THỰC HÀNH · DISTRIBUTED SYSTEMS · AWS

Từ Code đến Cloud

Microservices, K8s và DevOps trên AWS

huulinhcvp

First Edition · 2026

MICROSERVICES · KUBERNETES · DEVOPS TRÊN AWS

Từ Code đến Cloud

Tư duy hệ thống qua hành trình của TravelNow

huulinhcvp

Ấn bản PDF · 15 chương đã phát hành.

Tạo ngày 13/10/2026. Nội dung online có thể được cập nhật sau thời điểm này.

Bản online: ebook.fromcodetocloud.info

eBook miễn phí. Nếu muốn ủng hộ tác giả, xem trang cuối.

© 2026 huulinhcvp

Mục lục

Lời mở đầu 5

PHẦN I
Nền Tảng 9

Chương 1 Nghiệp vụ TravelNow 10

Chương 2 Deploy lên AWS lần đầu 20

PHẦN II
Container hóa và Tự động hóa 37

Chương 3 Docker và Container hóa ứng dụng 38

Chương 4 CI/CD Pipeline từ Code đến Cloud 57

Chương 5 Infrastructure as Code — Khi hạ tầng được triển khai bằng code 72

PHẦN III
Kiến trúc phân tán 85

Chương 6 Khi Monolith không còn đủ 87

Chương 7 Distributed Transaction, Timeout và Idempotency 102

Chương 8 Saga Pattern — Choreography và Orchestration 118

Chương 9 Triển khai hệ thống phân tán trên AWS 145

PHẦN IV
Vận hành ở quy mô lớn 179

Chương 10 Container Orchestration với Kubernetes 180

Chương 11 TravelNow trên Amazon EKS 202

Chương 12 Auto Scaling và High Availability 223

Chương 13 Tản mạn về kiến trúc Serverless 239

PHẦN V

Quan sát và Kiểm soát 250

Chương 14 Distributed Tracing và Logging 251

Chương 15 Bảo mật và Chi phí 265

Phần kết 281

Tài liệu tham khảo 285

LỜI MỞ ĐẦU

Tại sao lại là TravelNow?

Một người dùng nhấn "Xác nhận và thanh toán" để đặt chuyến đi. Khách sạn đã giữ phòng, vé máy bay đã được đặt, nhưng ứng dụng không nhận được phản hồi từ cổng thanh toán. Chuyến đi đã thành công chưa? Có nên gửi lại yêu cầu? Và nếu người dùng gọi đến nhờ hỗ trợ, đội vận hành dựa vào đâu để trả lời?

Những câu hỏi ấy là điểm xuất phát của eBook. Chúng dẫn đến ba vấn đề xuyên suốt: **giữ trạng thái nghiệp vụ nhất quán, xử lý an toàn khi kết quả chưa rõ, và tìm được bằng chứng để hiểu hệ thống đang làm gì.** Công nghệ chỉ có ý nghĩa khi giúp chúng ta giải quyết những vấn đề đó trong một bối cảnh cụ thể.

TravelNow là hệ thống đặt chuyến đi trọn gói được giả lập cho mục đích học tập. Nó kết hợp đặt khách sạn, đặt vé máy bay, thuê xe và thanh toán trong cùng một yêu cầu. Nghiệp vụ đủ gần gũi để không cần nhiều kiến thức ngành du lịch, nhưng có đủ tình huống thành công, thất bại và phục hồi để làm rõ các quyết định kiến trúc.

Mục tiêu của eBook là **xây dựng tư duy hệ thống thông qua quá trình phát triển và triển khai TravelNow trên AWS**. Chúng ta bắt đầu bằng một ứng dụng đơn giản, theo dõi điều gì thay đổi khi dữ liệu được lưu ở nơi khác, khi lời gọi hàm trở thành lời gọi qua mạng, và khi việc vận hành được giao một phần cho nền tảng. AWS giúp những ý tưởng ấy trở nên cụ thể; giá trị mà chúng ta có thể mang theo là cách đặt câu hỏi và đánh giá lựa chọn khi triển khai lên các nền tảng khác.

Cách tiếp cận của eBook

Một hệ thống chạy được chưa trả lời hết những câu hỏi về nó. Khi máy chủ bị thay thế, dữ liệu còn không? Khi một yêu cầu được gửi hai lần, tiền có bị trừ hai lần không? Khi triển khai phiên bản mới, làm sao biết người dùng vẫn đặt được chuyến đi?

Mỗi bước thay đổi của TravelNow được xem xét qua năm câu hỏi:

1. **Áp lực nào cần giải quyết?** Đó là vấn đề đã quan sát được hay một giả định cần kiểm chứng?
2. **Điều gì phải được giữ đúng?** Chẳng hạn, chỉ xác nhận chuyến đi khi các dịch vụ được yêu cầu và thanh toán đều thành công.
3. **Cơ chế nào đáp ứng yêu cầu ấy?** Cơ chế đó dựa vào những giả định và ranh giới nào?

4. **Ta phải chịu thêm chi phí và trách nhiệm gì?** Bao gồm công sức phát triển, vận hành, tiền hạ tầng và cách xử lý sự cố.
5. **Dựa vào đâu để biết lựa chọn có hiệu quả?** Cần kiểm thử tình huống nào, đo điều gì và khi nào nên đánh giá lại?

Chúng ta sẽ đi từ vấn đề đến cơ chế, rồi mới chọn công cụ triển khai. Tên dịch vụ và cú pháp có thể thay đổi; cách lập luận này vẫn giúp bạn đánh giá một phương án khi gặp công nghệ khác.

Thứ tự các chương phục vụ việc học, **không phải một lộ trình trưởng thành bắt buộc**. Monolith có thể đáp ứng tốt một sản phẩm trong nhiều năm. Microservices tạo thêm quyền triển khai độc lập nhưng cũng thêm ranh giới mạng và trách nhiệm phối hợp. Kubernetes và serverless giải quyết những nhóm nhu cầu khác nhau; một hệ thống có thể dùng kết hợp chúng, hoặc không cần đến chúng.

eBook dành cho ai?

eBook dành cho lập trình viên đã từng xây dựng một ứng dụng web và muốn hiểu mối liên hệ giữa code, dữ liệu, hạ tầng và vận hành. Bạn nên có kiến thức Java cơ bản, từng làm việc với HTTP/REST và cơ sở dữ liệu, đồng thời có thể đọc các ví dụ YAML và lệnh shell. Kinh nghiệm với AWS, Docker hoặc Kubernetes hữu ích nhưng không bắt buộc; các khái niệm cần thiết được giải thích khi xuất hiện.

Bạn có thể theo dõi toàn bộ lập luận ngay trong eBook, không cần tải mã nguồn hay dựng môi trường. Các đoạn code, lệnh và cấu hình minh họa cách một cơ chế hoạt động; tên file, class và tài nguyên là quy ước của ví dụ. Khi đọc, hãy chú ý đầu vào, trạng thái thay đổi và cách xử lý lỗi. Nếu áp dụng vào hệ thống của mình, bạn cần đối chiếu với phiên bản công cụ và môi trường thực tế. Những con số về tải, thời gian hoặc tài nguyên dùng làm ví dụ là giả định minh họa, trừ khi được nêu rõ là kết quả đo.

TravelNow không kết nối với nhà cung cấp du lịch hay cổng thanh toán thật. Ở điểm xuất phát, thao tác đặt chỗ và thanh toán được mô phỏng bằng thay đổi trong cùng một database; nhờ vậy, một transaction có thể bảo vệ chúng. Khi tách service, mỗi service quản lý dữ liệu riêng và các lời gọi giữa chúng tạo ra bài toán phân tán. Phân biệt hai giai đoạn này là điều cần thiết để hiểu đúng giới hạn của rollback và lý do xuất hiện Saga.

Bố cục nội dung

Phần I — Nền Tảng xác định nghiệp vụ và kiến trúc khởi điểm, rồi đưa ứng dụng lên AWS. Trọng tâm là phân biệt phần xử lý với dữ liệu cần giữ lâu dài, theo dõi nơi mỗi thành phần chạy và đường giao tiếp giữa chúng.

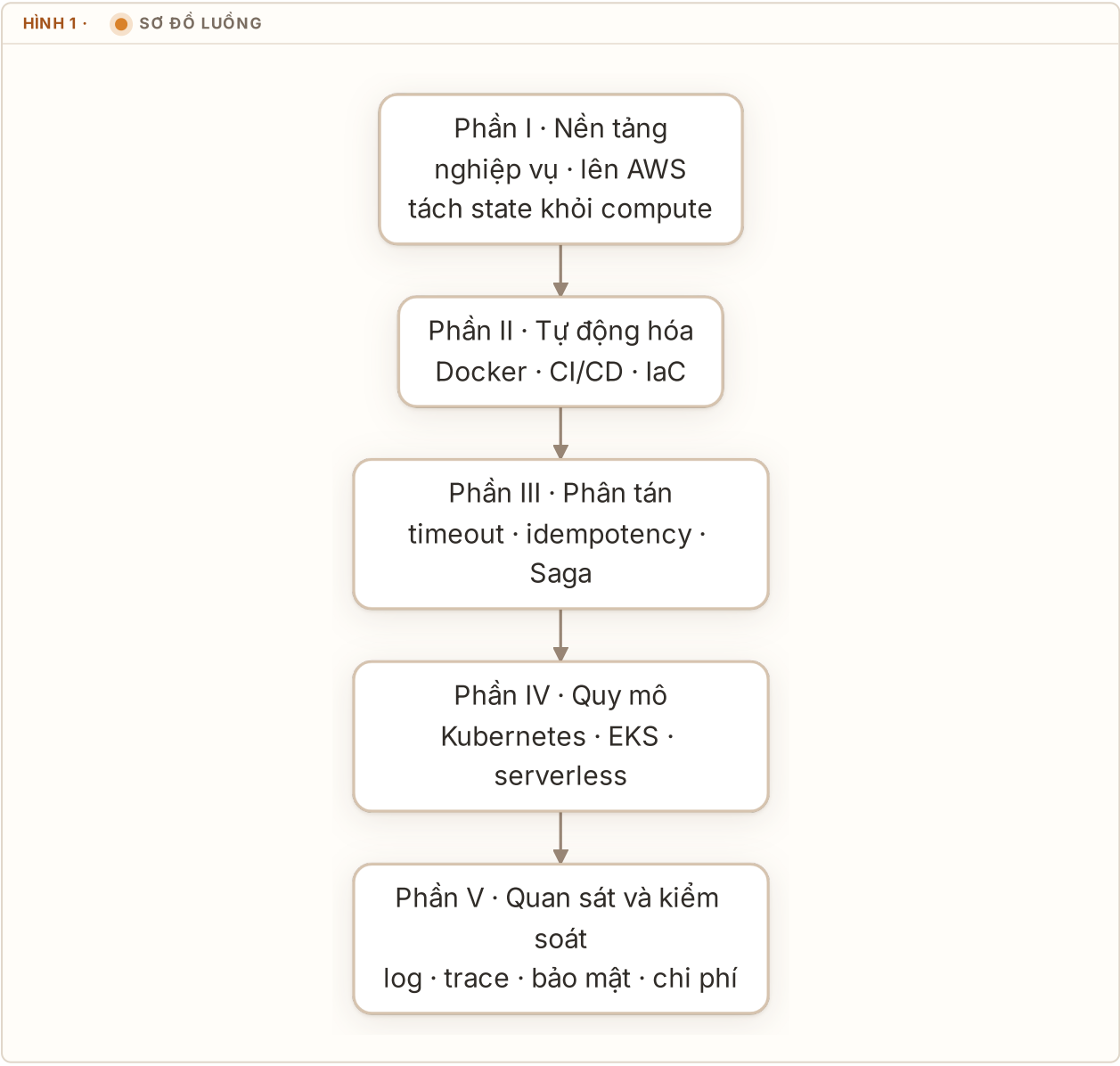
Phần II — Container hóa và Tự động hóa làm cho việc đóng gói, phát hành và dựng hạ tầng có thể lặp lại, kiểm tra và truy vết. Các khả năng này hữu ích ngay cả khi hệ thống vẫn chỉ có một ứng dụng.

Phần III — Kiến trúc phân tán đặt TravelNow vào một bối cảnh giả định có lý do để tách service. Chúng ta phân tích timeout, retry, idempotency và Saga, sau đó đối chiếu các cơ chế phối hợp với Step Functions, SNS/SQS và Lambda trên AWS.

Phần IV — Vận hành ở quy mô lớn tìm hiểu cách điều phối nhiều container, triển khai lên EKS, duy trì khả năng phục vụ khi có sự cố và điều chỉnh tài nguyên theo tải. Phần này khép lại bằng việc cân nhắc serverless cùng những trách nhiệm mà mô hình ấy chuyển giao hoặc giữ lại.

Phần V — Quan sát và Kiểm soát nối các góc nhìn trước đó qua log, metric, trace, bảo mật và chi phí. Những mối quan tâm này cần có từ khi hệ thống bắt đầu phục vụ người dùng; việc đặt chúng ở cuối eBook giúp ta tổng hợp trên một kiến trúc đã đủ rõ, không có nghĩa là đợi đến cuối quá trình phát triển mới thực hiện.

HÌNH 1 · SƠ ĐỒ LUỒNG



Bạn có thể đọc tuần tự để theo dõi các ranh giới thay đổi, hoặc quay lại một chương khi gặp bài toán tương ứng. Trong cả hai cách đọc, hãy giữ câu hỏi: *với yêu cầu và nguồn lực hiện tại, giải pháp này giúp ích ở đâu, và bằng chứng nào cho phép mình tin vào nó?*

Chúng ta bắt đầu từ một việc cụ thể: xác định thế nào là một chuyến đi được đặt thành công.

PHẦN I

Nền Tảng

The Foundation

Một chuyến đi đúng nghiệp vụ cần những điều gì, và dữ liệu của nó tồn tại ở đâu? Hai chương đầu xác định quy tắc của TravelNow, phạm vi giả lập và kiến trúc khởi điểm, rồi theo dõi việc tách dữ liệu khỏi nơi chạy ứng dụng.

Kết thúc phần này, bạn có một bản đồ gồm phần xử lý, dữ liệu và các đường giao tiếp. Những ranh giới ấy là cơ sở để đánh giá các thay đổi trong các phần sau.

Chương	Tiêu đề
1	Nghiệp vụ TravelNow
2	Deploy lên AWS lần đầu

CHƯƠNG 1

Nghiệp vụ TravelNow

Phần I — Nền Tảng · *The Foundation*

Một chuyến đi được đặt thành công nghĩa là gì? Câu trả lời cần làm rõ trước khi chúng ta chọn database, chia service hay đưa ứng dụng lên cloud. Chương này xác định các quy tắc nghiệp vụ của TravelNow và phạm vi của mô hình giả lập dùng xuyên suốt eBook.

Sau đó, chúng ta chọn một kiến trúc khởi điểm đủ đơn giản để nhìn rõ trạng thái và cách xử lý lỗi. Spring Boot là công cụ được dùng để triển khai các ví dụ; điều mà chúng ta cần theo dõi là ranh giới của từng cơ chế và những điều nó bảo đảm.

> **Ghi chú:** TravelNow là hệ thống giả định xuyên suốt eBook. Luồng nghiệp vụ, sơ đồ và các đoạn mã minh họa được trình bày ngay tại nơi cần giải thích; bạn có thể theo dõi mà không cần tải hay chạy một ứng dụng đi kèm.

1.1 Nghiệp vụ TravelNow

1.1.1 Nhìn từ phía người dùng

Bạn muốn đặt một chuyến công tác từ Hà Nội đến Đà Nẵng: chọn ngày đi, ngày về, khách sạn và thêm xe thuê nếu cần. Sau khi nhấn "Xác nhận và thanh toán", bạn mong nhận được kết quả rõ ràng: chuyến đi đã được đặt đầy đủ, hay chưa thể hoàn tất. Bạn không muốn tự kiểm tra từng dịch vụ để biết tiền đã bị trừ mà phòng có được giữ hay không.

Toàn bộ trải nghiệm đó, từ phía người dùng, chỉ là một cú click. Nhưng phía sau nó là cả một chuỗi thao tác phải diễn ra **đúng thứ tự, nhất quán và đáng tin cậy.**

1.1.2 Các thực thể nghiệp vụ

TravelNow xoay quanh năm thực thể cốt lõi sau đây.

Trip là trung tâm của mọi thứ. Một trip đại diện cho toàn bộ chuyến đi của một người dùng, bao gồm trạng thái tổng thể và thời điểm tạo. Mỗi trip được định danh bởi một mã duy nhất (`tripId`) — **do phía client sinh ra và gửi lên** cùng yêu cầu đặt chuyến đi, không phải do server cấp phát.

Nếu `tripId` đã tồn tại, yêu cầu bị từ chối. Mã do client cung cấp giúp nhận diện lại cùng một yêu cầu. Tuy nhiên, từ chối mã trùng mới chỉ là bước đầu; chương 7 sẽ phân biệt việc chặn trùng với việc trả lại kết quả an toàn khi client thử lại.

HotelBooking đại diện cho việc đặt phòng khách sạn, bao gồm các thuộc tính như: tên khách sạn, ngày nhận phòng, ngày trả phòng, và trạng thái hiện tại.

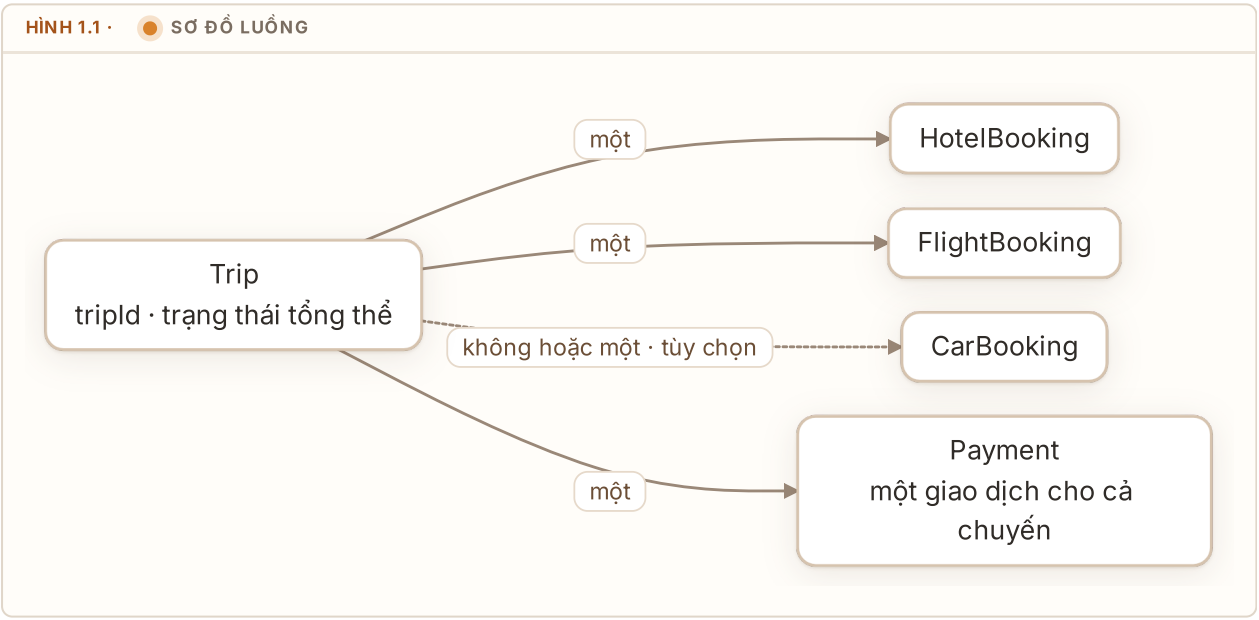
FlightBooking là thông tin đặt vé máy bay: điểm khởi hành, điểm đến, ngày đi, ngày đến, và trạng thái.

CarBooking là thông tin thuê xe: loại xe, thời gian thuê, và trạng thái. Đây là dịch vụ tùy chọn, không phải chuyến đi nào cũng cần thuê xe. Nếu người dùng không yêu cầu thì bước này được bỏ qua hoàn toàn.

Payment đại diện cho giao dịch thanh toán của toàn bộ chuyến đi: số tiền, trạng thái giao dịch, và mã xác nhận từ nhà cung cấp thanh toán.

Quan hệ giữa các thực thể: một Trip gom một đặt phòng, một vé máy bay, tối đa một xe thuê và một giao dịch thanh toán.

HÌNH 1.1 · SƠ ĐỒ LUỒNG



1.1.3 Vòng đời của một chuyến đi

Trước khi mô tả trạng thái, cần làm rõ giả định: **ở chương này, đặt chỗ và thanh toán đều được mô phỏng bằng thao tác ghi trong cùng một database**. Chúng chưa phải lời gọi đến đối tác bên ngoài. Mục 1.1.8 giải thích đầy đủ quy ước này.

Một chuyến đi có các trạng thái sau:

`PENDING` — khi người dùng vừa gửi yêu cầu, hệ thống đang xử lý từng bước theo thứ tự.

`BOOKED` — khi tất cả các bước đều được thực hiện thành công, bao gồm cả thanh toán. Đây là xác nhận nghiệp vụ dành cho người dùng.

`CANCELLED` — khi người dùng chủ động hủy một chuyến đi *đã đặt xong* (`BOOKED`).

Nếu một bước đặt chuyến đi thất bại, transaction bị rollback: các thay đổi của lần đặt đó không được commit. `PENDING` chỉ là trạng thái trung gian bên trong transaction, không phải một bản ghi đã được lưu bền vững để theo dõi tiến độ. Vì vậy, thất bại khi đang đặt khác với hủy một chuyến đã đặt thành công. Khi mỗi service có database riêng ở Phần III, ta phải thiết kế lại cách ghi nhận tiến độ và xử lý những bước đã commit.

Gom tất cả lại, vòng đời của một chuyến đi được biểu diễn bằng sơ đồ máy trạng thái (state machine diagram) dưới đây:

HÌNH 1.2 · MÁY TRẠNG THÁI



1.1.4 Luồng nghiệp vụ chính

Khi người dùng gửi yêu cầu đặt chuyến đi, hệ thống lần lượt thực hiện các bước sau:

Bước 1 — Tạo chuyến đi. Hệ thống ghi nhận yêu cầu và tạo một bản ghi chuyến đi mới ở trạng thái đang xử lý. Đây là điểm bắt đầu của toàn bộ luồng.

Bước 2 — Đặt khách sạn. Hệ thống mô phỏng việc giữ phòng theo yêu cầu và ghi nhận kết quả.

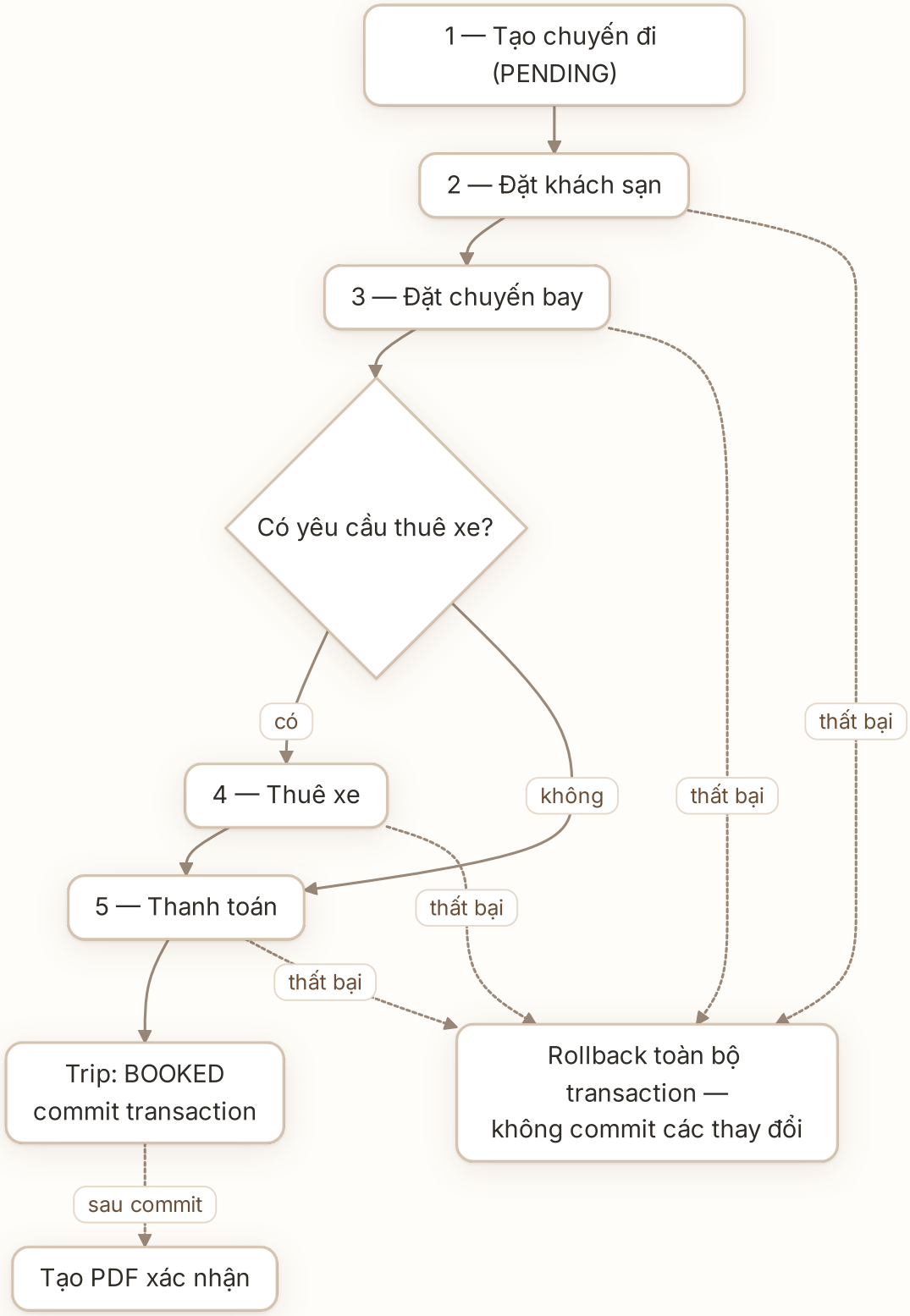
Bước 3 — Đặt chuyến bay. Tương tự, hệ thống đặt vé máy bay và ghi nhận kết quả.

Bước 4 — Thuê xe. Nếu người dùng có yêu cầu, hệ thống đặt xe và ghi nhận kết quả. Nếu không thì bỏ qua.

Bước 5 — Thanh toán. Đây là bước cuối cùng. Hệ thống thực hiện giao dịch thanh toán cho toàn bộ chuyến đi. Chỉ khi thanh toán thành công, hệ thống mới có thể hoàn tất transaction và trả về kết quả chuyến đi đã được đặt.

Toàn bộ luồng nghiệp vụ được mô tả trong sơ đồ dưới đây:

HÌNH 1.3 · SƠ ĐỒ LUỒNG



Sau khi chuyến đi được xác nhận, hệ thống tạo tệp PDF để người dùng xem lại hoặc tải về. PDF được tạo từ dữ liệu chuyến đi đã lưu; lỗi tạo file không có nghĩa là thanh toán thất bại và không được dẫn đến một lần thu tiền mới. Ghi file nằm ngoài database transaction, nên sơ đồ tách

riêng bước này. Cách bảo đảm công việc sau commit không bị bỏ sót sẽ được phân tích ở chương 8.

PDF cũng đưa vào mô hình một loại **state** — dữ liệu cần được lưu giữ — bên cạnh dữ liệu quan hệ. Chương 2 sẽ theo dõi hai cách lưu trữ tương ứng: database và object storage.

1.1.5 Khi có sự cố xảy ra

Giả sử bước đặt khách sạn và chuyến bay đã chạy, nhưng thanh toán bị từ chối. Trong mô hình cùng database, transaction rollback cả các bước trước đó để không giữ lại một chuyến đi chưa hoàn tất.

Cần phân biệt **đã biết thất bại** với **chưa biết kết quả**. Nếu client chỉ bị timeout, server có thể đã commit thành công. Khi đó, đặt lại bằng một mã mới có thể tạo thêm chuyến đi; client cần tra cứu hoặc gửi lại cùng định danh theo cơ chế được xây dựng ở chương 7.

1.1.6 Hủy chuyến đi

Trong chính sách giả lập của TravelNow, người dùng có thể hủy chuyến đi đã đặt thành công mà không chịu phí hủy. Luồng hủy thực hiện theo thứ tự dưới đây; đây là lựa chọn của ví dụ, không phải quy tắc chung của ngành du lịch:

1. Hoàn tiền
2. Hủy xe thuê, nếu có
3. Hủy đặt phòng khách sạn
4. Hủy vé máy bay

Chuyến đi chỉ chuyển sang `CANCELLED` khi các bước hủy cần thiết và hoàn tiền đã hoàn tất. Trong mô hình hiện tại, các thay đổi giả lập này cũng nằm trong một transaction.

1.1.7 Những gì hệ thống này không làm

Để giữ cho ví dụ đủ đơn giản và tập trung vào các khái niệm kỹ thuật cốt lõi, TravelNow cố tình bỏ qua một số thứ mà một hệ thống thực tế chắc chắn sẽ cần.

Không có tích hợp bên thứ ba thực sự. Các API đặt khách sạn, hãng bay, và thuê xe sẽ được giả lập đủ để minh họa các bài toán kỹ thuật, nhưng không kết nối với hệ thống thật. Tương tự, cổng thanh toán cũng là giả lập.

Không có xác thực người dùng phức tạp. Hệ thống sẽ có một lớp auth cơ bản, nhưng không đi sâu vào OAuth, SSO, hay quản lý phiên đa thiết bị.

Không có pricing engine. Giá của từng thành phần được trả về trực tiếp từ nhà cung cấp giả lập, không có logic tính giá động, không có discount, không có surge pricing.

Tất cả đều là những lựa chọn có chủ ý: TravelNow chỉ cần đủ phức tạp để minh họa các khái niệm kỹ thuật, mà không kéo chúng ta sa lầy vào chi tiết nghiệp vụ thực.

1.1.8 Quy ước giả lập

Vì không có kết nối thật đến bên thứ ba, chúng ta cần thống nhất với nhau một quy ước xuyên suốt eBook:

> **Quy ước giả lập:** một bản ghi trong database đại diện cho kết quả *đã được xác nhận ở phía nhà cung cấp*. `HotelBooking` ở trạng thái `BOOKED` nghĩa là "đối tác khách sạn đã giữ phòng xong"; chuyển nó sang `CANCELLED` nghĩa là "đã hủy giữ phòng với đối tác". Nói cách khác: **"đặt phòng" được giả lập bằng một thao tác ghi database** — không có lời gọi HTTP nào sang bên thứ ba.

Ở Phần I và II, các module vẫn nằm trong cùng ứng dụng. Từ Phần III, những service được tách ra sẽ giao tiếp qua mạng và quản lý dữ liệu riêng. Nghiệp vụ đối tác vẫn được giả lập, nhưng ranh giới giao tiếp giữa các service là ranh giới phân tán cần xử lý thật.

Các chương sau sẽ cùng xét một tình huống lỗi: khách sạn và chuyến bay đã được giữ, nhưng thanh toán bị từ chối. Giữ nguyên tình huống giúp ta thấy rõ cách xử lý thay đổi khi đi từ một database transaction sang hệ phân tán.

Một lưu ý nhỏ: trong thực tế, lời gọi đến bên thứ ba là side effect *nằm ngoài* database transaction, nên ngay cả một hệ thống monolith thật cũng không đạt được all-or-nothing trọn vẹn khi tích hợp với nhà cung cấp. Các cơ chế ở Phần III giúp phân tích ranh giới này; tích hợp với đối tác thật còn cần tuân theo đặc tả API mà họ cung cấp và quy trình đối soát của từng bên.

1.2 Bắt đầu xây dựng hệ thống này như thế nào?

Bây giờ chúng ta đã hiểu rõ nghiệp vụ. Câu hỏi tiếp theo là: *xây dựng hệ thống này như thế nào?*

Câu trả lời của eBook, ít nhất là ở giai đoạn đầu, là **một monolith application**: toàn bộ ứng dụng chạy trong một process duy nhất. Không message queue, không service-to-service call, không có gì phải phối hợp giữa các thành phần ngoài một database transaction.

Lựa chọn này phù hợp với bối cảnh khởi điểm: một đội nhỏ, các chức năng phát hành cùng nhau và dữ liệu giả lập nằm trong cùng database. Chúng ta vẫn tách code thành các module có trách nhiệm rõ ràng, nhưng chưa cần cho mỗi module một tiến trình và cách triển khai riêng. Một transaction cục bộ bảo vệ dữ liệu với ít thành phần phải vận hành.

Điều cần ghi nhớ là: **chỉ thêm độ phức tạp khi nó giải quyết một vấn đề đã xác định được**. Nếu có ràng buộc khác ngay từ đầu, chẳng hạn dữ liệu phải do những đơn vị riêng quản lý, kiến trúc khởi điểm cũng có thể khác.

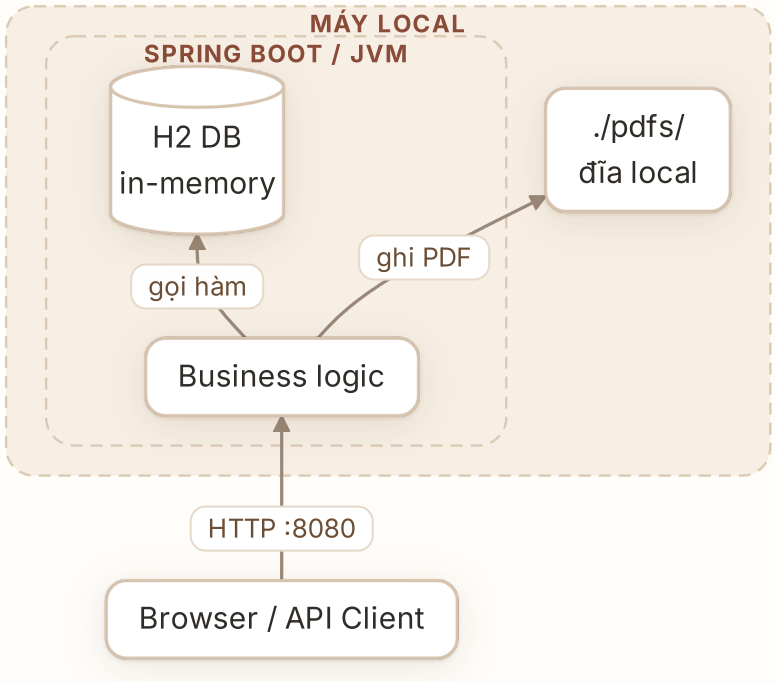
Hệ thống rồi sẽ tiến hóa: đến một lúc nào đó, phân bố tải thay đổi hoặc các domain cần tách biệt rõ ràng hơn, và khi ấy ta sẽ đánh giá lại ranh giới giữa các service. Chương 6 sẽ đặt baseline và mục tiêu trước khi tái kiến trúc, rồi Phần III xử lý những câu hỏi phát sinh sau đó: đặt phòng

khách sạn thành công nhưng đặt vé máy bay thất bại thì sao? Thử lại thanh toán liên tục có an toàn không? Còn hiện tại, nhiệm vụ của chúng ta đơn giản hơn nhiều: xây dựng một modular monolith với vài thành phần gọn gàng.

1.3 Phác họa kiến trúc khởi điểm

Nhìn từ bên ngoài, toàn bộ hệ thống ở thời điểm này trông như sau:

HÌNH 1.4 · SƠ ĐỒ LUỒNG



Không cần đi vào từng package hay từng dòng code; chỉ có **bốn điều đáng ghi nhớ về kiến trúc này** — bốn sợi chỉ sẽ xuyên suốt eBook và được nhắc lại thường xuyên:

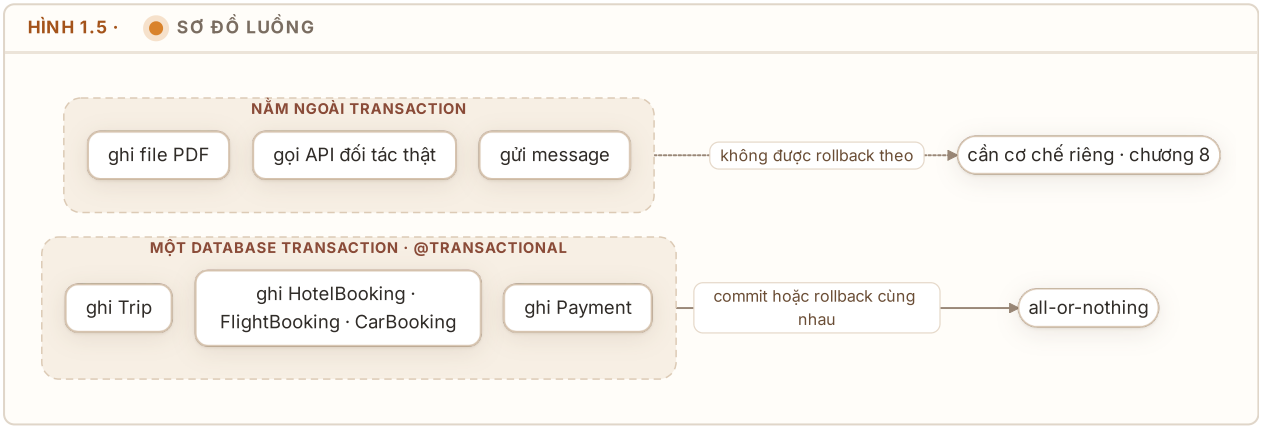
1. Ranh giới domain được xác định rõ ràng ngay từ đầu. Code được phân tầng theo kiểu cổ điển (API — nghiệp vụ — dữ liệu), và quan trọng hơn: coupling giữa hotel, flight, car, payment được phơi bày rõ ràng, không giấu đi đâu được.

2. Tính nhất quán được đảm bảo bằng một database transaction. Toàn bộ năm bước nghiệp vụ nằm trong một transaction duy nhất, với Spring Boot thì điều đó gói gọn trong một annotation:

```
@Transactional
public TripDetail bookTrip(BookTripCommand cmd) {
    // tạo trip → đặt hotel → đặt flight → thuê xe (nếu có) → thanh toán
    // lỗi RuntimeException thoát khỏi phương thức → rollback transaction
}
```

Với cấu hình mặc định, Spring rollback khi `RuntimeException` hoặc `Error` thoát khỏi phương thức được quản lý transaction; checked exception cần quy tắc rollback phù hợp. Ví dụ giả định các lỗi nghiệp vụ mô phỏng thuộc nhóm gây rollback. Tài liệu Spring về rollback giải thích chi tiết quy tắc này.

Khi mọi thay đổi cùng tham gia một database transaction, ta có tính all-or-nothing mà không phải dọn dẹp từng bảng bằng tay. Cơ chế này có ranh giới: gọi API bên ngoài, ghi file, gửi message — những side effect nằm ngoài transaction sẽ không được nó bao phủ. Và khi hệ thống tách thành microservices, một local transaction cũng không thể trải qua nhiều database độc lập. Chương 6 sẽ chỉ rõ ranh giới này, còn Saga và outbox ở chương 8 sẽ cung cấp cơ chế phối hợp thay thế. Ở giai đoạn hiện tại, gom được toàn bộ side effect mô phỏng vào một transaction là một lợi thế có chủ đích.



3. Chỗ nào hạ tầng có thể thay đổi, chỗ đó có một ranh giới giao tiếp. Việc lưu PDF nằm sau interface `PdfStorageService`: implementation hiện tại ghi ra đĩa local, tương lai có thể ghi sang object storage (S3). Interface giúp use case không phụ thuộc trực tiếp vào SDK của storage — dù adapter mới vẫn phải tự xử lý những khác biệt về semantics như lỗi mạng, retry, phân quyền và tính nhất quán. Nguyên tắc chung: tách quyết định hạ tầng khỏi quy tắc nghiệp vụ, nhưng đừng ảo tưởng rằng đổi hạ tầng là "miễn phí".

4. State đang "tạm bợ" một cách có chủ đích. Database là H2 chạy trong memory (tắt app là mất dữ liệu), file PDF nằm trên đĩa local. Đổi lại, ta được **zero setup** — đủ tốt cho nhu cầu phát triển, nhưng hoàn toàn không dùng được cho môi trường sản xuất (production). Chương 2 sẽ giải quyết chuyện này.

1.4 Chạy hệ thống

Ở điểm xuất phát, có thể hình dung quá trình khởi chạy qua ba bước: **mã nguồn Java → file JAR → một process JVM nhận request HTTP**. File JAR chứa ứng dụng đã biên dịch; JVM là môi trường thực thi trên máy đích. Trong mô hình này, process lắng nghe ở cổng `8080`, giữ dữ liệu H2 trong memory và ghi PDF ra đĩa local.

Sự đơn giản này là có chủ đích, nhưng nó cũng có giới hạn: file JAR chỉ là *ứng dụng*, chưa phải *ứng dụng cùng môi trường chạy của nó*. Máy đích vẫn phải cài đúng Java version, thiết lập đúng biến môi trường, đúng thư mục. Chừng nào chỉ có một máy và một người deploy, điều đó chưa thành vấn đề. Phần II sẽ đóng gói cả runtime lẫn dependency để giảm sai lệch giữa các môi trường — dù cấu hình, kernel, CPU, network và dịch vụ bên ngoài vẫn là những biến số cần kiểm chứng.

1.5 Những giới hạn của kiến trúc này

Kiến trúc hiện tại có những điểm mạnh rõ ràng: một lệnh để chạy, một chỗ để debug, một transaction đảm bảo nhất quán, tốc độ phát triển nhanh. Nhưng nó chưa sẵn sàng cho môi trường production:

Vấn đề	Hệ quả
H2 in-memory	Dữ liệu mất khi restart — không thể dùng cho production
PDF trên local filesystem	Chưa dùng chung giữa các instance; cần giữ hoặc di chuyển dữ liệu khi thay server/ổ đĩa
Log chỉ ra stdout	Chưa có cơ chế thu gom, tìm kiếm tập trung và cảnh báo
Một process, một server	Server chết là hệ thống chết theo — không có high availability

Chương 2 xử lý trước vòng đời dữ liệu và cách kết nối các thành phần trên AWS. Khả năng phục hồi của tầng ứng dụng, quan sát và bảo mật sẽ được phát triển tiếp ở các phần sau; đưa ứng dụng lên cloud tự nó chưa giải quyết hết những giới hạn trong bảng.

1.6 Tóm tắt

TravelNow chỉ xác nhận một chuyến đi khi khách sạn, chuyến bay, xe thuê nếu có và thanh toán đều thành công. Hủy chuyến đi đã đặt là một nghiệp vụ riêng; nó khác với rollback một lần đặt chưa hoàn tất.

Kiến trúc khởi điểm dùng một monolith application và một database transaction cho các thao tác giả lập. Giới hạn ấy làm rõ điều transaction bảo vệ: các thay đổi dữ liệu cùng tham gia giao dịch. Nó không bảo vệ việc ghi tệp PDF hay lời gọi đến đối tác bên ngoài.

Ứng dụng đã có quy tắc nghiệp vụ và một cách chạy đơn giản, nhưng dữ liệu còn phụ thuộc vào tiến trình và máy local. Chương 2 bắt đầu từ câu hỏi đó: **làm sao để thay thế nơi chạy ứng dụng mà vẫn giữ được dữ liệu?**

CHƯƠNG 2

Deploy lên AWS lần đầu

Phần I — Nền Tảng · *The Foundation*

Ở chương 1, dữ liệu trong H2 mất khi ứng dụng dừng, còn file PDF thì lại phụ thuộc vào ổ đĩa ở trên máy local. Trước khi nghĩ đến nhiều người dùng hay nhiều service, TravelNow cần một cách để giữ dữ liệu độc lập với nơi chạy ứng dụng.

Chương này lần lượt tách database và file khỏi vòng đời của app, rồi triển khai các thành phần trên AWS. Ở mỗi bước, chúng ta theo dõi ba điều: thành phần đó chạy ở đâu, giao tiếp qua con đường nào và ai chịu trách nhiệm vận hành nó. Đây là góc nhìn liên kết việc viết code với việc đưa hệ thống vào sử dụng.

Các dịch vụ AWS như EC2, VPC, Security Group, RDS và S3 sẽ xuất hiện theo nhu cầu cụ thể. Việc hiểu về máy ảo, mạng riêng, kiểm soát truy cập, managed database và object storage giúp bạn có thể đọc được cả kiến trúc trên các nền tảng Cloud khác. Tuy vậy thì việc chuyển nền tảng vẫn cần phải đánh giá lại API, cách phân quyền và đặc tính vận hành; chứ không chỉ thay tên dịch vụ.

2.1 "Lên cloud" nghĩa là gì?

Trước khi chạm vào AWS, hãy làm rõ thứ mà ta sắp bước vào.

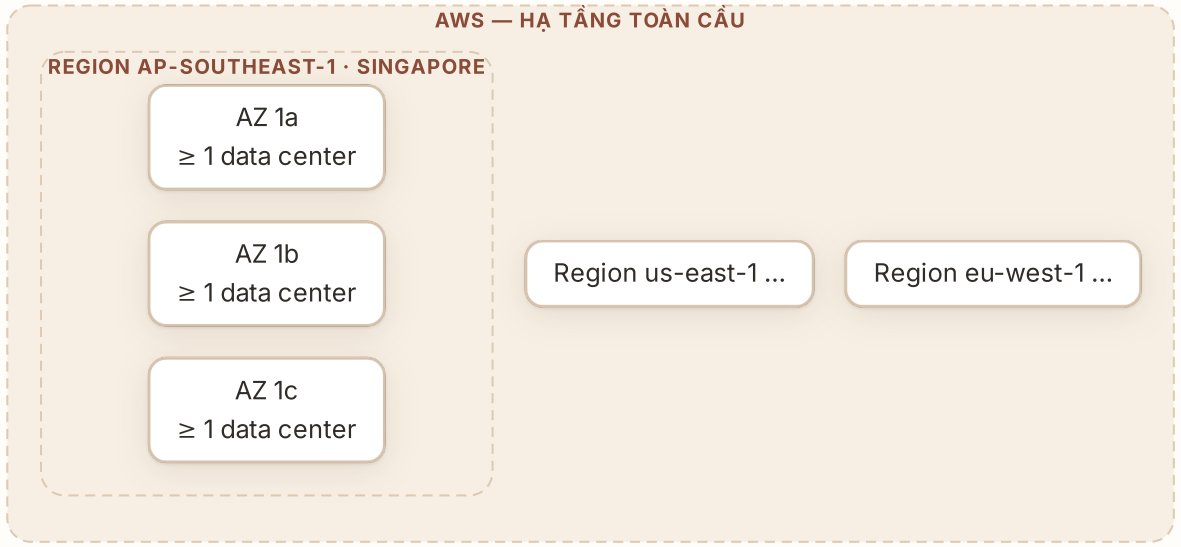
Phía sau cloud vẫn là máy chủ, mạng và thiết bị lưu trữ trong các trung tâm dữ liệu. Khác biệt đáng chú ý với việc tự mua và lắp đặt máy chủ là cách chúng ta tiếp cận và sử dụng tài nguyên. Trong phạm vi chương này, ba đặc tính sau có ảnh hưởng trực tiếp đến cách vận hành:

1. **Tự phục vụ qua API.** Muốn một máy chủ mới? Không cần gọi điện cho nhà cung cấp hay chờ duyệt mua phần cứng, một API call (hoặc vài cú click), vài phút sau máy chủ đã sẵn sàng. Chính tính chất này sẽ cho phép ta *lập trình hóa* hạ tầng ở chương 5.
2. **Dùng đến đâu trả tiền đến đấy (pay-as-you-go).** Chi phí gắn với loại tài nguyên và mức sử dụng: thời gian chạy, dung lượng lưu trữ, số request hoặc lượng dữ liệu truyền tải. Dừng compute không tự xóa chi phí của ổ đĩa, bản sao lưu hay tài nguyên khác còn giữ lại. Vì vậy, chi phí cần được theo dõi cùng quyết định kiến trúc; chương 15 sẽ phân tích cách làm này.

3. **Co giãn (elasticity).** Vì hai tính chất trên, việc "gấp đôi số máy chủ trong 10 phút rồi trả bớt vào cuối ngày" trở nên khả thi, tính co giãn chính là nền tảng của autoscaling ở Phần IV.

Máy tính của người khác thì vẫn phải nằm ở *một nơi nào đó*. AWS chia hạ tầng vật lý của mình thành các **Region** — cụm trung tâm dữ liệu (data center) tại một khu vực địa lý (eBook dùng `ap-southeast-1`, Singapore). Mỗi region gồm nhiều **Availability Zone (AZ)** — các cụm data center tách biệt về điện, mạng và hệ thống làm mát: đủ gần nhau để kết nối nhanh chóng qua mạng nội bộ, được thiết kế để giảm khả năng tất cả cùng bị ảnh hưởng bởi một sự cố vật lý. Phân bố trên nhiều AZ giúp giảm rủi ro, nhưng vẫn không loại bỏ mọi khả năng gián đoạn đồng thời.

HÌNH 2.1 · SƠ ĐỒ LUỒNG



Khái niệm AZ nghe có vẻ hành chính, nhưng hãy nhớ nó: AZ là đơn vị tư duy của **high availability** trên cloud — "hệ thống có sống sót khi một AZ gặp sự cố không?" — và ta sẽ gặp nó lần đầu ngay trong chương này, ở mục RDS Multi-AZ.

Cuối cùng, một nguyên tắc cần biết trước khi sử dụng cloud là **shared responsibility** — chia sẻ trách nhiệm. AWS chịu trách nhiệm về *bản thân* cloud, bao gồm: phần cứng, data center, mạng vật lý. Bạn chịu trách nhiệm về những gì mình đặt *trong* cloud: hệ điều hành bạn cài, cổng mạng bạn mở, dữ liệu bạn lưu trữ, quyền truy cập bạn cấp. Đường ranh này không cố định, nó còn phụ thuộc vào loại dịch vụ bạn chọn, chương này sẽ đưa các bạn qua một trục các dịch vụ: từ thuê máy "trần" (EC2, bạn lo gần hết), dịch vụ được quản lý (RDS, AWS lo phần vận hành khó) đến dịch vụ mà bạn không còn cần quan tâm về máy chủ phía sau (S3).

2.2 Bốn thành phần cần theo dõi

Để theo dõi sự thay đổi mà không bị ngợp bởi chi tiết, chương này dùng một mô hình rút gọn gồm bốn loại thành phần. Hệ thống thật có thể còn CDN, cache, queue, search engine và nhiều lớp khác — nhưng chúng đều phân tích được bằng cùng những câu hỏi về compute, state và đường giao tiếp giữa các thành phần đó.

Thành phần	Vai trò	Trong TravelNow
Client	Nơi phát ra request	Trình duyệt / API client
App server (compute)	Chạy business logic, xử lý request	JVM process (Spring Boot + embedded Tomcat)
Database server (state)	Lưu dữ liệu có cấu trúc, bền vững	H2 → PostgreSQL
Storage (state)	Lưu file/blob	File PDF trong `./pdfs/` → S3

Cách phân loại quan trọng hơn tên gọi là cặp **compute** và **state**:

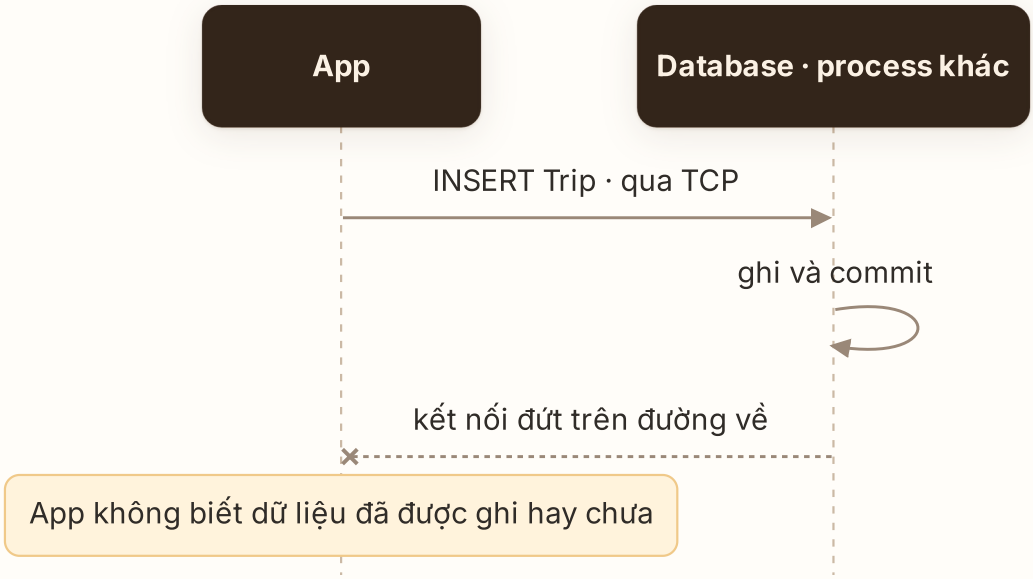
- **Compute** là phần *xử lý*. Ta **thiết kế** app server để nó không giữ state bền vững cục bộ, nhờ đó có thể thay thế nó mà không mất dữ liệu nghiệp vụ. Lưu ý: stateless là kết quả của thiết kế, không phải thuộc tính tự có — session để trong memory, file tạm thời bị dùng như dữ liệu thật, hay cache không có chiến lược khôi phục đều âm thầm phá vỡ giả định này.
- **State** là phần *nhớ* — dữ liệu phải sống sót qua restart, qua sự cố. Database và storage là state.

Một trục xuyên suốt hành trình hạ tầng của eBook: **tách state bền vững ra xa khỏi vòng đời của compute, để mỗi bên được scale và bảo vệ độc lập.** "Tách xa" ở đây là xa về vòng đời và quyền sở hữu, không nhất thiết xa về khoảng cách mạng, dữ liệu vẫn nên nằm gần compute khi độ trễ đòi hỏi.

Đến đây thì chúng ta cũng cần lưu ý về một khái niệm: **network hop**. Khi hai thành phần nằm trong cùng một process, bên gọi luôn quan sát được kết quả trực tiếp — hoặc trả về, hoặc ném lỗi; side effect vẫn có thể đã xảy ra trước khi lỗi được ném, nhưng không có sự mơ hồ kiểu "mất phản hồi giữa đường". Trong các bước tách của chương này, app sẽ dùng TCP để kết nối tới database ở process khác hoặc máy khác. Lời gọi lúc đó đi qua mạng: chậm hơn, và có thể đứt kết nối *sau khi* bên nhận đã xử lý xong. Mỗi ranh giới mới thường thêm một network hop và những failure mode mới; đổi lại, nó mở ra quyền triển khai, co giãn hoặc bảo vệ từng phần một cách độc lập. Toàn bộ chương này là câu chuyện về những ranh giới như thế xuất hiện dần.

Tình huống đặc trưng mà network hop mang lại: bên nhận đã xử lý xong, nhưng bên gọi không nhận được kết quả.

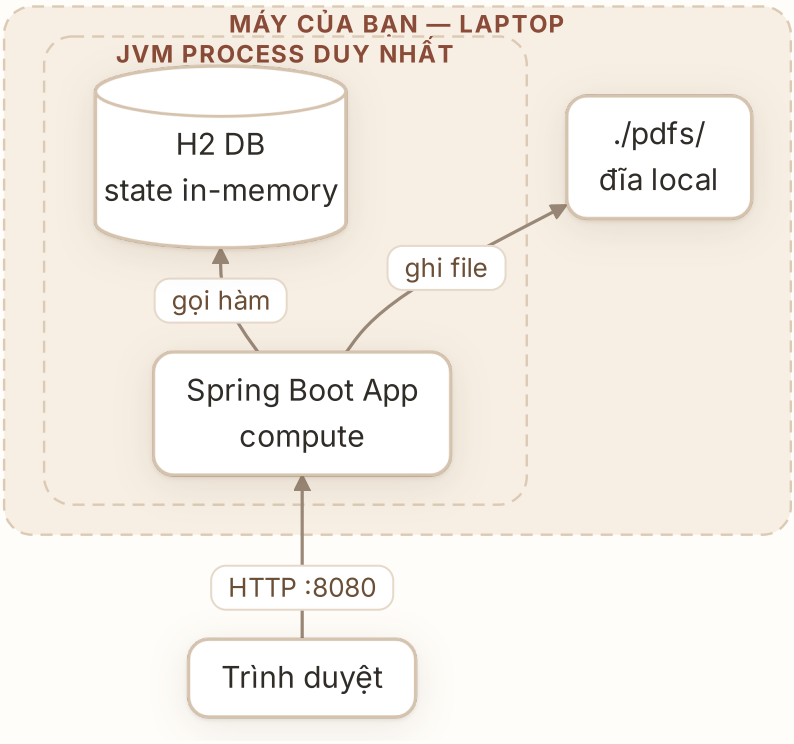
HÌNH 2.2 · SƠ ĐỒ TUẦN TỰ



2.3 Giai đoạn 0 — Mọi thứ trong một process (Local)

Đây là mô hình khởi đầu của TravelNow: một process Java, database H2 trong memory và tệp PDF trên đĩa local. Ta gọi nhóm cấu hình này là profile `dev`.

HÌNH 2.3 · SƠ ĐỒ LUỒNG



Điểm đặc biệt — và cực đoan — của giai đoạn này: **database không phải là một "server" riêng.** H2 in-memory chạy *bên trong* chính JVM process của app. Khi `TripBookingService` gọi xuống database, đó không phải một lời gọi qua mạng — đó là một lời gọi hàm Java thông thường, dữ liệu nằm ngay trong heap của process.

Giao tiếp:

- Client → App: HTTP qua cổng `8080`.
- App → DB: **không qua mạng** — cùng process, cùng vùng nhớ.
- App → Storage: ghi thẳng vào filesystem của máy.

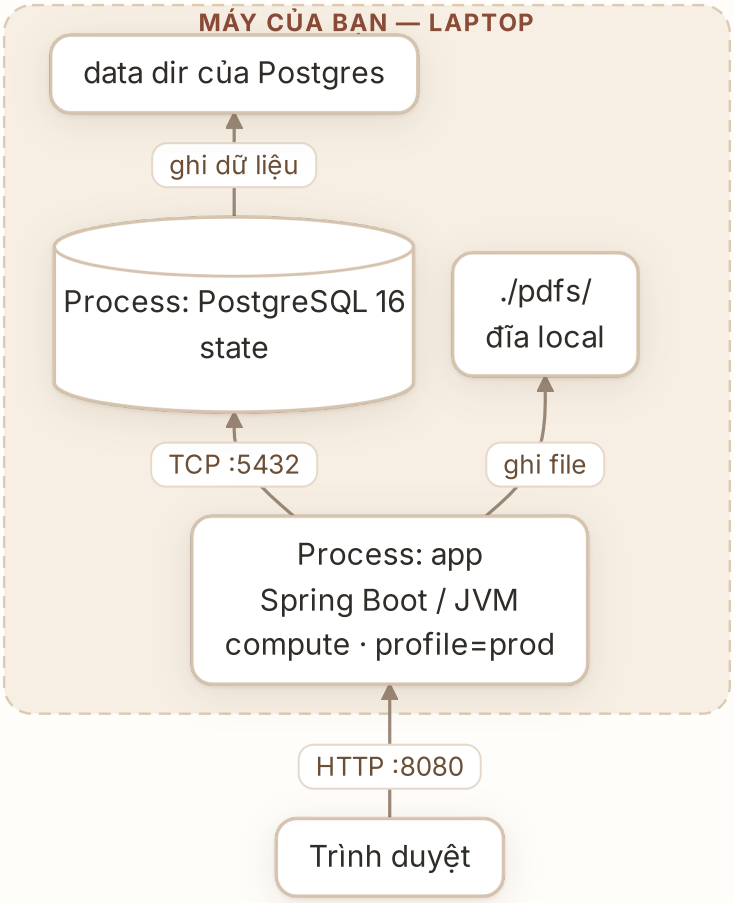
Ưu điểm: rất ít thành phần để chạy và debug. H2 nằm ngay trong process ứng dụng nên không cần một máy chủ database riêng, credential hay firewall rule cho kết nối tới database. Mô hình này giảm thiểu công việc chuẩn bị môi trường để tập trung vào luồng nghiệp vụ.

Nhược điểm: database sống chết *cùng* app — tắt app là mất sạch dữ liệu. Không thể có hai app cùng dùng chung dữ liệu. Máy này chết là cả hệ thống chết theo. Topology này không đáp ứng được yêu cầu production của TravelNow, nhưng nó đủ nhỏ để ta nhìn rõ từng bước tách bạch phía sau.

2.4 Giai đoạn 1 — Tách database thành process riêng (Local + PostgreSQL)

Bước đầu tiên hướng tới production: **database phải là một tiến trình độc lập, có vòng đời riêng.** Điều thú vị là ta chưa cần lên cloud để làm điều này — chỉ cần cài PostgreSQL lên chính máy local và chuyển app sang profile `prod`. Tên profile này chọn cấu hình database bên ngoài; nó không có nghĩa môi trường đã đáp ứng đầy đủ yêu cầu production.

HÌNH 2.4 · SƠ ĐỒ LUỒNG



Bây giờ có **hai process** thay vì một, và giữa chúng lần đầu tiên xuất hiện một **network hop**. App không còn "gọi hàm" tới database nữa — nó mở một kết nối TCP tới `localhost:5432`, đúng nghĩa một client nói chuyện với một server qua mạng, dù cuộc gọi chưa ra khỏi máy.

Chú ý dòng cấu hình quan trọng nhất:

```
SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/travelnow
```

Hãy nhớ kỹ dòng URL này. Xuyên suốt phần còn lại của chương, **contract truy cập database của app được giữ ổn định** — còn endpoint và credential đến từ cấu hình. Nói cho chính xác: bước chuyển H2 → PostgreSQL vẫn đòi hỏi đúng driver, migration và kiểm thử khác biệt SQL; nhưng từ PostgreSQL local lên PostgreSQL trên AWS thì business code không đổi.

Ưu điểm so với giai đoạn 0:

- Database có vòng đời riêng, nếu chúng ta restart app thì dữ liệu vẫn còn.
- Mô hình giao tiếp bước đầu đã có cùng phương thức với cloud — app là một database client nói chuyện qua TCP.

Nhược điểm: phức tạp hơn. Chúng ta phải cấu hình kết nối (URL, user, password), phải xử lý thứ tự khởi động (Postgres phải sẵn sàng trước khi app kết nối), và một lời gọi DB giờ *có thể* timeout hay bị từ chối. Ngoài ra, "zero setup" của chương 1 đã biến mất: ai muốn chạy hệ thống ở chế độ này phải cài Postgres, tạo database, tạo user bằng tay,... Hãy ghi nhớ cảm giác này; nó sẽ trở thành một chủ đề lớn ở Phần II.

2.5 Giai đoạn 2 — Thuê chiếc máy ảo đầu tiên: EC2

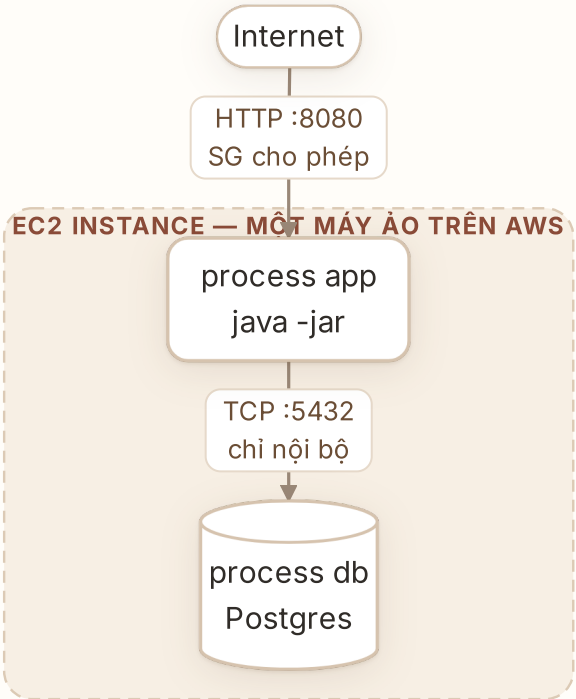
Giờ mới thực sự "lên cloud". Dịch vụ đầu tiên ta cần là thứ căn bản nhất mà mọi nhà cung cấp cloud đều có: **máy ảo** — trên AWS, nó tên là **EC2** (Elastic Compute Cloud).

Một EC2 instance là một lát cắt của máy chủ vật lý trong một AZ nào đó, được cấp cho bạn qua API trong vài phút. Bạn chọn **instance type** (bao nhiêu vCPU, bao nhiêu RAM), chọn **AMI** (ổ đĩa hệ điều hành khởi điểm — ví dụ Ubuntu), và trả chi phí theo phương án sử dụng. Bạn vẫn quản lý hệ điều hành, phần mềm và ổ đĩa, nhưng máy nằm trên hạ tầng AWS. Địa chỉ IP public và khả năng truy cập từ Internet phụ thuộc vào cấu hình mạng, không tự có với mọi instance.

2.5.1 App + DB trên cùng một EC2

Cách dễ nhất để bắt đầu: tái dựng y nguyên giai đoạn 1 trên một EC2 instance. Chúng ta SSH vào, cài JDK, cài PostgreSQL, copy file JAR lên, đăng ký app như một service của systemd để nó tự khởi động lại cùng máy.

HÌNH 2.5 · SƠ ĐỒ LUỒNG



Về topology, mô hình này **giống hệt giai đoạn 1** — chỉ là cái máy nằm ở Amazon thay vì trên bàn bạn. (Và để ý: "tái dựng y nguyên" nghĩa là lặp lại toàn bộ các bước cài đặt thủ công của giai đoạn 1, lần này qua SSH. Nỗi đau này đang được cộng dồn — Phần II chúng ta sẽ quay lại giải quyết nó.)

Nhưng có một thành phần AWS mới xuất hiện lần đầu: **Security Group** — tường lửa ảo bọc quanh EC2. Với một Security Group mới chưa có inbound rule, kết nối đến bị chặn. Trong ví dụ thử nghiệm, ta chỉ cho phép nguồn truy cập cần thiết vào cổng `8080`; cổng `5432` không cần mở ra ngoài vì app và database cùng máy. Khi phục vụ người dùng thực, đường vào cần HTTPS và cấu hình xác thực phù hợp.

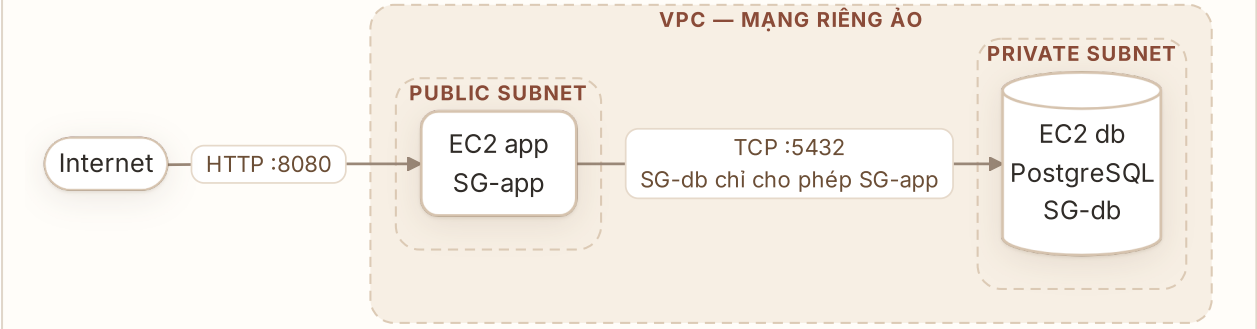
Đây là bài học vận hành đầu tiên về ranh giới mạng: kết nối thành công trên máy local chưa chứng minh rằng một đường truy cập qua mạng đã được cấu hình đúng. Trên cloud, mỗi cổng mở ra Internet là một cánh cửa cho kẻ tấn công. Nguyên tắc: *chỉ mở đúng thứ cần mở, cho đúng đối tượng cần truy cập.* (Chương 15 sẽ quay lại chủ đề bảo mật một cách hệ thống.)

Vấn đề của mô hình này: app và database cùng chung *số phận*. Chúng tranh nhau CPU và RAM của cùng một máy; muốn scale app thì vô tình đụng tới database; máy gặp sự cố thì cả ứng dụng và database đều có thể ngừng phục vụ. Ta vừa tách chúng thành hai process, nhưng chúng vẫn dính chặt vào cùng một phần cứng.

2.5.2 Tách DB sang một EC2 instance riêng

Bước tiếp theo tự nhiên: đặt database trên một máy ảo riêng.

HÌNH 2.6 · SƠ ĐỒ LUỒNG



Bây giờ compute và state nằm trên **hai máy khác nhau**, kết nối với nhau qua mạng riêng ảo của AWS — và đây là lần đầu network hop giữa app và DB đi *ra khỏi một máy*, băng qua mạng thật. Ba khái niệm hạ tầng xuất hiện cùng lúc:

- **VPC (Virtual Private Cloud)**: mạng riêng của bạn trên AWS, cách ly với mạng của khách hàng khác. Mọi thành phần bên trong nói chuyện với nhau bằng IP nội bộ (private IP).

- **Public subnet và private subnet**: public subnet có route trực tiếp tới Internet Gateway; private subnet không có route trực tiếp đó. Với IPv4 trong ví dụ, EC2 app còn cần địa chỉ public và các rule cho phép để nhận kết nối từ Internet. Database dùng địa chỉ private và chỉ nhận kết nối theo nhu cầu nội bộ. Tên loại subnet mô tả đường định tuyến, không tự thay thế việc kiểm soát truy cập.
- **Security Group tham chiếu nhau**: SG của DB cho phép cổng `5432` từ các network interface gắn SG-app. Cách này tránh phải cập nhật rule mỗi khi IP của app thay đổi. Nó kiểm soát nguồn kết nối ở tầng mạng; database vẫn cần xác thực và phân quyền riêng.

Tài liệu AWS về subnet mô tả sự khác nhau giữa các loại subnet và cấu hình địa chỉ IP.

Và dòng cấu hình quen thuộc chỉ đổi đúng phần host:

```
SPRING_DATASOURCE_URL = jdbc:postgresql://10.0.2.15:5432/travelnow
                                          └──── private IP của EC2 "db"
```

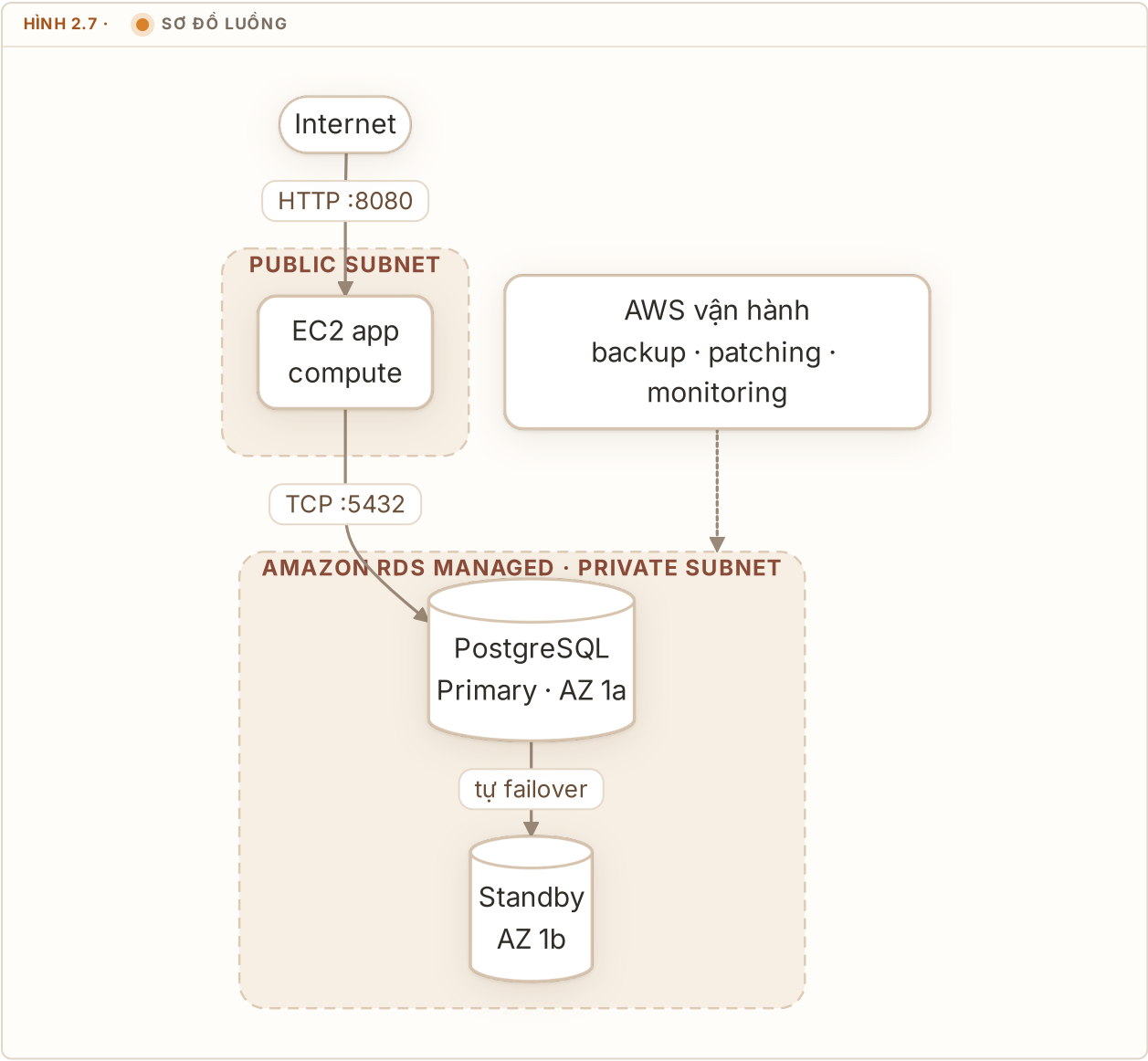
Ưu điểm: app và DB scale/restart độc lập. Chọn máy nhiều RAM cho DB, máy nhỏ cho app — hoặc ngược lại, tùy vào tải. App server chết không kéo theo dữ liệu.

Nhược điểm: bạn vẫn phải vận hành và bảo trì cái database đó — cài Postgres, vá lỗi bảo mật, cấu hình backup, và lo chuyện nó có thể sập lúc 2 giờ sáng. Vận hành database cần những kỹ năng khác với khởi chạy app, và những trách nhiệm đó hiện vẫn thuộc đội ngũ của bạn.

2.6 Giai đoạn 3 — Giao database cho AWS vận hành: RDS

Thay vì tự vận hành PostgreSQL trên EC2, TravelNow chọn **Amazon RDS** để chuyển một phần trách nhiệm đó cho AWS. Sơ đồ dưới đây minh họa cấu hình RDS Multi-AZ có một primary và một standby; Multi-AZ là lựa chọn cần bật, không phải thuộc tính mặc định của mọi database RDS.

HÌNH 2.7 · SƠ ĐỒ LUỒNG



Nhìn từ phía app, **gần như không có gì thay đổi** — vẫn một kết nối TCP tới cổng `5432`, vẫn một hostname trong connection string:

```
SPRING_DATASOURCE_URL = jdbc:postgresql://travelnow.abc123.ap-southeast-
1.rds.amazonaws.com:5432/travelnow
                                          └── endpoint do RDS cấp
```

Nhưng phía sau cái endpoint đó, bức tranh vật lý đã khác hẳn. Bạn không còn một cái máy để SSH vào nữa — bạn có một *dịch vụ*. Đường ranh shared responsibility ở mục 2.1 vừa dịch chuyển: AWS nhận về những phần khó nhất của việc vận hành database:

- **Automated backup** và point-in-time recovery khi bật sao lưu tự động với thời gian lưu giữ phù hợp. Đội ngũ vẫn phải đặt mục tiêu phục hồi và thử khôi phục dữ liệu.
- **Patching** tầng hạ tầng và engine, theo cửa sổ bảo trì bạn chọn. (Thành thực mà nói: chúng ta vẫn phải chọn thời điểm, kiểm thử tương thích và chuẩn bị cho khả năng gián đoạn — "managed" không có nghĩa là "không cần làm gì".)

- **Multi-AZ**: với cấu hình đang xét, RDS duy trì primary và standby ở hai Availability Zone khác nhau và tự động failover khi đủ điều kiện. Endpoint giữ nguyên, nhưng các kết nối đang mở sẽ bị ngắt, nên ứng dụng vẫn phải biết kết nối lại, và đội vận hành vẫn nên đo thời gian phục hồi thực tế. Đổi lại, chúng ta có **high availability** cho tầng dữ liệu mà không phải tự dựng cơ chế replication và failover từ đầu.

Về mặt kiến trúc, đây là bước hoàn tất việc **tách vòng đời state khỏi compute**: database giờ có endpoint ổn định và không sống cùng app server. Bạn có thể thay EC2 app, dựng lại rồi trỏ về cùng RDS mà không mất dữ liệu — miễn là backup, retention và quyền truy cập được cấu hình đúng. Compute đang trở thành thứ *có thể thay thế được* (disposable); state được bảo vệ theo một vòng đời riêng.

Nhược điểm: giá dịch vụ thường cao hơn phần compute thô tương đương, ít quyền kiểm soát OS hơn, và thêm một ranh giới trách nhiệm mới khi chẩn đoán sự cố. Đáng đánh đổi hay không phụ thuộc vào năng lực đội ngũ, nhu cầu tùy biến và chi phí thật của việc tự vận hành.

Khái niệm tổng quát rút ra ở đây là **managed service**: bạn thuê một dịch vụ được vận hành (một database với các khả năng backup và HA được cấu hình theo nhu cầu) thay vì mua một máy chủ về để *tự vận hành*. Mọi nhà cung cấp cloud đều có dải dịch vụ như vậy, và quyết định "tự vận hành hay dùng managed service" sẽ lặp lại với hầu hết mọi thành phần hạ tầng bạn gặp trong sự nghiệp.

2.7 Giai đoạn 4 — Tách File ra khỏi ổ đĩa với S3

Trong khi database đi từ in-process → EC2 → RDS, thành phần **storage** (file PDF) vẫn đang đứng yên: nó nằm trên đĩa của EC2 app.

Vấn đề của file trên đĩa local tinh vi hơn vấn đề của database, vì nó *chưa* gây ra vấn đề ngay lập tức. Chúng ta có một máy ảo, một app chạy trên đó, mọi thứ vẫn hoạt động tốt. Nhưng hãy để ý:

- File **dính vào vòng đời của máy**. Ở mục trước ta vừa nói EC2 app đang trở thành "thay thế được" — nhưng nếu chuyển sang instance khác là chúng ta sẽ để mất file (nếu không có phương án di chuyển hợp lý).
- Và một vấn đề đang chờ sẵn ở Phần IV: khi có **hai** app server để chịu tải, PDF do server A tạo ra nằm trên đĩa của server A; request tải file rơi vào server B thì… không thấy đâu cả.

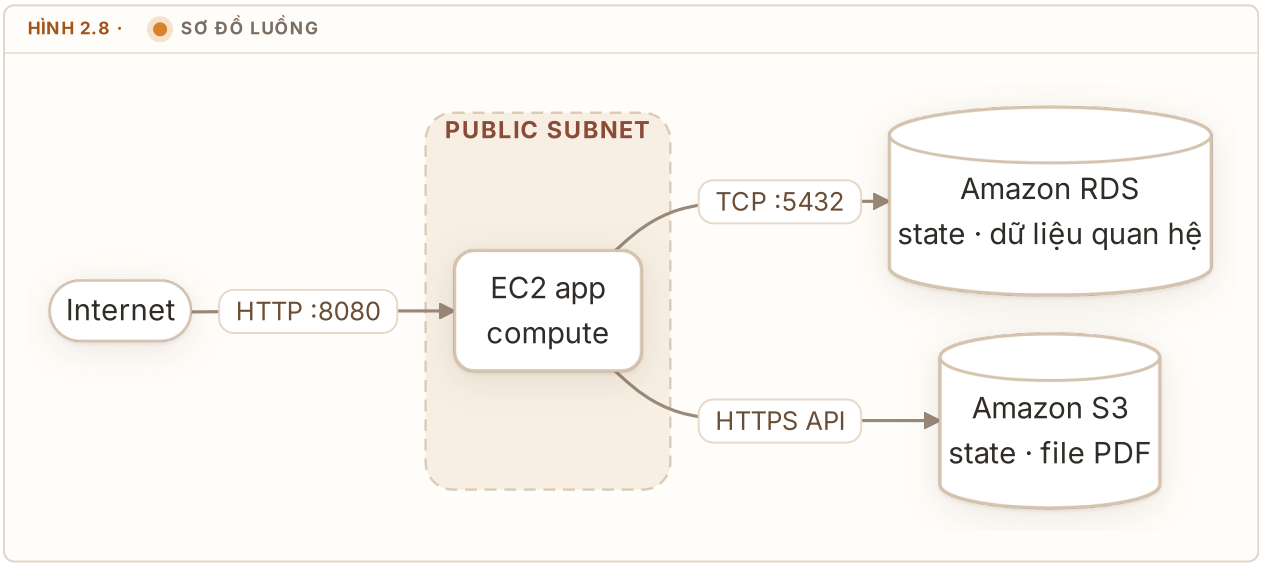
Đây chính xác là điều ta đã giải quyết với database nhưng xử lý cho file: *state bền vững còn nằm trong vòng đời của compute*. Lời giải trên AWS đó là **Amazon S3** (Simple Storage Service) — dịch vụ **object storage**.

Với general purpose bucket dùng trong eBook, S3 lưu **object** — dữ liệu cùng metadata — trong một **bucket được tạo tại Region đã chọn**. Mỗi object có một **key**, chẳng hạn `trips/abc-123/confirmation.pdf`; dấu `/` giúp nhóm tên theo prefix, không tạo ra thư mục như trên

filesystem. Ứng dụng dùng API để đọc, ghi và xóa object thay vì coi S3 như ổ đĩa local. Xem mô hình bucket của S3.

Mô hình này phù hợp với PDF của TravelNow vì hai lý do:

- **Lưu trữ độc lập với app**: lớp S3 Standard dùng ở đây lưu dữ liệu trên nhiều AZ. Đây không phải thuộc tính của mọi storage class; các lớp One Zone có phạm vi khác. Độ bền của hạ tầng cũng không bảo vệ khỏi mọi thao tác xóa hoặc ghi nhầm, nên vẫn cần phân quyền, versioning và chính sách lưu giữ phù hợp. Xem các lớp lưu trữ S3.
- **Truy cập dùng chung**: các app server có kết nối và quyền phù hợp đều truy cập cùng một object. PDF do server A tạo có thể được tải qua server B mà không phải chép file giữa hai máy.



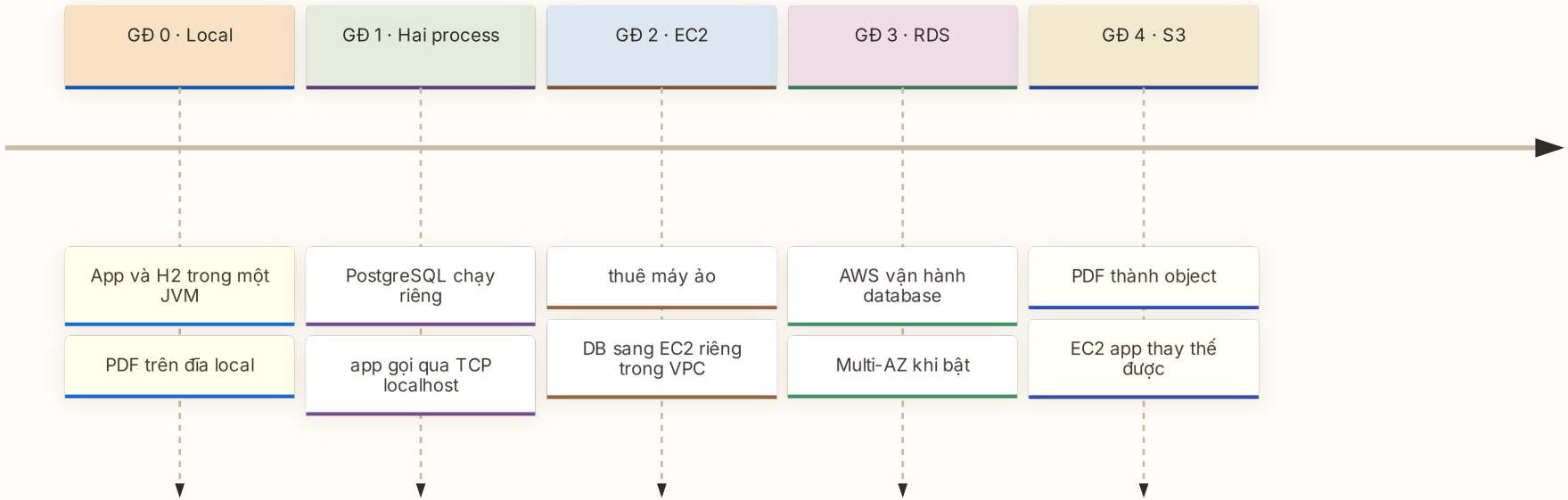
Còn nhớ điều số 3 trong bốn điều đáng ghi nhớ của chương 1? *Chỗ nào hạ tầng có thể thay đổi, chỗ đó có một ranh giới giao tiếp.* Khoản đầu tư đó giờ có chỗ dùng: việc lưu PDF đã nằm sau interface `PdfStorageService`, nên chuyển từ đĩa local sang S3 là viết một adapter mới dùng SDK của S3 — use case không đổi. Nhưng cũng như chương 1 đã cảnh báo, đổi hạ tầng không "miễn phí": adapter mới phải xử lý lỗi mạng, retry, quyền truy cập (app cần được *cấp quyền* ghi vào bucket — trên AWS, đó là việc của IAM), và ghi file giờ là một network hop có độ trễ thật thay vì một thao tác ghi ra đĩa.

Với S3, người dùng dịch vụ **không quản lý máy chủ lưu trữ**: không chọn số máy hay tự vá hệ điều hành. Đây là một ví dụ về mô hình serverless mà chương 13 sẽ phân tích thêm. Trách nhiệm vẫn còn ở cấp dịch vụ: chọn storage class, quyền truy cập, vòng đời dữ liệu và theo dõi chi phí lưu trữ, request, truyền dữ liệu.

2.8 Nhìn lại toàn bộ trục tiến hóa

Đặt tất cả cạnh nhau, ta thấy một trục duy nhất: **state ngày càng tách xa compute.**

HÌNH 2.9 · DÒNG THỜI GIAN



Thành phần	GĐ 0: Local	GĐ 1: Local, 2 process	GĐ 2: EC2	GĐ 3: + RDS	GĐ 4: + S3
App (compute)	trong JVM	JVM process	EC2	EC2	EC2
Database (state)	**trong JVM** (H2)	Postgres cạnh app	EC2 riêng	**RDS (managed)**	RDS
App ↔ DB	lời gọi hàm	TCP qua localhost	TCP qua VPC	TCP tới RDS endpoint	như GĐ 3
Storage (state)	đĩa local	đĩa local	đĩa EC2	đĩa EC2	**S3 (managed)**
DB độc lập với process app?	Không	Có	Có	Có	Có
PDF độc lập với ổ đĩa app?	Không	Không	Không	Không	Có
DB có HA trong cấu hình đang xét?	Không	Không	Không	Có, bật Multi-AZ	Có, bật Multi-AZ
Ai vận hành hạ tầng DB / file?	Bạn	Bạn	Bạn	AWS / bạn	AWS; bạn quản lý dữ liệu và cấu hình

Trong suốt các bước chuyển giữa các môi trường PostgreSQL, **connection string là thay đổi nhìn thấy rõ nhất ở phía ứng dụng:**

Giai đoạn	Database host trong JDBC URL	Mức ghép nối
1	`localhost`	Cùng máy
2	`10.0.2.15`	Private IP của một EC2 cụ thể
3+	`travelnow.abc123.ap-southeast-1.rds.amazonaws.com`	Endpoint ổn định do RDS cấp

Cùng một file JAR, cùng một dòng code `TripBookingService`. Đây là lý do cấu hình được đẩy ra **environment variable** thay vì hard-code: cùng một artifact chạy được trên các topology tương thích, chỉ khác nhau ở cấu hình lúc khởi động. (Tách config khỏi code loại bỏ nhu cầu build riêng cho từng môi trường — nhưng nó không xóa được khác biệt về runtime, dữ liệu, quyền hay network; ta sẽ còn quay lại điều này.)

2.9 Từ AWS nhìn ra cloud computing

Chương này đã đi qua các thành phần hạ tầng cơ bản trên AWS. Trước khi kết thúc, chúng ta cùng nhìn lại các khái niệm Cloud đã gặp.

Trên AWS	Khái niệm tổng quát	Dịch vụ cùng nhóm ở Azure / Google Cloud
Region / AZ	Phân vùng vật lý của hạ tầng	Region / Zone

Trên AWS	Khái niệm tổng quát	Dịch vụ cùng nhóm ở Azure / Google Cloud
EC2	Máy ảo (IaaS)	Virtual Machines / Compute Engine
VPC + subnet	Mạng riêng ảo	VNet / VPC
Security Group	Tường lửa mức instance	Network Security Group / firewall rules
RDS	Managed relational database	Azure Database / Cloud SQL
S3	Object storage	Blob Storage / Cloud Storage

Ba dịch vụ compute-và-state trong bảng còn xếp thành một dải — chính là đường ranh shared responsibility đang dịch chuyển dần trách nhiệm của bạn:

- **EC2** — thuê máy ảo: AWS lo phần cứng, *bạn lo từ OS trở lên* (cài đặt, vá lỗi, backup, HA).
- **RDS** — dùng database được quản lý: AWS vận hành hạ tầng và các cơ chế backup, failover đã chọn; bạn quản lý dữ liệu, quyền truy cập, truy vấn, cấu hình và kiểm chứng khả năng phục hồi.
- **S3** — dùng object storage: bạn không quản lý máy lưu trữ, nhưng vẫn quản lý dữ liệu, quyền truy cập, vòng đời và chi phí sử dụng.

Càng đi về bên phải, bạn càng đổi quyền kiểm soát lấy sự rảnh tay — và bài toán chọn điểm đứng trên dải này sẽ lặp lại suốt eBook.

Về giao tiếp, hệ thống đã xuất hiện bốn phạm vi kết nối. Mỗi phạm vi có những điểm cần kiểm chứng:

- **Trong process** (lời gọi hàm): độ trễ thấp nhất, kết quả trực tiếp — exception vẫn có, nhưng không có kiểu lỗi "mất phản hồi giữa đường". Giai đoạn 0.
- **Giữa các process cùng máy** (TCP qua localhost): đã có timeout, từ chối kết nối, giới hạn tài nguyên. Giai đoạn 1, 2.5.1.
- **Giữa các máy trong VPC** (mạng riêng AWS): cần đo độ trễ và xử lý lỗi kết nối, mất phản hồi hoặc timeout. Giai đoạn 2.5.2 trở đi.
- **Ra Internet** (client → app): chịu ảnh hưởng của mạng phía người dùng và đường truyền bên ngoài. Ngoài kiểm soát mạng, cần HTTPS, xác thực và phân quyền ở tầng ứng dụng.

Số network hop tăng lên cũng là lúc những câu hỏi ở lời mở đầu bắt đầu có sức nặng vật lý: *lời gọi qua mạng này thất bại thì sao? gửi lại có an toàn không?* Các câu hỏi ấy đã tồn tại ở monolith: client có thể mất phản hồi sau khi chuyến đi được commit, hoặc app mất kết nối khi chờ kết quả commit từ database. Tách thành nhiều service còn làm mất phạm vi một transaction chung cho toàn bộ luồng. Chương 7 phân tích cách xử lý kết quả chưa rõ; chương 8 xử lý việc phối hợp các bước đã commit riêng.

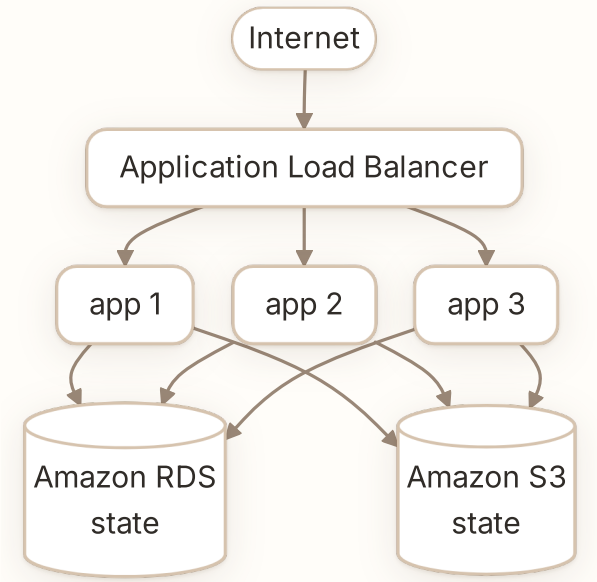
2.10 Tổng kết Phần I

Hệ thống đã "lên cloud" đúng nghĩa: một EC2 chạy app, một RDS giữ dữ liệu quan hệ, một bucket S3 dùng để lưu trữ file. Nghiệp vụ của chương 1 chạy được trên AWS, state đã tách khỏi compute, và những khái niệm cloud nền tảng — máy ảo, mạng riêng, tường lửa, managed service, object storage — đều đã xuất hiện theo đúng nhu cầu sinh ra chúng.

Nhưng hai bài toán đang chờ phía trước.

Bài toán thứ nhất — quy mô vật lý: vẫn chỉ có một app server. Máy đó chết là không ai đặt được vé — dù dữ liệu vẫn an toàn trong RDS và S3. Lời giải là nhân bản tầng compute:

HÌNH 2.10 · SƠ ĐỒ LUỒNG



Nhiều app server đứng sau một **Load Balancer**, tất cả cùng trỏ về một RDS và một S3 dùng chung. Vì app server không giữ state bền vững cục bộ (thành quả của toàn bộ chương này), nên ta có thể thay đổi số bản sao mà không phải di chuyển dữ liệu theo. (Để người dùng thực sự không nhận ra, còn cần readiness check, graceful shutdown, session phù hợp và đủ capacity — đây là nền tảng vật lý cho scalability và high availability, chưa phải bằng chứng đã đạt được chúng.)

Bài toán thứ hai — quy mô nghiệp vụ: khi team và tính năng phát triển, một đơn vị build/deploy/scale chung *có thể* trở thành nút thắt cổ chai. Cần đo lead time, ownership, coupling và nhu cầu scale trước khi kết luận monolith không còn phù hợp — Phần III sẽ tạo một bối cảnh đủ tín hiệu để TravelNow thử phân tách thành các service độc lập, và ghi lại cả chi phí của quyết định đó.

Cả hai hướng mở rộng đều làm tăng số lần phải dựng môi trường và triển khai ứng dụng. Ngay cả khi giữ nguyên một monolith application, việc cài runtime, sao chép tệp JAR và thiết lập cấu hình thủ công cũng dễ tạo sai lệch giữa các lần phát hành.

Phần II xử lý vấn đề đang có này: đóng gói ứng dụng bằng Docker, đưa kiểm thử và phát hành vào CI/CD, rồi quản lý hạ tầng bằng code. Nhờ đó, mỗi thay đổi có thể được lặp lại, kiểm tra và truy vết trước khi chúng ta cân nhắc mở rộng kiến trúc.

PHẦN II

Container hóa và Tự động hóa

Containerization & Automation

Làm sao để triển khai cùng một thay đổi nhiều lần mà vẫn kiểm soát được kết quả? Phần này chuẩn hóa cách đóng gói ứng dụng, kiểm thử và phát hành, rồi quản lý hạ tầng bằng code.

TravelNow vẫn là monolith application. Điều thay đổi là khả năng lặp lại, review và truy vết quá trình vận hành — những lợi ích cần thiết ngay cả khi chưa tăng số service.

Chương	Tiêu đề
3	Docker và Container hóa ứng dụng
4	CI/CD Pipeline từ Code đến Cloud
5	Infrastructure as Code — Khi hạ tầng được triển khai bằng code

CHƯƠNG 3

Docker và Container hóa ứng dụng

Phần II — Container hóa và Tự động hóa · *Containerization & Automation*

TravelNow đã lưu dữ liệu trên RDS và S3, nhưng mỗi lần triển khai ứng dụng lên EC2, chúng ta vẫn phải cài đặt Java, sao chép tệp JAR và cấu hình môi trường chạy. Nếu các máy sử dụng phiên bản Java khác nhau hoặc thiếu thư viện cần thiết, cùng một bản ứng dụng có thể chạy bình thường trên máy này nhưng gặp lỗi trên máy khác.

Chương này sử dụng Docker để **đóng gói ứng dụng cùng môi trường thực thi thành một image thống nhất**. Chúng ta sẽ tạo image cho TravelNow, chạy thử trên máy cá nhân rồi đưa lên Amazon ECR để EC2 tải về và triển khai. Qua đó, chúng ta làm rõ những thành phần được đóng gói trong image, những yêu cầu máy đích vẫn phải đáp ứng và cách xác định chính xác image được sử dụng trong mỗi lần phát hành.

3.1 Bài toán mà container giải quyết

3.1.1 Vấn đề: môi trường chạy được tái dựng bằng trí nhớ

Trước khi công nghệ container phổ biến, việc triển khai một ứng dụng từ máy dev lên server là một hành trình "tái dựng môi trường" gồm nhiều bước: cài đúng phiên bản Java, đúng biến môi trường, cấu hình đúng thư mục, đúng quyền,... và hy vọng không bỏ sót gì. Có một câu nói phổ biến: *"it works on my machine"*, đó là triệu chứng của một vấn đề có thật: **ứng dụng và môi trường chạy của nó là hai thứ tách rời**, trong đó phần môi trường được dựng lại thủ công ở mỗi máy đích mà chúng ta triển khai.

Đó chính xác là điều đã xảy ra ở chương 2. Và khi Phần III tách TravelNow thành **n** service, số môi trường cần chuẩn bị và kiểm tra sẽ tăng lên.

Container giải quyết vấn đề bằng cách đóng gói ứng dụng **cùng với** phần môi trường mà nó cần, bao gồm: runtime, thư viện, cấu trúc filesystem,... thành một đơn vị có thể định danh và phân phối. Image giảm sai lệch của phần môi trường được đóng gói; tính tương thích với nền tảng đích vẫn cần được kiểm tra.

3.1.2 Bản chất của container

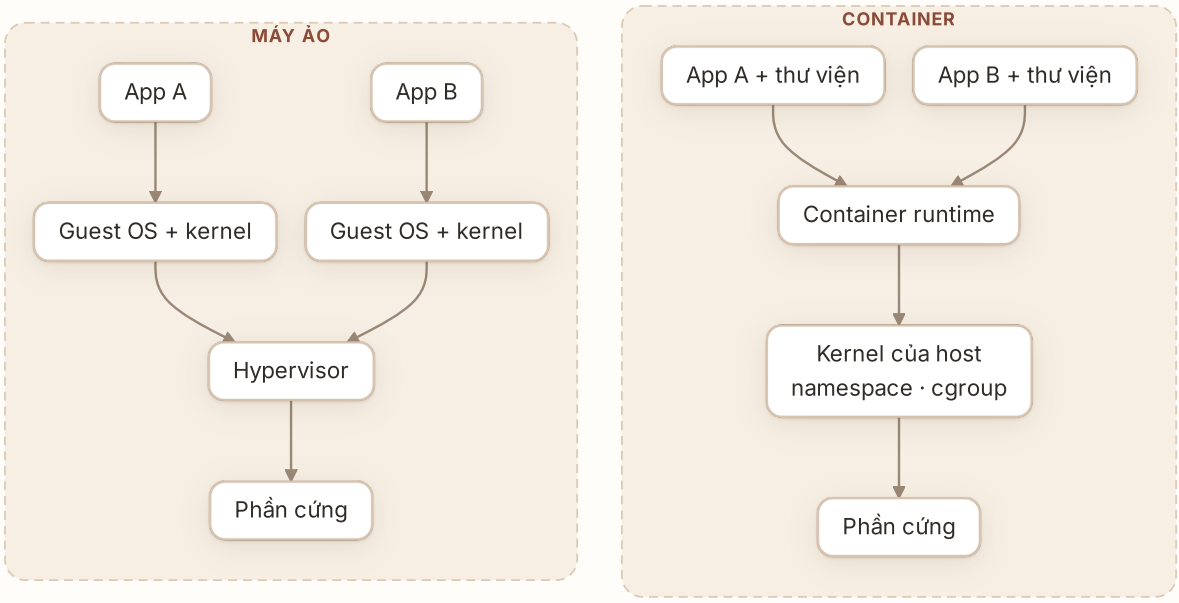
Điều dễ hiểu nhầm nhất là hình dung container như một "máy ảo nhỏ". Thực chất, **container chạy một hoặc nhiều process trên kernel của máy host**. Với Linux container, hai cơ chế chính kiểm soát cách ly và tài nguyên là namespace và cgroup; Docker còn quản lý filesystem của image và container:

- **Namespace** — giới hạn *những gì process nhìn thấy*: PID (chỉ thấy process trong container), mount (filesystem riêng), network (card mạng, bảng định tuyến riêng), user, IPC, hostname. Nhờ đó ứng dụng trong container tưởng mình đang chạy một mình trên một máy riêng.
- **Cgroup** (control group) — giới hạn *lượng tài nguyên process được dùng*: CPU, RAM, băng thông I/O. Đây là cơ chế đứng sau `--memory`, `--cpus` (sẽ nói ở mục 3.5).
- **Filesystem theo layer** (thường dùng cơ chế như OverlayFS) — ghép các **layer chỉ đọc** của image thành một cây thư mục, rồi phủ lên trên một **layer ghi được** riêng cho container. Vì vậy nhiều container từ cùng một image không nhân bản dữ liệu trên đĩa, và dữ liệu ghi vào layer riêng này sẽ mất khi container bị xóa; volume được gắn vào có vòng đời khác.

Điểm mấu chốt: **container dùng chung kernel với máy host.** Nó không boot một hệ điều hành riêng. Đây là khác biệt cơ bản so với máy ảo:

	Máy ảo (VM)	Container
Cơ chế cô lập	Hypervisor ảo hóa phần cứng	Namespace + cgroup của kernel host
Hệ điều hành	Mỗi VM một guest OS đầy đủ, có kernel riêng	Dùng chung kernel host, chỉ đóng gói phần user space
Thời gian khởi động	Hàng chục giây (phải boot OS)	Xấp xỉ thời gian khởi động một process
Dung lượng điển hình	Hàng GB	Vài chục đến vài trăm MB
Mức cách ly	Mạnh — ranh giới ở tầng phần cứng ảo	Yếu hơn — chung kernel, chung bề mặt tấn công

HÌNH 3.1 · SƠ ĐỒ LUỒNG



Hệ quả thực tế của việc chung kernel là một giới hạn cần biết trước: **image không mang theo kernel, nên tính di động của nó không phải vô điều kiện.** Một image build cho Linux/amd64 cần môi trường thực thi tương thích hoặc cơ chế giả lập để chạy trên máy ARM; muốn chạy native cần bản build cho kiến trúc đích, và không chạy trực tiếp trên kernel Windows hay macOS — Docker Desktop trên hai hệ điều hành đó thực chất chạy một máy ảo Linux ở bên dưới. Trong phạm vi một nền tảng đích đã xác định (ở đây là Linux/amd64 trên EC2), container cho ta môi trường chạy nhất quán hơn hẳn so với cài đặt thủ công. Đó là mức bảo đảm mà eBook này dựa vào.

3.1.3 Ba khái niệm cốt lõi

- **Image**: bản đóng gói tĩnh, gồm filesystem đã dựng sẵn cùng metadata (lệnh khởi chạy, biến môi trường mặc định, cổng...). Image được định danh bằng **digest** — một mã băm của nội dung, nên nội dung khác nhau thì digest khác nhau. Bên cạnh digest, image còn có **tag** (`travelnow:a1b2c3d`) cho dễ đọc; tag chỉ là một con trỏ và *có thể bị trỏ lại* sang image khác nếu registry không ngăn cấm. Với TravelNow, chúng ta sẽ thống nhất rằng: muốn thay đổi nội dung thì build image mới, không ghi đè tag cũ (mục 3.5).
- **Container**: một instance được tạo từ image, có cấu hình và trạng thái riêng; nó có thể đang chạy hoặc đã dừng. Từ cùng image có thể tạo nhiều container, nhưng cấu hình, dữ liệu và trạng thái lúc chạy của chúng vẫn có thể khác nhau. Khả năng tạo bản sao của ứng dụng là nền tảng cho việc scale ở Phần IV.
- **Registry**: kho lưu trữ và phân phối image, đóng vai trò với image giống như Git với source code: `docker push` để đẩy image lên, `docker pull` để kéo image về. Có nhiều registry phổ biến như Docker Hub, Google Artifact Registry, Harbor,... Mục 3.6 chúng ta sẽ dùng registry riêng trên AWS (ECR).

3.1.4 Layer và cache

Docker image không phải một khối duy nhất mà được cấu thành từ nhiều layer xếp chồng lên nhau. Các lệnh thay đổi filesystem như `RUN`, `COPY`, `ADD` có thể tạo layer dữ liệu; những lệnh như `ENV` hay `EXPOSE` thiết lập metadata. Vì vậy, không nên hiểu rằng mỗi dòng Dockerfile đều tạo một layer filesystem. Xem cách Docker tổ chức image layer. Cơ chế này giúp Docker tối ưu cả quá trình build, push và pull:

- **Khi build**: Docker tái sử dụng layer đã cache nếu lệnh và dữ liệu đầu vào không thay đổi. Nếu một layer thay đổi, cache từ layer đó trở đi sẽ bị vô hiệu và các layer phía sau phải build lại.
- **Khi push**: Docker chỉ đẩy lên registry những layer chưa tồn tại. Với TravelNow, mỗi lần phát hành một phiên bản mới thường chỉ có layer chứa file JAR là thay đổi; các layer nền đã nằm trên registry từ lần trước nên được tái sử dụng.
- **Khi pull**: Docker cũng chỉ tải những layer máy đích chưa có. Chẳng hạn, nếu máy đã có các layer của `eclipse-temurin:21-jre` từ một image khác thì chúng không cần được tải lại.

3.2 Dockerfile của TravelNow

Cách dựng một image được mô tả bằng một file công thức: **Dockerfile**. Ví dụ dưới đây giả định một ứng dụng Maven có `pom.xml` mô tả cách build, thư mục `src/` chứa mã nguồn và đầu ra là `target/travelnow-<phiên bản>.jar`. Ba tên này giúp đọc luồng đóng gói từ đầu vào đến image cuối:

```
# ─── Stage 1: Build ──────────────────────────────────────────────
FROM maven:3.9-eclipse-temurin-21 AS builder

WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q

COPY src ./src
RUN mvn package -DskipTests -q

# ─── Stage 2: Runtime ────────────────────────────────────────────
FROM eclipse-temurin:21-jre

WORKDIR /app
COPY --from=builder /app/target/travelnow-*.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

`-DskipTests` chỉ bỏ qua test trong bước đóng gói này. Quy trình phát hành ở chương 4 phải có bước kiểm thử riêng và kiểm tra image trước khi đưa cùng artifact sang môi trường tiếp theo.

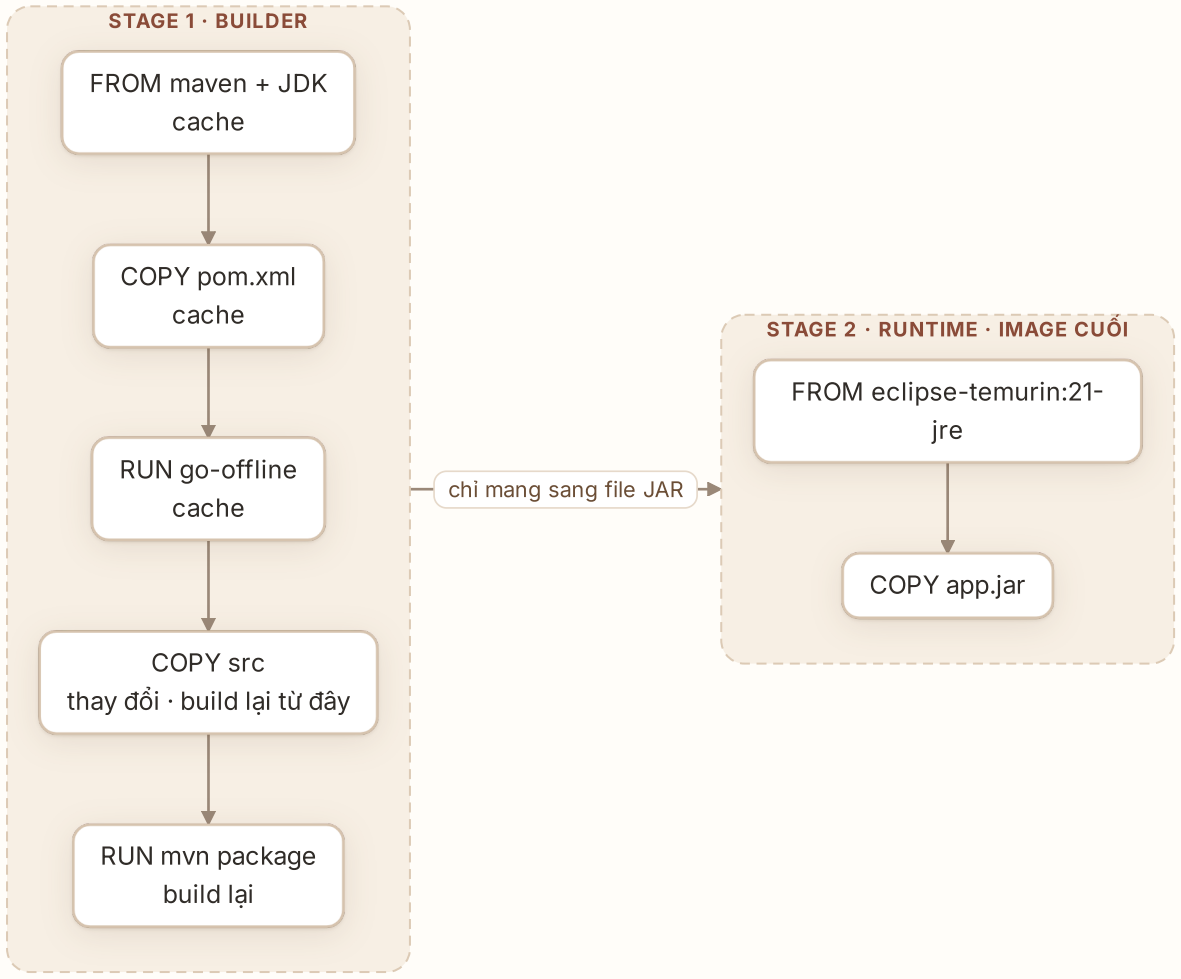
Có ba điểm đáng chú ý:

Multi-stage build. File này có hai chữ `FROM`, tức hai stage. Stage 1 chứa đầy đủ Maven và JDK để biên dịch; stage 2 bắt đầu lại từ một base image chỉ có JRE và **chỉ copy sang đúng file JAR** đã build. Toàn bộ compiler, dependency cache và source code bị bỏ lại ở stage 1, không có mặt trong image cuối cùng. Mức giảm dung lượng cụ thể phải đo trên image thật, nhưng nguyên tắc thì ổn định: image runtime càng ít package thì push/pull càng nhanh và bề mặt cần vá lỗi bảo mật càng nhỏ.

Thứ tự copy được xếp theo tần suất thay đổi. `pom.xml` được copy và dependency được tải về *trước*, source code copy vào *sau*. Vì layer tải dependency chỉ phụ thuộc `pom.xml`, những lần build mà ta chỉ sửa code Java, tức phần lớn các lần build trong ngày thì sẽ dùng lại layer dependency từ cache, giảm công việc phải thực hiện lại. Mức tiết kiệm thời gian phụ thuộc vào cache và môi trường build, cần được đo trên pipeline thật. Nguyên tắc chung là: **thứ ít thay đổi đặt lên trên, thứ hay thay đổi đặt xuống dưới.**

Với một lần build chỉ sửa code Java, hai stage và trạng thái cache trông như sau:

HÌNH 3.2 · SƠ ĐỒ LUỒNG



`ENTRYPOINT` **sử dụng exec form.** Cú pháp `["java", "-jar", "app.jar"]` khiến tiến trình Java được Docker khởi chạy **trực tiếp** và trở thành process chính (PID 1) của container. Nhờ đó, khi container bị dừng, tín hiệu `SIGTERM` được gửi thẳng tới JVM để ứng dụng có cơ hội thực hiện **graceful shutdown** trước khi thoát. Ngược lại, nếu viết `ENTRYPOINT` dưới dạng chuỗi (shell form), Docker sẽ chạy lệnh thông qua `/bin/sh -c`; khi đó shell trở thành PID 1 và JVM là tiến trình con, khiến việc chuyển tiếp tín hiệu không còn được đảm bảo. Vì vậy, exec form phù hợp hơn cho ứng dụng Java chạy trong container. Cơ chế graceful shutdown và ý nghĩa của nó sẽ được trình bày kỹ hơn ở mục 3.5.

Ngoài ra, `EXPOSE 8080` chỉ **khai báo rằng ứng dụng trong container sử dụng cổng 8080**, chứ không tự đưa cổng này ra bên ngoài. Muốn truy cập ứng dụng từ máy host, cần ánh xạ cổng khi khởi chạy container bằng tùy chọn `-p`, chẳng hạn `-p 8080:8080`.

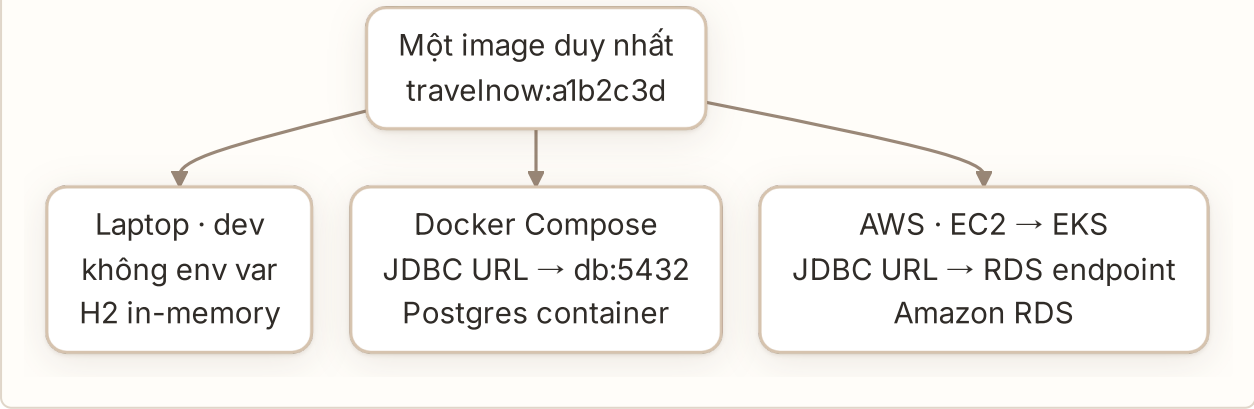
```
docker build -t travelnow:a1b2c3d .   # a1b2c3d = git commit SHA (xem mục 3.5)
docker run -p 8080:8080 travelnow:a1b2c3d
```

3.3 Một image cho nhiều môi trường

Một điểm đáng chú ý khác là những gì **không xuất hiện** trong Dockerfile: không có database URL, password hay cấu hình riêng cho từng môi trường. Image của TravelNow được xây dựng độc lập với nơi nó sẽ chạy; cùng một image có thể được triển khai ở dev, staging hay production. Các giá trị phụ thuộc vào môi trường chỉ được truyền vào **khi container khởi chạy**, chẳng hạn thông qua biến môi trường:

```
# application-prod.properties — connection lấy từ ENV, không nhúng vào image
spring.datasource.url=${SPRING_DATASOURCE_URL}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
```

HÌNH 3.3 · SƠ ĐỒ LUỒNG



Cách làm này tuân theo nguyên tắc "**store config in the environment**" của *The Twelve-Factor App*. Mục tiêu quan trọng là giữ nguyên artifact xuyên suốt quá trình triển khai: **image đã được kiểm thử cũng chính là image được đưa lên production.** Nếu mỗi môi trường lại build một image riêng, ta không còn đảm bảo rằng artifact chạy trên production chính là artifact đã vượt qua quá trình kiểm thử.

Tất nhiên, điều này không có nghĩa các môi trường sẽ hoàn toàn giống nhau. Dev và production vẫn có thể khác về dữ liệu, topology, quyền truy cập hay tải hệ thống. Việc dùng chung một image chỉ loại bỏ một nguồn sai lệch có thể kiểm soát được: **sự khác biệt của chính artifact triển khai**.

Tương tự, truyền password qua biến môi trường mới chỉ là một bước trong quá trình tách cấu hình khỏi ứng dụng, chưa phải giải pháp quản lý secret hoàn chỉnh. Ở chương 15, các thông tin nhạy cảm sẽ được chuyển sang AWS Secrets Manager. Có thể hình dung quá trình này theo ba bước: **nhúng cấu hình trong code → tách cấu hình khỏi artifact → quản lý thông tin nhạy cảm bằng secret store**.

3.4 Triển khai với Docker Compose

Ở chương 2, TravelNow với profile `prod` gồm hai thành phần: **ứng dụng và cơ sở dữ liệu (PostgreSQL)**. Khi chuyển sang container, mỗi thành phần có thể chạy trong một container riêng. Nếu sử dụng `docker run`, ta phải tự tạo network, gắn volume và kiểm soát thứ tự khởi động giữa các container. **Docker Compose** cho phép mô tả toàn bộ cấu hình đó trong một file duy nhất. PostgreSQL cũng không cần cài trực tiếp lên máy nữa mà được chạy từ một image có sẵn.

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: travelnow
      POSTGRES_USER: travelnow
      POSTGRES_PASSWORD: travelnow
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U travelnow"]
      interval: 5s
      retries: 10

  app:
    build: .
    environment:
      SPRING_PROFILES_ACTIVE: prod
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/travelnow
      SPRING_DATASOURCE_USERNAME: travelnow
      SPRING_DATASOURCE_PASSWORD: travelnow
    ports:
```

```
      - "127.0.0.1:8080:8080"
    depends_on:
      db:
        condition: service_healthy

volumes:
  pgdata:
```

Các password trong ví dụ chỉ dành cho môi trường local dùng dữ liệu mẫu; cổng app được bind vào localhost. Với cấu hình trên, chỉ cần chạy `docker compose up`, Compose sẽ tạo và kết nối các thành phần cần thiết để khởi động toàn bộ hệ thống. Có ba điểm quan trọng trong file này, đồng thời cũng là những khái niệm sẽ xuất hiện trở lại khi TravelNow chuyển sang Kubernetes ở Phần IV:

1. **Các service giao tiếp với nhau bằng tên:** Trong `jdbc:postgresql://db:5432`, chữ `db` là tên service chứ không phải một địa chỉ IP. Docker cung cấp DNS nội bộ để phân giải tên này thành địa chỉ của container tương ứng. Nhờ đó, ứng dụng chỉ cần biết tên của service cần gọi, thay vì phụ thuộc vào IP của container — vốn có thể thay đổi khi container được tạo lại. Đây chính là một dạng **service discovery**.
2. **Container đã khởi động chưa có nghĩa là ứng dụng bên trong đã sẵn sàng:** Docker có thể tạo và khởi động container PostgreSQL rất nhanh, nhưng PostgreSQL vẫn cần thời gian khởi tạo trước khi có thể nhận kết nối. `healthcheck` dùng `pg_isready` để xác định thời điểm database thực sự sẵn sàng, còn `depends_on` với condition: `service_healthy` yêu cầu Compose chờ trạng thái đó trước khi khởi động app. Điều kiện này kiểm soát thứ tự khởi động, không thay ứng dụng xử lý việc database mất kết nối sau đó. Sự khác biệt giữa **đã khởi động (started)** và **đã sẵn sàng phục vụ (ready)** cũng là nền tảng của cơ chế readiness probe trên Kubernetes.
3. **Dữ liệu được tách khỏi vòng đời container:** Volume `pgdata` lưu dữ liệu PostgreSQL bên ngoài filesystem riêng của container. Vì volume có vòng đời độc lập, container database có thể bị xóa và tạo lại mà dữ liệu vẫn được giữ nguyên. Với image PostgreSQL này, bỏ khai báo `pgdata` có thể khiến Docker tạo anonymous volume do image khai báo `VOLUME`. Dữ liệu chưa chắc bị xóa cùng container, nhưng việc nhận diện và gắn lại đúng volume khó kiểm soát hơn. Named volume làm rõ phần dữ liệu cần giữ; nó vẫn không thay thế backup. Xem vòng đời Docker volume.

Trong eBook, Compose phục vụ môi trường phát triển. Bản triển khai AWS ở cuối chương vẫn chạy trên một EC2 instance; Phần IV mới đặt thêm yêu cầu điều phối trên nhiều máy để nghiên cứu Kubernetes. Sự khác biệt về phạm vi điều phối, khả năng phục hồi và nguồn lực vận hành mới là cơ sở chọn công cụ, không phải tên gọi "dev" hay "production".

3.5 Container hóa đến đâu là đủ?

Mục 3.4 cho thấy PostgreSQL, vốn phải cài đặt và cấu hình thủ công ở chương 2, giờ có thể được khởi chạy từ một image chỉ với dòng `image: postgres:16-alpine`. Điều này dẫn đến một câu hỏi tự nhiên: **nếu container giúp triển khai database đơn giản như vậy, tại sao trên AWS không tiếp tục chạy PostgreSQL trong container cùng với ứng dụng, thay vì sử dụng RDS?**

3.5.1 Vì sao production vẫn dùng RDS thay vì container Postgres

Trước hết, cần xác định rõ **"container hóa" giải quyết bài toán gì**. Container giúp đóng gói ứng dụng cùng môi trường chạy để có thể triển khai nhất quán ở nhiều nơi. Với database, đây chỉ là một phần nhỏ của bài toán. Cài PostgreSQL bằng `apt install postgresql` hay khởi chạy bằng `docker run postgres` đều tương đối đơn giản; phần phức tạp nằm ở **việc vận hành database sau khi nó đã chạy**:

- Sao lưu định kỳ, kiểm tra tính toàn vẹn của bản sao lưu và hỗ trợ khôi phục tới một thời điểm cụ thể (*point-in-time recovery*).
- Duy trì replica dự phòng, phát hiện sự cố và thực hiện failover mà không làm mất dữ liệu đã commit.
- Nâng cấp phiên bản database, vá lỗi bảo mật và mở rộng dung lượng lưu trữ khi cần.
- Theo dõi các chỉ số đặc thù như số lượng connection, replication lag, slow query, lock và transaction kéo dài.

Container không tự giải quyết những vấn đề này; nó chủ yếu giúp **đóng gói và khởi chạy PostgreSQL**. Trong khi đó, Amazon RDS cung cấp sẵn nhiều khả năng vận hành như backup, replication, failover, patching và monitoring. Đây mới là giá trị chính mà TravelNow nhận được khi sử dụng một managed database thay vì tự vận hành PostgreSQL.

Ngoài vấn đề vận hành, còn có một khác biệt quan trọng về kiến trúc: **những lợi ích của container đối với tầng compute không hoàn toàn áp dụng được cho một primary database.**

Đặc tính	App server (compute)	Primary database (state)
Thay thế	Có thể xóa và tạo lại mà không làm mất dữ liệu nghiệp vụ	Việc thay thế phải đi kèm cơ chế bảo toàn và phục hồi dữ liệu
Nhân bản	Thêm instance/container giúp tăng khả năng xử lý request	Thêm container không đồng nghĩa với tăng khả năng ghi; dữ liệu vẫn cần được đồng bộ và kiểm soát nhất quán
Vị trí chạy	Có thể chuyển sang node khác tương đối dễ dàng	Phụ thuộc vào dữ liệu bền vững trên volume hoặc hệ thống storage bên ngoài

Đây chính là ranh giới giữa **compute và state** đã được đề cập ở chương 2, lần này nhìn từ góc độ container. Tầng compute được thiết kế để có thể thay thế và nhân bản dễ dàng, vì vậy rất phù hợp với container. Ngược lại, state phải tồn tại độc lập với vòng đời của tiến trình xử lý và cần những cơ chế riêng để bảo đảm độ bền, tính nhất quán và khả năng phục hồi.

Vậy tại sao PostgreSQL trong Docker Compose ở mục 3.4 lại phù hợp? Bởi môi trường development và production có mục tiêu khác nhau:

	Môi trường dev	Production trên AWS
PostgreSQL chạy bằng	Container (Docker Compose)	Amazon RDS
Mục tiêu chính	Dựng nhanh, xóa và tạo lại dễ dàng, chạy được trên máy developer	Độ bền dữ liệu, backup, HA và khả năng phục hồi
Khi mất dữ liệu	Dữ liệu mẫu có thể tạo lại; dữ liệu làm việc riêng vẫn cần được giữ	Là một sự cố cần xử lý theo mục tiêu phục hồi
Kiểm soát khác biệt	Dùng cùng major version với production và cùng bộ migration	—

Như vậy, việc dùng PostgreSQL container ở môi trường dev và RDS ở production không mâu thuẫn. **Dev ưu tiên khả năng dựng lại môi trường nhanh chóng; production ưu tiên độ bền và khả năng vận hành an toàn của dữ liệu.**

Điều này cũng không có nghĩa database **không nên chạy trong container**. Kubernetes cung cấp `StatefulSet` và `PersistentVolume` cho workload có state; các database operator như CloudNativePG còn có thể tự động hóa nhiều tác vụ như replication, failover và backup. Mô hình này phù hợp khi hệ thống có những yêu cầu như chạy on-premise, tránh phụ thuộc vào một nhà cung cấp cloud, cần tùy biến sâu database engine hoặc khi chi phí managed service không phù hợp.

Tuy nhiên, khi tự vận hành database theo cách đó, đội ngũ cũng phải **tự chịu trách nhiệm cho backup, replication, failover, nâng cấp, monitoring và phục hồi sự cố** — những công việc mà AWS hỗ trợ thực hiện một phần khi TravelNow sử dụng RDS ở chương 2. Vì TravelNow không có yêu cầu đặc biệt buộc phải tự vận hành PostgreSQL, RDS vẫn là lựa chọn phù hợp hơn cho production.

Nguyên tắc được sử dụng xuyên suốt phần còn lại của eBook là: **container hóa tầng compute; tách state khỏi vòng đời của container và giao nó cho thành phần được thiết kế để lưu trữ state bền vững.**

3.5.2 Một số lưu ý khi triển khai bằng container

Ở Phần IV, container của TravelNow sẽ không còn được khởi chạy thủ công bằng `docker run`. Thay vào đó, một orchestrator sẽ tự động tạo, theo dõi, dừng và thay thế container trong quá trình deploy, scale hoặc phục hồi sự cố. Điều đó đòi hỏi ứng dụng phải được thiết kế với giả định rằng **container có thể bị thay thế bất cứ lúc nào**. Có một số nguyên tắc cần chuẩn bị ngay từ bây giờ.

1. Không lưu dữ liệu cần tồn tại lâu dài trong container. Filesystem của container gắn với vòng đời của chính container; khi container bị xóa, dữ liệu ghi trực tiếp vào đó cũng mất theo. Vì vậy, mọi dữ liệu cần được bảo toàn phải nằm bên ngoài container: dữ liệu nghiệp vụ được lưu trong

RDS, file PDF được lưu trên S3 như đã thiết kế ở chương 2. Volume PostgreSQL trong Docker Compose ở mục 3.4 chỉ phục vụ môi trường development.

Container vẫn có thể lưu cache hoặc file tạm, miễn là việc mất chúng **không làm sai trạng thái nghiệp vụ**. Khi đó, orchestrator có thể tự do xóa, tạo lại hoặc nhân bản container mà không ảnh hưởng đến tính đúng đắn của hệ thống.

2. Ghi log ra `stdout` và `stderr`. Không nên lưu log vào file bên trong container, vì file đó sẽ biến mất khi container bị xóa. Hơn nữa, khi ứng dụng chạy trên nhiều container và nhiều node, việc truy cập từng container để đọc log riêng lẻ không còn khả thi.

Thay vào đó, ứng dụng chỉ cần ghi log ra `stdout` / `stderr`; hạ tầng chịu trách nhiệm thu thập, lưu trữ và cho phép tìm kiếm tập trung. Ở giai đoạn hiện tại, Docker có thể thu thập các luồng log này thông qua logging driver. Đến chương 14, cùng nguyên tắc đó sẽ được áp dụng với CloudWatch Logs.

3. Phân biệt "còn sống" và "sẵn sàng nhận request". Đây là hai trạng thái khác nhau và orchestrator cần phản ứng với chúng theo những cách khác nhau:

- **Liveness** trả lời câu hỏi: *tiến trình còn hoạt động bình thường hay đã rơi vào trạng thái cần khởi động lại?*
- **Readiness** trả lời câu hỏi: *container hiện tại đã sẵn sàng nhận traffic hay chưa?*

Một ứng dụng có thể vẫn sống nhưng chưa sẵn sàng, chẳng hạn khi đang khởi động, nạp cache hoặc tạm thời không truy cập được một dependency. Spring Boot Actuator hỗ trợ các health group tương ứng:

```
management.endpoints.web.exposure.include=health
management.endpoint.health.probes.enabled=true
```

Cần đặc biệt thận trọng khi đưa trạng thái của dependency bên ngoài vào liveness. Ví dụ, nếu liveness phụ thuộc vào RDS, một sự cố tạm thời của database có thể khiến hàng loạt container bị đánh giá là hỏng và đồng loạt khởi động lại. Khi đó, cơ chế vốn dùng để tự phục hồi lại có thể làm sự cố nghiêm trọng hơn. Dependency bên ngoài thường phù hợp hơn với readiness nếu việc dependency không khả dụng đồng nghĩa instance tạm thời không thể phục vụ request.

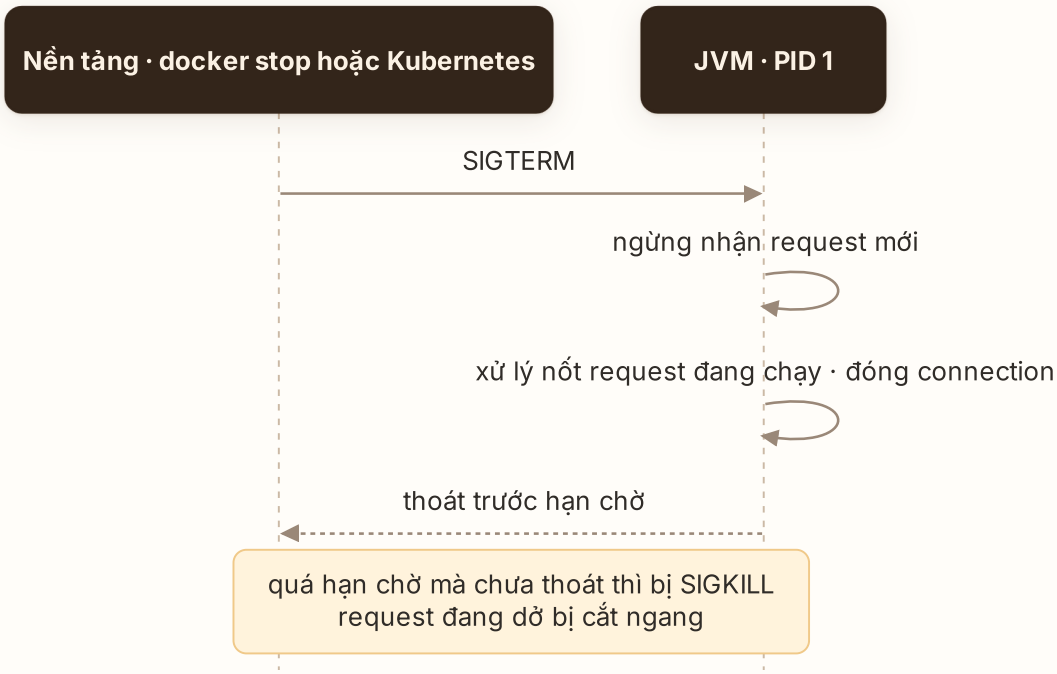
4. Cho phép ứng dụng tắt có trật tự (graceful shutdown). Khi container cần dừng, nền tảng trước tiên gửi `SIGTERM` cho tiến trình chính. Ứng dụng nên tận dụng khoảng thời gian này để ngừng nhận request mới, xử lý nốt các request đang thực hiện, đóng connection và giải phóng tài nguyên trước khi thoát.

Spring Boot có thể cấu hình:

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=25s
```

Để cơ chế này hoạt động đúng, có hai điều kiện. Thứ nhất, `SIGTERM` phải đến được JVM — đây là lý do `ENTRYPOINT` ở mục 3.2 sử dụng exec form. Thứ hai, thời gian graceful shutdown của ứng dụng phải nằm trong thời gian mà nền tảng cho phép trước khi buộc tiến trình dừng.

HÌNH 3.4 · SƠ ĐỒ TUẦN TỰ



Chẳng hạn, Kubernetes mặc định cho Pod khoảng 30 giây để kết thúc, nên timeout 25 giây ở trên vẫn nằm trong giới hạn đó. Trong khi đó, `docker stop` mặc định chỉ chờ 10 giây; nếu muốn kiểm thử đầy đủ cấu hình trên bằng Docker, có thể tăng thời gian chờ:

```
docker stop -t 30 <container>
```

Graceful shutdown đặc biệt quan trọng trong môi trường có orchestrator, vì việc container bị dừng là hoạt động bình thường khi deploy hoặc scale down. Nếu một request đặt chuyến đi bị cắt ngang giữa quá trình xử lý, client có thể rơi vào trạng thái không biết booking đã thành công hay chưa. Chương 7 sẽ quay lại chính tình huống này khi xây dựng các cơ chế bảo đảm tính nhất quán và khả năng phục hồi của nghiệp vụ.

5. Không chạy ứng dụng bằng quyền root. Theo mặc định, nếu image không chỉ định user khác, tiến trình trong container thường chạy với UID 0. Việc chạy ứng dụng với quyền cao hơn mức cần thiết làm tăng mức độ ảnh hưởng nếu ứng dụng hoặc container runtime xuất hiện lỗ hổng bảo mật.

TravelNow không cần quyền root để hoạt động, vì vậy có thể tạo một user riêng ngay trong runtime stage của Dockerfile:

```
RUN groupadd --gid 10001 appuser \
    && useradd --uid 10001 --gid 10001 --no-create-home appuser
USER 10001:10001
```

Sau thay đổi này, cần bảo đảm ứng dụng chỉ ghi vào những thư mục mà `appuser` có quyền truy cập và không yêu cầu các đặc quyền dành cho root. TravelNow sử dụng cổng 8080 nên cũng không cần quyền để bind vào privileged port. Đến chương 15, nguyên tắc **run as non-root** sẽ tiếp tục được áp dụng ở cấp Kubernetes.

6. Đặt giới hạn tài nguyên cho container. Nếu không có giới hạn, một container tiêu thụ bộ nhớ bất thường có thể ảnh hưởng đến các workload khác chạy trên cùng node. Vì vậy, production cần xác định rõ lượng CPU và memory mà mỗi container được phép sử dụng.

Với ứng dụng Java, cần quan tâm thêm đến cách JVM sử dụng giới hạn memory của container. Các JVM hiện đại nhận biết được giới hạn cgroup và tính kích thước heap dựa trên lượng memory đó, nhưng heap không phải toàn bộ bộ nhớ mà JVM sử dụng; còn có metaspace, thread stack, direct buffer và các vùng native memory khác. Vì vậy, không nên đặt heap bằng toàn bộ memory limit.

Ví dụ, với container có giới hạn 1 GiB, có thể dành khoảng 75% cho Java heap và giữ phần còn lại cho non-heap/native memory:

```
docker run \
  --memory=1g \
  -e JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75" \
  travelnow:a1b2c3d
```

Con số phù hợp cần được xác định bằng đo đạc thực tế thay vì coi `75%` là một giá trị cố định cho mọi ứng dụng. Khi chuyển sang Kubernetes, cùng nguyên tắc này sẽ được thể hiện thông qua resource requests và limits.

7. Sử dụng định danh image bất biến. Không nên triển khai production bằng một tag có thể bị ghi đè như `latest`. Nếu nội dung phía sau `latest` thay đổi theo thời gian, cùng một cấu hình deploy có thể kéo về những image khác nhau, khiến việc xác định chính xác phiên bản đang chạy và thực hiện rollback trở nên khó khăn.

TravelNow sử dụng **Git commit SHA** làm image tag, ví dụ:

```
travelnow:a1b2c3d
```

Tag này liên kết image với commit đã tạo ra nó. Ở mức chặt chẽ hơn, mỗi image còn có một **digest** xác định chính xác nội dung của image, chẳng hạn:

```
travelnow@sha256:<digest>
```

Nhờ sử dụng định danh bất biến, việc rollback phần ứng dụng có thể thực hiện bằng cách triển khai lại image của phiên bản trước. Tuy nhiên, rollback image không tự động hoàn tác database migration hoặc những side effect đã xảy ra; các thay đổi đó cần chiến lược rollback riêng.

Sang chương 4, việc build image và sinh tag từ Git commit SHA sẽ được đưa vào CI/CD pipeline, để định danh phiên bản trở thành một phần tự động của quy trình phát hành thay vì phụ thuộc vào thao tác thủ công.

3.6 Amazon ECR — registry trên AWS

Sau khi build, image mới chỉ tồn tại trên máy thực hiện quá trình build. Để các máy khác như EC2, và sau này là các node trong EKS, có thể tải và chạy image đó, chúng ta cần lưu nó trên một **container registry**.

Với TravelNow, toàn bộ hạ tầng production nằm trên AWS nên registry được sử dụng là **Amazon ECR (Elastic Container Registry)**. ECR lưu trữ các container image trong các repository private và tích hợp trực tiếp với IAM. Nhờ đó, quyền push và pull image có thể được quản lý bằng cùng cơ chế phân quyền đang sử dụng cho các tài nguyên AWS khác, thay vì phải duy trì thêm tài khoản và credential riêng cho registry.

Quy trình đưa image của TravelNow lên ECR gồm ba bước. Thay account ID mẫu `123456789012` bằng ID tài khoản của bạn. Repository được tạo với tag bất biến để không ghi đè một phiên bản đã phát hành:

```
# 1. Tạo repository cho service (chỉ cần thực hiện một lần)
aws ecr create-repository \
  --repository-name travelnow \
  --image-tag-mutability IMMUTABLE \
  --region ap-southeast-1

# 2. Xác thực Docker với ECR
aws ecr get-login-password --region ap-southeast-1 \
  | docker login \
      --username AWS \
      --password-stdin 123456789012.dkr.ecr.ap-southeast-1.amazonaws.com

# 3. Gắn địa chỉ ECR vào tên image và push
docker tag travelnow:a1b2c3d \
  123456789012.dkr.ecr.ap-southeast-1.amazonaws.com/travelnow:a1b2c3d

docker push \
  123456789012.dkr.ecr.ap-southeast-1.amazonaws.com/travelnow:a1b2c3d
```

Ở bước 2, AWS CLI lấy một authorization token tạm thời từ ECR và truyền token đó cho Docker qua `--password-stdin`. Token có hiệu lực trong 12 giờ, vì vậy không cần lưu một mật khẩu ECR cố định trên máy build. Máy hoặc pipeline thực hiện thao tác này vẫn phải có IAM permission phù hợp để truy cập ECR.

Bước 3 có một chi tiết quan trọng: **tên đầy đủ của image đồng thời xác định registry mà Docker sẽ push tới**. Image ban đầu có tên:

```
travelnow:a1b2c3d
```

chỉ tồn tại trong local image store. Sau khi `docker tag`, image có thêm tên đầy đủ:

```
123456789012.dkr.ecr.ap-southeast-1.amazonaws.com/travelnow:a1b2c3d
```

Trong đó, `123456789012.dkr.ecr.ap-southeast-1.amazonaws.com` xác định ECR registry, `travelnow` là repository và `a1b2c3d` là tag của phiên bản. `docker tag` không tạo thêm một bản sao của image; nó chỉ gắn thêm một tên cho cùng image để Docker biết image cần được đẩy tới đâu.

Khi `docker push` được thực hiện lần đầu, ECR chưa có các layer của image nên Docker phải upload toàn bộ. Ở những lần phát hành sau, Docker đối chiếu digest của từng layer với registry và chỉ upload những layer ECR chưa có. Với Dockerfile của TravelNow, nếu base image và dependency không thay đổi thì một thay đổi trong source code thường chỉ làm thay đổi các layer cuối liên quan đến file JAR. Cơ chế layer vì vậy không chỉ giúp tăng tốc build mà còn giảm lượng dữ liệu phải truyền khi phân phối image.

Ngoài việc lưu trữ image, repository production nên có thêm hai cơ chế quản lý:

Image scanning. ECR có thể quét các package và dependency có trong image để phát hiện các lỗ hổng bảo mật đã biết. Tuy nhiên, kết quả scan chỉ thực sự có giá trị khi được đưa vào quy trình phát hành: cần xác định mức độ nghiêm trọng nào sẽ chặn deployment, trường hợp nào được chấp nhận và ai chịu trách nhiệm xử lý. Chương 15 sẽ đưa bước kiểm tra này vào quy trình bảo mật của TravelNow.

Lifecycle policy. Mỗi lần phát hành tạo thêm một phiên bản image và theo thời gian repository sẽ tích lũy ngày càng nhiều image cũ. Lifecycle policy cho phép ECR tự động dọn dẹp các image không còn cần thiết theo những quy tắc định trước. Số phiên bản cần giữ lại không nên được chọn tùy ý mà phải đủ cho nhu cầu rollback, điều tra sự cố và lưu vết. Đặc biệt, không được xóa một image vẫn đang được bất kỳ môi trường nào sử dụng.

Từ thời điểm này, ECR trở thành **điểm trung chuyển artifact** giữa quá trình build và quá trình triển khai: máy build tạo image và push lên ECR; EC2 hoặc EKS chỉ cần pull đúng định danh image đã được phát hành. Sang chương 4, các thao tác build, gắn tag và push này sẽ được chuyển vào CI/CD pipeline thay vì thực hiện thủ công.

3.7 Deploy container lên AWS

Đến đây, TravelNow đã có đầy đủ các thành phần cần thiết để chuyển từ cách triển khai ở chương 2 — copy file JAR lên EC2 và cài JDK trực tiếp trên máy — sang triển khai bằng container. Kiến trúc ứng dụng chưa thay đổi: TravelNow vẫn chạy trên EC2 và kết nối tới RDS. Điểm khác biệt nằm ở **cách ứng dụng được đóng gói và đưa lên máy chủ**.

Ví dụ chuẩn bị EC2 dưới đây dùng Amazon Linux 2023. Máy cũng cần AWS CLI và đường mạng tới ECR; người thực hiện các lệnh Docker phải có quyền sử dụng Docker daemon. Docker được cài đặt một lần:

```
# Trên EC2 — chuẩn bị một lần
sudo dnf install -y docker
sudo systemctl enable --now docker
```

EC2 cũng cần quyền pull image từ ECR. Thay vì lưu `AWS_ACCESS_KEY_ID` và `AWS_SECRET_ACCESS_KEY` trực tiếp trên máy, chúng ta gán cho instance một **IAM role** có quyền đọc ECR thông qua instance profile. Ứng dụng và AWS CLI trên EC2 khi đó có thể sử dụng credential tạm thời do AWS cấp cho role này.

Cách làm này loại bỏ nhu cầu lưu access key dài hạn trên server. Credential của instance role được AWS tự động cấp và xoay vòng, trong khi một access key được ghi cố định trong file cấu hình có thể tồn tại lâu dài và trở thành rủi ro nếu bị lộ.

Sau bước chuẩn bị ban đầu, mỗi lần triển khai một phiên bản mới chỉ còn gồm ba thao tác chính: **pull image mới, dừng container cũ và khởi chạy container mới**.

```
# Trên EC2 — mỗi lần deploy
# Thay account ID mẫu bằng tài khoản của bạn
ECR=123456789012.dkr.ecr.ap-southeast-1.amazonaws.com

# 1. Xác thực Docker với ECR và pull đúng phiên bản cần deploy
aws ecr get-login-password --region ap-southeast-1 \
  | docker login \
      --username AWS \
      --password-stdin "$ECR"

docker pull "$ECR/travelnow:a1b2c3d"

# 2. Dừng và xóa container của phiên bản hiện tại
docker stop -t 30 travelnow 2>/dev/null || true
docker rm travelnow 2>/dev/null || true

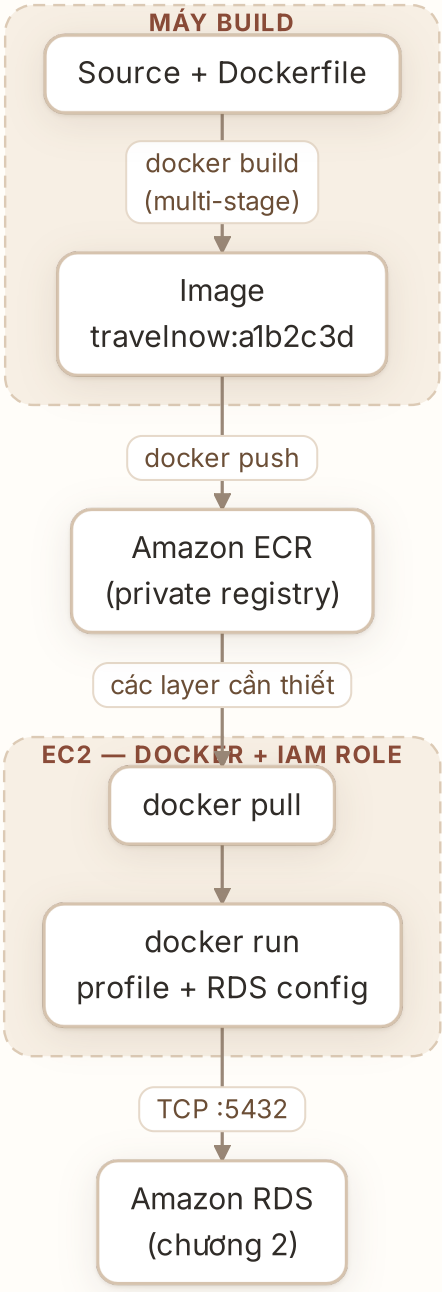
# 3. Khởi chạy phiên bản mới
docker run -d \
  --name travelnow \
  -p 8080:8080 \
  --restart unless-stopped \
  --memory=1g \
  -e JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75" \
  -e SPRING_PROFILES_ACTIVE=prod \
```

```
  -e SPRING_DATASOURCE_URL="jdbc:postgresql://<rds-endpoint>:5432/travelnow" \
  -e SPRING_DATASOURCE_USERNAME="..." \
  -e SPRING_DATASOURCE_PASSWORD="..." \
  "$ECR/travelnow:a1b2c3d"
```

Lệnh `docker stop -t 30` gửi `SIGTERM` tới tiến trình chính trong container và cho ứng dụng tối đa 30 giây để kết thúc. Đây chính là khoảng thời gian dành cho cơ chế graceful shutdown đã cấu hình ở mục 3.5.2. Sau khi container cũ được xóa, `docker run` tạo container mới từ image vừa pull và truyền vào các cấu hình dành riêng cho môi trường production.

Toàn bộ hành trình của một phiên bản TravelNow từ source code đến khi chạy trên AWS có thể hình dung như sau:

HÌNH 3.5 · SƠ ĐỒ LUỒNG



So với chương 2, thay đổi quan trọng nhất nằm ở **vai trò của EC2**. Trước đây, server phải được chuẩn bị theo công nghệ của ứng dụng: TravelNow viết bằng Java nên máy cần JDK, file JAR và các bước cấu hình tương ứng. Khi sử dụng container, những dependency cần thiết để chạy ứng dụng đã nằm trong image. EC2 chỉ cần container runtime và quyền pull image từ ECR.

Nói cách khác, việc **chuẩn bị môi trường chạy** đã được chuyển từ server vào image. Nếu sau này TravelNow có thêm một service viết bằng Python, Go hay một phiên bản Java khác, EC2 không cần cài thêm runtime tương ứng; mỗi service mang theo môi trường chạy của chính nó trong image.

Điều không thay đổi là **kiến trúc của hệ thống**. Container vẫn kết nối tới RDS được xây dựng ở chương 2, và các cấu hình phụ thuộc môi trường vẫn được truyền vào khi container khởi chạy. Chúng ta chỉ thay đổi cách đóng gói và phân phối ứng dụng:

```
Chương 2:
Source → build JAR → copy JAR lên EC2 → chạy bằng JDK trên EC2

Chương 3:
Source → build image → push ECR → EC2 pull image → chạy container
```

Cách triển khai hiện tại vẫn còn một hạn chế: TravelNow chỉ có **một instance ứng dụng**. Khi deploy, container cũ phải dừng trước khi container mới sẵn sàng nhận request, vì vậy sẽ xuất hiện khoảng gián đoạn phụ thuộc thời gian dừng và khởi động thực tế.

Container hóa tự nó không giải quyết vấn đề này. Muốn triển khai mà không làm gián đoạn dịch vụ, hệ thống cần nhiều instance chạy song song, một load balancer để phân phối traffic và một cơ chế thay thế các instance theo từng bước thay vì dừng tất cả cùng lúc. Đây sẽ là một trong những vấn đề được giải quyết ở Phần IV khi TravelNow chuyển sang Kubernetes.

3.8 Tóm tắt

TravelNow giờ được đóng gói và phân phối dưới dạng Docker image, với định danh để xác định chính xác nội dung của mỗi bản phát hành. Dockerfile mô tả các bước tạo image, Amazon ECR lưu trữ và phân phối image, còn cấu hình riêng của từng môi trường được cung cấp khi container khởi chạy.

Container hóa giúp môi trường thực thi nhất quán hơn giữa các máy. Để ứng dụng hoạt động ổn định, chúng ta vẫn cần lưu dữ liệu bền vững bên ngoài container, xử lý tín hiệu dừng, cung cấp thông tin về trạng thái sẵn sàng và cấu hình quyền truy cập, tài nguyên phù hợp. Database chạy bằng Docker Compose phục vụ việc phát triển và kiểm thử trên máy cá nhân; khi triển khai trên AWS, TravelNow tiếp tục sử dụng RDS để lưu trữ dữ liệu nghiệp vụ.

Sau khi thống nhất cách đóng gói ứng dụng, câu hỏi tiếp theo là **làm sao kiểm tra và triển khai đúng image tới đúng môi trường theo một quy trình nhất quán?** Chương 4 sẽ tự động hóa các bước build, test, push và deploy thành một CI/CD pipeline, với điều kiện rõ ràng để một bản phát hành được phép triển khai.

CHƯƠNG 4

CI/CD Pipeline từ Code đến Cloud

Phần II — Container hóa và Tự động hóa · *Containerization & Automation*

Chương 3 đã thống nhất cách đóng gói TravelNow thành Docker image và cung cấp cấu hình riêng cho từng môi trường. Tuy nhiên, mỗi lần triển khai vẫn cần người thực hiện lần lượt các bước build, test, push image và thay thế container. Chương này đưa các bước đó vào **pipeline**: một quy trình tự động có đầu vào xác định, điều kiện để chuyển sang bước tiếp theo và kết quả có thể kiểm tra.

Chúng ta sẽ tìm hiểu cách sử dụng cùng một image qua các môi trường, giới hạn quyền của từng job và xác nhận phiên bản mới đã sẵn sàng phục vụ. Sau phần khái niệm, GitHub Actions được dùng để xây dựng pipeline minh họa cho TravelNow. Tự động hóa giúp quy trình được thực hiện nhất quán hơn; độ tin cậy vẫn phụ thuộc vào chất lượng kiểm thử, tính chính xác của cấu hình và phạm vi quyền được cấp cho từng bước.

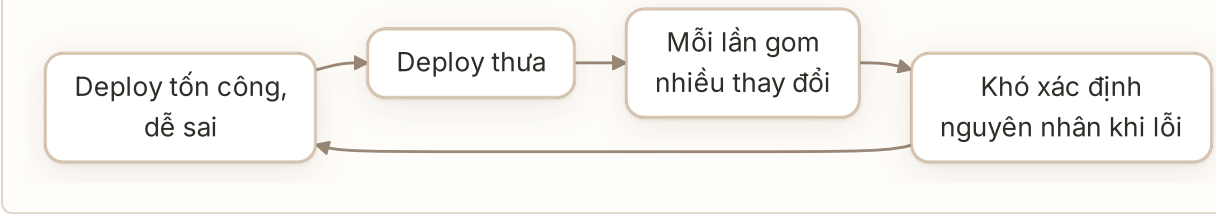
4.1 Vì sao phải tự động hóa

Ở cuối chương 3, quy trình triển khai đã có thể viết thành một chuỗi lệnh. Nếu một lần phát hành bỏ sót bước pull image hoặc dùng nhầm tag, phiên bản thực tế sẽ khác với điều người triển khai dự định. Khi có nhiều môi trường và phát hành thường xuyên, số cơ hội mắc lỗi cũng tăng lên.

Loại công việc thủ công, lặp lại, có thể tự động hóa và tăng theo quy mô thường được gọi là **toil**. Thời gian dành cho toil khiến đội ngũ có ít thời gian hơn để cải thiện hệ thống. Google SRE — Eliminating Toil.

Một hệ quả khác nằm ở cách tổ chức phát hành. Deploy càng tốn công, đội ngũ càng có xu hướng gom nhiều thay đổi vào một đợt. Khi có lỗi, phạm vi cần điều tra vì thế rộng hơn:

HÌNH 4.1 · SƠ ĐỒ LUỒNG



Pipeline giúp giảm công sức của mỗi lần phát hành, tạo điều kiện triển khai các thay đổi nhỏ hơn. Lợi ích này cần đi cùng bộ test đáng tin cậy, kiểm tra sau triển khai và đường rollback rõ ràng; chỉ tăng tần suất deploy là chưa đủ để làm hệ thống an toàn hơn.

Cuối cùng, một điều cần tỉnh táo: **tự động hóa khuếch đại cả sai sót.** Một lệnh sai trong pipeline sẽ được thực hiện chính xác và nhất quán trên mọi môi trường, nhanh hơn nhiều so với một người làm sai bằng tay. Vì vậy pipeline phải đi kèm các điều kiện kiểm tra và giới hạn quyền phù hợp:

- **Quality gate** — điều kiện tự động chặn luồng chạy tiếp: test không đạt thì không phát hành image để deploy.
- **Guardrail** — giới hạn giúp ngăn thao tác vượt phạm vi cho phép, chẳng hạn quyền riêng cho production hoặc quy tắc bắt buộc review. Phê duyệt thủ công là một cách kiểm soát trong ví dụ, không phải định nghĩa của mọi guardrail.

4.2 CI, CD và CD — ba mức tự động hóa

Ba chữ viết tắt này thường bị dùng lẫn lộn, trong khi chúng mô tả **ba mức độ nối tiếp nhau**, mức sau bao hàm mức trước:

Mức	Tên đầy đủ	Tự động hóa đến đâu	Điều kiện để đạt được
CI	Continuous Integration	Tích hợp thường xuyên vào nhánh chung, mỗi thay đổi được build và test; required checks chặn merge khi lỗi	Có bộ test chạy nhanh và đáng tin cậy
CD	Continuous **Delivery**	Mọi commit qua CI đều được đóng gói thành artifact hoàn chỉnh, *sẵn sàng* deploy	Artifact bất biến, môi trường dựng lại được
CD	Continuous **Deployment**	Bỏ nốt thao tác bấm nút: qua test là tự lên production	Các cổng tự động, quan sát sau phát hành và cơ chế phục hồi đáp ứng mức rủi ro

Giá trị cốt lõi của **CI** là *tín hiệu sớm*: một thay đổi làm hỏng hệ thống bị phát hiện sau vài phút thay vì vài tuần, khi tác giả của nó còn nhớ mình vừa sửa gì và khi phạm vi nghi vấn còn nhỏ.

Khác biệt giữa hai chữ **CD** chỉ nằm ở đúng một chỗ: ai bấm nút phát hành. Continuous Delivery giữ lại quyết định đó cho con người, nhưng bắt buộc mọi thứ trước quyết định ấy phải sẵn sàng — nghĩa là bấm nút xong thì không còn công đoạn thủ công nào nữa. Continuous Deployment bỏ luôn nút bấm.

Trong eBook, TravelNow chọn **CI kết hợp Continuous Delivery**: môi trường dev được triển khai tự động, còn production có bước phê duyệt. Đây là chính sách phát hành của ví dụ. Một người phê duyệt cần có kết quả kiểm thử và thông tin thay đổi để quyết định; sự hiện diện của nút phê duyệt tự nó không làm deployment an toàn hơn.

4.3 Giải phẫu một pipeline

Các nền tảng CI/CD có nhiều khái niệm tương ứng về kích hoạt, thực thi và chuyển artifact. Nắm các khái niệm này giúp đọc workflow của công cụ khác; cách cô lập runner, quản lý quyền và áp dụng quy tắc phê duyệt vẫn cần được tìm hiểu riêng.

4.3.1 Bộ từ vựng chung

- **Trigger (sự kiện kích hoạt)** — điều gì làm pipeline chạy: một commit đẩy lên nhánh main, một pull request được mở, một lịch định kỳ, hoặc một lệnh thủ công.
- **Stage (chặng)** — một nhóm công việc logic: build, test, đóng gói, triển khai. Các chặng nối nhau thành chuỗi phụ thuộc.
- **Job** — đơn vị công việc được cấp phát tài nguyên để chạy. Các job độc lập có thể chạy song song.
- **Step (bước)** — một thao tác bên trong job: chạy một lệnh, hoặc gọi một khối logic đóng gói sẵn dùng lại được.
- **Agent / runner** — máy thực thi job.
- **Artifact** — sản phẩm mà pipeline tạo ra và chuyển tiếp: với TravelNow là container image, với dự án khác có thể là file JAR, npm package hay bundle tĩnh.

4.3.2 Mỗi job có môi trường thực thi riêng

Trong ví dụ dùng runner Ubuntu do GitHub cung cấp, **mỗi job chạy trên một môi trường mới**. Cách này giảm ảnh hưởng của file và tiến trình còn sót từ lần chạy trước. Với runner tự quản lý, tính chất ấy phải được thiết kế bằng cơ chế cô lập và dọn dẹp; nó không tự xuất hiện chỉ vì có CI. Mô hình runner này dẫn đến ba yêu cầu trong ví dụ:

1. **Không có gì tự động truyền từ job này sang job khác** — không thư mục làm việc, không file đã build, không thông tin xác thực. Job nào cần mã nguồn thì job đó phải tự lấy về.
2. **Muốn chuyển sản phẩm giữa các job hoặc giữa các môi trường thì phải đi qua một nơi lưu trữ chung.** Với TravelNow, nơi đó là ECR.

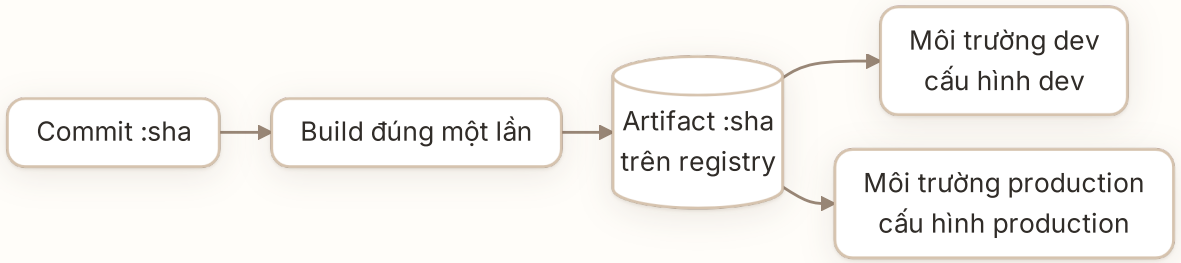
3. **Mỗi job cần gọi vào hạ tầng đều phải tự lấy quyền cho riêng nó** — và như mục 4.4 sẽ chỉ ra, ràng buộc nghe có vẻ phiền phức này hóa ra lại là một thuộc tính bảo mật tốt.

4.3.3 Artifact promotion: build một lần, triển khai qua nhiều môi trường

Đây là nguyên tắc quan trọng nhất của phần CD, và là hệ quả trực tiếp của "một image cho mọi môi trường" ở chương 3.

Artifact được build **đúng một lần**, ở chặng đầu tiên. Từ đó trở đi, nó không được build lại ở bất kỳ môi trường nào nữa — nó được chuyển tiếp để triển khai (*promote*) qua các môi trường: dev trước, production sau, vẫn là chính artifact đó. Cấu hình khác nhau giữa các môi trường được nạp vào lúc chạy, không được đóng gói cố định trong artifact.

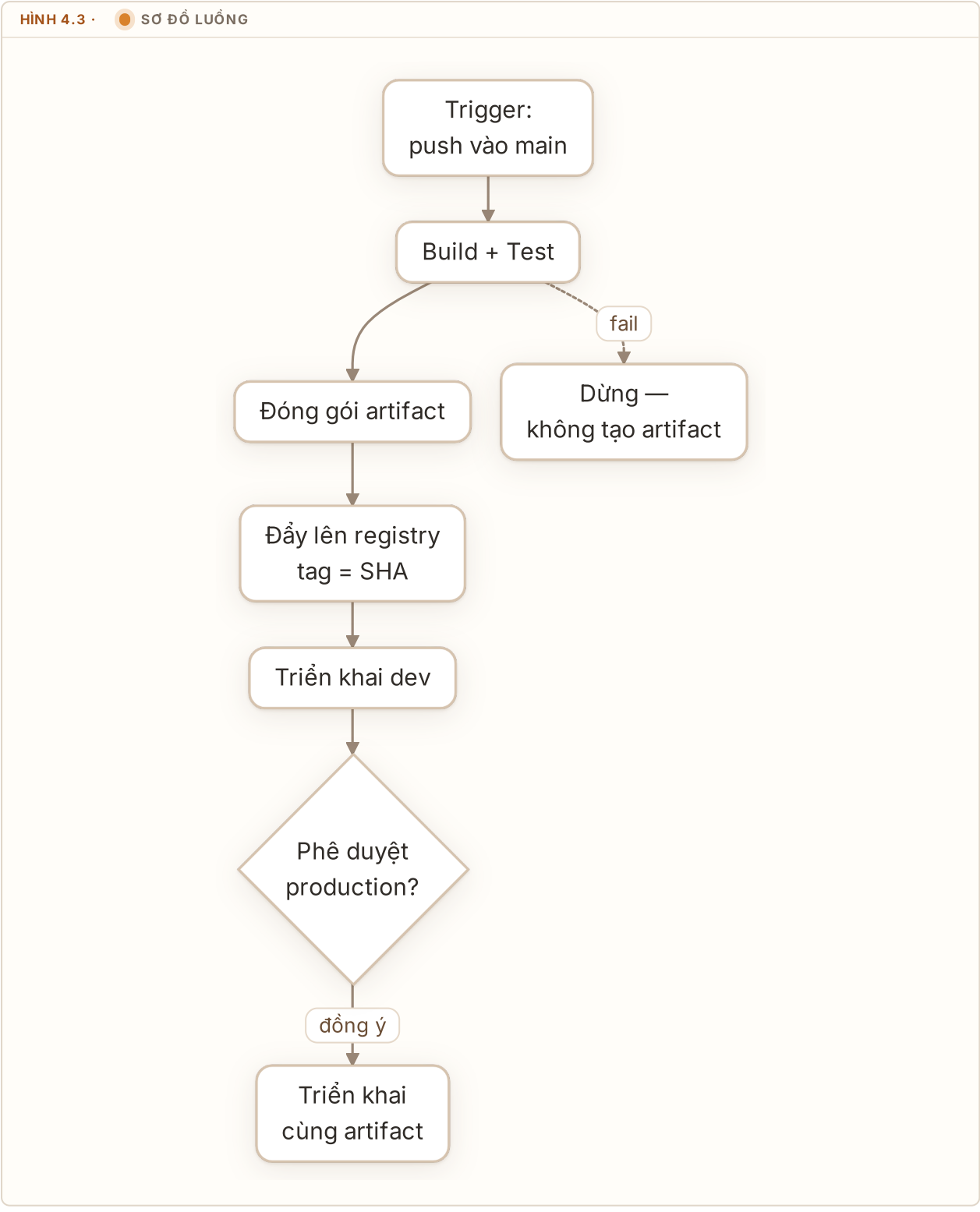
HÌNH 4.2 · SƠ ĐỒ LUỒNG



Nếu mỗi môi trường build lại từ mã nguồn, ta không còn bảo đảm artifact trên production là artifact đã được kiểm thử ở dev: phiên bản dependency hoặc môi trường build có thể khác nhau. Artifact promotion biến khẩu hiệu "thứ được kiểm thử là thứ được phát hành" từ một quy ước thành một ràng buộc cấu trúc: pipeline truyền cùng định danh image qua các chặng và kiểm tra định danh đó khi triển khai.

4.3.4 Thứ tự các chặng theo mức cam kết tăng dần

HÌNH 4.3 · SƠ ĐỒ LUỒNG



Thứ tự này không tùy tiện: những bước rẻ và có khả năng phát hiện lỗi cao đứng trước, những bước tốn kém và khó thu hồi đứng sau. Test chạy trước khi đóng gói, vì không có lý do gì để tốn công đóng gói một thứ đã hỏng. Artifact được đẩy lên registry trước khi bất kỳ môi trường nào chạy nó. Fail ở chặng nào thì dừng ở chặng đó — đó chính là cơ chế quality gate ở mục 4.1, được cài đặt bằng chuỗi phụ thuộc giữa các chặng.

4.3.5 Cùng khái niệm, khác tên gọi

Khái niệm	GitHub Actions	GitLab CI	Jenkins (Declarative Pipeline)	AWS CodePipeline
Tập tin mô tả	Workflow YAML	`.gitlab-ci.yml`	`Jenkinsfile`	Định nghĩa pipeline
Sự kiện kích hoạt	`on:`	`rules` / `workflow`	`triggers`	Source stage
Chặng	`needs:` giữa các job	`stages`	`stage`	`stage`
Đơn vị chạy	`job`	`job`	`steps` trong một `stage`	`action`
Máy thực thi	Runner	Runner	Agent / node	Do dịch vụ quản lý
Khối logic dùng lại	Action	`include` / template	Shared library	—
Chờ phê duyệt	Environment protection rule	`when: manual` + protected environment	Bước `input`	Manual approval action
Danh tính ngắn hạn	OIDC → IAM role	ID token → IAM role	Plugin / IRSA	Service role

Nhìn theo hàng ngang của bảng này thì việc chọn công cụ nào phần lớn là quyết định về hệ sinh thái và chi phí vận hành, không phải quyết định kiến trúc. Điều thực sự cần thiết kế nằm ở cột đầu tiên.

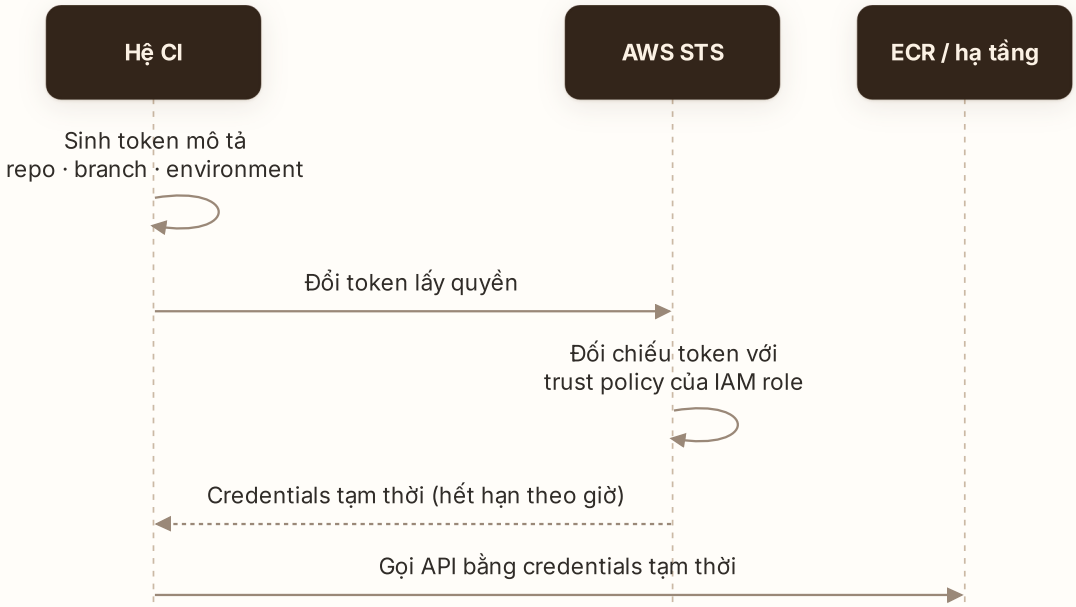
4.4 Danh tính của pipeline

Pipeline cần quyền gọi vào hạ tầng: đẩy image lên registry, ra lệnh triển khai. Nói cách khác, **pipeline là một chủ thể có quyền lực trong hệ thống** — có khả năng thay đổi production thông qua các quyền được cấp. Nó lấy danh tính ở đâu là một câu hỏi kiến trúc, không phải chi tiết cấu hình.

Cách làm trực tiếp nhất — tạo một tài khoản kỹ thuật, sinh cặp khóa truy cập, dán vào kho secret của CI — có một vấn đề cố hữu: **đó là một credential dài hạn được sao chép sang hệ thống CI.** Nó không tự hết hạn, tồn tại bản sao ở một hệ thống khác, và nếu bị lộ thì kẻ tấn công dùng được cho tới khi có người phát hiện và thu hồi thủ công.

Lời giải hiện đại là **danh tính liên kết với credential ngắn hạn**: hệ CI ký một token mô tả *bối cảnh của lần chạy này* — repository nào, nhánh nào, môi trường nào — rồi đổi token đó lấy quyền tạm thời từ nhà cung cấp hạ tầng. Trên AWS với GitHub Actions, cơ chế cụ thể là **OIDC** (OpenID Connect):

HÌNH 4.4 · SƠ ĐỒ TUẦN TỰ



Điểm mấu chốt nằm ở bước đối chiếu: **trust policy quy định chính xác token nào được chấp nhận.** Quyền không còn được cấp cho "ai giữ chuỗi ký tự này" mà cho "lần chạy diễn ra đúng trong bối cảnh này". Pipeline không cần giữ access key AWS dài hạn. Token và credential tạm thời vẫn phải được bảo vệ trong lúc job chạy. GitLab CI có cơ chế tương đương qua ID token, và các hệ chạy trong Kubernetes có thể dùng service account của chính cụm — khác nhau ở cài đặt, giống nhau ở nguyên tắc.

Nguyên tắc thứ hai là **mỗi chặng một danh tính riêng, với đúng quyền nó cần**:

Chặng	Quyền được cấp
Đóng gói và đẩy artifact	Xác thực registry, push image và đọc metadata/digest của repository cần phát hành
Triển khai dev	Chỉ tác động lên hạ tầng gắn nhãn dev
Triển khai production	Chỉ tác động lên hạ tầng production, và chỉ khi lần chạy đã qua cửa phê duyệt

Việc tách này không phải hình thức. Nếu dùng chung một danh tính có quyền chạm vào production ngay từ chặng build, thì cửa phê duyệt ở mục 4.5 chỉ còn là một nút bấm trên giao diện — vì mọi chặng trước đó đã có sẵn quyền làm điều mà nút bấm được sinh ra để kiểm soát. **Guardrail chỉ là ranh giới thật khi nó đồng thời là ranh giới quyền.** Đây cũng là lúc thấy rõ giá trị của đặc điểm "mỗi job một môi trường sạch" ở mục 4.3: credential không được truyền ngầm qua môi trường dùng chung. Tuy vậy, artifact và script từ chặng build vẫn đi tiếp, nên tính toàn vẹn của chúng cũng cần được kiểm soát.

Với GitHub Actions, trust policy cần kiểm tra `aud` và `sub` đúng định dạng mà repository đang dùng. Khi job gắn environment, subject tham chiếu environment; phải giới hạn nhánh được deploy trong quy tắc bảo vệ environment, thay vì giả định subject luôn chứa nhánh `main`. Chỉ

viết `environment: production` trong YAML chưa tạo ra bước phê duyệt. GitHub — OIDC trên AWS.

Cần nói rõ giới hạn: credential tạm thời vẫn dùng được trong thời gian hiệu lực của nó, nên trust policy hẹp, phạm vi quyền tối thiểu và môi trường thực thi đáng tin cậy vẫn là điều kiện bắt buộc. Bức tranh đầy đủ về IAM thuộc về chương 15.

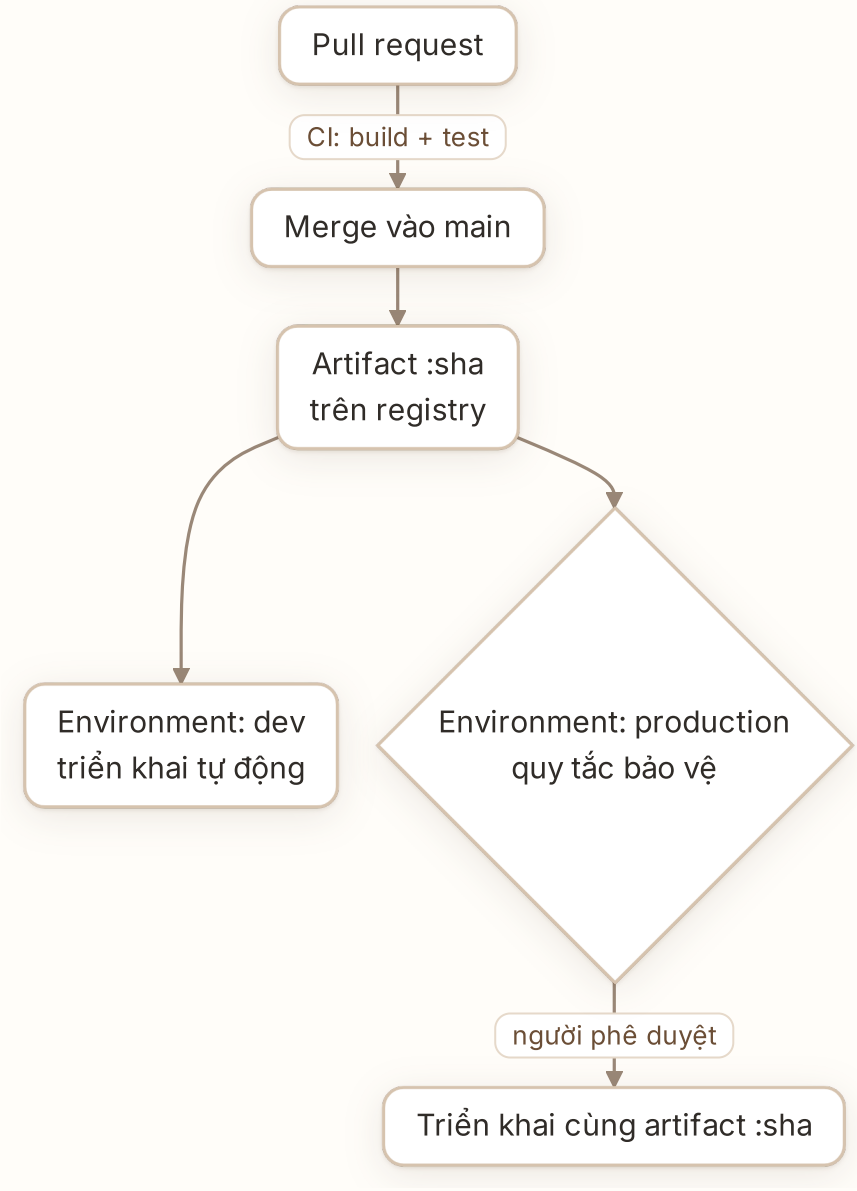
4.5 Quản lý nhánh, môi trường và phê duyệt triển khai

TravelNow dùng **trunk-based development**: các thay đổi được thực hiện trên nhánh tồn tại trong thời gian ngắn, gửi qua pull request và được CI kiểm tra trước khi merge vào `main`. Cách làm này giúp các thay đổi sớm được tích hợp vào nhánh chung, nhưng cần bộ kiểm thử đáng tin cậy và phạm vi thay đổi đủ nhỏ. Dự án phải duy trì nhiều dòng sản phẩm hoặc nhiều phiên bản hỗ trợ có thể cần chiến lược nhánh khác.

Sau khi được build và kiểm thử, cùng một artifact được triển khai lần lượt qua các **môi trường — environment**. Nhánh quản lý thay đổi mã nguồn; môi trường xác định nơi chạy ứng dụng và các điều kiện triển khai. Trong pipeline của TravelNow, mỗi môi trường có ba phần cần được quản lý:

- **Cấu hình riêng** — biến môi trường và secret được cung cấp cho ứng dụng khi chạy, giữ nguyên nội dung image.
- **Điều kiện triển khai** — các kiểm tra cần đạt và yêu cầu phê duyệt trước khi job được phép thực hiện.
- **Danh tính triển khai riêng** — job sử dụng danh tính có quyền phù hợp với môi trường đích, như Mục 4.4 đã mô tả.

HÌNH 4.5 · SƠ ĐỒ LUỒNG



Trong phương án Continuous Delivery này, production cần được phê duyệt trước khi triển khai. Người phê duyệt cần biết artifact nào sẽ được sử dụng, kết quả kiểm thử và những thay đổi liên quan đến cấu hình hoặc dữ liệu. Tùy yêu cầu vận hành, quy trình có thể bổ sung thời gian được phép triển khai, khoảng thời gian tạm ngừng thay đổi (freeze window) và điều kiện dựa trên trạng thái hệ thống. Phê duyệt cho phép bắt đầu triển khai; kết quả vẫn phải được kiểm tra tại môi trường đích.

4.6 Hiện thực hóa: pipeline TravelNow trên GitHub Actions

Phần này minh họa pipeline bằng cấu hình GitHub Actions cho hai chặng build/test và phát hành image, sau đó mô tả hai chặng triển khai bằng bảng. Cách đọc xuyên suốt là: **đầu vào nào được nhận, quyền nào được cấp, điều kiện nào cho phép đi tiếp**. Trong mô hình này, ECR là nơi lưu trữ image; IAM OIDC cấp quyền cho từng job; môi trường `dev` / `production` giữ cấu hình và quy tắc phê duyệt.

```yaml
name: Build & Deploy

on:                              # trigger
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  AWS_REGION: ap-southeast-1
  ECR_REPO: 123456789012.dkr.ecr.ap-southeast-1.amazonaws.com/travelnow

permissions:
  contents: read

concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: false       # không để hai lần triển khai cùng sửa một môi trường

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 21
          cache: maven          # cache ~/.m2 giữa các lần chạy

      - name: Build & run tests
        run: mvn verify -q      # quality gate: fail = dừng toàn bộ

  build-push-image:
    needs: build-test           # chuỗi phụ thuộc giữa các chặng
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    outputs:
      digest: ${{ steps.publish.outputs.digest }}
    permissions:
      id-token: write           # cho phép job yêu cầu OIDC token
      contents: read
    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
```

```
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-actions-ecr-publisher
          aws-region: ${{ env.AWS_REGION }}

      - name: Login to ECR
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build & push image
        id: publish
        env:
          IMAGE_TAG: ${{ github.sha }}
        run: |
          docker build -t "$ECR_REPO:$IMAGE_TAG" .
          docker push "$ECR_REPO:$IMAGE_TAG"
          DIGEST=$(aws ecr describe-images \
            --repository-name travelnow \
            --image-ids imageTag="$IMAGE_TAG" \
            --query 'imageDetails[0].imageDigest' --output text)
          echo "digest=$DIGEST" >> "$GITHUB_OUTPUT"
```

Hai job triển khai nối tiếp phần cấu hình trên như sau:

Job	Điều kiện bắt đầu và quyền	Công việc và điều kiện hoàn tất
`deploy-dev`	`needs: build-push-image`; môi trường `dev`; OIDC nhận role triển khai dev	Nhận digest từ job build, cập nhật EC2 dev bằng đúng image đó, kiểm tra health endpoint và một luồng đặt chuyến; chỉ thành công khi mọi kiểm tra đạt.
`deploy-prod`	`needs: [build-push-image, deploy-dev]`; môi trường `production` có phê duyệt; OIDC nhận role triển khai production	Nhận lại chính digest đã qua dev, thực hiện cùng quy trình trên EC2 production và kiểm tra kết quả.

Trong cả hai job, thao tác triển khai có cùng một luồng:

1. Nhận môi trường đích và địa chỉ image dạng `repository@digest`.
2. Xác định các EC2 thuộc môi trường đó; không tìm thấy máy đích thì dừng và báo lỗi.
3. Gửi lệnh qua Systems Manager (SSM). Trên từng máy, xác thực với ECR, pull image và thay container theo cấu hình của mục 3.7, giữ restart policy, memory limit và graceful shutdown.
4. Chờ kết quả thực thi trên từng máy, sau đó kiểm tra health endpoint và luồng nghiệp vụ. Một máy thất bại hoặc một kiểm tra không đạt khiến job thất bại, chặn chặng tiếp theo.

EC2 cần SSM Agent, instance role cho SSM/ECR, đường mạng tới các API cần thiết và cấu hình môi trường của ứng dụng. Phản hồi thành công từ `send-command` mới chỉ xác nhận AWS đã nhận lệnh; kết quả thực thi và kiểm tra ứng dụng mới quyết định triển khai có đạt hay không. Trạng thái lệnh SSM.

Vài điểm đáng dừng lại:

`mvn verify` **là quality gate của pipeline này.** Đây cũng là chỗ trả lời câu hỏi còn treo từ chương 3: `-DskipTests` trong Dockerfile không có nghĩa là bỏ test, mà là chuyển test sang đúng nơi có quyền chặn phát hành. Cần nhìn nhận đúng phạm vi bảo vệ của nó — một bộ test chỉ bảo đảm những gì nó thực sự kiểm tra. Quét lỗ hổng dependency, code review, contract test giữa các service (chương 9) và kiểm tra sau khi triển khai đều là những cổng riêng biệt, không thay thế được cho nhau.

`github.sha` **liên kết image với commit; digest xác định nội dung image.** Cùng một commit vẫn có thể build ra các image khác nhau nếu đầu vào build thay đổi. Vì vậy repository ECR của ví dụ phải bật tag immutability, không cho ghi đè tag đã phát hành; hai job deploy dùng cùng digest được chặng build xuất ra. Khi chạy lại pipeline cho commit đã có image, quy trình phát hành cần tái sử dụng artifact đã lưu hoặc cấp định danh build mới. Tài liệu ECR về tag immutability.

`if: github.event_name == 'push'` **phân biệt hai loại trigger.** Khi sự kiện là pull request, pipeline chỉ chạy build và test rồi dừng: pull request được kiểm tra nhưng không được phép tạo ra artifact hay chạm vào bất kỳ môi trường nào.

Job `deploy-prod` **không build lại bất cứ thứ gì** — đúng nguyên tắc artifact promotion ở mục 4.3. Nó triển khai chính image mà môi trường dev đã chạy.

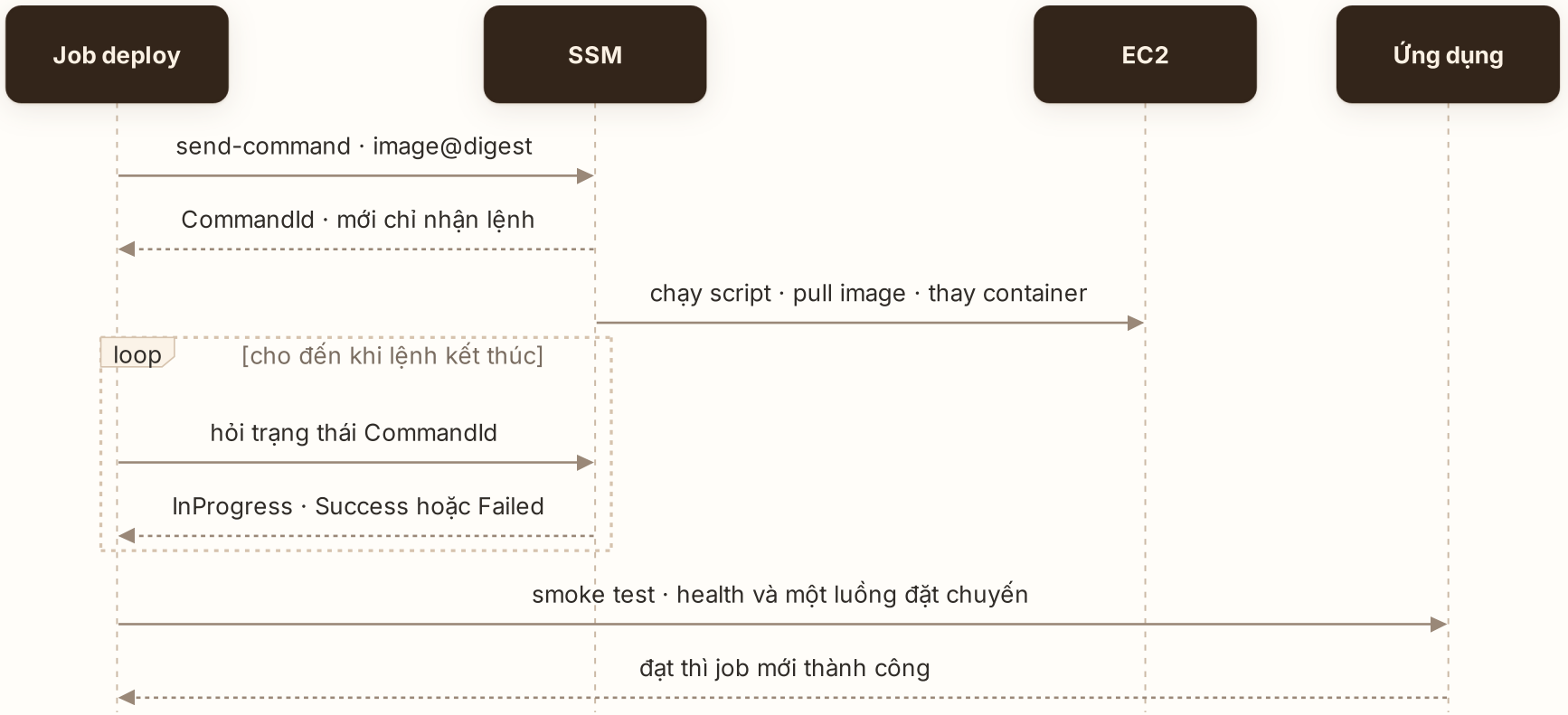
`concurrency` **ngăn hai lần triển khai cùng sửa một môi trường.** Hai commit merge sát nhau sẽ tạo hai lần chạy; với `cancel-in-progress: false`, lần đang chạy được hoàn tất và lần tiếp theo phải chờ. Cần hiểu đúng giới hạn: khi đang có một lần chạy chờ mà một lần mới hơn xuất hiện, GitHub thay thế lần đang chờ chứ không xếp hàng cả hai. Cơ chế này bảo đảm *một môi trường chỉ bị một lần chạy thay đổi tại một thời điểm*, chứ không bảo đảm mọi commit đều lần lượt được phát hành. Nếu nghiệp vụ yêu cầu điều thứ hai, cần một cơ chế hàng đợi phát hành riêng.

Hợp đồng của script SSM. `aws ssm send-command` là lệnh **bất đồng bộ**: nó trả về một `CommandId` ngay lập tức, không chờ lệnh chạy xong trên máy đích. Một pipeline production phải lấy `CommandId` đó, chờ trạng thái hoàn tất, rồi chạy smoke test. Bỏ qua các bước này sẽ dẫn tới nhầm lẫn nguy hiểm nhất trong tự động hóa triển khai: coi *"gửi được lệnh"* là *"triển khai thành công"*.

Xem Hình 4.6 (sơ đồ khổ ngang) ở trang 69.

Về việc cố định phiên bản của khối logic dùng lại. Ví dụ dùng major tag như `actions/checkout@v4` cho dễ đọc. Trong pipeline production, các khối logic của bên thứ ba nên được pin bằng commit SHA đầy đủ kèm một công cụ cập nhật tự động — vì một major tag có thể được trỏ lại sang code mới, và code đó chạy trên môi trường có quyền truy cập vào secret của bạn. Đây là cùng một lập luận với "không deploy bằng tag `latest`" ở chương 3, lần này áp dụng cho chuỗi cung ứng của chính pipeline.

HÌNH 4.6 · SƠ ĐỒ TUẦN TỰ



4.7 Triển khai phiên bản, phát hành tính năng và khôi phục khi có lỗi

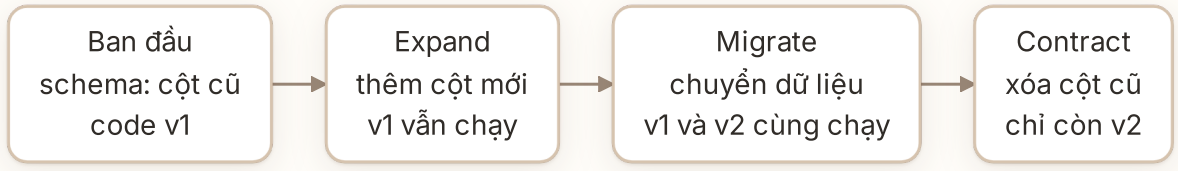
Triển khai phiên bản (deploy) và phát hành tính năng (release) là hai quyết định riêng. Deploy đưa phiên bản mới vào môi trường chạy; release cho phép người dùng sử dụng tính năng mới. Với những thay đổi cần theo dõi thận trọng, TravelNow có thể triển khai code nhưng chưa bật tính năng bằng **feature flag**, sau đó mở cho một nhóm nhỏ, theo dõi metric và log rồi mới mở rộng phạm vi sử dụng.

Feature flag cho phép tạm tắt tính năng bằng cấu hình và giúp tích hợp code sớm vào nhánh chung. Tuy nhiên, tắt flag không hoàn tác dữ liệu hoặc những thao tác đã thực hiện. Mỗi flag cần có người phụ trách, kế hoạch kiểm thử các trạng thái bật/tắt và thời điểm dự kiến loại bỏ để tránh tích lũy những nhánh điều kiện khó bảo trì.

Khôi phục phiên bản ứng dụng cần xét cả tính tương thích của dữ liệu. Triển khai lại image cũ không tự hoàn tác các thay đổi schema hoặc dữ liệu do phiên bản mới tạo ra. Với database thử nghiệm có thể xóa và tạo lại, Hibernate có thể tạo schema từ entity. RDS lưu dữ liệu nghiệp vụ của TravelNow cần được cập nhật bằng migration có kiểm soát, chẳng hạn với Flyway hoặc Liquibase, theo chiến lược **expand** → **migrate** → **contract**:

1. **Expand** — thêm cấu trúc mới (cột, bảng) theo cách mà code phiên bản cũ vẫn chạy bình thường.
2. **Migrate** — chuyển dữ liệu và chuyển traffic sang dùng cấu trúc mới.
3. **Contract** — chỉ xóa cấu trúc cũ sau khi không còn phiên bản đang chạy hoặc phiên bản cần giữ để rollback phụ thuộc vào nó.

HÌNH 4.7 · SƠ ĐỒ LUỒNG



Trong giai đoạn chuyển đổi, schema và dữ liệu cần tương thích với các phiên bản còn được hỗ trợ. Bước contract kết thúc khả năng dùng cấu trúc cũ, nên chỉ thực hiện sau khi đã xác định rõ phạm vi rollback. Yêu cầu tương thích sẽ xuất hiện lại ở Chương 9, khi schema của event thay đổi trong lúc các consumer được triển khai ở những thời điểm khác nhau.

Triển khai lại phiên bản ổn định (rollback). Quy trình cần trả lời được câu hỏi: nếu phiên bản mới gặp lỗi, mất bao lâu để đưa phiên bản ổn định trước đó trở lại phục vụ? Với image đã lưu trong ECR, chúng ta có thể triển khai lại bằng digest đã được kiểm chứng mà không cần build lại. Cấu hình tương ứng cũng cần được lưu phiên bản và kiểm tra với dữ liệu hiện tại.

Rollback chỉ phù hợp khi phiên bản cũ vẫn xử lý được schema, dữ liệu và message hiện có. Những thao tác đã thực hiện tại đối tác cũng cần được xác minh riêng. Nếu các điều kiện tương thích không còn đáp ứng, chúng ta có thể cần **triển khai bản sửa mới (roll forward)**, kèm migration sửa dữ liệu hoặc đối soát nghiệp vụ. Kế hoạch phục hồi phải xác định cả phiên bản sẽ chạy và cách xử lý những ảnh hưởng đã xảy ra.

Sau khi triển khai phiên bản mới, rollback hoặc đưa bản sửa lên, pipeline đều cần **kiểm tra khả năng phục vụ tại môi trường đích**. Trong phạm vi chương này, các kiểm tra gồm health endpoint và một luồng nghiệp vụ cơ bản. Phần IV bổ sung việc theo dõi tiến độ rollout, còn Phần V mở rộng sang cảnh báo và kết quả booking. Mỗi kiểm tra chỉ xác nhận phạm vi được đo; lệnh triển khai hoàn tất vẫn cần được bổ sung bằng kết quả kiểm thử ứng dụng.

4.8 Tóm tắt

Pipeline tổ chức việc phát hành thành một quy trình có thể kiểm tra và truy vết. Với TravelNow, image được build một lần, kiểm thử rồi triển khai qua các môi trường bằng cùng một định danh. Bước triển khai lên production cần được phê duyệt và sử dụng danh tính riêng với quyền truy cập phù hợp.

Kết quả thành công của pipeline cần được xác nhận bằng khả năng phục vụ của ứng dụng tại môi trường đích. Vì vậy, sau khi triển khai, chúng ta phải kiểm tra phiên bản mới đã hoạt động đúng hay chưa. Quay lại image cũ không tự khôi phục dữ liệu đã thay đổi; đưa code lên production cũng chưa nhất thiết đồng nghĩa với việc mở tính năng cho người dùng.

Ứng dụng đã có quy trình kiểm soát thay đổi; hạ tầng cũng cần được quản lý nhất quán và có thể truy vết. Chương 5 sẽ mô tả mạng, máy chủ, database và quyền truy cập bằng mã cấu hình, để các thay đổi có thể được lưu phiên bản, review và áp dụng theo quy trình.

CHƯƠNG 5

Infrastructure as Code — Khi hạ tầng được triển khai bằng code

Phần II — Container hóa và Tự động hóa · *Containerization & Automation*

Pipeline ở chương 4 có thể đưa đúng image lên EC2 instance, nhưng vẫn phụ thuộc vào hạ tầng đã được chuẩn bị. Nếu cần một môi trường mới, làm sao để dựng lại được network, database và quyền truy cập mà không bỏ sót một thiết lập nào trên Console?

Infrastructure as Code (IaC) đưa những quyết định ấy vào mã nguồn để review, xem trước thay đổi và thực thi có kiểm soát. Chương này trình bày mô hình khai báo, sau đó dùng Terraform để mô tả hạ tầng TravelNow. Điều chúng ta cần theo dõi là quan hệ giữa cấu hình mong muốn, tài nguyên thực tế và state của công cụ — cùng giới hạn quan trọng: dựng lại hạ tầng không có nghĩa là khôi phục được dữ liệu.

5.1 Những hạn chế khi quản lý hạ tầng thủ công

Ở chương 2, chúng ta tạo hạ tầng TravelNow qua từng thao tác: mở AWS Console, tạo VPC, click qua wizard tạo subnet, mở Security Group, điền form tạo RDS. Cách làm này, dân vận hành gọi đùa là **ClickOps** — hoàn toàn đúng đắn *ở thời điểm học*, vì nó cho ta nhìn thấy từng thành phần và từng lựa chọn. Vấn đề nằm ở việc *vận hành lâu dài* bằng Console, và nó gồm bốn khoản:

- **Khó tái tạo nhất quán.** Dựng thêm một môi trường staging nghĩa là click lại từ đầu theo trí nhớ. Và thứ bị sót thường không phải cái dễ thấy, mà là một tùy chọn nhỏ trong wizard — đúng loại sai lệch gây ra bug khó tìm nhất.
- **Trôi dạt (drift).** Ai đó mở cổng 5432 "tạm thời để debug" rồi quên đóng. Hạ tầng thật lệch dần khỏi hiểu biết chung của team, và không ai biết nó đã sai lệch từ lúc nào.
- **Thiếu lịch sử về ý định.** CloudTrail có thể ghi lại việc ai gọi API nào và lúc nào, nhưng thường không giải thích vì sao cần thay đổi. Commit message và pull request bổ sung phần ngữ cảnh ấy.
- **Không review được.** Một thao tác có thể mở database ra Internet được thực hiện bởi một người, một mình, không ai nhìn thấy trước khi nó có hiệu lực.

Đây chính là chẩn đoán của chương 3 về các bước deploy thủ công — "tái dựng môi trường bằng trí nhớ" — chỉ khác tầng áp dụng. Chương 3 và 4 quản lý thay đổi của ứng dụng bằng file và pipeline; chương này áp dụng cách làm đó cho cấu hình hạ tầng.

5.2 Tư duy khai báo: mô tả đích đến thay vì các bước

Trước khi chọn công cụ, cần nắm tư duy khai báo mà Terraform và CloudFormation sử dụng, vì nó khác hẳn cách ta viết script.

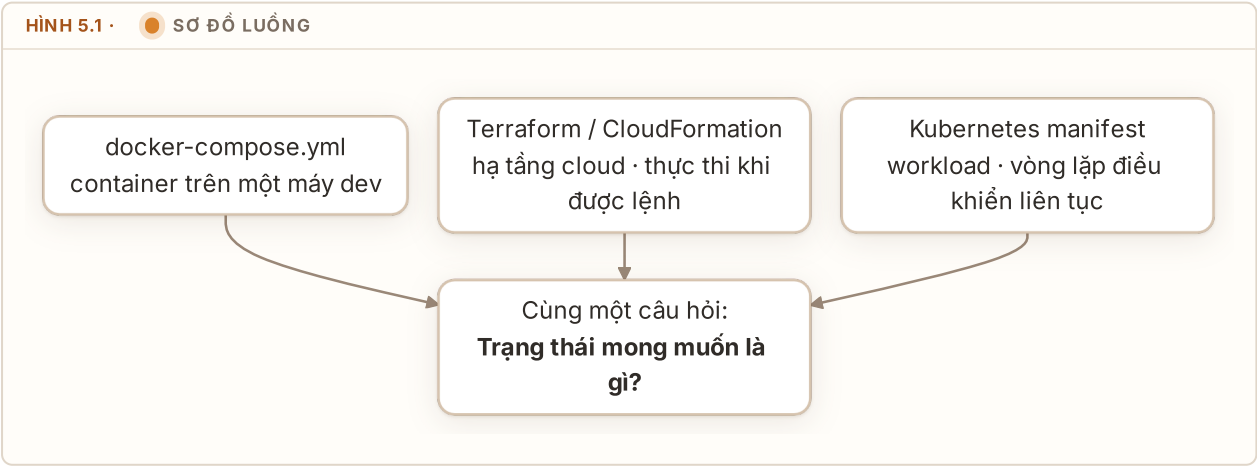
Một script deploy hoạt động theo lối **mệnh lệnh (imperative)**: nó mô tả *các bước* — tạo VPC, rồi tạo subnet, rồi tạo Security Group. Nếu script chỉ gọi API tạo tài nguyên mà không kiểm tra hiện trạng, chạy lần hai có thể tạo thêm tài nguyên hoặc gặp lỗi trùng tên. Muốn đổi instance type thì phải viết thêm một script sửa. Script mô tả một *hành trình*, nên nó chỉ đúng khi hệ thống xuất phát từ đúng điểm mà người viết hình dung.

Hai công cụ IaC dùng trong chương này, Terraform và CloudFormation, chủ yếu hoạt động theo lối **khai báo (declarative)**: nó mô tả *đích đến* — "hệ thống của tôi gồm một VPC như thế này, hai subnet như thế này, một database như thế này". Công cụ tự so sánh mô tả đó với hiện trạng rồi quyết định phải tạo, sửa hay xóa những gì. Chạy lại lần thứ hai với cùng đầu vào thì không có gì xảy ra, vì hiện trạng đã khớp với mô tả.

Tính chất vừa nêu chính là **idempotency** — thực hiện một lần hay nhiều lần đều cho cùng kết quả — mà chương 7 sẽ gặp lại ở tầng nghiệp vụ. Cần lưu ý đây là đặc tính của *mô hình*, không phải một bảo đảm tuyệt đối: provider và các API bên ngoài vẫn có thể chứa thao tác không idempotent, nên "declarative" giúp hệ thống hội tụ về trạng thái mong muốn chứ không miễn nhiễm với mọi sai lệch.

Tư duy này không mới với chúng ta. `docker-compose.yml` ở chương 3 đã là một bản khai báo — "tôi muốn hai container nối với nhau như thế này" — và `docker compose up` lo phần còn lại. Kubernetes manifest ở Phần IV cũng sẽ là khai báo, thậm chí đi xa hơn: một vòng lặp điều khiển *liên tục* so sánh và tự sửa sai lệch. IaC là mắt xích giữa của cùng một triết lý:

HÌNH 5.1 · SƠ ĐỒ LUỒNG



Nắm được trục này thì Phần IV sẽ không phải một thế giới mới, mà là một ứng dụng tiếp theo của tư duy đã bắt đầu từ chương 3.

5.3 Những khái niệm của IaC theo mô hình khai báo

Dưới đây là bộ khái niệm mà công cụ nào cũng phải có, vì chúng là hệ quả tất yếu của tư duy khai báo.

Resource và trạng thái mong muốn. Đơn vị khai báo là một *resource* — một VPC, một subnet, một database instance. Tập hợp mọi khai báo tạo thành **trạng thái mong muốn** (desired state), và nhiệm vụ của công cụ là đưa hiện trạng về khớp với nó.

Đồ thị phụ thuộc. Vì ta mô tả đích đến chứ không mô tả các bước, công cụ phải tự suy ra thứ tự thực hiện. Nó làm điều này bằng cách đọc các **tham chiếu** giữa các resource: nếu subnet tham chiếu tới ID của VPC, thì VPC phải được tạo trước. Thứ tự dòng trong file không có ý nghĩa; những resource không phụ thuộc nhau còn có thể được tạo song song.

Ánh xạ giữa khai báo và thực thể thật. Đây là mảnh ghép dễ bị bỏ qua nhất. Công cụ phải nhớ được rằng khối khai báo tên `main` trong file của bạn tương ứng với thực thể `vpc-0a1b2c3d` trên AWS — nếu không, nó không phân biệt được "sửa cái đang có" với "tạo thêm cái mới". Mọi công cụ IaC đều phải giữ ánh xạ này ở đâu đó; chúng chỉ khác nhau ở *chỗ giữ*, và như mục 5.7 sẽ cho thấy, lựa chọn đó kéo theo hệ quả vận hành rất khác nhau.

Xem trước thay đổi. Trước khi thực hiện, công cụ so sánh ba nguồn thông tin — khai báo, ánh xạ, và hiện trạng đọc về từ API — rồi in ra danh sách việc nó *định* làm. Đây là điểm khác biệt lớn nhất so với thao tác trên giao diện, nơi ta bấm nút rồi mới biết chuyện gì xảy ra.

Độ lệch (drift). Khi ai đó sửa trực tiếp trên Console, hiện trạng lệch khỏi khai báo. Công cụ phát hiện được điều này, nhưng chỉ với những thuộc tính mà nó quản lý và đọc được.

Đơn vị tái sử dụng. Một nhóm resource đi liền nhau, có tham số đầu vào, gọi được nhiều lần — về bản chất là một hàm trong thế giới hạ tầng. Đây là thứ biến "dựng thêm một môi trường" từ công việc chép tay thành một lời gọi.

Các công cụ thể hiện những khái niệm trên theo những cách khác nhau:

Khái niệm	Terraform	CloudFormation / AWS CDK	Pulumi
Khai báo một thực thể	`resource`	Resource trong template / construct sinh resource	Resource trong chương trình
Toàn bộ khai báo	Configuration (HCL)	Template viết trực tiếp hoặc tổng hợp từ CDK	Chương trình mô tả hạ tầng
Ánh xạ khai báo ↔ thực thể thật	State tại backend đã chọn	Stack do CloudFormation quản lý	State tại backend đã chọn
Xem trước thay đổi	`terraform plan`	Change set / `cdk diff`	`pulumi preview`

Khái niệm	Terraform	CloudFormation / AWS CDK	Pulumi
Đơn vị tái sử dụng	Module	Nested stack / construct	Component
Phát hiện độ lệch	Refresh trong quá trình `plan`	Drift detection của CloudFormation	`pulumi refresh`, hoặc preview có refresh

5.4 Bức tranh công cụ trên AWS và lựa chọn của eBook

CloudFormation — công cụ gốc của AWS. Bạn viết template YAML/JSON mô tả resource, nộp cho AWS, AWS thực thi và giữ trạng thái dưới dạng *stack*. Ưu điểm: tích hợp sâu với hệ sinh thái AWS, AWS quản lý trạng thái stack, còn bạn vẫn phải kiểm soát quyền đọc template và cấu hình, có sẵn drift detection. Nhược điểm: template thuần thường dài dòng, cơ chế tái sử dụng kém linh hoạt hơn, và không dùng được cho nhà cung cấp khác. Đáng chú ý là nhiều công cụ AWS khác — SAM, CDK, `eksctl` — sinh ra CloudFormation ở phía dưới, nên hiểu stack và change set vẫn có giá trị dù eBook không chọn nó làm cú pháp chính.

Terraform (và bản mã nguồn mở OpenTofu) — công cụ độc lập, làm việc với nhiều nhà cung cấp qua cơ chế provider. Cú pháp HCL mô tả resource và quan hệ phụ thuộc, module giúp tái sử dụng khai báo, và `terraform plan` cho ta bản "diff của hạ tầng" trước khi bất cứ thứ gì thay đổi. Đổi lại, bạn phải tự quản lý state file (mục 5.7), và công cụ chỉ nhìn thấy những gì nó quản — resource tạo ngoài luồng là vô hình với nó.

AWS CDK và Pulumi cho phép mô tả hạ tầng bằng ngôn ngữ quen thuộc như TypeScript hoặc Python. AWS CDK tổng hợp chương trình thành template CloudFormation và triển khai qua stack; Pulumi dùng engine cùng backend trạng thái của mình. Cả hai thuận tiện khi xây thư viện hạ tầng dùng chung, nhưng cần giữ code đủ rõ để reviewer hiểu những tài nguyên thực tế sẽ được tạo.

eBook chọn **Terraform** vì HCL cho phép trình bày ví dụ gọn, và vì `terraform plan` làm cho khái niệm "xem trước thay đổi" hiện ra một cách trực quan. Cần nhắc lại điều đã nói ở đầu chương: kỹ năng chuyển được sang công cụ khác là đọc đồ thị phụ thuộc, hiểu ánh xạ trạng thái, đọc diff và xử lý drift; cú pháp cụ thể thì không.

5.5 Hiện thực hóa: hạ tầng chương 2 bằng Terraform

Cách ôn lại tốt nhất cho những khái niệm đã dựng bằng tay ở chương 2 là nhìn chúng hiện ra dưới dạng code. Đây là trích đoạn hạ tầng TravelNow — VPC, Security Group, RDS — viết bằng HCL:

```
# ─── network.tf (trích đoạn) ──────────────────────────────────────
data "aws_availability_zones" "available" {
  state = "available"
```

```
}

resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags       = { Name = "travelnow-vpc" }
}

resource "aws_subnet" "public" {
  count                   = 2
  vpc_id                  = aws_vpc.main.id
  availability_zone       = data.aws_availability_zones.available.names[count.index]
  cidr_block              = cidrsubnet(aws_vpc.main.cidr_block, 8, count.index)
  map_public_ip_on_launch = true
}

resource "aws_subnet" "private" {
  count             = 2
  vpc_id            = aws_vpc.main.id
  availability_zone = data.aws_availability_zones.available.names[count.index]
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 8, count.index + 10)
}

# ─── security.tf ──────────────────────────────────────────────────────────
resource "aws_security_group" "app" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = [var.demo_client_cidr] # CIDR máy thử nghiệm, ví dụ một địa chỉ /32
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"] # ví dụ; production thu hẹp theo đường mạng cần thiết
  }
}

resource "aws_security_group" "db" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port       = 5432
    to_port         = 5432
    protocol        = "tcp"
    security_groups = [aws_security_group.app.id] # CHỈ SG-app được vào :5432
  }
}

# ─── database.tf ──────────────────────────────────────────────────────────
resource "aws_db_subnet_group" "main" {
  subnet_ids = aws_subnet.private[*].id # ít nhất hai AZ cho Multi-AZ
}

resource "aws_db_instance" "main" {
  engine                = "postgres"
  engine_version        = "16"
```

```
  instance_class         = "db.t4g.micro"
  allocated_storage      = 20
  db_name                = "travelnow"
  multi_az               = var.db_multi_az
  db_subnet_group_name   = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.db.id]
  publicly_accessible    = false
  storage_encrypted      = true
  username               = "travelnow"
  password               = var.db_password # state vẫn chứa secret — mục 5.7
  backup_retention_period = 7 # minh họa; chọn theo nhu cầu khôi phục
  deletion_protection    = true
  skip_final_snapshot    = false
  final_snapshot_identifier = "travelnow-${var.environment}-final"

  lifecycle {
    prevent_destroy = true
  }
}
```

Hãy đọc chậm và đối chiếu với chương 2. `aws_db_subnet_group` đặt RDS vào hai private subnet thuộc hai AZ khác nhau — đúng điều kiện để bật Multi-AZ. `vpc_security_group_ids` gắn đúng Security Group chỉ nhận kết nối từ nhóm app. `publicly_accessible = false` là phiên bản khai báo của lựa chọn không cấp địa chỉ public cho database. Toàn bộ topology mà chương 2 mô tả bằng lời giờ nằm gọn trong vài chục dòng đọc được và review được.

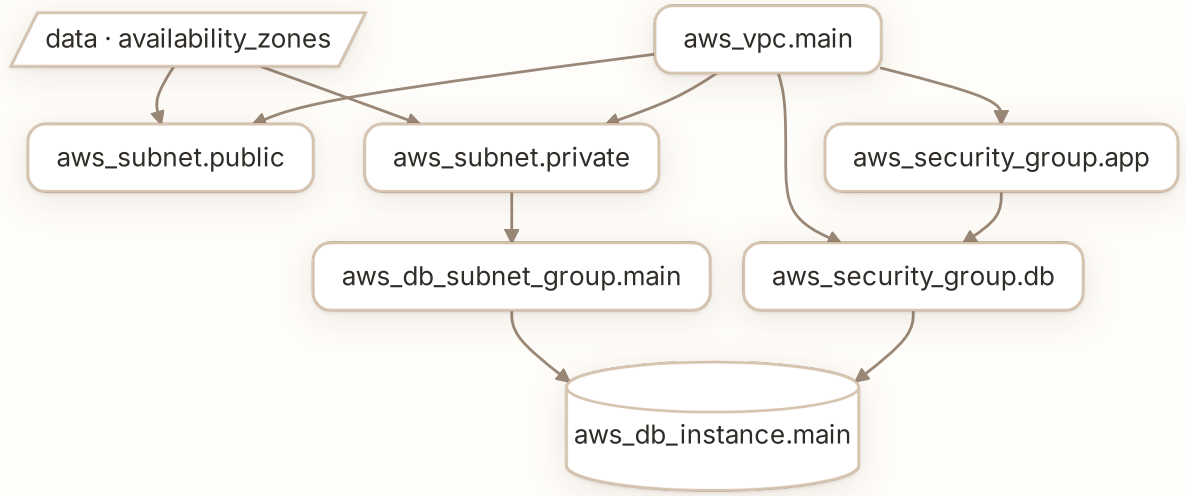
Trích đoạn này đã lược bớt Internet Gateway, route table, NAT hoặc VPC endpoint, EC2, ALB và cấu hình backup để giữ trọng tâm. Một module dùng cho production phải khai báo đầy đủ những resource đó — đặc biệt là route table, vì một subnet không trở thành "public" chỉ nhờ bật cấp IP public, mà nhờ có đường đi ra Internet Gateway.

Ba khái niệm của mục 5.3 xuất hiện ngay trong đoạn code trên:

- **Đồ thị phụ thuộc**: `aws_subnet.public` tham chiếu `aws_vpc.main.id`, nên Terraform tự biết phải tạo VPC trước. Không có bước nào được đánh số, và cũng không cần.
- **Data source**: khối `data "aws_availability_zones"` không tạo ra gì cả — nó *đọc* thông tin có sẵn từ AWS để dùng làm đầu vào. Đây là cách IaC tham chiếu tới những thứ nó không quản lý.
- **Variable**: những giá trị thay đổi theo môi trường (mật khẩu, instance type, dải CIDR) được đẩy ra biến — chính là bài "tách cấu hình khỏi artifact" của chương 3, lần này ở tầng hạ tầng.

Đồ thị phụ thuộc mà Terraform suy ra từ các tham chiếu trong đoạn code trên:

HÌNH 5.2 · SƠ ĐỒ LUỒNG



Toàn bộ các file trên được gói lại thành một **module** — đơn vị tái sử dụng, hoạt động như một hàm có tham số:

```
module "travelnow_base" {
  source        = "./modules/travelnow-base"
  environment   = "dev"
  instance_type = "t3.small"
  db_multi_az   = false # dev không cần Multi-AZ — tiết kiệm chi phí
}
```

Đây là giao diện minh họa của module: bên trong, các tham số phải được nối với resource tương ứng, chẳng hạn `db_multi_az` đi vào `aws_db_instance.multi_az`. Các biến, provider và toàn bộ tài nguyên mạng chưa được trình bày đủ để apply độc lập. Module hoàn chỉnh cần pin phiên bản Terraform/provider, commit `.terraform.lock.hcl` và có input riêng cho từng môi trường.

5.6 `terraform plan` — xem trước khi thay đổi

Quy trình làm việc gồm ba lệnh, và lệnh ở giữa là quan trọng nhất:

```
terraform init      # tải provider, kết nối tới nơi lưu state
terraform plan      # SO SÁNH khai báo với hiện trạng → in ra diff
terraform apply     # thực thi (tự lập plan mới để duyệt, hoặc chạy plan đã lưu)
```

`terraform plan` trả lời câu hỏi: *với khai báo, ánh xạ và hiện trạng đọc được ở thời điểm này, công cụ định làm những gì?* Ví dụ, đổi `instance_class` của RDS rồi chạy plan:

```
Terraform will perform the following actions:
```

```
  # aws_db_instance.main will be updated in-place
  ~ resource "aws_db_instance" "main" {
      ~ instance_class = "db.t4g.micro" -> "db.t4g.small"
        # (các thuộc tính khác giữ nguyên)
    }

Plan: 0 to add, 1 to change, 0 to destroy.
```

Ba ký hiệu cần nhận biết: `+` tạo mới, `~` sửa tại chỗ, và `-` xóa. Ký hiệu thứ ba là lý do thực sự khiến `plan` tồn tại. Có những thuộc tính không sửa được tại chỗ, nên khi ta đổi chúng, cách duy nhất là **xóa resource cũ rồi tạo lại resource mới** — plan hiển thị điều này bằng dòng `-/+ destroy and then create replacement`. Với một EC2 chạy ứng dụng duy nhất, thay thế có thể gây gián đoạn; với một resource mang dữ liệu như RDS thì đó là rủi ro mất dữ liệu nếu thiếu backup và thiếu rào chắn. Một dòng như vậy trong plan phải được đọc kỹ trước khi apply.

Từ đó, hai quy tắc kỷ luật:

1. **Không apply thứ chưa plan** — và ở mức một team vận hành: plan phải được người thứ hai đọc. Mục 5.8 sẽ đưa plan vào pull request để điều này trở thành cấu trúc thay vì trông vào thiện chí.
2. **Đặt rào chắn cho những resource không được phép mất.** Với resource mang dữ liệu, khai báo `prevent_destroy` để công cụ từ chối thực hiện thay vì hỏi lại. Việc dựng rồi xóa toàn bộ môi trường là một phép thử tốt cho khả năng tái tạo, nhưng chỉ áp dụng cho môi trường thử nghiệm không chứa dữ liệu cần giữ.

`prevent_destroy` chỉ có hiệu lực khi cấu hình bảo vệ còn tồn tại; xóa cả resource block có thể gỡ luôn rào chắn này. Vì vậy dữ liệu quan trọng còn cần deletion protection của dịch vụ, phân quyền và backup đã thử restore. Giới hạn của `prevent_destroy`.

Cũng cần biết giới hạn của plan: nó là bản dự kiến tính trên hiện trạng *tại thời điểm chạy*. Nó không dự đoán được lỗi phát sinh lúc thực thi, và nếu hiện trạng thay đổi trong khoảng thời gian giữa plan và apply thì kết quả có thể khác. Đó là lý do plan để lâu nên được loại bỏ và làm lại thay vì cố apply.

Trong CloudFormation, **change set** phục vụ cùng mục đích: xem tác động dự kiến trước khi thực hiện cập nhật stack.

5.7 State và drift

Một câu hỏi cần làm rõ khi dùng Terraform là: *làm sao nó biết hiện trạng để mà so sánh?* Nó không quét toàn bộ AWS account mỗi lần chạy. Nó giữ một **state file** (`terraform.tfstate`) — chính là "ánh xạ giữa khai báo và thực thể thật" ở mục 5.3, dưới dạng một file: `aws_vpc.main` ↔ `vpc-0a1b2c3d`.

Đây là chỗ Terraform và CloudFormation khác nhau rõ nhất về mô hình trách nhiệm. CloudFormation giữ ánh xạ đó bên trong dịch vụ; Terraform giao nó cho bạn, kèm theo hai vấn đề vận hành phải giải quyết ngay.

Vấn đề thứ nhất: state không nên sống trên laptop của một người. State chỉ lưu trữ cục bộ khiến các thành viên khó cùng quản lý một tập tài nguyên; mất bản state duy nhất cũng làm mất ánh xạ đang dùng. Cấu hình thông dụng trên AWS là đặt state trên S3, bật versioning và bật khóa:

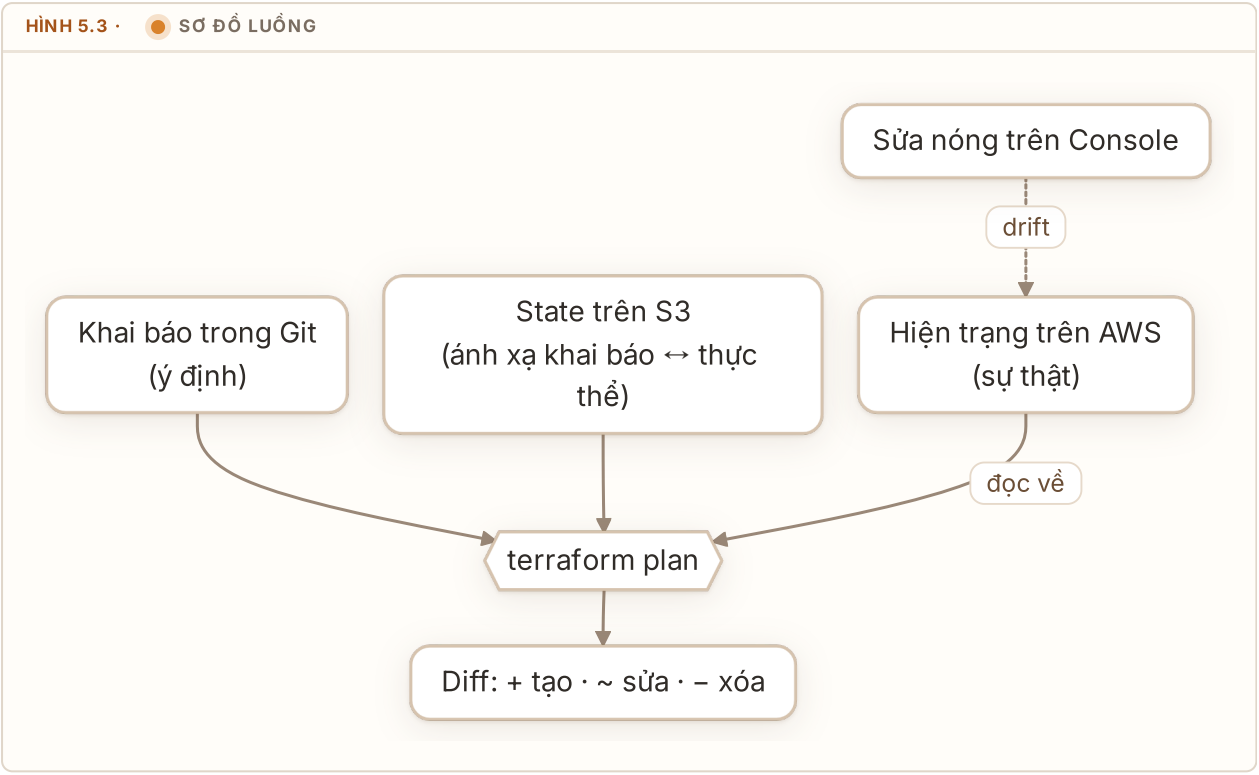
```
terraform {
  backend "s3" {
    bucket       = "travelnow-terraform-state"
    key          = "base/terraform.tfstate"
    region       = "ap-southeast-1"
    use_lockfile = true # chống hai tiến trình cùng apply
    encrypt      = true
  }
}
```

`use_lockfile` giải bài toán hai người (hoặc hai lần chạy pipeline) cùng apply một lúc: một tiến trình giữ khóa, tiến trình còn lại dừng hoặc chờ, thay vì cả hai cùng ghi đè state và làm hỏng ánh xạ. Versioning trên bucket là đường lùi khi có thao tác sai. Terraform vẫn hỗ trợ khóa bằng DynamoDB cho hệ thống cũ, nhưng cơ chế đó đã bị đánh dấu deprecated.

Vấn đề thứ hai: state chứa secret. Với cách truyền `password = var.db_password` trong ví dụ, giá trị mật khẩu được lưu trong state. Đánh dấu biến `sensitive` chỉ ẩn giá trị ở một số đầu ra, không tự loại bỏ nó khỏi state. Một số tính năng ephemeral/write-only có thể tránh lưu giá trị nếu phiên bản Terraform và provider hỗ trợ, nhưng ví dụ này chưa sử dụng chúng. Vì vậy quy tắc là: **đối xử với state như đối xử với secret.** Bucket riêng, bật mã hóa, chặn truy cập public, và chỉ cấp quyền cho pipeline cùng một số ít người vận hành.

Từ đây rút ra một mô hình tư duy đáng ghi nhớ: có **ba nguồn thông tin** cần phân biệt.

HÌNH 5.3 · SƠ ĐỒ LUỒNG



Khai báo ghi **ý định**, state giữ **ánh xạ**, còn API của nhà cung cấp mới là **sự thật**. Bình thường ba nguồn này khớp nhau. Khi ai đó sửa nóng (hotfix) trên Console, chúng có sự sai lệch — đó là **drift**. Chạy `plan` định kỳ là cách phát hiện ra sai lệch đó, và có hai hướng xử lý: hoặc đưa thay đổi hợp lệ đó vào khai báo, hoặc apply để kéo hiện trạng về đúng khai báo. Điều không nên làm là để một thay đổi khẩn cấp tồn tại ngoài code vô thời hạn — khi đó ý định thật của hệ thống không còn nằm ở nơi mọi người đang đọc.

5.8 Đưa hạ tầng vào pipeline

5.8.1 Ánh xạ trực tiếp từ chương 4

Hạ tầng đã là code, vậy nó đi đúng con đường mà code đã đi:

Sự kiện	Code ứng dụng — chương 4	Hạ tầng — chương 5
Mở pull request	Chạy build + test, đỏ thì chặn merge	Chạy `terraform plan`, đăng kết quả vào PR để reviewer đọc
Merge vào `main`	Đóng gói artifact và triển khai dev	`terraform apply` trên môi trường dev
Production	Chờ phê duyệt rồi triển khai	Chờ phê duyệt rồi apply

Điểm mạnh nhất của quy trình này là reviewer đọc được **cả hai lớp thông tin**: khai báo HCL cho biết ý định lâu dài, còn plan cho biết tác động cụ thể lên hiện trạng đang có. Một thay đổi trông vô hại trong HCL vẫn có thể sinh ra một dòng `destroy and then create replacement` trong plan

— và đó chính là dòng mà review cần bắt được.

Để bước apply thực hiện đúng thứ đã được duyệt, nên lưu plan lại và apply chính nó (`terraform plan -out=tfplan` → review → `terraform apply tfplan`), thay vì để công cụ tự tính một diff mới ở thời điểm apply. Lưu ý plan file có thể chứa dữ liệu nhạy cảm nên phải được bảo vệ như state.

Plan phải được tạo riêng cho từng môi trường từ commit đã xác định; không dùng plan của dev để apply production. Mỗi môi trường cũng cần state và phạm vi quyền riêng.

Danh tính của pipeline hạ tầng thì không có gì mới phải phát minh: **cùng cơ chế credential ngắn hạn của chương 4.** Chỉ có một khác biệt về mức độ, nhưng là khác biệt lớn — role của pipeline hạ tầng mạnh hơn nhiều, vì nó tạo và xóa được cả database. Vì vậy trust policy của nó càng phải hẹp: chỉ repository hạ tầng, chỉ nhánh main, và bước apply lên production đứng sau cửa phê duyệt như một điều hiển nhiên.

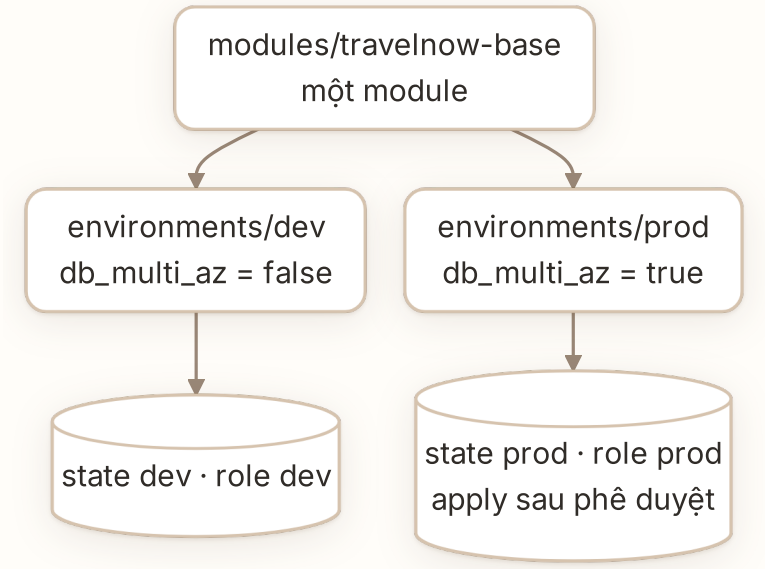
5.8.2 Một module, mọi môi trường

Chương 3 có nguyên tắc "một image, mọi môi trường — chỉ khác biến môi trường". Phiên bản hạ tầng của nó: **một module, mọi môi trường — chỉ khác tham số đầu vào.**

```
environments/
├── dev/
│   ├── main.tf          # module "travelnow_base" { source = "../../modules/..." }
│   └── dev.tfvars       # instance_type = "t3.small",  db_multi_az = false
└── prod/
    ├── main.tf          # CÙNG module đó
    └── prod.tfvars      # instance_type = "t3.large",  db_multi_az = true
```

Dev và prod không phải hai bộ hạ tầng được chép tay thành hai bản, mà là hai lần gọi của cùng một module với tham số khác nhau. Nhờ vậy, những khác biệt có chủ đích giữa các môi trường — dev không bật Multi-AZ để tiết kiệm — trở thành **khác biệt được khai báo**, đọc được và review được, thay vì khác biệt do lịch sử click để lại mà không ai nhớ.

HÌNH 5.4 · SƠ ĐỒ LUỒNG



Cần giữ kỳ vọng đúng mức: tái sử dụng module làm giảm phần công việc cơ học của việc dựng một môi trường mới, nhưng mỗi môi trường vẫn cần account và quyền riêng, dữ liệu riêng, hạn mức, chi phí và kiểm thử riêng. Nó cũng có mặt trái: một thay đổi sai trong module dùng chung sẽ lan tới mọi môi trường gọi nó — nên chính module cũng cần được phát hành theo phiên bản và triển khai dần, không sửa trực tiếp rồi apply đồng loạt.

5.9 Giới hạn của IaC

IaC không thay thế được hiểu biết về hạ tầng. Muốn review plan, ta phải hiểu tài nguyên nào đang thay đổi, đường mạng nào bị ảnh hưởng và dữ liệu nào cần giữ lại. Đó là lý do chương 2 giới thiệu các thành phần qua từng thao tác trước khi chương này chuyển chúng thành code.

IaC quản lý resource, không quản lý dữ liệu. Xóa và dựng lại một RDS bằng Terraform sẽ cho ra một database đúng cấu hình — và trống rỗng. Khả năng khôi phục dữ liệu đến từ backup, snapshot và quy trình restore đã được diễn tập, không đến từ file HCL. Một hệ thống "tái tạo được hoàn toàn bằng IaC" là mô tả về phần hạ tầng, không phải về phần dữ liệu.

Công cụ chỉ nhìn thấy những gì mà nó quản lý. Resource được tạo ngoài luồng là vô hình với plan; nó vẫn chạy, vẫn tính tiền, và vẫn có thể là lỗ hổng bảo mật mà không xuất hiện trong bất kỳ bản review nào. Vì vậy kỷ luật "mọi thay đổi đều đi qua code" quan trọng ngang với bản thân công cụ.

Có những bước cần khởi tạo và phối hợp riêng. Bản thân bucket chứa state phải tồn tại trước khi Terraform dùng được nó — vì vậy cần một cấu hình bootstrap có state riêng trước khi chuyển các cấu hình khác sang backend này. Tương tự là các thao tác một lần, các thay đổi cần thực hiện

theo trình tự thủ công, hay việc phối hợp giữa nhiều hệ thống bên ngoài. IaC bao phủ phần lớn công việc, không phải toàn bộ; phần còn lại cần được ghi thành runbook thay vì giả vờ rằng nó không tồn tại.

5.10 Tóm tắt Phần II

Phần II đã xây dựng cách quản lý nhất quán cho ba công việc: đóng gói ứng dụng, triển khai phiên bản và tạo lập hạ tầng.

Chương	Nội dung được chuẩn hóa	Kết quả cần kiểm chứng
3 — Docker	Cách đóng gói ứng dụng cùng môi trường thực thi	Image được xác định chính xác và chạy được trên nền tảng đích
4 — CI/CD	Quy trình kiểm thử và triển khai	Đúng image được đưa tới đúng môi trường và ứng dụng hoạt động như mong đợi
5 — IaC	Cấu hình và quy trình thay đổi hạ tầng	Tài nguyên được tạo hoặc cập nhật theo kế hoạch đã review

Những cơ chế này giúp giảm thao tác thủ công và hỗ trợ kiểm tra, truy vết mỗi thay đổi. Chúng ta vẫn cần duy trì module hạ tầng, pipeline và quyền truy cập tương ứng. IaC giúp tạo lại tài nguyên theo cấu hình đã lưu; dữ liệu nghiệp vụ cần được khôi phục bằng backup và quy trình restore đã kiểm chứng.

TravelNow có thể áp dụng những cải tiến này ngay khi vẫn là một ứng dụng monolith. Sang Phần III, chúng ta sẽ xem xét những nhu cầu dẫn đến việc tách service và các trách nhiệm phát sinh. Công cụ đóng gói, triển khai và quản lý hạ tầng vẫn có thể được sử dụng, nhưng khi các service chạy trong những tiến trình riêng và quản lý dữ liệu độc lập, cách phối hợp để hoàn tất nghiệp vụ cần được thiết kế lại.

PHẦN III

Kiến trúc phân tán

The Hard Parts

Khi các bước nghiệp vụ được thực hiện bằng những transaction riêng biệt, làm sao bảo đảm một chuyến đi vẫn được xử lý chính xác? Phần này bắt đầu từ việc xem xét lý do tách service, sau đó tìm hiểu cách xử lý một lời gọi chưa rõ kết quả và cách phối hợp, phục hồi toàn bộ quy trình đặt chuyến.

Trên sơ đồ microservices, một mũi tên thường biểu diễn lời gọi giữa hai service. Khi hệ thống hoạt động, lời gọi đó có thể phản hồi chậm, mất kết nối hoặc đã được xử lý nhưng bên gửi chưa nhận được kết quả. Một service cũng có thể dừng khi công việc còn dang dở. Qua quy trình đặt chuyến của TravelNow, chúng ta sẽ phân tích từng tình huống và những cơ chế cần có để tiếp tục xử lý an toàn.

Trong mỗi tình huống, hãy theo dõi ba câu hỏi: **trạng thái nào đã được lưu, thành phần nào quản lý trạng thái đó và ai xử lý tiếp khi có lỗi?** Timeout giới hạn thời gian chờ; idempotency giúp tránh lặp tác động nghiệp vụ khi thực hiện lại thao tác; Saga phối hợp các bước và hoạt động bù trừ. Trạng thái được lưu bền vững cùng cơ chế giao nhận message giúp hệ thống tiếp tục công việc sau gián đoạn.

Chương 9 triển khai hai phương án phối hợp Saga trên AWS: orchestration với Step Functions và choreography với SNS/SQS. Cả năm service sử dụng Lambda; dữ liệu được lưu trên một RDS instance với database riêng cho từng domain, sau khi cân nhắc các lựa chọn lưu trữ. Với mỗi dịch vụ, chúng ta sẽ xác định khả năng mà nền tảng cung cấp và những yêu cầu ứng dụng vẫn phải tự đáp ứng.

Các sơ đồ và tình huống tập trung giải thích cách các thành phần phối hợp và phân chia trách nhiệm; chi tiết thiết lập môi trường được trình bày ở mức cần thiết để theo dõi. Phần IV sẽ tiếp tục xem xét trách nhiệm vận hành và điều kiện lựa chọn môi trường thực thi phù hợp.

Chương	Tiêu đề
6	Khi Monolith không còn đủ
7	Distributed Transaction, Timeout và Idempotency
8	Saga Pattern — Choreography và Orchestration
9	Triển khai hệ thống phân tán trên AWS

CHƯƠNG 6

Khi Monolith không còn đủ

Phần III — Kiến trúc phân tán · *The Hard Parts*

Đến cuối Phần II, TravelNow vẫn là một ứng dụng monolith với các module có trách nhiệm rõ ràng, dữ liệu được lưu bên ngoài ứng dụng và quy trình phát hành được kiểm soát. Kiến trúc này có thể tiếp tục đáp ứng nhu cầu khi các chức năng vẫn phù hợp để được triển khai và vận hành cùng nhau.

Chương này xem xét một bối cảnh mới: các nhóm cần phát hành theo lịch khác nhau, tải tập trung ở một số chức năng và phạm vi phụ trách dần được phân chia rõ hơn. Chúng ta sẽ đánh giá những nhu cầu đó trước khi lựa chọn tách TravelNow thành năm service. Mục tiêu là tăng khả năng thay đổi, triển khai và vận hành độc lập. Đổi lại, khi mỗi service quản lý dữ liệu riêng, **các bước đặt chuyến không còn nằm trong một transaction chung**.

Chương 7 và Chương 8 sẽ phân tích cách xử lý lỗi và phối hợp nghiệp vụ khi các bước được thực hiện riêng biệt. Trước hết, chúng ta cần xác định phần nào nên tách và dùng kết quả nào để kiểm chứng việc tách service có giải quyết được vấn đề ban đầu hay không.

6.1 Những tín hiệu cho thấy monolith không còn đủ

Trước hết, có một thực tế là: **monolith không "hết hạn sử dụng" theo thời gian.** Nó không tự dưng trở nên tồi đi khi hệ thống chạy đủ lâu. Thứ thay đổi là *bối cảnh xung quanh nó* — lưu lượng, đội ngũ, tốc độ ra tính năng. Khi bối cảnh thay đổi, những tính chất từng là điểm mạnh bắt đầu trở thành điểm nghẽn. Hãy xét ba tình huống giả định trên TravelNow; các con số bên dưới dùng để minh họa, chưa phải kết quả đo của ứng dụng.

6.1.1 Tín hiệu 1 — Phạm vi phát hành quá rộng

Giả sử team payment cần sửa một dòng: đổi format của `paymentRef` để tương thích với cổng thanh toán mới. Trong monolith, "sửa một dòng" nghĩa là:

1. Build lại **toàn bộ** ứng dụng.
2. Chạy bộ kiểm thử của đơn vị phát hành chung theo pipeline hiện tại, kể cả các module không sửa trực tiếp.

3. Deploy lại **toàn bộ** hệ thống — mọi domain cùng đi qua một rollout và cùng chịu blast radius nếu rollout lỗi. Một quy trình rolling tốt có thể tránh downtime, nhưng không làm phạm vi thay đổi nhỏ lại.

Trong tình huống giả định này, build mất 20 phút, test mất 40 phút và các nhóm phải phối hợp lịch deploy. Pipeline vẫn tự động, nhưng mỗi lần phát hành phải xử lý cả đơn vị triển khai chung. Nếu điều đó khiến đội ngũ gom nhiều thay đổi vào một đợt, khi có lỗi sẽ khó xác định nguyên nhân hơn.

Trước khi coi đây là lý do tách thành các microservice độc lập, cần đo thời gian chờ thực tế và thử tối ưu build, test cùng quy trình phát hành. Monolith không mặc nhiên có pipeline chậm; vấn đề là phạm vi triển khai chung có còn phù hợp với cách các nhóm làm việc hay không.

6.1.2 Tín hiệu 2 — Chi phí cộng tác giữa các nhóm tăng lên

Một codebase, bốn team (hotel, flight, car, payment). Ranh giới module trong code giúp các phần không chồng chéo, nhưng có những thứ ranh giới code không ngăn được:

- **Chung một database schema** — team hotel muốn đổi cấu trúc bảng phải hỏi xem có ai đang JOIN vào bảng đó không.
- **Chung một dependency tree** — team payment muốn nâng cấp thư viện HTTP client, cả bốn team phải kiểm tra lại.
- **Chung một release** — tính năng của team flight đã xong từ thứ Hai, nhưng phải chờ team car sửa xong bug mới được lên production.

Khi nhiều đội cùng thay đổi một đơn vị phát hành, chi phí phối hợp có thể gia tăng. Cần phân biệt phần chi phí đến từ ranh giới kỹ thuật với phần đến từ cách tổ chức công việc; tách service chỉ tác động trực tiếp đến phần thứ nhất.

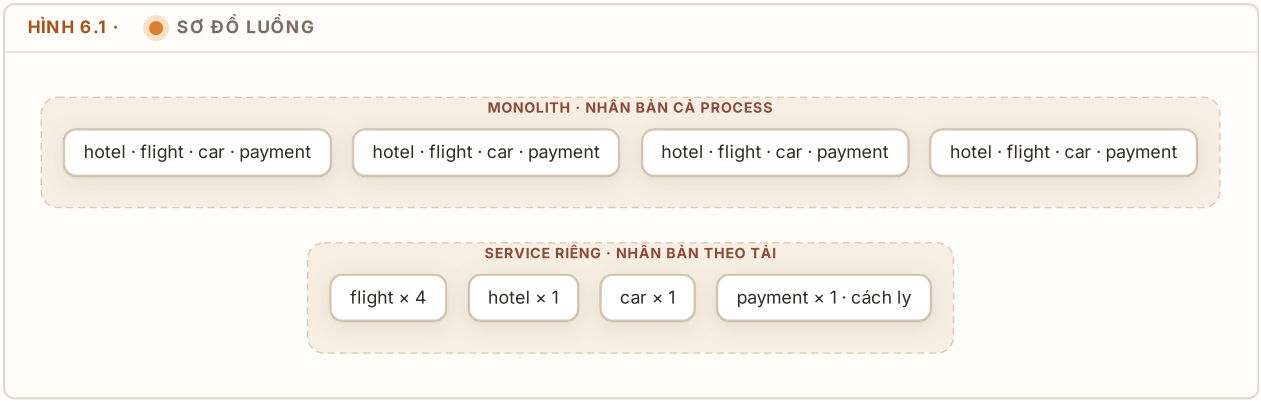
6.1.3 Tín hiệu 3 — Nhu cầu scale lệch nhau

Không phải mọi phần của TravelNow chịu tải như nhau:

Domain	Đặc điểm tải	Nhu cầu riêng
Flight	Tìm kiếm/đặt vé spike mạnh theo mùa, giờ vàng	Cần scale ngang nhanh
Hotel	Tải đều, bước xử lý giả lập có độ trễ cao	Cần nhiều connection chờ I/O
Car	Tải thấp (tính năng tùy chọn)	Chạy tối thiểu là đủ
Payment	Tải thấp nhưng **nhạy cảm nhất**	Cần cách ly, audit, độ ổn định cao

Trong monolith, đơn vị scale là cả process. Flight cần thêm 4 instance vào giờ cao điểm thì hotel, car và payment cũng được nhân bản theo, có thể làm hiệu quả sử dụng tài nguyên kém đi. Một bug làm sập JVM cũng ảnh hưởng mọi module trong process đó. Tuy nhiên, tách service không

tự động cô lập mọi sự cố: dependency đồng bộ, database chung hoặc thiếu bulkhead vẫn có thể tạo cascading failure. Ranh giới process chỉ tạo **khả năng** thu hẹp blast radius; timeout, capacity và quyền truy cập mới biến khả năng ấy thành thực tế.



Nói đến đây, ta cũng nên làm rõ chữ **scalability**. "Hệ thống cần mở rộng tốt" vẫn còn quá chung chung: ta cần phục vụ thêm lượt tìm kiếm, thêm giao dịch đặt vé, hay thêm dữ liệu lịch sử? Mỗi loại tải có thể chạm một điểm nghẽn khác nhau. Với TravelNow, một mục tiêu giả định cụ thể hơn là: tăng gấp đôi số lượt tìm kiếm mỗi giây, vẫn giữ p95 dưới ngưỡng đã thống nhất, với chi phí tăng trong mức chấp nhận được.

Thêm gấp đôi tài nguyên mà xử lý được gần gấp đôi công việc là một kết quả đáng mong đợi, nhưng không phải điều cứ thêm máy là có. Nếu điểm nghẽn nằm ở database hoặc hạn mức của hãng bay, nhân đôi flight-service có thể chỉ tạo thêm các lời gọi đang chờ. Trước khi tách để scale, cần biết **tải nào tăng, nghẽn ở đâu và thêm tài nguyên ở đâu mới có ích**. Chương 12 sẽ trở lại câu chuyện này từ góc độ vận hành.

6.1.4 Điều kiện để quyết định tách

Ba tín hiệu trên chủ yếu là vấn đề của **tổ chức và vận hành**, không phải số dòng code. Microservices cũng làm tăng chi phí compute, network, dữ liệu, kiểm thử và observability. Trước khi tách, đội ngũ nên viết mục tiêu có thể kiểm chứng, chẳng hạn: giảm thời gian lead time của payment từ ba ngày xuống một ngày, cô lập sự cố payment khỏi tìm kiếm chuyến bay, hoặc scale-in, scale-out flight mà không phải scale toàn hệ thống. Nếu không có mục tiêu và baseline, sau sáu tháng sẽ không thể biết kiến trúc mới đã giải quyết vấn đề hay chỉ phân phối lại độ phức tạp.

Ngân sách cũng không chỉ là hóa đơn cloud. Mỗi service mới cần người hiểu nó, phát hành nó và xử lý khi nó có vấn đề. Một đội có tiền thuê thêm máy nhưng chưa có thời gian xây pipeline, quan sát và trực vận hành vẫn phải trả khoản chi phí ấy theo cách khác. Nghiệp vụ còn đang thay đổi nhanh cũng khiến ranh giới vừa tách hôm nay trở thành chỗ phải nối lại ngày mai.

Với một đội nhỏ, pipeline còn nhanh và tải chưa khác biệt giữa các domain, modular monolith thường vẫn là lựa chọn tốt hơn. TravelNow tách để minh họa các vấn đề phân tán; một hệ thống production chỉ nên tách khi lợi ích dự kiến lớn hơn chi phí vận hành dài hạn.

6.2 Từ package thành service

Trước khi tách thành các service độc lập, cần thử các biện pháp ít tốn kém hơn: tối ưu test chậm, thực thi quyền sở hữu schema trong monolith và tách các tác vụ chạy nền có tải riêng. Trong phần minh họa tiếp theo, giả định những biện pháp ấy chưa đáp ứng mục tiêu đã đặt ra. Câu hỏi tiếp theo: **tách theo con đường nào?**

Điểm xuất phát đã được chuẩn bị từ chương 1: **ranh giới domain nghiệp vụ.** `HotelBooking`, `FlightBooking`, `CarBooking` và `Payment` có vòng đời và quy tắc riêng, chỉ được phối hợp thông qua `Trip`. Ranh giới đó là ứng viên cho ranh giới service.

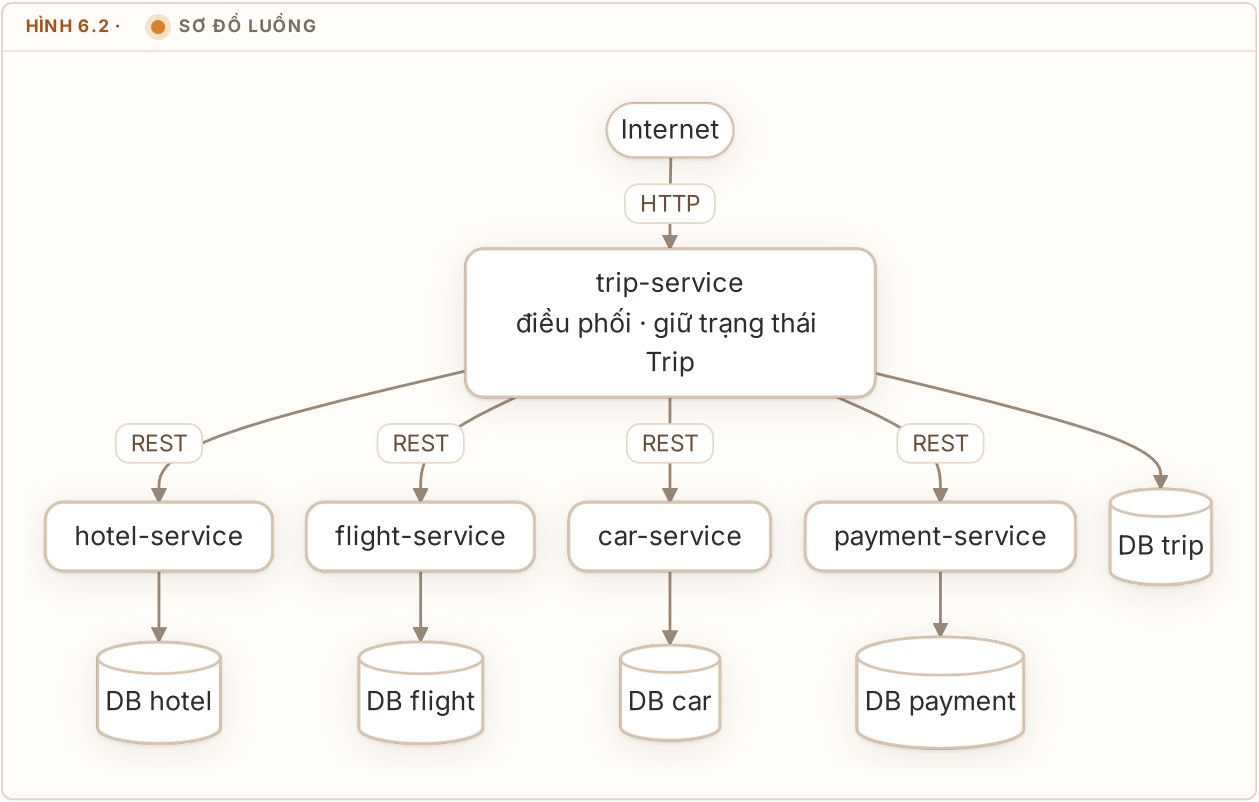
Trước khi biến ranh giới ấy thành một lời gọi qua mạng, cần hỏi thêm: **khi mất kết nối, phần nào vẫn phải hoạt động, dữ liệu nào nó được quyền quyết định và kết quả sẽ được đồng bộ lại ra sao?** Chẳng hạn, một hệ thống POS cần tiếp tục bán hàng khi mất mạng có thể phải giữ một phần xử lý trên thiết bị, dù cùng nghiệp vụ còn có phần chạy trên Cloud. Vì vậy, ranh giới triển khai còn phải xét độ trễ và khả năng hoạt động khi mất kết nối; chia đúng domain chưa đủ để trả lời những câu hỏi đó.

Một ranh giới tốt không được quyết định bằng kích thước code. Nó cần đạt ba tính chất:

- **Cohesion cao:** những hành vi thay đổi cùng nhau nằm trong cùng một service. Quy tắc giữ phòng và hủy phòng thuộc hotel-service; chúng không bị chia giữa trip-service và hotel-service.
- **Coupling thấp:** service chỉ công bố "hợp đồng" cần thiết và che giấu cấu trúc dữ liệu bên trong. Thay đổi cách hotel-service lưu giá phòng không buộc consumer phải deploy lại.
- **Khả năng deploy độc lập:** một thay đổi tương thích trong payment-service có thể build, kiểm thử và release mà không cần phát hành đồng thời service khác. Nếu mọi lần release vẫn phải phối hợp cả năm service, ta đã tạo một distributed monolith.

Các tiêu chí này cũng giúp phát hiện ranh giới sai. Nếu một request chỉ chuyển tiếp qua nhiều service mà mỗi service không có quyết định nghiệp vụ riêng, đó là pass-through coupling. Nếu nhiều service cùng sửa một bảng hoặc cùng phụ thuộc vào một thư viện domain dùng chung, coupling chỉ đổi hình thức chứ chưa biến mất.

HÌNH 6.2 · SƠ ĐỒ LUỒNG



Năm service, năm phạm vi dữ liệu do từng service sở hữu. Các ký hiệu DB trên sơ đồ chưa quyết định loại database hay số máy chủ. Vai trò của từng service:

- **trip-service** — cửa ngõ của hệ thống. Nhận request đặt chuyến đi từ client, gọi lần lượt các service domain, và giữ bản ghi `Trip` với trạng thái tổng thể. Nó *không* biết chi tiết đặt phòng diễn ra thế nào — chỉ biết bước đó thành công hay thất bại.
- **hotel-service, flight-service, car-service** — mỗi service sở hữu trọn vẹn một domain: API riêng, logic riêng, database riêng. `HotelBooking` giờ sống trong database của hotel-service, và *chỉ* hotel-service được đọc/ghi nó.
- **payment-service** — sở hữu domain nhạy cảm nhất. Ranh giới process, dữ liệu và quyền truy cập giảm blast radius trực tiếp từ lỗi ở domain khác; luồng đặt chuyến đi vẫn phải xử lý việc dependency phía trước thất bại.

Và đây là khoảnh khắc quy ước giả lập của mục 1.1.8 phát huy đúng vai trò được thiết kế: từ cú tách này, mỗi domain service chính thức **đóng vai nhà cung cấp** của domain nó — database riêng của service chính là "sổ sách của đối tác". Dưới góc nhìn trip-service, hotel-service là một bên thứ ba đúng nghĩa: giao tiếp qua mạng, có thể chậm, có thể lỗi, và commit của nó nằm ngoài tầm với của bất kỳ transaction nào phía trip-service. Các bài toán phối hợp trong Phần III xuất phát từ ranh giới đó.

Có bốn điều đáng chú ý trong việc phân tách này.

Thứ nhất, cấu trúc bên trong service vẫn quen thuộc, nhưng semantics không giữ nguyên. Mỗi service có thể tiếp tục dùng các tầng API — nghiệp vụ — dữ liệu như monolith. Tuy nhiên, nó cần contract mạng/event, idempotency, local transaction và khả năng quan sát riêng. Ranh

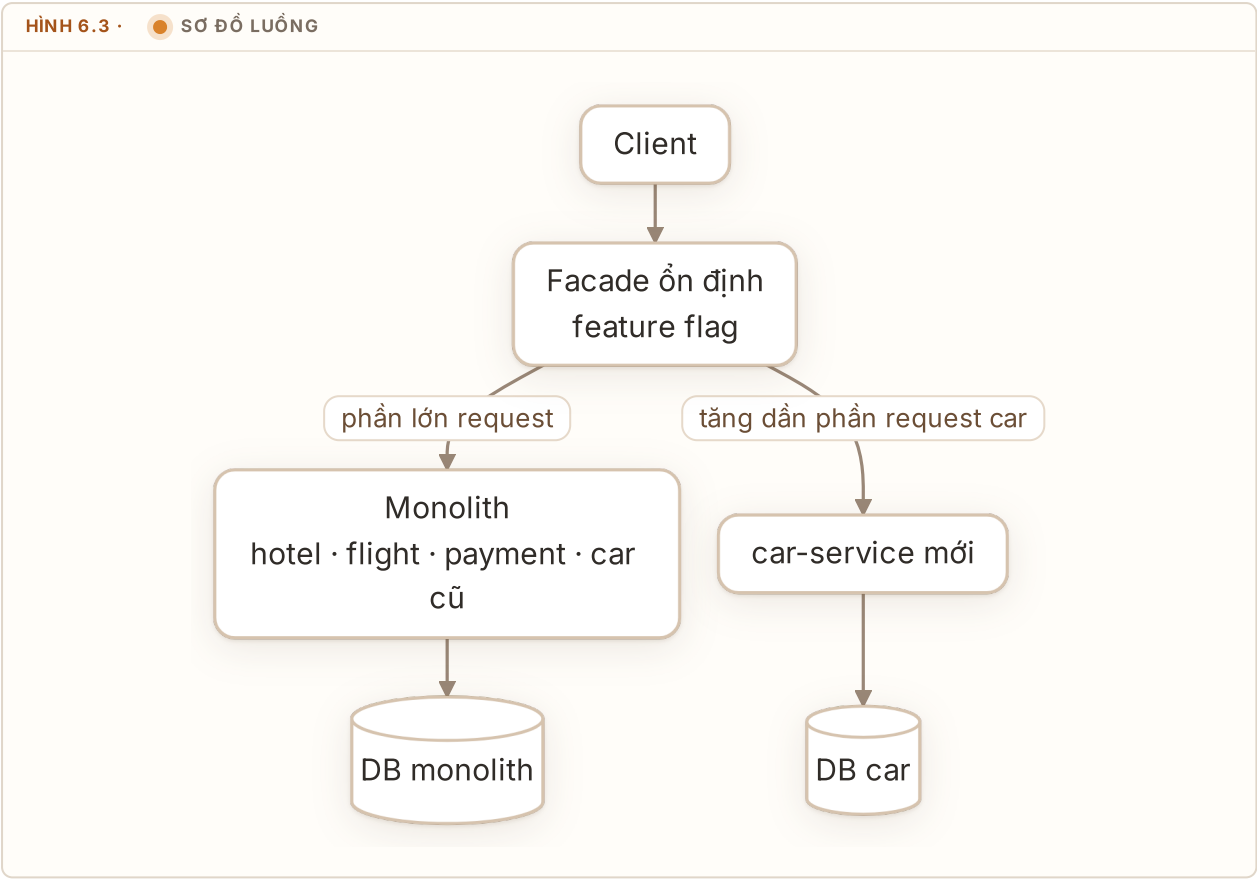
giới process mới thay đổi cả cách service nhận yêu cầu lẫn cách nó trả về kết quả; không thể chỉ cắt package rồi coi như migration đã xong.

Thứ hai, đây là lúc khoản đầu tư từ chương 1 phát huy tác dụng. Vì monolith đã vẽ đúng ranh giới domain, phần lớn công việc là chuyển module thành đơn vị build/deploy độc lập và bổ sung adapter mạng, không phải gỡ coupling trong business logic. Các service có thể tiếp tục ở cùng monorepo; ranh giới deploy không bắt buộc ranh giới repository. Nếu logic payment gọi thẳng vào dữ liệu của hotel, việc tách sẽ cần xử lý coupling trước khi di chuyển process.

Thứ ba, Phần II làm giảm chi phí cơ học của cú tách. Năm service nghĩa là năm artifact, nhiều pipeline và nhiều resource hơn. Với thao tác SSH thủ công, quy mô đó nhanh chóng mất kiểm soát. Khi đã có Dockerfile chuẩn, pipeline template và module Terraform, phần lặp lại có thể được tự động hóa. Chi phí biên không về 0 — mỗi service vẫn cần ownership, on-call, test và observability — nhưng đội ngũ không phải phát minh lại quy trình build/deploy cho từng service.

Thứ tư, migration nên diễn ra theo lát cắt nhỏ. Sơ đồ trình bày trạng thái đích để dễ quan sát, nhưng production không nên tắt monolith rồi thay bằng năm service trong một lần. Một lộ trình an toàn là chọn domain có ranh giới rõ ràng và rủi ro vừa phải — ví dụ car — đặt một facade ổn định trước nó, chuyển traffic từng phần sang car-service, so sánh kết quả, rồi mới xóa code cũ. Đây là tinh thần của **Strangler Fig pattern**: hệ thống mới bao quanh và thay dần từng năng lực của hệ thống cũ. Feature flag giúp chuyển hướng các yêu cầu mới. Chạy song song để đối chiếu chỉ phù hợp với luồng đọc hoặc bản sao không tạo side effect; không gửi cùng lệnh đặt vé hay thanh toán thật vào cả hai hệ thống. Với dữ liệu đã ghi ở service mới, rollback còn cần kế hoạch đồng bộ dữ liệu và xác định bên đang có quyền ghi. Payment, domain nhạy cảm nhất, nên được tách sau khi đội ngũ đã chứng minh được pipeline, quan sát và rollback trên một domain ít rủi ro hơn.

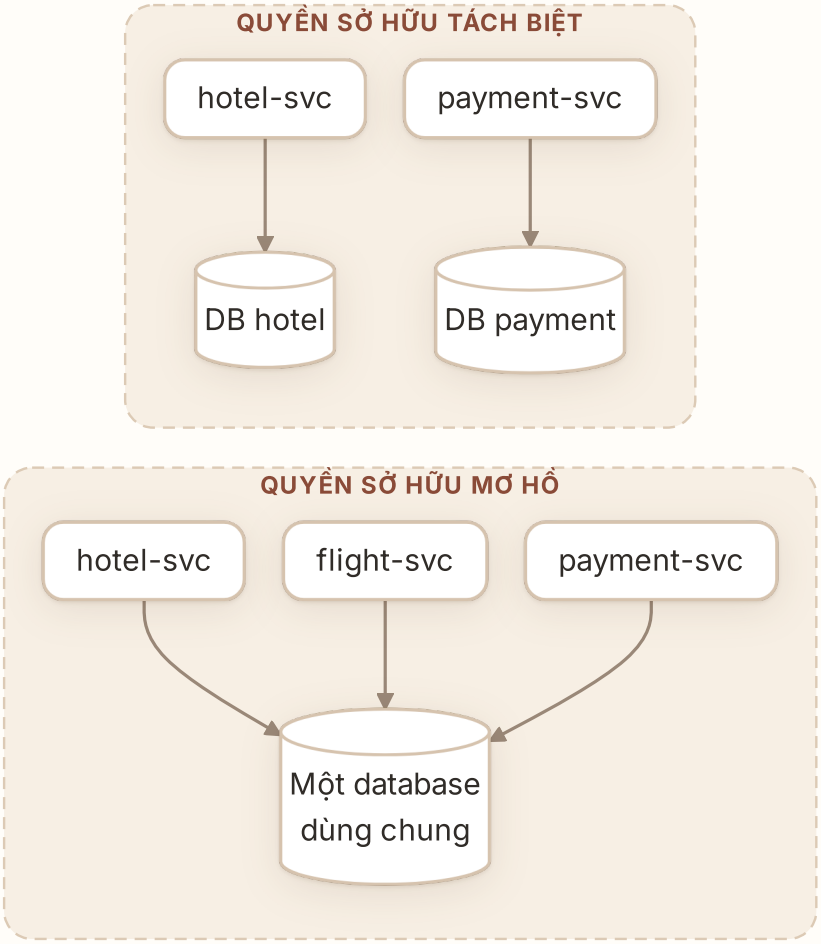
HÌNH 6.3 · SƠ ĐỒ LUỒNG



6.3 Database-per-service: phân chia dữ liệu và quyền truy cập giữa các service

Nhìn lại sơ đồ trên, quyết định *"tốn kém"* nhất không phải là tách code — mà là **tách database**. Sẽ có một cám dỗ rất lớn ở bước này: *"Hay là cứ tách service thôi, còn database dùng chung cho đơn giản?"*

HÌNH 6.4 · SƠ ĐỒ LUỒNG



Dùng chung database và cho các service tùy ý đọc/ghi schema của nhau có thể làm mất phần lớn quyền tự chủ mà việc phân tách nhắm tới:

- **Coupling quay lại bằng cửa sau.** Team hotel đổi schema bảng `hotel_bookings` → query của trip-service (đang JOIN vào bảng đó) sẽ hỏng ngay trên production. Ranh giới service trở thành trang trí; ranh giới thật vẫn là schema chung — và giờ nó còn *vô hình* hơn trước, vì không compiler nào bắt được.
- **Không thể deploy độc lập thật sự.** Mọi migration đều là một cuộc đàm phán bốn team — chính xác vấn đề ở tín hiệu 2 mà ta muốn thoát khỏi.
- **Khó scale hoặc cách ly độc lập.** Một query nặng của flight có thể tranh CPU/I/O/connection với payment. Resource group, replica hay quota có thể giảm tác động, nhưng shared cluster vẫn là failure và capacity domain cần quản lý chung.

Điểm cần tránh không phải một label kiến trúc, mà là **quyền sở hữu mơ hồ**: service nào cũng có thể đổi schema và đọc dữ liệu nội bộ của service khác. Khi đó coupling vẫn tồn tại nhưng không còn được compiler hay API contract làm lộ rõ, nên thay đổi càng khó dự đoán.

TravelNow chọn quy tắc: **mỗi service sở hữu dữ liệu của mình, và service khác không đọc/ghi trực tiếp vào schema đó.** Muốn dữ liệu của hotel thì đi qua contract do hotel-service công bố — API, event hoặc read model được cấp có chủ đích. Đây là quyết định để đạt deploy independence trong bối cảnh eBook, không phải định luật rằng mọi service trên đời phải có một database machine riêng.

"Database riêng" có thể là các database/schema logic trên cùng một cụm RDS, với credential và quyền truy cập riêng. Trong chương 9, TravelNow tiếp tục dùng PostgreSQL trên RDS cho các service để tập trung vào cách phối hợp qua những transaction độc lập. Mỗi service vẫn có thể chọn cách lưu trữ khác khi nhu cầu thực tế đòi hỏi. Điều phải giữ là **quyền sở hữu dữ liệu và ranh giới transaction**; số máy chủ hay tên sản phẩm chưa quyết định được hai điều ấy.

Nhưng sự dứt khoát này phải đánh đổi. Ba thứ mà ta *mất ngay lập tức* khi phân tách database:

1. **Mất JOIN trực tiếp qua dữ liệu nội bộ trong request transaction.** Màn hình "chi tiết chuyến đi" cần API composition, read model/materialized view hoặc data product được công bố. Vẫn có thể dùng công cụ query nhiều nguồn dữ liệu, nhưng làm vậy phải giữ quyền sở hữu và chấp nhận consistency/coupling tương ứng.
2. **Mất foreign key.** Không còn ràng buộc nào ở tầng database đảm bảo `HotelBooking.tripId` trỏ đến một `Trip` có thật. Tính toàn vẹn tham chiếu giờ là trách nhiệm của tầng ứng dụng.
3. **Mất transaction chung.** Đây là mất mát lớn nhất — lớn đến mức nó cần riêng mục 6.5.

6.4 Giao tiếp giữa các service: bắt đầu bằng REST đồng bộ

Các service nói chuyện với nhau như thế nào? Ta bắt đầu với hai cách phối hợp quen thuộc: **đồng bộ**, bên gọi cần kết quả để đi tiếp; và **bất đồng bộ**, bên gửi giao việc rồi nhận kết quả sau, thường qua queue/topic. REST hay gRPC là cách giao tiếp; một API HTTP vẫn có thể nhận việc rồi trả `202` để xử lý nền. Điều cần phân biệt ở đây là luồng nghiệp vụ có phải chờ bên kia hoàn tất ngay hay không. Chương này chọn REST đồng bộ, vì hai lý do: nó là bước tiến hóa *tự nhiên nhất* từ monolith — lời gọi hàm trở thành lời gọi HTTP, luồng xử lý giữ nguyên hình dạng — và nó để dành messaging cho chương 8, nơi ta có lý do nghiệp vụ thật sự để cần nó.

Trong monolith, bước đặt khách sạn gọi logic trong cùng process. Sau khi tách, trip-service gửi một request HTTP sang hotel-service. Luồng nghiệp vụ vẫn nhận diện được, nhưng cách thực hiện đã có những ranh giới mới:

Trong cùng process	Khi gọi qua HTTP
Truyền dữ liệu trực tiếp vào hàm	Biểu diễn yêu cầu thành payload theo contract.
Biết đối tượng cần gọi	Cần địa chỉ service, kết nối và quyền gọi.
Nhận kết quả trong cùng process	Chờ response có thể đến chậm hoặc bị mất.
Các thay đổi giả lập dùng chung transaction ở Phần I	Bên nhận commit trong database do nó sở hữu.

Ví dụ, `POST /api/v1/hotel-bookings` mang `tripId`, khách sạn và ngày nhận/trả phòng. Hotel-service quyết định có giữ phòng được không, lưu kết quả rồi trả về định danh booking cùng trạng thái. Trip-service cần hiểu contract đó, không cần biết hotel-service dùng framework hay database nào.

Địa chỉ service đến từ cấu hình — tiếp nối bài học chương 2. Trên máy phát triển có thể là các cổng khác nhau; trên AWS là endpoint được cấp đường mạng và quyền truy cập phù hợp; trên Kubernetes có thể là tên Service. Thay địa chỉ không tự giải quyết timeout, TLS hay xác thực; mỗi môi trường vẫn phải đáp ứng các điều kiện của lời gọi.

Nhưng endpoint chỉ là địa chỉ; thứ ràng buộc hai service là **contract**. Với `POST /api/v1/hotel-bookings`, contract gồm schema request/response, ý nghĩa từng field, status code, idempotency key và quy tắc tương thích. Hotel-service nên công bố contract bằng OpenAPI và kiểm tra nó trong pipeline. Consumer contract test xác nhận những giả định trip-service thực sự dùng; provider test xác nhận phiên bản mới của hotel-service vẫn đáp ứng các giả định đó.

Quy tắc thay đổi API là expand → migrate → contract: thêm field mới ở dạng optional và cho provider đọc được cả dạng cũ/mới; deploy consumer sử dụng dạng mới; chỉ xóa field cũ sau khi không còn consumer phụ thuộc. Version `/v2` chỉ cần khi semantic thật sự không thể tương thích ngược. Đây là điều kiện thực tế của "deploy độc lập": nếu một thay đổi API luôn yêu cầu năm service release cùng lúc, ranh giới process có tồn tại nhưng autonomy thì chưa.

Một cái bẫy khác là đưa DTO và cả quy tắc của một domain vào thư viện dùng chung. Khi quy tắc đặt phòng hoặc model của hotel thay đổi, trip, flight và payment có thể bị buộc phải nâng phiên bản thư viện rồi phát hành theo, dù chúng không sở hữu nghiệp vụ khách sạn. Coupling qua mã nguồn chỉ chuyển thành coupling qua package.

TravelNow chỉ nên chia sẻ những thành phần ổn định, không mang tri thức nghiệp vụ qua ranh giới — chẳng hạn logging/tracing hoặc client được sinh từ contract. Quy tắc đặt phòng vẫn thuộc hotel-service; service khác gọi API hoặc nhận event theo contract mà hotel-service công bố. Trong thời gian migrate, nhiều phiên bản client có thể cùng tồn tại. Một đoạn mapping nhỏ bị lặp lại thường rẻ hơn một thư viện chung khiến các service mất khả năng deploy độc lập.

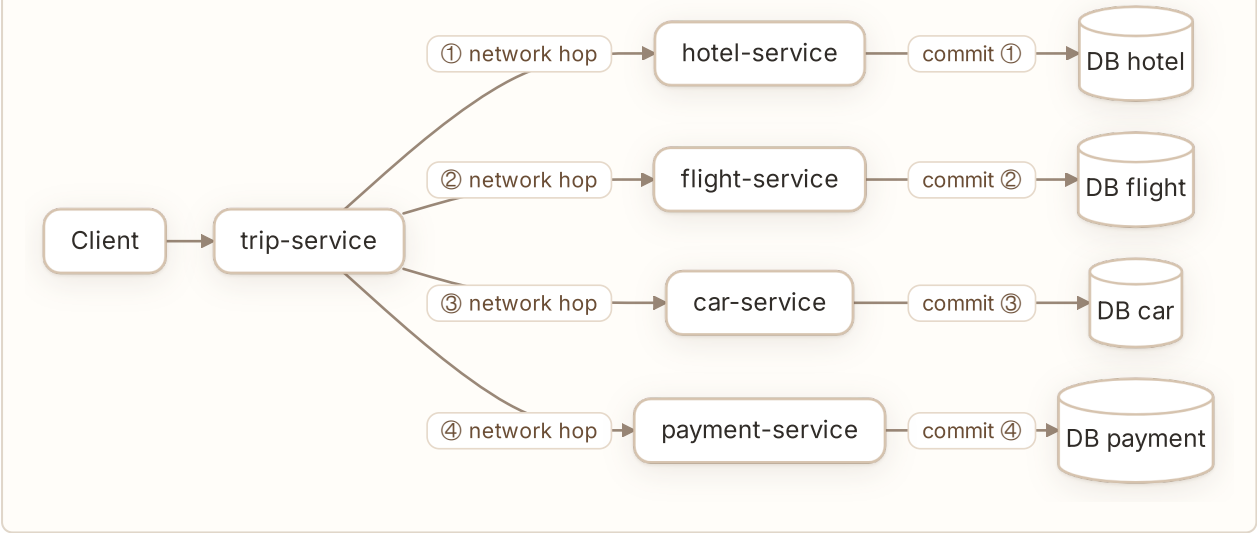
Hãy dừng lại và gọi tên đúng điều vừa xảy ra, bằng ngôn ngữ của chương 2: **luồng nghiệp vụ có thêm bốn lời gọi qua mạng khi người dùng chọn cả xe thuê.** Chương 2 đã tách *state* khỏi compute — app gọi database qua mạng. Nhưng bản thân business flow khi đó vẫn nguyên vẹn trong một process. Bây giờ, chính giữa các *bước nghiệp vụ* xuất hiện những bước giao tiếp qua mạng — nghĩa là bước 2 có thể timeout, bước 3 có thể trả về lỗi, và trip-service có thể chết giữa bước 3 và bước 4. Side effect và exception vẫn cần xử lý trong một process; điểm mới của lời gọi qua mạng là bên gọi có thể mất phản hồi sau khi bên nhận đã commit, nên xuất hiện trạng thái “không biết”.

Trước khi thêm một lời gọi, cũng đáng để đặt câu hỏi: **bước này có thật sự cần dữ liệu hay quyết định từ service kia không?** Nếu luồng đặt chuyến chỉ cần hiển thị tên khách sạn, trip-service có thể dùng thông tin đã lưu thay vì gọi hotel-service. Nhưng khi cần kiểm tra còn phòng

hay không, nó vẫn phải hỏi hotel-service — nơi sở hữu dữ liệu tồn phòng. Mỗi dependency không cần thiết được loại bỏ cũng giảm thêm một điểm có thể làm luồng chậm hoặc gián đoạn.

Một mũi tên từ trip-service sang hotel-service trên sơ đồ che đi DNS, kết nối, TLS, serialization, latency, timeout, retry, versioning và ownership của contract. Với luồng TravelNow hiện tại, REST đồng bộ còn gắn availability của booking với hotel-service tại cùng thời điểm: nếu hotel không phản hồi và không có fallback nghiệp vụ, request đặt chuyến không thể hoàn tất. Chương 8 sẽ đổi coupling theo thời gian này bằng queue, đồng thời nhận lại bài toán backlog và deadline.

HÌNH 6.5 · SƠ ĐỒ LUỒNG



Chú ý dòng cuối của sơ đồ: bốn lần commit **độc lập**. Đây là nguyên nhân trực tiếp của trạng thái dở dang ở mục tiếp theo.

6.5 Giới hạn của @Transactional

Ở chương 1, một local transaction đủ bảo vệ các side effect giả lập trong cùng database. Sau khi tách service, cần xác định lại chính xác phạm vi mà annotation ấy còn bảo vệ.

Trong monolith mô phỏng, toàn bộ thay đổi database của năm bước nằm trong một method `@Transactional` — một transaction, một lần commit. Giờ hãy nhìn phiên bản microservices của `TripBookingService` trong trip-service:

```
@Transactional  // ← vẫn đứng đó... nhưng còn bảo vệ được gì?
public TripDetail bookTrip(BookTripCommand cmd) {
    Trip trip = Trip.create(cmd.getTripId());          // DB của trip-service
    tripRepository.save(trip);

    hotelClient.book(cmd);      // hotel-service  → commit vào DB hotel
    flightClient.book(cmd);     // flight-service → commit vào DB flight
    if (cmd.getCar() ≠ null) {
        carClient.book(cmd);    // car-service    → commit vào DB car
```

```
    }
    paymentClient.pay(cmd);     // payment-service có thể trả lỗi sau các commit trước

    trip.booked();
    return TripDetail.from(trip);
}
```

Bây giờ, xét lại kịch bản sad path của chương 1: các bước giữ chỗ đã thành công nhưng thanh toán bị từ chối. Trong monolith, kết quả là HTTP 422 và các thay đổi của lần đặt chuyến đó không được commit. Còn ở đây?

1. `hotelClient.book(cmd)` — hotel-service nhận request, ghi `HotelBooking` với status `BOOKED`, **commit vào database của nó**. Xong. Từ góc nhìn của hotel-service, nghiệp vụ đã kết thúc thành công.
2. `flightClient.book(cmd)` — tương tự, `FlightBooking` được **commit** vào DB flight.
3. `paymentClient.pay(cmd)` — payment-service trả về lỗi. Exception ném ra trong trip-service.
4. `@Transactional` của trip-service làm đúng nhiệm vụ của nó: rollback... **database của trip-service**. Chỉ vậy thôi. Với giả định exception không được bắt và thuộc loại kích hoạt rollback, bản ghi `Trip` vừa tạo bị rollback. Nếu bắt exception hoặc commit `PENDING` trong transaction khác, kết quả sẽ khác.

Cùng kịch bản, nhìn theo trục thời gian:

Xem Hình 6.6 (sơ đồ khổ ngang) ở trang 99.

Kết quả cuối cùng trong bốn database:

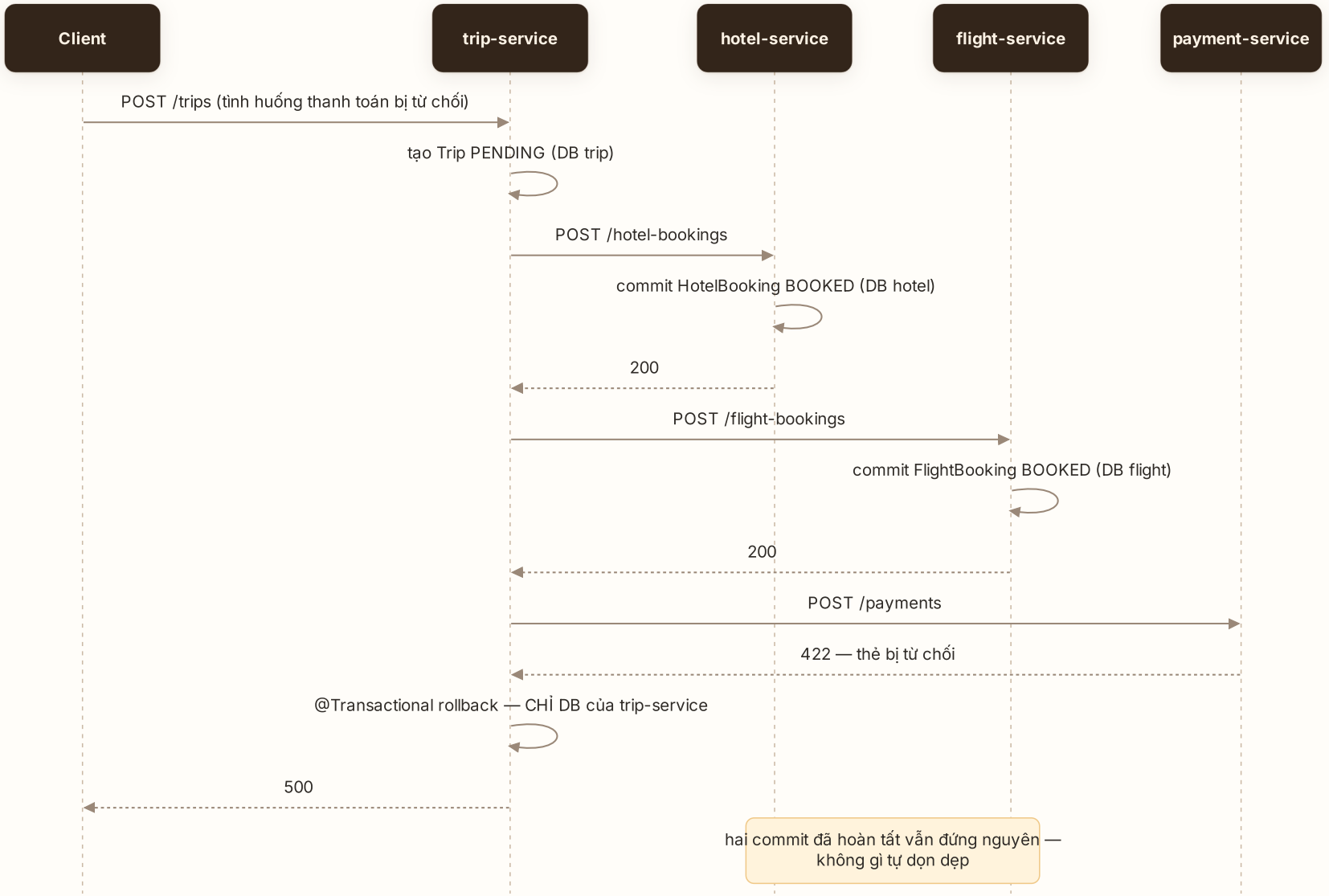
Database	Bản ghi	Trạng thái
DB trip (trip-service)	`Trip`	rollback trong ví dụ này
DB hotel (hotel-service)	`HotelBooking`	`BOOKED` — phòng đang bị giữ
DB flight (flight-service)	`FlightBooking`	`BOOKED` — ghế đang bị giữ
DB payment (payment-service)	—	không có gì — tiền chưa trừ

Kết quả là phòng và vé vẫn bị giữ dù trip-service đã rollback chuyến đi. Nếu khách gửi một yêu cầu mới với ID khác, hệ thống còn có thể giữ thêm tài nguyên. **`@Transactional` vẫn bảo vệ database của trip-service, nhưng không hoàn tác các commit ở service khác.** Phiên bản này chưa có nơi ghi nhận những bước cần hoàn tác, nên trạng thái dở dang không tự được xử lý.

6.5.1 Hoàn tác thủ công — đúng hướng nhưng chưa đủ

Một cách xử lý dễ nghĩ đến là:

HÌNH 6.6 · SƠ ĐỒ TUẦN TỰ



```
try {
    hotelClient.book(cmd);
    flightClient.book(cmd);
    paymentClient.pay(cmd);
} catch (Exception e) {
    flightClient.cancel(cmd.getTripId());   // gọi ngược để dọn dẹp
    hotelClient.cancel(cmd.getTripId());
    throw e;
}
```

Trực giác này *đúng hướng* — đây là dạng sơ khai của Saga pattern ở chương 8. Tuy nhiên, phiên bản chỉ dựa vào `try-catch` chưa lưu được tiến trình hoàn tác và chưa trả lời các tình huống sau:

- Nếu chính lời gọi `flightClient.cancel(...)` **cũng thất bại** thì sao? (Mạng vừa chập chờn làm payment fail thì khả năng cao cancel cũng chập chờn.) Retry? Bao nhiêu lần? Rồi bỏ cuộc giữa chừng thì ai xử lý tiếp?
- Nếu trip-service **chết ngay giữa** khối `catch` — sau khi cancel flight, trước khi cancel hotel? Process mới khởi động lên làm sao biết mình đang nợ một lời cancel?
- Nếu `paymentClient.pay(...)` **timeout** — không phải "thất bại" mà là "không biết"? Tiền có thể đã bị trừ. Cancel toàn bộ rồi tiền tính sao? Mà gửi lại lệnh pay thì có bị trừ hai lần không?

Ba câu hỏi này không thể vá bằng thêm try-catch. Chúng cần những khái niệm nền tảng — timeout, retry, idempotency (chương 7) — và một pattern có hệ thống để tổ chức việc "dọn dẹp" một cách đáng tin cậy (Saga, chương 8).

6.6 Lợi ích và chi phí khi tách service trong TravelNow

Đặt hai kiến trúc cạnh nhau, với đúng những gì ta đã chứng kiến:

Khía cạnh	Monolith (Phần I)	Microservices (từ chương này)
Deploy	Chung một đơn vị phát hành	Có thể phát hành từng service khi contract tương thích
Scale	Cả process; có thể nhân bản cả phần ít tải	Từng service theo đúng tải của nó
Blast radius khi có bug	Có thể ảnh hưởng mọi module trong cùng JVM	Có thể giới hạn theo service nếu dependency, capacity và dữ liệu cũng được cô lập
Team	Chung codebase, thường phát hành cùng đợt	Team sở hữu service, có thể phát hành độc lập; vẫn có thể chung monorepo
Tính nhất quán	Một local transaction cho các side effect nằm trong cùng DB	Mỗi service có local transaction; tính nhất quán liên service cần protocol và vận hành riêng (ch. 7, 8)
Truy vấn dữ liệu	Một câu lệnh JOIN	API composition hoặc read model theo contract

Khía cạnh	Monolith (Phần I)	Microservices (từ chương này)
Độ trễ một request đặt vé	0 network hop giữa các bước	+4 network hop, mỗi hop có thể fail
Debug một request lỗi	Một log, một stack trace	Log rải trên 5 service
Chạy local để dev	Một process ứng dụng và một database	5 app + 5 DB logic — thêm cấu hình kết nối và thứ tự khởi động
Số "thứ" phải vận hành	1 app + 1 DB	5 app + 5 DB + cấu hình kết nối chúng với nhau

Có thể hình dung microservices như các căn hộ trong một chung cư. Mỗi căn có không gian riêng, nhưng cả tòa nhà còn cần điện, nước, thang máy và người bảo trì. Chia xong các căn hộ mới chỉ là một phần công việc; chia xong service cũng vậy. Những phần dùng chung cần người sở hữu và giới hạn rõ ràng, nếu không một sự cố ở đó vẫn có thể ảnh hưởng nhiều service.

Khi các service giao tiếp qua mạng, hệ thống có thể cần thêm service discovery, giới hạn lỗi lan truyền, messaging và khả năng quan sát phân tán. Những cơ chế này cũng có ích với monolith kết nối dịch vụ bên ngoài; chúng không phải đặc quyền của microservices. Các chương tiếp theo chọn từng cơ chế theo vấn đề cụ thể của TravelNow.

Bảng này cần được đọc như một bảng đánh đổi, không phải bảng xếp hạng. Microservices tăng quyền tự chủ về triển khai, co giãn và tổ chức nhóm, đồng thời làm tăng độ phức tạp của giao tiếp, dữ liệu, kiểm thử và vận hành. Khi các tín hiệu ở mục 6.1 đủ rõ và đo được, sự đánh đổi có thể hợp lý; trước thời điểm đó, nó là chi phí trả trước.

6.7 Tóm tắt

TravelNow được chia thành năm service: trip, hotel, flight, car và payment. Mỗi service quản lý dữ liệu thuộc phạm vi nghiệp vụ của mình và cung cấp contract để các thành phần khác tương tác. Để đạt mục tiêu phát hành độc lập, những thay đổi tương thích với contract cần được kiểm thử và triển khai riêng. Nếu mỗi thay đổi vẫn buộc toàn bộ hệ thống phát hành cùng lúc, chúng ta cần xem lại mức độ phụ thuộc giữa các service.

Khi dữ liệu được quản lý riêng, các bước giữ chỗ, thanh toán và cập nhật trạng thái Trip cũng được thực hiện trong những transaction riêng biệt. Một bước thất bại không tự hoàn tác những bước đã commit trước đó. Nếu lời gọi bị timeout, bên gọi còn có thể chưa biết thao tác đã thành công hay chưa, nên chưa đủ thông tin để quyết định thử lại, tiếp tục hay bù trừ.

Chương 7 sẽ phân tích cách xử lý từng lời gọi khi có lỗi hoặc chưa xác định được kết quả. Trên cơ sở đó, Chương 8 xây dựng cơ chế phối hợp, theo dõi tiến độ và phục hồi toàn bộ quy trình đặt chuyến.

CHƯƠNG 7

Distributed Transaction, Timeout và Idempotency

Phần III — Kiến trúc phân tán · *The Hard Parts*

Sau khi tách service, một lần đặt chuyến đi có thể dừng ở giữa: khách sạn đã giữ phòng, nhưng trip-service chưa biết thanh toán đã thành công hay chưa. Trước khi xây dựng cơ chế phục hồi cả chuỗi xử lý, chúng ta phải hiểu chính xác kết quả của từng lời gọi.

Chương này bắt đầu từ phạm vi của distributed transaction, rồi tập trung vào tình huống thường gặp hơn: **bên nhận có thể đã xử lý xong nhưng bên gọi không nhận được phản hồi**. Timeout giới hạn thời gian chờ; retry tạo thêm lần gọi; idempotency kiểm soát tác động khi cùng một ý định được gửi lại. Ba cơ chế này phải được thiết kế cùng nhau.

Câu hỏi xuyên suốt là: khi chưa biết tiền đã bị trừ hay chưa, dựa vào đâu để tra cứu, thử lại hoặc chuyển sang đối soát mà không tạo thêm một lần thu tiền?

7.1 Phối hợp nghiệp vụ khi không còn transaction chung

Trước hết, gọi tên chính xác bài toán ta vừa gặp. Một **transaction** (giao dịch) là một nhóm thao tác với các tính chất ACID — trong đó tính chất ta quan tâm nhất ở đây là **Atomicity**: hoặc toàn bộ transaction thành công, hoặc toàn bộ transaction thất bại mà không để lại thay đổi dở dang.

Khi một nghiệp vụ trải trên **nhiều database hoặc hệ thống độc lập** nhưng vẫn cần duy trì tính nhất quán cho toàn bộ quy trình, ta bước vào bài toán **distributed transaction**. Với TravelNow, một lần đặt chuyến có thể đi qua database của trip-service cùng các database của hotel-service, flight-service, car-service và payment-service. Nếu tích hợp với nhà cung cấp bên ngoài, ranh giới còn mở rộng sang hệ thống của họ.

Một cách tiếp cận lâu đời là cố duy trì atomicity xuyên qua nhiều resource bằng một **atomic distributed transaction**. Các giao thức như **Two-Phase Commit (2PC)** sử dụng một coordinator để các participant cùng chuẩn bị rồi commit hoặc rollback theo một quyết định chung. Cách tiếp cận này vẫn có giá trị khi các data source, driver và yêu cầu availability phù hợp. Nhưng với

TravelNow, các service giao tiếp qua API HTTP và không cùng tham gia một transaction protocol; việc giữ tài nguyên trong khi chờ nhiều participant cũng làm tăng coupling và khiến failure mode phức tạp hơn.

Vì vậy, TravelNow đưa ra một quyết định thiết kế: **không sử dụng một atomic distributed transaction bao trùm toàn bộ chuỗi đặt chuyến**. Thay vào đó, mỗi service thực hiện local transaction của riêng mình và hệ thống chấp nhận rằng dữ liệu giữa các service có thể tạm thời chưa nhất quán. Để toàn bộ quy trình cuối cùng trở về một trạng thái nghiệp vụ hợp lệ, ta phải chủ động thiết kế các cơ chế phục hồi: retry có giới hạn, idempotency, lưu trữ trạng thái tiến triển, bù trừ những bước đã hoàn thành, cảnh báo và đối soát khi hệ thống không thể tự phục hồi.

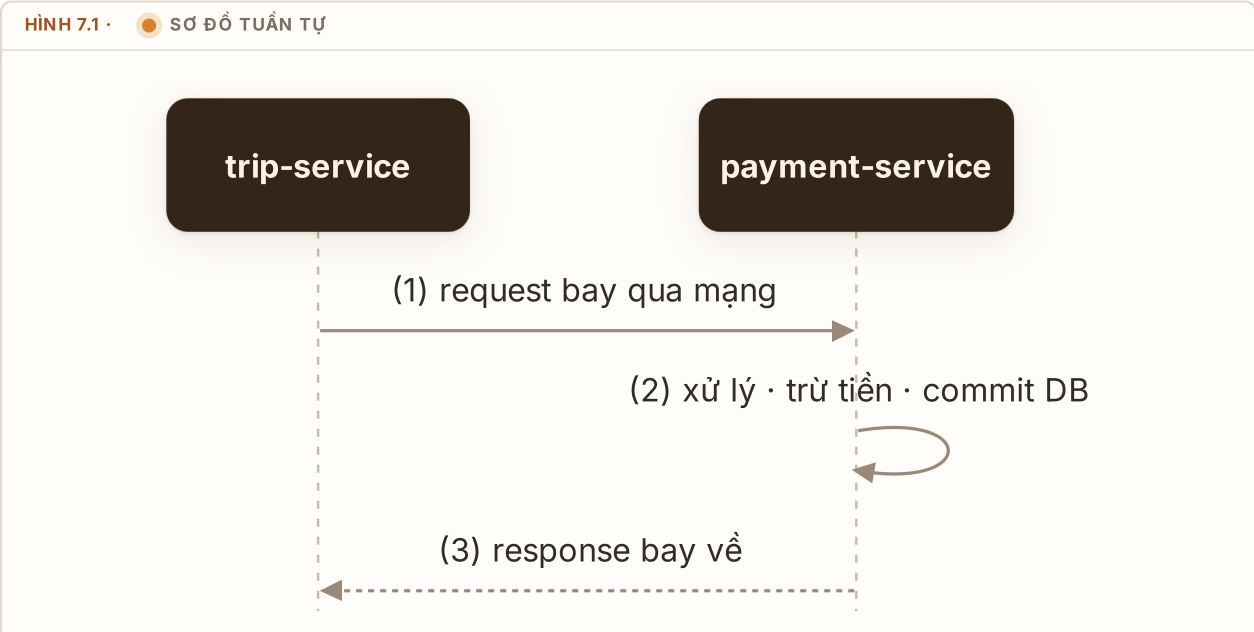
Nói cách khác, **eventual consistency không có nghĩa là dữ liệu sẽ tự trở nên nhất quán nếu ta chờ đủ lâu**. Hệ thống phải có cơ chế để phát hiện công việc còn dang dở, tiếp tục xử lý hoặc sửa chữa sai lệch. Chương này tập trung vào những viên gạch nền tảng như timeout, retry và idempotency; sang Chương 8, chúng ta sẽ dùng chúng để xây dựng **Saga — một cách tổ chức distributed transaction bằng chuỗi local transaction và các hành động bù trừ**.

7.2 Trạng thái thứ ba: "không biết"

Trước khi cấu hình timeout hay retry, cần phân biệt kết quả mà phía nhận đã xử lý với kết quả mà phía gọi quan sát được.

Khi trip-service gửi lệnh trừ tiền sang payment-service, ta quen nghĩ về hai kết quả: **thành công** hoặc **thất bại**. Với lời gọi trong cùng process, bên gọi ít nhất quan sát được return/exception trong cùng failure boundary; side effect vẫn có thể xảy ra trước exception. Khi chuyển sang giao tiếp qua mạng, mất response sẽ tạo thêm một loại mơ hồ:

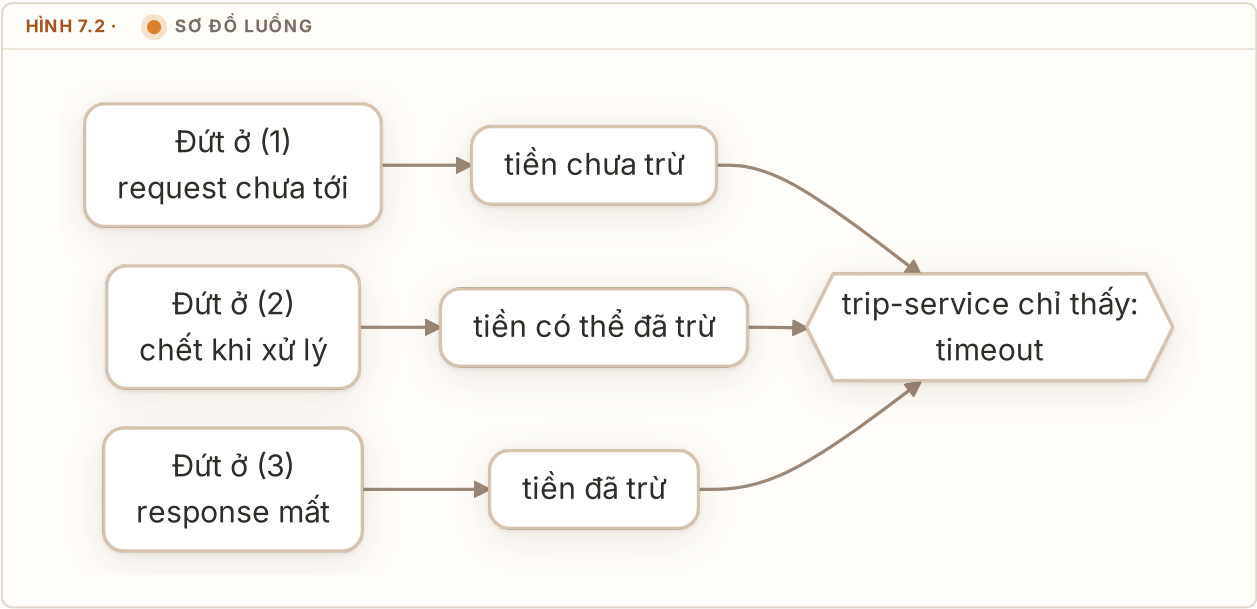
HÌNH 7.1 · SƠ ĐỒ TUẦN TỰ



Và sự cố có thể xảy ra ở **cả ba đoạn**:

- Đứt ở **(1)**: request chưa bao giờ đến nơi. Tiền **chưa** bị trừ.
- Đứt ở **(2)**: payment-service nhận được nhưng chết giữa chừng khi xử lý. Tiền **có thể** đã bị trừ.
- Đứt ở **(3)**: xử lý đã xong, tiền **đã** bị trừ — chỉ có tờ biên nhận là mất trên đường về.

HÌNH 7.2 · SƠ ĐỒ LUỒNG



Từ phía trip-service, **cả ba tình huống trên có thể biểu hiện giống hệt nhau**: chờ quá lâu nhưng không nhận được response. Vì vậy, timeout không cho trip-service biết payment-service thực sự đã xử lý đến đâu hay tiền đã bị trừ hay chưa. Muốn xác định kết quả, nó cần thêm cơ chế như query trạng thái, idempotency key hoặc một protocol khác giữa hai bên. Do đó, với một lời gọi qua mạng, ta cần tính đến ba khả năng: **thành công**, **thất bại đã được xác nhận**, và **chưa biết kết quả** (unknown).

Giữ trạng thái "chưa biết" có vẻ bất tiện, nhưng nó giúp hệ thống không quyết định dựa trên một kết quả chưa được xác nhận. Ba mục tiếp theo sẽ lần lượt giải quyết vấn đề đó: **timeout** giới hạn thời gian chờ; **retry hoặc reconciliation** xác định nên làm gì khi chưa biết kết quả; còn **idempotency** giúp việc gửi lại request không tạo ra side effect trùng lặp.

7.3 Timeout — giới hạn thời gian chờ

7.3.1 Tại sao remote call cần deadline

Nếu thời gian chờ không được giới hạn theo nhu cầu nghiệp vụ, các request có thể giữ tài nguyên quá lâu. Trong mô hình HTTP đồng bộ dùng thread pool đang xét, nhiều request chờ payment cùng lúc có thể làm cạn pool của trip-service, ảnh hưởng cả các request khác. Đây là **cascading failure** — lỗi lan truyền từ một thành phần sang các thành phần phụ thuộc.

Timeout giúp giới hạn thời gian giữ tài nguyên, nhưng cần phối hợp với giới hạn đồng thời và cách từ chối tải. Mục tiêu là **resiliency**: duy trì phần phục vụ có thể duy trì khi có lỗi, rồi phục hồi trong phạm vi đã thiết kế.

Giả sử client có tối đa 2 giây để thiết lập kết nối và 5 giây để chờ phản hồi. Hai ngưỡng này bảo vệ từng chặng của lời gọi. Chúng chưa phải **deadline tổng** của lần đặt chuyến: ngân sách tổng còn bao gồm các bước khác, lần thử lại và khoảng chờ giữa các lần thử. Các con số ở đây chỉ để phân biệt phạm vi, không phải cấu hình mặc định cho mọi hệ thống.

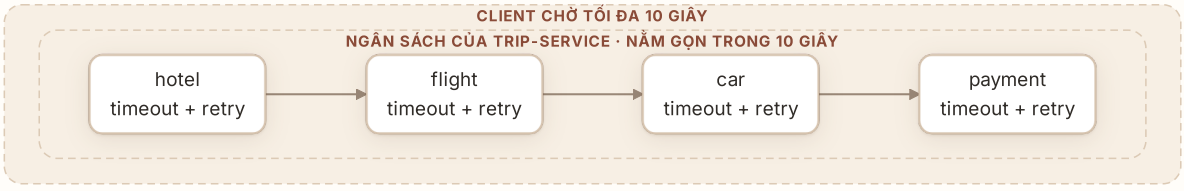
Hai loại timeout khác nhau về ý nghĩa chẩn đoán: **connect timeout** cho biết client không thiết lập được kết nối tới endpoint đã chọn; **read timeout** cho biết không nhận được dữ liệu đúng hạn sau khi kết nối. Chúng là tín hiệu vận hành, không phải bằng chứng nghiệp vụ tuyệt đối — proxy, connection reuse và nhiều tầng mạng có thể làm đường đi phức tạp hơn mô hình tối giản.

7.3.2 Chọn giá trị bao nhiêu?

Không có con số đúng cho mọi hệ thống, nhưng có phương pháp thay cho đoán mò:

- **Nhìn từ dưới lên:** đo thời gian phản hồi thực tế của downstream. Nếu p99 của payment-service là 800ms, đây là dữ liệu để chọn timeout và đo tỷ lệ timeout giả. Cần tính thêm kết nối, cold start và biến động tải; riêng p99 chưa đủ để chốt một ngưỡng cho mọi tình huống.
- **Nhìn từ trên xuống:** upstream cho ta bao nhiêu thời gian? Nếu client bỏ cuộc sau 10 giây, thì tổng thời gian trip-service dành cho cả bốn bước (kể cả retry!) phải nằm trong 10 giây đó. Ngân sách thời gian (timeout budget) phải được chia lồng nhau: timeout của bên ngoài phải *lớn hơn* tổng những gì diễn ra bên trong — nếu tổng ngân sách vượt quá thời gian client chờ, backend có thể còn xử lý sau khi client đã bỏ cuộc. Khi đó cần trả trạng thái đang xử lý và cho phép tra cứu kết quả.

HÌNH 7.3 · SƠ ĐỒ LUỒNG



7.3.3 Điều timeout KHÔNG có nghĩa

Sai lầm phổ biến nhất khi xử lý timeout nằm trong một dòng suy nghĩ tự nhiên: *"timeout rồi — vậy là thất bại, coi như tiền chưa trừ, báo lỗi cho khách và xong."* Nhìn lại mục 7.2: đứt ở đoạn (3) — xử lý **đã thành công** — cũng biểu hiện thành timeout. Nếu ta coi timeout là "chưa trừ tiền" và cho khách đặt lại chuyến mới, ta có thể vừa trừ tiền khách **hai lần**.

Timeout chỉ kết thúc việc chờ; nó không xác nhận rằng side effect đã thất bại. Và khi không biết, ta có hai lựa chọn: đi hỏi (query trạng thái bên kia) hoặc gửi lại một cách an toàn. Cả hai đều dẫn đến mục tiếp theo.

7.4 Retry — gửi lại, nhưng có kỷ luật

Một số lỗi mạng hoặc tình trạng quá tải chỉ xảy ra **tạm thời (transient)**, vì vậy thử lại sau một khoảng thời gian có thể thành công. Tuy nhiên, **không phải cứ gặp lỗi là retry**. Tùy từng tình huống, hệ thống có thể cần kiểm tra trạng thái của request trước đó, đưa công việc vào queue để xử lý sau, trả về trạng thái chưa xác định hoặc chuyển sang đối soát thủ công. Retry không đúng cách có thể gây ra hai vấn đề: **thực hiện cùng một side effect nhiều lần** và **tạo thêm tải cho downstream đúng lúc nó đang gặp sự cố**. Vì vậy, khi sử dụng retry, ta cần tuân theo năm nguyên tắc sau:

1. Chỉ retry những lỗi có khả năng chỉ xảy ra tạm thời. Timeout, connection refused hoặc HTTP `503` thường là những trường hợp có thể thử lại. Ngược lại, một lỗi nghiệp vụ đã rõ ràng, chẳng hạn HTTP `422` báo "thẻ bị từ chối", thì retry cũng không giải quyết được gì. Cũng không nên đơn giản cho rằng mọi `5xx` đều retry còn mọi `4xx` đều không: `429` thường yêu cầu chờ theo `Retry-After`, `408` có thể thử lại, còn một số `5xx` của thao tác không idempotent lại cần kiểm tra trạng thái trước khi gửi lại. Vì vậy, **retry policy phải dựa trên contract cụ thể của từng endpoint**.

2. Giới hạn số lần retry. Không được thử lại vô hạn. Trong ví dụ bên dưới, ta cho phép tối đa 3 lượt gọi, tính cả lần đầu tiên. Con số này chỉ để minh họa; trong thực tế phải lựa chọn dựa trên deadline của request và khả năng chịu tải của downstream. Nếu đã hết số lượt mà vẫn chưa xác định được kết quả, hãy dừng retry và chuyển trạng thái hiện tại lên tầng điều phối để quyết định bước tiếp theo.

3. Dùng exponential backoff — mỗi lần retry chờ lâu hơn một chút. Nếu downstream đang quá tải, retry ngay lập tức chỉ khiến tình hình tệ hơn. Thay vào đó, khoảng thời gian giữa các lần thử nên tăng dần, chẳng hạn `1s → 2s → 4s`. Cách này tạo cho downstream một khoảng thời gian để phục hồi trước khi nhận thêm request.

4. Thêm jitter — tránh tất cả cùng retry một lúc. Giả sử payment-service gặp sự cố trong 5 giây. Nếu 1.000 request đều sử dụng cùng một lịch backoff, chúng có thể đồng loạt retry vào đúng những thời điểm giống nhau. Downstream vừa phục hồi đã phải nhận một lượng lớn request cùng lúc và có thể lại quá tải. Đây là **retry storm** hay **thundering herd**.

Để tránh tình trạng đó, ta thêm **jitter**: thay vì tất cả cùng chờ một khoảng thời gian cố định, mỗi request sẽ có một chút ngẫu nhiên trong thời gian chờ, nhưng vẫn nằm trong giới hạn của backoff và deadline. Nhờ vậy, các lần retry được giãn ra theo thời gian thay vì dồn vào cùng một thời điểm. Cũng cần tránh retry ở nhiều tầng: nếu client thử tối đa 3 lượt và trip-service cũng thử tối đa 3 lượt, một thao tác của người dùng có thể tạo ra tới 9 lần gọi xuống payment-service.

5. Retry phải được quan sát và cảnh báo. Retry dựa trên giả định rằng lỗi chỉ là tạm thời và lần thử sau có thể thành công. Nhưng nếu downstream thực sự đang gặp sự cố kéo dài, retry có thể khiến hệ thống bên ngoài trông như vẫn hoạt động trong khi bên trong ngày càng có nhiều request phải thử lại. Vì vậy, **retry rate phải là một metric được theo dõi**. Khi tỷ lệ retry tăng bất thường, đó có thể là tín hiệu sớm cho thấy downstream đang có vấn đề, ngay cả khi phần lớn request cuối cùng vẫn thành công.

```
# Mã giả: retry cho endpoint đã có contract idempotency.
# maxAttempts tính cả lần gọi đầu tiên.
for attempt in 1..3:
    nếu deadline tổng đã hết: trả UNKNOWN để tầng điều phối xử lý
    result = gọi payment với cùng key, payload và timeout còn lại
    nếu thành công hoặc lỗi nghiệp vụ xác nhận: trả result
    nếu lỗi không thuộc policy retry: chuyển sang query/đối soát
    nếu còn lượt thử: chờ backoff có jitter trong ngân sách còn lại
trả UNKNOWN nếu chưa xác nhận được kết quả thanh toán
```

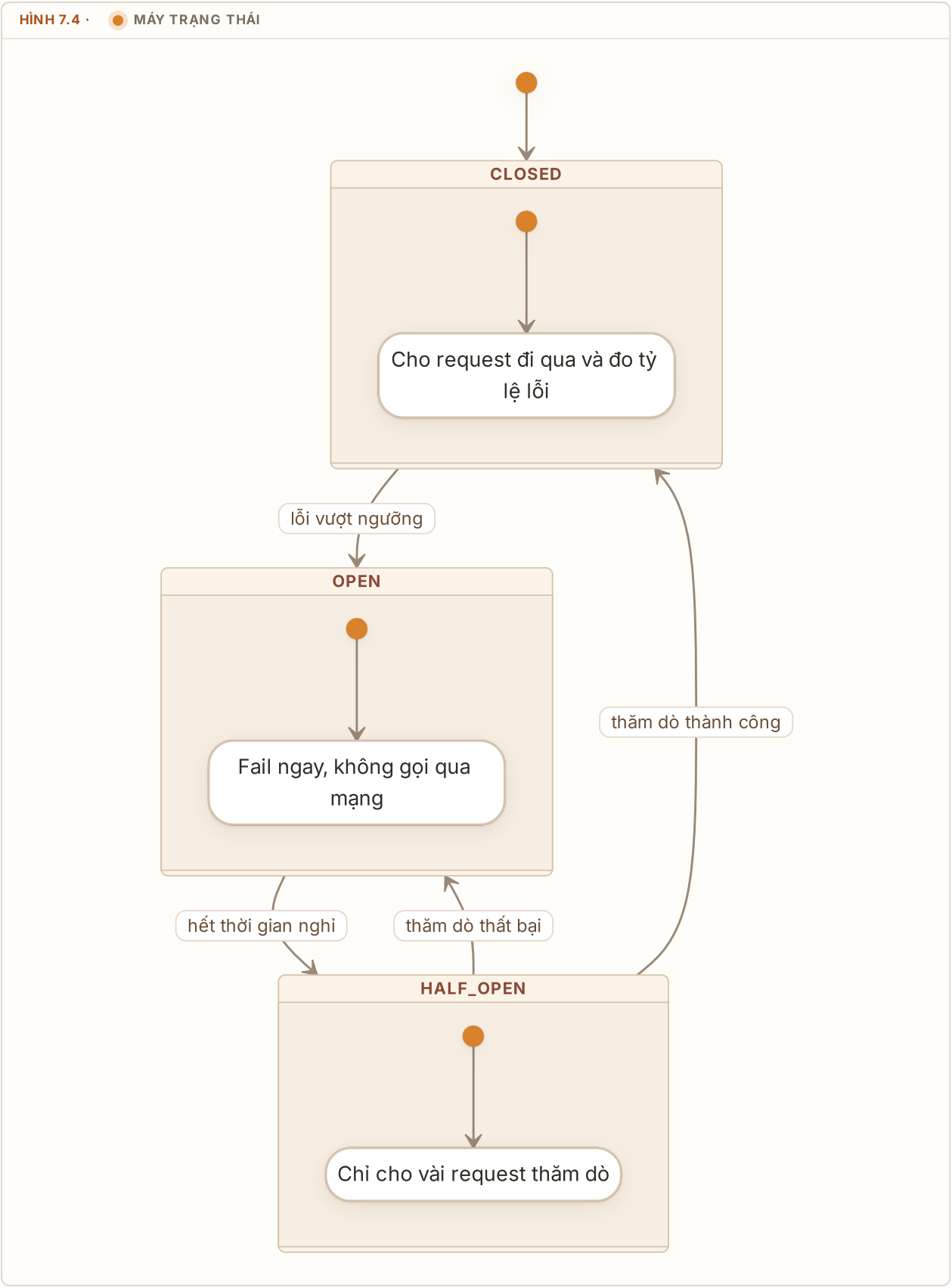
Trong ví dụ này, retry policy được đặt tại tầng trực tiếp gọi payment-service để tránh nhiều tầng cùng retry và vô tình nhân số request. Khi sử dụng các thư viện như Spring Retry hoặc Resilience4j, cần cấu hình policy cho đúng: **lỗi nào được retry, thử tối đa bao nhiêu lần, deadline còn lại là bao nhiêu, và retry phối hợp với circuit breaker theo thứ tự nào**.

7.4.1 Circuit breaker — biết khi nào nên ngừng gọi

Các nguyên tắc retry ở trên giúp giảm tải cho downstream khi sự cố chỉ xảy ra tạm thời. Nhưng nếu downstream gặp sự cố **kéo dài**, retry vẫn chưa đủ. Giả sử payment-service ngừng hoạt động trong 10 phút: mỗi request mới vẫn cố gọi, chờ timeout rồi retry. Khi có hàng nghìn request cùng làm như vậy, chúng vừa tiếp tục tạo tải lên một service đang gặp sự cố, vừa chiếm thread và connection của chính upstream cho những lời gọi có rất ít khả năng thành công.

Trong trường hợp này, hệ thống cần biết **khi nào nên tạm ngừng gọi đến một downstream đang gặp sự cố**. Đó là vai trò của **circuit breaker** — cơ chế hoạt động tương tự một cầu dao tự ngắt. Circuit breaker được đặt trước lời gọi tới downstream và theo dõi kết quả của các request. Khi số lượng hoặc tỷ lệ lỗi vượt quá ngưỡng đã cấu hình, nó sẽ tạm thời chặn các lời gọi tiếp theo thay vì tiếp tục gửi chúng tới downstream.

HÌNH 7.4 · MÁY TRẠNG THÁI



Khi tỷ lệ lỗi vượt quá ngưỡng đã cấu hình, circuit breaker chuyển sang trạng thái **OPEN**. Lúc này, các request mới sẽ bị từ chối ngay tại upstream thay vì tiếp tục gọi xuống downstream. Nhờ đó, downstream có thời gian phục hồi, còn upstream không phải lãng phí thread, connection và thời gian chờ cho những lời gọi ít có khả năng thành công. Lỗi do circuit breaker đang OPEN cũng

không nên tiếp tục bị retry, nếu không hệ thống sẽ lặp lại những lần gọi mà breaker đã chủ động chặn.

Sau một khoảng thời gian, circuit breaker chuyển sang **HALF_OPEN** và cho phép một số request thử nghiệm đi xuống downstream. Nếu các request này thành công, breaker chuyển về **CLOSED** và hoạt động bình thường trở lại. Nếu chúng tiếp tục thất bại, breaker lại chuyển về **OPEN** và tiếp tục chặn request.

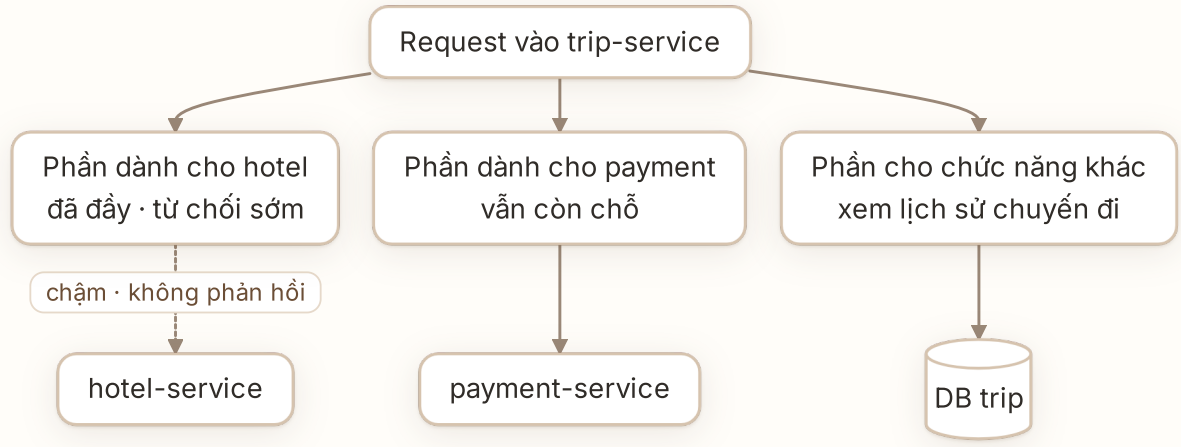
Trong hệ Java, **Resilience4j** là một thư viện phổ biến để triển khai circuit breaker, retry và time limiter. Với TravelNow, mỗi downstream nên có policy riêng phù hợp với đặc điểm của nó thay vì áp dụng một cấu hình chung cho tất cả. Chẳng hạn, hotel-service gặp sự cố chỉ làm circuit breaker của hotel mở, không trực tiếp ảnh hưởng đến breaker của payment-service. Tuy nhiên, **circuit breaker chỉ quyết định có nên tiếp tục gọi một dependency hay không; nó không quyết định nghiệp vụ phải làm gì khi dependency đó không khả dụng**. Việc tiếp tục, hủy booking, chờ xử lý hay thực hiện một phương án thay thế vẫn thuộc về logic nghiệp vụ ở tầng cao hơn.

7.4.2 Bulkhead và load shedding — đừng để một dependency chiếm hết tài nguyên

Timeout và circuit breaker vẫn chưa giải quyết hết vấn đề. Trước khi circuit breaker kịp phát hiện sự cố và chuyển sang OPEN, những lời gọi chậm có thể đã chiếm gần hết thread, connection hoặc các tài nguyên thực thi **dùng chung**. Chẳng hạn, nếu các request tới hotel-service chiếm hết connection pool, ngay cả một request chỉ muốn xem lịch sử chuyến đi — vốn không liên quan đến hotel — cũng có thể bị ảnh hưởng.

Để tránh một dependency làm cạn tài nguyên của cả hệ thống, TravelNow **tách và giới hạn tài nguyên cho từng downstream hoặc từng nhóm công việc**. Tùy runtime, giới hạn này có thể được thực hiện bằng connection pool, semaphore, thread pool hoặc reserved concurrency. Đây là nguyên tắc **bulkhead**: nếu hotel-service gặp vấn đề, nó chỉ được phép sử dụng hết phần tài nguyên dành cho hotel, không thể chiếm sang phần dành cho payment-service hay các chức năng khác.

HÌNH 7.5 · SƠ ĐỒ LUỒNG



Nhưng giới hạn tài nguyên cũng đặt ra một câu hỏi khác: **điều gì xảy ra khi phần tài nguyên đó đã đầy?** Nếu tiếp tục đưa request vào một hàng đợi không giới hạn, hệ thống chỉ chuyển từ quá tải sang hàng loạt request chờ lâu rồi timeout. Thay vào đó, hệ thống nên **từ chối sớm một phần request** khi đã vượt quá khả năng xử lý. Với HTTP, có thể trả `429` hoặc `503` kèm `Retry-After`; với xử lý bất đồng bộ, chỉ nên nhận thêm công việc khi queue và deadline vẫn nằm trong giới hạn đã thiết kế. Cách chủ động loại bớt tải để bảo vệ phần còn lại của hệ thống được gọi là **load shedding**. Số request bị từ chối cũng cần được theo dõi bằng metric để phát hiện tình trạng quá tải.

Một vấn đề khác là **fallback**. Circuit breaker hay thư viện resiliency chỉ phát hiện và hạn chế ảnh hưởng của sự cố; chúng không thể tự quyết định hệ thống nên trả kết quả gì cho người dùng. Đó là quyết định nghiệp vụ. Ví dụ, TravelNow có thể tạm thời không hiển thị gợi ý thuê xe nếu đây chỉ là một tính năng bổ sung. Nhưng nếu khách đã chọn thuê xe như một phần của booking, hệ thống không thể âm thầm bỏ qua bước đó rồi vẫn báo toàn bộ chuyến đi đã được đặt thành công. Với payment, càng không thể dùng dữ liệu cache hay một kết quả giả để thay thế. Nếu chưa xác định được tiền đã bị trừ hay chưa, trạng thái đúng vẫn phải là `PENDING` hoặc `UNKNOWN` để hệ thống tiếp tục tra cứu hoặc đối soát.

Cuối cùng, quay lại retry. Mã giả ở đầu mục có một điều kiện rất quan trọng: **endpoint được retry phải có contract idempotency phù hợp**. Nếu không, lần gọi đầu tiên có thể đã trừ tiền thành công nhưng response bị mất; retry sau đó lại thực hiện thêm một lần trừ tiền nữa. Vì vậy, muốn retry một thao tác có side effect một cách an toàn, phía nhận phải có khả năng nhận ra request được gửi lại và **không thực hiện side effect thêm lần nữa**. Đó chính là vấn đề mà **idempotency** giải quyết ở mục tiếp theo.

7.5 Idempotency — biến "gửi lại" thành thao tác an toàn

7.5.1 Định nghĩa

Một thao tác được gọi là **idempotent** khi thực hiện lại cùng một yêu cầu nhiều lần vẫn tạo ra hiệu ứng cuối cùng giống như chỉ thực hiện một lần. Ví dụ, yêu cầu bật công tắc sang trạng thái ON là idempotent: gửi lệnh bật 1 lần hay 5 lần thì kết quả cuối cùng vẫn là đèn sáng. Ngược lại, thao tác “cộng thêm 100.000 đồng vào tài khoản” không idempotent, vì thực hiện 5 lần sẽ làm số dư tăng 5 lần.

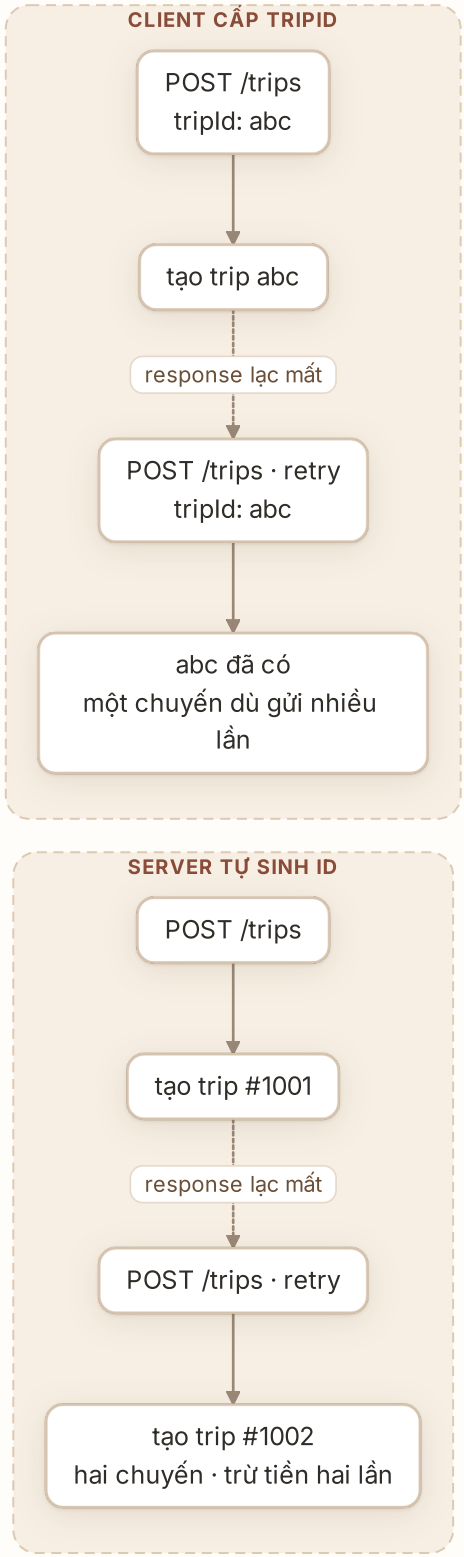
Khái niệm này cũng xuất hiện trong HTTP. `GET`, `PUT` và `DELETE` có semantics idempotent theo **hiệu ứng dự kiến (intended effect)**: gửi lại cùng một request nhiều lần không làm thay đổi kết quả cuối cùng so với thực hiện một lần, dù response, status code hay log của từng lần gọi có thể khác nhau. `POST` thì không có đảm bảo này theo mặc định. Tuy nhiên, một API sử dụng `POST` vẫn có thể tự thiết kế **idempotency contract**, thường bằng cách yêu cầu client gửi kèm một idempotency key. Vì các thao tác đặt chuyến đi và thanh toán của TravelNow sử dụng `POST`, ta cần chủ động thiết kế cơ chế này ở tầng nghiệp vụ thay vì dựa vào HTTP method.

7.5.2 Chìa khóa đã nằm sẵn trong thiết kế: tripId do client cấp

Nhìn lại một quyết định có vẻ vu vơ từ chương 1: `tripId` là UUID **do client sinh ra và gửi lên**, không phải do server cấp phát. Bây giờ quyết định đó lộ rõ vai trò: `tripId` chính là **idempotency key** — định danh giúp server nhận ra *"yêu cầu này tôi từng gặp rồi."*

So sánh hai thiết kế khi client gửi lại sau timeout:

HÌNH 7.6 · SƠ ĐỒ LUỒNG



Nếu không có idempotency key, server rất khó biết một request vừa nhận được là **một yêu cầu mới** hay chỉ là **yêu cầu cũ được gửi lại sau sự cố**. Chỉ so sánh nội dung request là chưa đủ, bởi hai request giống hệt nhau vẫn có thể đại diện cho hai ý định khác nhau — chẳng hạn một khách hàng thực sự muốn đặt hai chuyến giống nhau cho hai người khác nhau.

Vì vậy, trong TravelNow, client tạo một idempotency key trước lần gửi đầu tiên và **giữ nguyên key đó trong mọi lần retry của cùng một yêu cầu**. Cũng có thể thiết kế để server cấp trước một mã cho thao tác rồi client sử dụng mã đó khi gửi request. Dù theo cách nào, nguyên tắc quan trọng vẫn là: **key phải được xác định trước khi side effect có thể xảy ra và được sử dụng lại khi cùng yêu cầu được gửi lại**.

Một cách khác thường được nhắc đến là **fingerprinting**: tạo một hash từ nội dung request để nhận diện các request giống nhau. Tuy nhiên, hash payload không thể thay thế hoàn toàn idempotency key. Nếu đưa những trường thay đổi như timestamp vào hash, hai lần retry của cùng một yêu cầu có thể tạo ra hai hash khác nhau. Ngược lại, nếu hai yêu cầu độc lập tình cờ có cùng nội dung, chúng có thể tạo ra cùng một hash dù thực chất là hai ý định khác nhau.

Vì vậy, TravelNow sử dụng **idempotency key để xác định ý định của request**, còn **hash payload để kiểm tra nội dung gắn với ý định đó**. Nếu cùng một key được gửi lại với cùng payload, hệ thống có thể coi đó là retry của yêu cầu trước. Nếu cùng key nhưng payload thay đổi, hệ thống biết rằng key đang bị sử dụng sai và có thể từ chối request.

7.5.3 Trùng idempotency key không đồng nghĩa với có lỗi xảy ra

Ở monolith trong Phần I, nếu nhận được một `tripId` đã tồn tại, hệ thống trả về HTTP `400` vì coi đó là request trùng từ client. Nhưng trong hệ phân tán, một request trùng chưa chắc là lỗi. Client có thể đã gửi request trước đó, bị timeout vì không nhận được response, rồi **gửi lại cùng request để xác định hoặc hoàn tất kết quả**.

Nếu server vẫn trả `400` khi gặp key đã tồn tại, client lại phải gọi thêm API để kiểm tra request trước đó thực sự đã được xử lý hay chưa. Như vậy, mỗi client đều phải tự xử lý một phần logic phục hồi sau timeout.

Với idempotency, ta thiết kế contract đơn giản hơn: **nếu nhận lại cùng idempotency key cho cùng một yêu cầu, server không thực hiện side effect lần nữa mà trả lại kết quả của lần xử lý trước**. Handler có thể được gọi nhiều lần, nhưng trong phạm vi mà idempotency contract bảo vệ, side effect chỉ có hiệu lực một lần:

```
# Mã giả ở điểm nhận yêu cầu; không giữ transaction khi gọi qua mạng.
đăng ký duy nhất (client, operation, key) cùng hash payload
nếu key đã có:
    payload khác → từ chối xung đột
    đã hoàn tất → trả kết quả đã lưu
    đang xử lý → trả trạng thái hiện tại và đường dẫn tra cứu
nếu key mới:
    ghi Trip PENDING và ý định khởi chạy workflow trong cùng local transaction
    để worker tiếp tục xử lý từ trạng thái đã lưu
    trả 202 cùng tripId và đường dẫn tra cứu
```

Cách xử lý trên cũng chuẩn bị cho **Saga ở Chương 8**: HTTP request không nhất thiết phải giữ kết nối và chờ cho toàn bộ quy trình nghiệp vụ hoàn tất. Server có thể tiếp nhận yêu cầu, lưu lại trạng thái cần thiết rồi tiếp tục xử lý qua các bước phía sau. Với những thao tác đơn giản chỉ liên

quan đến một database, việc xử lý nghiệp vụ và lưu kết quả idempotency có thể được thực hiện ngay trong cùng một local transaction.

Tuy nhiên, đoạn code trên mới chỉ minh họa nguyên lý. Trong hệ thống production, mỗi idempotency key thường cần một bản ghi riêng, bao gồm `key`, hash của payload, trạng thái xử lý như `PROCESSING`, `SUCCEEDED` hoặc `FAILED`, kết quả/response đã lưu và thời điểm hết hạn của key. Chỉ lưu key là chưa đủ; để cơ chế idempotency thực sự an toàn khi có retry, concurrent request và sự cố giữa chừng, cần xử lý đúng năm vấn đề sau:

1. **Việc đăng ký key phải an toàn khi có nhiều request đến đồng thời.** Hai request mang cùng idempotency key có thể đến server gần như cùng lúc. Vì vậy, database phải có cơ chế đảm bảo chỉ một request được quyền tạo bản ghi đầu tiên, chẳng hạn unique constraint hoặc conditional write. Cách "kiểm tra key chưa tồn tại rồi mới ghi" bằng hai thao tác độc lập là chưa đủ, vì cả hai request có thể cùng kiểm tra trước khi request nào kịp ghi. Chương 9 sẽ minh họa nguyên tắc này bằng unique constraint và local transaction trong PostgreSQL trên RDS. Nếu request thứ hai gặp key đang ở trạng thái `PROCESSING`, nó cũng không thể giả định rằng kết quả đã có; tùy contract, server có thể chờ trong một khoảng thời gian giới hạn, trả `202 Accepted`, hoặc yêu cầu client kiểm tra lại sau.

2. **Caller và receiver đều phải tham gia vào idempotency contract.** Phía gọi có trách nhiệm tạo hoặc lưu idempotency key và sử dụng lại đúng key đó khi retry. Phía nhận có trách nhiệm ràng buộc key với request, kiểm tra payload và đảm bảo side effect không bị thực hiện lại. Chỉ một phía làm đúng là chưa đủ: caller không thể tự ngăn receiver thực hiện side effect hai lần, còn receiver cũng không thể biết hai request giống nhau có phải cùng một ý định hay không nếu caller không cung cấp một key ổn định.

3. **Cùng key nhưng khác payload phải bị từ chối.** Một idempotency key đại diện cho một ý định nghiệp vụ cụ thể, vì vậy không được sử dụng lại key đó cho một yêu cầu khác. Ví dụ, nếu `tripId` đã được dùng để thanh toán 2 triệu đồng, client không thể gửi lại cùng key nhưng đổi số tiền thành 3 triệu đồng. Lưu thêm hash của những trường quan trọng trong payload giúp server phát hiện trường hợp này thay vì vô tình trả lại kết quả của request cũ.

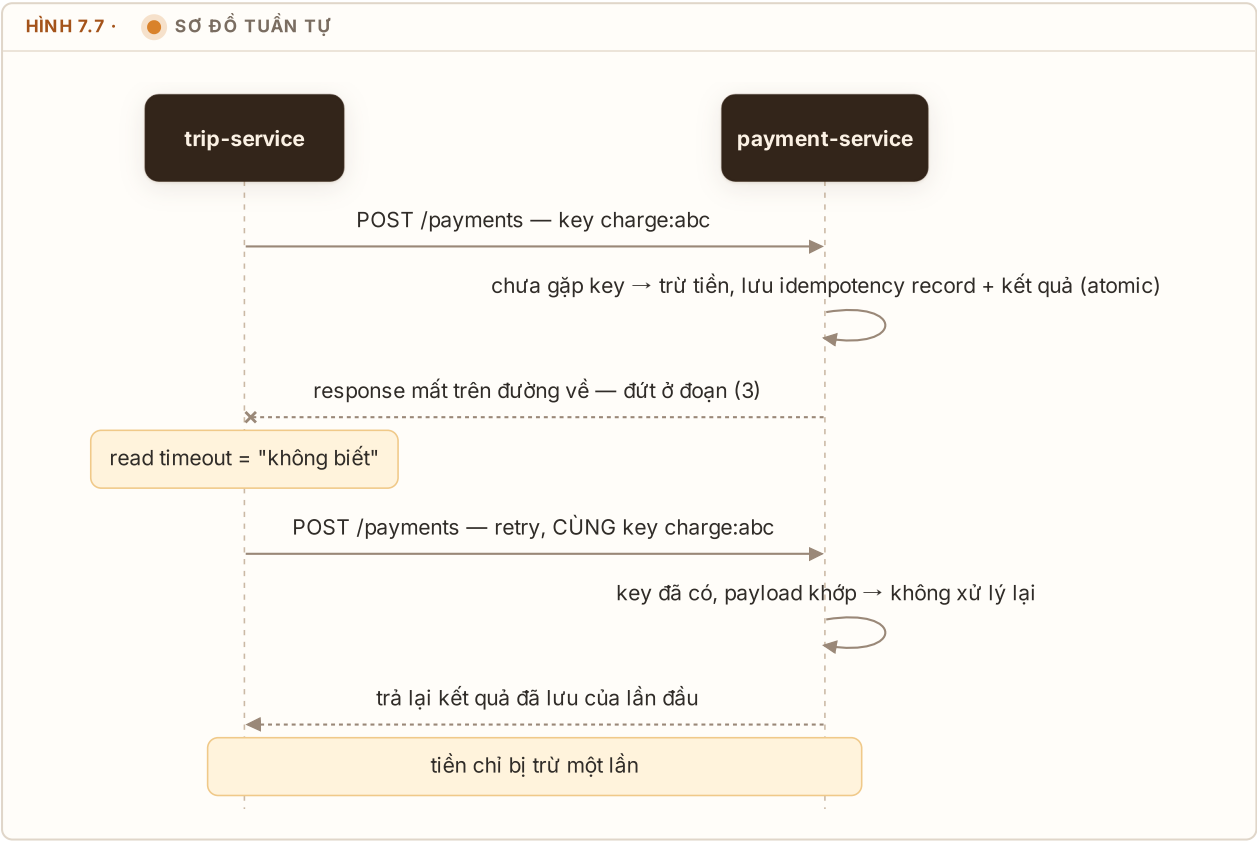
4. **Mỗi loại side effect cần có key riêng.** Một chuyến đi có thể tạo ra nhiều side effect khác nhau: giữ phòng, trừ tiền, hoàn tiền... Vì vậy, thay vì dùng cùng một UUID cho tất cả, TravelNow có thể tạo các key theo từng loại thao tác như `book-hotel:<tripId>`, `charge:<tripId>` và `refund:<tripId>`. Quan trọng hơn, idempotency key phải được truyền tới **service thực sự tạo ra side effect**. Nếu payment-service gọi tiếp một cổng thanh toán bên ngoài nhưng cổng đó không hỗ trợ cơ chế idempotency tương đương, payment-service vẫn có thể rơi vào trạng thái không biết giao dịch bên ngoài đã thành công hay chưa và cần thêm cơ chế tra cứu hoặc đối soát.

5. **Idempotency record cần có chính sách lưu giữ rõ ràng.** Nếu các idempotency key được lưu trong một bảng riêng, số lượng bản ghi sẽ tăng theo traffic và không thể giữ mãi mãi. Hệ thống cần xác định key có hiệu lực trong bao lâu dựa trên cửa sổ retry và đối soát của nghiệp vụ. TTL có thể được dùng để dọn các bản ghi cũ, nhưng cần phân biệt **thời điểm key hết**

hiệu lực với **thời điểm database thực sự xóa bản ghi**. Nếu thời gian lưu quá ngắn, một retry đến muộn có thể bị coi là request mới và tạo thêm side effect; nếu quá dài, chi phí lưu trữ và yêu cầu bảo vệ dữ liệu sẽ tăng lên. Vì vậy, thời hạn của idempotency key phải là một phần của contract của từng hệ thống, không nên lấy một con số từ dịch vụ khác rồi áp dụng máy móc.

Ghép timeout, retry và idempotency key vào đúng kịch bản khó nhất của mục 7.2 — đứt ở đoạn (3):

HÌNH 7.7 · SƠ ĐỒ TUẦN TỰ



7.5.4 Trả lời câu hỏi mở đầu

Đến đây, ta có thể quay lại câu hỏi ở đầu chương: **một lệnh trừ tiền bị timeout thì có thể gửi lại an toàn hay không?**

Câu trả lời là **có, nhưng chỉ khi cơ chế idempotency thực sự bảo vệ thao tác trừ tiền**: lần retry sử dụng cùng idempotency key và cùng payload, phía nhận đảm bảo side effect không bị thực hiện lại, kết quả xử lý được lưu nhất quán với bản ghi idempotency, và key vẫn còn hiệu lực.

Nếu không chắc những điều kiện này được đáp ứng, hệ thống không nên đơn giản gửi lại lệnh trừ tiền. Khi đó, lựa chọn an toàn hơn là **tra cứu trạng thái của giao dịch trước đó hoặc chuyển sang đối soát** để xác định điều gì đã thực sự xảy ra.

7.6 At-most-once, at-least-once, và "exactly-once" thực sự có nghĩa là gì?

Sau khi có retry và idempotency, một câu hỏi tự nhiên xuất hiện: **TravelNow đang đảm bảo request được xử lý một lần, nhiều lần hay chính xác một lần?** Để trả lời, cần phân biệt việc một request được **gửi bao nhiêu lần** với việc side effect của nó **có hiệu lực bao nhiêu lần**.

Với **at-most-once**, hệ thống gửi request một lần và không retry khi không nhận được kết quả. Cách này tránh nguy cơ gửi trùng, nhưng đổi lại một request có thể bị mất. Ví dụ, nếu lệnh thanh toán không tới được payment-service mà trip-service cũng không gửi lại, giao dịch sẽ không bao giờ được thực hiện.

Với **at-least-once**, hệ thống cho phép gửi lại khi chưa nhận được kết quả. Cách này làm tăng khả năng request cuối cùng được xử lý, nhưng đồng nghĩa receiver có thể nhận cùng một yêu cầu nhiều lần. Đây chính là mô hình mà retry đưa chúng ta tới. Vì vậy, với những thao tác có side effect như trừ tiền, **at-least-once phải đi cùng idempotency** để những lần gửi lại không tạo thêm side effect.

Còn **exactly-once** cần được hiểu cẩn thận. Trong hệ phân tán, không nên nói chung chung rằng toàn bộ request được "xử lý chính xác một lần" nếu không chỉ rõ phạm vi của đảm bảo đó. Một hệ thống có thể đảm bảo một message và thay đổi database được xử lý atomic trong một transaction cụ thể, hoặc đảm bảo cùng một idempotency key chỉ tạo side effect một lần trong thời hạn hiệu lực của key. Nhưng đảm bảo đó không tự động mở rộng sang những hệ thống bên ngoài transaction boundary.

Ví dụ, payment-service có thể bảo đảm rằng `charge:<tripId>` chỉ tạo một bản ghi thanh toán trong database của nó. Nhưng nếu service tiếp tục gọi sang một cổng thanh toán bên ngoài, việc tiền thực sự bị trừ bao nhiêu lần còn phụ thuộc vào contract mà cổng thanh toán cung cấp. Nếu phía ngoài không hỗ trợ idempotency tương đương, payment-service vẫn cần query hoặc đối soát khi kết quả không xác định.

Vì vậy, cách mô tả chính xác hơn cho TravelNow là:

At-least-once delivery + idempotent side effect → cùng một ý định có thể được gửi và xử lý nhiều lần, nhưng side effect nghiệp vụ chỉ có hiệu lực một lần trong phạm vi mà idempotency contract bảo vệ.

Đây không phải là lời hứa rằng mọi thứ trong toàn bộ hệ thống xảy ra "exactly once". Mỗi khi gặp một tuyên bố về exactly-once, hãy đặt ba câu hỏi: **thao tác nào được đảm bảo, trong transaction boundary nào, và các side effect bên ngoài boundary đó được xử lý ra sao?**

7.7 Tóm tắt

Một lời gọi tới service khác không phải lúc nào cũng kết thúc bằng hai trạng thái thành công hoặc thất bại. Có những trường hợp bên gọi **không biết kết quả thực sự là gì**. Đặc biệt, timeout chỉ cho biết bên gọi đã ngừng chờ response; nó không có nghĩa rằng bên nhận chưa xử lý request hay side effect chưa xảy ra.

Retry giúp hệ thống thử lại khi gặp những lỗi có tính tạm thời, nhưng phải được kiểm soát: chỉ retry những lỗi phù hợp, giới hạn số lần và thời gian, đồng thời tránh tạo thêm tải khi downstream đang gặp sự cố. Với các thao tác tạo side effect, **idempotency** giúp những lần gửi lại cùng một ý định không thực hiện nghiệp vụ thêm lần nữa. Và khi hệ thống vẫn không đủ thông tin để xác định điều gì đã xảy ra, **query trạng thái hoặc đối soát** cũng phải được xem là một phần của thiết kế phục hồi.

Timeout, retry, circuit breaker, bulkhead và idempotency giúp TravelNow kiểm soát sự cố ở **từng lời gọi và từng bước nghiệp vụ**. Nhưng chúng chưa giải quyết được toàn bộ một quy trình gồm nhiều bước. Chẳng hạn, nếu phòng và vé máy bay đã được giữ nhưng payment-service sau đó xác nhận thanh toán thất bại, hệ thống phải quyết định cách hoàn tác những gì đã thực hiện trước đó.

Đó là bài toán của **Saga** trong Chương 8: tổ chức một quy trình phân tán thành nhiều bước, lưu lại tiến độ của quy trình và thực hiện các thao tác bù trừ khi một bước không thể hoàn thành. Quan trọng hơn, Saga cũng phải tính đến trường hợp **chính thao tác bù trừ bị lỗi** và cần được retry, tiếp tục xử lý hoặc chuyển sang đối soát.

CHƯƠNG 8

Saga Pattern — Choreography và Orchestration

Phần III — Kiến trúc phân tán · *The Hard Parts*

Chương 7 tập trung vào độ tin cậy của **từng lời gọi giữa các service**: làm gì khi request bị timeout, khi kết quả chưa rõ, và làm thế nào để retry mà không tạo side effect trùng lặp. Nhưng một quy trình đặt chuyến gồm nhiều bước lại đặt ra bài toán lớn hơn. Nếu phòng và vé máy bay đã được giữ, sau đó payment-service xác nhận thanh toán bị từ chối, retry không còn giải quyết được vấn đề. Các bước trước đã thành công và đã commit; hệ thống phải xử lý những gì chúng đã để lại.

Đó là bài toán **distributed transaction** mà Saga giúp giải quyết. Thay vì sử dụng một atomic transaction bao trùm toàn bộ các service, Saga chia quy trình thành một chuỗi **local transaction**, mỗi bước commit độc lập trong phạm vi của service tương ứng. Nếu quy trình không thể tiếp tục, hệ thống thực hiện các hành động bù trừ cho những bước đã hoàn thành để đưa toàn bộ quy trình về một trạng thái nghiệp vụ hợp lệ. Tiến độ của Saga cũng phải được lưu bền vững để hệ thống có thể tiếp tục phục hồi khi service restart hoặc quá trình bù trừ bị gián đoạn. Nếu không thể tự động hoàn tất, hệ thống phải biết công việc nào còn dang dở để cảnh báo và chuyển sang đối soát hoặc xử lý thủ công.

Trong chương này, chúng ta sẽ xem xét cùng một nghiệp vụ đặt chuyến đi theo hai cách phối hợp Saga. Với **orchestration**, một thành phần điều phối nắm luồng xử lý và quyết định bước nào được thực hiện tiếp theo. Với **choreography**, các service phối hợp bằng cách phát và phản ứng với event, không có một orchestrator điều khiển toàn bộ chuỗi. Qua cùng một tình huống, chúng ta sẽ xem ai đưa ra quyết định, trạng thái tiến triển được lưu ở đâu và hệ thống tiếp tục công việc như thế nào khi một bước gặp sự cố. Sang Chương 9, những trách nhiệm này sẽ được ánh xạ sang Step Functions, Lambda và SNS/SQS trên AWS, với dữ liệu nghiệp vụ tiếp tục được lưu trong RDS.

8.1 Saga là gì

8.1.1 Ý tưởng cốt lõi

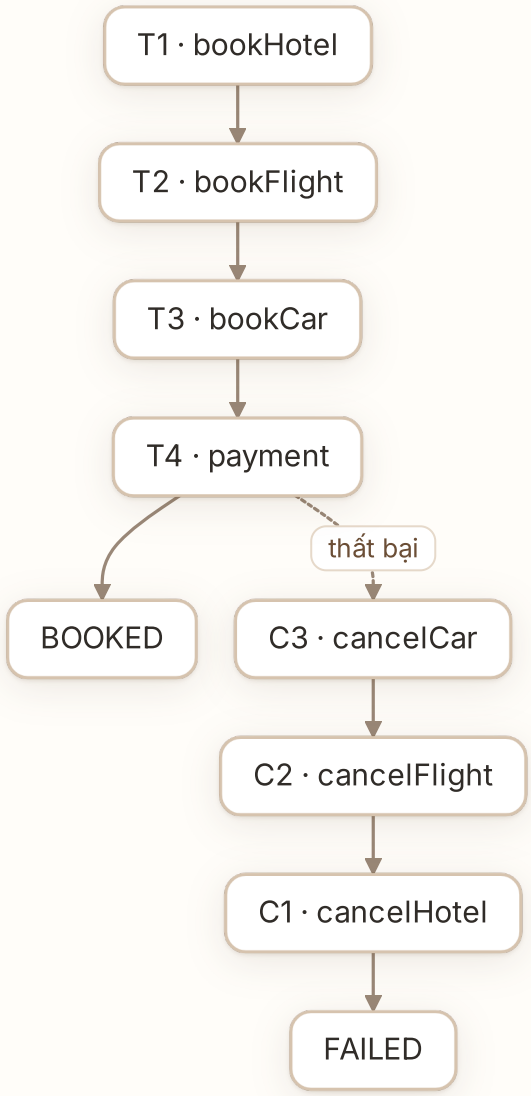
Saga không phải là một khái niệm ra đời cùng microservices. Nó đã xuất hiện từ năm 1987 trong một bài báo của Hector Garcia-Molina và Kenneth Salem, ban đầu nhằm giải quyết bài toán **long-lived transaction** trong database: làm thế nào xử lý những transaction kéo dài mà không phải giữ tài nguyên và khóa trong suốt quá trình thực hiện.

Đặt ý tưởng đó vào bài toán distributed transaction của chúng ta, có thể hiểu Saga như sau:

> Thay vì đặt toàn bộ quy trình nghiệp vụ trong **một transaction lớn**, hãy chia nó thành một **chuỗi local transaction** T_1, T_2, ... T_n. Mỗi transaction thực hiện một phần của quy trình và commit độc lập trong phạm vi của service tương ứng. Với mỗi bước T_i cần khả năng bù trừ, ta định nghĩa một **compensating transaction** C_i — một transaction mới nhằm hoàn tác **về mặt nghiệp vụ** những gì T_i đã thực hiện. Nếu bước T_k thất bại và quy trình không thể tiếp tục, hệ thống có thể thực hiện các transaction bù trừ cho những bước đã hoàn thành trước đó, thường theo thứ tự ngược: C_{k-1}, C_{k-2}, ... C_1. Nếu kết quả của một bước vẫn chưa rõ, hệ thống phải xác định điều gì thực sự đã xảy ra trước khi quyết định tiếp tục tiến về phía trước hay bắt đầu bù trừ.

Áp ý tưởng này vào TravelNow:

HÌNH 8.1 · SƠ ĐỒ LUỒNG



Chuỗi đi xuôi là các bước thực hiện nghiệp vụ; chuỗi đi ngược là các bước bù trừ. Khi một bước thất bại, hệ thống đã có sẵn một **đường phục hồi** thay vì đến lúc có lỗi mới ứng biến bằng các khối `catch`.

Tuy nhiên, bù trừ không phải lúc nào cũng hoàn tất ngay. Nếu một dependency tiếp tục gặp sự cố, Saga có thể nằm ở trạng thái dở dang trong một khoảng thời gian dài. Vì vậy, bên cạnh happy path và luồng bù trừ tự động, Saga còn cần **deadline**, trạng thái **cần can thiệp** và cơ chế **reconciliation** để xử lý những trường hợp hệ thống không thể tự phục hồi.

8.1.2 Semantic rollback — bù trừ thay cho xóa bỏ lịch sử

Có một khác biệt quan trọng giữa **compensating transaction C_i** và rollback của database: **rollback cố làm cho thay đổi chưa từng được commit, còn bù trừ xử lý một thay đổi đã thực sự xảy ra**.

Ở Chương 1, nếu quá trình đặt chuyến thất bại, local transaction có thể rollback toàn bộ thay đổi trước khi chúng được commit. Với Saga thì khác. Mỗi bước là một local transaction và có thể đã commit từ trước. Khi cần phục hồi, hệ thống không thể quay ngược thời gian để xóa những transaction đó; thay vào đó, nó thực hiện **một transaction mới để bù trừ về mặt nghiệp vụ**.

Ví dụ, `HotelBooking` đã được commit với trạng thái `BOOKED`. Khi Saga cần bù trừ, hotel-service không xóa booking này mà thực hiện một thao tác mới để chuyển nó sang `CANCELLED`. Nếu service phát event, việc hủy cũng có thể tạo thêm một event tương ứng. Lịch sử lúc này phản ánh đúng những gì đã thực sự xảy ra: **phòng đã được đặt, sau đó bị hủy**.

Đó là ý nghĩa của **semantic rollback**: hoàn tác về mặt **nghiệp vụ**, chứ không rollback dữ liệu như một database transaction.

Sự khác biệt này cũng làm thay đổi cách TravelNow lưu trạng thái. Ở Chương 1, một lần đặt chuyến thất bại giữa chừng có thể biến mất cùng với local transaction bị rollback. Với Saga, Trip cần lưu lại tiến trình đã thực hiện, bước nào đã thành công và kết quả của các bước bù trừ.

Quan trọng hơn, lịch sử ấy đôi khi còn hiện diện bên ngoài hệ thống. Khách sạn có thể đã kịp gửi email xác nhận trước khi nhận được lệnh hủy. Một khoản thanh toán có thể đã xuất hiện trên tài khoản của khách hàng rồi được hoàn lại vài phút sau, để lại hai giao dịch ngược chiều trên sao kê. **Hệ phân tán không thể xóa những gì đã xảy ra; nó chỉ có thể thực hiện thêm những hành động để đưa nghiệp vụ về một trạng thái hợp lệ.** Vì vậy, cả hệ thống lẫn trải nghiệm người dùng đều phải được thiết kế với thực tế này.

8.1.3 Phân loại thất bại trước khi quyết định retry hay bù trừ

Một Saga không nên coi mọi lỗi đều giống nhau. Trước khi quyết định retry hay chuyển sang bù trừ, hệ thống cần xác định **loại thất bại mà nó đang đối mặt**.

Nếu thẻ bị từ chối, đó là một **business failure**: kết quả đã rõ và việc gửi lại cùng một yêu cầu thường không làm thay đổi kết quả. Saga có thể coi bước thanh toán là thất bại và bắt đầu bù trừ những bước đã hoàn thành trước đó.

Ngược lại, timeout hoặc HTTP `503` thường là **technical failure**. Khi đó, hệ thống trước hết sử dụng các cơ chế đã xây dựng ở Chương 7: timeout budget, retry có giới hạn và idempotency. Nếu vẫn còn trong deadline, Saga có thể tiếp tục chờ hoặc retry theo policy đã định.

Vấn đề khó hơn xuất hiện khi đã hết deadline hoặc cạn retry policy mà kết quả của side effect vẫn chưa rõ. Saga không được tự suy diễn rằng thao tác đã thất bại, bởi phía bên kia có thể đã thực hiện nó thành công. Thay vào đó, quy trình cần chuyển sang một trạng thái như `SUSPENDED` hoặc `MANUAL_REVIEW` để tiếp tục tra cứu, đối soát hoặc chờ can thiệp.

Theo nghĩa đó, Saga không thay thế timeout, retry hay idempotency. **Saga phối hợp các cơ chế tin cậy ở tầng dưới, lưu lại kết quả của chúng và quyết định quy trình nghiệp vụ phải đi theo hướng nào tiếp theo.** Khi các cơ chế tự động không thể đưa ra một kết quả đủ chắc chắn, Saga cũng phải có một đường thoát rõ ràng thay vì để quy trình mắc kẹt hoặc tự kết luận sai.

8.1.4 Ba điều kiện để Saga đứng vững

Saga chỉ hoạt động đáng tin cậy khi từng bước của nó tuân thủ một số nguyên tắc chặt chẽ. Và phần lớn những nguyên tắc này thực ra là sự tiếp nối của những gì chúng ta đã xây dựng ở Chương 7.

1. **Mỗi bước tiến T_i phải idempotent.** Một bước như `bookHotel` có thể được thực hiện lại do retry hoặc message bị giao lại theo cơ chế at-least-once. Service nhận lệnh phải nhận ra rằng cùng một ý định, chẳng hạn cùng `tripId` và operation tương ứng, đã được xử lý và không tạo thêm một booking mới.
2. **Mỗi bước bù trừ C_i cũng phải idempotent.** Bù trừ cũng là một thao tác phân tán nên bản thân nó có thể timeout, được retry hoặc nhận cùng một message nhiều lần. Nếu `cancelHotel` được gọi lại cho một booking đã `CANCELLED`, kết quả vẫn phải là `CANCELLED` mà không tạo thêm side effect.
3. **Bù trừ phải có khả năng retry và có đường xử lý khi không thể tự hoàn tất.** Các bước bù trừ nên được thiết kế càng đơn giản càng tốt, có tính idempotent và hạn chế những business precondition có thể khiến chúng thất bại. Khi một bước bù trừ gặp technical failure, hệ thống retry theo policy đã định và lưu lại tiến độ để có thể tiếp tục sau khi restart. Nếu đã quá deadline hoặc gặp lỗi không thể tự phục hồi, Saga phải ghi nhận phần việc bù trừ còn tồn đọng, phát cảnh báo và chuyển sang reconciliation hoặc xử lý thủ công.

Điều quan trọng là **"retry mãi cho đến khi thành công" không phải một chiến lược vận hành**. Một booking chưa được hủy vẫn có thể đang giữ phòng, ghế hoặc những tài nguyên có giá trị thực; retry vô hạn vừa tạo thêm chi phí vừa có nguy cơ che giấu một vấn đề cần con người can thiệp.

Ngoài ra, không phải mọi side effect đều có thể bù trừ một cách hoàn hảo. Vé không hoàn, phí hủy hay một khoản thanh toán đang tranh chấp đều có thể cần những quy tắc xử lý riêng. Trong TravelNow, chúng ta sử dụng các quy ước đơn giản để tập trung vào cách Saga phối hợp quy trình; trong hệ thống thực tế, **khả năng và điều kiện bù trừ của từng bước phải được xác định rõ ở cấp nghiệp vụ trước khi tự động hóa chúng**.

8.1.5 Thứ tự các bước là một quyết định thiết kế

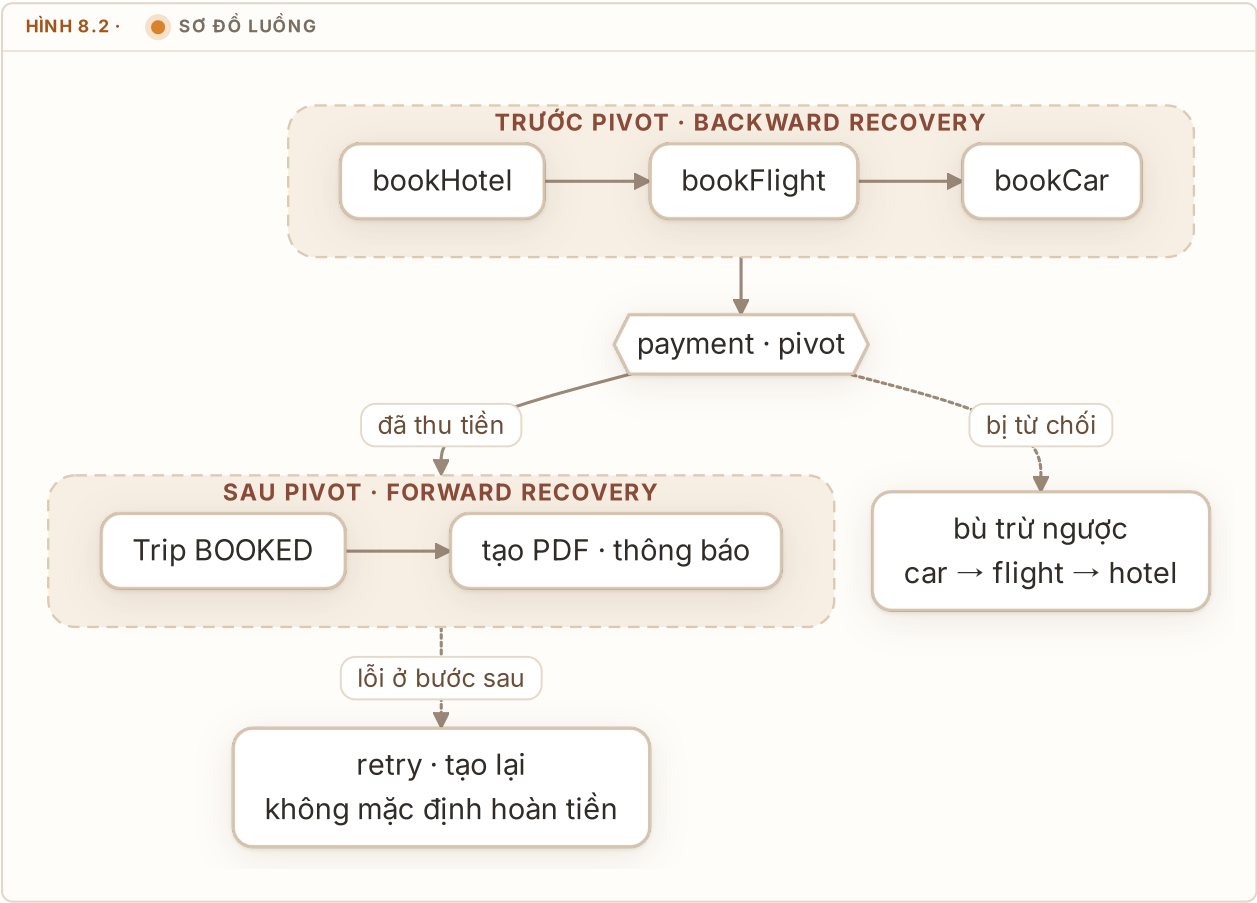
Tại sao payment lại nằm **cuối** chuỗi? Một nguyên tắc hữu ích khi thiết kế Saga là: những side effect càng khó hoặc càng tốn kém để bù trừ thì càng nên được thực hiện muộn, nếu nghiệp vụ cho phép. Hủy một lượt giữ phòng thường đơn giản hơn hoàn lại một khoản tiền đã thu. Vì vậy, TravelNow chỉ thực hiện payment sau khi các bước đặt phòng, vé máy bay và xe đã hoàn tất.

Trong thiết kế này, payment đóng vai trò **pivot transaction** — một điểm chuyển quan trọng của Saga. **Trước pivot**, nếu một bước thất bại, hệ thống vẫn có thể chọn **backward recovery**: bù trừ những bước đã hoàn thành để đưa quy trình về một trạng thái nghiệp vụ hợp lệ khác. **Sau khi pivot đã commit**, hệ thống ưu tiên **forward recovery**: tiếp tục hoàn thành những công việc còn lại thay vì quay ngược và bù trừ toàn bộ các bước trước đó.

Điều đó không có nghĩa là sau payment sẽ không còn lỗi. TravelNow vẫn có thể gặp sự cố khi cập nhật trạng thái Trip, phát event hoặc tạo PDF xác nhận. Nhưng những lỗi này không nên khiến hệ thống thu tiền lại hay mặc định hủy toàn bộ booking. Các bước sau pivot cần được thiết kế để có thể tiếp tục phục hồi: retry theo policy, lưu lại tiến độ, cảnh báo khi cần và cuối cùng chuyển sang xử lý thủ công nếu hệ thống không thể tự hoàn tất. Chẳng hạn, nếu việc tạo PDF thất bại sau khi payment đã thành công, hướng xử lý hợp lý là tạo lại PDF hoặc yêu cầu hỗ trợ thủ công, chứ không mặc định hủy chuyến và hoàn tiền.

Một Saga thực tế vì thế có thể sử dụng cả hai hướng phục hồi: **backward recovery trước pivot và forward recovery sau pivot**. Pivot chính là ranh giới giúp workflow xác định nên quay lại bằng bù trừ hay tiếp tục tiến về phía trước.

HÌNH 8.2 · SƠ ĐỒ LUỒNG



Ngoài ra, khi sắp xếp các bước, có thể áp dụng một nguyên tắc khác: **Nếu nghiệp vụ cho phép, những bước kiểm tra đơn giản và dễ phát hiện thất bại nên được thực hiện trước các side effect tốn kém hoặc khó bù trừ**. Fail sớm giúp giảm số thao tác phải bù trừ về sau. Tuy nhiên, không nên đổi thứ tự chỉ để tối ưu Saga nếu điều đó làm sai semantics nghiệp vụ hoặc tạo ra khoảng trống giữa lúc kiểm tra và lúc sử dụng dữ liệu. Ví dụ, nếu TravelNow có bước kiểm tra hạn mức chỉ nhằm xác nhận đầu vào, bước này có thể được thực hiện sớm; nhưng nếu hạn mức có thể thay đổi đồng thời, việc kiểm tra sớm không thể thay thế cho cơ chế giữ hoặc xác nhận hạn mức tại thời điểm thực sự sử dụng.

8.1.6 Trạng thái của Trip trong Saga

Khi quy trình được điều phối bằng Saga, `Trip` cần thêm một số trạng thái để phản ánh **quy trình đang ở bước nào và việc phục hồi đã hoàn tất hay chưa**:

Xem Hình 8.3 (sơ đồ khổ ngang) ở trang 125.

`COMPENSATING` cho biết quy trình đang ở trạng thái chưa nhất quán và hệ thống đang thực hiện các bước bù trừ để tự phục hồi. Nếu quá deadline hoặc xuất hiện vấn đề không thể xử lý tự động, Trip được chuyển sang `MANUAL_REVIEW` để ghi nhận rằng cần đối soát hoặc can thiệp thủ công.

`FAILED` chỉ được sử dụng khi quy trình đặt chuyến đã thất bại **và tất cả các bước bù trừ cần thiết đã hoàn tất**. Trong khi đó, `CANCELLED` mang ý nghĩa khác: chuyến đi trước đó đã được đặt thành công nhưng sau đó được hủy theo yêu cầu của người dùng.

Việc biểu diễn rõ những trạng thái trung gian này, theo dõi chúng tồn tại bao lâu và phát cảnh báo khi chúng kéo dài quá mức giúp **eventual consistency trở thành một cơ chế có thể vận hành và giám sát**, thay vì chỉ hy vọng rằng hệ thống rồi sẽ tự trở về trạng thái nhất quán.

8.1.7 Saga phục hồi lỗi, nhưng không tự tạo isolation

Saga chia một transaction lớn thành nhiều local transaction độc lập. Đổi lại, toàn bộ quy trình không còn được bảo vệ bởi **isolation** như khi mọi thao tác nằm trong cùng một database transaction. Vì vậy, ngay cả khi từng bước đều đúng và idempotent, hai workflow hợp lệ vẫn có thể chạy đồng thời và xung đột với nhau.

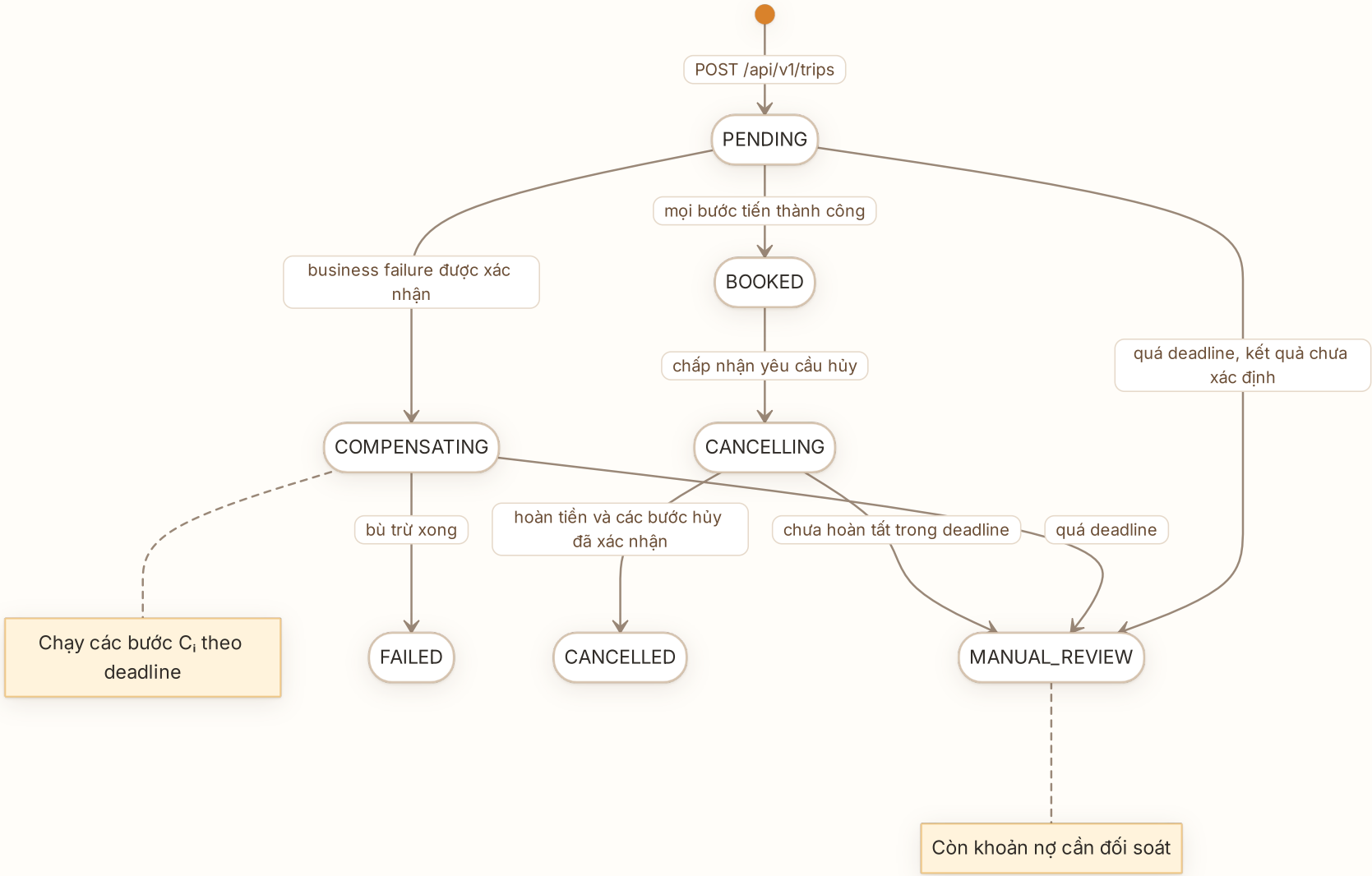
Ví dụ, người dùng gửi yêu cầu hủy trong khi payment đang ở trạng thái chưa rõ kết quả. Vài giây sau, một `PaymentSucceeded` đến muộn. Nếu mỗi handler chỉ cập nhật trạng thái theo kiểu **last-write-wins**, event đến sau có thể đưa một Trip đang được hủy trở lại `BOOKED`. Ngược lại, luồng hủy cũng có thể kết thúc trước khi hệ thống nhận ra rằng payment vừa thành công và khoản tiền đó vẫn cần được hoàn lại.

Vì vậy, TravelNow không chỉ bảo vệ từng message bằng idempotency mà còn phải bảo vệ **máy trạng thái của Trip**. Mỗi lần chuyển trạng thái được thực hiện bằng conditional update: trạng thái hiện tại phải thuộc tập trạng thái cho phép và `version` phải đúng với giá trị mà handler đã đọc. Command hoặc event cũng mang `operationId` để nhận diện thao tác tương ứng. Nếu phát hiện `version` đã thay đổi, handler phải đọc lại trạng thái mới nhất và quyết định cách xử lý theo transition policy hiện tại, thay vì ghi đè kết quả một cách mù quáng.

Các command có thể được tuần tự hóa theo `tripId` để giảm khả năng cạnh tranh, nhưng đó không phải lớp bảo vệ duy nhất. Retry, event đến muộn hoặc các nguồn sự kiện khác vẫn có thể xuất hiện, nên conditional update trên trạng thái vẫn đóng vai trò **hàng rào cuối cùng** chống lại những transition không còn hợp lệ.

Trong phạm vi chính của eBook, chúng ta giữ quy ước từ Chương 1: người dùng chỉ hủy một chuyến đã ở trạng thái `BOOKED`. Khi đó, Trip chuyển sang `CANCELLING` và lưu tiến độ của chuỗi xử lý: hoàn tiền → hủy xe → hủy khách sạn → hủy chuyến bay.

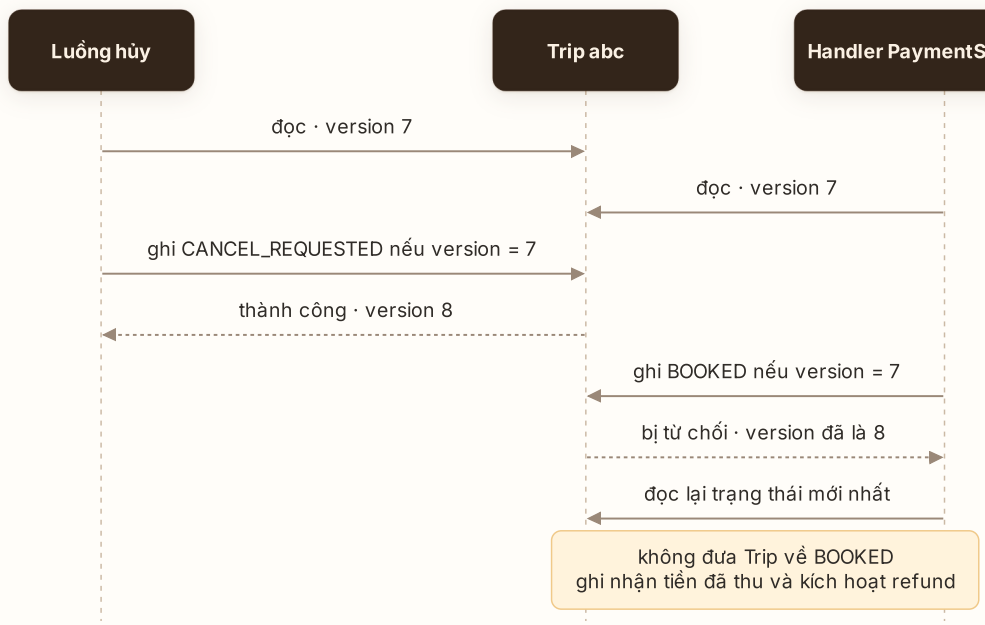
HÌNH 8.3 · MÁY TRẠNG THÁI



Nếu sau này TravelNow cho phép người dùng yêu cầu hủy ngay cả khi quá trình đặt chuyến chưa hoàn tất, máy trạng thái cần thêm `CANCEL_REQUESTED`. Trạng thái này phải được lưu bền vững trước khi thực hiện các side effect tiếp theo. Nếu `PaymentSucceeded` đến muộn sau đó, handler không được đưa Trip trở lại `BOOKED`; thay vào đó, hệ thống ghi nhận khoản tiền đã được thu rồi kích hoạt `refund` cùng những bước hủy còn thiếu. Tương tự, các lượt giữ phòng hoặc vé cũng cần xác định rõ chủ sở hữu và thời hạn để một Saga cũ không giữ tài nguyên vô thời hạn.

Với trạng thái mở rộng ấy, conditional update chặn `PaymentSucceeded` đến muộn như sau:

HÌNH 8.4 · SƠ ĐỒ TUẦN TỰ



Điểm cần phân biệt ở đây là: **idempotency xử lý việc cùng một ý định bị thực hiện nhiều lần; còn version và transition policy xử lý nhiều ý định hợp lệ nhưng cạnh tranh với nhau trên cùng một trạng thái**. Saga cần cả hai để quy trình vẫn đúng khi có retry, concurrency và event đến không đúng thứ tự.

8.2 Bảng bù trừ của TravelNow

Trước khi chọn trường phái triển khai, phần "nghiệp vụ thuần" của Saga phải được chốt — và phần này **giống hệt nhau** ở cả hai trường phái:

Bước tiến (T_i)	Service	Bù trừ (C_i)	Ghi chú
T_1 `bookHotel`	hotel-service	C_1 `cancelHotel`	`BOOKED → CANCELLED`; hủy cái đã hủy = OK
T_2 `bookFlight`	flight-service	C_2 `cancelFlight`	như trên

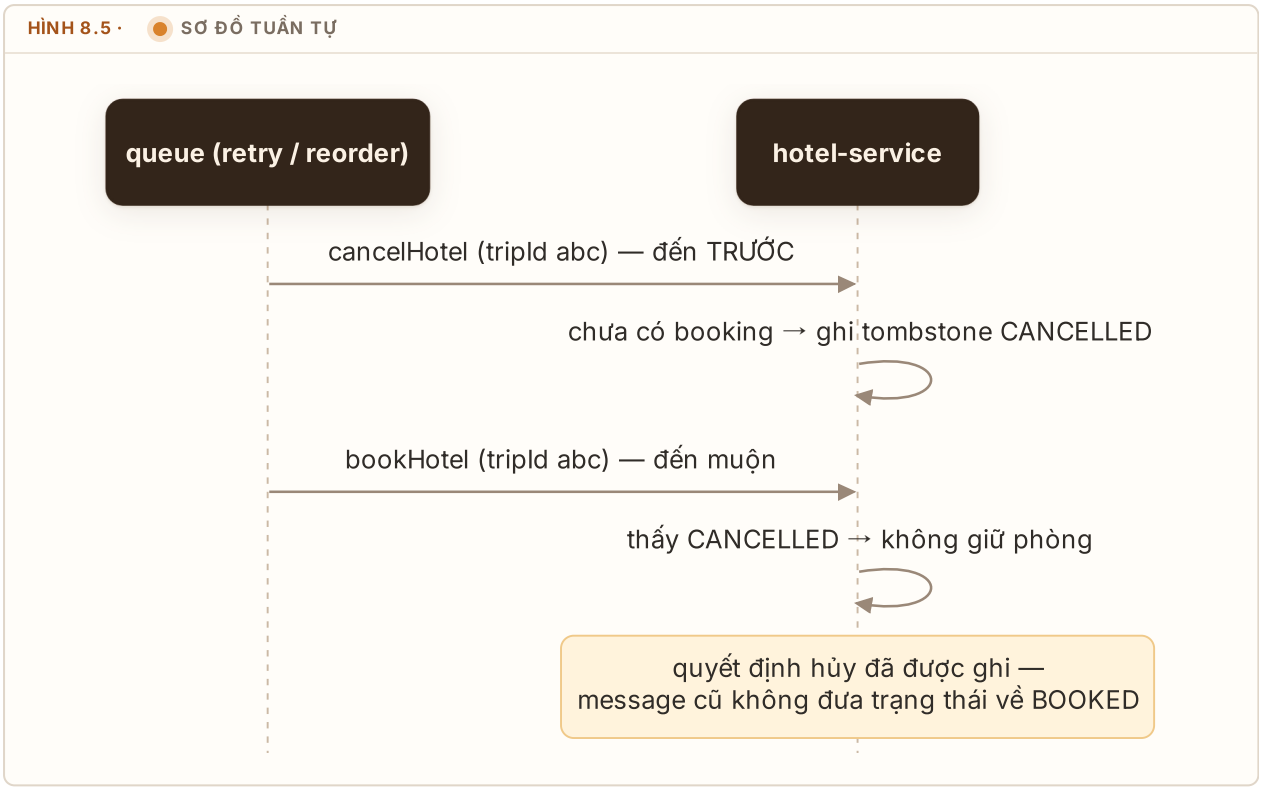
Bước tiến (T_i)	Service	Bù trừ (C_i)	Ghi chú
T_3 `bookCar` *(tùy chọn)*	car-service	C_3 `cancelCar`	bỏ qua cả T_3 lẫn C_3 nếu trip không thuê xe
T_4 `payment` — **pivot**	payment-service	C_4 `refund`	chỉ dùng cho luồng *user chủ động hủy* sau khi BOOKED

Mỗi C_i là một thao tác trên service tương ứng, và cả bốn đều tuân thủ ba điều kiện ở mục 8.1: idempotent, có quy tắc bù trừ rõ ràng và có đường xử lý khi chưa hoàn tất. Trong giả lập, hủy một bản ghi không bị từ chối vì điều kiện thương mại; lỗi database và lỗi mạng vẫn có thể xảy ra. Với hotel-service, quy tắc có thể diễn đạt bằng bảng:

Trạng thái đang lưu	Khi nhận `cancelHotel`	Nếu `bookHotel` đến sau đó với cùng ý định đặt chuyến
`BOOKED`	Chuyển sang `CANCELLED`	Giữ nguyên kết quả hủy.
`CANCELLED`	Trả lại kết quả đã hủy	Giữ nguyên kết quả hủy.
Chưa có bản ghi	Lưu dấu hủy `CANCELLED` cho định danh đó	Không tạo booking mới từ lệnh đến muộn.

Dấu hủy ấy gọi là **tombstone**. Nếu `cancel` đến trước `book` mà ta chỉ trả OK khi "không tìm thấy", một lệnh `book` đến muộn vẫn có thể giữ phòng sau khi saga đã hủy. Bản ghi `CANCELLED` giúp phía nhận nhớ quyết định đã có. Cả `book` và `cancel` phải được bảo vệ bằng ràng buộc duy nhất, khóa hoặc ghi có điều kiện; bảng quy tắc chỉ có ý nghĩa khi hai request đồng thời không thể vượt qua nó. **Sau khi quyết định hủy đã được ghi, message cũ không được làm trạng thái quay lại `BOOKED`.**

Kịch bản reorder mà tombstone chặn:



Phần còn lại của chương trả lời câu hỏi duy nhất còn lại, cũng là câu hỏi thú vị nhất: **ai là người quyết định gọi T_i nào, C_i nào, vào lúc nào?** Hai cách triển khai đặt trách nhiệm điều phối ở hai nơi khác nhau.

8.3 Saga Orchestration — có orchestrator trung tâm

8.3.1 Ý tưởng

Cách tiếp cận thứ nhất là **orchestration**: một thành phần trung tâm, gọi là **orchestrator**, chịu trách nhiệm điều phối toàn bộ quy trình. Nó biết Saga gồm những bước nào, chúng được thực hiện theo thứ tự nào, và nếu một bước thất bại thì cần kích hoạt những bước bù trừ nào.

Orchestrator nắm **logic của luồng**, nhưng không thay thế business logic của các domain service. Nó có thể yêu cầu hotel-service *"đặt phòng cho trip abc"*, còn hotel-service vẫn tự quyết định yêu cầu đó có hợp lệ hay không và trả về kết quả thành công hoặc thất bại. Nói cách khác, orchestrator quyết định **bước nào cần được thực hiện tiếp theo**, còn mỗi service vẫn làm chủ **cách bước đó được thực hiện trong domain của mình**.

Thực ra, code ở Chương 6 đã mang hình dáng sơ khai của mô hình này: trip-service lần lượt gọi hotel-service, flight-service, car-service rồi payment-service. Theo nghĩa đó, trip-service đã đóng vai trò một orchestrator đơn giản. Vấn đề không nằm ở cách tổ chức luồng, mà ở **độ tin cậy của việc điều phối**. Nếu toàn bộ tiến trình chỉ tồn tại trong các câu lệnh `try-catch` và bộ nhớ của process, orchestrator có thể quên mình đã đi đến đâu khi process chết giữa chừng.

Vì vậy, một orchestrator đáng tin cậy không thể chỉ là một đoạn code gọi tuần tự các service. **Nó phải hoạt động như một durable state machine: trạng thái và tiến độ của workflow được lưu bền vững để quá trình có thể tiếp tục sau sự cố.** Cụ thể, orchestrator cần bốn năng lực:

1. **Lưu trạng thái bền vững** — trạng thái và tiến độ của Saga không chỉ tồn tại trong memory của process mà được lưu vào persistent storage. Nếu orchestrator crash hoặc restart, một process khác vẫn có thể đọc lại trạng thái và tiếp tục từ nơi quy trình đang dở.
2. **Ghi nhận bước tiếp theo trước khi thực hiện** — trước khi gọi một side effect, orchestrator lưu và commit trạng thái cho biết bước nào sắp được thực hiện. Nhờ đó, nếu process chết giữa chừng, hệ thống vẫn biết Saga đang chờ xử lý bước nào thay vì mất dấu tiến trình.
3. **Retry và deadline theo policy** — mỗi bước có quy tắc rõ ràng về thời gian chờ, số lần retry và cách xử lý khi không thể tiếp tục. Một bước bị kẹt phải được phát hiện để retry, chuyển sang bù trừ hoặc đưa Saga vào `MANUAL_REVIEW`, ngay cả khi process ban đầu không còn tồn tại.
4. **Theo dõi được từng execution** — trạng thái của mỗi Saga phải có thể được truy vấn trực tiếp. Câu hỏi *"Saga của trip abc đang ở bước nào, bước nào đã hoàn thành và đang chờ điều gì?"* nên được trả lời bằng một câu query, thay vì phải lần theo hàng loạt log để dựng lại toàn bộ diễn biến.

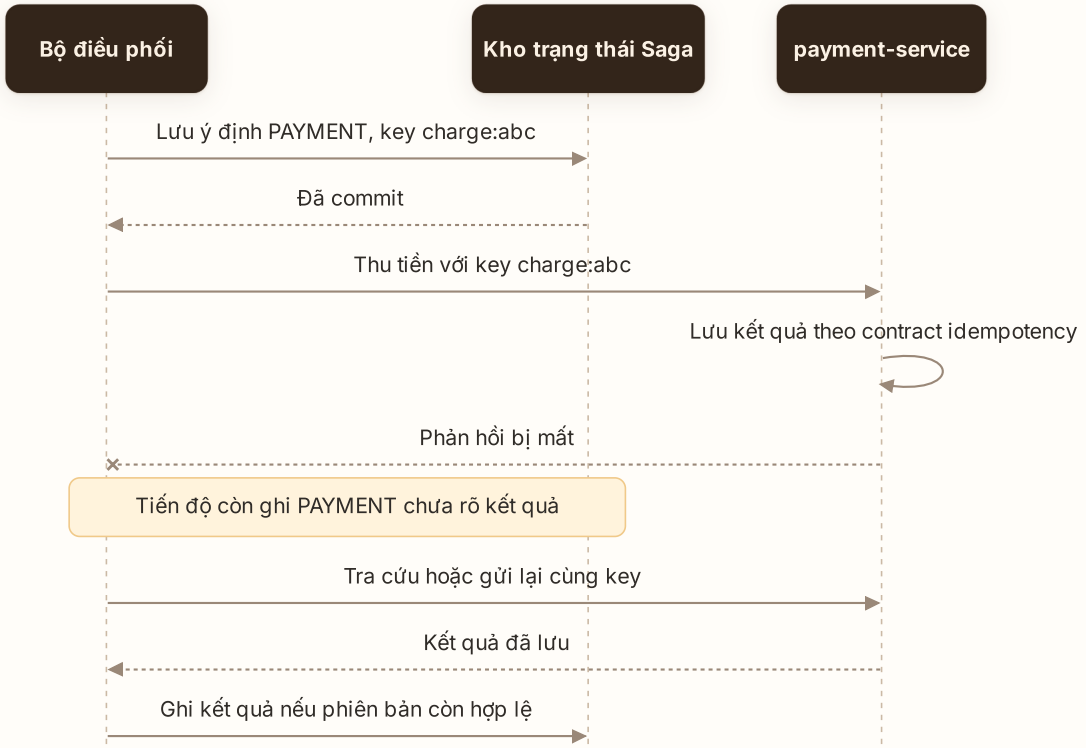
8.3.2 Một orchestrator cần nhớ những gì?

Có thể hình dung orchestrator duy trì một **sổ tiến độ bền vững** cho mỗi Saga. Với mỗi chuyến đi, sổ này ghi lại những thông tin cần thiết để biết quy trình đã đi đến đâu và cần làm gì tiếp theo:

Thông tin	Ví dụ	Dùng để quyết định điều gì?
Định danh chuyến và thao tác	`tripId = abc`, `charge:abc`	Lần thử lại có còn là cùng một ý định?
Bước hiện tại và kết quả đã biết	Đang chờ kết quả payment	Cần gọi tiếp, tra cứu hay bù trừ?
Hướng phục hồi	Tiến tới hoặc bù trừ	Bước tiếp theo thuộc luồng nào?
Số lần thử và hạn xử lý	Còn một lượt thử trước deadline	Tiếp tục tự động hay chuyển đối soát?
Phiên bản trạng thái	`version = 7`	Worker này còn được phép ghi kết quả không?

Mỗi bước của orchestrator có thể hình dung theo ba giai đoạn: **ghi ý định → gọi service → ghi kết quả**. Trước tiên, orchestrator lưu lại bước sắp thực hiện cùng một định danh ổn định trong một transaction ngắn. Sau khi transaction đã commit, nó mới thực hiện lời gọi đến service tương ứng. Khi nhận được kết quả, orchestrator cập nhật lại trạng thái của Saga, nhưng chỉ khi `version` và lượt xử lý vẫn còn hợp lệ. Nhờ đó, một worker cũ hoặc một request đến muộn không thể ghi đè lên tiến độ mới hơn của Saga.

HÌNH 8.6 · SƠ ĐỒ TUẦN TỰ



Nếu process điều phối dừng ở bất kỳ thời điểm nào, một worker khác có thể đọc lại **trạng thái Saga đã được lưu** và tiếp tục xử lý. Việc process cũ dừng không cho biết side effect trước đó đã thành công hay thất bại. Nếu kết quả của một bước vẫn chưa rõ, worker mới phải tra cứu trạng thái hoặc gửi lại yêu cầu một cách an toàn theo cơ chế idempotency. Các bước bù trừ cũng được lưu và tiếp tục theo cùng nguyên tắc.

Đó chính là ý nghĩa của **durable**: khả năng tiếp tục workflow dựa trên trạng thái được lưu bền vững, thay vì phụ thuộc vào memory của một process cụ thể.

Tuy nhiên, chỉ lưu trạng thái là chưa đủ. Hệ thống còn cần một **cơ chế phục hồi** để phát hiện những Saga hoặc bước xử lý đã quá hạn, thực hiện retry theo policy và chuyển sang `MANUAL_REVIEW` khi không thể tự động xác định hoặc hoàn tất bước tiếp theo. Nếu thiếu cơ chế này, kho trạng thái Saga chỉ cho biết công việc đang dở ở đâu, nhưng không khiến công việc tự tiếp tục.

Vì vậy, cần phân biệt hai trách nhiệm: **persistent storage lưu trạng thái của Saga, còn cơ chế phục hồi chịu trách nhiệm đánh thức và tiếp tục những công việc đang dở**. Kết hợp cả hai mới tạo thành một orchestrator thực sự durable.

8.3.3 Trách nhiệm khi tự xây orchestrator và lựa chọn công cụ điều phối

TravelNow hoàn toàn có thể tự xây orchestrator bằng cách lưu trạng thái Saga trong database và sử dụng worker hoặc scheduler để tiếp tục các bước đang dở. Tuy nhiên, khi đó đội ngũ không chỉ phải viết logic của workflow mà còn phải tự giải quyết hàng loạt vấn đề về độ tin cậy: nhiều

worker cùng xử lý một Saga, một bước bị thực hiện lại, quản lý retry và deadline, lưu lịch sử execution, phát cảnh báo và khôi phục workflow sau sự cố.

Nói cách khác, **tự xây orchestrator đồng nghĩa với việc tự xây cả cơ chế thực thi và phục hồi workflow**, chứ không đơn thuần là viết code gọi các service theo đúng thứ tự.

Các công cụ như Temporal, Camunda hay AWS Step Functions cung cấp những cách khác nhau để đảm nhận một phần hoặc phần lớn những trách nhiệm này. Khi lựa chọn, trước hết cần xem chúng đáp ứng bốn năng lực của durable orchestrator ở trên như thế nào, sau đó mới xét đến các yếu tố như giới hạn của nền tảng, mô hình chi phí, khả năng quan sát và cách xử lý khi định nghĩa workflow thay đổi trong lúc vẫn còn những execution đang chạy.

Ở Chương 9, chúng ta sẽ sử dụng AWS Step Functions để xem cụ thể **những trách nhiệm nào có thể giao cho một dịch vụ được quản lý và những trách nhiệm nào vẫn thuộc về ứng dụng**.

8.3.4 Những đánh đổi khi tập trung logic điều phối

Dù được tự xây hay triển khai bằng một workflow engine, orchestration vẫn có một số đánh đổi xuất phát từ chính việc **đưa logic điều phối về một nơi**.

- **Coupling về luồng:** orchestrator phải biết các service tham gia, contract của chúng, thứ tự thực hiện và cách diễn giải kết quả của từng bước. Khi thêm, bỏ hoặc thay đổi một bước trong quy trình, kịch bản của orchestrator thường cũng phải thay đổi. Việc tập trung điều phối giúp theo dõi toàn bộ workflow, đồng thời tạo ra sự phụ thuộc giữa kịch bản điều phối và contract của các service.
- **Nguy cơ kéo business logic về orchestrator:** ranh giới trách nhiệm rất dễ bị mờ nếu orchestrator bắt đầu quyết định giá phòng, điều kiện hoàn tiền hay cách giữ vé. Vai trò của orchestrator nên giới hạn ở việc quyết định **bước nào cần chạy tiếp và xử lý kết quả của bước đó như thế nào**; các quy tắc nghiệp vụ bên trong từng bước vẫn thuộc về service sở hữu domain tương ứng. Những workflow khác nhau, chẳng hạn đặt chuyến và hủy chuyến, cũng có thể được tách thành các kịch bản riêng để giữ phạm vi điều phối rõ ràng.
- **Một điểm tập trung cần được quản trị:** nhiều execution cùng đi qua hạ tầng điều phối chung, vì vậy capacity, quota, giới hạn của nền tảng và khả năng quan sát của orchestrator đều có thể ảnh hưởng đến toàn bộ workflow. Đồng thời, đội ngũ cũng phải xác định rõ ai sở hữu từng workflow, ai được phép thay đổi contract và những giới hạn nào cần được giám sát.

Orchestration giúp tập trung việc theo dõi tiến độ và quyết định bước tiếp theo. Để duy trì lợi ích đó, chúng ta cần **quản lý sự phụ thuộc giữa các service, giữ rõ phạm vi nghiệp vụ của orchestrator và bảo đảm khả năng vận hành của hạ tầng điều phối**.

8.4 Saga Choreography — không có orchestrator

8.4.1 Ý tưởng

Với **choreography**, trách nhiệm điều phối không nằm ở một orchestrator trung tâm mà được phân tán giữa các service. Khi hoàn thành phần việc của mình, một service phát ra event mô tả điều vừa xảy ra, chẳng hạn `HotelBooked`. Những service quan tâm đến event đó sẽ nhận và thực hiện bước tiếp theo theo contract đã thống nhất. Không có một thành phần trung tâm lần lượt ra lệnh cho toàn bộ chuỗi.

Tuy nhiên, **không có orchestrator không có nghĩa là không có thiết kế hay quyền sở hữu workflow**. Producer chịu trách nhiệm về ý nghĩa và cách công bố event; mỗi consumer chịu trách nhiệm về cách mình phản ứng với event đó; còn toàn bộ hệ thống vẫn cần cơ chế để theo dõi một chuyến đi đã đi đến đâu và đang chờ bước nào.

Việc producer không gọi trực tiếp consumer cũng không loại bỏ coupling hoàn toàn. Các service vẫn phụ thuộc vào **event contract**. Vì vậy, schema và ý nghĩa của event phải được quản lý như một contract chung; producer không thể tùy ý thay đổi chúng nếu những consumer khác vẫn đang dựa vào đó để xử lý.

8.4.2 Hai năng lực chính của hạ tầng sự kiện

Để các service phối hợp bằng event, hệ thống cần một **messaging infrastructure** đứng giữa producer và consumer. Trước khi lựa chọn công nghệ cụ thể, hãy xác định hai năng lực cơ bản mà hạ tầng này cần cung cấp. Chúng ta sẽ quay lại chính các yêu cầu này khi triển khai trên AWS ở Chương 9.

1. **Fan-out:** một event do producer phát ra có thể được phân phối đến nhiều consumer khác nhau mà producer không cần biết những consumer đó là ai. Chẳng hạn, cùng một `PaymentSucceeded` có thể được trip-service sử dụng để cập nhật trạng thái chuyến đi, trong khi notification-service sử dụng nó để gửi thông báo cho người dùng.
2. **Queue bền vững cho từng consumer:** mỗi consumer có hàng đợi riêng để giữ những message mà nó cần xử lý. Message tồn tại độc lập với vòng đời của consumer, vì vậy nếu consumer tạm thời dừng hoạt động, các message vẫn được giữ lại để xử lý khi nó hoạt động trở lại. Hạ tầng cũng cần hỗ trợ các cơ chế vận hành như retry khi xử lý thất bại và **dead-letter queue (DLQ)** cho những message không thể xử lý thành công sau số lần thử cho phép.

Tuy nhiên, queue chỉ giúp **giữ công việc lại để xử lý sau**, chứ không cung cấp dung lượng vô hạn. Retention, sức chứa và tốc độ xử lý đều có giới hạn. Nếu producer liên tục tạo message nhanh hơn khả năng xử lý của consumer, backlog sẽ ngày càng lớn và thời gian chờ của message cũng tăng theo.

Vì vậy, các chỉ số như **độ sâu của queue, tuổi của message lâu nhất và tốc độ xử lý backlog (drain rate)** cần được theo dõi và gắn với SLO của hệ thống. Điều này đúng bất kể bên dưới TravelNow sử dụng công nghệ messaging nào.

8.4.3 Queue cho phép chờ, nhưng chờ được bao lâu?

Queue giúp producer và consumer không phải xử lý công việc với cùng một tốc độ tại mọi thời điểm. Nhưng khả năng này chỉ hữu ích khi consumer có thể bắt kịp trở lại.

Giả sử producer liên tục tạo **100 message/giây**, trong khi toàn bộ consumer chỉ xử lý được **80 message/giây**. Mỗi giây sẽ có thêm 20 message chưa được xử lý, tương đương khoảng **1.200 message sau một phút**, chưa tính các message phải retry. Queue vẫn giữ được công việc, nhưng backlog và thời gian chờ sẽ liên tục tăng. Nếu một lượt giữ phòng chỉ có hiệu lực trong vài phút, message dù vẫn còn nguyên trong queue cũng có thể được xử lý quá muộn để hoàn tất chuyến đi.

Queue cũng làm thay đổi cách công việc được chuyển giao. Producer giao message cho broker rồi tiếp tục công việc của mình; consumer xử lý message theo khả năng của nó. Tuy nhiên, tách rời producer và consumer không có nghĩa là hệ thống có thể nhận tải không giới hạn. Vẫn cần **flow control** để quyết định có bao nhiêu công việc được xử lý đồng thời và hệ thống phải phản ứng thế nào khi tốc độ nhận việc vượt quá khả năng xử lý.

Với TravelNow, điều này dẫn đến ba nguyên tắc. Trước hết, số message được xử lý đồng thời phải được giới hạn để không làm quá tải database hoặc các nhà cung cấp bên ngoài. Khi backlog tăng đến mức không còn khả năng đáp ứng deadline, hệ thống cần áp dụng **backpressure** để giảm hoặc từ chối nhận thêm công việc thay vì tiếp tục chất chúng vào queue. Cuối cùng, trước khi xử lý một message đã nằm trong queue quá lâu, consumer phải kiểm tra deadline nghiệp vụ. Nếu công việc đã quá hạn, hệ thống cần ghi nhận trạng thái và chuyển sang đường phục hồi phù hợp thay vì âm thầm bỏ message.

Queue giúp hệ thống có thêm thời gian để hấp thụ những biến động tạm thời; nó không tạo thêm năng lực xử lý. Nếu tốc độ nhận việc liên tục lớn hơn tốc độ xử lý, backlog chắc chắn sẽ tăng. Và sau khi sự cố qua đi, consumer phải xử lý nhanh hơn tốc độ message mới đi vào thì backlog mới có thể giảm trở lại.

8.4.4 Từ một event đến nhiều queue

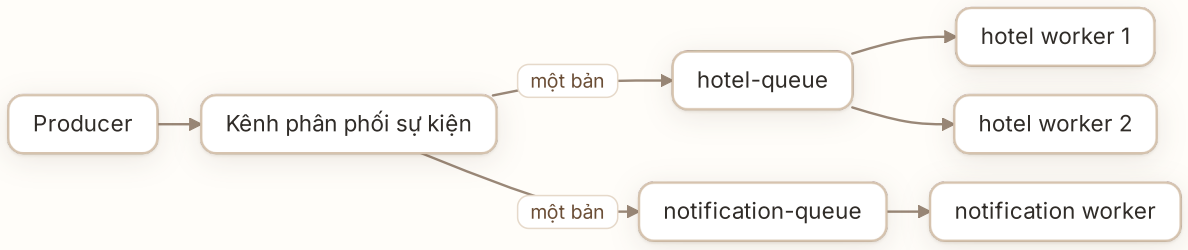
Giả sử cả hotel-service và notification-service đều cần xử lý cùng một event. Nếu worker của hai service cùng đọc từ **một queue**, chúng sẽ cạnh tranh để nhận message: một message được hotel-service xử lý thì không có nghĩa notification-service cũng nhận được message đó. Mô hình này phù hợp khi nhiều worker cùng chia tải cho **một loại công việc**, nhưng không phù hợp khi nhiều service đều cần nhận cùng một event.

Để mỗi service có thể xử lý event một cách độc lập, event cần được **fan-out tới các queue riêng**. Hotel-service nhận event qua `hotel-queue`, còn notification-service nhận cùng event qua `notification-queue`. Mỗi queue giữ backlog và tiến độ xử lý của riêng mình, vì vậy một service tạm thời chậm hoặc ngừng hoạt động không cản trở service còn lại.

Trong phạm vi một service, nhiều worker vẫn có thể cùng đọc từ **cùng một queue** để chia tải. Chẳng hạn, nhiều worker của hotel-service cùng xử lý `hotel-queue`; mỗi message chỉ cần được một trong các worker đó xử lý thành công. Như vậy, **fan-out được dùng để phân phối cùng một**

event cho nhiều nhóm consumer, còn nhiều worker trên cùng một queue được dùng để chia tải trong một nhóm consumer.

HÌNH 8.7 · SƠ ĐỒ LUỒNG



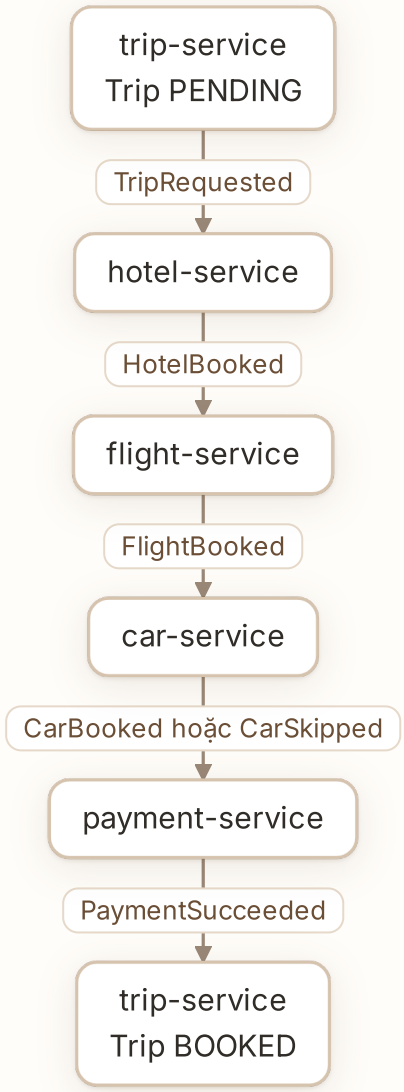
Đây chính là sự khác nhau giữa **pub/sub** và **chia việc giữa các worker trong cùng một nhóm consumer**. Với pub/sub, mỗi nhóm consumer quan tâm đến event cần nhận được một bản riêng; còn trong một nhóm, nhiều worker có thể cùng đọc một queue để chia nhau xử lý công việc.

Công nghệ cụ thể có thể khác nhau nhưng nguyên tắc này không đổi. Trong RabbitMQ, `exchange` và `binding` định tuyến message tới các queue phù hợp. Trên AWS, Chương 9 sẽ sử dụng SNS để phân phối event và SQS để lưu message chờ từng consumer xử lý. Dù sử dụng công nghệ nào, câu hỏi cần trả lời vẫn là: **mỗi bên có cần nhận một bản của event hay chỉ cần một worker trong nhóm xử lý nó?**

Mỗi consumer chịu trách nhiệm cho queue, bộ lọc event và cách xử lý lỗi của mình. Nếu một service mới cần nhận event, đường nhận bền vững của nó phải được thiết lập trước khi producer phát những event mà service đó cần xử lý. Việc đăng ký sau không mặc nhiên giúp consumer nhận lại những event đã đi qua trước đó.

Trong choreography, **happy path được hình thành từ một chuỗi phản ứng với event**. Trip-service tạo `Trip` ở trạng thái `PENDING` và phát `TripRequested`. Hotel-service nhận event này, đặt phòng rồi phát `HotelBooked`. Flight-service phản ứng với `HotelBooked`, đặt chuyến bay và phát `FlightBooked`. Nếu người dùng yêu cầu thuê xe, car-service tiếp tục xử lý bước tương ứng. Sau đó payment-service thực hiện thanh toán và phát `PaymentSucceeded`; khi nhận được event này, trip-service chuyển `Trip` sang `BOOKED`.

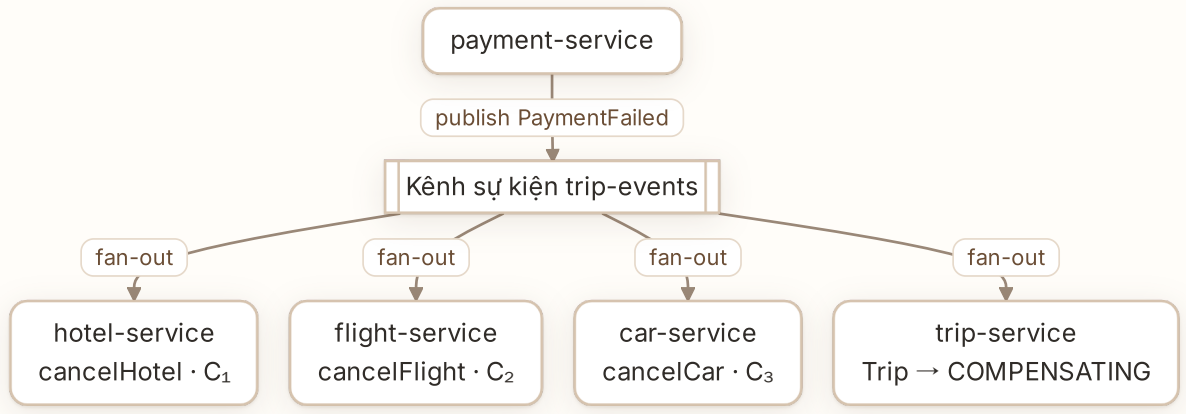
HÌNH 8.8 · SƠ ĐỒ LUỒNG



Điểm quan trọng là các event trên **mô tả điều đã xảy ra, không phải mệnh lệnh trực tiếp từ một orchestrator**. Không có thành phần trung tâm yêu cầu flight-service *"hãy đặt chuyến bay ngay bây giờ"*. Chính flight-service đã được thiết kế để biết rằng khi nhận `HotelBooked` phù hợp, nó cần thực hiện phần việc của mình và công bố kết quả cho những service tiếp theo.

Sad path cũng được hình thành theo cách tương tự. Nếu payment thất bại và được xác nhận là không thể tiếp tục, payment-service phát `PaymentFailed`. Các service quan tâm đến event này nhận thông báo và thực hiện phần bù trừ thuộc trách nhiệm của mình:

HÌNH 8.9 · SƠ ĐỒ LUỒNG



Trip-service tiếp tục theo dõi tiến trình bù trừ thông qua các event xác nhận như `HotelCancelled`, `FlightCancelled` và `CarCancelled`. Trip chỉ được chuyển từ `COMPENSATING` sang `FAILED` khi **tất cả các bước bù trừ cần thiết đã hoàn thành**. Do event có thể đến không đúng thứ tự, projection của trip-service phải xử lý được cả trường hợp event xác nhận bù trừ đến trước event báo lỗi. Nếu quá deadline mà vẫn chưa đủ kết quả cần thiết, Trip được chuyển sang `MANUAL_REVIEW`.

Sơ đồ trên chỉ minh họa trường hợp payment thất bại. Trong thực tế, nếu lỗi xảy ra ở hotel, flight hoặc car, luồng event cũng phải có nhánh phục hồi tương ứng để bù trừ những bước đã hoàn thành trước đó.

Các bước tùy chọn cũng cần được biểu diễn rõ trong event contract. Chẳng hạn, nếu người dùng không thuê xe, hệ thống có thể phát `CarSkipped` để payment-service biết rằng bước car đã được bỏ qua hợp lệ và có thể tiếp tục thanh toán. Workflow không nên bị mắc kẹt chỉ vì một bước tùy chọn không tạo ra event mà bước tiếp theo đang chờ.

Một điểm đáng chú ý là hotel-service, flight-service và car-service đều có thể nhận `PaymentFailed`, vì vậy các bước bù trừ của chúng **có thể được thực hiện song song**. Tuy nhiên, điều này chỉ an toàn khi các bước bù trừ thực sự độc lập với nhau. Nếu việc bù trừ một bước phụ thuộc vào kết quả của bước khác, thứ tự đó phải được thể hiện trong chuỗi event hoặc được một coordinator điều phối. **Choreography phân tán quyền điều phối; nó không tự động biến các bước thành độc lập hay cho phép chúng chạy song song.**

Một đặc điểm quan trọng khác của mô hình này là producer không cần consumer phải hoạt động **ngay tại thời điểm event được phát ra**. Nếu hotel-service ngừng hoạt động trong mười phút, message vẫn có thể nằm trong queue và được xử lý khi service hoạt động trở lại, miễn là queue vẫn còn trong giới hạn retention và capacity. Workflow vì thế có thể chậm lại thay vì buộc một request đồng bộ phải chờ consumer phục hồi.

Đây là **temporal decoupling**: producer và consumer không cần cùng hoạt động tại một thời điểm. Tuy nhiên, nó không làm vấn đề availability biến mất mà chuyển một phần bài toán sang **độ trễ, backlog, deadline và cách thông báo trạng thái cho người dùng**. Hệ thống đồng bộ vẫn

có thể resilient nếu được thiết kế tốt với timeout, fallback và redundancy; ngược lại, hệ thống bất đồng bộ vẫn có thể gặp sự cố nghiêm trọng nếu queue và backlog không được giám sát, giới hạn và phục hồi đúng cách.

8.4.5 Consumer trong thế giới at-least-once

Với mô hình queue có cơ chế xác nhận, consumer nhận message, xử lý rồi gửi **ack** để báo cho broker rằng công việc đã hoàn tất. Nhưng nếu consumer xử lý xong rồi chết trước khi gửi ack, broker không biết công việc đã thực sự hoàn thành và có thể giao lại message cho một consumer khác.

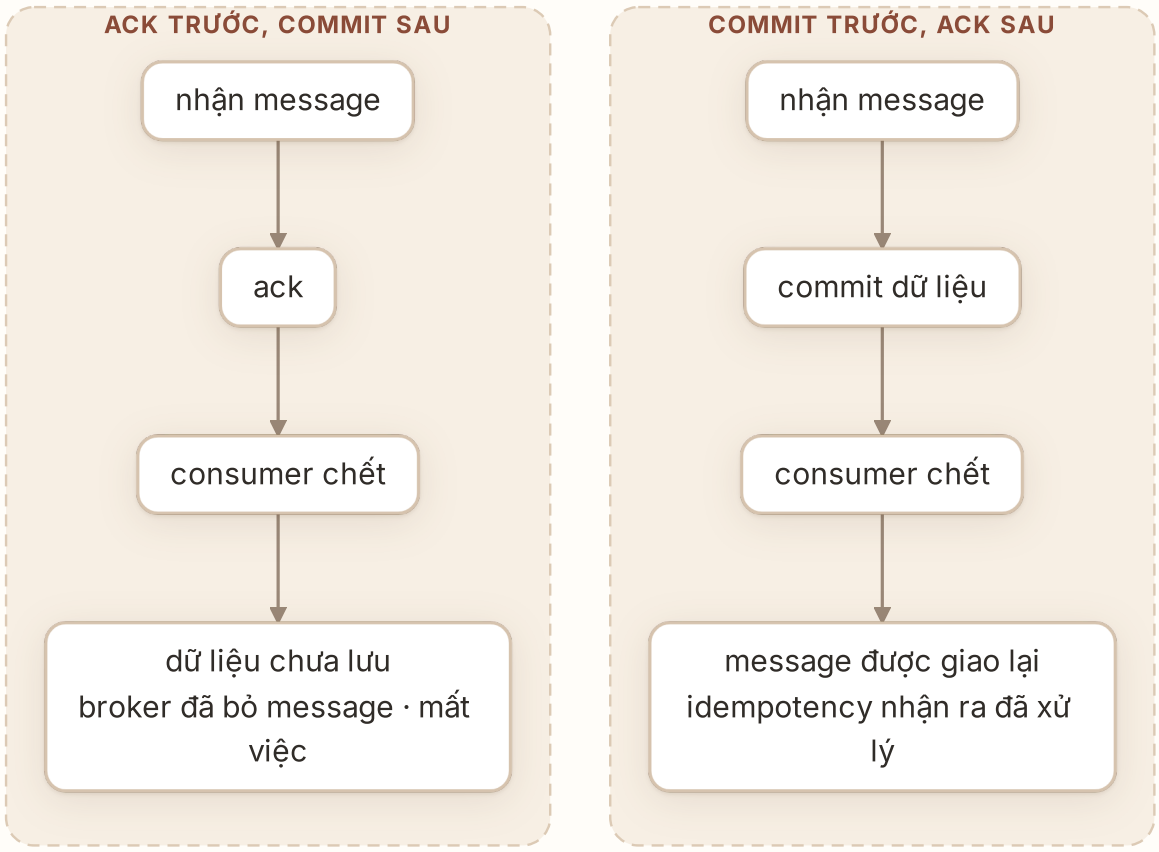
Đây chính là trạng thái *"không biết"* đã gặp ở Chương 7, nhưng lần này xuất hiện trong hạ tầng messaging. Với cơ chế **at-least-once delivery**, một message có thể được giao nhiều hơn một lần, vì vậy consumer phải được thiết kế để xử lý an toàn khi có redelivery. Tuy nhiên, đây không phải đặc tính mặc định của mọi queue; guarantee cụ thể vẫn phụ thuộc vào broker và cách chúng ta cấu hình cơ chế giao nhận, ack và retry.

Vì vậy, handler không thể giả định rằng mỗi message chỉ xuất hiện một lần. Với event `HotelBooked`, flight-service có thể xử lý theo ba bước:

1. **Xác định message đã được xử lý hay chưa** dựa trên định danh thao tác và trạng thái hiện tại, bằng một ràng buộc concurrency-safe để hai worker không thể đồng thời thực hiện cùng một side effect.
2. **Trong cùng một local transaction**, lưu kết quả giữ vé, đánh dấu event đã được xử lý và ghi lại ý định phát `FlightBooked`.
3. **Chỉ ack sau khi transaction đã commit thành công.** Nếu transaction thất bại, message vẫn phải còn khả năng được xử lý lại theo retry policy.

Thứ tự này giúp tránh hai failure mode quan trọng. Nếu **ack trước khi commit**, consumer có thể chết sau khi ack nhưng trước khi dữ liệu được lưu; broker cho rằng công việc đã hoàn tất và message có nguy cơ bị mất. Ngược lại, nếu **commit trước khi ack**, consumer có thể chết sau khi dữ liệu đã được lưu nhưng trước khi gửi ack; message sẽ được giao lại, nhưng idempotency giúp lần xử lý tiếp theo nhận ra công việc đã hoàn thành và không tạo thêm side effect.

HÌNH 8.10 · SƠ ĐỒ LUỒNG



Cơ chế trên bảo vệ tốt những thay đổi nằm trong cùng transaction của service. Nếu handler còn tạo side effect bên ngoài database — chẳng hạn gọi API của một nhà cung cấp — local transaction không thể bao trùm side effect đó. Khi ấy, service vẫn cần **idempotency contract, tra cứu trạng thái hoặc cơ chế đối soát** như đã thảo luận ở Chương 7.

8.4.6 Local transaction và event: lấp khoảng trống dual write

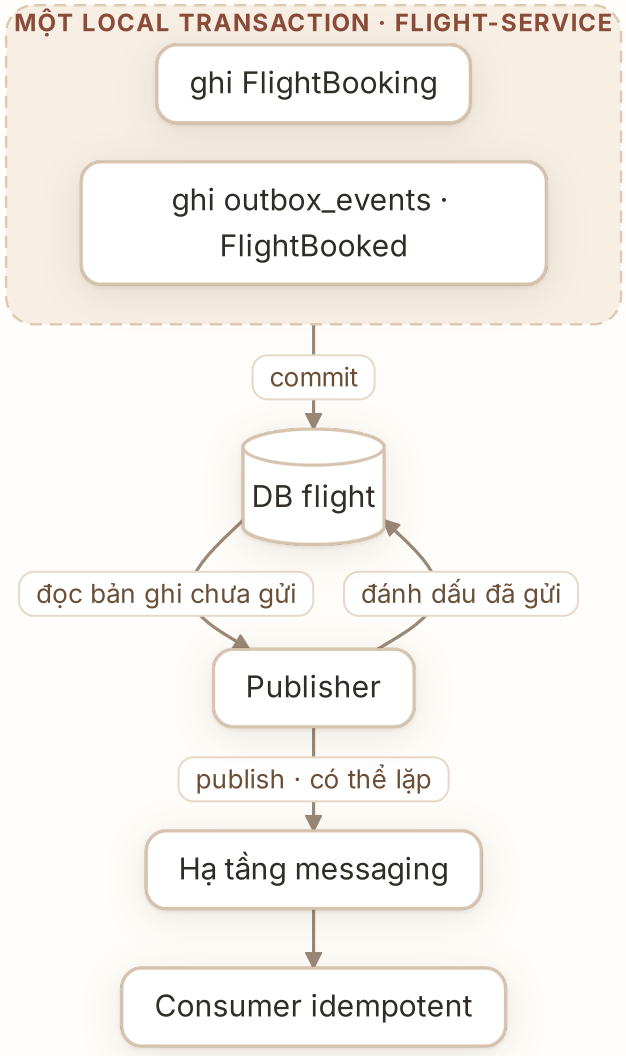
Một vấn đề khác xuất hiện khi consumer vừa phải **thay đổi dữ liệu trong database**, vừa phải **phát event cho bước tiếp theo**. Nếu hai việc này được thực hiện độc lập, hệ thống có thể rơi vào trạng thái không nhất quán.

Giả sử flight-service lưu booking thành công rồi mới phát `FlightBooked`. Nếu database đã commit nhưng process chết trước khi event được publish, booking đã tồn tại nhưng các service phía sau không biết để tiếp tục Saga. Ngược lại, nếu `FlightBooked` được publish trước rồi transaction trong database thất bại, downstream có thể xử lý một event mô tả điều chưa thực sự xảy ra. Đây là **dual-write problem**.

TravelNow giải quyết khoảng trống này bằng **transactional outbox**. Thay vì trực tiếp publish event trong lúc xử lý request, service ghi hai thứ trong **cùng một local transaction**: thay đổi dữ liệu nghiệp vụ và một bản ghi trong `outbox_events` mô tả event cần phát. Khi transaction commit, cả hai cùng được lưu; nếu transaction rollback, cả hai cùng biến mất.

Sau đó, một publisher riêng đọc các bản ghi trong outbox và phát chúng lên hạ tầng messaging. Khi publish thành công, bản ghi tương ứng được đánh dấu đã gửi. Publisher vẫn có thể chết sau khi event đã được phát nhưng trước khi cập nhật trạng thái outbox, khiến cùng một event được phát lại. Vì vậy, transactional outbox **không loại bỏ duplicate**; consumer vẫn phải idempotent.

HÌNH 8.11 · SƠ ĐỒ LUỒNG



Có thể triển khai việc đọc outbox bằng polling, CDC hoặc những cơ chế khác mà nền tảng cung cấp. Cách triển khai có thể thay đổi, nhưng invariant cần giữ nguyên: **thay đổi state và ý định phát event phải được ghi atomically trong cùng một transaction boundary**. Transactional outbox và idempotency vì thế giải quyết hai vấn đề khác nhau: **outbox tránh khoảng trống "đã commit nhưng chưa ghi nhận event cần phát", còn idempotency giúp hệ thống an toàn khi event được phát hoặc xử lý nhiều lần**.

Một vấn đề khác là message có thể liên tục thất bại dù đã retry. Khi đó, hệ thống cần một retry policy hữu hạn và **dead-letter queue (DLQ)** để tách những message chưa thể xử lý khỏi luồng bình thường. Tuy nhiên, đưa message vào DLQ không có nghĩa vấn đề đã được giải quyết. Cần có người hoặc quy trình chịu trách nhiệm điều tra nguyên nhân, sửa lỗi và quyết định message có

thể được replay hay phải chuyển sang đối soát hoặc xử lý thủ công. Các chỉ số như tuổi của message và số lần xử lý thất bại cũng cần được theo dõi để phát hiện vấn đề trước khi Saga bị treo quá lâu.

Ngay cả khi đã có outbox và DLQ, **thứ tự xử lý event vẫn cần được xem xét riêng**. Redelivery, nhiều worker chạy song song và thời gian xử lý khác nhau có thể khiến thứ tự hoàn thành không giống thứ tự event được phát. Vì vậy, các cơ chế như tombstone, `version` và transition policy đã trình bày ở phần trước vẫn cần thiết để ngăn một event cũ ghi đè lên trạng thái mới hơn.

Quan trọng hơn, không phải toàn bộ hệ thống đều cần một thứ tự chung. Cần xác định **thứ tự nào thực sự có ý nghĩa đối với nghiệp vụ và trong phạm vi nào**. Chẳng hạn, các thao tác đặt và hủy của cùng một `tripId` có quan hệ với nhau nên cần được bảo vệ theo cùng business key. Trong khi đó, hai chuyến đi độc lập thường không cần chờ nhau. Ép tất cả message vào một thứ tự toàn cục không chỉ không cần thiết mà còn có thể khiến một Saga bị chậm làm ảnh hưởng đến những Saga hoàn toàn không liên quan.

8.4.7 Khó khăn khi theo dõi toàn bộ quy trình nghiệp vụ

Choreography giúp các service ít phụ thuộc trực tiếp vào nhau hơn, nhưng đổi lại, việc nhìn thấy toàn bộ workflow trở nên khó hơn. Hãy quay lại câu hỏi rất đơn giản ở phần orchestration: *"Saga của trip abc đang ở đâu?"*

Với orchestration, trạng thái và tiến độ của Saga được lưu tập trung nên câu trả lời có thể lấy trực tiếp từ kho trạng thái Saga. Với choreography, **không mặc nhiên tồn tại một nơi lưu toàn bộ tiến độ**. Một phần thông tin nằm trong database của hotel-service, một phần ở flight-service, một số event đang chờ trong queue, và những event khác có thể đã được xử lý từ trước. Nếu không thiết kế thêm cơ chế quan sát, muốn biết một Trip đã đi đến đâu sẽ phải ghép lại dấu vết từ nhiều nơi.

Vì vậy, TravelNow bổ sung hai cơ chế. Thứ nhất là **correlation ID**: mọi event thuộc cùng một Saga mang một định danh ổn định, ở đây là `tripId`. Các log quan trọng tại điểm nhận event, phát event, chuyển trạng thái và xử lý lỗi cũng ghi lại định danh này để có thể nối các hoạt động thuộc cùng một chuyến đi. Sang Chương 14, chúng ta sẽ mở rộng ý tưởng này khi nói về distributed tracing.

Thứ hai là một **projection trạng thái Saga**. Trong thiết kế của TravelNow, trip-service lắng nghe những event cần thiết và tổng hợp chúng thành một góc nhìn về tiến độ của chuyến đi, chẳng hạn: `HotelBooked` đã hoàn thành, `FlightBooked` đã hoàn thành và đang chờ payment. Projection này cũng phải được thiết kế như một thành phần đáng tin cậy: xử lý được duplicate, event đến không đúng thứ tự và những khoảng trống có thể xuất hiện do retention. Nó không đơn thuần là một bảng phụ để phục vụ giao diện, mà là dữ liệu vận hành giúp hệ thống biết workflow đang ở trạng thái nào.

Choreography còn có một chi phí khác khó nhận thấy hơn: **thay đổi workflow thường liên quan đến nhiều service**. Giả sử TravelNow muốn chèn thêm bước *"kiểm tra hạn mức khách hàng"* vào giữa chuỗi. Với orchestration, thứ tự mới chủ yếu được thể hiện bằng cách thay đổi workflow của

orchestrator. Với choreography, service thực hiện bước kiểm tra phải tham gia vào chuỗi event, đồng thời những consumer phía sau cũng phải biết khi nào điều kiện mới đã được thỏa mãn để tiếp tục. Producer hiện tại có thể không cần thay đổi nếu event contract vẫn phù hợp, nhưng việc thay đổi thứ tự của toàn bộ luồng vẫn cần được phối hợp giữa các service liên quan.

Khi số lượng event và consumer tăng lên, quan hệ giữa các service cũng trở nên khó theo dõi hơn. Hai service thậm chí có thể vô tình phản ứng qua lại với event của nhau và tạo thành **event loop**. Vì vậy, choreography vẫn cần một sơ đồ mô tả toàn bộ workflow, quản lý rõ event contract và có các bài kiểm thử để xác nhận rằng những thay đổi ở một service không làm sai luồng nghiệp vụ ở các service khác.

8.4.8 Cùng bộ câu hỏi khi thay hạ tầng messaging

Các loại hạ tầng messaging khác nhau được thiết kế cho những nhu cầu khác nhau. **Queue** phù hợp với việc phân phối công việc cho consumer và theo dõi việc xử lý thông qua cơ chế xác nhận. **Topic** phù hợp khi cùng một event cần được phân phối tới nhiều nhóm consumer. Trong khi đó, **distributed log** cho phép mỗi consumer duy trì vị trí đọc riêng và có thể đọc lại dữ liệu còn nằm trong khoảng thời gian retention.

Vì vậy, việc chọn một công nghệ quen thuộc chưa đủ để đảm bảo rằng hệ thống có đúng những đặc tính mà workflow cần. Khi thay đổi hạ tầng messaging, thay vì bắt đầu từ tên sản phẩm, hãy quay lại những câu hỏi nền tảng: **message được giữ trong bao lâu, ai chịu trách nhiệm xác nhận việc xử lý, duplicate được nhận diện và xử lý như thế nào, thứ tự được bảo đảm trong phạm vi nào, và hệ thống phục hồi công việc đang dở ra sao?**

Chương 9 sẽ sử dụng chính những câu hỏi này khi ánh xạ thiết kế của TravelNow sang SNS và SQS trên AWS. Công nghệ có thể thay đổi, nhưng các yêu cầu về độ tin cậy của workflow vẫn còn nguyên.

Đặc biệt, cần nhớ rằng **guarantee của hạ tầng messaging chỉ có hiệu lực trong phạm vi mà nó kiểm soát**. Broker có thể đảm bảo cách message được lưu và phân phối, nhưng không tự bảo vệ những thay đổi mà consumer thực hiện trong database hoặc hệ thống bên ngoài. Các side effect đó vẫn cần được bảo vệ bằng transaction, idempotency và những cơ chế phục hồi phù hợp ở phía ứng dụng.

8.5 So sánh hai cách phối hợp

Cùng một nghiệp vụ, cùng một cơ chế bù trừ, nhưng hai cách tổ chức và vận hành khác nhau:

Khía cạnh	Orchestration	Choreography
Luồng nghiệp vụ được định nghĩa ở đâu	**Tập trung** trong workflow của orchestrator; có thể đọc tương đối trực tiếp để hiểu toàn bộ luồng	**Phân tán** trong logic phản ứng với event của các service; cần ghép các phản ứng lại để thấy toàn bộ luồng

Khía cạnh	Orchestration	Choreography
Biết Saga đang ở đâu	Truy vấn trạng thái và execution history của orchestrator	Cần correlation ID và một projection hoặc cơ chế tổng hợp trạng thái nếu muốn có góc nhìn tập trung
Coupling	Orchestrator phụ thuộc vào contract và kết quả của các bước mà nó điều phối	Service không cần biết địa chỉ của nhau nhưng vẫn phụ thuộc vào schema và semantics của event
Thêm một bước bắt buộc	Chủ yếu thay đổi workflow của orchestrator và tích hợp với service mới	Thêm phản ứng/event mới và điều chỉnh các consumer liên quan để phản ánh thứ tự mới
Thay đổi thứ tự các bước	Thứ tự chủ yếu được thay đổi trong workflow tập trung	Thứ tự được thể hiện qua quan hệ giữa các event và consumer nên có thể cần thay đổi ở nhiều service
Rủi ro đặc trưng	Orchestrator dễ tích tụ quá nhiều kiến thức hoặc business logic; hạ tầng điều phối trở thành một điểm tập trung cần quản trị	Workflow khó quan sát toàn cục; quan hệ event phức tạp có thể tạo dependency khó thấy hoặc event loop
Retry, deadline và bù trừ	Policy có thể được quản lý tập trung trong workflow; vẫn cần đường xử lý thủ công khi không thể tự phục hồi	Policy được phân bổ giữa consumer và hạ tầng messaging; cần phối hợp retry, DLQ, reconciliation và runbook
Đặc điểm thực thi	Dễ biểu diễn rõ các bước tuần tự, nhánh và dependency trong workflow	Các bước độc lập có thể xử lý song song; đổi lại mỗi bước có thể chịu thêm thời gian chờ và xử lý qua queue
Hạ tầng cần vận hành	Orchestrator hoặc workflow engine cùng persistent storage và cơ chế phục hồi	Broker cùng topic/queue, subscription, DLQ và các cơ chế theo dõi backlog
Phù hợp khi	Workflow dài, nhiều bước hoặc nhánh, cần theo dõi tiến độ và audit tập trung	Luồng event tương đối tự nhiên, các bước ít phụ thuộc chặt về thứ tự hoặc cùng một event cần phục vụ nhiều consumer

Nếu rút gọn bảng trên thành một ý: **orchestration đưa việc điều phối về một nơi để luồng dễ quan sát và kiểm soát hơn; choreography phân tán việc điều phối để các bên tham gia ít phụ thuộc trực tiếp và dễ mở rộng hơn.** Đổi lại, orchestration chấp nhận một điểm điều phối tập trung, còn choreography chấp nhận việc toàn bộ workflow khó nhìn thấy hơn. Đây là một trong những đánh đổi quan trọng khi lựa chọn giữa hai mô hình.

Trong thực tế, có thể cân nhắc một số tiêu chí sau.

Nếu workflow **dài, có nhiều bước hoặc nhiều nhánh điều kiện, liên quan đến tiền và thường xuyên cần trả lời câu hỏi "quy trình đang ở bước nào?"**, orchestration thường giúp việc theo dõi và kiểm soát đơn giản hơn. Luồng booking chính của TravelNow thuộc trường hợp này: hệ thống cần biết phòng, chuyến bay, xe và payment đã đi đến đâu, đồng thời cần một đường phục hồi rõ ràng khi một bước thất bại.

Ngược lại, choreography đặc biệt hữu ích khi **cùng một event cần kích hoạt nhiều phản ứng tương đối độc lập**. Chẳng hạn, sau khi một chuyến đi được đặt thành công, các service khác nhau có thể gửi email, cộng điểm thưởng hoặc cập nhật analytics. Một consumer mới thường có thể

được bổ sung mà không cần thay đổi producer, miễn là event contract và hạ tầng phân phối hiện tại đã cung cấp đủ thông tin cho nó.

Dù chọn mô hình nào, câu hỏi **"ai chịu trách nhiệm cho kết quả cuối cùng của Saga?"** vẫn phải có câu trả lời rõ ràng. Với orchestration, cần xác định ai sở hữu workflow, contract giữa orchestrator và các service, cũng như quyền thay đổi kịch bản. Với choreography, dù không có orchestrator trung tâm, vẫn cần một bên chịu trách nhiệm nhìn toàn bộ chuỗi phản ứng và đảm bảo rằng các event kết hợp với nhau vẫn tạo thành một workflow hợp lệ. Vì vậy, số lượng team tham gia không tự nó quyết định nên chọn mô hình nào.

Năng lực vận hành của đội ngũ cũng là một yếu tố cần tính đến. Với một team nhỏ hoặc chưa có nhiều kinh nghiệm về messaging, một workflow điều phối tập trung thường dễ lần theo hơn. Choreography đòi hỏi đầu tư sớm hơn vào event contract, correlation, projection trạng thái, backlog, DLQ và khả năng truy vết công việc đang dở. Những chi phí này nên được tính ngay từ lúc lựa chọn kiến trúc, thay vì chỉ xuất hiện sau khi hệ thống đã đi vào vận hành.

Hai mô hình cũng **không loại trừ nhau**. TravelNow có thể sử dụng orchestration làm xương sống cho Saga booking, nơi thứ tự các bước, payment và recovery cần được kiểm soát rõ ràng. Khi booking hoàn tất, hệ thống vẫn có thể phát event để notification, loyalty, analytics và các chức năng phụ trợ phản ứng theo choreography. Khi đó, orchestration quản lý phần workflow cần phối hợp chặt chẽ, còn choreography mở rộng những phản ứng ít phụ thuộc vào luồng chính.

Vì vậy, không có một lựa chọn phù hợp cho mọi hệ thống. Quyết định phụ thuộc vào **đặc điểm của workflow, mức độ coupling có thể chấp nhận, yêu cầu quan sát và năng lực vận hành của đội ngũ**. Điều quan trọng không phải chọn pattern theo tên gọi, mà hiểu **sự phức tạp sẽ nằm ở đâu sau khi lựa chọn**: tập trung trong orchestrator hay phân tán giữa event, consumer và hạ tầng messaging.

8.6 Tóm tắt

Saga giải quyết distributed transaction bằng cách chia một quy trình lớn thành **chuỗi local transaction**, mỗi bước commit độc lập và tiến độ của toàn bộ workflow được lưu lại. Khi quy trình không thể tiếp tục, hệ thống sử dụng các **compensating transaction** để bù trừ về mặt nghiệp vụ cho những bước đã hoàn thành. Vì bản thân việc bù trừ cũng có thể timeout, thất bại hoặc bị gián đoạn, Saga cần retry an toàn, lưu lại tiến độ, có cơ chế reconciliation và một trạng thái rõ ràng như `MANUAL_REVIEW` khi hệ thống không thể tự phục hồi. Đồng thời, Saga không tự cung cấp isolation giữa các workflow cạnh tranh; idempotency, version và transition policy vẫn cần thiết để bảo vệ trạng thái nghiệp vụ.

Có hai cách chính để phối hợp các bước của Saga. **Orchestration** tập trung logic điều phối vào một orchestrator, giúp thứ tự, trạng thái và đường phục hồi của workflow được nhìn thấy rõ hơn. **Choreography** phân tán việc phối hợp giữa các service thông qua event, giúp các bên tham gia ít phụ thuộc trực tiếp vào nhau nhưng khiến toàn bộ luồng khó quan sát hơn. Dù chọn mô hình

nào, hệ thống vẫn phải xử lý duplicate và event đến muộn, bảo vệ các chuyển trạng thái và có khả năng theo dõi một Saga từ đầu đến cuối. Trong kiến trúc event-driven, **transactional outbox** còn giúp khép khoảng trống dual write giữa việc commit dữ liệu và ghi nhận ý định phát event.

Đến đây, những thành phần hạ tầng mà Saga cần đã hiện ra khá rõ: **cơ chế điều phối workflow, nơi lưu trạng thái, cơ chế phục hồi và hạ tầng messaging**. Sang Chương 9, chúng ta sẽ ánh xạ những trách nhiệm này lên các dịch vụ AWS để xem phần nào có thể giao cho nền tảng quản lý, và quan trọng hơn, **những trách nhiệm về tính đúng đắn của nghiệp vụ vẫn phải nằm trong ứng dụng dù hạ tầng bên dưới được managed đến đâu**.

CHƯƠNG 9

Triển khai hệ thống phân tán trên AWS

Phần III — Kiến trúc phân tán · *The Hard Parts*

Đến cuối Chương 8, chúng ta đã biết để một quy trình Saga hoạt động đáng tin cậy, hệ thống cần lưu tiến độ bền vững, xử lý an toàn những thao tác được gọi lại và bảo đảm công việc bù trừ có thể tiếp tục ngay cả khi chương trình dừng giữa chừng. Với phương án choreography, hệ thống còn phải phân phối sự kiện đến từng consumer và bảo đảm không bỏ sót sự kiện sau khi dữ liệu đã được commit.

Những yêu cầu này có thể được hiện thực trên nhiều nền tảng hoặc bằng các thành phần mã nguồn mở do đội ngũ tự vận hành. Trong chương này, chúng ta chọn AWS để triển khai một phương án cụ thể cho TravelNow, từ tiếp nhận yêu cầu, thực thi nghiệp vụ đến xử lý sự cố. **Thông qua từng tình huống của TravelNow, chúng ta sẽ tìm hiểu các dịch vụ AWS giải quyết vấn đề gì, cung cấp những bảo đảm nào và đâu là trách nhiệm mà ứng dụng vẫn phải tự đảm nhận.**

Trong chương này, chúng ta sẽ triển khai hai phương án Saga đã phân tích: **orchestration với AWS Step Functions** và **choreography với SNS/SQS**. Cả năm service của TravelNow đều chạy trên Lambda, với API Gateway làm điểm tiếp nhận yêu cầu từ client. Về dữ liệu, mô hình minh họa sử dụng một RDS PostgreSQL instance chứa các database riêng cho từng service, đồng thời xem xét những đánh đổi khi tách thành nhiều RDS instance hoặc chuyển sang DynamoDB. Với mỗi phương án, chúng ta sẽ theo dõi toàn bộ luồng xử lý, từ khi tiếp nhận yêu cầu đến khi hoàn tất, cũng như cách hệ thống phục hồi khi có sự cố.

Cách đọc chương này

Bạn nên đọc mục 9.1–9.2 trước để theo dõi luồng chính: tiếp nhận yêu cầu, lưu ý định, thực thi Saga và trả kết quả. Sau đó, mục 9.3 giới thiệu phương án thay thế sử dụng sự kiện để bạn so sánh hai cách triển khai. Cuối cùng, mục 9.4 sẽ xem xét cách cả hai phương án xử lý và phục hồi khi xảy ra sự cố.

Ba thuật ngữ sẽ xuất hiện thường xuyên trong chương: **workflow** là quy trình gồm nhiều bước; **consumer** là thành phần nhận và xử lý thông điệp; **relay** là thành phần chuyển công việc đã được lưu sang hệ thống thực thi. Bạn có thể bỏ qua các mục đọc thêm trong lần đọc đầu tiên mà vẫn theo dõi được nội dung chính.

9.1 Từ thiết kế của TravelNow đến kiến trúc trên AWS

9.1.1 Giữ ranh giới nghiệp vụ khi thay đổi hạ tầng

Khi triển khai TravelNow lên AWS, 5 service đã giới thiệu ở Chương 6 vẫn giữ nguyên trách nhiệm nghiệp vụ. `trip-service` tiếp nhận yêu cầu và quản lý trạng thái chuyến đi; `hotel`, `flight`, `car` và `payment` quản lý nghiệp vụ cùng dữ liệu tương ứng. Điều thay đổi không phải là cách phân chia trách nhiệm, mà là môi trường thực thi, phương thức giao tiếp giữa các service và cơ chế duy trì công việc khi xảy ra sự cố.

Trong phương án triển khai này, cả năm service đều sử dụng AWS Lambda. Mỗi service có thể bao gồm nhiều Lambda function đảm nhận những tác vụ khác nhau, chẳng hạn tiếp nhận yêu cầu, giữ chỗ, hủy chỗ, tra cứu kết quả hoặc phát các sự kiện đã lưu trong outbox. Dù được triển khai thành nhiều function, chúng vẫn thuộc cùng một domain, tuân theo các contract và chỉ truy cập dữ liệu trong phạm vi trách nhiệm của service đó. **Một function là đơn vị thực thi, còn một service là ranh giới trách nhiệm nghiệp vụ.**

AWS chịu trách nhiệm quản lý môi trường thực thi Lambda, trong khi chúng ta vẫn phải quản lý mã nguồn, quyền truy cập và giới hạn tài nguyên của từng function. Khác với mô hình mỗi service chạy trên một máy chủ riêng, Lambda có thể tạo nhiều lượt thực thi đồng thời. Vì vậy, những trạng thái cần duy trì phải được lưu trữ bên ngoài bộ nhớ của function.

Bản chất phân tán của TravelNow cũng không phụ thuộc vào số lượng máy chủ hay Lambda function. Điều quan trọng là các service giao tiếp qua mạng, thực hiện transaction độc lập và có thể gặp sự cố tại những thời điểm khác nhau. Chính những đặc điểm này đặt ra các yêu cầu về phối hợp nghiệp vụ và khả năng phục hồi mà chúng ta sẽ hiện thực hóa trên AWS.

9.1.2 Vì sao chọn Lambda cho chương này?

Các thao tác nghiệp vụ của TravelNow có thể được chia thành những lượt xử lý tương đối ngắn, như tiếp nhận yêu cầu, giữ chỗ, ghi nhận kết quả, phát sự kiện hoặc tra cứu trạng thái. Thay vì duy trì một tiến trình chạy liên tục để chờ các bước hoàn tất, chúng ta giao việc điều phối và lưu giữ công việc cho workflow hoặc queue.

Với giả định lưu lượng truy cập biến động và đội ngũ muốn giảm công việc quản lý máy chủ, AWS Lambda là lựa chọn phù hợp cho mô hình minh họa. Lambda cho phép chúng ta tập trung vào logic nghiệp vụ và tích hợp trực tiếp với API Gateway, Step Functions và SQS.

Tuy nhiên, lựa chọn này cũng đi kèm những đánh đổi. Trước khi đi sâu vào triển khai, hãy so sánh Lambda với các phương án thực thi khác:

Phương án thực thi	Ưu điểm	Đánh đổi và trách nhiệm vận hành
AWS Lambda	Không cần quản lý máy chủ; tự động mở rộng theo nhu cầu; tích hợp trực tiếp với API Gateway, Step Functions và SQS.	Có thể gặp cold start; giới hạn thời gian thực thi và concurrency; cần xây dựng handler phù hợp với từng nguồn gọi. Chi phí phụ thuộc vào số lượt gọi và thời gian thực thi.

Phương án thực thi	Ưu điểm	Đánh đổi và trách nhiệm vận hành
Container trên EC2	Duy trì mô hình triển khai ứng dụng quen thuộc; chủ động kiểm soát môi trường thực thi và các tiến trình chạy dài hạn.	Phải tự quản lý máy chủ, tài nguyên, triển khai và phục hồi. Nếu nhiều service chạy trên cùng một máy, sự cố của máy có thể ảnh hưởng đến toàn bộ hệ thống.
Container trên ECS/Fargate hoặc EKS	Phù hợp với ứng dụng cần chạy liên tục, yêu cầu kiểm soát môi trường thực thi hoặc muốn tận dụng nền tảng container hiện có.	Cần cấu hình tài nguyên, mạng, tự động co giãn và giám sát. Riêng EKS còn đòi hỏi kiến thức và năng lực vận hành Kubernetes.

Lambda không phải lúc nào cũng tiết kiệm chi phí hơn, đặc biệt với những ứng dụng có tải cao và ổn định. Bên cạnh đó, chúng ta cần đánh giá ảnh hưởng của **cold start** — độ trễ phát sinh khi Lambda phải khởi tạo một môi trường thực thi mới, nhất là với ứng dụng có thời gian khởi tạo dài. Mỗi invocation có thời gian thực thi tối đa 15 phút, vì vậy không thể duy trì một function để chờ toàn bộ quy trình Saga hoàn tất. Những công việc kéo dài cần được chia thành các bước nhỏ hoặc sử dụng cơ chế chờ và tiếp nhận kết quả phù hợp. AWS — Lambda timeout.

Trong chương này, chúng ta chọn Lambda để triển khai kiến trúc phân tán của TravelNow trên AWS. Sang Phần IV, chúng ta sẽ chuyển sang mô hình container để tìm hiểu Kubernetes và Amazon EKS, từ đó so sánh hai phương án dựa trên đặc điểm tải và yêu cầu vận hành. **Đây là hai cách triển khai cho cùng một hệ thống nghiệp vụ, không phải lộ trình bắt buộc phải chuyển từ Lambda sang Kubernetes.**

9.1.3 Một RDS instance có đủ cho năm service không?

Có, nếu tài nguyên của instance đáp ứng được nhu cầu xử lý. Trong mô hình minh họa, chúng ta sử dụng một RDS PostgreSQL instance chứa năm database riêng biệt: `trip`, `hotel`, `flight`, `car` và `payment`. Mỗi service có tài khoản và quyền truy cập riêng, chỉ được phép kết nối tới database thuộc phạm vi trách nhiệm của mình, thay vì dùng chung tài khoản quản trị. Các service cũng tự quản lý migration, bảng lưu kết quả idempotency và outbox. Khi tạo tài khoản ứng dụng, cần rà soát và giới hạn các quyền mặc định của PostgreSQL. PostgreSQL — Quản lý database.

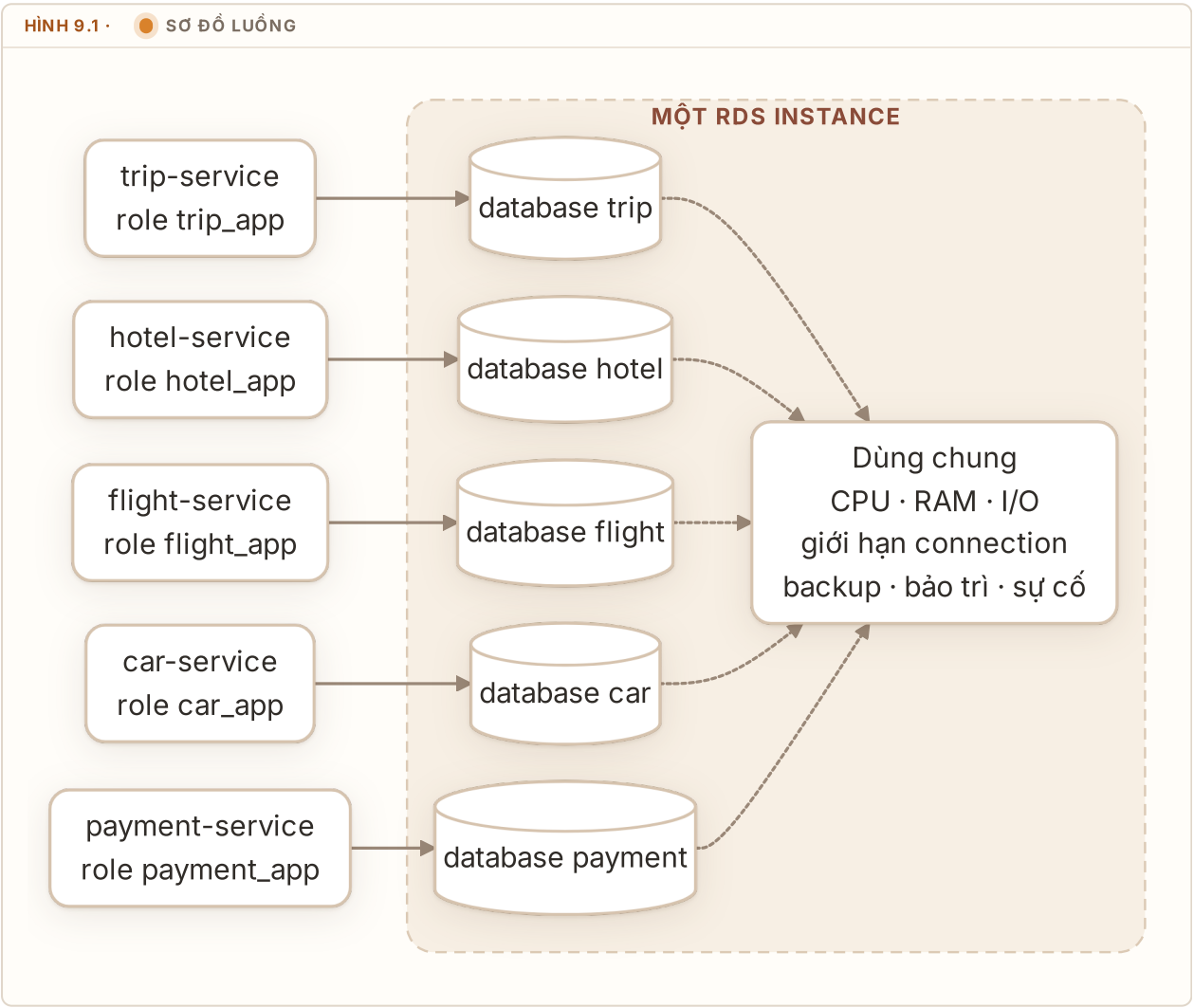
Ở đây, cần phân biệt **database logic và RDS instance**. Việc mỗi service sở hữu dữ liệu riêng không có nghĩa chúng ta phải triển khai năm RDS instance độc lập. Ngoài cách tạo database riêng, PostgreSQL còn cho phép phân chia dữ liệu bằng các schema với quyền truy cập khác nhau trong cùng một database. Tuy nhiên, chương này chọn database riêng để làm rõ ranh giới kết nối và transaction. Các service không truy vấn trực tiếp dữ liệu của nhau và không sử dụng transaction xuyên domain.

Chẳng hạn, hotel-service đã commit giao dịch giữ phòng thành công, nhưng payment-service sau đó không thể kết nối tới cổng thanh toán. Dù hai service sử dụng chung một RDS instance, transaction của hotel cũng không tự động rollback. Tương tự, trạng thái Trip, tiến độ workflow và giao dịch với cổng thanh toán không nằm trong cùng một transaction. Vì vậy, chúng ta vẫn

cần Saga, idempotency, outbox và cơ chế đối soát để bảo đảm tính nhất quán giữa các service. Ngược lại, nếu toàn bộ nghiệp vụ có thể được xử lý trong một service với một transaction cục bộ, không cần thiết phải chia nhỏ hệ thống chỉ để áp dụng Saga.

Tuy nhiên, việc dùng chung một RDS instance cũng có những đánh đổi. Cả năm database cùng sử dụng CPU, bộ nhớ, I/O và giới hạn kết nối của instance. Một truy vấn tiêu tốn nhiều tài nguyên ở hotel-service có thể làm chậm cả payment-service. Các hoạt động bảo trì, sao lưu, khôi phục hoặc sự cố ở cấp instance cũng có thể ảnh hưởng đến toàn bộ hệ thống. **Như vậy, các service đã có quyền sở hữu dữ liệu riêng, nhưng vẫn chia sẻ tài nguyên và chưa có khả năng phục hồi độc lập.**

HÌNH 9.1 · SƠ ĐỒ LUỒNG



Khi xuất hiện nhu cầu cách ly tải, phục hồi riêng, tăng cường bảo mật hoặc triển khai theo chu kỳ độc lập, chúng ta có thể chuyển từng database sang RDS instance riêng. Phương án này làm tăng chi phí và công việc vận hành nhưng không nhất thiết phải thay đổi mô hình dữ liệu hay thiết kế Saga.

Cần phân biệt việc tách RDS với Multi-AZ. Tách RDS giúp các service có tài nguyên và phạm vi sự cố độc lập hơn, trong khi Multi-AZ chủ yếu cải thiện độ sẵn sàng của từng database deployment. Với cấu hình Multi-AZ DB instance có standby, AWS duy trì một bản dự phòng tại Availability

Zone khác để phục vụ failover, không phải một database riêng cho service khác hay read replica để chia tải đọc. Mô hình thực hành có thể bắt đầu với Single-AZ; khi triển khai production, cần lựa chọn cấu hình phù hợp với mục tiêu độ sẵn sàng và thời gian phục hồi (SLO). AWS — RDS Multi-AZ.

9.1.4 Lựa chọn cơ sở dữ liệu dựa trên những bảo đảm cần thiết

Nếu việc tách riêng các RDS instance làm tăng chi phí và công việc vận hành, liệu DynamoDB có phải là một phương án thay thế đáng cân nhắc? Câu trả lời là có, nhưng quyết định không nên chỉ dựa trên số lượng instance. Chúng ta cần xem xét đặc điểm truy cập dữ liệu, yêu cầu về transaction, khả năng phục hồi và tổng chi phí của từng phương án trước khi lựa chọn:

Phương án lưu trữ	Phù hợp khi	Đánh đổi cần cân nhắc
Một RDS PostgreSQL với database riêng cho từng service	Tải ban đầu ở mức vừa phải; ứng dụng cần SQL, các ràng buộc quan hệ và transaction quen thuộc.	Các service chia sẻ tài nguyên và phạm vi sự cố của instance; vẫn phát sinh chi phí duy trì, quản lý kết nối và cấu hình HA.
Nhiều RDS deployment, tách theo nhu cầu của từng domain	Vẫn muốn sử dụng PostgreSQL nhưng cần cách ly tài nguyên, phục hồi hoặc vận hành độc lập cho từng service.	Tăng chi phí duy trì và số lượng database deployment phải quản lý; vẫn cần Saga để phối hợp transaction giữa các service.
DynamoDB với bảng và quyền truy cập riêng cho từng domain	Có các mẫu truy cập dữ liệu (access pattern) theo khóa được xác định rõ; muốn sử dụng cơ sở dữ liệu serverless mà không phải quản lý connection pool.	Phải thiết kế partition key và index theo nhu cầu truy cập; không thể chuyển nguyên mô hình SQL, join và ràng buộc quan hệ sang DynamoDB. Cần đánh giá phân vùng quá tải (hot key), mức độ nhất quán dữ liệu và chi phí đọc/ghi.

Trong chương này, **RDS PostgreSQL là phương án lưu trữ chính** vì cho phép chúng ta tiếp tục sử dụng transaction cục bộ, các ràng buộc ngăn ghi trùng và cách tổ chức dữ liệu đã xây dựng ở những chương trước. Việc đặt năm database trên cùng một RDS instance cũng giúp mô hình minh họa đơn giản hơn, chưa cần triển khai và quản lý hạ tầng cơ sở dữ liệu riêng cho từng service. Đổi lại, các service phải chia sẻ tài nguyên của instance, đồng thời cần kiểm soát số lượng kết nối và mức độ thực thi đồng thời của Lambda để tránh gây quá tải.

Đọc thêm — nếu chọn DynamoDB thay cho PostgreSQL

Khi chuyển sang DynamoDB, những nguyên tắc về transaction cục bộ, idempotency và outbox vẫn được giữ nguyên, nhưng cách triển khai cụ thể sẽ khác. Chẳng hạn, `car-service` có thể sử dụng `TransactWriteItems` để ghi thông tin đặt xe, kết quả xử lý và một outbox item trong cùng một transaction. Các điều kiện trên khóa và phiên bản dữ liệu giúp ngăn đặt trùng hoặc kích hoạt lại một booking đã bị hủy. Tuy nhiên, transaction này không bao trùm cổng thanh toán hay SNS. Chúng ta cũng không nên tập trung quyền ghi dữ liệu của nhiều domain vào một function chỉ để thực hiện transaction chung. AWS — DynamoDB transactions.

DynamoDB Streams có thể kích hoạt Lambda khi dữ liệu thay đổi, cho phép relay chuyển các outbox item thành sự kiện mà không cần quét database định kỳ. Tuy nhiên, cơ chế này không loại bỏ khả năng phát sự kiện trùng lặp, chẳng hạn khi relay đã publish thành công nhưng chưa kịp ghi nhận kết

quả. Do DynamoDB Streams chỉ lưu giữ bản ghi trong 24 giờ, hệ thống cần giám sát độ trễ xử lý và có cơ chế khôi phục từ outbox được lưu trữ bền vững nếu quá trình chuyển tiếp bị gián đoạn kéo dài. AWS — DynamoDB Streams.

Cuối cùng, cần thiết kế cơ chế tra cứu và đối soát phù hợp với mức độ nhất quán dữ liệu (consistency). Một lần truy vấn chưa tìm thấy kết quả không có nghĩa thao tác trước đó chưa được thực hiện. DynamoDB hỗ trợ strongly consistent read trên bảng và local secondary index, trong khi global secondary index chỉ hỗ trợ eventual consistency. Vì vậy, đường tra cứu trạng thái của từng operation cần được xác định ngay từ đầu, thay vì giả định mọi truy vấn đều nhìn thấy dữ liệu mới nhất sau khi transaction được commit. AWS — Read consistency.

9.1.5 Vai trò và trách nhiệm của từng dịch vụ AWS

Trước khi đi vào sơ đồ triển khai, chúng ta hãy đối chiếu những yêu cầu đã phân tích ở Chương 6–8 với các dịch vụ AWS được sử dụng để hiện thực hóa chúng trong TravelNow. Bảng dưới đây giúp chúng ta xác định mỗi dịch vụ giải quyết vấn đề gì, cung cấp những bảo đảm nào và đâu là trách nhiệm mà ứng dụng vẫn phải tự đảm nhận:

Trách nhiệm trong kiến trúc	Dịch vụ AWS đảm nhận	Trách nhiệm của ứng dụng
Lưu tiến độ và điều phối Saga	AWS Step Functions	Xác định thứ tự thực thi, phân loại kết quả, thiết kế các bước bù trừ và cơ chế đối soát.
Tiếp nhận yêu cầu từ client	API Gateway HTTP API và Lambda của `trip-service`	Xác thực người dùng, kiểm soát quyền truy cập Trip, kiểm tra dữ liệu đầu vào và định nghĩa contract phản hồi.
Thực thi từng bước nghiệp vụ	Lambda của từng service	Hiện thực quy tắc nghiệp vụ, bảo đảm idempotency và quản lý transaction cục bộ.
Phân phối sự kiện đến nhiều bên nhận	Amazon SNS với các topic và subscription	Định nghĩa event contract, cấu hình bộ lọc và kiểm soát quyền công bố sự kiện.
Lưu giữ công việc cho từng consumer	Amazon SQS	Xác định điều kiện xác nhận xử lý thành công, giới hạn xử lý đồng thời, chính sách retry và cách xử lý thông điệp trong DLQ.
Kích hoạt xử lý các sự kiện còn trong outbox	EventBridge Scheduler và Lambda relay	Kiểm soát việc nhiều relay cùng nhận công việc, phát lại sự kiện, giám sát độ trễ và đối soát các công việc chưa hoàn tất.
Lưu dữ liệu nghiệp vụ và ý định phát sự kiện	Amazon RDS for PostgreSQL	Bảo đảm quyền sở hữu dữ liệu của từng service, thiết lập ràng buộc chống ghi trùng, quản lý chuyển trạng thái và triển khai transactional outbox.

API Gateway, Lambda, Step Functions, SNS/SQS và EventBridge Scheduler giúp chúng ta giảm bớt công việc tự xây dựng và vận hành các thành phần hạ tầng như API frontend, môi trường thực thi, workflow engine, message broker và bộ lập lịch. Tuy nhiên, **sử dụng dịch vụ được quản lý không có nghĩa ứng dụng không còn trách nhiệm vận hành**. Chúng ta vẫn phải cấu hình quyền truy cập, kiểm soát tải, thiết kế chính sách xử lý lỗi và giám sát tiến độ công việc.

Trong kiến trúc hiện tại, RDS PostgreSQL vẫn sử dụng tài nguyên được cấp phát trước. Vì vậy, việc triển khai toàn bộ tầng ứng dụng bằng Lambda không đồng nghĩa cả hệ thống đã trở thành serverless. Chúng ta có thể áp dụng mô hình serverless cho từng thành phần phù hợp, thay vì buộc toàn bộ ứng dụng phải chuyển đổi cùng lúc.

Ở các mục tiếp theo, chúng ta sẽ triển khai **hai phương án thay thế cho cùng một quy trình Saga**: orchestration và choreography. Cả hai sử dụng chung mô hình dữ liệu và quy tắc nghiệp vụ của TravelNow để thuận tiện so sánh. Với mỗi yêu cầu đặt chuyến, hệ thống chỉ sử dụng một phương án điều phối, tránh việc hai cơ chế cùng khởi chạy và kiểm soát một quy trình nghiệp vụ.

9.2 Triển khai Saga orchestration bằng Step Functions

9.2.1 Workflow trở thành một state machine được quản lý

Ở Chương 8, chúng ta đã biết orchestrator cần lưu lại tiến độ Saga, ghi nhận kết quả của từng bước và xác định công việc phải tiếp tục khi xảy ra gián đoạn. Với **AWS Step Functions**, chúng ta hiện thực hóa cơ chế này bằng một **state machine**, trong đó định nghĩa các bước xử lý, điều kiện chuyển tiếp và cách ứng phó với lỗi.

Mỗi yêu cầu đặt chuyến của TravelNow sẽ tạo ra một **execution** — một lượt thực thi state machine với dữ liệu và tiến độ riêng của chuyến đi đó.

Để hiểu rõ các khái niệm của Step Functions, hãy xem cách chúng được áp dụng vào quy trình đặt chuyến của TravelNow:

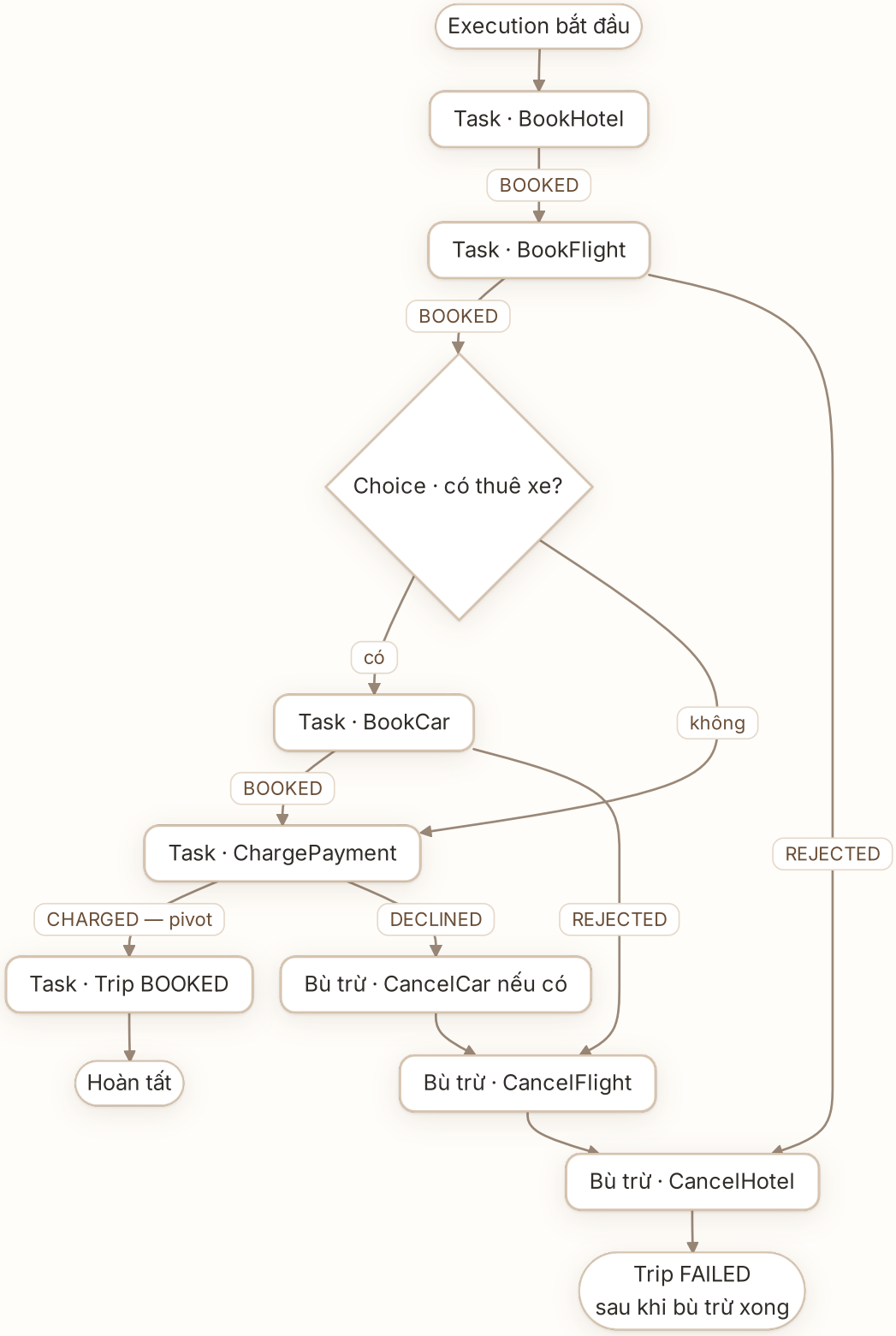
Thành phần của Step Functions	Vai trò trong quy trình đặt chuyến của TravelNow
`Task`	Thực thi một bước nghiệp vụ, chẳng hạn giữ phòng, giữ xe hoặc xử lý thanh toán.
`Choice`	Xác định bước tiếp theo dựa trên điều kiện hoặc kết quả xử lý, chẳng hạn chuyến đi có yêu cầu thuê xe hay không.
`Wait`	Tạm dừng workflow trước khi kiểm tra lại trạng thái của một thao tác chưa có kết quả rõ ràng.
`Retry` và `Catch`	Tự động thử lại những lỗi đã xác định hoặc chuyển sang nhánh xử lý lỗi tương ứng.
Execution history	Lưu lại tiến độ, kết quả và các bước đã thực hiện trong từng lượt chạy của workflow.

Qua tình huống đã giữ phòng thành công nhưng chưa xác định được kết quả thanh toán, chúng ta có thể thấy rõ vai trò của Step Functions: **lưu tiến độ và điều phối các bước tiếp theo của Saga**. Nhờ tiến độ được lưu trữ bền vững, workflow có thể tiếp tục sau khi xảy ra gián đoạn. Tuy nhiên, để xác định khách hàng đã bị trừ tiền hay chưa, workflow vẫn phải tra cứu kết quả từ service sở hữu giao dịch thanh toán. Đây là điểm khác biệt quan trọng giữa việc lưu trạng thái workflow và quản lý transaction nghiệp vụ.

TravelNow sử dụng **Standard Workflow** vì quy trình đặt chuyến cần lưu giữ tiến độ bền vững và có thể phải chờ, tra cứu kết quả hoặc thực hiện các bước bù trừ. Step Functions còn cung cấp Express Workflow cho những nhu cầu khác, với các bảo đảm thực thi riêng: *asynchronous Express* sử dụng cơ chế *at-least-once*, trong khi *synchronous Express* sử dụng *at-most-once*. Vì vậy, Standard và Express không đơn thuần là hai lựa chọn khác nhau về tốc độ xử lý. AWS — Các loại workflow.

Toàn bộ kịch bản đặt chuyến, khi viết thành state machine, có hình dạng như sau. Nhánh thành công đi từ trên xuống; mỗi lần thất bại nghiệp vụ rẽ sang cột bù trừ, hủy ngược những bước đã giữ.

HÌNH 9.2 · SƠ ĐỒ LUỒNG



Standard Workflow cung cấp bảo đảm exactly-once ở phạm vi thực thi workflow theo công bố của AWS, nhưng điều đó không có nghĩa mỗi thao tác nghiệp vụ chỉ được gọi đúng một lần. Một `Task` được cấu hình `Retry` vẫn có thể thực thi nhiều lần, và Step Functions cũng không tạo ra

transaction chung giữa RDS và cổng thanh toán. Do đó, **mỗi bước nghiệp vụ vẫn phải bảo đảm idempotency và có cơ chế xử lý kết quả chưa rõ**, như chúng ta đã phân tích ở Chương 7.

9.2.2 Kết nối workflow với các service của TravelNow

Kiến trúc Saga orchestration được minh họa trong sơ đồ dưới đây. Các mũi tên thể hiện kết nối giữa các thành phần, còn trình tự thực thi nghiệp vụ do Step Functions điều phối: **giữ phòng → đặt vé máy bay → thuê xe (nếu có yêu cầu) → thanh toán.**

Xem Hình 9.3 (sơ đồ khổ ngang) ở trang 155.

API Gateway tiếp nhận các yêu cầu HTTPS từ client, sử dụng JWT authorizer để xác thực token dựa trên `issuer`, `audience` và `scope` được cấu hình, sau đó chuyển yêu cầu đến Lambda của `trip-service`. Tuy nhiên, xác thực tại API Gateway không thay thế việc kiểm tra quyền nghiệp vụ: `trip-service` vẫn phải xác định người gọi có quyền truy cập hoặc thực hiện thao tác trên chuyến đi tương ứng hay không. Các Lambda nội bộ, chẳng hạn giữ phòng, hủy xe hoặc xử lý thanh toán, được Step Functions gọi thông qua quyền IAM và không cần công khai HTTP endpoint riêng. AWS — JWT authorizer.

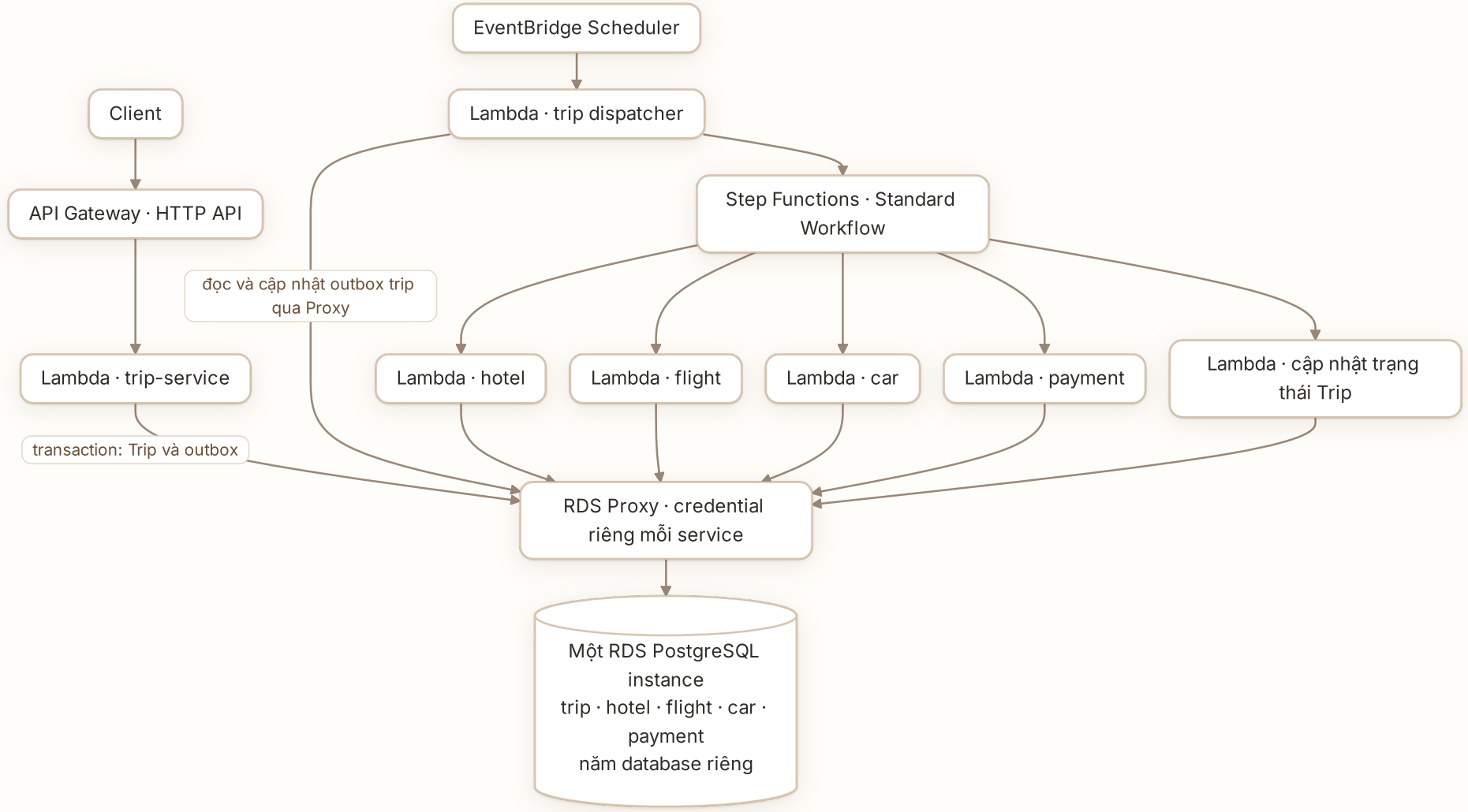
Trong sơ đồ, **RDS Proxy đóng vai trò quản lý và tái sử dụng kết nối** đến RDS instance dùng chung, chứ không phải một database mới hay một instance riêng cho từng Lambda function. Mỗi service vẫn sử dụng danh tính và quyền truy cập database của mình. RDS Proxy giúp hạn chế áp lực tạo kết nối khi Lambda mở rộng số lượt thực thi đồng thời, nhưng không làm tăng vô hạn năng lực xử lý của database, không tạo transaction chung giữa các service và cũng phát sinh chi phí riêng. Với môi trường thử nghiệm có mức độ đồng thời thấp, các function có thể kết nối trực tiếp đến RDS. Tuy nhiên, mô hình minh họa trong chương này sử dụng RDS Proxy để xem xét cả bài toán quản lý kết nối khi Lambda tự động co giãn (scale-in/scale-out theo tải). AWS — Lambda với RDS.

9.2.3 Một invocation trở thành bước nghiệp vụ như thế nào?

Khi workflow đến bước `BookCar`, Step Functions gọi Lambda của `car-service` với các thông tin cần thiết, bao gồm `tripId`, yêu cầu thuê xe và idempotency key. Lambda handler tiếp nhận dữ liệu, gọi logic nghiệp vụ của domain và trả kết quả cho workflow. Cách tổ chức này tương tự HTTP controller hoặc SQS consumer: mỗi thành phần đảm nhận việc tiếp nhận yêu cầu từ một nguồn khác nhau, trong khi logic nghiệp vụ có thể được tái sử dụng nếu đã được tách khỏi cơ chế giao tiếp. Các bước giữ phòng, đặt vé máy bay, thanh toán và bù trừ cũng được triển khai theo nguyên tắc tương tự.

Trong phương án này, chúng ta sử dụng **Lambda integration đồng bộ**: Step Functions chờ function thực thi xong và đọc kết quả từ `Payload` để quyết định bước tiếp theo. Nếu sử dụng `InvocationType: Event`, kết quả trả về chỉ xác nhận yêu cầu thực thi bất đồng bộ đã được chấp nhận, chứ chưa chứng minh thao tác giữ xe thành công. Với những công việc kéo dài hoặc phải chờ phản hồi từ đối tác, workflow cần có cơ chế theo dõi và tra cứu kết quả riêng. AWS — Lambda integration.

HÌNH 9.3 · SƠ ĐỒ LUỒNG

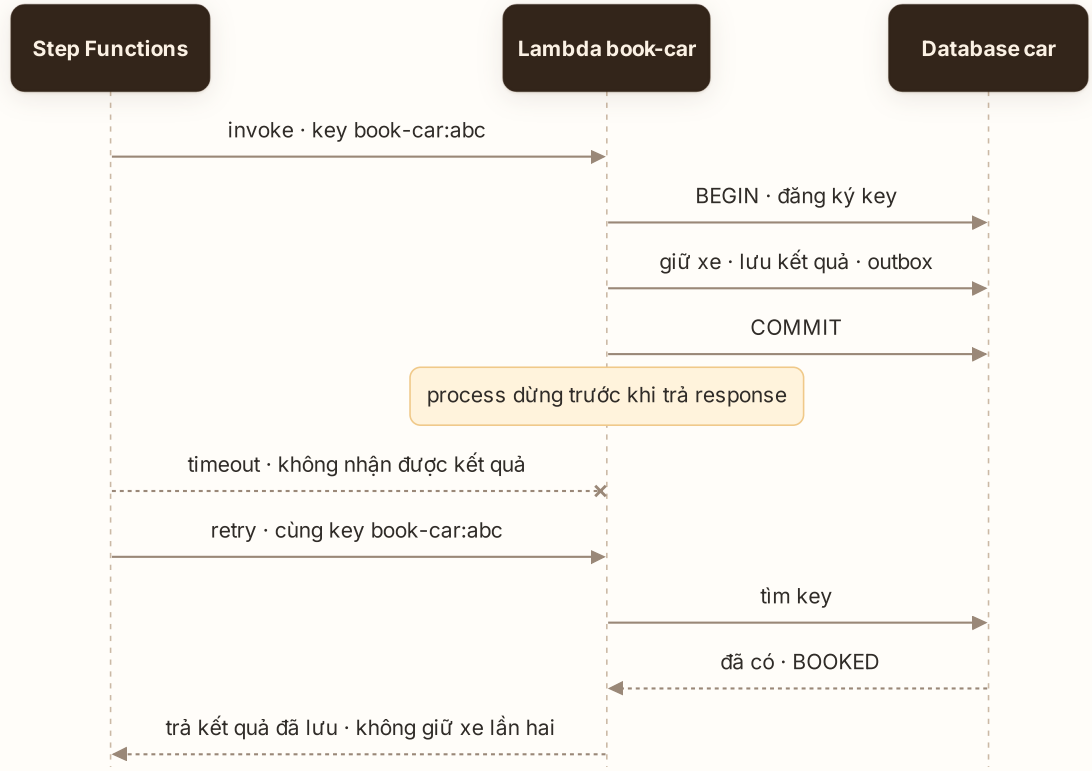


Đối với bước giữ xe, chúng ta áp dụng nguyên tắc **idempotency** đã phân tích ở Chương 7 thông qua một transaction cục bộ gồm ba bước:

1. **Đăng ký và kiểm tra idempotency key**. `car-service` ghi nhận một key, chẳng hạn `book-car:abc`, trong bảng lưu kết quả thao tác, với ràng buộc duy nhất theo phạm vi client và operation. Nếu key đã tồn tại, service kiểm tra payload để xác định đây có phải yêu cầu được gửi lại hay không, sau đó trả kết quả đã lưu hoặc trạng thái đang xử lý theo contract.
2. **Kiểm tra và cập nhật trạng thái booking**. Với yêu cầu mới, service kiểm tra trạng thái hiện tại trước khi ghi nhận kết quả giữ xe. Việc kiểm tra và cập nhật phải bảo đảm an toàn ngay cả khi lệnh đặt xe và lệnh hủy được xử lý đồng thời. Nếu booking đã được đánh dấu `CANCELLED`, những yêu cầu đặt xe đến muộn không được phép tạo lại hoặc chuyển booking về trạng thái đã đặt. Service có thể sử dụng ràng buộc duy nhất (unique constraint) kết hợp với câu lệnh cập nhật có điều kiện (conditional update) để ngăn các thao tác cạnh tranh làm sai lệch trạng thái booking.
3. **Lưu kết quả và commit transaction**. Kết quả idempotency được ghi cùng các thay đổi nghiệp vụ trong một transaction. Nếu cần phát sự kiện, service cũng ghi bản ghi outbox trong transaction này. Chỉ sau khi commit thành công, function mới trả kết quả cho Step Functions.

Cơ chế trên giúp bảo vệ các thay đổi nghiệp vụ khi function gặp sự cố. Nếu `car-service` dừng sau khi commit nhưng trước khi trả response, lần gọi lại vẫn có thể tìm thấy kết quả đã lưu trong RDS mà không thực hiện lại thao tác giữ xe. Ngược lại, nếu transaction chưa commit, những thay đổi của lần xử lý đó sẽ không có hiệu lực. **Step Functions có thể gọi lại một bước, nhưng chính transaction và idempotency của service mới bảo đảm thao tác nghiệp vụ không bị thực hiện trùng**.

HÌNH 9.4 · SƠ ĐỒ TUẦN TỰ



Trong mô hình TravelNow hiện tại, thao tác giữ xe được mô phỏng bằng dữ liệu do `car-service` sở hữu, như giả định từ Chương 1. Khi tích hợp với một hãng xe thực tế, transaction trong RDS không thể bao trùm lời gọi đến hệ thống của đối tác. Khi đó, service cần lưu ý định thực hiện, sử dụng cơ chế idempotency do đối tác hỗ trợ và đối soát những giao dịch chưa xác định được kết quả. Không nên duy trì một transaction database trong suốt thời gian chờ phản hồi qua mạng.

9.2.4 Cấu hình timeout, retry và nhánh phục hồi

Step Functions sử dụng **Amazon States Language (ASL)** để định nghĩa các bước thực thi, điều kiện chuyển tiếp và cách xử lý lỗi của workflow. Đoạn mã dưới đây minh họa cách cấu hình timeout, retry và nhánh phục hồi cho state `BookCar` trong một state machine sử dụng JSONPath.

ARN của Lambda function version chỉ mang tính minh họa và cần được thay bằng ARN của tài nguyên thực tế. Hai state `EvaluateCarResult` và `ResolveCarResult` cũng cần được định nghĩa khi xây dựng workflow hoàn chỉnh.

```
{
  "BookCar": {
    "Type": "Task",
    "Resource": "arn:aws:states:::lambda:invoke",
    "Parameters": {
      "FunctionName": "arn:aws:lambda:ap-southeast-1:123456789012:function:book-car:7",
      "Payload": {
        "tripId.$": "$.tripId",
        "operationKey.$": "$.carOperationKey",
        "car.$": "$.car"
```

```
      }
    },
    "TimeoutSeconds": 15,
    "ResultSelector": { "result.$": "$.Payload" },
    "ResultPath": "$.carTask",
    "Retry": [{
      "ErrorEquals": ["Lambda.ServiceException", "Lambda.TooManyRequestsException"],
      "IntervalSeconds": 1,
      "BackoffRate": 2,
      "MaxAttempts": 2,
      "JitterStrategy": "FULL"
    }],
    "Catch": [{
      "ErrorEquals": ["States.ALL"],
      "ResultPath": "$.carError",
      "Next": "ResolveCarResult"
    }],
    "Next": "EvaluateCarResult"
  }
}
```

Trong ví dụ này, idempotency key được tạo trước khi thực thi và giữ nguyên qua tất cả các lần retry. Cần lưu ý rằng `MaxAttempts: 2` trong Step Functions cho phép **hai lần thử lại sau lần gọi đầu tiên**, tức tối đa ba lượt thực thi. Cách tính này khác với biến `maxAttempts` trong mã giả ở Chương 7, vốn biểu thị tổng số lượt thử. Các giá trị timeout và retry trong ví dụ chỉ mang tính minh họa; khi triển khai thực tế, chúng cần được điều chỉnh phù hợp với thời gian xử lý của Lambda và ngân sách thời gian của toàn bộ Saga. AWS — Retry và Catch.

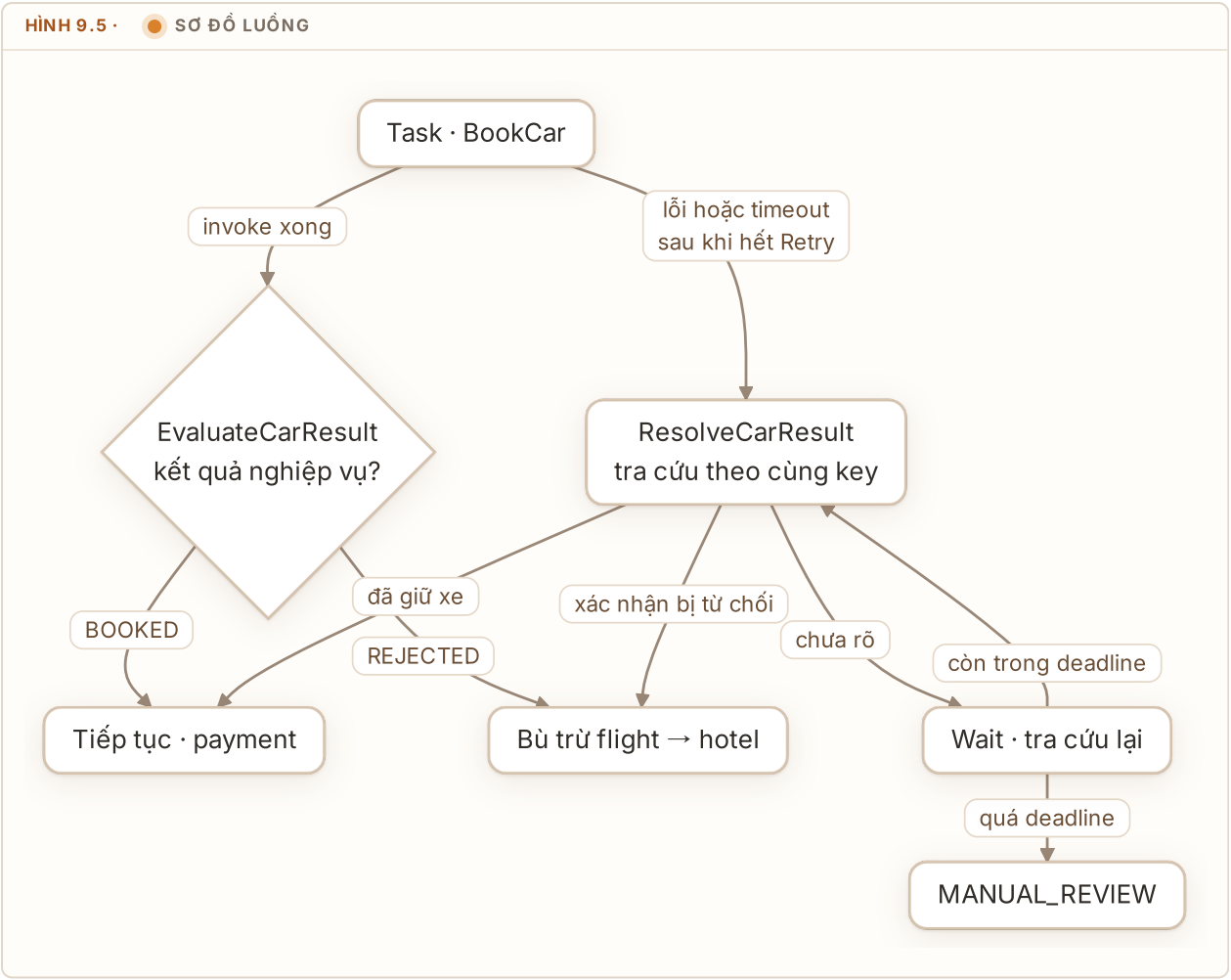
Sau khi Lambda thực thi thành công, `EvaluateCarResult` vẫn phải kiểm tra kết quả nghiệp vụ trong `$.carTask.result` để quyết định bước tiếp theo. Chẳng hạn, `car-service` có thể trả về `REJECTED` vì không còn xe, dù function đã chạy bình thường và không phát sinh lỗi kỹ thuật. Trong trường hợp này, Saga cần chuyển sang nhánh bù trừ thay vì gọi lại function. Nếu khách hàng không yêu cầu thuê xe, một state `Choice` trước `BookCar` sẽ bỏ qua bước này và chuyển thẳng đến thanh toán.

Khi `BookCar` gặp lỗi hoặc timeout, workflow chuyển sang `ResolveCarResult` để tra cứu trạng thái thao tác bằng cùng định danh đã sử dụng trước đó. Tùy theo kết quả tra cứu, Saga sẽ tiếp tục hoặc thực hiện các bước phục hồi tương ứng:

- **Đã giữ xe thành công**: Tiếp tục thực hiện các bước tiếp theo của Saga.
- **Đã xác nhận giữ xe thất bại**: Thực hiện bù trừ theo thứ tự ngược lại, trước tiên là flight, sau đó là hotel.
- **Chưa xác định được kết quả**: Chờ và tiếp tục tra cứu trong thời hạn cho phép. Nếu vẫn không thể xác định trạng thái, chuyển Trip sang `MANUAL_REVIEW` để xử lý thủ công.

Điểm quan trọng là `Catch` **chỉ cho biết Task gặp lỗi, không chứng minh thao tác nghiệp vụ chưa xảy ra**. Tương tự, một Task bị timeout không đồng nghĩa Lambda đã ngừng thực thi. Vì vậy, workflow phải tra cứu trạng thái thực tế trước khi quyết định thử lại, tiếp tục hay bù trừ.

HÌNH 9.5 · SƠ ĐỒ LUỒNG



Đối với bước thanh toán, nguyên tắc này càng quan trọng. Một khi đã xác nhận thu tiền thành công, Saga cần ưu tiên phục hồi theo hướng hoàn tất chuyến đi, tương tự *pivot transaction* đã phân tích ở Chương 8. Những lỗi xảy ra sau đó, chẳng hạn không thể tạo PDF xác nhận hoặc cập nhật Trip sang `BOOKED`, phải được xử lý mà không thực hiện lại giao dịch thu tiền. Các Task bù trừ cũng cần idempotency key riêng, chính sách retry và cơ chế theo dõi những thao tác chưa hoàn tất.

Tuy nhiên, không phải mọi lỗi đều được xử lý bởi `Catch`. Một số lỗi ở cấp execution hoặc lỗi runtime liên quan đến định nghĩa workflow có thể khiến toàn bộ execution kết thúc trước khi ứng dụng kịp ghi nhận trạng thái nghiệp vụ mong muốn. Do đó, TravelNow còn cần giám sát những execution bị lỗi hoặc quá hạn, đồng thời định kỳ đối chiếu các Trip đang chờ với execution tương ứng.

Cần phân biệt **trạng thái execution của Step Functions với trạng thái nghiệp vụ của Trip**. Một execution có trạng thái `FAILED` không có nghĩa Trip đã hoàn tất quá trình xử lý thất bại. Trip chỉ nên được ghi nhận thất bại hoàn tất sau khi các thao tác bù trừ cần thiết đã được xử lý thành công.

9.2.5 Khởi chạy workflow và trả kết quả cho khách hàng

Một ranh giới phân tán xuất hiện ngay từ thời điểm khởi chạy Saga: `trip-service` ghi nhận Trip ở trạng thái `PENDING`, sau đó gọi API `StartExecution` để bắt đầu workflow. Tuy nhiên, RDS và Step Functions không tham gia cùng một transaction. Nếu service dừng sau khi lưu Trip nhưng trước khi gọi `StartExecution`, yêu cầu đặt chuyến đã được tiếp nhận nhưng chưa có workflow nào xử lý.

Để giải quyết vấn đề này, chúng ta sử dụng **Transactional Outbox**, ghi nhận Trip và ý định khởi chạy workflow trong cùng một transaction. Sau khi transaction được commit, một dispatcher đọc các ý định còn chờ, gọi `StartExecution` bằng tên execution ổn định gắn với yêu cầu đặt chuyến, rồi lưu thông tin liên kết giữa Trip và execution tương ứng.

Với Standard Workflow, việc gọi lại `StartExecution` bằng cùng tên và input khi execution vẫn đang chạy có tính idempotent. Nếu execution đã kết thúc, lời gọi trùng sẽ trả về lỗi và dispatcher cần đối chiếu với execution đã được tạo trước đó. Ngoài ra, Step Functions chỉ duy trì cơ chế chống trùng tên trong một khoảng thời gian nhất định. Vì vậy, TravelNow vẫn phải lưu thông tin nghiệp vụ và thực thi quy tắc chống tạo trùng theo contract của ứng dụng, thay vì phụ thuộc hoàn toàn vào tên execution. AWS — StartExecution.

Trong kiến trúc minh họa, dispatcher cũng là một Lambda thuộc `trip-service`. **EventBridge Scheduler kích hoạt dispatcher mỗi phút** để kiểm tra những ý định khởi chạy chưa được xử lý. Mỗi lần chạy, dispatcher nhận một batch có giới hạn và sử dụng lease được lưu trữ bền vững để tránh nhiều lượt xử lý cùng nhận một công việc. Lease xác định lượt xử lý nào đang giữ quyền thực hiện và thời điểm quyền đó hết hạn. Nếu dispatcher dừng giữa chừng, lượt chạy tiếp theo có thể nhận lại công việc sau khi lease hết hạn. Transaction database phải được đóng trước khi gọi `StartExecution`, thay vì duy trì trong suốt thời gian chờ phản hồi từ AWS.

Phương án polling này tương đối đơn giản nhưng có độ trễ: một yêu cầu mới có thể phải chờ đến lượt quét tiếp theo, trong khi EventBridge Scheduler không bảo đảm kích hoạt chính xác đến từng giây. Nếu yêu cầu về thời gian phản hồi đòi hỏi workflow bắt đầu sớm hơn, `trip-service` có thể chủ động gọi `StartExecution` ngay sau khi commit, sử dụng chính tên và input đã lưu trong outbox. Dispatcher định kỳ khi đó đóng vai trò phục hồi những yêu cầu chưa được khởi chạy thành công.

Dù áp dụng cách nào, lời gọi `StartExecution` vẫn độc lập với transaction trong RDS. **Outbox bảo đảm ý định khởi chạy không bị mất nếu lời gọi trực tiếp thất bại hoặc service dừng đột ngột**. Không nên dựa vào một thread chạy nền sau khi HTTP handler trả response để bảo đảm công việc sẽ được thực hiện. AWS — EventBridge Scheduler.

Về phía khách hàng, HTTP request không cần chờ toàn bộ Saga hoàn tất. Ngay sau khi yêu cầu được lưu trữ, `trip-service` trả về `202 Accepted`, kèm `tripId` và header `Location` trỏ đến API tra cứu trạng thái chuyến đi. Client có thể định kỳ gọi API này hoặc nhận cập nhật chủ động qua SSE/WebSocket nếu ứng dụng có nhu cầu.

Trong cả hai trường hợp, trạng thái Trip được lưu lại trong `trip-service`, cho phép khách hàng đóng trình duyệt và quay lại kiểm tra kết quả sau đó. Giao diện cần phân biệt rõ **yêu cầu đã được tiếp nhận** với **chuyến đi đã được đặt thành công**: phản hồi `202 Accepted` chỉ xác nhận hệ thống đã ghi nhận yêu cầu và sẽ tiếp tục xử lý, không phải cam kết rằng toàn bộ quy trình đặt chuyến đã hoàn tất. Microsoft — Asynchronous Request-Reply.

Ghép các phần trên lại, một yêu cầu đặt chuyến đi qua hai chặng tách biệt: chặng trả lời khách và chặng giao việc cho workflow.

Xem Hình 9.6 (sơ đồ khổ ngang) ở trang 162.

9.2.6 Cập nhật workflow khi các execution cũ chưa kết thúc

Định nghĩa workflow, cấu hình retry và quyền truy cập của từng Task cần được quản lý bằng IaC, tương tự cách chúng ta đã thực hiện ở Phần II. Step Functions hỗ trợ **version bất biến và alias**, cho phép triển khai phiên bản mới và điều hướng các execution mới mà không làm thay đổi định nghĩa của những execution đang chạy.

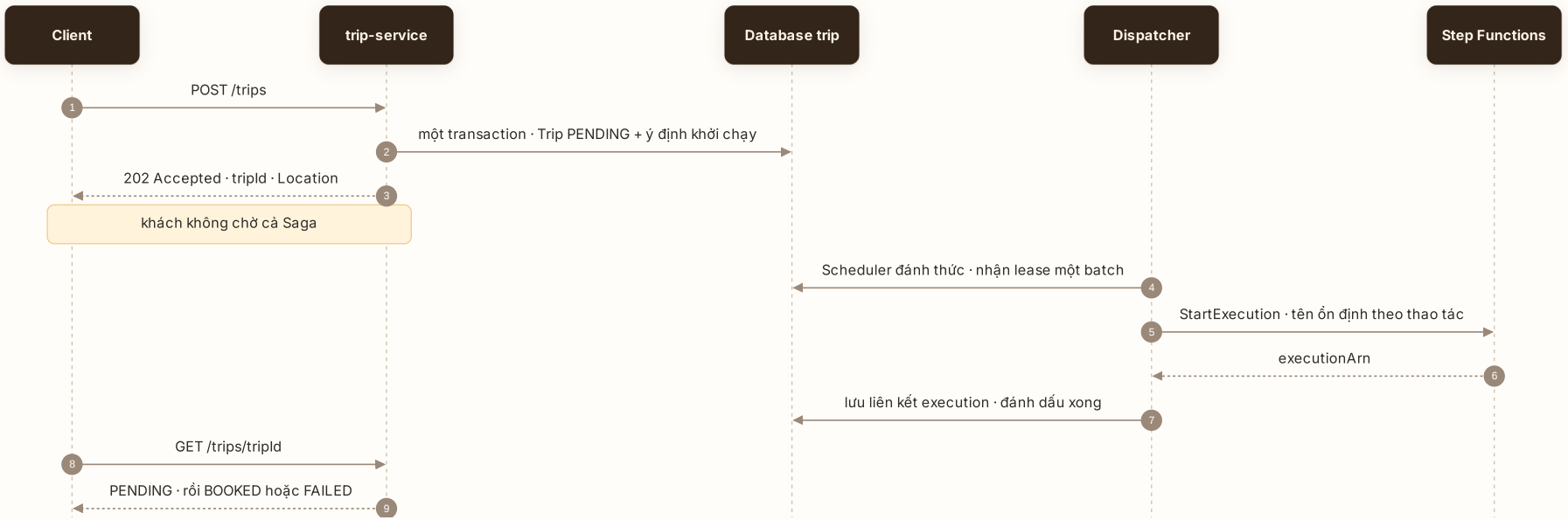
Trong thiết kế TravelNow, dispatcher lưu ARN của workflow version đã chọn cùng với ý định khởi chạy trong outbox. Nhờ đó, nếu phải gọi lại `StartExecution`, hệ thống vẫn sử dụng đúng phiên bản ban đầu thay vì vô tình chuyển sang workflow mới. Nếu chỉ lưu ARN của alias, chúng ta cần xử lý trường hợp alias đã được chuyển sang version khác giữa các lần thử. Những execution đã khởi chạy vẫn tiếp tục sử dụng định nghĩa workflow ban đầu. AWS — Version và alias.

Tuy nhiên, **cố định phiên bản workflow không đồng nghĩa với cố định phiên bản của tất cả thành phần mà nó gọi**. Chẳng hạn, nếu một Task sử dụng Lambda alias và alias này được chuyển sang phiên bản code mới, execution cũ vẫn có thể gọi code mới dù định nghĩa workflow không thay đổi.

Vì vậy, khi triển khai phiên bản mới, chúng ta cần sử dụng Lambda version bất biến hoặc duy trì contract tương thích cho toàn bộ các thao tác đặt, hủy và tra cứu trạng thái. Các thay đổi về database schema cũng phải tương thích với những execution cũ trong suốt thời gian chúng còn hoạt động hoặc cần phục hồi.

Một đợt triển khai chỉ thực sự hoàn tất khi cả luồng xử lý thành công lẫn các nhánh phục hồi của những execution chưa kết thúc vẫn được hỗ trợ.

HÌNH 9.6 · SƠ ĐỒ TUẦN TỰ



9.3 Triển khai Saga choreography bằng SNS và SQS

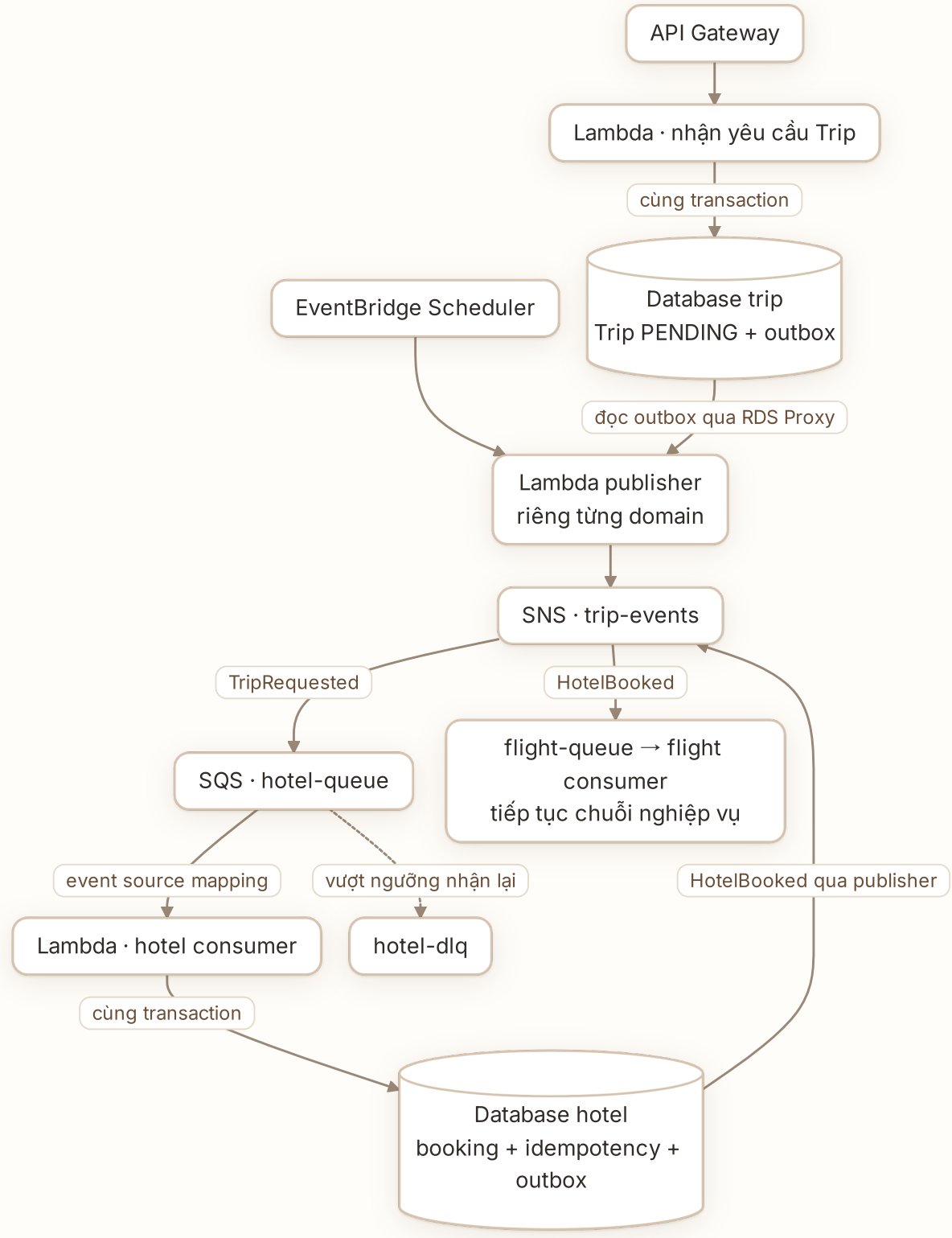
9.3.1 Một sự kiện được phân phối, mỗi consumer có hàng đợi riêng

Trong phương án choreography, các service phối hợp với nhau thông qua việc phát hành và xử lý sự kiện, thay vì được điều phối bởi một workflow trung tâm. **Amazon SNS** đảm nhận việc phân phối sự kiện từ một topic đến các subscription phù hợp, trong khi **Amazon SQS** lưu giữ message để từng consumer có thể xử lý theo tiến độ riêng. Kết hợp hai dịch vụ này giúp hiện thực hóa hai yêu cầu đã phân tích ở Chương 8: phân phối một sự kiện đến nhiều bên (fan-out) và duy trì hàng đợi bền vững cho từng bên nhận.

Ví dụ, khi `hotel-service` tạm thời ngừng hoạt động, SQS vẫn giữ các message để service tiếp tục xử lý sau khi phục hồi. Ngược lại, khi TravelNow bổ sung một service mới cần nhận cùng sự kiện, SNS có thể phân phối sự kiện đến hàng đợi của service đó thông qua một subscription mới. Qua đó, chúng ta thấy rõ **phân phối sự kiện và lưu giữ công việc là hai trách nhiệm khác nhau**. Một nền tảng messaging khác có thể cung cấp cả hai khả năng trong cùng một sản phẩm, nhưng vẫn cần xem xét những bảo đảm mà nền tảng đó cung cấp cho từng trách nhiệm.

Sơ đồ dưới đây tập trung vào nhánh `hotel-service` để minh họa đường đi của một sự kiện, từ khi được phát hành đến khi consumer tiếp nhận và xử lý. Các service còn lại, bao gồm `flight`, `car`, `payment` và `trip`, cũng được tổ chức theo nguyên tắc tương tự: mỗi service có hàng đợi và consumer riêng, đồng thời chỉ đọc và ghi dữ liệu thuộc phạm vi sở hữu của mình.

HÌNH 9.7 · SƠ ĐỒ LUỒNG



Mỗi queue cần có chính sách xử lý lỗi và dead-letter queue (DLQ) riêng, tương tự `hotel-dlq` trong sơ đồ. Năm database logic của TravelNow vẫn được triển khai trên cùng RDS instance đã chọn ở Mục 9.1, với các kết nối thông qua RDS Proxy. Publisher của mỗi service sử dụng danh

tính database riêng để đọc outbox thuộc phạm vi sở hữu của mình. Trong sơ đồ, mũi tên `HotelBooked` biểu thị việc publisher của `hotel-service` phát hành sự kiện lên SNS, chứ không phải database trực tiếp gửi message.

Luồng xử lý bắt đầu khi `trip-service` ghi nhận `Trip PENDING` và sự kiện `TripRequested` vào outbox trong cùng một transaction. Sau khi transaction được commit, publisher phát hành sự kiện lên SNS. Subscription tương ứng chuyển message vào `hotel-queue`, nơi hotel consumer tiếp nhận và thực hiện thao tác giữ phòng. Khi giữ phòng thành công, `hotel-service` ghi sự kiện `HotelBooked` vào outbox của mình. Sự kiện này tiếp tục được phát hành lên SNS và chuyển đến hàng đợi của `flight-service` để bắt đầu bước giữ vé máy bay.

Quy trình tiếp tục qua các bước thuê xe và thanh toán, trong khi `trip-service` tiếp nhận những sự kiện liên quan để theo dõi và cập nhật tiến độ của chuyến đi. Nếu khách hàng không yêu cầu thuê xe, luồng xử lý sử dụng sự kiện `CarSkipped` theo contract đã thiết kế ở Chương 8.

Để mỗi consumer chỉ nhận những sự kiện cần thiết, SNS hỗ trợ **lọc message theo** `eventType`, thông qua message attributes hoặc nội dung message, tùy theo cấu hình `FilterPolicyScope`. Producer và subscription phải thống nhất vị trí của trường này trong event contract. Đồng thời, queue policy phải cho phép đúng SNS topic gửi message vào hàng đợi. Chỉ tạo topic và queue là chưa đủ; subscription, bộ lọc và quyền truy cập đều phải được cấu hình chính xác để sự kiện có thể đi từ producer đến consumer. AWS — SNS message filtering.

Ngoài SNS, **EventBridge event bus** cũng là một lựa chọn cho lớp phân phối sự kiện, đặc biệt khi hệ thống cần các quy tắc định tuyến linh hoạt hoặc tích hợp nhiều nguồn sự kiện. Tuy nhiên, dù sử dụng nền tảng nào, chúng ta vẫn phải xem xét riêng cơ chế lưu giữ message, retry và xác nhận xử lý. Trong phạm vi TravelNow, kết hợp SNS và SQS giúp minh họa rõ sự phân tách giữa phân phối sự kiện và xử lý bất đồng bộ.

Một vấn đề khác cần lưu ý là **producer và consumer không nhất thiết được triển khai cùng thời điểm**. Vì vậy, event contract phải duy trì khả năng tương thích trong quá trình nâng cấp. Chẳng hạn, nếu muốn đổi tên trường `checkIn` thành `checkInDate`, chúng ta có thể áp dụng chiến lược *expand-migrate-contract*:

1. **Expand**: Nâng cấp consumer để chấp nhận cả `checkIn` và `checkInDate`.
2. **Migrate**: Chuyển producer sang định dạng mới, đồng thời tiếp tục xử lý những sự kiện cũ còn tồn tại trong queue, outbox hoặc DLQ.
3. **Contract**: Chỉ loại bỏ khả năng hỗ trợ `checkIn` sau khi xác nhận các sự kiện cũ đã được xử lý và không còn cần phát lại.

Contract test cần kiểm tra cả định dạng sự kiện cũ lẫn mới. Nếu chỉ kiểm thử phiên bản mới nhất của producer và consumer cùng lúc, chúng ta có thể bỏ sót những lỗi tương thích xuất hiện trong giai đoạn hai bên được nâng cấp lệch thời điểm.

9.3.2 Outbox nối transaction với việc phát hành sự kiện

Hãy xem xét `flight-service`. Nếu service chỉ ghi booking vào database rồi gọi SNS để phát `FlightBooked`, một khoảng trống có thể xuất hiện giữa hai thao tác: transaction đã commit nhưng process dừng trước khi sự kiện được gửi. Khi đó, vé đã được giữ nhưng các service phía sau không nhận được `FlightBooked` để tiếp tục Saga.

Để tránh tình huống này, `flight-service` ghi **booking, kết quả idempotency và bản ghi `outbox_events` trong cùng một transaction PostgreSQL**. Sau khi transaction được commit, publisher đọc những bản ghi outbox chưa được gửi, phát sự kiện lên SNS và chỉ đánh dấu chúng đã gửi sau khi nhận được xác nhận từ SNS.

Xem Hình 9.8 (sơ đồ khổ ngang) ở trang 167.

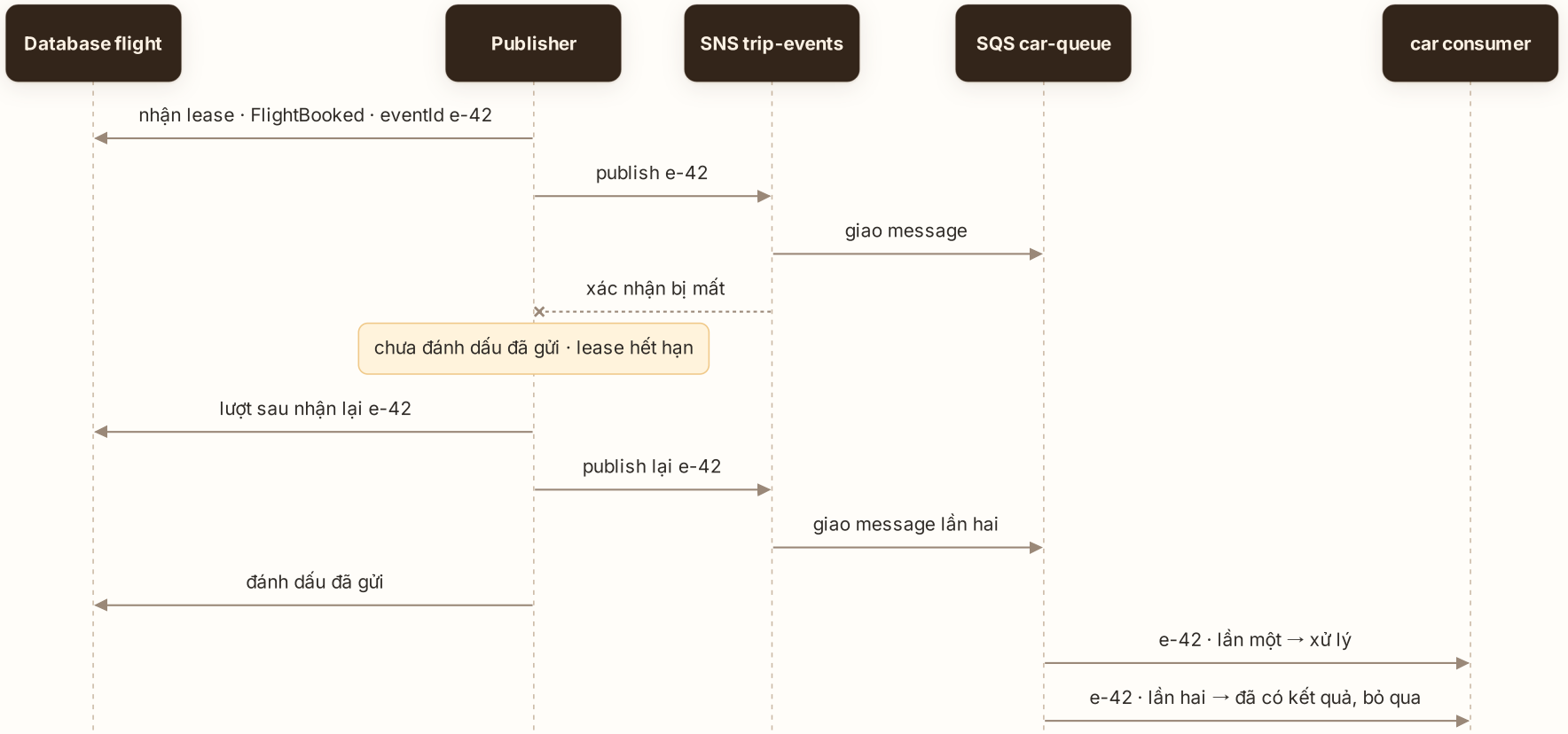
Tuy nhiên, publisher vẫn có thể gặp trường hợp SNS đã nhận sự kiện nhưng phản hồi xác nhận bị mất. Khi đó, lần xử lý tiếp theo có thể phát lại cùng sự kiện với cùng `eventId`. Đây là hành vi bình thường trong mô hình **at-least-once**. Consumer phải sử dụng định danh sự kiện hoặc thao tác, kết hợp với transaction cục bộ, để nhận biết message đã được xử lý và không tạo thêm booking hay side effect trùng lặp. Ngược lại, việc SNS xác nhận đã nhận sự kiện chỉ chứng minh sự kiện đã đến SNS, không có nghĩa tất cả consumer đã xử lý thành công. Vì vậy, hệ thống vẫn cần theo dõi lỗi delivery của subscription, backlog của queue và trạng thái của Saga. AWS — Transactional outbox.

Trong mô hình TravelNow, publisher được EventBridge Scheduler kích hoạt định kỳ và xử lý một batch outbox có giới hạn, tương tự dispatcher ở Mục 9.2. Khi nhiều invocation chạy chồng lên nhau, *lease* giúp xác định publisher nào đang giữ quyền xử lý từng bản ghi. Publisher chỉ đánh dấu sự kiện đã gửi sau khi SNS xác nhận và khi nó vẫn còn sở hữu lease. Nếu publisher dừng sau khi phát sự kiện nhưng trước khi cập nhật trạng thái outbox, lần chạy tiếp theo sẽ phát lại cùng `eventId`. **Lease giúp phân công công việc giữa các publisher, còn idempotency ở consumer mới là cơ chế bảo vệ side effect khi sự kiện được gửi lặp lại.**

Polling định kỳ có ưu điểm đơn giản nhưng cũng tạo thêm độ trễ ở từng chặng của choreography. Nếu publisher chỉ chạy mỗi phút, chúng ta không thể mặc định một chuỗi gồm nhiều service sẽ hoàn tất trong vài giây. Một cách cải thiện là thử phát sự kiện ngay sau khi transaction commit, trong khi publisher định kỳ vẫn đóng vai trò phục hồi những bản ghi bị bỏ sót. Cả hai đường xử lý phải sử dụng cùng `eventId` và cùng cơ chế nhận việc để tránh tạo ra hai luồng phát hành độc lập.

Khi hệ thống yêu cầu độ trễ thấp ổn định hoặc throughput lớn hơn, chúng ta có thể cân nhắc cơ chế relay dựa trên **Change Data Capture (CDC)**: theo dõi những thay đổi đã được commit trong database và chuyển chúng thành công việc cho thành phần phân phối sự kiện. Với DynamoDB, Streams cung cấp sẵn một cơ chế theo hướng này; còn với PostgreSQL, CDC đòi hỏi thêm cấu hình và các thành phần vận hành.

HÌNH 9.8 · SƠ ĐỒ TUẦN TỰ



Việc chọn RDS giúp TravelNow giữ được mô hình transaction quan hệ quen thuộc, nhưng đồng thời đặt ra một câu hỏi phải được trả lời rõ trong thiết kế: thành phần nào chịu trách nhiệm chuyển các bản ghi outbox thành sự kiện, và sự kiện được chuyển đi nhanh đến mức nào.

9.3.3 Lambda nhận và xử lý message từ SQS

Trong phương án choreography này, Step Functions không còn đóng vai trò điều phối luồng đặt chuyến. Thay vào đó, **Lambda event source mapping** liên tục polling SQS queue và gọi consumer Lambda với một batch message. Handler không cần tự triển khai vòng lặp `ReceiveMessage`; nhiệm vụ của nó là chuyển từng message thành thao tác nghiệp vụ, xử lý chúng và trả về kết quả của batch. AWS — Lambda với SQS.

Khi một message được nhận để xử lý, SQS tạm thời ẩn nó khỏi các consumer khác trong khoảng **visibility timeout**. Consumer phải commit các thay đổi nghiệp vụ, kết quả idempotency và bản ghi outbox trước khi báo xử lý thành công. Sau đó, event source mapping mới xóa message khỏi queue. Nếu Lambda dừng hoặc thất bại trước khi hoàn tất, message có thể xuất hiện trở lại và được giao cho một invocation khác. Với SQS Standard, message vẫn có thể được giao trùng, vì vậy visibility timeout chỉ giúp hạn chế xử lý đồng thời chứ không tạo ra một khóa độc quyền hay bảo đảm exactly-once. AWS — Visibility timeout.

Ví dụ, giả sử `car-service` đã commit booking cùng sự kiện `CarBooked` vào outbox nhưng Lambda gặp lỗi trước khi báo xử lý thành công. Message sau đó có thể được giao lại. Ở lần xử lý tiếp theo, consumer sử dụng cùng idempotency key để tìm kết quả đã lưu thay vì thực hiện thêm một lần giữ xe. Đồng thời, `CarBooked` vẫn có thể được publisher phát hành vì ý định phát sự kiện đã được lưu bền trong RDS, không phụ thuộc vào bộ nhớ hay vòng đời của invocation trước đó.

Khi Lambda nhận một batch gồm cả message xử lý thành công và message thất bại, event source mapping nên bật `ReportBatchItemFailures`. Handler trả về `batchItemFailures` chứa `messageId` của những item chưa xử lý thành công để chỉ các message đó được giao lại. Nếu không cấu hình partial batch response hoặc để toàn bộ invocation thất bại, cả batch có thể được xử lý lại. Ngược lại, nếu handler bắt exception nhưng vẫn báo toàn bộ batch thành công, những message chưa thực sự được xử lý có thể bị xóa khỏi queue và làm mất công việc.

Partial batch response giúp giảm những lần xử lý lại không cần thiết, nhưng không thay thế idempotency. Ngay cả khi chỉ những message thất bại được đưa trở lại queue, consumer vẫn phải chịu được việc cùng một message được giao nhiều lần. AWS — SQS partial batch response.

Xem Hình 9.9 (sơ đồ khổ ngang) ở trang 170.

9.3.4 Khả năng co giãn phải đi cùng kiểm soát luồng

Lambda có thể tự động tăng số lượng invocation để xử lý backlog trong SQS. Tuy nhiên, khả năng mở rộng của consumer không có nghĩa các thành phần phía sau cũng có thể mở rộng tương ứng. Trong TravelNow, cả năm domain vẫn chia sẻ capacity của cùng một RDS instance.

Nếu mỗi invocation đồng thời mở kết nối database hoặc gọi đến một đối tác có giới hạn lưu lượng, việc tăng concurrency quá nhanh có thể làm các tài nguyên này quá tải, kéo theo nhiều lỗi và retry hơn.

Vì vậy, khả năng co giãn cần đi kèm với **kiểm soát concurrency và tốc độ xử lý**. Chúng ta cần phối hợp `reserved concurrency` của Lambda, `maximum concurrency` của SQS event source mapping, `batch size` và `visibility timeout`. Khi xác định các giới hạn này, phải tính đến toàn bộ workload cùng sử dụng RDS, bao gồm request từ API, các Task được Step Functions gọi và các publisher, chứ không chỉ riêng SQS consumer. RDS Proxy giúp quản lý và tái sử dụng kết nối hiệu quả hơn, nhưng không loại bỏ các giới hạn thực tế của database như CPU, I/O và lock contention. AWS — SQS scaling.

Bên cạnh độ dài của queue, hệ thống cần theo dõi **tuổi của message cũ nhất** và **tuổi của các bản ghi outbox chưa được phát hành**. Những chỉ số này cho biết công việc đang tồn đọng bao lâu và liệu chúng còn có khả năng hoàn tất trong deadline nghiệp vụ hay không.

Giới hạn concurrency chỉ kiểm soát số lượng công việc được xử lý đồng thời; nó không tự làm producer gửi chậm lại. Vì vậy, TravelNow vẫn cần thiết kế chính sách tiếp nhận yêu cầu, backpressure và thời hạn xử lý phù hợp với năng lực thực tế của hệ thống. **Queue có thể giữ message an toàn trong lúc hệ thống quá tải, nhưng điều đó không có nghĩa yêu cầu đặt chuyến vẫn còn giá trị khi đến lượt được xử lý.**

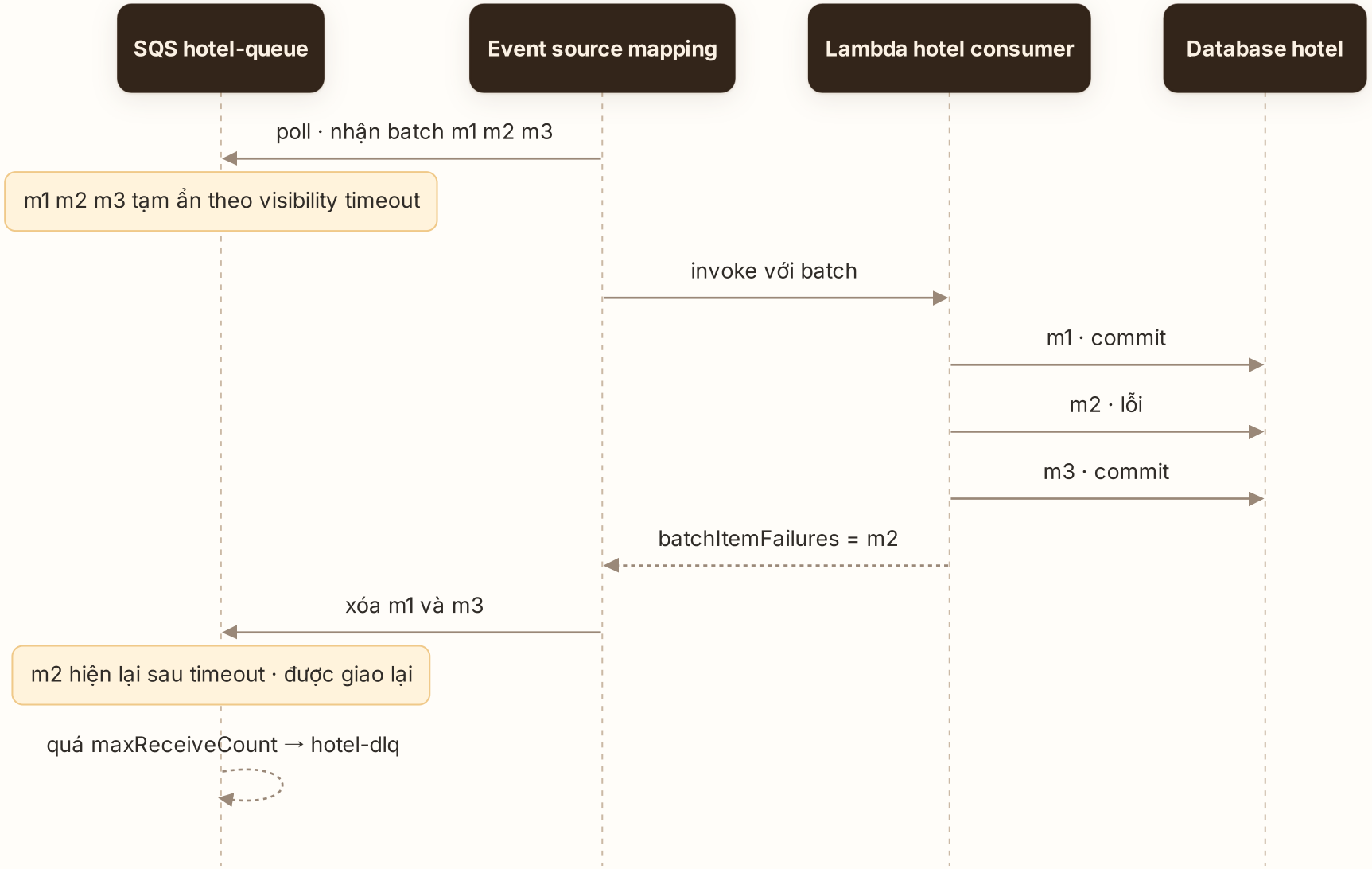
9.3.5 Bù trừ, message đến muộn và công việc còn lại trong DLQ

Khi `payment-service` xác nhận thanh toán thất bại, service ghi sự kiện `PaymentFailed` vào outbox để phát hành. Các service liên quan nhận sự kiện này và thực hiện bù trừ cho những bước đã hoàn thành trước đó. Đồng thời, `trip-service` chuyển Trip sang trạng thái `COMPENSATING` và chỉ ghi `FAILED` sau khi nhận đủ xác nhận rằng các thao tác bù trừ cần thiết đã hoàn tất.

Trong choreography, các sự kiện có thể được giao nhiều lần hoặc đến không đúng thứ tự. Vì vậy, mỗi service vẫn phải thực thi các cơ chế đã thiết kế ở Chương 8, bao gồm tombstone để ghi nhớ tài nguyên đã bị hủy, kiểm tra `version` và giới hạn những phép chuyển trạng thái hợp lệ. Các quy tắc này được thực thi ngay tại database của từng service để một sự kiện đến muộn không thể đảo ngược trạng thái đã được xác lập.

Nếu consumer liên tục xử lý thất bại và message vượt quá số lần nhận được cấu hình trong policy, SQS chuyển message sang **dead-letter queue (DLQ)**. DLQ lưu giữ những công việc chưa được giải quyết để đội ngũ vận hành có thể điều tra, nhưng bản thân nó không tiếp tục hay hoàn tất Saga. Sau khi nguyên nhân được khắc phục, quá trình redrive phải giữ nguyên định danh của message, đồng thời kiểm tra deadline và trạng thái nghiệp vụ hiện tại trước khi xử lý lại. Chẳng hạn, một lệnh đặt xe đến muộn không được phép tạo lại booking đã bị hủy trước đó.

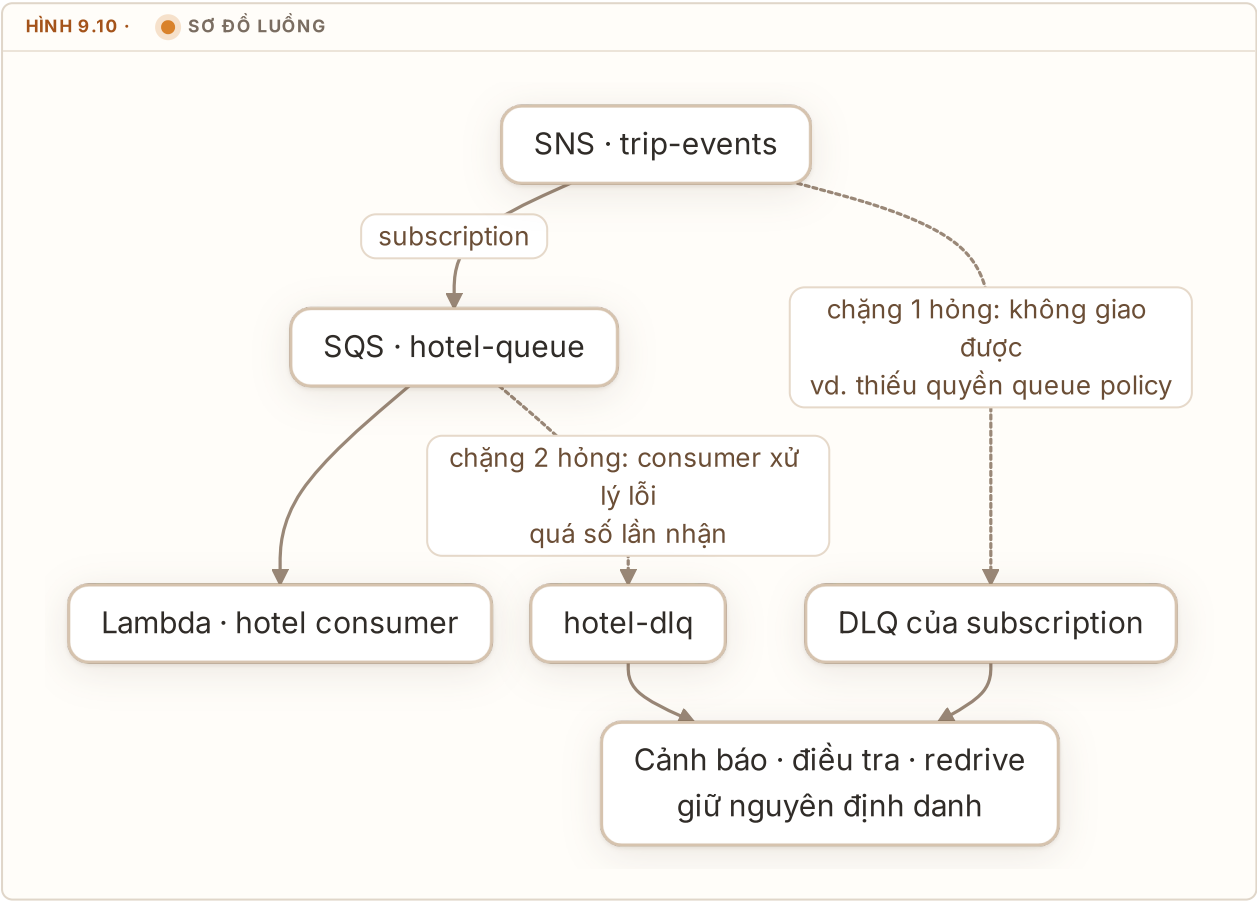
HÌNH 9.9 · SƠ ĐỒ TUẦN TỰ



Cũng cần phân biệt hai loại lỗi xảy ra ở hai chặng khác nhau của đường truyền sự kiện. **SNS subscription DLQ** xử lý trường hợp SNS không thể chuyển message đến SQS theo delivery policy; trong khi **SQS DLQ** lưu những message đã vào queue nhưng consumer không thể xử lý thành công sau nhiều lần nhận. Hai DLQ vì thế bảo vệ hai ranh giới khác nhau trong luồng `SNS → SQS → consumer`.

Nếu chỉ theo dõi DLQ và backlog của consumer queue, chúng ta có thể bỏ sót những sự kiện chưa từng được SNS chuyển thành công tới queue đó. Vì vậy, việc giám sát cần bao quát cả chặng phân phối từ SNS đến SQS và chặng xử lý từ SQS đến consumer. AWS — SNS dead-letter queues.

HÌNH 9.10 · SƠ ĐỒ LUỒNG



9.4 Đưa các thành phần vào một môi trường triển khai hoàn chỉnh

9.4.1 Mỗi đường kết nối cần cả mạng và quyền truy cập

Những mũi tên trong sơ đồ kiến trúc không tự xuất hiện khi các resource được tạo. Mỗi kết nối đều cần được thiết lập ở cả **lớp mạng** và **lớp quyền truy cập**. API Gateway cần quyền invoke Lambda tiếp nhận request; Step Functions sử dụng execution role để gọi đúng Lambda version; EventBridge Scheduler cần role để kích hoạt các relay. Tương tự, mỗi Lambda có execution role phù hợp với nhiệm vụ của mình: dispatcher được phép khởi chạy workflow, publisher được phép publish lên SNS topic, còn consumer được phép đọc SQS queue và truy cập các tài nguyên cần thiết cho domain mà nó phục vụ.

Cần phân biệt **IAM của AWS với quyền bên trong PostgreSQL**. Việc một Lambda được phép lấy secret không có nghĩa nó tự động có quyền đọc hoặc ghi mọi dữ liệu trong database. Quyền đó vẫn được kiểm soát bằng role và grant của PostgreSQL. Tương tự, quyền invoke một Lambda chỉ cho phép thực hiện lời gọi, không mặc nhiên trao quyền đối với dữ liệu mà function đó quản lý.

Ở đường vào từ người dùng, thông tin xác thực do API Gateway kiểm tra cần được chuyển thành ngữ cảnh nghiệp vụ đáng tin cậy trước khi đưa vào workflow. Các lời gọi giữa những thành phần nội bộ sử dụng service identity thay vì danh tính trực tiếp của người dùng, nhưng mỗi service vẫn phải kiểm tra contract và chỉ cho phép những thao tác phù hợp với phạm vi trách nhiệm của nó.

Các Lambda cần truy cập RDS được đặt trong VPC và cấu hình subnet, Security Group để kết nối tới RDS Proxy và database nằm trong mạng private. Tuy nhiên, **đưa Lambda vào VPC không đồng nghĩa function tự có đường ra Internet**. Nếu Lambda cần gọi API của đối tác bên ngoài, VPC phải có đường egress phù hợp. Tương tự, khi code cần truy cập các dịch vụ AWS như SNS, Step Functions hoặc Secrets Manager, chúng ta phải bảo đảm có đường mạng thông qua NAT hoặc VPC endpoint nếu dịch vụ đó hỗ trợ. Việc đặt Lambda trong public subnet cũng không tự cấp public IP cho function. Các thành phần mạng như NAT Gateway và VPC endpoint đồng thời tạo thêm chi phí cần được tính vào kiến trúc tổng thể. AWS — Lambda trong VPC.

Dựa trên quy trình triển khai bằng IaC ở Phần II, một môi trường TravelNow có thể được chuẩn bị theo thứ tự sau:

1. **Chuẩn bị lớp dữ liệu.** Tạo RDS instance với năm database logic, các database role, secret, RDS Proxy cùng cấu hình mạng cần thiết. Đồng thời xác định rõ môi trường sử dụng Single-AZ cho mục đích thực hành hay Multi-AZ khi có yêu cầu cao hơn về tính sẵn sàng.
2. **Triển khai các service.** Chạy database migration theo hướng tương thích ngược, sau đó triển khai Lambda của từng service. Mỗi Lambda version cần truy vết được về commit và artifact đã qua kiểm thử. Các handler dành cho HTTP, Step Functions Task và SQS batch consumer có contract riêng phù hợp với cách chúng được gọi.
3. **Triển khai cơ chế phối hợp Saga.** Với orchestration, tạo state machine cùng các role cần thiết để invoke đúng Lambda version. Với choreography, tạo SNS topic, subscription, SQS queue, DLQ và event source mapping trước khi producer bắt đầu phát những sự kiện mà các consumer phải xử lý.
4. **Bật các tác vụ nền và phục hồi.** Cấu hình EventBridge Scheduler cho dispatcher, publisher và các công việc đối soát. Trước khi bật relay, cần kiểm tra nơi nhận đã sẵn sàng, đồng thời cấu hình retry, xử lý invocation thất bại và cảnh báo cho những bản ghi outbox tồn đọng quá lâu.
5. **Mở đường vào hệ thống.** Cấu hình API Gateway, cơ chế xác thực và các giới hạn tiếp nhận request. Kiểm tra toàn bộ luồng bằng dữ liệu thử trước khi mở traffic theo kế hoạch phát hành.

Các resource, IAM policy, cấu hình mạng và alarm được quản lý bằng Terraform; cấu hình của từng môi trường cung cấp các giá trị như endpoint, ARN và tên secret cho ứng dụng. Tuy nhiên, việc đặt hậu tố `dev` hay `prod` trong tên resource chỉ giúp nhận diện môi trường, **không tạo ra sự cách ly thực sự**. Việc cách ly còn phải được bảo đảm bằng account hoặc phạm vi IAM phù hợp, ranh giới mạng và dữ liệu riêng, sao cho một thao tác thử nghiệm trong `dev` không thể vô tình tác động đến booking của `prod`.

9.4.2 Kiểm chứng những điều kiến trúc đã hứa

Một request được xử lý thành công mới chỉ chứng minh **happy path** của hệ thống có thể hoạt động. Để đánh giá kiến trúc phân tán có thực sự đáp ứng những bảo đảm đã thiết kế hay không, chúng ta cần chủ động tạo sự cố tại chính các ranh giới lỗi đã phân tích ở những phần trước:

Tình huống kiểm chứng	Kết quả mong đợi
Trip đã được commit nhưng dispatcher chưa gọi Step Functions	Ý định khởi chạy vẫn còn trong outbox và được dispatcher xử lý ở lượt tiếp theo.
`car-service` đã commit việc giữ xe nhưng phản hồi bị mất	Lần gọi lại với cùng idempotency key trả về kết quả đã lưu, không tạo thêm booking.
`payment-service` xác nhận thanh toán bị từ chối	Các bước giữ chỗ đã hoàn thành được bù trừ; Trip chỉ chuyển sang `FAILED` sau khi các thao tác bù trừ cần thiết hoàn tất.
Payment timeout và chưa xác định được tiền đã bị thu hay chưa	Hệ thống kích hoạt luồng tra cứu hoặc đối soát thay vì mặc định giao dịch chưa xảy ra.
Publisher dừng sau khi SNS đã nhận event	Event có thể được phát lại với cùng định danh; consumer xử lý an toàn mà không lặp lại side effect.
Lambda consumer xử lý được một phần batch rồi dừng	Những item chưa hoàn tất được giao lại theo cấu hình; các item đã commit vẫn an toàn nếu bị nhận lại.
Relay bỏ lỡ một lượt kích hoạt hoặc dừng sau khi nhận lease	Công việc được nhận lại sau khi lease hết hạn; độ trễ của các bản ghi outbox tồn đọng được phát hiện và theo dõi.
RDS gián đoạn hoặc hết capacity khi Lambda tăng tải	Lỗi hạ tầng không bị hiểu nhầm thành từ chối nghiệp vụ; retry được giới hạn và không tạo thêm side effect.
Lệnh đặt đến sau khi booking đã được quyết định hủy	Booking giữ nguyên trạng thái đã hủy và không quay trở lại `BOOKED`.
Step Functions execution kết thúc lỗi hoặc message bị chuyển vào DLQ	Các Trip chưa hoàn tất được phát hiện, theo dõi và có đường phục hồi hoặc xử lý tiếp theo.

Mỗi bài kiểm chứng cần được đánh giá dựa trên **trạng thái nghiệp vụ và dấu vết xử lý**, không chỉ dựa vào kết quả bên ngoài. Chúng ta cần đối chiếu dữ liệu của Trip, booking, idempotency và outbox với execution history của Step Functions hoặc các metric của queue để xác định hệ thống đã xử lý đúng hay chưa. Một HTTP response `200 OK` hoặc một queue đang trống chưa đủ để chứng minh toàn bộ quy trình nghiệp vụ đã hoàn tất chính xác.

Các tình huống trong chương này chủ yếu minh họa cơ chế và kết quả mà kiến trúc kỳ vọng đạt được. Khi triển khai thực tế, đội ngũ vẫn cần kiểm chứng chúng trên môi trường AWS với cấu hình thật, bởi môi trường local không thể tái hiện đầy đủ các yếu tố như IAM, quota, concurrency cũng như hành vi giao nhận và retry của các dịch vụ managed.

9.4.3 Khi tầng dữ liệu gián đoạn, điều gì còn lại?

Việc chạy ứng dụng trên Lambda giúp chúng ta không phải duy trì một EC2 chứa toàn bộ hệ thống, nhưng không loại bỏ các **điểm phụ thuộc chung**. Trong kiến trúc TravelNow hiện tại, cả năm service vẫn sử dụng chung một RDS instance. Vì vậy, khi RDS gián đoạn, nhiều service có thể đồng thời mất khả năng đọc hoặc ghi dữ liệu. Step Functions vẫn lưu tiến độ workflow và SQS vẫn giữ message trong thời gian retention đã cấu hình, nhưng các thành phần này không thể thay database hoàn thành thao tác nghiệp vụ.

Trong thời gian RDS chưa phục hồi, workflow chỉ có thể tiếp tục theo các chính sách retry, trạng thái chờ và deadline đã thiết kế. Các SQS consumer có thể xử lý thất bại khiến message được giao lại hoặc cuối cùng chuyển sang DLQ. Hệ thống cần tránh để nhiều thành phần retry đồng loạt gây thêm áp lực lên database, đồng thời có thể phải hạn chế lượng request mới được tiếp nhận. RDS Multi-AZ giúp giảm thời gian gián đoạn thông qua cơ chế failover, nhưng ứng dụng vẫn phải chịu được việc connection bị ngắt và những trường hợp chưa xác định chắc chắn transaction trước đó đã commit hay chưa.

Khi database hoạt động trở lại, hệ thống tiếp tục dựa trên **trạng thái đã được lưu trữ**. Những bản ghi outbox còn tồn đọng sẽ được relay xử lý; message còn trong SQS được consumer nhận lại; các thao tác chưa rõ kết quả được tra cứu bằng idempotency key. Nhờ đó, quá trình phục hồi không phụ thuộc vào bộ nhớ của những Lambda invocation đã kết thúc.

Tuy nhiên, một Step Functions execution đã kết thúc với trạng thái lỗi sẽ không tự động chạy lại khi database phục hồi. Công việc đối soát cần phát hiện những Trip vẫn còn dang dở, xác định trạng thái thực tế của các bước đã thực hiện và lựa chọn đường phục hồi phù hợp.

Trường hợp phải **restore database từ backup** còn phức tạp hơn. Việc khôi phục RDS chỉ đưa dữ liệu database trở về một thời điểm trước đó, trong khi các sự kiện đã phát ra hoặc giao dịch đã thực hiện với hệ thống bên ngoài không tự động quay ngược trở lại. Vì vậy, sau khi restore, TravelNow cần đối chiếu dữ liệu trong database với các event đã phát hành và trạng thái của những giao dịch bên ngoài trước khi tiếp tục xử lý.

9.4.4 Khả năng phục hồi còn cần con người tiếp tục công việc

Giả sử một Trip được chuyển sang `MANUAL_REVIEW` lúc hai giờ sáng. Hệ thống vẫn lưu đầy đủ dữ liệu và cảnh báo đã được phát đi, nhưng nếu không ai biết mình chịu trách nhiệm xử lý hoặc được phép thực hiện những thao tác nào tiếp theo, thì từ góc nhìn của khách hàng, sự cố vẫn chưa được giải quyết.

Vì vậy, **khả năng phục hồi không chỉ nằm trong code và hạ tầng mà còn phụ thuộc vào quy trình vận hành**. Đội ngũ cần xác định rõ người chịu trách nhiệm, quyền hạn và hướng dẫn xử lý cho từng tình huống. Người trực phải có khả năng lần theo `tripId` để tìm execution tương ứng, trạng thái các booking và kết quả thanh toán; đồng thời biết trường hợp nào có thể retry an toàn, trường hợp nào cần tra cứu trạng thái hoặc đối soát trực tiếp với đối tác trước khi thực hiện bước tiếp theo.

Những quy trình này không nên chỉ tồn tại trên tài liệu mà cần được **diễn tập trước khi xảy ra sự cố thực tế**. Qua mỗi lần kiểm thử hoặc xử lý sự cố, những điểm còn thiếu trong code, cấu hình, khả năng quan sát và quy trình vận hành trở thành đầu vào cho vòng cải tiến tiếp theo của hệ thống.

9.4.5 Nếu hiện thực TravelNow ngoài AWS

Những nguyên tắc đã áp dụng cho TravelNow không phụ thuộc hoàn toàn vào AWS. Hãy giữ nguyên một tình huống đã gặp: `hotel-service` đã commit việc giữ phòng nhưng phản hồi bị mất. Dù logic này chạy trong Lambda hay một worker container, lần xử lý tiếp theo vẫn phải sử dụng cùng idempotency key để xác định thao tác đã xảy ra và tra cứu kết quả đã lưu, thay vì tạo thêm một booking. **Điều cần giữ nguyên là ranh giới transaction, định danh thao tác và các bảo đảm phục hồi; cách triển khai handler, giao thức giao tiếp hay nền tảng thực thi có thể thay đổi.**

Với **orchestration**, một workflow engine mã nguồn mở như **Temporal** có thể đảm nhận việc lưu tiến độ và tiếp tục workflow sau sự cố. Temporal cho phép mô tả workflow bằng code và sử dụng lịch sử thực thi để tái tạo trạng thái của workflow. Đây là một cách khác để hiện thực những yêu cầu mà Saga đặt ra, chứ không phải nền tảng có thể nhận nguyên định nghĩa ASL của Step Functions. Khi chuyển sang Temporal, chúng ta phải thiết kế lại cách biểu diễn workflow, gọi các công việc nghiệp vụ, cấu hình retry và quản lý quá trình khởi chạy theo mô hình của engine này. Temporal — Workflow, mã nguồn Temporal.

Với **choreography**, RabbitMQ có thể thay thế lớp messaging bằng mô hình **exchange và queue**. Exchange chịu trách nhiệm định tuyến message đến các queue phù hợp, còn consumer nhận và xử lý message từ queue. Khi sử dụng acknowledgement thủ công, consumer chỉ xác nhận sau khi các thay đổi nghiệp vụ cần thiết đã được lưu lại. Nếu message được giao lại, consumer vẫn phải nhận diện thao tác đã xử lý trước đó để tránh xử lý trùng lặp.

Ở đây cũng cần phân biệt hai ranh giới xác nhận. **Publisher confirm** cho biết broker đã tiếp nhận trách nhiệm đối với message từ publisher, còn **consumer acknowledgement** xác nhận quá trình xử lý message ở phía consumer. Hai cơ chế này bảo vệ hai chặng khác nhau của đường truyền, tương tự việc SNS xác nhận đã nhận một event không chứng minh rằng `hotel-service` đã giữ phòng thành công. Khi chuyển đổi nền tảng, chúng ta cũng không thể sao chép nguyên các khái niệm như visibility timeout hay chính sách DLQ của SQS sang RabbitMQ mà phải thiết kế theo cơ chế và bảo đảm của broker mới. RabbitMQ — Publish/Subscribe, các loại xác nhận.

PostgreSQL cũng có thể tiếp tục lưu trữ dữ liệu nghiệp vụ, idempotency và outbox mà không cần chạy trên RDS. Nếu tự vận hành PostgreSQL, đội ngũ phải đảm nhận thêm các công việc như sao lưu, phục hồi, nâng cấp, giám sát và xây dựng cơ chế dự phòng. Nếu chuyển sang một dịch vụ database được quản lý trên nền tảng khác, chúng ta cần xác định lại ranh giới trách nhiệm giữa ứng dụng và nhà cung cấp. Dù lựa chọn cách nào, **bảng tình huống lỗi ở Mục 9.4 vẫn có thể được sử dụng làm đầu vào để kiểm chứng những bảo đảm mà kiến trúc cần duy trì.**

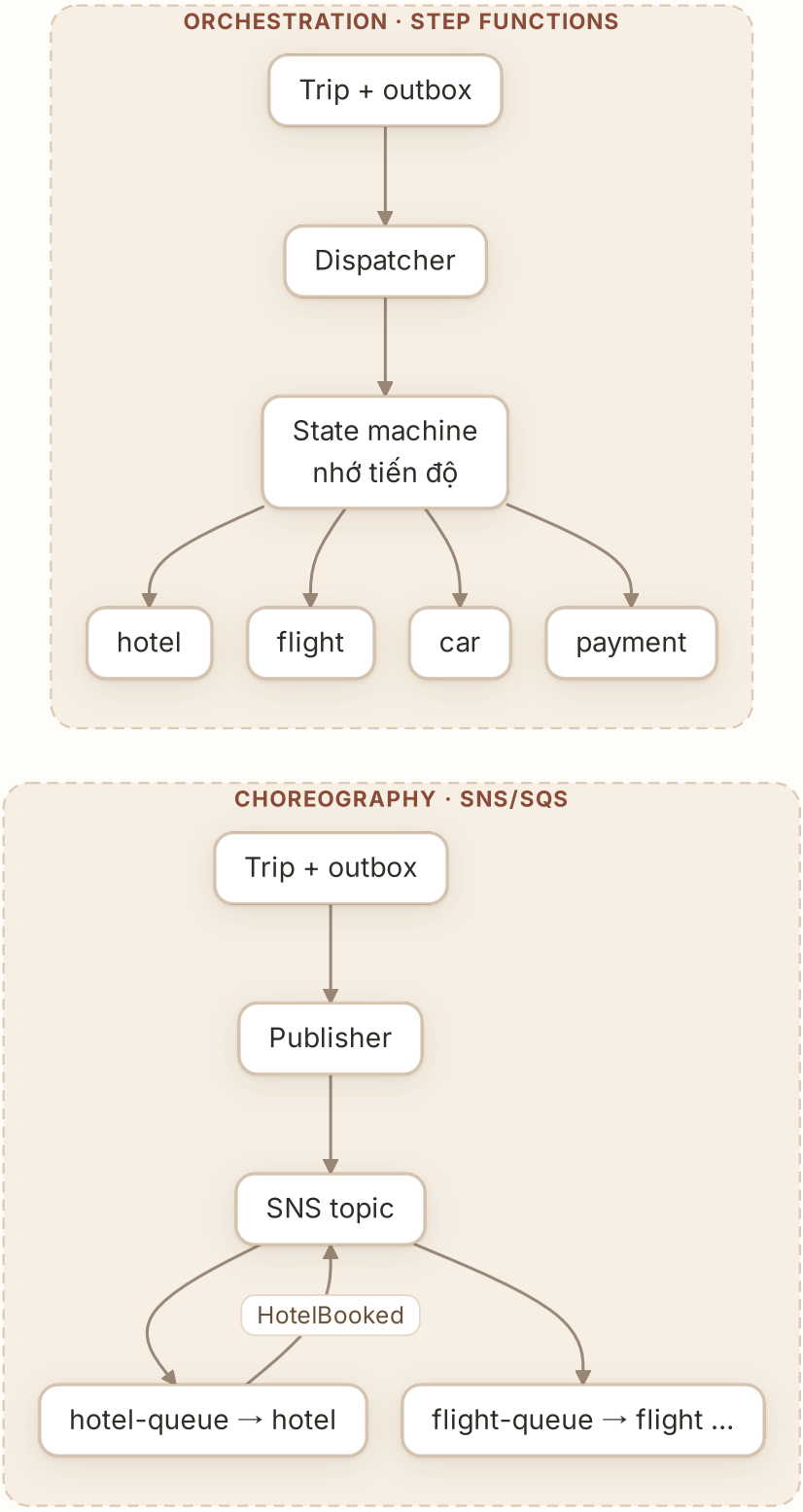
Như vậy, khi chuyển TravelNow từ AWS sang một nền tảng khác, điều cần giữ lại không phải là tên dịch vụ hay cấu hình cụ thể, mà là **những yêu cầu mà kiến trúc phải đáp ứng và cách xử lý tại từng ranh giới lỗi**. Với nền tảng mới, chúng ta cần xem xét lại API, cơ chế retry, cách message được gửi và xử lý lại, các giới hạn kỹ thuật cũng như trách nhiệm vận hành. eBook vì thế sử dụng một bản triển khai AWS xuyên suốt để có thể đi sâu vào những chi tiết này, thay vì chỉ thay tên các dịch vụ trên cùng một sơ đồ kiến trúc.

9.5 Tóm tắt — từ phối hợp nghiệp vụ đến vận hành compute

Chương 9 đã hiện thực cùng một quy trình Saga theo hai cách khác nhau. Với **orchestration**, Step Functions lưu tiến độ và điều phối thứ tự thực hiện các bước. Với **choreography**, SNS phân phối sự kiện còn SQS lưu giữ công việc để từng consumer xử lý theo tiến độ riêng. Trong cả hai phương án, Lambda thực thi logic nghiệp vụ, còn RDS lưu dữ liệu cùng các thông tin cần thiết để nhận diện và ngăn thao tác nghiệp vụ bị thực hiện trùng.

Dù lựa chọn cách phối hợp nào, vẫn tồn tại một ranh giới quan trọng giữa **việc lưu ý định thực hiện và việc thực sự đưa công việc vào xử lý**. Vì hai thao tác này không nằm trong cùng một transaction, hệ thống cần có cơ chế phục hồi để những công việc đã được ghi nhận không bị bỏ sót khi xảy ra sự cố.

HÌNH 9.11 · SƠ ĐỒ LUỒNG



Từ TravelNow, chúng ta có thể thấy rõ hơn **ranh giới trách nhiệm của từng dịch vụ AWS**. Step Functions lưu tiến độ và điều phối workflow, nhưng không thể tự xác định một giao dịch thanh toán đã thực sự thành công hay chưa. SNS phân phối sự kiện nhưng không chứng minh

consumer đã xử lý nghiệp vụ thành công. SQS lưu giữ message để công việc có thể được xử lý lại, nhưng không biết các quy tắc nghiệp vụ của quá trình đặt chuyến. Lambda thực thi code, nhưng không phải nơi lưu trạng thái bền vững của Saga.

Vì vậy, tính đúng đắn của hệ thống vẫn phụ thuộc vào những nguyên tắc đã xuyên suốt các chương trước: **lưu trữ bền vững ý định trước khi thực hiện công việc bên ngoài transaction, giữ nguyên định danh thao tác khi retry, chỉ bù trừ dựa trên trạng thái đã được xác định và theo dõi những công việc còn dang dở cho đến khi chúng được giải quyết**. Những nguyên tắc này không phụ thuộc riêng vào AWS và vẫn cần được duy trì khi hệ thống chuyển sang một nền tảng khác.

Sang Phần IV, trọng tâm chuyển từ **phối hợp nghiệp vụ phân tán** sang **quản lý môi trường thực thi ứng dụng**. Chúng ta sẽ sử dụng Kubernetes và Amazon EKS để tìm hiểu cách triển khai, phân bố, thay thế và điều chỉnh số lượng container theo tải. Các service của TravelNow sẽ được chạy bằng container thay cho Lambda, trong khi những quy tắc nghiệp vụ và mô hình dữ liệu trên RDS vẫn được giữ nguyên. Qua đó, chúng ta có thể tập trung tìm hiểu cách Kubernetes vận hành ứng dụng container và so sánh trách nhiệm mà đội ngũ phải đảm nhận so với khi sử dụng mô hình serverless trên AWS.

Đến Chương 13, hai phương án sẽ được đặt cạnh nhau để xem xét những trường hợp phù hợp với từng cách tiếp cận cũng như khả năng kết hợp chúng trong cùng một hệ thống. **Chuyển từ Lambda sang container không phải bước nâng cấp bắt buộc và cũng không làm Saga trở nên đúng hơn; đó là một lựa chọn về nền tảng thực thi và mô hình vận hành, cần được đánh giá dựa trên đặc điểm của workload.**

PHẦN IV

Vận hành ở quy mô lớn

Operating at Scale

Khi lượng yêu cầu và số nhóm tham gia phát triển tăng lên, làm thế nào để triển khai thay đổi an toàn và duy trì khả năng phục vụ? Phần này sử dụng TravelNow để tìm hiểu cách điều phối container, chuẩn bị điều kiện tích hợp giữa các nhóm, điều chỉnh tài nguyên theo tải và phục hồi công việc sau sự cố.

Trong Chương 10–12, cả năm service của TravelNow được triển khai bằng container thay cho Lambda ở Chương 9. Dữ liệu tiếp tục lưu trên RDS và các quy tắc nghiệp vụ được giữ nguyên; cách tiếp nhận yêu cầu HTTP, xử lý message và kết nối với workflow được điều chỉnh cho môi trường container. Nhờ đó, chúng ta có thể tập trung phân tích những thay đổi về triển khai và vận hành khi sử dụng Kubernetes trên EKS. Chương 13 sẽ đối chiếu phương án này với serverless và xem xét khả năng kết hợp theo nhu cầu của từng workload. Thứ tự trình bày phục vụ việc học, không hàm ý hệ thống dùng Lambda phải chuyển sang Kubernetes khi phát triển lớn hơn.

Qua các tình huống thay thế Pod, bổ sung tài nguyên và duy trì hoạt động khi có sự cố, chúng ta sẽ làm rõ **nền tảng quản lý những gì và ứng dụng vẫn phải tự bảo đảm những gì**. Kubernetes giúp giải thích cơ chế điều phối container; EKS cho thấy cách AWS quản lý một phần hạ tầng đó; serverless mở rộng việc so sánh sang mô hình giao thêm trách nhiệm quản lý môi trường thực thi cho nền tảng. Dù lựa chọn cách triển khai nào, hệ thống vẫn cần hoàn tất nghiệp vụ đúng, phục hồi được công việc đang dở và có người chịu trách nhiệm xử lý khi cơ chế tự động không đủ.

Chương	Nội dung	Câu hỏi trọng tâm
10	Container Orchestration với Kubernetes	Kubernetes quản lý vòng đời container như thế nào, và ứng dụng vẫn phải tự bảo đảm những gì?
11	TravelNow trên Amazon EKS	Khi nào nên chọn EKS, và trách nhiệm vận hành được phân chia giữa AWS và chúng ta ra sao?
12	Auto Scaling và High Availability	Tăng tài nguyên có cải thiện khả năng xử lý không, và hệ thống còn đủ năng lực phục vụ khi mất một AZ không?
13	Tản mạn về kiến trúc Serverless	Serverless phù hợp với workload nào, và chúng ta vẫn cần quản lý những gì khi nền tảng cung cấp môi trường thực thi?

CHƯƠNG 10

Container Orchestration với Kubernetes

Phần IV — Vận hành ở quy mô lớn · *Operating at Scale*

Ở cuối Phần III, TravelNow đã có cơ chế lưu tiến độ của quá trình đặt chuyến, thực hiện lại những công việc chưa hoàn tất và nhận diện các thao tác được gửi đến nhiều lần. Bây giờ, hãy giả sử năm service của TravelNow được đóng gói thành container và chạy trên nhiều máy chủ. Nếu máy chủ đang chạy một instance của `flight-service` gặp sự cố, instance đó sẽ được thay thế như thế nào và workload mới sẽ được chạy ở đâu? Khi instance thay thế đã khởi động, làm thế nào để đưa nó vào phục vụ traffic? Và khi phát hành một phiên bản mới của `flight-service`, làm thế nào để thay thế dần các instance đang chạy phiên bản cũ mà vẫn duy trì khả năng phục vụ?

Đó là bài toán của **container orchestration**: quản lý nơi các workload được chạy, vòng đời của chúng và cách tài nguyên được phân bổ trên nhiều máy chủ. Cần phân biệt khái niệm này với **Saga orchestration** ở Chương 8. Container orchestration duy trì các instance của ứng dụng theo trạng thái mong muốn, trong khi Saga orchestration phối hợp các bước của một quy trình nghiệp vụ. Kubernetes có thể phát hiện một instance của flight-service không còn hoạt động và tạo instance thay thế trên một node phù hợp, nhưng nó không biết thao tác giữ vé mà instance trước đó đang xử lý đã thành công hay cần được bù trừ.

Thông qua `flight-service`, chúng ta sẽ theo dõi cách Kubernetes triển khai workload, thay thế instance khi xảy ra sự cố và đưa các instance sẵn sàng vào phục vụ. Qua đó, chúng ta sẽ thấy rõ **Kubernetes có thể đảm nhận những trách nhiệm nào cho TravelNow và những trách nhiệm nào vẫn phải do ứng dụng tự bảo đảm**. Thay vì tập trung ghi nhớ tên resource hay cú pháp YAML, chương này đi vào các cơ chế phía sau chúng: cách Kubernetes phân bổ tài nguyên, kết nối các workload, xử lý state và kiểm soát quá trình phát hành phiên bản mới.

Cách đọc chương này

Mục 10.1–10.3 xây dựng nền tảng về **container orchestration**: những vấn đề một bộ điều phối cần giải quyết, điểm khác biệt của Kubernetes so với các nền tảng khác và cách các resource cốt lõi phối hợp để duy trì workload theo trạng thái mong muốn. Mục 10.4–10.5 chuyển sang hai vấn đề quan trọng khi đưa ứng dụng vào vận hành: **networking và state**. Cuối cùng, Mục 10.6–10.7 tập trung vào quá trình **phát hành phiên bản mới** và cách Kubernetes xác định một instance có sẵn sàng nhận traffic hay cần được khởi động lại.

10.1 Container orchestration giải quyết bài toán gì?

10.1.1 Từ chạy container trên một máy đến điều phối workload trên nhiều máy

Ở Chương 3, Docker Compose đã giúp TravelNow khai báo và chạy nhiều container cùng với network, volume và các cấu hình cần thiết. Khi toàn bộ ứng dụng chạy trên một máy chủ, Compose vẫn có thể được sử dụng trong production nếu đội ngũ có quy trình triển khai, giám sát và khôi phục phù hợp.

Bài toán trở nên khác khi ứng dụng cần chạy trên **nhiều máy chủ và sử dụng tài nguyên của chúng như một môi trường thực thi chung**. Nếu máy chủ A gặp sự cố, Compose không tự tìm một máy chủ khác còn đủ tài nguyên để khởi chạy lại workload đang bị mất. Tương tự, khi cần nhiều instance của cùng một service, Compose trên từng máy chủ không tự quyết định instance nào nên chạy ở đâu hay làm thế nào để duy trì số lượng instance mong muốn khi một máy chủ hoặc container gặp sự cố.

Khi số lượng máy chủ và workload tăng lên, những quyết định này cần được thực hiện ở phạm vi toàn bộ hệ thống thay vì trên từng máy riêng lẻ. Đây chính là khoảng trống mà các nền tảng container orchestration giải quyết. Docker — Compose trong production.

Một bộ điều phối container trên nhiều máy chủ thường cần giải quyết năm bài toán chính:

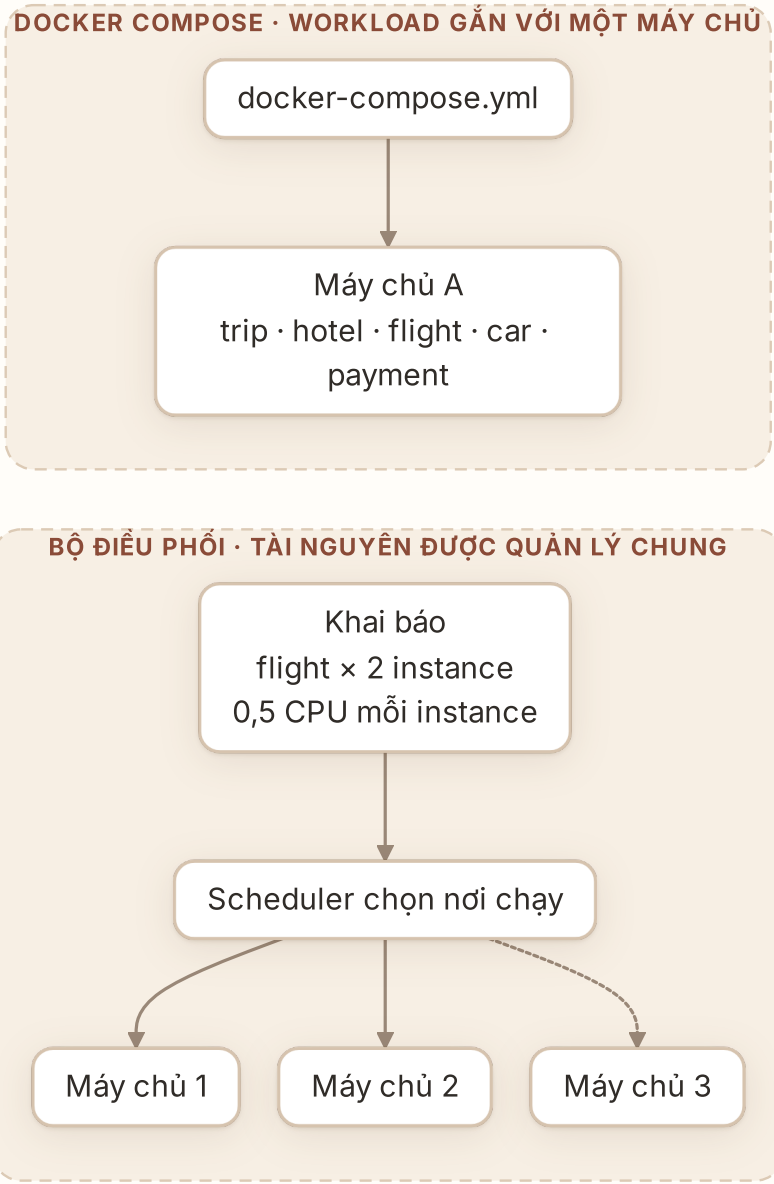
TravelNow cần gì?	Cơ chế cần có
Luôn duy trì hai instance của `flight-service` hoạt động	**Trạng thái mong muốn** và cơ chế liên tục đối chiếu với trạng thái thực tế
Chọn máy chủ phù hợp để chạy từng instance	**Scheduling** dựa trên tài nguyên và các ràng buộc đã khai báo
Gọi `flight-service` mà không cần biết địa chỉ của từng instance	**Service discovery** và định tuyến traffic đến các instance sẵn sàng
Phát hành phiên bản mới mà vẫn duy trì khả năng phục vụ	**Rollout có kiểm soát**: thay thế dần các instance và dừng khi phát hiện vấn đề
Tự động thay thế instance bị lỗi hoặc mất khi máy chủ gặp sự cố	**Self-healing** dựa trên trạng thái của workload và node

Sự thay đổi quan trọng nằm ở cách chúng ta **mô tả nơi ứng dụng sẽ chạy**. Với Compose, cách triển khai thường gắn workload với một máy chủ cụ thể, chẳng hạn: "`flight-service` chạy trên máy chủ A". Với một bộ điều phối, chúng ta chuyển sang khai báo nhu cầu: "`flight-service` cần hai instance, mỗi instance cần 0,5 CPU và 512 MiB bộ nhớ". Nền tảng sau đó quyết định từng instance sẽ được chạy trên máy chủ nào phù hợp.

Theo cách này, **tài nguyên của nhiều máy chủ được nhìn nhận và phân bổ như một nguồn capacity chung**. Đổi lại, ứng dụng phải khai báo nhu cầu tài nguyên và các ràng buộc cần thiết đủ chính xác để bộ điều phối đưa ra quyết định phù hợp. Một instance của `flight-service` vì thế

không còn gắn cố định với một máy chủ cụ thể và có thể được tạo lại trên một máy chủ khác khi cần.

HÌNH 10.1 · SƠ ĐỒ LUỒNG



10.1.2 Kubernetes khác gì các nền tảng khác?

Kubernetes không phải lựa chọn duy nhất để chạy và điều phối container. Đặt các nền tảng phổ biến cạnh nhau theo một số tiêu chí chung sẽ giúp chúng ta thấy rõ **mỗi nền tảng giải quyết bài toán ở mức nào, cung cấp sẵn những gì và để lại những trách nhiệm nào cho đội ngũ vận hành**:

Nền tảng	Phạm vi	Đơn vị khai báo chính	Mô hình vận hành	Phù hợp khi
Docker Compose	Một máy chủ	Service trong file Compose	Đội ngũ vận hành máy chủ và Docker; không có lớp điều phối nhiều máy chủ	Ứng dụng chạy trên một máy chủ hoặc không cần khả năng điều phối workload giữa nhiều máy
Docker Swarm	Nhiều máy chủ	Service, stack	Đội ngũ tự vận hành cluster	Muốn điều phối container trên nhiều máy chủ với mô hình và cách sử dụng gần Docker Compose
HashiCorp Nomad	Nhiều máy chủ, nhiều loại workload	Job	Đội ngũ tự vận hành cluster	Cần điều phối nhiều loại workload với kiến trúc tương đối gọn
Amazon ECS	AWS	Task definition, service	AWS quản lý lớp điều phối; đội ngũ quản lý workload và các tài nguyên liên quan	Muốn chạy và điều phối container trên AWS mà không phải tự vận hành control plane
Kubernetes	Nhiều máy chủ, nhiều môi trường	Resource thông qua Kubernetes API	Có thể tự vận hành cluster hoặc sử dụng dịch vụ managed như EKS	Cần nền tảng có khả năng mở rộng thông qua controller và hệ sinh thái công cụ phong phú

Điểm đáng chú ý của Kubernetes không chỉ nằm ở khả năng chạy container trên nhiều máy chủ, mà còn ở **mô hình API khai báo có khả năng mở rộng**. Pod, Deployment và Service là những loại resource được Kubernetes cung cấp sẵn. Bên cạnh đó, chúng ta có thể định nghĩa thêm các loại resource mới thông qua *CustomResourceDefinition* (CRD) và xây dựng controller để quản lý chúng. Theo cách này, nhiều thành phần như chứng chỉ TLS, load balancer, cơ chế autoscaling hay thậm chí database có thể được quản lý theo cùng một nguyên tắc: khai báo trạng thái mong muốn và để controller liên tục đối chiếu, điều chỉnh trạng thái thực tế.

Khả năng mở rộng này đồng thời làm tăng trách nhiệm vận hành. Mỗi controller được bổ sung vào cluster là một thành phần phần mềm cần được triển khai, cấu hình quyền, giám sát và nâng cấp. Trong thực tế, một Kubernetes cluster dùng cho production thường còn có nhiều thành phần bổ sung như network plugin, ingress controller, autoscaler hay hệ thống thu thập log. **Kubernetes càng được mở rộng để giải quyết nhiều bài toán, đội ngũ càng phải hiểu và quản lý nhiều thành phần hơn.**

Vì vậy, số lượng microservice không phải là tiêu chí đủ để quyết định sử dụng Kubernetes. Nếu ứng dụng có thể đáp ứng yêu cầu về tải và khả năng khôi phục trên một máy chủ, Docker Compose có thể đã phù hợp. Nếu cần điều phối container trên nhiều máy chủ nhưng muốn giảm trách nhiệm vận hành nền tảng, một dịch vụ managed như Amazon ECS có thể là một lựa chọn khác. Kubernetes phù hợp khi đội ngũ thực sự cần mô hình quản lý thống nhất, khả năng mở rộng của nền tảng và mức độ kiểm soát mà nó cung cấp.

Trong phần này, TravelNow sử dụng Kubernetes để chúng ta có thể đi sâu vào các cơ chế của container orchestration. Sang Chương 11, những cơ chế này sẽ được đặt vào môi trường AWS để xem cách chúng được triển khai với Amazon EKS và trách nhiệm vận hành được phân chia như thế nào.

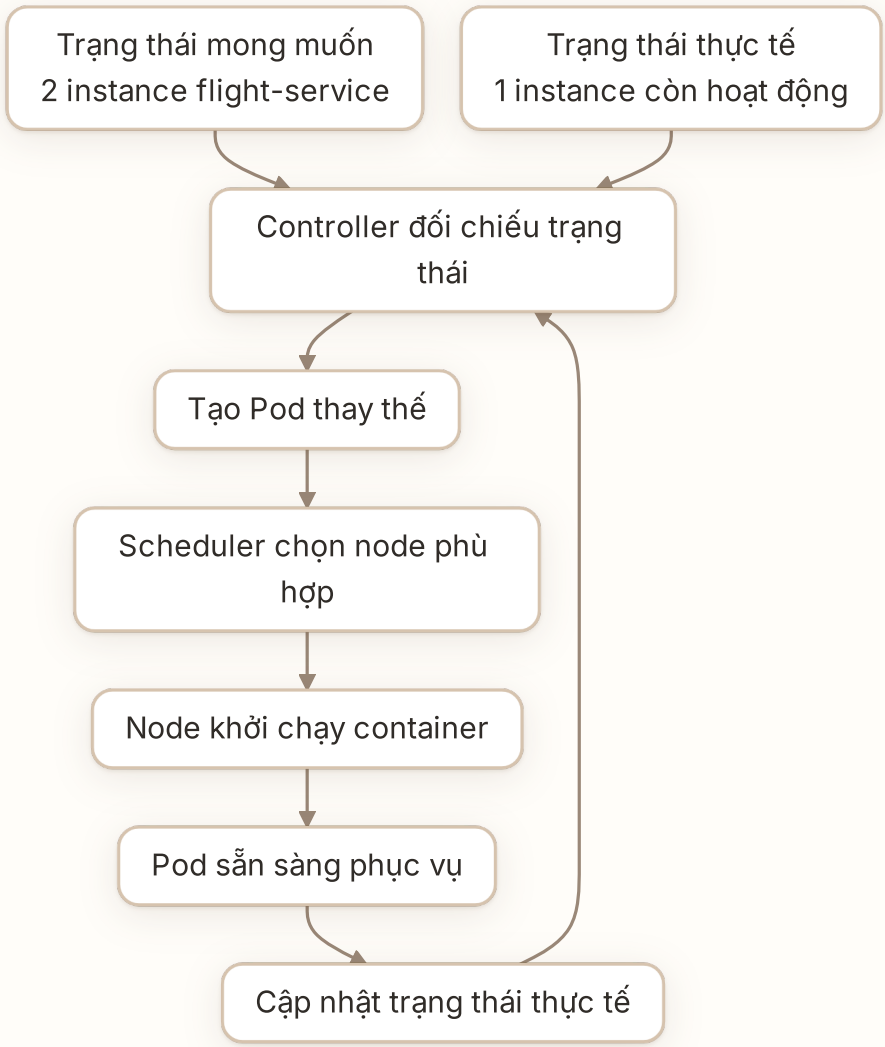
10.2 Trạng thái mong muốn và reconciliation

10.2.1 Từ trạng thái mong muốn đến trạng thái thực tế

Giả sử chúng ta khai báo `flight-service` cần duy trì hai instance hoạt động. Kubernetes liên tục quan sát trạng thái thực tế của hệ thống và so sánh với trạng thái đã khai báo. Nếu chỉ còn một instance hoạt động, nền tảng sẽ tạo thêm một instance để đưa hệ thống trở về trạng thái mong muốn.

Quá trình liên tục **quan sát, đối chiếu và thực hiện điều chỉnh** này được gọi là **reconciliation**:

HÌNH 10.2 · SƠ ĐỒ LUỒNG



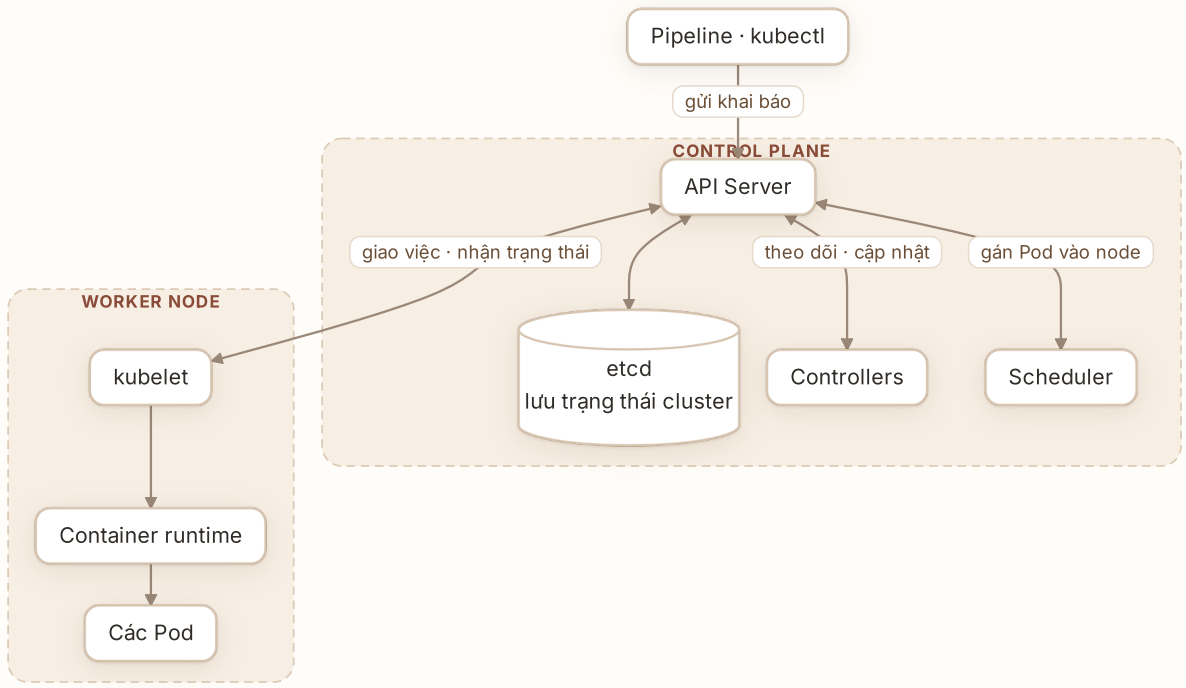
Ở Chương 5, chúng ta đã làm quen với **tư duy khai báo** thông qua Terraform: mô tả trạng thái mong muốn của hạ tầng rồi để công cụ xác định những thay đổi cần thực hiện. Tuy nhiên, Terraform chỉ thực hiện quá trình đối chiếu khi chúng ta chạy `plan` hoặc `apply`, trong khi các controller của Kubernetes **liên tục quan sát và điều chỉnh trạng thái của hệ thống**.

Chẳng hạn, nếu một Pod thuộc Deployment bị xóa, controller sẽ phát hiện số lượng instance thực tế thấp hơn trạng thái mong muốn và tạo Pod thay thế. Tương tự, nếu số lượng replica được thay đổi trực tiếp trên cluster nhưng pipeline vẫn khai báo giá trị cũ, lần triển khai tiếp theo có thể đưa cấu hình trở lại giá trị trong pipeline.

Điều này dẫn đến một nguyên tắc quan trọng: **cần xác định rõ đâu là nguồn cấu hình chuẩn và thành phần nào được phép thay đổi từng thuộc tính của resource**. Vấn đề này càng trở nên quan trọng khi hệ thống có thêm những controller có khả năng tự động điều chỉnh cấu hình, chẳng hạn cơ chế autoscaling sẽ được trình bày ở Chương 12.

10.2.2 Vai trò của các thành phần trong cluster

HÌNH 10.3 · SƠ ĐỒ LUỒNG



Control plane lưu trạng thái của cluster và thực hiện các quyết định cần thiết để đưa hệ thống về trạng thái mong muốn, trong khi **worker node** là nơi các workload thực sự được chạy. Một điểm quan trọng trong kiến trúc Kubernetes là các thành phần phối hợp với nhau thông qua **API Server**. Controller không trực tiếp yêu cầu một node khởi chạy Pod, và scheduler cũng không tự tạo container. Thay vào đó, mỗi thành phần quan sát trạng thái thông qua Kubernetes API, ghi lại quyết định của mình, rồi các thành phần khác tiếp tục phản ứng với những thay đổi đó.

Thiết kế này dẫn đến hai hệ quả quan trọng. Thứ nhất, nếu control plane tạm thời gián đoạn, các Pod đã chạy trên worker node thường vẫn tiếp tục hoạt động và có thể tiếp tục phục vụ request. Tuy nhiên, Kubernetes lúc này không thể thực hiện đầy đủ các công việc quản lý cluster như tiếp nhận thay đổi cấu hình, lên lịch cho Pod mới hoặc thay thế những Pod bị mất. Nói cách khác, **ứng dụng có thể vẫn tiếp tục phục vụ trong khi khả năng điều phối của Kubernetes đang bị gián đoạn**. Chương 11 sẽ quay lại đặc điểm này khi chúng ta tìm hiểu kiến trúc của Amazon EKS.

Thứ hai, **việc Kubernetes quyết định cần tạo một Pod không có nghĩa Pod đó chắc chắn có thể chạy và phục vụ traffic**. Controller có thể tạo Pod để đáp ứng trạng thái mong muốn, nhưng từ đó đến khi Pod thực sự sẵn sàng còn có nhiều bước khác: scheduler phải tìm được node phù hợp, node phải tải được image và khởi chạy container, ứng dụng bên trong container phải khởi động thành công và cuối cùng Pod phải đạt trạng thái sẵn sàng. Vì vậy, một Pod chưa thể phục vụ có thể xuất phát từ những vấn đề ở các giai đoạn khác nhau:

Trạng thái quan sát được	Nguyên nhân thường gặp	Cần kiểm tra
Pod ở trạng thái `Pending`	Không có node đủ tài nguyên hoặc thỏa mãn các ràng buộc để chạy Pod	Tài nguyên của cluster, `requests` và các ràng buộc scheduling
Không thể kéo image	Sai tên hoặc tag của image, thiếu quyền truy cập registry hoặc node không kết nối được tới registry	Image, registry, quyền truy cập và kết nối mạng
Container liên tục khởi động rồi thoát	Cấu hình không hợp lệ, thiếu Secret hoặc ứng dụng gặp lỗi khi khởi động	Log của container, cấu hình ứng dụng và Secret
Pod đã chạy nhưng không nhận traffic	Readiness probe chưa đạt hoặc Service không chọn Pod do label không khớp	Readiness probe, label và selector của Service (Mục 10.3)

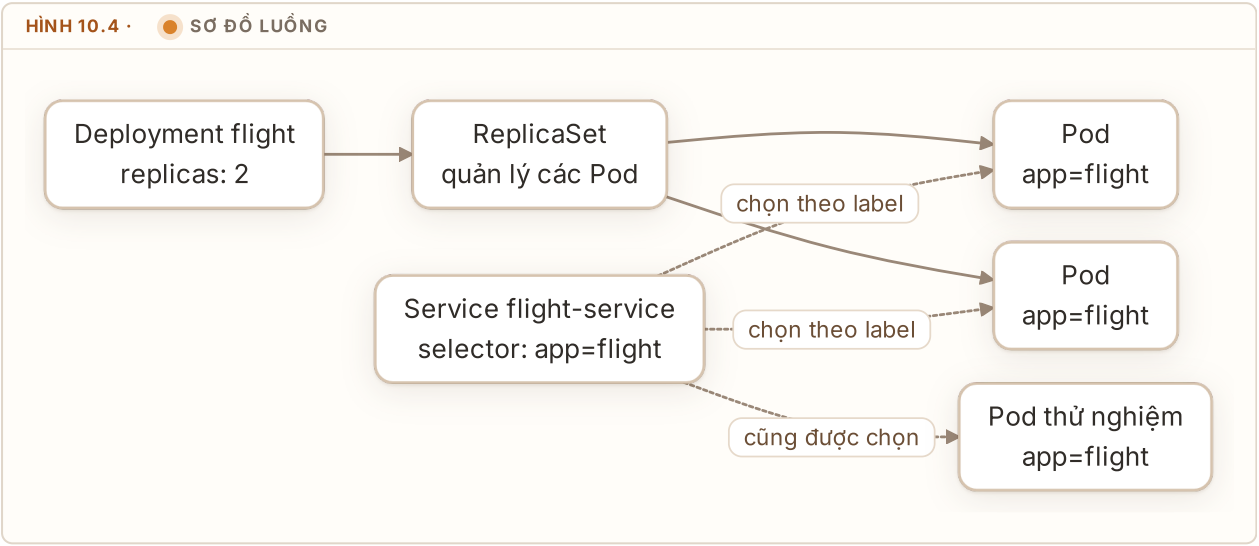
10.3 Một vài resource cốt lõi của Kubernetes

Kubernetes cung cấp nhiều loại resource để quản lý các khía cạnh khác nhau của ứng dụng và cluster. Tuy nhiên, để hiểu cách TravelNow được triển khai và vận hành, chúng ta chỉ cần tập trung vào một nhóm resource cốt lõi:

Resource	Vai trò trong TravelNow	Giới hạn cần nhớ
Pod	Đơn vị chạy một instance của `flight-service`; có thể chứa một hoặc nhiều container cần chạy cùng nhau	Pod có thể bị thay thế bất kỳ lúc nào; địa chỉ và dữ liệu lưu cục bộ trong Pod không nên được xem là trạng thái bền vững
Deployment	Duy trì số lượng Pod mong muốn và quản lý việc thay thế chúng khi triển khai phiên bản mới	Quản lý vòng đời của Pod nhưng không hiểu trạng thái hay tính đúng đắn của quy trình booking
Service	Cung cấp tên và điểm truy cập ổn định tới một nhóm Pod	Không quyết định workload nào được phép gọi API và cũng không tự đưa ứng dụng ra Internet
ConfigMap / Secret	Cung cấp cấu hình cho workload; Secret dành cho dữ liệu cần được kiểm soát truy cập chặt chẽ hơn	Secret không thay thế các cơ chế mã hóa, phân quyền và xoay vòng credential
ServiceAccount	Cung cấp danh tính Kubernetes cho workload	Quyền truy cập Kubernetes API và quyền truy cập tài nguyên AWS là những cơ chế cần được cấu hình riêng
Job / CronJob	Chạy tác vụ cần hoàn thành một lần hoặc theo lịch, chẳng hạn tác vụ đối soát	Tác vụ có thể được thực hiện lại do retry hoặc lịch chạy; logic xử lý cần an toàn khi cùng một công việc được thực hiện nhiều lần
DaemonSet	Duy trì một Pod trên mỗi node phù hợp, chẳng hạn agent thu thập log	Phân bố workload theo node, không phải cơ chế điều chỉnh số lượng instance của ứng dụng theo tải
Namespace	Nhóm các resource theo môi trường, đội ngũ hoặc phạm vi quản lý	Tạo phạm vi cho tên resource và hỗ trợ phân quyền, nhưng không mặc nhiên tạo ra ranh giới cách ly mạng

Các resource này không hoạt động độc lập mà được liên kết với nhau theo những cách khác nhau. Pod có thể tham chiếu trực tiếp đến **ConfigMap, Secret và ServiceAccount** theo tên. Trong khi đó, **Service không tham chiếu trực tiếp đến từng Pod** mà sử dụng **label** — các cặp khóa/giá trị được gắn vào resource — để xác định những Pod thuộc phạm vi phục vụ của mình:

HÌNH 10.4 · SƠ ĐỒ LUỒNG



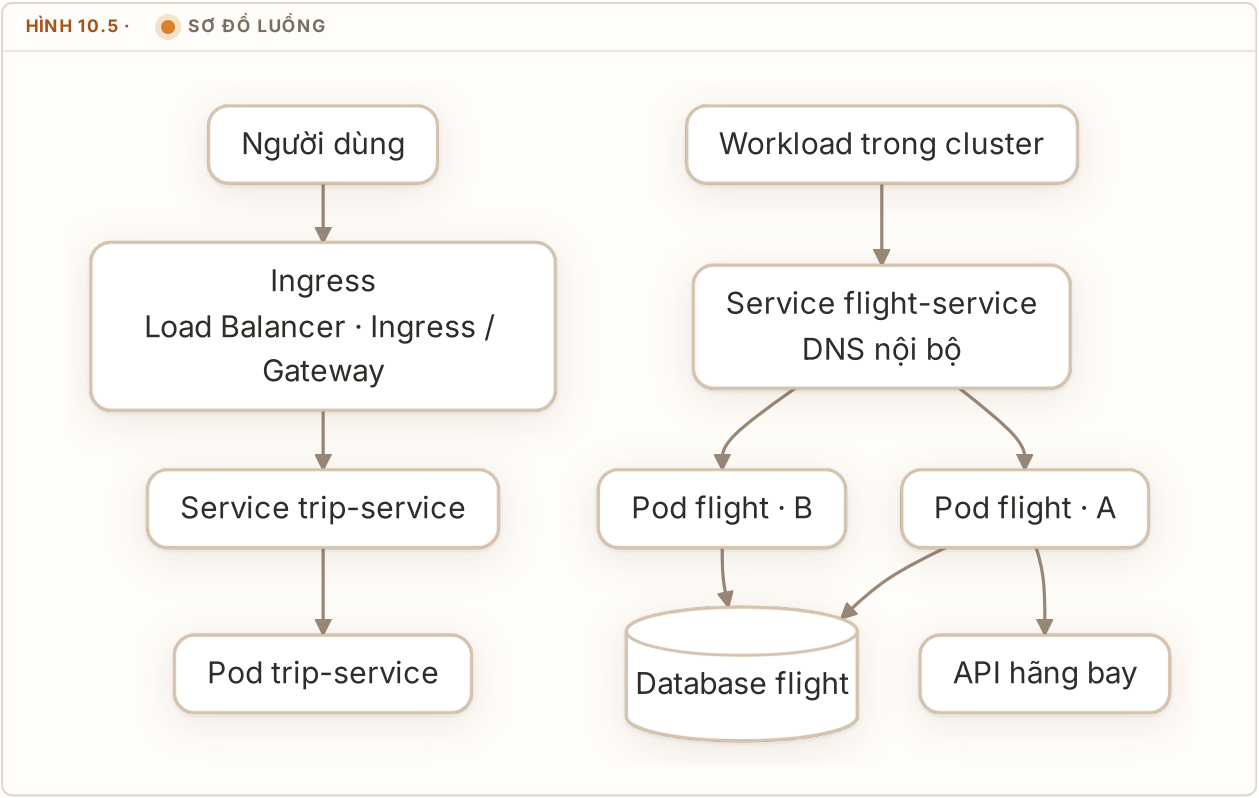
Service không biết Pod nào được tạo bởi Deployment nào. Nó chỉ dựa vào **selector** để tìm những Pod có **label** phù hợp và đưa các Pod đó vào nhóm endpoint của Service. Vì vậy, nếu label của Pod không khớp với selector, Service có thể không có endpoint để chuyển tiếp traffic dù các Pod vẫn đang chạy bình thường. Ngược lại, nếu một Pod thử nghiệm vô tình mang label `app=flight`, Pod đó cũng có thể được Service chọn và nhận traffic của ứng dụng thật. Do đó, khi một service đã chạy nhưng không thể được truy cập thông qua Service, **label và selector là một trong những nơi nên kiểm tra đầu tiên**.

Cũng cần hiểu đúng ý nghĩa của `replicas: 2`. Khai báo này yêu cầu Deployment duy trì hai Pod, nhưng **không đồng nghĩa ứng dụng chắc chắn có hai instance sẵn sàng phục vụ**. Hai Pod có thể cùng được xếp lên một node và cùng mất nếu node đó gặp sự cố. Kubernetes cũng có thể tạo đủ hai Pod nhưng cả hai đều không sẵn sàng, chẳng hạn do cùng sử dụng sai thông tin kết nối database. Vì vậy, số lượng replica chỉ là một phần của bài toán availability. Chương 12 sẽ mở rộng từ khai báo này sang cách phân bố Pod giữa các vùng lỗi và duy trì capacity dự phòng.

10.4 Networking trong Kubernetes: private networking, ingress và egress

Để hiểu networking trong Kubernetes, trước hết cần phân biệt ba hướng kết nối của TravelNow: **private networking giữa các workload trong cluster, ingress đưa traffic từ bên ngoài vào ứng dụng, và egress cho phép workload kết nối tới database hoặc các API bên ngoài**. Mỗi hướng sử dụng những cơ chế khác nhau và có những nguyên nhân gây lỗi riêng.

HÌNH 10.5 · SƠ ĐỒ LUỒNG



10.4.1 Service discovery và địa chỉ nội bộ ổn định

Service kiểu `ClusterIP` cung cấp một điểm truy cập ổn định bên trong cluster. Thông qua DNS của Kubernetes, các workload có thể gọi `flight-service` bằng tên thay vì phải biết địa chỉ IP của từng Pod. Phía sau Service, Kubernetes duy trì danh sách các **endpoint** tương ứng với những Pod được Service chọn và đang sẵn sàng nhận traffic. Khi một Pod bị thay thế, danh sách endpoint được cập nhật để phản ánh các Pod hiện có, vì vậy phía gọi không cần tự theo dõi địa chỉ của từng instance.

Điểm quan trọng ở đây là **trạng thái sẵn sàng của Pod**. Service chỉ chuyển traffic tới những Pod được xác định là sẵn sàng, tạo ra mối liên hệ trực tiếp giữa networking và readiness probe sẽ được trình bày ở Mục 10.7.

Tuy nhiên, Service không loại bỏ hoàn toàn lỗi kết nối. Khi một Pod bị dừng hoặc thay thế, các kết nối đang mở hay request đang được xử lý vẫn có thể thất bại trong quá trình chuyển đổi. Vì vậy, phía gọi vẫn cần sử dụng **timeout và retry có kiểm soát** như đã trình bày ở Chương 7.

10.4.2 Ingress: từ khai báo đến định tuyến traffic

Ingress và **Gateway API** cho phép chúng ta khai báo cách traffic từ bên ngoài được đưa vào và định tuyến tới các Service bên trong cluster. Tuy nhiên, các resource này chỉ mô tả **trạng thái mong muốn**; bản thân chúng không trực tiếp nhận hay chuyển tiếp traffic. Cần có một controller đọc các khai báo đó và cấu hình hạ tầng thực tế, chẳng hạn proxy hoặc load balancer, để thực hiện việc định tuyến.

Tương tự, Service kiểu `LoadBalancer` cũng phụ thuộc vào khả năng tích hợp của môi trường nơi Kubernetes đang chạy. Load balancer được tạo ra là public hay private, sử dụng địa chỉ nào và có thể được truy cập từ đâu đều phụ thuộc vào cấu hình và implementation cụ thể. Vì vậy, chỉ nhìn thấy một Service kiểu `LoadBalancer` chưa đủ để kết luận ứng dụng đã có thể được truy cập từ Internet. Để người dùng thực sự truy cập được TravelNow, **load balancer phải có khả năng nhận traffic từ bên ngoài, DNS phải trỏ tới đúng địa chỉ, TLS phải được cấu hình nếu sử dụng HTTPS và kết nối mạng phải cho phép traffic đi tới ứng dụng**. Kubernetes — Mạng và Service.

Cần phân biệt **controller cấu hình đường đi** với **thành phần thực sự nhận và chuyển tiếp traffic**. Hai vai trò có thể nằm trong cùng một implementation hoặc ở các thành phần khác nhau; sự cố ở mỗi vai trò vì thế cũng có tác động khác nhau.

Ở Chương 11, chúng ta sẽ hiện thực cơ chế này trên AWS bằng **AWS Load Balancer Controller**. Đây cũng là một ví dụ điển hình cho mô hình controller của Kubernetes: một resource được khai báo bên trong cluster có thể khiến controller tạo hoặc thay đổi hạ tầng nằm bên ngoài cluster.

10.4.3 Egress và ranh giới tin cậy

Việc các Pod có thể giao tiếp với nhau trong cluster không có nghĩa `flight-service` chắc chắn kết nối được tới database trên RDS hoặc API của hãng bay. Để một kết nối egress hoạt động, nhiều điều kiện phải đồng thời được đáp ứng: **đường mạng phải tồn tại, DNS phải phân giải được địa chỉ, firewall phải cho phép kết nối và hệ thống đích phải chấp nhận danh tính cũng như quyền của bên gọi**. Khi chẩn đoán sự cố, cần phân biệt hai nhóm lỗi này. Một kết nối bị timeout thường hướng việc kiểm tra về đường mạng hoặc khả năng tiếp cận hệ thống đích, trong khi lỗi từ chối truy cập thường liên quan đến danh tính, thông tin xác thực hoặc quyền.

Một nhầm lẫn khác là coi **"nội bộ" đồng nghĩa với "đáng tin cậy"**. Việc một workload nằm trong cùng cluster không mặc nhiên có nghĩa nó được phép truy cập mọi workload khác. Service cung cấp khả năng tìm kiếm và định tuyến tới Pod, nhưng không đóng vai trò như firewall; Namespace cũng không tự tạo ra ranh giới cách ly mạng.

Kubernetes cung cấp **NetworkPolicy** để khai báo Pod nào được phép nhận hoặc gửi traffic tới những đích nào. Tuy nhiên, các policy này chỉ có tác dụng khi network plugin của cluster hỗ trợ và thực thi chúng. Và ngay cả khi kết nối mạng đã được kiểm soát đúng, service nhận request vẫn cần xác thực danh tính và kiểm tra quyền của bên gọi. Chương 15 sẽ quay lại ranh giới này khi xem xét trường hợp một service trong hệ thống bị chiếm quyền. Kubernetes — NetworkPolicy.

10.5 Stateless và stateful: khả năng thay thế instance

TravelNow lưu thông tin chuyến đi, kết quả thanh toán và tiến độ Saga, nên toàn bộ hệ thống rõ ràng có state. Tuy nhiên, khi chọn cách triển khai trên Kubernetes, điều chúng ta cần xác định là **state nào gắn với từng instance và một instance khác có thể tiếp tục công việc của nó hay không**. Câu hỏi này quyết định cách thay Pod, tăng số replica và phục hồi khi có sự cố.

10.5.1 Stateless không có nghĩa là không có dữ liệu

Giả sử Pod A của `trip-service` tiếp nhận yêu cầu đặt chuyến, ghi Trip cùng outbox trong một transaction rồi trả HTTP `202`. Sau đó, Pod A bị mất. Pod B không cần khôi phục bộ nhớ của A: nó đọc trạng thái Trip từ RDS; relay tìm công việc chưa chuyển tiếp trong outbox; workflow tiếp tục từ tiến độ đã lưu. Nếu client gửi lại yêu cầu, idempotency record giúp nhận ra thao tác trước đó. **Thông tin cần để tiếp tục xử lý không chỉ tồn tại trong instance đã mất.**

Đó là ý nghĩa của **stateless workload** trong mô hình này: các instance có thể thay thế cho nhau mà không cần chuyển giao state riêng của instance trước. Chúng vẫn có bộ nhớ, connection pool, cache và file tạm; những state đó có thể được tạo lại hoặc mất đi trong phạm vi ứng dụng chấp nhận được. Điều cần tránh là đặt thông tin quyết định tính đúng của nghiệp vụ vào nơi chỉ một instance đọc được. Thay instance vẫn có thể làm đứt request đang xử lý; ứng dụng cần timeout, retry và trạng thái bền vững để phục hồi.

State của TravelNow	Nếu chỉ nằm trên Pod A	Thiết kế để Pod khác tiếp tục được
Session đăng nhập	Request tới Pod B không tìm thấy session; mất A khiến khách phải đăng nhập lại	Dùng token phù hợp hoặc session store dùng chung; vẫn kiểm tra quyền trên Trip
Tiến độ tạo PDF	Worker thay thế không biết tài liệu nào còn thiếu	Lưu công việc bền vững, cho phép tạo lại; xác nhận hoàn tất sau khi PDF đã lưu trên S3
Kết quả thu tiền	Pod khác không biết có nên gọi thanh toán lại	Lưu định danh thao tác, kết quả và cơ chế đối soát với cổng thanh toán
Cache giá vé	Phải tải lại; nhiều Pod mới có thể đồng thời gọi hãng bay	Chấp nhận mất cache, nhưng kiểm soát tải khi làm nóng lại

Từ đây, chúng ta có thể đọc đúng một số lựa chọn dễ gây nhầm lẫn. **Sticky session** đưa khách về cùng một Pod có thể che giấu việc session đang nằm trong RAM, nhưng không giúp Pod khác tiếp tục khi Pod đó mất. Tương tự, mount volume giúp dữ liệu sống lâu hơn process, nhưng khả năng thay instance còn phụ thuộc cách sử dụng dữ liệu. Nếu Pod thay thế phải giữ danh tính và nhận lại storage riêng của thành viên cũ, workload vẫn có ràng buộc stateful. Ngược lại, tầng ứng dụng cùng truy cập một nguồn dữ liệu bên ngoài có thể vẫn stateless khi các instance thực sự thay thế được cho nhau.

Việc dùng chung database cũng chưa đủ để nhiều instance xử lý an toàn. Hai relay của TravelNow có thể cùng nhìn thấy một outbox record chưa gửi. Cơ chế nhận quyền xử lý như lease, cùng idempotency ở bên nhận, mới giúp chúng phối hợp đúng. **Stateless tạo điều kiện để thay thế và nhân bản tầng compute; transaction, quyền xử lý và idempotency vẫn quyết định tính đúng khi các instance chạy đồng thời.** Các cơ chế ở Phần III vì thế là nền tảng của việc co giãn ở Chương 12.

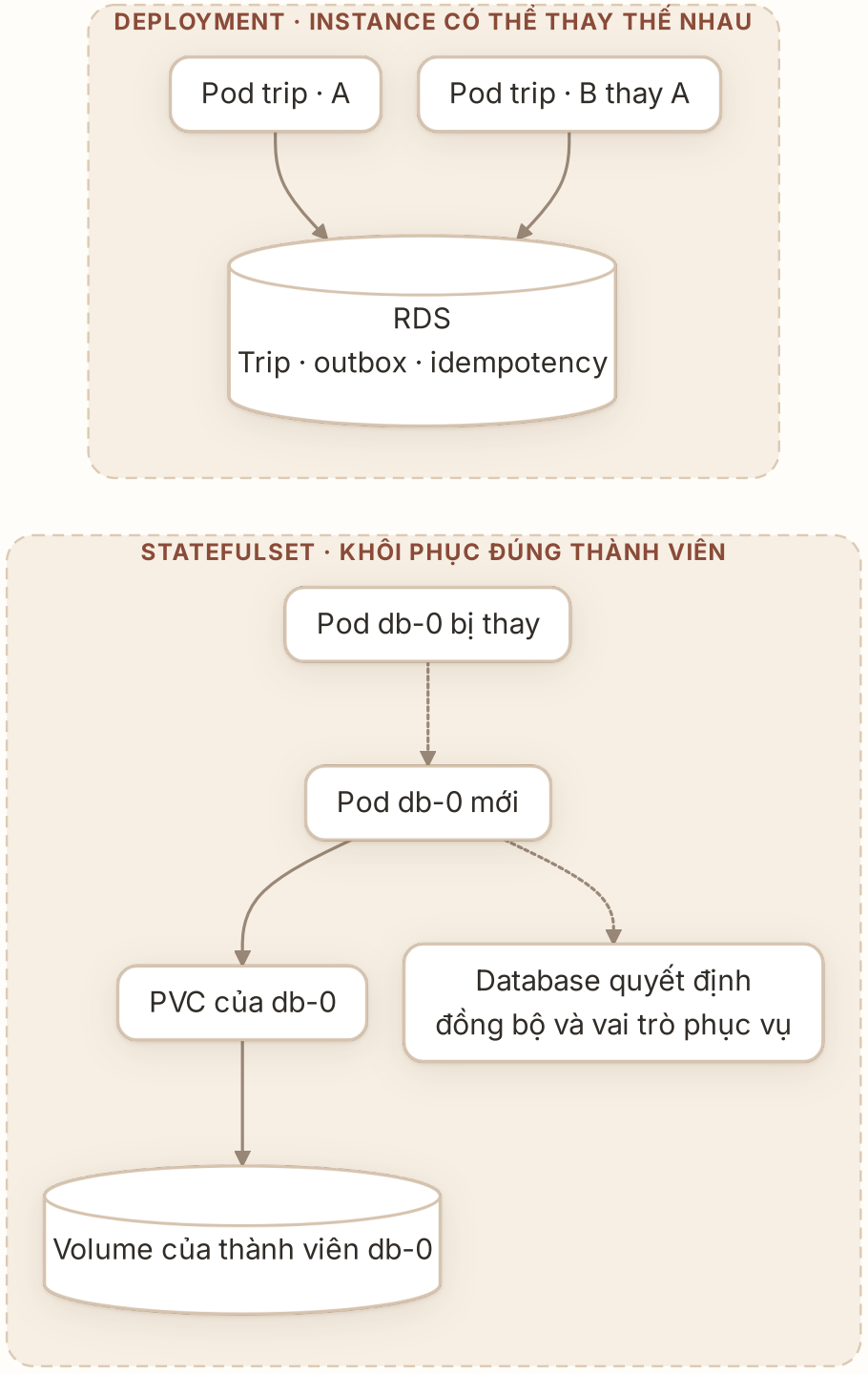
10.5.2 Khi state gắn với danh tính của từng instance

Để thấy rõ sự khác biệt, hãy xét một phương án giả định trong đó đội ngũ tự vận hành PostgreSQL cho TravelNow trên Kubernetes. Mỗi thành viên của cụm database lưu dữ liệu trên storage riêng và tham gia vào cơ chế replication với các thành viên khác. Trong trường hợp này, chỉ khởi chạy thêm một container PostgreSQL với volume rỗng chưa đủ để tạo ra một replica có thể hoạt động. Thành viên mới còn phải được khởi tạo dữ liệu, đồng bộ với các thành viên hiện có và gia nhập cụm theo cơ chế của chính database. Tương tự, khi một thành viên bị thay thế, hệ thống cần xác định **đó là thành viên nào và storage nào thuộc về nó**.

Đây là đặc điểm của **stateful workload**: các instance không hoàn toàn có thể thay thế cho nhau vì **state, danh tính hoặc vai trò của từng instance có ý nghĩa đối với hoạt động và khả năng phục hồi của hệ thống**. Database, message broker và các hệ thống lưu trữ phân tán thường có những yêu cầu như vậy. Các workload này vẫn có thể được tự động hóa và điều chỉnh số lượng instance, nhưng quá trình đó không chỉ là tạo hoặc xóa container mà còn phải xử lý việc đồng bộ dữ liệu, quản lý thành viên và xác định instance nào được phép đảm nhận từng vai trò.

Kubernetes cung cấp **StatefulSet** để hỗ trợ một phần những yêu cầu này. Mỗi Pod có một danh tính ổn định, chẳng hạn `db-0`, và khi Pod được tạo lại, nó có thể tiếp tục sử dụng **PersistentVolumeClaim (PVC)** gắn với danh tính đó. PVC mô tả yêu cầu về storage và được liên kết với PersistentVolume cung cấp storage thực tế. Với `volumeClaimTemplates`, StatefulSet có thể tạo một PVC riêng cho từng Pod và duy trì mối quan hệ giữa **danh tính của Pod và storage tương ứng** ngay cả khi Pod cũ bị xóa và một Pod mới được tạo ra để thay thế. Kubernetes — StatefulSet.

HÌNH 10.6 · SƠ ĐỒ LUỒNG



Cần quản lý	Deployment	StatefulSet
Danh tính	Các Pod được thiết kế để có thể thay thế cho nhau; không phụ thuộc vào danh tính cố định của từng Pod	Mỗi Pod có tên và định danh mạng ổn định, chẳng hạn `db-0`, `db-1`; địa chỉ IP vẫn có thể thay đổi

Cần quản lý	Deployment	StatefulSet
Storage	Có thể sử dụng PVC, nhưng Deployment không tự duy trì một PVC riêng gắn với danh tính của từng replica	Có thể tạo PVC riêng cho từng Pod và duy trì mối quan hệ giữa danh tính của Pod với storage tương ứng khi Pod được tạo lại
Vòng đời	Các Pod được tạo và thay thế theo chiến lược của Deployment, không dựa trên thứ tự cố định giữa các replica	Có thể kiểm soát thứ tự tạo, dừng và cập nhật các Pod; hành vi cụ thể phụ thuộc vào policy được cấu hình
Trách nhiệm của ứng dụng	Ứng dụng vẫn phải phối hợp khi nhiều instance cùng truy cập dữ liệu, xác định quyền xử lý và bảo vệ các thao tác khỏi việc thực hiện trùng lặp	StatefulSet không tự quản lý dữ liệu; hệ thống vẫn phải xử lý việc quản lý thành viên, replication, vai trò của từng thành viên và tính nhất quán của dữ liệu

Cần hiểu rõ rằng những cơ chế trên **không khiến StatefulSet trở thành một hệ thống quản lý database**. Việc một Pod có tên `db-0` không có nghĩa Pod đó mặc nhiên giữ vai trò primary; tương tự, khai báo `replicas: 3` chỉ yêu cầu StatefulSet duy trì ba Pod, chứ không bảo đảm dữ liệu giữa ba thành viên được replication và duy trì nhất quán.

StatefulSet quản lý danh tính, storage gắn với danh tính và vòng đời của các Pod; chính database mới quyết định cách dữ liệu được replication, thành viên nào giữ vai trò primary và thành viên nào được phép phục vụ thao tác ghi. Một operator chuyên biệt có thể tự động hóa nhiều công việc này dựa trên cơ chế của database, nhưng đội ngũ vẫn cần xác định cách backup, phục hồi dữ liệu và bảo đảm khả năng tương thích khi nâng cấp.

10.5.3 Dữ liệu còn tồn tại chưa có nghĩa workload đã phục hồi

Với `trip-service` được thiết kế theo hướng stateless, khi một Pod bị mất, Kubernetes có thể tạo Pod thay thế trên một node phù hợp; Pod mới sau đó kết nối tới RDS và tiếp tục làm việc dựa trên state đã được lưu bên ngoài. Với PostgreSQL sử dụng EBS trong phương án giả định, quá trình phục hồi phức tạp hơn. Nếu Pod được chuyển sang node khác, volume có thể phải được tách khỏi node cũ, gắn vào node mới, sau đó PostgreSQL mới có thể mở lại dữ liệu và tiếp tục hoạt động. Vì vậy, **cluster còn đủ CPU và bộ nhớ chưa có nghĩa stateful workload có thể lập tức được chạy ở một node khác**. Với EBS, node mới còn phải nằm trong cùng Availability Zone với volume; một node còn tài nguyên ở AZ khác không thể đơn giản gắn volume cũ để tiếp tục chạy workload. AWS — Gắn EBS volume.

Ở đây cũng cần hiểu chính xác ý nghĩa của access mode. `ReadWriteOnce` **cho phép volume được mount ở chế độ đọc/ghi bởi một node, chứ không có nghĩa volume chỉ có thể được sử dụng bởi một Pod.** Nhiều Pod trên cùng node vẫn có thể truy cập volume đó; còn việc nhiều process cùng đọc hoặc ghi dữ liệu có an toàn hay không phụ thuộc vào storage và chính ứng dụng. Nếu yêu cầu là volume chỉ được một Pod mount ở chế độ đọc/ghi, Kubernetes cung cấp `ReadWriteOncePod` với CSI driver phù hợp. Kubernetes — Access modes.

Storage vẫn chỉ là một phần của bài toán phục hồi. Khi một node mất liên lạc, Kubernetes không thể ngay lập tức khẳng định process trên node đó đã thực sự dừng. Với những hệ thống mà nhiều thành viên không được phép đồng thời ghi vào cùng một state, việc khởi động thành viên

thay thế trước khi chắc chắn thành viên cũ đã mất quyền ghi có thể dẫn đến xung đột hoặc hỏng dữ liệu. Vì vậy, trước khi trao quyền cho thành viên mới, hệ thống có thể cần một cơ chế **fencing** để bảo đảm thành viên cũ không còn khả năng tiếp tục ghi. Việc force delete một Pod chỉ loại bỏ đối tượng Pod khỏi Kubernetes API; nó không chứng minh process trên node cũ đã ngừng hoạt động. Kubernetes — Force delete StatefulSet Pods.

Với TravelNow, chúng ta tiếp tục giữ lựa chọn đã đưa ra từ Chương 2–3: database được đặt trên RDS, file PDF được lưu trên S3, còn tầng ứng dụng được thiết kế để các instance có thể thay thế cho nhau. State không biến mất khỏi hệ thống; nó vẫn cần được replication, backup và phục hồi, nhưng một phần trách nhiệm đó được giao cho các dịch vụ được quản lý. Nếu lựa chọn tự vận hành PostgreSQL trên Kubernetes, đội ngũ sẽ phải tiếp nhận thêm những trách nhiệm tương ứng. Vì vậy, **vấn đề cốt lõi không chỉ là lựa chọn Deployment hay StatefulSet, mà là xác định thành phần nào chịu trách nhiệm quản lý vòng đời, tính nhất quán và khả năng phục hồi của state**.

10.6 Chiến lược phát hành và tương thích phiên bản

10.6.1 Lựa chọn chiến lược phát hành

Giả sử phiên bản v2 của `trip-service` thay đổi trường `checkIn` thành `checkInDate`. Kubernetes có thể triển khai phiên bản mới đúng theo chiến lược đã cấu hình, các Pod mới đều khởi động thành công và vượt qua health check, nhưng những consumer vẫn sử dụng cấu trúc dữ liệu cũ có thể không xử lý được event do phiên bản mới tạo ra. Điều này cho thấy một ranh giới quan trọng: **một phiên bản được triển khai thành công ở tầng hạ tầng chưa đồng nghĩa nó tương thích với các thành phần khác của hệ thống**.

Vì vậy, khi lựa chọn chiến lược phát hành, chúng ta không chỉ quan tâm đến cách Kubernetes thay thế Pod cũ bằng Pod mới. Các chiến lược khác nhau chủ yếu ở ba khía cạnh: **ứng dụng có duy trì khả năng phục vụ trong suốt quá trình phát hành hay không, có cần thêm capacity để chạy đồng thời phiên bản cũ và mới hay không, và có thể đưa một phần traffic sang phiên bản mới trước khi triển khai rộng hơn hay không**.

Chiến lược	Cách triển khai	Đánh đổi cần cân nhắc
Recreate	Dừng toàn bộ Pod của phiên bản cũ trước khi tạo Pod chạy phiên bản mới	Có khoảng thời gian ứng dụng không thể phục vụ; phù hợp khi hai phiên bản không thể chạy đồng thời
Rolling update	Thay thế lần lượt các Pod cũ bằng Pod mới, vì vậy hai phiên bản cùng tồn tại trong một khoảng thời gian	Cần đủ capacity trong quá trình rollout và ứng dụng phải hỗ trợ việc phiên bản cũ và mới hoạt động đồng thời
Blue/green	Chuẩn bị một nhóm Pod chạy phiên bản mới song song với nhóm hiện tại, sau đó chuyển traffic sang nhóm mới	Cần thêm capacity để duy trì đồng thời hai môi trường; chuyển traffic trở lại phiên bản cũ không tự hoàn tác những thay đổi đã xảy ra với dữ liệu

Chiến lược	Cách triển khai	Đánh đổi cần cân nhắc
Canary	Chỉ đưa một phần traffic hoặc workload sang phiên bản mới, theo dõi kết quả rồi mới quyết định mở rộng phạm vi triển khai	Cần cơ chế phân chia traffic và các chỉ số đủ tin cậy để quyết định tiếp tục, mở rộng hay dừng phiên bản mới

Kubernetes Deployment hỗ trợ trực tiếp hai chiến lược **Recreate** và **RollingUpdate**. Với **blue/green** hoặc **canary**, chúng ta cần kết hợp thêm các nhóm workload chạy song song, cơ chế định tuyến traffic và một quy trình hoặc controller để điều phối quá trình phát hành. Kubernetes — Deployment.

10.6.2 Rolling update diễn ra như thế nào?

Với **RollingUpdate**, Kubernetes kiểm soát quá trình thay thế Pod thông qua hai tham số chính. `maxSurge` xác định số Pod tối đa có thể được tạo thêm so với số replica mong muốn trong quá trình rollout, còn `maxUnavailable` xác định số Pod tối đa có thể ở trạng thái chưa sẵn sàng trong cùng thời điểm.

Giả sử `flight-service` được cấu hình với ba replica, `maxSurge: 1` và `maxUnavailable: 0`. Kubernetes có thể tạo thêm một Pod chạy phiên bản mới, nhưng chỉ dừng Pod cũ sau khi Pod mới đã sẵn sàng. Quá trình thay thế lần lượt diễn ra như sau:

HÌNH 10.7 · SƠ ĐỒ LUỒNG



Sơ đồ trên cho thấy hai điểm quan trọng. Thứ nhất, trong một phần của quá trình rollout, **các Pod chạy v1 và v2 cùng tồn tại và có thể cùng nhận traffic**. Vì vậy, hai phiên bản phải có khả năng hoạt động đồng thời với các contract mà hệ thống đang sử dụng, chẳng hạn API, event hoặc schema dữ liệu.

Thứ hai, **readiness của Pod v2 ảnh hưởng trực tiếp đến quá trình rollout**. Kubernetes chỉ tiếp tục thay thế Pod cũ khi Pod mới được xác định là sẵn sàng. Nếu Pod v2 không đạt trạng thái này, rollout không thể tiếp tục và các Pod v1 vẫn được giữ lại để phục vụ. Ngược lại, nếu readiness probe báo sẵn sàng khi ứng dụng chưa thực sự có khả năng xử lý request, Kubernetes có thể tiếp tục dừng các Pod v1 và đưa một phiên bản chưa hoạt động đúng vào phục vụ.

Do đó, cấu hình `maxUnavailable: 0` không tự bảo đảm một rollout không gián đoạn. Cluster vẫn cần đủ capacity để tạo thêm Pod mới theo `maxSurge`; readiness phải phản ánh khả năng nhận việc, còn Pod cũ cần dừng an toàn với công việc đang xử lý. Deployment cũng không tự rollback khi rollout không tiến triển: pipeline hoặc người vận hành cần quyết định dừng, sửa hay quay lại phiên bản trước. Kubernetes — Failed Deployment.

10.6.3 Giới hạn của rollback: dữ liệu và cấu hình

Ở Chương 4, chúng ta đã sử dụng nguyên tắc **expand → migrate → contract** để thay đổi contract một cách an toàn: trước tiên mở rộng hệ thống để chấp nhận cả cấu trúc cũ và mới, sau đó chuyển dần sang cách ghi mới, và chỉ loại bỏ cấu trúc cũ khi chắc chắn không còn thành phần nào cần sử dụng nó. Với Kubernetes, nguyên tắc này càng quan trọng vì trong quá trình rolling update, phiên bản cũ và mới có thể cùng hoạt động, rollout có thể dừng giữa chừng và `kubectl rollout undo` có thể đưa các Pod chạy phiên bản cũ trở lại. Nếu database schema hoặc event schema đã được thay đổi theo cách phiên bản cũ không còn hiểu được, khả năng rollback ở tầng Deployment cũng không thể đưa toàn bộ hệ thống trở về trạng thái trước khi phát hành.

Cấu hình cũng cần được xem là một phần của phiên bản phát hành. Chẳng hạn, ngay cả khi image của `flight-service` không thay đổi, việc tăng số lần retry vẫn có thể làm tăng đáng kể lượng request gửi tới API của hãng bay. Với ConfigMap được truyền vào container qua biến môi trường, Pod đang chạy không tự nhận giá trị mới mà cần được thay thế. Khi ConfigMap được mount dưới dạng volume, nội dung có thể được cập nhật nhưng điều đó không đồng nghĩa ứng dụng sẽ tự động đọc lại cấu hình. Vì vậy, mỗi lần phát hành cần xác định rõ phiên bản cấu hình được sử dụng và cách thay đổi đó được áp dụng vào workload. Rollback image nhưng vẫn giữ cấu hình gây ra sự cố có thể không khôi phục được hành vi của phiên bản trước. Ý này phát triển từ phần quản lý cấu hình trong *Kubernetes: Up and Running*; hành vi cập nhật ConfigMap được mô tả trong tài liệu Kubernetes.

Giới hạn tương tự cũng áp dụng cho **canary**. Với TravelNow, việc phiên bản mới nhận request thành công chưa đủ để kết luận bản phát hành hoạt động đúng. HTTP `202` chỉ cho biết hệ thống đã tiếp nhận yêu cầu; kết quả nghiệp vụ có thể chỉ xuất hiện sau khi các bước phía sau hoàn tất. Vì vậy, các tín hiệu dùng để đánh giá canary cần phản ánh cả **kết quả của quá trình booking và thanh toán**, thay vì chỉ dựa vào trạng thái HTTP ở điểm tiếp nhận request.

10.6.4 Khi rollout bị giới hạn bởi tài nguyên dành riêng

Các tham số rollout chỉ hữu ích khi phiên bản mới thực sự có nơi chạy. Một tình huống từ bài viết *Kubernetes Controllers, Deployment Strategies và một vài bài học* làm rõ điều này: workload cần dùng một cổng cố định trên node, nhưng các node đủ điều kiện đều đã có Pod cũ chiếm cổng đó. Với rolling update tạo Pod mới trước khi dừng Pod cũ, Pod mới có thể không được xếp vào node hoặc không bind được cổng, tùy cách khai báo. Còn CPU và RAM không giải quyết được xung đột cổng.

Đổi sang **Recreate** giải phóng tài nguyên trước khi tạo phiên bản mới, nhưng đồng thời chấp nhận gián đoạn. Nếu yêu cầu phục vụ không cho phép gián đoạn, cần xem lại khả năng bổ sung node phù hợp hoặc cách workload sử dụng cổng. Bài học ở đây là **capacity dành cho rollout bao gồm cả những tài nguyên và ràng buộc không thể chia sẻ**, thay vì chỉ số CPU và dung lượng bộ nhớ.

Ràng buộc tương tự xuất hiện khi các Pod của một Deployment cùng tham chiếu một PVC: Pod mới trên node khác có thể không mount được volume đang dùng trên node cũ. Chuyển sang StatefulSet không tự giải quyết việc chia sẻ dữ liệu; PVC riêng cho từng Pod cũng không tự sao

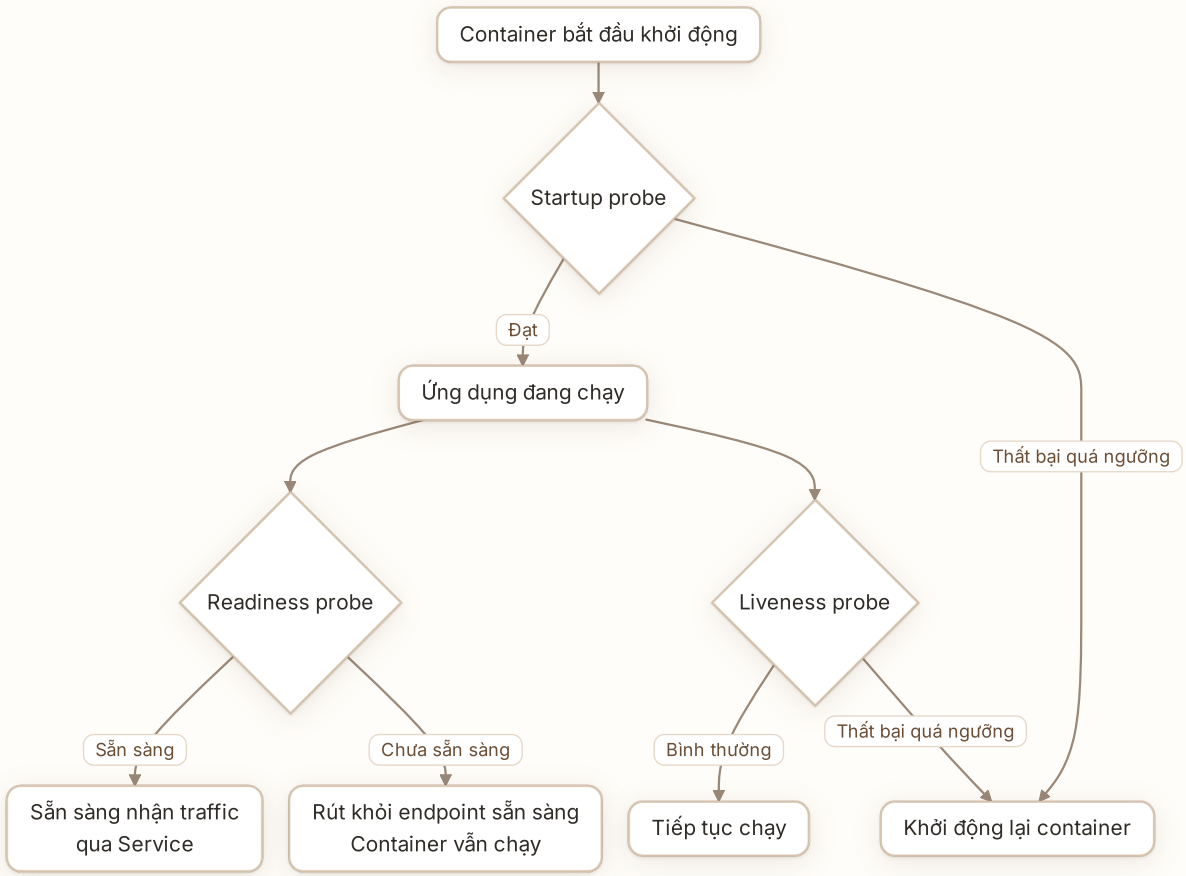
chép dữ liệu giữa chúng. Cách cập nhật phải phù hợp với mô hình sở hữu và phục hồi state ở Mục 10.5.

Cũng cần phân biệt hai controller: **RollingUpdate của StatefulSet** theo cơ chế thông thường thay từng Pod theo danh tính, không dùng mô hình tạo thêm replica với `maxSurge` của Deployment. Với **OnDelete**, controller chỉ áp dụng template mới khi Pod cũ bị xóa và được tạo lại. Khi vận hành database, thứ tự cập nhật còn phải phù hợp với replication và vai trò của từng thành viên. Kubernetes — StatefulSet update strategies.

10.7 Probe và quyết định phục hồi

Ở Chương 3, chúng ta đã phân biệt **liveness** và **readiness** khi chuẩn bị ứng dụng để chạy trong container. Trong Kubernetes, sự khác biệt này đặc biệt quan trọng vì kết quả của mỗi loại probe dẫn đến **một hành động khác nhau của nền tảng**. Vì vậy, thay vì bắt đầu bằng câu hỏi *"health endpoint nên kiểm tra những gì?"*, chúng ta nên bắt đầu từ câu hỏi **"Kubernetes nên làm gì khi kiểm tra này thất bại?"**

HÌNH 10.8 · SƠ ĐỒ LUỒNG



Hai nguyên tắc sau giúp tránh phần lớn sai lầm khi thiết kế probe.

Liveness chỉ nên kiểm tra những trạng thái mà việc restart container có thể khắc phục. Giả sử RDS tạm thời không thể truy cập và liveness probe của tất cả Pod đều kiểm tra kết nối tới database. Khi RDS gặp sự cố, các probe có thể đồng loạt thất bại, khiến nhiều container bị restart cùng lúc. Việc restart không giải quyết được sự cố của RDS; ngược lại, các Pod mất trạng thái tạm thời như cache và khi khởi động lại có thể đồng loạt tạo kết nối mới đúng lúc database đang phục hồi. Một sự cố của dependency vì thế có thể bị khuếch đại thành sự cố của cả service. Liveness phù hợp hơn để phát hiện những trạng thái lỗi nằm trong chính process và có khả năng được khắc phục bằng restart, chẳng hạn ứng dụng bị treo và không còn xử lý được công việc.

Readiness phản ánh khả năng nhận traffic của Pod, không phải tình trạng sống hay chết của process. Việc readiness có nên phụ thuộc vào database hay một dependency khác cần được quyết định dựa trên khả năng thực tế của ứng dụng: khi dependency đó không hoạt động, Pod còn có thể phục vụ những API nào và việc loại tất cả Pod khỏi Service sẽ ảnh hưởng tới người dùng ra sao?

Cũng cần nhớ rằng readiness chủ yếu ảnh hưởng đến việc Pod có được sử dụng làm endpoint để nhận traffic qua Service hay không. Nó không tự dừng những công việc mà ứng dụng chủ động lấy về xử lý. Chẳng hạn, một worker đọc message từ SQS không ngừng polling chỉ vì readiness chuyển sang `false`. Khi Pod cần dừng, chính worker phải phản ứng với tín hiệu shutdown, ngừng nhận công việc mới, hoàn tất hoặc xử lý an toàn công việc đang thực hiện và chỉ xác nhận message sau khi kết quả cần thiết đã được lưu thành công. Đây chính là quy trình **graceful shutdown** đã được trình bày ở Chương 3. Kubernetes — Container probes.

Thời gian chờ shutdown cũng là một phần của cấu hình vận hành. Khi dừng Pod theo quy trình thông thường, Kubernetes cho container một khoảng thời gian để kết thúc trước khi buộc dừng. Thời gian chạy hook `preStop`, nếu có, nằm trong `terminationGracePeriodSeconds`; không nên dành toàn bộ khoảng này cho một lệnh chờ rồi mới để ứng dụng xử lý tín hiệu dừng. Việc cập nhật endpoint và ngừng nhận traffic cũng không diễn ra tức thời trên mọi tầng. Vì vậy, cần đo quá trình dừng qua chính đường ALB, Service và worker sẽ dùng ở môi trường thật. Kubernetes — Pod termination.

Giả sử Pod `trip-service` dừng ngay sau khi commit Trip và outbox nhưng trước khi dispatcher khởi chạy Saga. Graceful shutdown có thể giúp hoàn tất chặng đang chạy, nhưng nếu hết thời gian hoặc node mất đột ngột, ý định còn trong outbox mới là thứ cho phép tiếp tục. Nếu chỉ lưu Trip rồi gửi công việc bằng một luồng trong RAM, Pod thay thế không biết còn việc phải gửi. **Shutdown giảm gián đoạn; trạng thái bền vững và idempotency bảo vệ công việc khi không thể dừng êm.** Chương 12 sẽ kiểm chứng hai trường hợp này dưới tải.

> **Nếu muốn quan sát cơ chế trên máy cá nhân**
>
> kind, minikube và k3d là những lựa chọn mã nguồn mở để tạo Kubernetes cluster trên máy cá nhân. Chỉ cần một ứng dụng HTTP đơn giản với hai replica, chúng ta đã có thể xóa một Pod để quan sát controller tạo Pod thay thế, hoặc thực hiện rolling update để theo dõi các Pod và endpoint thay đổi trong quá trình rollout.

> Không cần triển khai lại toàn bộ TravelNow để quan sát những cơ chế này. Tuy nhiên, cluster local chỉ giúp minh họa hành vi của Kubernetes; nó không kiểm chứng được những đặc điểm phụ thuộc vào môi trường AWS, chẳng hạn đường mạng thực tế hay khả năng chịu lỗi khi workload được phân bố giữa nhiều Availability Zone.

10.8 Tóm tắt

Container orchestration cho phép tài nguyên của nhiều máy chủ được quản lý như một nguồn capacity chung. Thay vì gắn từng workload với một máy chủ cụ thể, chúng ta khai báo **trạng thái mong muốn** và để nền tảng liên tục quan sát, đối chiếu và điều chỉnh trạng thái thực tế. Kubernetes thực hiện mô hình này thông qua một API khai báo có khả năng mở rộng bằng các resource và controller. Khả năng mở rộng đem lại nhiều lựa chọn, nhưng mỗi thành phần được bổ sung vào cluster cũng kéo theo trách nhiệm cấu hình, giám sát và vận hành tương ứng.

Các resource cốt lõi của Kubernetes đảm nhận những vai trò khác nhau và phối hợp để duy trì workload. **Deployment** quản lý vòng đời và số lượng Pod, **Service** cung cấp điểm truy cập ổn định tới các Pod phù hợp, còn **probe** giúp Kubernetes quyết định khi nào một Pod sẵn sàng nhận traffic và khi nào container cần được khởi động lại. Ở tầng networking, cần phân biệt **private networking giữa các workload, ingress và egress**; việc một workload nằm bên trong cluster không mặc nhiên khiến nó trở thành một thành phần đáng tin cậy.

Với state, điều cần xác định là **một instance khác có thể tiếp tục công việc hay phải khôi phục state gắn với danh tính của instance cũ**. Stateless workload vẫn có dữ liệu và state tạm thời; khả năng thay thế đến từ việc giữ thông tin cần thiết ở nơi bền vững, kết hợp transaction, quyền xử lý và idempotency. StatefulSet hỗ trợ danh tính cùng quan hệ storage cho từng thành viên, nhưng replication, vai trò phục vụ và phục hồi dữ liệu vẫn thuộc về hệ thống lưu trữ. Có Pod mới hoặc còn volume chưa chứng minh workload đã phục hồi.

Cuối cùng, phát hành phiên bản mới không chỉ là quá trình thay Pod cũ bằng Pod mới. Trong rolling update, hai phiên bản có thể cùng hoạt động trong một khoảng thời gian; rollback cũng chỉ thực sự hữu ích nếu dữ liệu, contract và cấu hình vẫn tương thích với phiên bản được đưa trở lại. Vì vậy, **khả năng tương thích giữa các phiên bản và tính đúng đắn của ứng dụng vẫn là trách nhiệm nằm phía trên những cơ chế rollout mà Kubernetes cung cấp**.

Sang Chương 11, chúng ta sẽ đặt những cơ chế này vào môi trường AWS. Trước hết, chúng ta sẽ xem AWS cung cấp những lựa chọn nào để chạy container, trách nhiệm được phân chia giữa AWS và đội ngũ vận hành ra sao trong từng mô hình, và từ đó đi đến lựa chọn **Amazon EKS** cho TravelNow.

CHƯƠNG 11

TravelNow trên Amazon EKS

Phần IV — Vận hành ở quy mô lớn · *Operating at Scale*

Chương 10 đã cho thấy cách Kubernetes duy trì workload theo trạng thái mong muốn, kết nối các instance thông qua Service và kiểm soát quá trình phát hành phiên bản mới. Tuy nhiên, những cơ chế đó mới được xem xét trong một cluster chung chung. Khi đưa TravelNow lên AWS, chúng ta phải trả lời thêm nhiều câu hỏi cụ thể hơn: cluster được tạo và nâng cấp như thế nào, image được lưu ở đâu, mỗi workload dùng danh tính nào để gọi API AWS, và các Pod kết nối ra sao với RDS, S3, Step Functions cùng những dịch vụ đã có từ Phần III.

AWS cung cấp nhiều phương án để chạy container, từ ECS và EKS ở tầng orchestration đến EC2 và Fargate ở tầng compute. Mỗi lựa chọn phân chia trách nhiệm vận hành giữa AWS và team của bạn theo một cách khác nhau. Chương này bắt đầu bằng việc đối chiếu các phương án đó, sau đó triển khai TravelNow trên **Amazon EKS**. Qua từng bước, chúng ta sẽ làm rõ **AWS chịu trách nhiệm vận hành những gì, Kubernetes đảm nhận việc điều phối những gì và những gì vẫn thuộc trách nhiệm của ứng dụng**.

Việc chọn EKS trong chương này phục vụ mục tiêu tìm hiểu container orchestration trên một nền tảng managed, không có nghĩa phương án Lambda ở Chương 9 kém phù hợp hơn. Với nhiều workload, Lambda hoặc ECS vẫn là lựa chọn hợp lý; Chương 13 sẽ đặt các phương án này cạnh nhau để đánh giá.

Cách đọc chương này

Mục 11.1 so sánh các cách chạy container trên AWS và giải thích lý do TravelNow dùng EKS. Mục 11.2 làm rõ phạm vi trách nhiệm của managed control plane. Mục 11.3–11.5 đi qua ba nhóm công việc khi chuyển TravelNow lên EKS: **ánh xạ kiến trúc Phần III, cấp danh tính cho workload và thiết lập networking**. Mục 11.6 trình bày cách kiểm chứng một lần triển khai; Mục 11.7 đánh giá giá trị và chi phí của một nền tảng Kubernetes dùng chung.

11.1 Các lựa chọn chạy container trên AWS

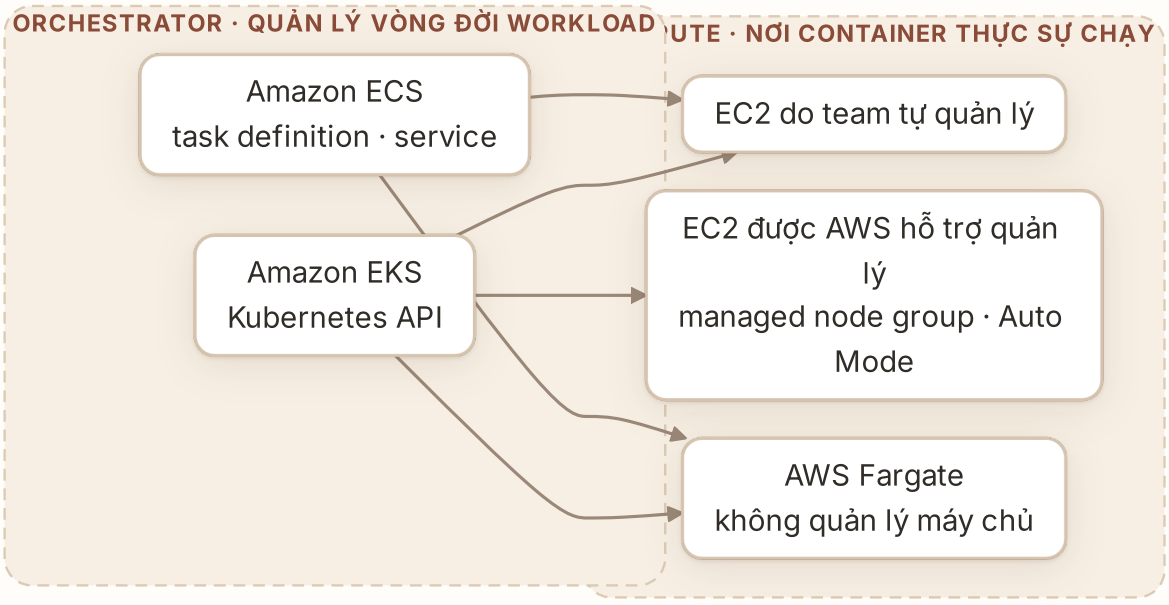
11.1.1 Orchestrator và compute: hai tầng lựa chọn

Khi lựa chọn cách chạy container trên AWS, chúng ta thường gặp câu hỏi: **"Nên dùng ECS hay Fargate?"**. Tuy nhiên, ECS và Fargate giải quyết **hai bài toán khác nhau**. Để lựa chọn đúng, trước hết cần tách hai quyết định:

1. **Chọn orchestrator (container orchestration):** sử dụng mô hình task definition và service của **Amazon ECS**, hay sử dụng Kubernetes API thông qua **Amazon EKS** để triển khai, duy trì và quản lý vòng đời workload?
2. **Chọn compute (môi trường thực thi):** workload chạy trên EC2 do team tự quản lý, trên node được AWS hỗ trợ quản lý vòng đời, hay trên Fargate, nơi bạn không còn phải quản lý máy chủ?

Hai tầng này có thể kết hợp theo nhiều cách, trong phạm vi tính năng mà mỗi phương án hỗ trợ. ECS có thể chạy task trên EC2 hoặc Fargate. EKS có thể chạy Pod trên node tự quản lý, managed node group, node do EKS Auto Mode cấp phát hoặc Fargate, tùy yêu cầu và mức độ tương thích của workload. Như vậy, **Fargate là một lựa chọn compute có thể được sử dụng với cả ECS và EKS**. Phân biệt hai tầng giúp chúng ta xác định rõ orchestrator nào quản lý workload và compute nào cung cấp tài nguyên để workload thực thi.

HÌNH 11.1 · SƠ ĐỒ LUỒNG



11.1.2 Phân chia trách nhiệm giữa AWS và team vận hành

Sau khi phân biệt hai quyết định trên, chúng ta có thể xem xét **AWS đảm nhận những phần việc nào và team bạn vẫn phải chịu trách nhiệm những gì** trong từng phương án. Bảng dưới đây bổ sung Kubernetes tự vận hành trên EC2 và Lambda ở Chương 9 làm hai mốc tham chiếu. Không có phương án phù hợp cho mọi hệ thống; lựa chọn cuối cùng cần dựa trên **yêu cầu của workload và phạm vi vận hành mà bạn muốn tự đảm nhận**.

Phương án	AWS vận hành	Bạn vẫn chịu trách nhiệm
Kubernetes tự dựng trên EC2	Máy ảo và hạ tầng mạng vật lý	Control plane, `etcd`, node, nâng cấp phiên bản, add-on và workload
EKS với node tự quản lý hoặc managed node group	Control plane trên nhiều AZ; managed node group hỗ trợ thêm vòng đời node	Phiên bản cluster, add-on, loại và số lượng node, workload, phân quyền
EKS Auto Mode	Control plane, cùng việc cấp phát, cập nhật node và một số thành phần hạ tầng của cluster	Chính sách cho node, workload, phân quyền, networking của ứng dụng, lịch nâng cấp phiên bản
ECS trên Fargate	Tầng orchestration và môi trường chạy task	Task definition, tài nguyên cho mỗi task, networking, phân quyền, chiến lược phát hành
Lambda	Cấp phát môi trường thực thi và tự động điều chỉnh năng lực xử lý theo số lượng request	Mã nguồn, giới hạn concurrency, timeout, nguồn kích hoạt, phân quyền

Dù chọn phương án nào, bạn vẫn phải chịu trách nhiệm về contract giữa các service, tính đúng đắn của dữ liệu và phạm vi quyền của ứng dụng. Lambda giúp chúng ta không phải quản lý máy chủ, nhưng ở Chương 9 chúng ta vẫn phải thiết kế outbox, idempotency và giới hạn concurrency để bảo vệ RDS. Nói cách khác, **managed service thu hẹp phạm vi vận hành hạ tầng, nhưng không thu hẹp trách nhiệm đối với tính đúng đắn của nghiệp vụ**. AWS — Compute cho EKS.

11.1.3 Khi nào chọn Amazon ECS, khi nào chọn Amazon EKS?

Với TravelNow, câu hỏi đầu tiên không phải "hệ thống đã đủ lớn để dùng Kubernetes chưa", mà là **team có thực sự cần Kubernetes API hay không**. Nhu cầu đó thường xuất phát từ một trong ba lý do: tổ chức đã vận hành một nền tảng Kubernetes dùng chung, workload phụ thuộc vào một controller hoặc công cụ trong hệ sinh thái Kubernetes, hoặc cần duy trì cùng một mô hình triển khai trên nhiều môi trường, kể cả ngoài AWS.

Nếu mô hình task và service của ECS đã đáp ứng được yêu cầu, ECS thường là lựa chọn đơn giản hơn: bạn không phải quản lý phiên bản cluster hay add-on, và chỉ cần chọn tiếp giữa EC2 và Fargate theo yêu cầu compute. Nếu chọn EKS, chúng ta còn phải quyết định mức độ quản lý node, dựa trên tính năng mà workload cần và năng lực vận hành của đội ngũ.

Về mặt kỹ thuật, cả năm service của TravelNow đều có thể chạy trên ECS. Chương này chọn EKS để hiện thực và quan sát các cơ chế Kubernetes đã phân tích ở Chương 10. Với một hệ thống thực tế, quyết định sử dụng Kubernetes cần được ghi lại cùng lý do cụ thể. Số lượng service hay mức

tải cao, xét riêng, chưa đủ để lựa chọn giữa ECS và EKS. AWS — Amazon ECS.

> **Phương án triển khai xuyên suốt Chương 11–12**
>
> Cả năm service của TravelNow được triển khai dưới dạng container trên EKS, trong khi RDS, S3, Step Functions và SNS/SQS tiếp tục hoạt động bên ngoài cluster. Ở Chương 11, cluster sử dụng một managed node group gồm ba node làm **capacity ban đầu**. Sang Chương 12, nhóm node này được giữ lại để chạy các thành phần nền tảng và controller, còn **Karpenter** sẽ đảm nhận việc cấp phát thêm node cho workload ứng dụng khi cần. Cách triển khai này giúp chúng ta quan sát rõ cơ chế quản lý capacity; **EKS Auto Mode** sẽ được xem như một phương án trong đó AWS đảm nhận nhiều hơn trách nhiệm cấp phát và quản lý vòng đời node.

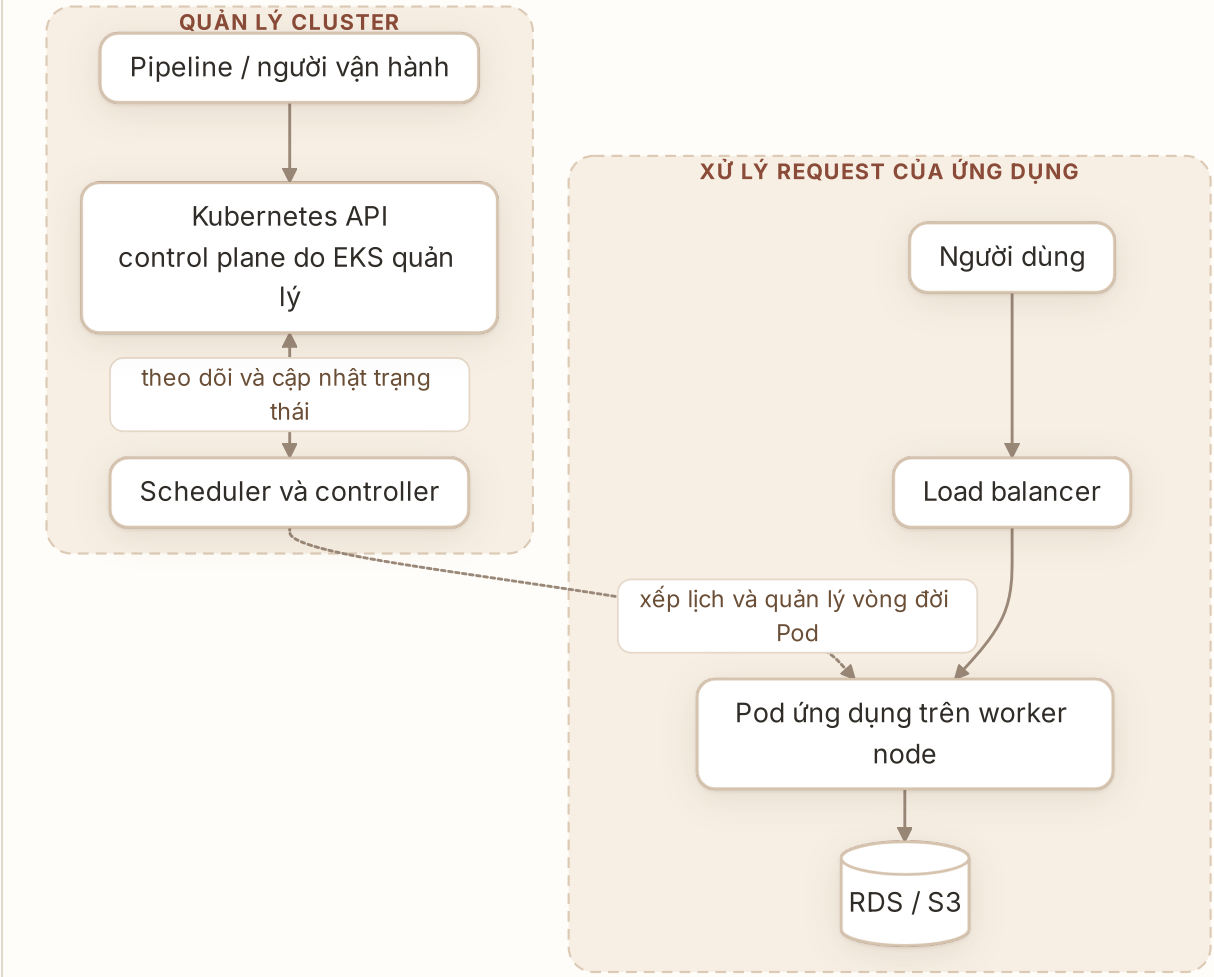
11.2 Managed control plane: phạm vi quản lý của AWS và trách nhiệm của chúng ta

11.2.1 Phân biệt hoạt động quản lý cluster với luồng xử lý request

Với EKS, AWS vận hành control plane của Kubernetes, bao gồm API server và `etcd`, trên nhiều Availability Zone. Chúng ta không còn phải tự triển khai, sao lưu hay thay thế các máy chủ chạy những thành phần này. Tuy nhiên, để hiểu phạm vi của dịch vụ, cần phân biệt **hoạt động quản lý cluster** với **luồng xử lý request của ứng dụng**:

- **Quản lý cluster:** pipeline hoặc người vận hành gửi cấu hình qua Kubernetes API. Các controller theo dõi và điều chỉnh trạng thái resource, còn scheduler chọn node phù hợp cho những Pod chưa được xếp lịch.
- **Xử lý request:** yêu cầu của người dùng đi qua load balancer tới Pod ứng dụng. Ứng dụng thực hiện nghiệp vụ và truy cập RDS hoặc S3 khi cần. Kubernetes API không trực tiếp tham gia vào luồng xử lý này.

HÌNH 11.2 · SƠ ĐỒ LUỒNG



Sự phân biệt này giúp chúng ta đánh giá đúng ảnh hưởng khi control plane gián đoạn. Như đã phân tích ở Chương 10, việc tiếp nhận cấu hình mới, xếp lịch Pod mới và tạo Pod thay thế đều phụ thuộc vào Kubernetes API. Khi API không khả dụng, pipeline không thể triển khai thay đổi vào cluster và các hoạt động điều phối bị gián đoạn. Tuy nhiên, những Pod đang chạy vẫn có thể tiếp tục xử lý request qua cấu hình định tuyến hiện có, miễn là load balancer, mạng và các dịch vụ mà ứng dụng sử dụng vẫn hoạt động. Nếu một Pod hoặc node gặp sự cố trong thời gian này, khả năng tạo và đưa Pod thay thế vào phục vụ sẽ bị ảnh hưởng.

Ngược lại, Kubernetes API hoạt động bình thường cũng chưa cho biết ứng dụng có đang phục vụ đúng hay không. Chẳng hạn, cluster vẫn có thể tiếp nhận cấu hình và duy trì đủ Pod, trong khi `payment-service` không kết nối được tới cổng thanh toán.

Vì vậy, **độ sẵn sàng của control plane và độ sẵn sàng của ứng dụng cần được đo và kiểm chứng riêng**. EKS quản lý hạ tầng control plane trong phạm vi dịch vụ của mình; khả năng hoàn tất một booking còn phụ thuộc vào các Pod ứng dụng, kết nối mạng, dữ liệu và những dịch vụ bên ngoài. AWS — Kiến trúc EKS.

11.2.2 Quản lý vòng đời phiên bản Kubernetes trên EKS

Phần việc thường bị đánh giá thấp nhất khi chọn EKS là **nâng cấp phiên bản Kubernetes**. Kubernetes phát hành khoảng ba phiên bản minor mỗi năm. Trên EKS, mỗi phiên bản có một giai đoạn hỗ trợ tiêu chuẩn, sau đó chuyển sang giai đoạn hỗ trợ mở rộng với phí control plane cao hơn. Lịch nâng cấp cluster vì thế cần là một hạng mục cố định trong kế hoạch vận hành của TravelNow, thay vì chỉ được xử lý khi phiên bản sắp hết thời hạn hỗ trợ.

Trong một lần nâng cấp, AWS thực hiện thay đổi trên control plane; phần việc còn lại thuộc về đội ngũ:

- Rà soát các Kubernetes API mà manifest đang sử dụng, đặc biệt là những API đã bị deprecated hoặc bị loại bỏ ở phiên bản mới.
- Đối chiếu và nâng cấp các add-on như Amazon VPC CNI, CoreDNS và `kube-proxy`; mỗi add-on có vòng đời và yêu cầu tương thích riêng.
- Nâng cấp các controller do đội ngũ tự cài đặt, chẳng hạn AWS Load Balancer Controller.
- Cập nhật hoặc thay thế node để phiên bản `kubelet` nằm trong phạm vi tương thích với control plane.

Chi phí vận hành cluster vì vậy không dừng lại ở công sức dựng môi trường ban đầu, mà bao gồm cả **chu kỳ nâng cấp và kiểm chứng lặp lại** trong suốt vòng đời hệ thống. AWS — Vòng đời phiên bản EKS.

11.3 Thay đổi môi trường thực thi, giữ nguyên ranh giới kiến trúc

11.3.1 Ánh xạ kiến trúc Phần III sang EKS

Trong phương án Saga orchestration của Phần III, Step Functions lưu tiến độ Saga; mỗi service sở hữu dữ liệu thuộc domain của mình; outbox ghi nhận công việc cần chuyển tiếp, còn idempotency giúp ngăn side effect trùng lặp khi một thao tác được thực hiện lại. Khi chuyển tầng ứng dụng sang EKS, cách phân chia trách nhiệm này vẫn được giữ nguyên. Điều thay đổi là môi trường thực thi và cách các thành phần giao tiếp với nhau. Vì vậy, chúng ta cần kiểm thử lại các tình huống lỗi trên EKS để xác nhận rằng hệ thống vẫn có thể tiếp tục công việc, xử lý kết quả chưa rõ và phục hồi đúng như thiết kế.

Như đã phân tích ở Chương 10, **container orchestration** và **Saga orchestration** phục vụ hai mục đích khác nhau. Kubernetes trên EKS quản lý vòng đời và duy trì các Pod theo trạng thái mong muốn, còn Step Functions lưu tiến độ và điều phối các bước trong quy trình đặt chuyến. Vì vậy, khi chuyển các service sang EKS, chúng ta vẫn sử dụng Step Functions để điều phối Saga.

Thành phần TravelNow	Cách triển khai trên EKS
trip, hotel, flight, car, payment	Mỗi service chạy bằng container, được Deployment quản lý. Các workload phục vụ HTTP API sử dụng thêm Service để cung cấp điểm truy cập ổn định; worker xử lý công việc nền. Tăng hoặc giảm số Pod không làm thay đổi trách nhiệm nghiệp vụ của từng service.
Dữ liệu của từng domain	Tiếp tục lưu trên RDS. Các Pod của cùng một service sử dụng chung dữ liệu thuộc domain đó, với quyền truy cập riêng biệt giữa các service.
Tài liệu PDF	Tiếp tục lưu trên S3. Dữ liệu lưu cục bộ trong Pod chỉ phục vụ nhu cầu tạm thời và có thể bị mất khi Pod được thay thế.
Saga orchestration	Step Functions tiếp tục lưu tiến độ và điều phối Saga. Thay vì gọi Lambda để thực hiện nghiệp vụ, workflow gọi qua Lambda adapter tới API của các service trong cluster, như mô tả ở Mục 11.5.4.
Outbox dispatcher/publisher	Chạy dưới dạng worker, đọc outbox rồi khởi chạy workflow hoặc phát sự kiện tương ứng. Các worker vẫn cần phối hợp quyền xử lý, retry có kiểm soát và giữ nguyên định danh thao tác để bên nhận ngăn xử lý trùng.
Consumer SQS (phương án choreography)	Chạy dưới dạng worker, chủ động nhận message từ SQS, xử lý nghiệp vụ và chỉ xóa message sau khi commit thành công. Worker quản lý việc nhận và xác nhận message thay cho Lambda event source mapping.

Hàng cuối mô tả cách triển khai consumer nếu TravelNow chọn **choreography**, giúp đối chiếu với phương án **orchestration** đang sử dụng trong chương. Đây là hai phương án thay thế cho cùng quy trình đặt chuyến; mỗi yêu cầu booking được xử lý theo một phương án đã chọn.

Vì sao vẫn giữ Step Functions khi đã có Kubernetes?

Kubernetes duy trì số lượng Pod của `payment-service` theo cấu hình, nhưng không theo dõi chuyến đi nào đã giữ phòng và đang chờ thanh toán. Khi một Pod bị mất, Kubernetes có thể tạo Pod thay thế; Pod mới vẫn cần thông tin đã lưu để tiếp tục công việc còn dang dở. Trong phương án orchestration của TravelNow, Step Functions lưu tiến độ Saga, quyết định bước tiếp theo và điều phối các thao tác bù trừ khi cần.

Phần IV tiếp tục sử dụng Step Functions để tập trung phân tích những thay đổi khi chuyển ứng dụng từ Lambda sang container trên EKS. Việc giữ nguyên cơ chế điều phối Saga cũng giúp chúng ta đối chiếu kết quả kiểm thử ở Mục 11.6 với các tình huống đã xét trong Phần III.

Trong thực tế, chúng ta có thể lựa chọn phương án khác theo yêu cầu của hệ thống: phối hợp bằng sự kiện với SNS/SQS như Mục 8.4, sử dụng workflow engine như Temporal hoặc Camunda, hoặc tự xây orchestrator khi quy trình đủ đơn giản. Phương án tự xây cần cả nơi lưu trạng thái Saga lẫn cơ chế tiếp tục công việc sau sự cố, cùng những chi phí đã phân tích ở Mục 8.3.3. Dù chọn phương án nào, từng service vẫn phải bảo đảm idempotency, duy trì định danh thao tác và hỗ trợ tra cứu hoặc đối soát khi kết quả chưa rõ.

11.3.2 Phân chia trách nhiệm quản lý cấu hình

Terraform tiếp tục quản lý các tài nguyên hạ tầng như VPC, subnet, cluster, node group và IAM theo cách đã trình bày ở Chương 5. Manifest Kubernetes mô tả cấu hình workload, còn pipeline hoặc GitOps controller áp dụng cấu hình đó vào cluster. Khi sử dụng nhiều công cụ, chúng ta cần xác định rõ **mỗi công cụ được phép quản lý và thay đổi những thuộc tính nào**, tránh để chúng ghi đè kết quả của nhau. Hai ví dụ thường gặp là:

- **HPA và pipeline cùng thay đổi số replica.** HPA tăng hoặc giảm số Pod theo tải, nhưng manifest trong pipeline vẫn khai báo một giá trị `replicas` cố định. Lần triển khai tiếp theo có thể đưa số replica về giá trị trong manifest, làm mất hiệu lực điều chỉnh trước đó của HPA.
- **Terraform và AWS Load Balancer Controller cùng thay đổi cấu hình load balancer.** Nếu cả hai cùng quản lý những thuộc tính như listener hoặc target group, mỗi công cụ có thể áp dụng lại cấu hình của mình và ghi đè thay đổi do công cụ còn lại tạo ra.

Để tránh xung đột, **mỗi thuộc tính nên do một công cụ quản lý**. Chẳng hạn, khi giao số replica cho HPA, cấu hình triển khai cần được điều chỉnh để pipeline không tiếp tục áp đặt giá trị `replicas` cố định. Tương tự, những thành phần của load balancer do AWS Load Balancer Controller tạo và cập nhật không nên đồng thời được Terraform quản lý.

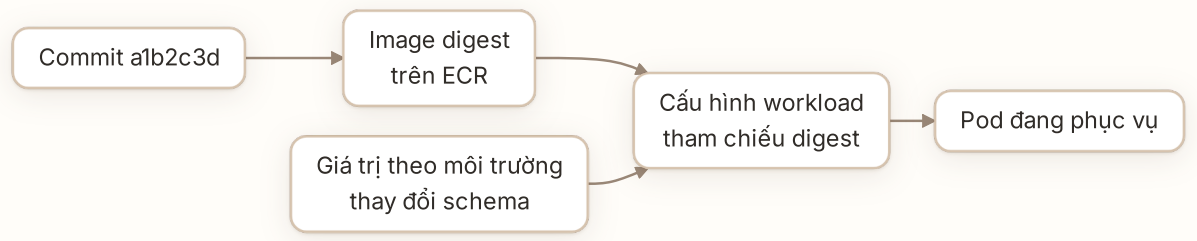
Bên cạnh việc phân chia phạm vi giữa các công cụ, chúng ta cũng cần quy định cách thay đổi cấu hình khi xử lý sự cố. Giả sử người trực sửa trực tiếp timeout của `flight-service` trên cluster, nhưng cấu hình trong Git vẫn giữ giá trị cũ. Nếu GitOps controller được thiết lập để tự động khôi phục cấu hình theo Git, thay đổi vừa thực hiện có thể bị ghi đè. Với Argo CD, hành vi này liên quan đến tùy chọn `selfHeal` trong chính sách đồng bộ tự động. Vì vậy, người xử lý sự cố cần biết controller sẽ phản ứng thế nào trước một thay đổi trực tiếp trên cluster. Argo CD — Automated sync.

Quy trình vận hành cần xác định ai được phép sửa trực tiếp, cách kiểm soát việc đồng bộ trong thời gian xử lý và ai chịu trách nhiệm cập nhật lại cấu hình trong Git. Sau khi xác nhận thay đổi có hiệu quả, chúng ta cần ghi nhận nó vào Git; nếu thay đổi chỉ mang tính tạm thời hoặc không còn cần thiết, phải hoàn tác có kiểm soát. Trước khi kết thúc xử lý sự cố, cần đối chiếu cấu hình đang chạy với cấu hình trong Git và khôi phục chính sách đồng bộ nếu đã tạm điều chỉnh. Bỏ sót bước này có thể khiến lần triển khai tiếp theo đưa cấu hình gây lỗi trở lại.

11.3.3 Truy vết từ commit đến workload đang phục vụ

Image tiếp tục được lưu trên ECR. Khi chạy trên EKS, chuỗi truy vết phải nối được từ commit mã nguồn tới đúng phiên bản đang phục vụ người dùng:

HÌNH 11.3 · SƠ ĐỒ LUỒNG



Manifest hoặc Helm chart đóng gói cấu hình của workload, nhưng chưa đủ để tái tạo một bản phát hành. Chúng ta còn cần image digest, các giá trị cấu hình theo môi trường, thay đổi schema database và những bước chuyển đổi dữ liệu đi kèm. Tham chiếu image bằng digest bảo đảm mọi Pod chạy đúng nội dung image đã được kiểm thử. Bộ thông tin này liên kết mã nguồn với workload đang phục vụ, giúp team xác định chính xác phiên bản cần điều tra khi xảy ra sự cố ở Chương 14.

11.3.4 Kiểm tra cấu hình triển khai theo từng môi trường

Giả sử Helm chart dùng chung đã được kiểm thử, nhưng bộ giá trị cấu hình (*values*) của môi trường tích hợp lại khai báo sai `targetPort` hoặc ARN của queue. Helm vẫn có thể sinh manifest hợp lệ và container vẫn khởi động bình thường, trong khi một bước của quy trình booking không thể thực hiện. Vì vậy, chúng ta cần kiểm tra **manifest được tạo từ đúng phiên bản chart và bộ values của môi trường đích**, thay vì chỉ kiểm tra chart dùng chung.

Mỗi bước kiểm tra giúp phát hiện một nhóm vấn đề và có phạm vi xác nhận riêng:

Bước kiểm tra	Nội dung có thể kiểm tra hoặc xác nhận	Giới hạn
Lint chart và render với values của môi trường đích	Phát hiện một số lỗi cấu trúc, cú pháp và template; xem manifest được sinh ra từ cấu hình đã chọn.	Chưa xác nhận các tài nguyên được tham chiếu tồn tại hoặc ứng dụng có thể kết nối tới chúng.
Kiểm tra schema, policy và quan hệ giữa các manifest	Phát hiện trường sai kiểu, thiếu cấu hình bắt buộc, selector hoặc cổng không khớp, trong phạm vi các quy tắc đã thiết lập.	Chưa xác nhận ứng dụng thực sự lắng nghe ở cổng đã khai báo hoặc sử dụng đúng thông tin xác thực.
Thực hiện server-side dry-run qua Kubernetes API	Xác nhận yêu cầu vượt qua các bước validation và admission của cluster mà không lưu thay đổi.	Không khởi chạy Pod, nên chưa kiểm tra được kết nối tới RDS hay khả năng xử lý booking.
Triển khai vào môi trường kiểm thử và chạy các kịch bản tích hợp	Kiểm tra quyền truy cập, kết nối mạng và contract với các thành phần thực tế của môi trường.	Kết quả chỉ có giá trị với những kịch bản và mức tải đã thử.

`helm template` cho phép xem manifest được sinh ra từ chart và values. Server-side dry-run bổ sung việc kiểm tra manifest qua Kubernetes API của cluster đích. Kết hợp hai bước giúp phát hiện lỗi trước khi triển khai, nhưng vẫn cần chạy ứng dụng để kiểm chứng hành vi thực tế. Helm — Debugging templates, Kubernetes — Dry-run.

Như đã trình bày ở Chương 4, một bước kiểm tra chỉ có thể ngăn bản phát hành lỗi nếu nó **thực sự được chạy và kết quả thất bại khiến pipeline dừng lại**. Báo cáo cần phân biệt rõ ba trạng thái: **đạt**, **thất bại** và **chưa chạy**. Một job bị bỏ qua không thể được tính là kiểm thử thành công.

Nếu vẫn cho phép phát hành khi một kiểm tra chưa đạt hoặc chưa được thực hiện, cần ghi rõ lý do, người phê duyệt, phạm vi ảnh hưởng và người chịu trách nhiệm xử lý phần còn thiếu, kèm thời hạn cụ thể. Nhờ đó, nhóm tiếp nhận bản phát hành biết những gì đã được kiểm chứng và những gì còn cần kiểm tra trước khi tích hợp hoặc đưa vào sử dụng.

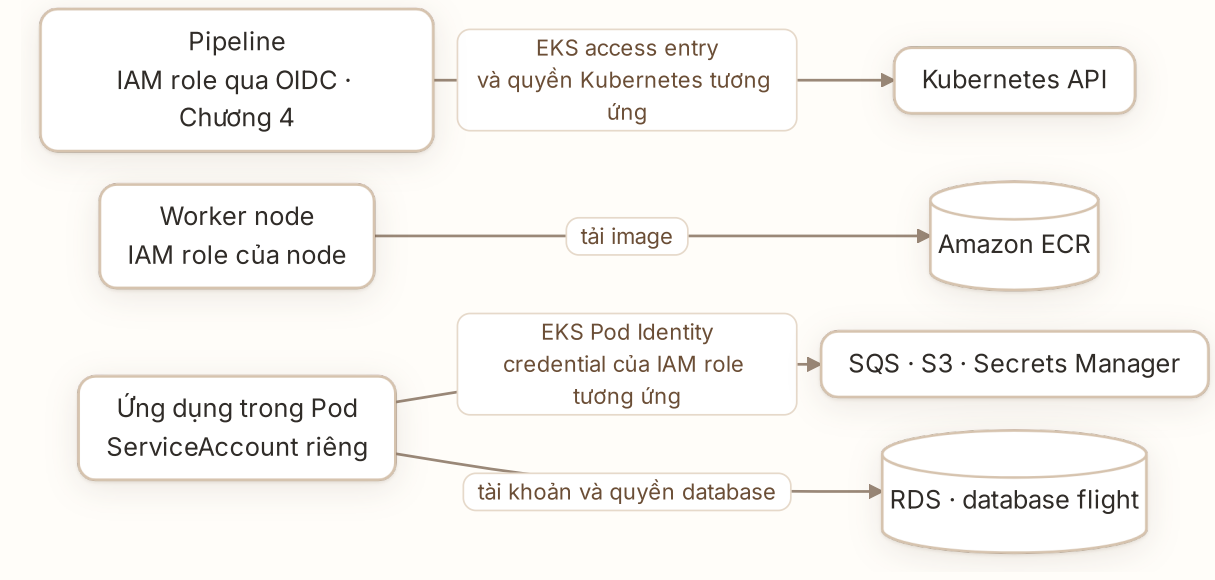
11.4 Danh tính và phân quyền cho workload trên EKS

11.4.1 Phân biệt quyền của pipeline, node và ứng dụng

Giả sử `flight-service` không khởi động được sau một lần phát hành. Để xác định nguyên nhân, chúng ta cần biết lỗi xảy ra ở bước nào: pipeline không được phép cập nhật Deployment, node không tải được image từ ECR, hay ứng dụng không đọc được secret cần thiết để khởi động? Mỗi thao tác sử dụng một danh tính và phạm vi quyền khác nhau.

Trong phương án EKS sử dụng managed node group của TravelNow, cần phân biệt ba thành phần:

HÌNH 11.4 · SƠ ĐỒ LUỒNG



Thành phần	Quyền cần có	Biểu hiện khi thiếu quyền hoặc không kết nối được
Pipeline	Truy cập Kubernetes API và cập nhật các resource trong phạm vi được cấp.	Không áp dụng được cấu hình mới vào cluster.
Worker node	Tải image từ ECR để khởi chạy container.	Container chưa thể khởi động vì lỗi tải image.

Thành phần	Quyền cần có	Biểu hiện khi thiếu quyền hoặc không kết nối được
Ứng dụng trong Pod	Truy cập những queue, bucket, secret và dữ liệu cần thiết cho nghiệp vụ.	Ứng dụng có thể lỗi ngay khi khởi động hoặc khi thực hiện một thao tác cụ thể.

Sự phân biệt này giúp chúng ta chọn đúng nơi cần kiểm tra. Với lỗi `ImagePullBackOff`, cần xem địa chỉ image, quyền truy cập ECR và kết nối mạng của node. Với lỗi `AccessDenied` trong log ứng dụng, cần xác định ứng dụng đang sử dụng danh tính nào và danh tính đó có được phép thực hiện thao tác trên tài nguyên đích hay không. **Cấp thêm quyền cho pipeline không giải quyết được việc ứng dụng thiếu quyền ghi vào S3**, vì hai thành phần sử dụng những danh tính khác nhau.

11.4.2 Cấp quyền AWS riêng cho từng workload

Giả sử `trip-service` bị từ chối khi ghi PDF vào S3. Nếu bổ sung quyền S3 vào IAM role của node, quyền đó sẽ gắn với máy chủ chạy ứng dụng. Những Pod khác truy cập được **Instance Metadata Service (IMDS)** có thể lấy credential của node và sử dụng cùng quyền này, dù chúng không cần lưu PDF. Nếu ứng dụng phụ thuộc vào credential của node, việc chuyển Pod sang node có role khác cũng có thể làm thay đổi quyền truy cập của ứng dụng.

Vì vậy, chúng ta cần **cấp quyền theo từng workload**, đồng thời hạn chế khả năng Pod truy cập credential của node qua IMDS. Khi đó, quyền ghi PDF thuộc về `trip-service` và không phụ thuộc vào node mà Pod được xếp lịch.

Với **EKS Pod Identity**, chúng ta liên kết một ServiceAccount trong một namespace của cluster với một IAM role. Pod sử dụng ServiceAccount đó có thể nhận credential tạm thời của role thông qua Pod Identity Agent chạy trên node. AWS SDK trong ứng dụng sử dụng credential này khi được cấu hình phù hợp và hỗ trợ cơ chế Pod Identity.

Một lựa chọn khác là **IAM Roles for Service Accounts (IRSA)**, sử dụng OIDC provider của cluster để liên kết ServiceAccount với IAM role. Việc lựa chọn giữa Pod Identity và IRSA cần phù hợp với môi trường compute và khả năng hỗ trợ của nền tảng.

Dù sử dụng cơ chế nào, nguyên tắc vẫn là **mỗi workload có danh tính phù hợp và chỉ được cấp những quyền cần cho công việc của mình**. Chẳng hạn, `trip-service` có thể được ghi vào vùng lưu PDF trên S3, còn một consumer chỉ được nhận và xóa message trong queue mà nó xử lý. AWS — EKS Pod Identity.

11.4.3 Phân biệt quyền Kubernetes, quyền AWS và quyền database

Một ứng dụng chạy trên EKS có thể tương tác với Kubernetes API, các dịch vụ AWS và database. Mỗi hệ thống kiểm soát quyền trong phạm vi riêng:

- **Kubernetes RBAC** quy định những thao tác được phép thực hiện trên resource của cluster, chẳng hạn cập nhật Deployment trong namespace của TravelNow. Trong phương án sử dụng RBAC, EKS access entry liên kết IAM role của pipeline với các nhóm Kubernetes được

cấp quyền tương ứng.

- **IAM** kiểm soát quyền gọi API AWS, chẳng hạn đọc secret từ Secrets Manager, ghi object vào S3 hoặc nhận message từ SQS.
- **Cơ chế xác thực và phân quyền của PostgreSQL** kiểm soát việc kết nối tới database và thao tác trên dữ liệu. Mỗi service sử dụng tài khoản với các quyền được cấp trong phạm vi domain của mình.

Quyền ở một hệ thống không tự động mở rộng sang hệ thống khác. Ví dụ, IAM cho phép `flight-service` đọc secret chứa thông tin kết nối database; sau đó, PostgreSQL vẫn kiểm tra tài khoản trong secret có được phép kết nối và ghi vào bảng cần thiết hay không. Tương tự, quyền cập nhật Deployment không đồng nghĩa với quyền truy cập dữ liệu nghiệp vụ của ứng dụng.

Ngoài quyền truy cập, các thành phần còn cần kết nối mạng phù hợp. Node trong private subnet phải có đường kết nối tới ECR để tải image; ứng dụng phải kết nối được tới các API AWS và RDS mà nó sử dụng. Tùy đích đến, kết nối có thể đi qua mạng nội bộ VPC, VPC endpoint hoặc NAT gateway. Khi chẩn đoán sự cố, chúng ta cần kiểm tra riêng **khả năng kết nối** và **quyền thực hiện thao tác**, vì cấu hình đúng một phần chưa đủ để lời gọi thành công.

11.5 Networking: kết nối TravelNow với người dùng và dịch vụ AWS

11.5.1 Tiếp nhận request qua ALB

Ở Chương 10, chúng ta đã phân biệt **Ingress**, resource khai báo các quy tắc định tuyến, với thành phần thực hiện việc định tuyến. Trên EKS, **AWS Load Balancer Controller** theo dõi cấu hình Ingress và gọi API AWS để tạo hoặc cập nhật ALB, listener và target group tương ứng. Controller cần quyền IAM và kết nối mạng để thực hiện các thao tác này. Khi người dùng truy cập ứng dụng, **ALB trực tiếp tiếp nhận và chuyển tiếp request tới các target; request không đi qua process của controller**.

Sự phân chia này giúp chúng ta hiểu ảnh hưởng khi controller gặp sự cố. ALB hiện có vẫn có thể tiếp tục phục vụ theo cấu hình đã được thiết lập, nhưng những thay đổi mới trong cluster có thể chưa được cập nhật lên ALB. Chẳng hạn, với target group sử dụng IP mode, controller cần đăng ký địa chỉ IP của Pod mới. Nếu controller ngừng hoạt động trong lúc rollout, một Pod có thể đã khởi động và vượt qua readiness probe nhưng vẫn chưa được ALB chuyển traffic tới. Đây là ví dụ cụ thể cho sự khác biệt giữa **hoạt động quản lý hạ tầng và khả năng xử lý request** đã trình bày ở Mục 11.2. AWS Load Balancer Controller — Cơ chế hoạt động.

Trong phương án của TravelNow, các node ứng dụng nằm trong private subnet. ALB công khai nhận request từ Internet và chuyển trực tiếp tới Pod của `trip-service` thông qua target group sử dụng **IP mode**. Để hiểu cách kết nối này, cần phân biệt hai chế độ đăng ký target:

Chế độ	Target được đăng ký	Cách traffic đến ứng dụng
IP mode	Địa chỉ IP của Pod	ALB chuyển request trực tiếp tới Pod.
Instance mode	Node trong cluster	ALB chuyển request tới NodePort trên node, sau đó traffic được chuyển tiếp tới Pod phù hợp, có thể nằm trên node khác.

Chế độ được chọn quyết định những kết nối và Security Group cần kiểm tra. Vì vậy, khi ứng dụng không nhận được traffic, chúng ta cần xác định ALB đang gửi request tới IP của Pod hay tới NodePort trên node trước khi phân tích lỗi.

11.5.2 Cấp địa chỉ IP cho Pod và quy hoạch subnet

Trong cấu hình sử dụng **Amazon VPC CNI** của TravelNow, mỗi Pod nhận một địa chỉ IP thuộc subnet của VPC. Cách cấp địa chỉ này cho phép ALB sử dụng IP của Pod làm target và chuyển traffic trực tiếp tới ứng dụng, với điều kiện các cấu hình mạng cho phép kết nối.

Tuy nhiên, địa chỉ IP cũng là một loại tài nguyên cần tính toán. **Subnet có thể hết địa chỉ IP dành cho Pod trong khi node vẫn còn CPU và bộ nhớ.** Khi đó, việc tăng số replica hoặc bổ sung node chưa chắc giúp ứng dụng có thêm capacity để phục vụ.

Prefix delegation giúp tăng khả năng cấp IP trên mỗi node bằng cách gán các dải địa chỉ cho network interface, thay vì gán từng địa chỉ riêng lẻ. Các dải này vẫn được lấy từ subnet của VPC và cần một vùng địa chỉ liên tục còn trống. Nếu không gian địa chỉ bị phân mảnh, việc cấp prefix có thể thất bại dù tổng số IP chưa sử dụng vẫn còn nhiều.

Vì vậy, khi quy hoạch CIDR như ở Chương 5, chúng ta cần tính đến:

- Số Pod dự kiến chạy trong mỗi subnet.
- Nhu cầu tăng số Pod và node khi tải cao.
- Số Pod tồn tại thêm trong quá trình rollout, khi phiên bản mới khởi động trước khi phiên bản cũ dừng.
- Phần địa chỉ dự phòng để phục hồi workload khi có sự cố.

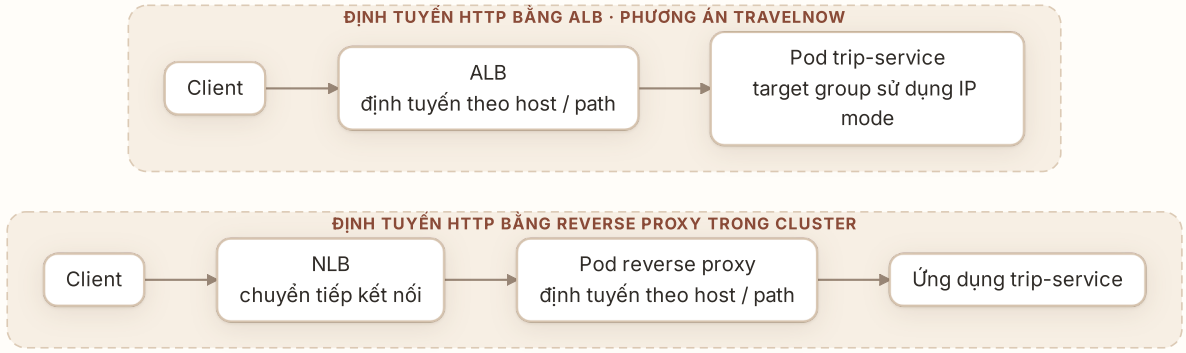
Có địa chỉ IP trong VPC mới chỉ đáp ứng một phần điều kiện kết nối. Để Pod truy cập được RDS, route table, Security Group và các cơ chế kiểm soát mạng liên quan vẫn phải cho phép traffic giữa hai bên. Sau khi kết nối được thiết lập, database tiếp tục kiểm tra danh tính và quyền truy cập của ứng dụng. AWS — Prefix mode, AWS — VPC CNI.

11.5.3 Lựa chọn định tuyến bằng ALB hoặc reverse proxy

Bài viết *Về Ingress Controller trên EKS Cluster* phân tích hai cách đưa request vào ứng dụng: thực hiện định tuyến HTTP trên load balancer bên ngoài cluster, hoặc chuyển kết nối tới một **reverse proxy** chạy trong cluster để proxy thực hiện việc định tuyến.

Với phương án thứ nhất, ALB đọc thông tin như hostname và path để chọn target phù hợp. Với phương án thứ hai, NLB chuyển tiếp kết nối tới các Pod reverse proxy; proxy đọc request HTTP và chuyển tiếp tới ứng dụng theo cấu hình.

HÌNH 11.5 · SƠ ĐỒ LUỒNG



Khác biệt quan trọng nằm ở **thành phần thực hiện định tuyến HTTP và phạm vi công việc chúng ta phải vận hành**:

Tiêu chí	ALB và AWS Load Balancer Controller	NLB và reverse proxy trong cluster
Nơi định tuyến HTTP	ALB, do AWS vận hành.	Các Pod reverse proxy trong cluster.
Công việc cần quản lý	Cấu hình Ingress, quyền của controller và việc tích hợp ALB với workload.	Cấu hình định tuyến, số replica, tài nguyên, cập nhật và khả năng phục hồi của proxy, cùng việc tích hợp với NLB.
Khả năng tùy chỉnh	Theo những tính năng ALB cung cấp.	Theo phần mềm proxy được lựa chọn.
Sử dụng trên nhiều môi trường	Cấu hình tích hợp gắn với AWS.	Có thể sử dụng cùng phần mềm proxy ở nhiều môi trường, nhưng vẫn cần điều chỉnh cách tích hợp với hạ tầng.

Trong phương án thứ hai, một Service kiểu `LoadBalancer` có thể được cấu hình để tạo NLB phía trước proxy. Các quy tắc định tuyến HTTP được khai báo riêng, chẳng hạn qua Ingress hoặc Gateway API, rồi được controller tương ứng áp dụng vào proxy. **Tạo load balancer và thiết lập quy tắc định tuyến HTTP là hai phần cấu hình cần phân biệt.** Kubernetes — Ingress controllers.

TravelNow tiếp tục sử dụng ALB vì các yêu cầu định tuyến hiện tại nằm trong phạm vi tính năng của dịch vụ. Nếu ứng dụng cần khả năng tùy chỉnh riêng của một reverse proxy hoặc cần thống nhất phần mềm định tuyến giữa nhiều môi trường, chúng ta có thể đánh giá phương án chạy proxy trong cluster. Lựa chọn này cần tính cả công sức cập nhật, giám sát và phục hồi proxy; số lượng service chưa đủ để quyết định phương án nào phù hợp hơn.

Với cả hai phương án, chúng ta cần xác định rõ **kết nối TLS kết thúc tại đâu**, ứng dụng lấy địa chỉ IP gốc của client bằng cách nào và health check kiểm tra tới thành phần nào. Những cấu hình này ảnh hưởng trực tiếp đến bảo mật, khả năng ghi log và việc xác định ứng dụng có thực sự sẵn sàng nhận request hay không.

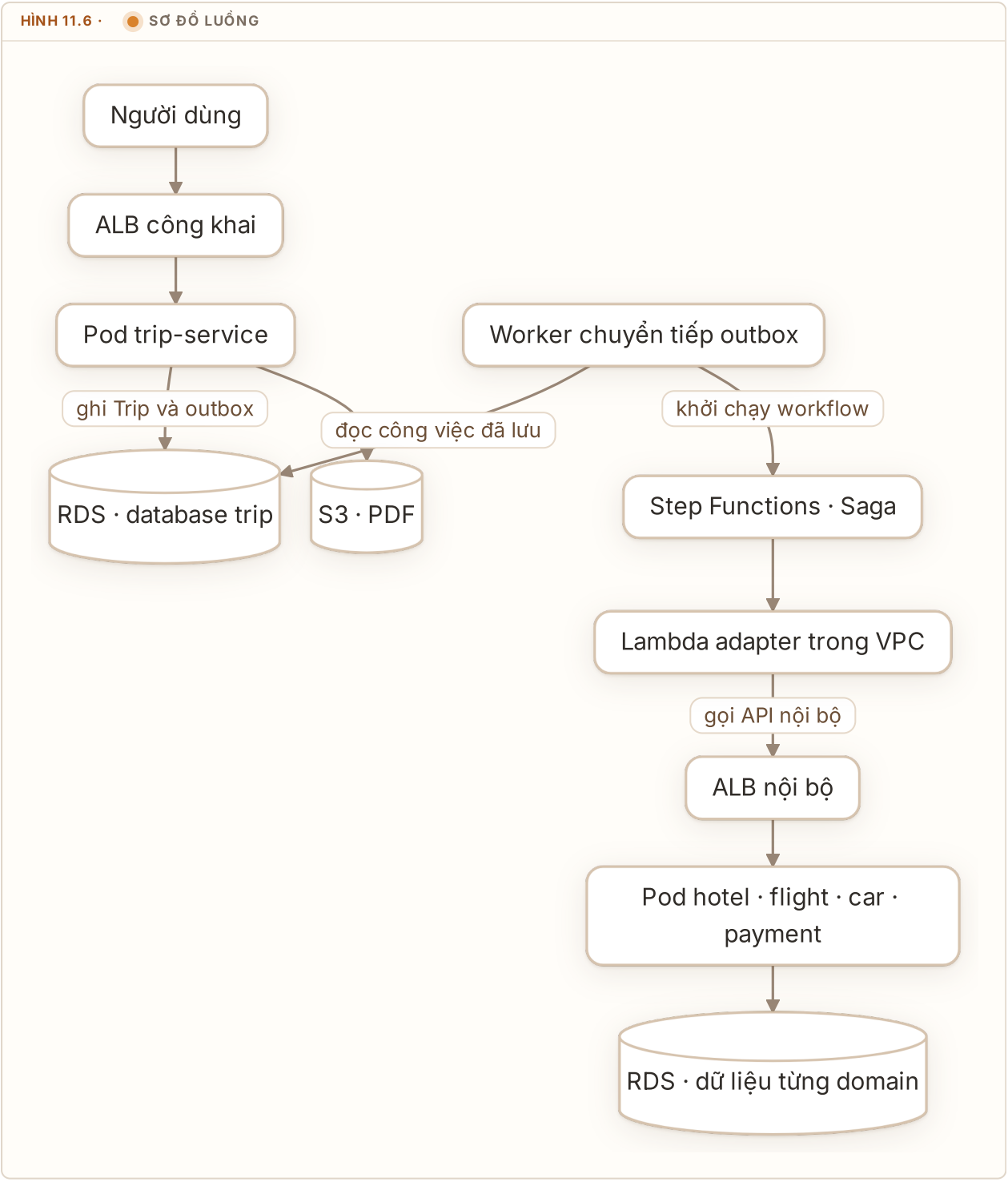
> **Mô hình kiến trúc và vòng đời phần mềm**
>
> Bài viết năm 2025 sử dụng ingress-nginx để minh họa mô hình reverse proxy trong cluster. Dự án **ingress-nginx** có lịch kết thúc bảo trì vào tháng 3/2026. Điều này liên quan đến vòng đời của một phần mềm controller, không có nghĩa Kubernetes loại bỏ resource Ingress. Khi triển khai mới, chúng ta cần lựa chọn controller còn được bảo trì và phù hợp với Ingress hoặc Gateway API đang sử dụng. Kubernetes — Thông báo về ingress-nginx.

11.5.4 Kết nối Step Functions với các service trong cluster

Ở Chương 9, Step Functions gọi Lambda để thực hiện từng bước nghiệp vụ. Khi chuyển sang EKS, logic đó chạy trong các Pod, còn Step Functions vẫn hoạt động bên ngoài cluster. Vì vậy, chúng ta cần thiết lập cách để workflow gọi được API của từng service.

Phương án minh họa trong chương sử dụng **Lambda adapter trong VPC**, kết nối tới các service thông qua **ALB nội bộ**. Step Functions gọi adapter với dữ liệu của bước cần thực hiện; adapter chuyển dữ liệu thành request HTTP, gọi API tương ứng rồi trả kết quả về workflow. Logic nghiệp vụ vẫn nằm trong service, còn Step Functions tiếp tục lưu tiến độ và điều phối Saga.

HÌNH 11.6 · SƠ ĐỒ LUỒNG



Lambda adapter tạo thêm một tầng thực thi giữa Step Functions và các service trên EKS. Khi số lời gọi tăng hoặc service phản hồi chậm, số lượt thực thi đồng thời của adapter cũng có thể tăng vì adapter vẫn chờ kết quả. **Lambda và các Pod co giãn độc lập**; adapter được cấp thêm môi trường thực thi không có nghĩa các service phía sau đã đủ tài nguyên để tiếp nhận tải. Vì vậy, chúng ta cần phối hợp giới hạn concurrency, timeout và retry của adapter với khả năng xử lý của các service. Phương án này giúp giữ cơ chế điều phối Saga đã xây dựng, nhưng làm tăng chi phí thực thi và số thành phần cần triển khai, theo dõi, xử lý sự cố.

Để kết nối này hoạt động, chúng ta phải cấu hình route, Security Group, TLS và cơ chế xác thực cho API nội bộ. Lambda nằm trong VPC không mặc nhiên phân giải được các tên như `flight-service.namespace.svc.cluster.local`, vì những tên này thuộc hệ thống DNS nội bộ của Kubernetes. Trong phương án trên, adapter sử dụng endpoint của ALB nội bộ để truy cập các service.

Quyền IAM cho phép Step Functions gọi Lambda cũng không tự cấp quyền gọi API nghiệp vụ cho adapter. Service nhận request vẫn cần xác thực adapter và kiểm tra adapter có được phép thực hiện thao tác tương ứng hay không, theo các nguyên tắc ở Mục 11.4.

Một phương án khác là sử dụng **HTTP Task** của Step Functions cùng EventBridge connection được thiết lập kết nối private phù hợp. Trước khi lựa chọn, cần đối chiếu các yêu cầu về endpoint, chứng chỉ, xác thực và giới hạn thời gian chờ với API mà workflow sẽ gọi. AWS — HTTP Task và private connectivity.

Dù sử dụng adapter hay HTTP Task, `tripId` và định danh thao tác phải được truyền đầy đủ tới service nhận để hỗ trợ idempotency và đối soát. Nếu adapter bị timeout sau khi gửi yêu cầu thanh toán, workflow vẫn chưa biết tiền đã được thu hay chưa. Hệ thống phải tra cứu hoặc thử lại an toàn theo contract đã thiết kế ở Chương 7–8.

Như vậy, chuyển ứng dụng sang EKS vẫn giữ nguyên các yêu cầu về tính đúng đắn của Saga, đồng thời bổ sung những kết nối và thành phần trung gian cần cấu hình, giám sát và kiểm chứng khi xảy ra lỗi.

11.6 Kiểm chứng TravelNow sau khi triển khai

11.6.1 Chuẩn bị đầy đủ thông tin và điều kiện tích hợp

Giả sử nhóm phát triển `hotel-service` bàn giao một image đã vượt qua unit test. Khi triển khai lên EKS, nhóm tích hợp phát hiện môi trường chưa có tài khoản database, secret và kết nối tới RDS. Mã nguồn đã được kiểm thử trong phạm vi phát triển, nhưng service vẫn chưa đủ điều kiện tham gia quy trình booking.

Tình huống này cho thấy việc bàn giao cần bao gồm cả **những điều kiện để service hoạt động trong môi trường đích**. Khi nhiều nhóm hoặc nhà cung cấp cùng tham gia, mỗi bên cần biết phần việc nào đã hoàn tất, phần nào còn thiếu và ai chịu trách nhiệm chuẩn bị.

Với `hotel-service`, thông tin bàn giao có thể được tổ chức như sau:

Nội dung bàn giao	Thông tin cần cung cấp
Phiên bản và contract	Image digest, phiên bản API hoặc event, các phiên bản bên gọi còn được hỗ trợ.
Tài nguyên và cấu hình	Database, tham chiếu secret, biến môi trường; topic, subscription, queue và DLQ nếu sử dụng choreography.

Nội dung bàn giao	Thông tin cần cung cấp
Quyền truy cập và kết nối mạng	Danh tính workload, quyền trong database, nguồn kết nối, đích đến và cổng sử dụng; bên chịu trách nhiệm thiết lập và kiểm tra từng phần.
Dữ liệu và hướng dẫn kiểm thử	Dữ liệu mẫu có thể tạo lại, cách gửi yêu cầu, kết quả mong đợi và cách dọn dữ liệu sau khi thử.
Thay đổi và giới hạn của bản phát hành	Migration cần thực hiện, điều kiện rollback, những kiểm thử đã chạy hoặc chưa chạy và các lỗi đã biết.
Hỗ trợ sau bàn giao	Nhóm phụ trách service, đầu mối phối hợp tích hợp và quy trình tiếp nhận, xử lý lỗi.

Release note cần liên kết tới cấu hình và kết quả kiểm thử của đúng bản phát hành, đồng thời nêu rõ những hạng mục còn chờ hoàn thành. Chẳng hạn, thông tin "đã kiểm thử API với đối tác giả lập; chưa kiểm thử timeout trên môi trường sandbox của đối tác" giúp bên tiếp nhận hiểu chính xác phạm vi đã được xác nhận.

Hoàn thành mã nguồn, sẵn sàng tích hợp và kiểm chứng toàn bộ quy trình là ba mốc khác nhau. Thống nhất tiêu chí cho từng mốc giúp các nhóm đánh giá tiến độ và phối hợp dựa trên cùng một cách hiểu.

11.6.2 Kiểm tra từ trạng thái rollout đến kết quả booking

`kubectl rollout status` giúp theo dõi và xác nhận rollout theo các điều kiện của Deployment. Tuy nhiên, kết quả đó chưa chứng minh ứng dụng kết nối được với các dịch vụ liên quan hoặc xử lý đúng một yêu cầu đặt chuyến. Để đánh giá bản triển khai, chúng ta cần kiểm tra qua ba bước:

1. **Kiểm tra điều kiện vận hành.** Xác nhận image, cấu hình, quyền truy cập, kết nối mạng và dữ liệu đã được chuẩn bị đầy đủ. API, event và database schema phải tương thích với những phiên bản đang cùng hoạt động. Nếu kế hoạch phục hồi sử dụng rollback, cần kiểm chứng phiên bản trước vẫn đọc và xử lý được dữ liệu, event cùng cấu hình sau thay đổi.
2. **Kiểm tra khả năng tiếp nhận traffic.** Xác nhận Pod khởi động thành công, readiness phản ánh đúng khả năng phục vụ và các kết nối qua ALB công khai lẫn ALB nội bộ đều hoạt động. Cluster cũng cần đủ tài nguyên để chạy các Pod bổ sung trong quá trình rollout. Những kiểm tra này giúp phân biệt lỗi triển khai, lỗi kết nối và lỗi nghiệp vụ.
3. **Theo dõi booking đến khi hoàn tất.** Thực hiện một yêu cầu với dữ liệu thử và theo dõi từ lúc tiếp nhận, ghi outbox, khởi chạy Saga, giữ chỗ và thanh toán đến khi Trip có kết quả cuối cùng. Sau đó, kiểm tra việc tạo và truy cập PDF xác nhận nếu booking thành công. HTTP `202` chỉ xác nhận yêu cầu đã được tiếp nhận, nên chưa phải điểm kết thúc của bài kiểm thử.

11.6.3 Kiểm thử khả năng phục hồi khi có sự cố

Sau khi xác nhận quy trình thông thường hoạt động, chúng ta cần kiểm thử những tình huống một thành phần bị gián đoạn hoặc không nhận được kết quả từ thành phần khác. Đặc biệt, cần kiểm tra trường hợp **thao tác đã được thực hiện nhưng phản hồi hoặc xác nhận chưa tới nơi**

nhận.

Tình huống trong môi trường kiểm thử	Kết quả cần xác nhận
Thanh toán bị từ chối và xác nhận chưa thu tiền	Các bước giữ chỗ đã hoàn tất được bù trừ theo chính sách. Trip chỉ chuyển sang `FAILED` sau khi những thao tác bù trừ cần thiết đã thành công.
Lời gọi thanh toán timeout sau khi bên nhận đã xử lý	Hệ thống tra cứu hoặc thử lại an toàn bằng cùng định danh thao tác, xác định kết quả thực tế và không thu tiền trùng. Việc bù trừ không được quyết định chỉ dựa vào timeout.
Worker dừng sau khi commit nhưng trước khi xác nhận message	Trong phương án choreography, message được giao lại có thể được nhận diện là đã xử lý, không tạo side effect trùng lặp; công việc tiếp theo vẫn được tiếp tục từ trạng thái đã lưu.
Pod bị thay thế trong lúc rollout	Request và công việc đang thực hiện được hoàn tất hoặc phục hồi trong thời hạn đã đặt ra, kể cả khi Pod cũ không kịp kết thúc toàn bộ công việc.

Các phép thử này kiểm tra cách những cơ chế của hệ thống phối hợp với nhau. Kubernetes trên EKS duy trì workload và tạo Pod thay thế; Step Functions lưu tiến độ Saga; RDS lưu dữ liệu đã commit; còn SQS, trong phương án choreography, cho phép giao lại message khi chưa nhận được xác nhận hoàn tất. Việc giao lại vẫn có thể xảy ra sau khi consumer đã xử lý thành công, nên ứng dụng phải bảo đảm idempotency.

Tính đúng đắn của toàn bộ quy trình booking phụ thuộc vào sự phối hợp giữa các thành phần và logic nghiệp vụ của từng service. Để kiểm chứng điều đó, chúng ta cần đối chiếu trạng thái Trip, kết quả giữ chỗ, thanh toán và các công việc còn dang dở, thay vì chỉ nhìn trạng thái Pod hoặc kết quả rollout.

Log và các định danh như `tripId`, định danh thao tác và execution ID giúp liên kết những thông tin này để theo dõi một booking qua nhiều service. Chương 14 sẽ phát triển tiếp khả năng quan sát đó, phục vụ cả kiểm thử lẫn điều tra sự cố trong quá trình vận hành.

11.7 Nền tảng Kubernetes dùng chung: lợi ích và chi phí

11.7.1 Chi phí hạ tầng và công sức vận hành

Khi lựa chọn EKS, chúng ta cần tính cả **chi phí tài nguyên và công sức duy trì hệ thống**. Ngoài phí control plane, hạ tầng còn phát sinh chi phí cho node, load balancer, NAT Gateway, lưu trữ log và truyền dữ liệu. Trong quá trình sử dụng, chúng ta vẫn phải lên kế hoạch nâng cấp Kubernetes, duy trì các add-on và controller, theo dõi hoạt động của cluster và diễn tập phục hồi sự cố.

Những công việc này cần thời gian và người phụ trách trong suốt vòng đời hệ thống. Vì vậy, chi phí của một nền tảng Kubernetes không thể chỉ được đánh giá qua hóa đơn AWS hoặc công sức triển khai ban đầu.

EKS Auto Mode chuyển thêm một phần công việc quản lý hạ tầng sang AWS. Tuy nhiên, chúng ta vẫn phải thiết kế contract giữa các service, giới hạn quyền truy cập và xác định mục tiêu phục vụ của ứng dụng. Mỗi phương án cần được đánh giá theo phần việc AWS quản lý và phần việc chúng ta tiếp tục thực hiện.

11.7.2 Giới hạn ảnh hưởng của sự cố trên nền tảng dùng chung

Một nền tảng Kubernetes dùng chung cho phép nhiều nhóm sử dụng cùng các cơ chế triển khai, thu thập log, giám sát và cấp quyền. Nhờ đó, mỗi nhóm có thể tập trung hơn vào nghiệp vụ của service thay vì tự xây dựng lại toàn bộ môi trường vận hành.

Tuy nhiên, các workload cũng có thể cùng chịu ảnh hưởng từ một sự cố của nền tảng. Một lần nâng cấp gây lỗi, cấu hình controller không đúng hoặc tình trạng thiếu tài nguyên trong cluster có thể tác động đến nhiều service. Đây là bài toán **blast radius — phạm vi ảnh hưởng của sự cố** đã được đề cập ở Chương 6, lần này áp dụng cho hạ tầng dùng chung.

Chúng ta cần cân nhắc mức độ chia sẻ phù hợp với yêu cầu cách ly của hệ thống. Chẳng hạn, có thể tách cluster theo môi trường hoặc dành cluster riêng cho những workload có dữ liệu nhạy cảm và yêu cầu vận hành khác biệt. Cách tổ chức này giúp giới hạn một số sự cố trong phạm vi từng cluster, nhưng làm tăng số thành phần cần nâng cấp, giám sát và duy trì. Các cluster vẫn có thể phụ thuộc vào cùng database, mạng hoặc dịch vụ bên ngoài, nên việc tách cluster cần được xem xét cùng những phụ thuộc đó.

11.7.3 Xây dựng nền tảng như một sản phẩm nội bộ

Nền tảng dùng chung có thể cung cấp template tạo service, thư viện xử lý lỗi và truyền trace, Helm chart cùng pipeline triển khai. Với TravelNow, các nhóm thống nhất định dạng lỗi, cấu trúc log, cách cấp danh tính và quy trình phát hành. Những quy ước này giúp lập trình viên hiểu và vận hành các service khác nhau mà không phải học lại từ đầu. Các quy tắc nghiệp vụ như giữ phòng hay hoàn tiền vẫn do service thuộc domain tương ứng quản lý.

Bản thân những thành phần dùng chung cũng cần có **phiên bản, cam kết tương thích và hướng dẫn nâng cấp**. Giả sử một phiên bản thư viện logging đổi tên trường mà dashboard đang sử dụng. Trong thời gian các service nâng cấp khác nhau, hệ thống quan sát phải đọc được log từ những phiên bản còn được hỗ trợ. Nhóm phát triển nền tảng cần công bố thay đổi, cách kiểm thử và thời điểm ngừng hỗ trợ phiên bản cũ; các nhóm ứng dụng cần biết phiên bản mình đang dùng và cách chuyển sang phiên bản mới.

Vì vậy, chúng ta có thể xem nền tảng như **một sản phẩm phục vụ các nhóm phát triển trong tổ chức**. Giá trị của nó được thể hiện qua những kết quả cụ thể: mất bao lâu để đưa một service mới vào môi trường tích hợp, nhóm ứng dụng tự thực hiện được những thao tác nào và mỗi lần nâng cấp cần bao nhiêu công sức.

Nếu mọi thay đổi đều phải chờ nhóm nền tảng thực hiện, thời gian phối hợp có thể tăng dù công cụ đã được chuẩn hóa. Khả năng tự phục vụ, tài liệu rõ ràng và quy trình hỗ trợ vì thế cần được phát triển cùng các tính năng kỹ thuật. Góc nhìn này được trình bày trong *Platform Engineering*

on Kubernetes.

11.7.4 Khả năng chuyển ứng dụng sang môi trường khác

Khi chuyển TravelNow sang một nền tảng khác, chúng ta vẫn cần nơi lưu container image, cơ chế cấp danh tính, kết nối mạng, hệ thống lưu dữ liệu và công cụ điều phối container. Nếu môi trường đích cũng sử dụng Kubernetes, một phần cấu hình workload như Deployment có thể được tái sử dụng. Những cấu hình gắn với AWS, chẳng hạn Pod Identity, ALB hoặc storage, cần được thay thế hay điều chỉnh theo nền tảng mới.

Khả năng chuyển đổi phụ thuộc vào mức độ tách biệt giữa logic nghiệp vụ và các tích hợp hạ tầng. Ranh giới rõ ràng giúp xác định phần nào có thể giữ lại, phần nào cần thay đổi và những hành vi nào phải kiểm thử lại.

Chẳng hạn, logic giữ phòng của `hotel-service` có thể tiếp tục được sử dụng, trong khi cách nhận credential hoặc kết nối tới database thay đổi. Sau khi chuyển, chúng ta vẫn phải kiểm chứng các yêu cầu đã có: retry không tạo booking trùng, dữ liệu và outbox được lưu nhất quán, công việc đang dở có thể tiếp tục sau sự cố. Những kiểm thử này mới xác nhận ứng dụng giữ được hành vi mong muốn trên môi trường mới.

11.8 Tóm tắt

Khi chạy container trên AWS, chúng ta cần lựa chọn **công cụ điều phối workload** và **môi trường cung cấp tài nguyên thực thi**. ECS và EKS cung cấp hai mô hình điều phối khác nhau; EC2, Fargate và các phương án quản lý node quyết định phần công việc vận hành mà chúng ta cần thực hiện. TravelNow sử dụng EKS để tìm hiểu cụ thể các cơ chế Kubernetes của Chương 10 trong môi trường AWS. Lựa chọn cho một hệ thống thực tế vẫn cần dựa trên yêu cầu ứng dụng và khả năng vận hành.

Với EKS, AWS vận hành control plane, còn chúng ta quản lý workload, cấu hình tích hợp, quyền truy cập và việc cập nhật các thành phần đang sử dụng. Một bản phát hành cần liên kết được với image digest, manifest theo môi trường, migration và kết quả kiểm thử. Thông tin bàn giao phải làm rõ những gì đã được xác nhận và những phần còn cần hoàn thành.

Triển khai thành công cần được kiểm chứng bằng kết quả nghiệp vụ: booking được xử lý đến trạng thái cuối cùng, đồng thời các tình huống lỗi đã phân tích ở Phần III vẫn được phục hồi đúng khi ứng dụng chạy trên EKS. Trạng thái Pod và kết quả rollout là những bằng chứng cần thiết, nhưng chỉ phản ánh một phần của quá trình này.

Ba node và số Pod được cấu hình trong chương mới cung cấp **năng lực xử lý ban đầu**. Sang Chương 12, chúng ta sẽ xem TravelNow điều chỉnh tài nguyên như thế nào khi tải thay đổi và cần chuẩn bị những gì để tiếp tục phục vụ khi **một Availability Zone gặp sự cố**.

CHƯƠNG 12

Auto Scaling và High Availability

Phần IV — Vận hành ở quy mô lớn · *Operating at Scale*

TravelNow đã được triển khai trên EKS. Giả sử trong một đợt mở bán vé, API vẫn trả HTTP `202` nhanh chóng, nhưng thời gian từ lúc tiếp nhận yêu cầu đến khi có kết quả đặt chuyến ngày càng kéo dài. Chúng ta tăng số replica để xử lý thêm công việc đồng thời. Ban đầu, lượng công việc tồn đọng giảm, nhưng sau đó các lời gọi tới database bắt đầu bị timeout. Tình huống này cho thấy **bổ sung tài nguyên cho ứng dụng chỉ giúp tăng tốc độ xử lý khi database và các dịch vụ liên quan còn đủ khả năng đáp ứng**.

Chương này phân biệt hai yêu cầu có liên quan nhưng phục vụ những mục tiêu khác nhau: **autoscaling** điều chỉnh tài nguyên theo tải, còn **high availability (HA)** duy trì khả năng phục vụ khi một phần hệ thống gặp sự cố. Từ tình huống mở bán của TravelNow, chúng ta sẽ tìm thành phần đang giới hạn tốc độ xử lý, đo thời gian để tài nguyên bổ sung thực sự phục vụ được và tính toán mức dự phòng cần thiết khi mất một phần hạ tầng.

Các giả định này sẽ được kiểm chứng bằng thử tải và diễn tập sự cố, theo dõi cả khả năng phục hồi hạ tầng lẫn kết quả của những booking bị ảnh hưởng. Trên EKS, chúng ta xem xét cách phân bổ Pod, cấp thêm node và bố trí workload qua các Availability Zone (AZ). Khi sử dụng nền tảng khác, những câu hỏi cốt lõi vẫn giữ nguyên: **hệ thống xử lý được bao nhiêu công việc, mất bao lâu để tăng khả năng xử lý và còn phục vụ được đến đâu khi có sự cố?**

> **Kiểm chứng cấu hình bằng kết quả thực tế**
>
> Các con số trong chương là ví dụ tính toán hoặc cấu hình minh họa, không phải kết quả đo hiệu năng của TravelNow. Chúng giúp xác định những giả định cần kiểm tra và các chỉ số cần đo. Một cấu hình chỉ được xem là phù hợp sau khi được kiểm chứng với tải và điều kiện vận hành thực tế.

12.1 Phân biệt tự động co giãn và tính sẵn sàng cao

Autoscaling và **high availability (HA)** bổ sung cho nhau, nhưng cần được thiết kế và đánh giá theo những mục tiêu riêng. Autoscaling điều chỉnh tài nguyên khi tải thay đổi; HA giúp hệ thống tiếp tục phục vụ khi một phần hạ tầng hoặc ứng dụng gặp sự cố.

Tiêu chí	Autoscaling	High availability
Câu hỏi cần trả lời	Khi tải thay đổi, cần điều chỉnh tài nguyên nào và mất bao lâu để đáp ứng?	Khi một phần hệ thống gặp sự cố, các thành phần còn lại có tiếp tục phục vụ được không?
Cơ chế chính	Tăng hoặc giảm tài nguyên theo các tín hiệu như mức sử dụng CPU, lưu lượng request hoặc lượng công việc trong queue.	Duy trì tài nguyên dự phòng, phân bố các bản sao qua những vùng lỗi độc lập và chuyển việc xử lý sang các thành phần còn hoạt động.
Điều cần đo	Thời gian bổ sung tài nguyên và mức cải thiện thông lượng, độ trễ sau khi tài nguyên mới bắt đầu phục vụ.	Mức độ gián đoạn, thời gian phục hồi và năng lực xử lý còn lại khi mất một vùng lỗi.
Sai lầm thường gặp	Tăng số instance ở tầng ứng dụng trong khi nút thắt nằm ở database hoặc dịch vụ bên ngoài.	Có nhiều bản sao nhưng tất cả cùng phụ thuộc vào một node hoặc một AZ.

Một hệ thống có thể tự động tăng số Pod theo tải nhưng vẫn ngừng phục vụ khi mất một AZ nếu toàn bộ Pod nằm trong AZ đó. Ngược lại, phân bố workload qua ba AZ giúp giảm ảnh hưởng của sự cố tại một vùng, nhưng chưa bảo đảm hệ thống xử lý kịp lượng yêu cầu tăng mạnh trong đợt mở bán. Nếu tổng năng lực xử lý không đủ, công việc vẫn tồn đọng và khách hàng phải chờ lâu hơn.

Vì vậy, chúng ta cần kiểm tra cả **khả năng đáp ứng khi tải thay đổi** lẫn **khả năng duy trì phục vụ khi có sự cố**. Các mục tiếp theo sẽ lần lượt phân tích hai yêu cầu này trên TravelNow.

12.2 Xác định nút thắt trước khi tăng tài nguyên

12.2.1 Tăng số Pod ảnh hưởng thế nào đến database và các dịch vụ liên quan?

Ở Mục 10.5, chúng ta đã thấy khả năng thay thế instance phụ thuộc vào việc lưu trạng thái cần thiết bên ngoài từng Pod. TravelNow lưu dữ liệu nghiệp vụ trên RDS, PDF trên S3 và tiến độ xử lý trong workflow hoặc database của từng service. Khi một Pod bị mất, instance khác có thể tiếp tục công việc dựa trên những thông tin đã lưu. Transaction, lease và idempotency giúp các instance phối hợp, kiểm soát quyền xử lý và ngăn thao tác trùng lặp.

Thiết kế này cho phép tăng số Pod để xử lý nhiều công việc đồng thời. Tuy nhiên, **mỗi Pod bổ sung cũng có thể tạo thêm tải cho database và các dịch vụ mà nó sử dụng**. Các Pod mới có connection pool riêng, cần nạp dữ liệu vào cache và có thể cùng nhận message hoặc gọi API đối tác.

Giả sử mỗi Pod được cấu hình tối đa 20 kết nối database. Khi tăng từ 5 lên 20 Pod, tổng số kết nối tối đa của nhóm Pod này tăng từ 100 lên 400, chưa tính các worker khác và những Pod được tạo thêm trong quá trình rollout. Số kết nối thực tế phụ thuộc vào tải và cách quản lý pool, nhưng database vẫn cần được đánh giá theo tổng nhu cầu của tất cả thành phần sử dụng nó.

Mức sử dụng CPU thấp trên từng Pod chưa chứng minh hệ thống còn khả năng tiếp nhận thêm công việc. Các Pod có thể đang dành phần lớn thời gian chờ database phản hồi. Nếu database đã đạt giới hạn xử lý, tăng replica có thể làm tăng số truy vấn chờ, gây thêm timeout và retry. Vì vậy, chúng ta cần xem xét **khả năng xử lý của cả ứng dụng lẫn các thành phần phụ thuộc** trước khi quyết định tăng số Pod.

12.2.2 Đo thời gian xử lý và thời gian chờ

Trước khi cấu hình autoscaling, chúng ta cần xác định **thành phần nào đang giới hạn tốc độ hoàn tất công việc**. Có hai tình huống cần phân biệt:

- **Ứng dụng thiếu tài nguyên xử lý.** Nếu CPU hoặc giới hạn xử lý đồng thời của ứng dụng là nút thắt, tăng số Pod có thể giúp hoàn tất nhiều công việc hơn mỗi giây, với điều kiện database và các dịch vụ liên quan vẫn đáp ứng được tải bổ sung.
- **Ứng dụng đang chờ một thành phần đã quá tải.** Nếu database hoặc API hãng bay đã đạt giới hạn, tăng số Pod chủ yếu tạo thêm lời gọi cùng chờ kết quả. Khi đó, cần xử lý nút thắt tại thành phần này hoặc giới hạn lượng công việc gửi tới.

Để phân biệt, chúng ta cần đo thời gian ứng dụng thực hiện logic, thời gian chờ lấy kết nối và thời gian chờ database hoặc API bên ngoài phản hồi. Các số liệu này giúp xác định chặng đang làm chậm quy trình và nơi việc bổ sung tài nguyên có thể mang lại hiệu quả.

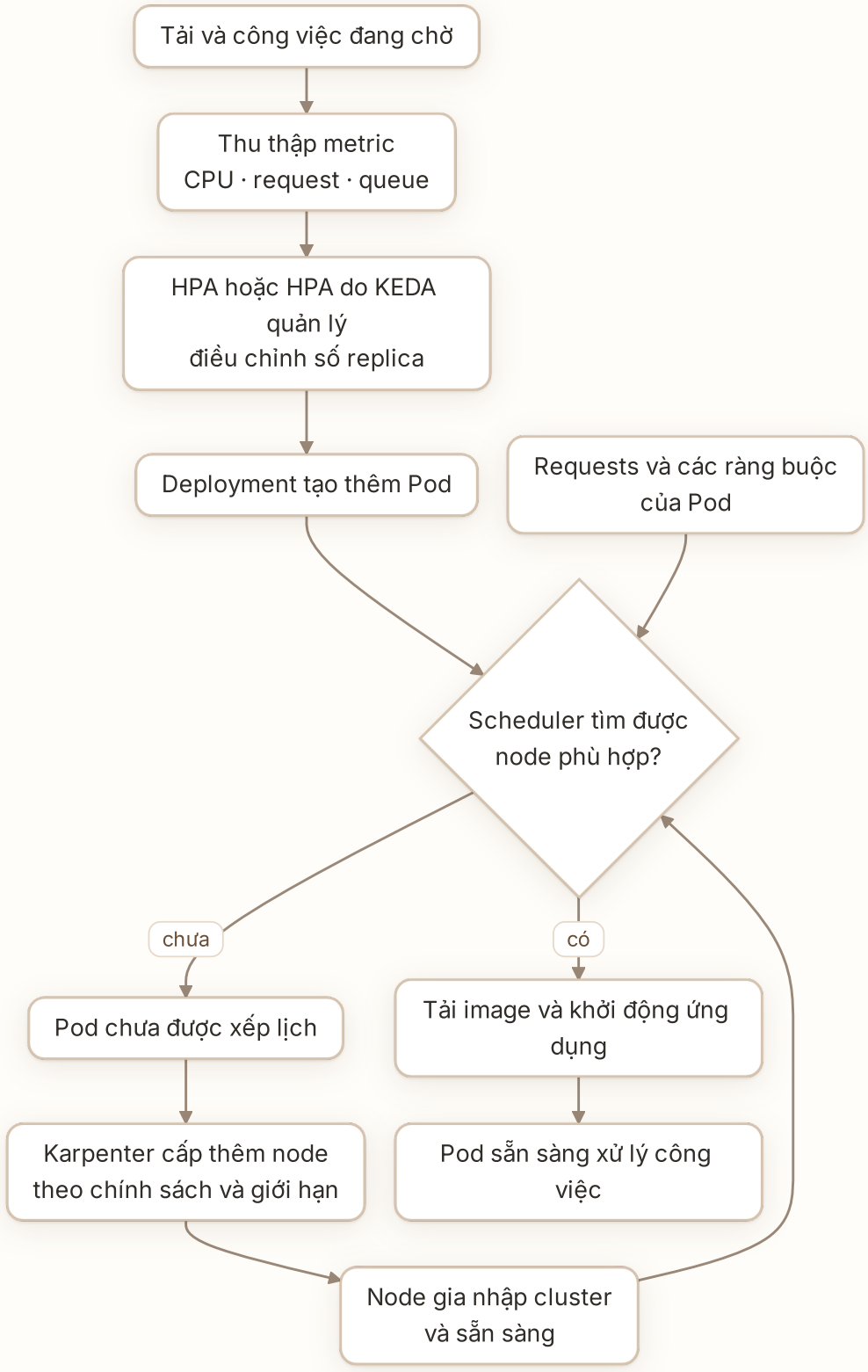
Đồng thời, cần theo dõi cả **thời gian hoàn tất một yêu cầu** và **số công việc hoàn tất mỗi giây**. Với TravelNow, thời gian API trả HTTP `202` chỉ phản ánh bước tiếp nhận; việc tăng tài nguyên cần được đánh giá bằng thời gian và tốc độ hoàn tất booking phía sau.

12.3 Điều chỉnh tài nguyên và thời gian đáp ứng khi tải tăng

12.3.1 Tài nguyên mỗi Pod, số replica và số node

Khả năng xử lý của ứng dụng trên EKS phụ thuộc vào ba yếu tố: **tài nguyên dành cho mỗi Pod, số Pod sẵn sàng làm việc và số node có thể chạy các Pod đó**. Khi tải tăng, yêu cầu thêm replica chỉ mang lại hiệu quả sau khi các Pod mới được xếp lịch, khởi động và bắt đầu xử lý công việc.

HÌNH 12.1 · SƠ ĐỒ LUỒNG



Requests khai báo nhu cầu CPU và bộ nhớ để scheduler lựa chọn node cho Pod. Giá trị quá cao có thể khiến node bị xem là không còn đủ tài nguyên để nhận thêm Pod, dù mức sử dụng thực tế vẫn thấp. Ngược lại, khai báo quá thấp có thể khiến nhiều workload được xếp lên cùng node và tranh chấp tài nguyên khi tải tăng.

Limits giới hạn mức tài nguyên container được sử dụng. Khi chạm giới hạn CPU, container bị hạn chế thời gian sử dụng CPU (*throttling*); khi vượt giới hạn bộ nhớ, container có thể bị kết thúc do thiếu bộ nhớ (*OOM-kill*). Requests và limits cần được xác định từ kết quả đo, bao gồm cả giai đoạn khởi động, xử lý tải cao và rollout.

Horizontal Pod Autoscaler (HPA) điều chỉnh số replica dựa trên metric. Nếu sử dụng CPU utilization với mục tiêu 70%, tỷ lệ này được tính so với CPU request đã khai báo. Nó không có nghĩa toàn hệ thống đang sử dụng 70% năng lực xử lý hay còn 30% tài nguyên dự phòng. Vì vậy, metric cần phản ánh đúng nhu cầu của workload, còn requests phải đủ sát với thực tế để HPA đưa ra quyết định phù hợp. Khi HPA quản lý số replica, pipeline cũng cần tránh ghi đè giá trị này như tình huống đã phân tích ở Chương 11. Kubernetes — HPA.

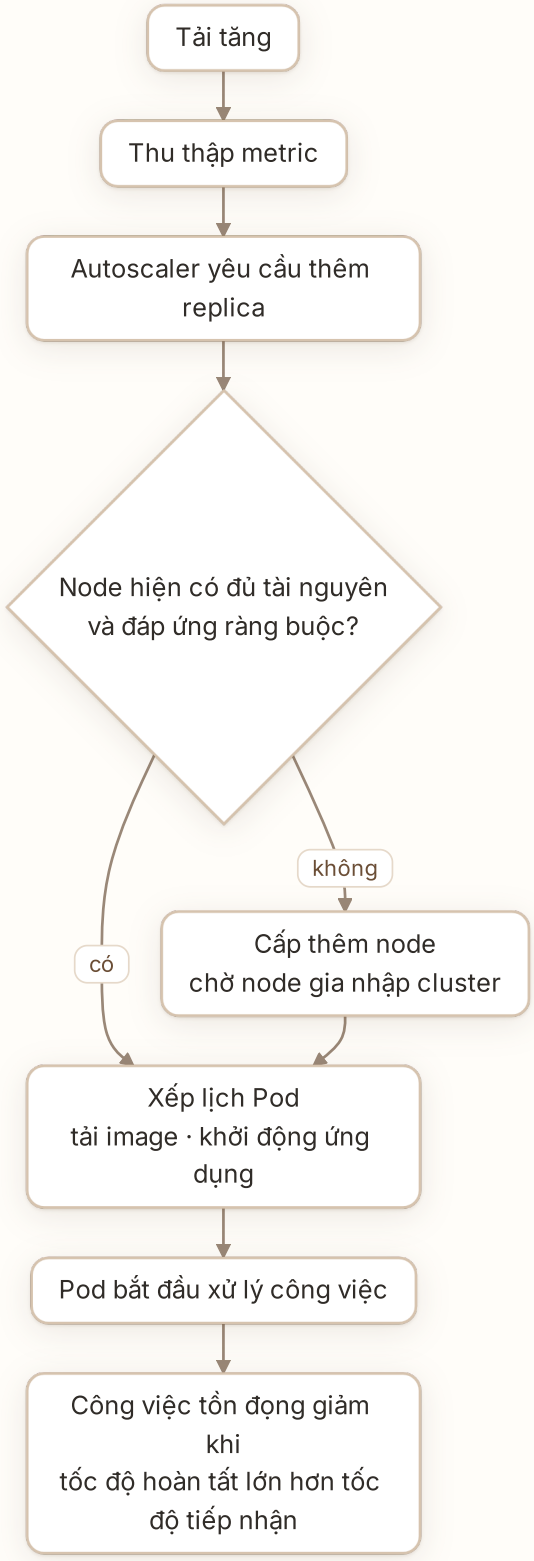
KEDA giúp sử dụng các tín hiệu bên ngoài, chẳng hạn lượng message trong SQS, để điều chỉnh số replica. Với worker xử lý message, lượng công việc đang chờ thường phản ánh nhu cầu bổ sung worker rõ hơn mức sử dụng CPU. KEDA tạo và quản lý HPA cho workload tương ứng; chúng ta không nên cấu hình thêm một HPA độc lập cùng thay đổi số replica của workload đó. KEDA — SQS scaler.

Karpenter xử lý nhu cầu bổ sung node khi các Pod chưa thể được xếp lịch trên những node hiện có. Việc cấp node phải tuân theo chính sách, giới hạn tài nguyên và các ràng buộc đã cấu hình. Trong phương án tiếp nối Chương 11, managed node group cung cấp tài nguyên cho các thành phần nền tảng và controller, còn Karpenter cấp thêm node cho ứng dụng khi cần. Nếu sử dụng EKS Auto Mode, AWS quản lý thêm cơ chế cấp phát và vòng đời node; chúng ta vẫn phải xác định chính sách và giới hạn phù hợp với workload. Karpenter — Cơ chế hoạt động.

12.3.2 Thời gian bổ sung tài nguyên và deadline của booking

Autoscaling không tạo ra năng lực xử lý mới ngay khi tải tăng. Hệ thống cần thời gian thu thập metric, tính số replica và chuẩn bị các Pod mới. Thời gian chờ còn phụ thuộc vào việc cluster đã có node phù hợp hay phải cấp thêm node:

HÌNH 12.2 · SƠ ĐỒ LUỒNG



Nếu node hiện có còn đủ tài nguyên và đáp ứng các ràng buộc của Pod, quá trình chỉ cần chờ xếp lịch và khởi động ứng dụng. Nếu phải bổ sung node, hệ thống cần thêm thời gian cấp máy, khởi tạo các thành phần trên node và đưa node vào trạng thái sẵn sàng.

Trong khoảng chờ đó, các yêu cầu mới vẫn tiếp tục đến. Với xử lý bất đồng bộ, công việc có thể tích lũy trong queue và chiếm thêm một phần thời hạn xử lý. Ngay cả khi Pod mới đã bắt đầu làm việc, lượng tồn đọng chỉ giảm nếu tổng tốc độ hoàn tất vượt tốc độ tiếp nhận công việc mới.

Với TravelNow, câu hỏi cần kiểm chứng là: **tài nguyên bổ sung có bắt đầu phục vụ đủ sớm và xử lý đủ nhanh để booking hoàn tất trong deadline hay không?** Để cải thiện, chúng ta cần đo thời gian của từng giai đoạn và chọn biện pháp tương ứng:

- **Rút ngắn giai đoạn đang gây chậm.** Nếu phần lớn thời gian nằm ở tải image và khởi động ứng dụng, tối ưu hai bước này giúp Pod phục vụ sớm hơn. Nếu chậm ở thu thập metric hoặc cấp node, cần điều chỉnh tại những giai đoạn đó.
- **Chuẩn bị trước cho tải có thể dự đoán.** Với đợt mở bán đã biết thời điểm, chúng ta có thể tăng tài nguyên và xác nhận ứng dụng sẵn sàng trước giờ mở bán. Autoscaling tiếp tục điều chỉnh theo mức tải thực tế và xử lý những biến động ngoài dự kiến.

12.4 Tốc độ xử lý và thời gian hoàn tất công việc tồn đọng

12.4.1 Ước tính thời gian xử lý backlog

Xét `hotel-worker` trong phương án choreography: worker chạy bằng container, nhận message từ SQS và chỉ xác nhận sau khi transaction đã commit. Giả sử queue có **5.000 công việc đang chờ**, tiếp tục nhận thêm **80 việc mỗi giây**, và mỗi worker có thể xử lý ổn định **10 việc mỗi giây**. Đây là các giá trị giả định để tính toán; tạm thời bỏ qua thời gian khởi động, retry và sự khác biệt về thời lượng giữa các công việc.

Số worker sẵn sàng	Tổng tốc độ xử lý	Khả năng xử lý phần tồn đọng	Thời gian dự kiến để hết 5.000 việc đang chờ
5	50 việc/giây	Lượng tồn tăng thêm 30 việc/giây.	Không giảm được backlog.
8	80 việc/giây	Chỉ đủ đáp ứng lượng công việc mới.	Backlog giữ nguyên.
10	100 việc/giây	Xử lý thêm được 20 việc tồn/giây.	Khoảng 250 giây.
20	200 việc/giây	Xử lý thêm được 120 việc tồn/giây.	Khoảng 42 giây.

Với những giả định trên, thời gian để xử lý hết backlog được tính bằng:

> **Thời gian xử lý hết backlog = Số công việc đang chờ ÷ (Tốc độ hoàn tất − Tốc độ tiếp nhận)**

Công thức chỉ áp dụng khi tốc độ hoàn tất lớn hơn tốc độ tiếp nhận và các tốc độ này được duy trì trong khoảng thời gian tính toán. Nếu hai tốc độ bằng nhau, hệ thống chỉ đủ sức xử lý lượng việc mới và không giảm được phần tồn đọng.

Bảng cũng cho thấy việc tăng từ 8 lên 10 worker, tương đương 25%, tạo ra phần năng lực xử lý cần thiết để backlog bắt đầu giảm. Tuy nhiên, nếu database chỉ đáp ứng được tối đa 110 việc mỗi giây, 20 worker cũng không thể duy trì tổng tốc độ 200 việc mỗi giây như giả định. **Backlog giảm nhanh hay chậm phụ thuộc vào chênh lệch giữa tốc độ hoàn tất thực tế và tốc độ tiếp nhận, không chỉ vào số Pod.**

Giả sử KEDA được cấu hình với mục tiêu 50 message đang chờ trên mỗi replica và giới hạn tối đa 20 replica. Với backlog 5.000 message, nhu cầu tính theo tỷ lệ này vượt giới hạn, nên số replica vẫn chỉ có thể tăng tới 20. Mục tiêu 50 message mỗi replica cũng chưa chứng minh booking sẽ hoàn tất trong 30 giây. Thời gian thực tế còn phụ thuộc vào tốc độ của worker, thời gian bổ sung tài nguyên, công việc đang xử lý, retry và giới hạn của database hoặc đối tác.

Vì vậy, bên cạnh số replica, chúng ta cần theo dõi **thời gian công việc đã chờ và tốc độ hoàn tất thực tế** để biết hệ thống có đang bắt kịp tải hay không.

12.4.2 Khi autoscaling làm tình trạng quá tải nghiêm trọng hơn

Nếu backlog tăng do database xử lý chậm, việc bổ sung worker có thể tạo ra một vòng lặp khiến sự cố ngày càng nghiêm trọng:

HÌNH 12.3 · SƠ ĐỒ LUỒNG



Trong tình huống này, autoscaler vẫn hoạt động đúng theo cấu hình: backlog tăng thì số worker tăng. Vấn đề nằm ở chỗ database đã là nút thắt, nên worker mới chủ yếu tạo thêm truy vấn và kết nối tới thành phần đang quá tải.

Vì vậy, **giới hạn số replica cần dựa trên khả năng đáp ứng của database và các dịch vụ liên quan**, đồng thời phù hợp với ngân sách. Giới hạn này cần phối hợp với bulkhead, giới hạn xử lý đồng thời và chính sách retry có khoảng nghỉ đã trình bày ở Chương 7.

Tuy nhiên, giới hạn số worker chưa làm giảm lượng công việc được gửi vào hệ thống. Nếu tốc độ tiếp nhận vẫn cao hơn tốc độ hoàn tất, queue tiếp tục tích lũy message. Chúng ta cần lựa chọn cách giảm tốc độ tiếp nhận, từ chối có kiểm soát hoặc tăng tài nguyên tại thành phần đang giới hạn khả năng xử lý.

Queue giúp giữ công việc trong những khoảng chênh lệch tạm thời giữa tốc độ tiếp nhận và xử lý. Nếu chênh lệch kéo dài, thời gian chờ vẫn tăng và có thể vượt deadline nghiệp vụ.

12.4.3 Xử lý công việc tồn đọng trong khi vẫn tiếp nhận yêu cầu mới

Giả sử database vừa phục hồi sau một lần gián đoạn và hệ thống còn 5.000 công việc đang chờ. Cùng lúc, worker tiếp nhận yêu cầu mới, relay chuyển tiếp các bản ghi outbox cũ và người vận hành chuẩn bị đưa message từ DLQ trở lại xử lý (*redrive*). Từng hoạt động có thể hợp lệ khi xét riêng, nhưng tổng lượng công việc có thể vượt khả năng của database vừa phục hồi.

Chúng ta cần xác định **tổng mức tải cho phép, bao gồm yêu cầu mới, công việc tồn đọng và retry**. Quá trình xử lý lại nên bắt đầu ở tốc độ đã được kiểm chứng, sau đó tăng dần nếu thời gian phản hồi của database và tốc độ hoàn tất vẫn ổn định. Nếu các chỉ số xấu đi, cần giảm tốc độ hoặc tạm dừng redrive.

Ví dụ, nếu database xử lý được 110 việc mỗi giây trong cùng điều kiện tải và hệ thống tiếp tục nhận 80 việc mới mỗi giây, phần còn lại để xử lý backlog chỉ là 30 việc mỗi giây, trước khi tính thêm retry và các tác vụ khác. Giới hạn của từng nhóm worker cần được phối hợp để tổng tải không vượt mức này.

Trước khi xử lý lại, hệ thống cũng phải kiểm tra trạng thái hiện tại của công việc. Một yêu cầu giữ phòng đã quá deadline hoặc thuộc Trip đã hủy không được coi như yêu cầu đặt phòng mới. Nó cần được xử lý theo các quy tắc kiểm tra trạng thái, bù trừ hoặc đối soát đã thiết kế ở Phần III, đồng thời giữ nguyên định danh thao tác.

Do đó, kế hoạch phục hồi phải trả lời hai câu hỏi: **có thể xử lý lại với tốc độ bao nhiêu, và những công việc nào vẫn còn được phép tiếp tục?**

12.5 Thử tải dựa trên kết quả booking

Một bài thử tải cần cho biết hệ thống sử dụng tài nguyên như thế nào và người dùng nhận được kết quả sau bao lâu. Với TravelNow, chúng ta cần theo dõi đồng thời bốn nhóm số liệu:

- **Tốc độ tiếp nhận và hoàn tất:** số yêu cầu được nhận, số booking đạt trạng thái cuối cùng mỗi giây; phân biệt kết quả nghiệp vụ hợp lệ với lỗi kỹ thuật.
- **Thời gian xử lý toàn bộ booking:** từ lúc yêu cầu được tiếp nhận và trả HTTP `202` đến khi có kết quả cuối cùng, cùng thời gian chờ của những booking chưa hoàn tất.
- **Khả năng bổ sung tài nguyên:** số replica mong muốn, số Pod sẵn sàng và thời gian từ lúc yêu cầu tăng tài nguyên đến khi các Pod mới bắt đầu xử lý công việc.
- **Tình trạng của các thành phần phụ thuộc:** số kết nối, thời gian phản hồi và tỷ lệ lỗi của database cùng API đối tác, để phát hiện khi nút thắt chuyển từ ứng dụng sang thành phần khác.

Các dạng tải khác nhau giúp kiểm tra những khía cạnh khác nhau. Tăng tải từng bước giúp xác định mức hệ thống có thể xử lý ổn định. Tăng tải đột ngột giúp đo thời gian phản ứng của autoscaling. Giảm tải giúp kiểm tra việc thu hẹp số Pod hoặc node có làm gián đoạn công việc

đang thực hiện hay không.

Trong môi trường kiểm thử, chúng ta cũng cần thực hiện rollout hoặc scale-in khi vẫn còn công việc đang xử lý, đồng thời chủ động dừng worker tại những thời điểm quan trọng:

Thời điểm dừng worker	Kết quả cần kiểm tra sau khi worker khác tiếp tục
Sau khi lưu Trip và outbox, trước khi chuyển tiếp công việc	Relay vẫn tìm thấy và chuyển tiếp được công việc. Thời gian phục hồi cần tính cả thời gian chờ lease cũ hết hạn nếu có.
Sau khi publish thành công, trước khi đánh dấu outbox đã gửi	Event được phát lại với cùng định danh; bên nhận nhận diện được việc đã xử lý và không lặp side effect.
Sau khi consumer commit, trước khi xác nhận message	Message được giao lại nhưng kết quả đã lưu được nhận diện, giúp quá trình xử lý tiếp tục an toàn.
Khi công việc chưa hoàn tất nhưng đã hết thời gian shutdown	Worker đã ngừng nhận việc mới; phần việc còn dang dở có thể được nhận lại hoặc đối soát dựa trên trạng thái lưu bền vững.

Hai tình huống publish và xác nhận message áp dụng cho phương án choreography. Với orchestration, chúng ta kiểm tra tình huống tương ứng: domain service đã hoàn tất thao tác nhưng adapter không nhận được phản hồi. Hệ thống phải xác định lại kết quả bằng cùng định danh thao tác trước khi quyết định bước tiếp theo.

Ngoài việc dừng có graceful shutdown, cần kiểm tra cả trường hợp process bị kết thúc đột ngột trong môi trường cách ly. Khả năng phục hồi phải dựa trên trạng thái bền vững và idempotency, vì ứng dụng không phải lúc nào cũng có cơ hội chạy xong các bước shutdown.

Kết quả bài thử cần nêu rõ **mức tải đáp ứng được mục tiêu, cấu hình đã sử dụng và những giới hạn quan sát được**. Chẳng hạn, "HPA đã tăng lên 10 Pod" chỉ xác nhận autoscaler đã phản ứng. Để kết luận việc tăng Pod có hiệu quả, chúng ta còn phải kiểm tra booking có hoàn tất trong thời hạn yêu cầu, backlog có giảm và các thao tác nghiệp vụ có được thực hiện đúng hay không.

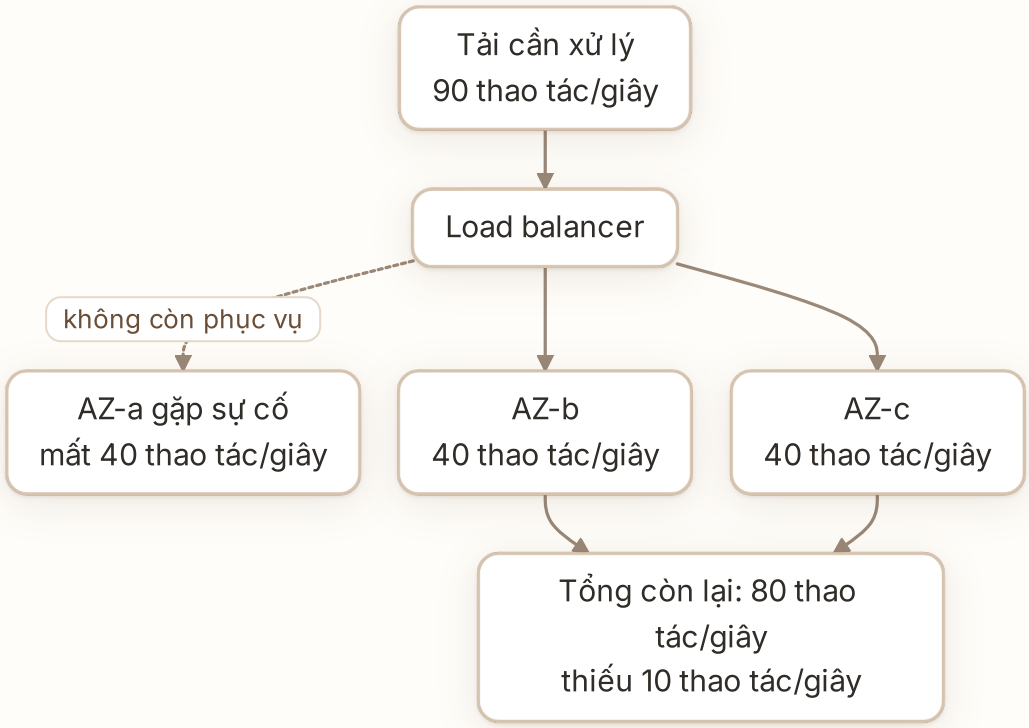
12.6 Duy trì khả năng phục vụ khi một vùng hạ tầng gặp sự cố

12.6.1 Tính toán tài nguyên dự phòng theo phạm vi sự cố

Autoscaling điều chỉnh tài nguyên theo tải, còn thiết kế HA cần tính đến khả năng phục vụ khi một phần tài nguyên không còn hoạt động. Để đánh giá điều này, chúng ta cần xác định **vùng lỗi (*failure domain*)**: nhóm tài nguyên có thể cùng bị ảnh hưởng bởi một sự cố, chẳng hạn một node hoặc một Availability Zone.

Giả sử `payment-service` có ba instance, mỗi instance xử lý ổn định 40 thao tác mỗi giây và được phân bố đều trên ba AZ. Với tải 90 thao tác mỗi giây, tổng năng lực xử lý 120 thao tác mỗi giây đáp ứng được nhu cầu hiện tại. Nhưng khi mất một AZ, hai instance còn lại chỉ xử lý được 80 thao tác mỗi giây. Ứng dụng vẫn hoạt động, nhưng không còn đủ khả năng theo kịp tải.

HÌNH 12.4 · SƠ ĐỒ LUỒNG



Chúng ta có thể ước tính mức dự phòng trước khi kiểm chứng bằng diễn tập. Giả sử tải cần phục vụ là **L**, năng lực xử lý được chia đều trên **N vùng lỗi độc lập**, với N lớn hơn 1, và toàn bộ tải có thể chuyển sang những vùng còn hoạt động. Để chịu được việc mất một vùng mà không phải chờ bổ sung tài nguyên, mỗi vùng cần có năng lực xử lý tối thiểu:

> **Năng lực xử lý mỗi vùng ≥ L ÷ (N − 1)**

Với tải 90 thao tác mỗi giây, kết quả tính toán như sau:

Số AZ	Năng lực xử lý tối thiểu mỗi AZ	Tổng năng lực xử lý cần duy trì	Tỷ lệ so với tải
2	90 thao tác/giây	180 thao tác/giây	2 lần
3	45 thao tác/giây	135 thao tác/giây	1,5 lần

Trong mô hình này, phân bố qua ba AZ cần ít tổng tài nguyên compute dự phòng hơn hai AZ để chịu được sự cố mất một vùng. Tuy nhiên, phép tính chưa xét giới hạn của database, API đối tác, kết nối mạng hay việc tài nguyên phải được cấp theo kích thước instance cụ thể. Nó cũng chưa đủ để kết luận phương án nào có tổng chi phí thấp hơn.

Duy trì tài nguyên dự phòng giúp hệ thống tiếp tục xử lý ngay khi sự cố xảy ra. Nếu lựa chọn chờ autoscaler cấp bù, chúng ta có thể giảm tài nguyên nhàn rỗi nhưng phải chấp nhận một khoảng thời gian suy giảm khả năng phục vụ. Việc cấp thêm máy trong thời gian mất AZ cũng có thể gặp

hạn chế, nên khả năng phục hồi không nên phụ thuộc hoàn toàn vào giả định rằng tài nguyên mới luôn có sẵn.

12.6.2 Phân bố Pod và kiểm soát gián đoạn khi bảo trì

Trên EKS, **topology spread constraints** giúp phân bố Pod giữa các node hoặc AZ đáp ứng điều kiện chạy workload. Cấu hình này cần cân bằng hai yêu cầu: tránh tập trung các bản sao vào cùng một vùng lỗi và cho phép hệ thống bố trí lại Pod khi một vùng không còn khả dụng.

Nếu ràng buộc quá chặt, Pod có thể tiếp tục ở trạng thái `Pending` dù những vùng còn lại vẫn có tài nguyên. Nếu ràng buộc quá lỏng, nhiều bản sao có thể tập trung vào cùng một vùng và cùng mất khi xảy ra sự cố. Vì vậy, cần kiểm tra cách scheduler phân bố Pod cả trong điều kiện bình thường lẫn khi thiếu một AZ.

Số node cũng không trực tiếp cho biết mức độ dự phòng. Ba node có thể nằm trong cùng một AZ. Nếu mở rộng từ VPC hai AZ ở Chương 5 sang ba AZ, chúng ta phải chuẩn bị subnet, route và cấu hình cấp node tương ứng; autoscaler không tự tạo đầy đủ các điều kiện mạng này.

PodDisruptionBudget (PDB) giúp giới hạn việc gián đoạn Pod do những thao tác chủ động sử dụng cơ chế eviction, chẳng hạn drain node để nâng cấp. PDB không ngăn được node hỏng hoặc mất AZ, cũng không thay thế các tham số `maxUnavailable` và `maxSurge` của Deployment khi rollout.

Một PDB quá chặt có thể khiến quá trình bảo trì không tiếp tục được nếu cluster thiếu tài nguyên để chạy Pod thay thế. Vì vậy, chính sách bảo trì phải được xem xét cùng số replica, cách phân bố Pod và phần tài nguyên dự phòng. Kubernetes — Disruptions.

12.6.3 Kiểm tra khả năng chịu lỗi của từng thành phần

Tầng ứng dụng còn đủ Pod chưa chứng minh toàn bộ quy trình booking có thể tiếp tục. Request vẫn cần đi qua load balancer, ứng dụng phải truy cập dữ liệu và các bước nghiệp vụ còn phụ thuộc vào workflow hoặc dịch vụ bên ngoài. Mỗi thành phần có phạm vi chịu lỗi và điều kiện phục hồi riêng:

Thành phần	Nội dung cần kiểm chứng với TravelNow
Load balancer và compute	Request vẫn tới được các target khỏe; những vùng còn hoạt động có đủ tài nguyên để tiếp nhận tải chuyển sang.
Database	RDS Multi-AZ đã được cấu hình cho môi trường cần HA; ứng dụng có thể kết nối lại và xử lý các kết quả chưa rõ sau failover.
Lưu trữ PDF	Storage class của S3 đáp ứng phạm vi dự phòng cần thiết; phân biệt lớp lưu trên nhiều AZ với lớp One Zone.
Queue và workflow	Giới hạn dịch vụ, thời gian lưu giữ và chính sách retry cho phép tiếp tục những công việc chưa hoàn tất.
API đối tác	Có timeout, cơ chế đối soát và chính sách xử lý khi đối tác không thể phục vụ.

Khi triển khai ngoài AWS, chúng ta vẫn cần xác định các vùng lỗi thực tế như máy chủ, rack, nguồn điện, thiết bị mạng hoặc trung tâm dữ liệu. Việc gắn nhãn `zone-a` và `zone-b` chỉ có ý nghĩa khi cách bố trí hạ tầng thực sự giảm được nguy cơ hai vùng cùng gián đoạn.

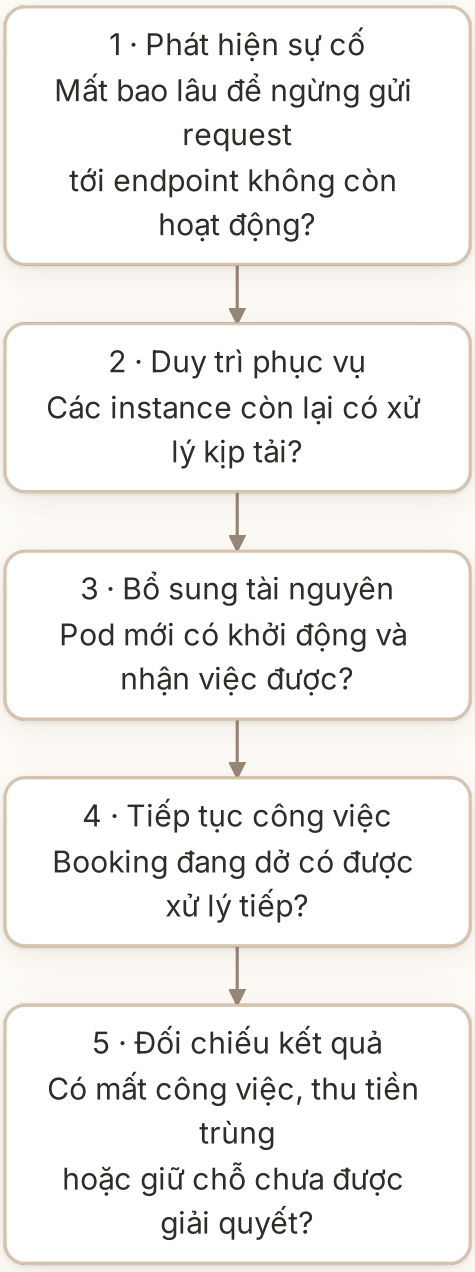
12.7 Kiểm chứng khả năng phục hồi bằng diễn tập

Thiết kế HA mô tả cách hệ thống dự kiến ứng phó với sự cố. Diễn tập giúp kiểm tra những giả định đó khi ứng dụng đang xử lý công việc.

Trong môi trường kiểm thử cách ly, chúng ta có thể cho TravelNow chạy ở mức tải đã xác định rồi mô phỏng mất các node ứng dụng tại một AZ. Kịch bản cần nêu rõ phạm vi tác động, thời gian thực hiện, điều kiện dừng và cách khôi phục. Phép thử này đánh giá sự cố ở tầng compute; database failover hoặc sự cố ảnh hưởng rộng hơn tới một AZ cần những kịch bản riêng.

Trong quá trình diễn tập, chúng ta theo dõi từ lúc phát hiện sự cố đến khi giải quyết xong các booking bị ảnh hưởng:

HÌNH 12.5 · SƠ ĐỒ LUỒNG



Hai bước cuối kiểm tra mối liên hệ giữa phục hồi hạ tầng và phục hồi nghiệp vụ. Trong kịch bản RDS failover, ứng dụng phải kết nối lại sau khi các connection cũ bị ngắt và xác định kết quả của những transaction chưa rõ trạng thái. Với SQS, message có thể được giao lại sau khi worker mất, nên lần xử lý tiếp theo phải nhận diện các thao tác đã hoàn tất và tránh lặp side effect.

Pod đạt trạng thái sẵn sàng cho biết các điều kiện readiness đã được đáp ứng; chúng ta còn phải kiểm tra công việc đang dở có được tiếp tục và hoàn tất đúng hay không.

Khi kết thúc diễn tập, cần đối chiếu tập `tripId` đã tiếp nhận và phân loại từng kết quả: booking thành công, thất bại sau khi đã bù trừ, hoặc còn chờ xử lý với người phụ trách cụ thể. Việc kiểm tra phải bao gồm cả công việc chưa được chuyển tiếp khỏi outbox và message đã vào DLQ. Queue

chính không còn message chưa có nghĩa toàn bộ công việc đã được giải quyết.

Báo cáo nên ghi riêng **thời điểm hệ thống có thể phục vụ trở lại** và **thời điểm các booking bị ảnh hưởng được xử lý xong**. Cách ghi nhận này giúp thể hiện đầy đủ phần việc còn lại sau khi hạ tầng phục hồi.

Hai mục tiêu cần được xác định trước khi diễn tập là:

- **RTO — Recovery Time Objective:** thời gian khôi phục mục tiêu của chức năng hoặc dịch vụ đang xét.
- **RPO — Recovery Point Objective:** mức mất dữ liệu có thể chấp nhận, biểu thị bằng khoảng thời gian dữ liệu có thể phải khôi phục lùi lại.

Chúng ta cần đặt RTO và RPO theo yêu cầu của từng chức năng, loại dữ liệu rồi kiểm chứng bằng kết quả diễn tập. HA giúp giảm gián đoạn do một số sự cố hạ tầng; những tình huống xóa nhầm hoặc ghi sai dữ liệu vẫn cần backup, restore và đối soát. Chương 15 sẽ phân tích kỹ hơn các yêu cầu này.

12.8 Tóm tắt

Autoscaling và HA giải quyết hai yêu cầu khác nhau. **Autoscaling điều chỉnh tài nguyên theo tải**, dựa trên tín hiệu phù hợp với công việc, thời gian phản ứng và giới hạn của các thành phần liên quan. Tăng số Pod chỉ cải thiện kết quả khi tài nguyên bổ sung giúp tăng tốc độ hoàn tất thực tế. Backlog bắt đầu giảm khi tốc độ hoàn tất vượt tốc độ tiếp nhận.

HA duy trì khả năng phục vụ khi một phần hệ thống gặp sự cố. Các bản sao cần được phân bố qua những vùng lỗi phù hợp, đồng thời phần còn hoạt động phải đủ sức tiếp nhận tải. Trong mô hình chia đều qua N vùng độc lập và chuyển được toàn bộ tải sang các vùng còn lại, mỗi vùng cần năng lực xử lý tối thiểu $L \div (N - 1)$ để chịu mất một vùng mà không chờ cấp bù. Đây là cơ sở tính toán ban đầu, cần được đối chiếu với giới hạn của từng thành phần.

Thử tải và diễn tập phải theo dõi đến **kết quả cuối cùng của booking**, bao gồm những công việc còn trong outbox, queue, DLQ hoặc đang chờ đối soát. Số Pod tăng hay hạ tầng hoạt động trở lại mới chỉ phản ánh một phần của khả năng phục hồi.

Sau khi tìm hiểu công việc quản lý tài nguyên và duy trì khả năng phục vụ trên EKS, Chương 13 quay lại mô hình serverless: khi nền tảng quản lý thêm việc cấp phát môi trường thực thi, chúng ta vẫn cần đưa ra những quyết định nào để TravelNow hoạt động đúng và đáp ứng yêu cầu?

CHƯƠNG 13

Tản mạn về kiến trúc Serverless

Phần IV — Vận hành ở quy mô lớn · *Operating at Scale*

Giả sử `car-service` của TravelNow nhận ít yêu cầu trong phần lớn thời gian, nhưng lượng đặt xe tăng đột ngột vào cuối tuần. Khi chạy service này trên EKS, chúng ta cần tính toán lượng tài nguyên sẵn có và thời gian bổ sung tài nguyên để đáp ứng tải tăng. Nếu chuyển sang Lambda, nền tảng sẽ quản lý thêm những phần nào, và chúng ta vẫn cần quyết định những gì để service hoạt động ổn định?

Chương 9 đã triển khai TravelNow bằng Lambda. Những kiến thức về Kubernetes, EKS và autoscaling ở Chương 10–12 giúp chúng ta có thêm cơ sở để đánh giá lựa chọn đó. Lấy `car-service` làm ví dụ, chương này phân tích **cách phân chia trách nhiệm khi sử dụng serverless**, những đặc điểm tải phù hợp, giới hạn xử lý đồng thời và chi phí khi kết hợp nhiều mô hình thực thi trong cùng một hệ thống.

Trước hết, cần làm rõ hai khái niệm: **container** là cách đóng gói ứng dụng cùng các thành phần cần thiết để chạy; **serverless** là mô hình trong đó nền tảng quản lý việc cấp phát tài nguyên và một phần hoạt động của môi trường thực thi. Hai khái niệm này có thể kết hợp với nhau: Fargate chạy container mà chúng ta không cần trực tiếp quản lý node, còn Lambda cũng hỗ trợ triển khai bằng container image. Vì vậy, để so sánh các phương án, chúng ta cần xem xét **nền tảng quản lý những gì và phần việc nào vẫn thuộc trách nhiệm của người phát triển, vận hành ứng dụng**.

13.1 Phân chia trách nhiệm khi sử dụng serverless

13.1.1 Trách nhiệm của AWS và ứng dụng khi xử lý yêu cầu giữ xe

Trong phương án EKS ở Chương 12, chúng ta khai báo nhu cầu tài nguyên của Pod, chọn tín hiệu autoscaling, cấu hình cơ chế bổ sung node và tính toán tài nguyên dự phòng khi mất một AZ. Với Lambda, AWS cấp phát và duy trì môi trường thực thi. Chúng ta vẫn cần cấu hình tài nguyên cho function, giới hạn số lượt thực thi đồng thời (concurrency), timeout và nguồn kích hoạt.

Xét một yêu cầu giữ xe của `car-service`, trách nhiệm được phân chia như sau:

Câu hỏi	Cách giải quyết trong phương án Lambda
Code chạy trên máy nào, và máy gặp sự cố được thay thế ra sao?	AWS quản lý hạ tầng phục vụ việc thực thi function
Khi tải tăng, môi trường thực thi được bổ sung như thế nào?	Lambda cấp thêm môi trường thực thi trong các giới hạn co giãn và concurrency áp dụng
Cho phép bao nhiêu yêu cầu giữ xe được xử lý đồng thời?	Chúng ta đặt giới hạn dựa trên hạn mức dịch vụ và khả năng tiếp nhận của database, API đối tác
Timeout có đồng nghĩa với việc giữ xe chưa thành công không?	Ứng dụng phải đối soát kết quả với đối tác trước khi quyết định bước tiếp theo
Thực hiện lại yêu cầu có tạo thêm một lần đặt xe không?	Cơ chế idempotency của `car-service` và contract với đối tác phải bảo đảm thao tác không bị thực hiện trùng
Khi không có yêu cầu, hệ thống còn phát sinh chi phí không?	Cần tính cả cấu hình tài nguyên cấp trước, database, mạng, log và các dịch vụ đi kèm

Việc AWS quản lý môi trường thực thi giúp giảm công việc vận hành hạ tầng, nhưng chúng ta vẫn phải xác định giới hạn xử lý phù hợp với các thành phần mà ứng dụng gọi tới. Mỗi yêu cầu giữ xe cần có định danh thao tác, thời hạn hoàn tất và cơ chế xác định kết quả nếu quá trình xử lý bị gián đoạn. **AWS quản lý môi trường thực thi; tính đúng đắn của thao tác giữ xe vẫn phụ thuộc vào thiết kế ứng dụng và cách phối hợp với đối tác.**

13.1.2 Serverless trên nền tảng tự vận hành

Cách phân chia trách nhiệm của serverless cũng có thể áp dụng trên nền tảng do tổ chức tự vận hành. Chẳng hạn, **Knative Serving** hỗ trợ triển khai và co giãn container trên Kubernetes, bao gồm khả năng giảm số replica về 0 khi không có request, tùy cấu hình. Nhóm phát triển `car-service` sử dụng nền tảng này để chạy ứng dụng, còn nhóm nền tảng quản lý cluster và hạ tầng bên dưới. Khi workload giảm về 0 replica, cluster và các thành phần dùng chung vẫn có thể tiếp tục tiêu thụ tài nguyên và phát sinh chi phí. Knative — Serving.

Trong mô hình này, công việc quản lý máy chủ được chuyển từ nhóm phát triển ứng dụng sang nhóm nền tảng. Vì vậy, khi sử dụng serverless, chúng ta cần hiểu rõ **nền tảng quản lý những phần nào, cam kết đến đâu và ứng dụng còn phải tự xử lý những gì**. Sự phân chia đó giúp xác định trách nhiệm khi triển khai, mở rộng tài nguyên và khôi phục hoạt động sau sự cố.

13.2 Lựa chọn môi trường thực thi theo đặc điểm tải

Để lựa chọn môi trường thực thi cho `car-service`, chúng ta cần đánh giá cách mỗi phương án đáp ứng cùng một lượng công việc dưới những dạng tải khác nhau. Có thể bắt đầu với ba trường hợp giả định:

Đặc điểm tải	Mục tiêu	Những thông tin cần đo và kiểm chứng
Ít yêu cầu, thường xuyên có khoảng thời gian không phát sinh công việc	Giảm chi phí duy trì tài nguyên khi ứng dụng không xử lý yêu cầu	Tổng chi phí và độ trễ của yêu cầu đầu tiên sau thời gian không hoạt động
Lượng yêu cầu tăng đột ngột trong thời gian ngắn	Bổ sung khả năng xử lý kịp thời	Thời gian cấp thêm tài nguyên, số yêu cầu bị giới hạn tốc độ, thời gian hoàn tất và khả năng tiếp nhận của API đối tác
Tải cao và ổn định trong thời gian dài	Duy trì độ trễ và chi phí xử lý phù hợp	Chi phí cho cùng lượng công việc, bao gồm tài nguyên dự phòng và công sức vận hành

Khi service thường xuyên không có yêu cầu, mô hình tính phí theo mức sử dụng có thể giúp giảm chi phí duy trì tài nguyên chờ sẵn. Khi tải cao và ổn định, tài nguyên được cấp trước và sử dụng hiệu quả có thể mang lại chi phí thấp hơn trên mỗi yêu cầu. Tuy nhiên, cả hai phương án đều cần đáp ứng mục tiêu về độ trễ, khả năng chịu lỗi và các giới hạn của dịch vụ. Vì vậy, **việc so sánh phải dựa trên cùng lượng công việc và cùng mục tiêu phục vụ**, đồng thời tính đến công sức triển khai, theo dõi và xử lý sự cố.

Kết luận còn phụ thuộc vào hạ tầng hiện có. Khi không có lượt gọi, Lambda on-demand không phát sinh phí thời lượng thực thi, nhưng RDS, lưu trữ, mạng và các thành phần khác của TravelNow vẫn có thể phát sinh chi phí. Tương tự, nếu cluster EKS dùng chung còn đủ tài nguyên, chi phí bổ sung `car-service` sẽ khác với chi phí xây dựng một cluster riêng cho service này.

Do đó, cần đánh giá **tổng chi phí của từng phương án trong bối cảnh hạ tầng thực tế**, phân biệt rõ phần tài nguyên dùng chung, phần chi phí phát sinh thêm và phần có thể cắt giảm khi thay đổi môi trường thực thi.

13.3 Khả năng xử lý đồng thời và giới hạn của các thành phần phụ thuộc

13.3.1 Ước tính số lượt thực thi đồng thời

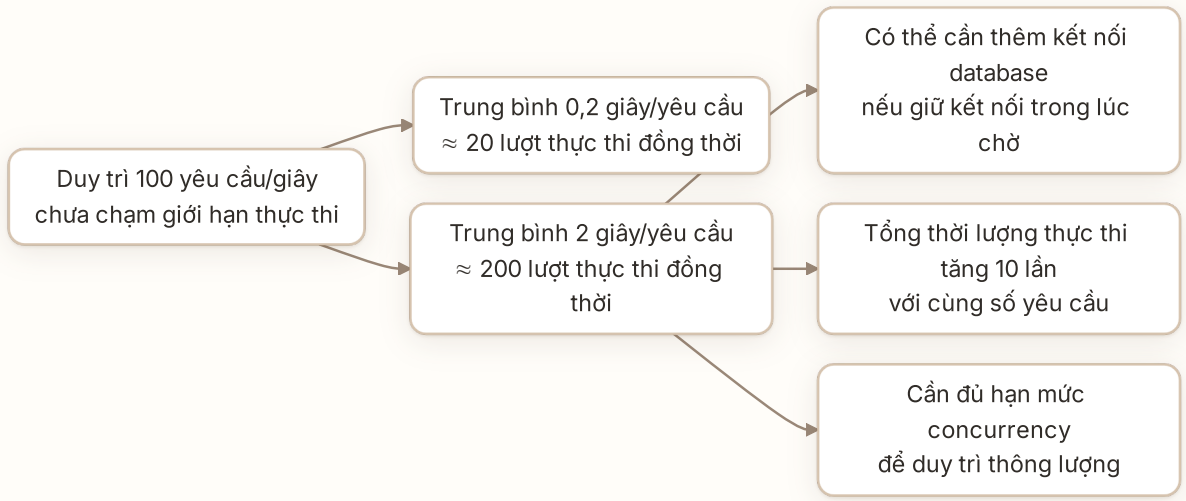
Để duy trì một tốc độ xử lý nhất định, ứng dụng cần có đủ số lượt thực thi chạy cùng lúc. Số lượt thực thi đồng thời này được gọi là **concurrency**. Khi tải tương đối ổn định, có thể ước tính giá trị trung bình bằng công thức:

> **Concurrency trung bình ≈ số yêu cầu xử lý mỗi giây × thời lượng thực thi trung bình của một yêu cầu (giây)**

Giả sử `car-service` xử lý 100 yêu cầu mỗi giây, chưa chạm giới hạn thực thi và không phát sinh retry. Nếu mỗi lượt thực thi mất trung bình 0,2 giây, concurrency trung bình vào khoảng 20. Khi API đối tác phản hồi chậm khiến thời lượng này tăng lên 2 giây, ứng dụng cần trung bình khoảng

200 lượt thực thi đồng thời để duy trì cùng thông lượng. Thời lượng trong phép tính bao gồm toàn bộ thời gian function xử lý yêu cầu: chạy code, truy cập database và chờ phản hồi từ đối tác. AWS — Tính concurrency của Lambda.

HÌNH 13.1 · SƠ ĐỒ LUỒNG



Như vậy, **ngay cả khi lượng yêu cầu không tăng, ứng dụng vẫn có thể cần nhiều lượt thực thi đồng thời hơn vì mỗi yêu cầu mất nhiều thời gian hơn để hoàn tất**. Nếu chạm trần concurrency, ứng dụng có thể không còn duy trì được tốc độ 100 yêu cầu mỗi giây. Tùy cơ chế tiếp nhận và kích hoạt, yêu cầu có thể bị từ chối do vượt giới hạn thực thi (throttling) hoặc phải chờ lâu hơn trước khi được xử lý. Công thức trên chỉ ước tính giá trị trung bình; khi đặt giới hạn, chúng ta còn cần xét biến động của tải và thời lượng thực thi.

Thời gian chờ cũng ảnh hưởng đến chi phí. Trong ví dụ này, tổng thời lượng thực thi tăng gấp mười với cùng số yêu cầu. Nếu giữ nguyên cấu hình bộ nhớ và đơn giá theo thời lượng, phần chi phí tương ứng cũng tăng theo. Tổng hóa đơn còn bao gồm phí request, dữ liệu và các dịch vụ đi kèm.

Số kết nối database không nhất thiết tăng theo cùng tỷ lệ. Mức tăng phụ thuộc vào việc function có giữ kết nối trong lúc chờ đối tác hay không, cũng như cách sử dụng connection pool và proxy. Trên EKS, việc đối tác phản hồi chậm cũng có thể khiến thread pool bị sử dụng hết hoặc công việc chờ xử lý tích tụ trong Pod. Vì vậy, với bất kỳ môi trường thực thi nào, chúng ta đều cần xác định **mỗi yêu cầu đang giữ những tài nguyên gì trong thời gian chờ và khi nào các tài nguyên đó được giải phóng**.

13.3.2 Giới hạn concurrency để bảo vệ database và dịch vụ bên ngoài

Khi chạy trên Lambda, `car-service` vẫn sử dụng RDS như phương án ở Chương 9. Số lượt thực thi đồng thời tăng có thể kéo theo nhu cầu kết nối database tăng, ngay cả khi CPU của database chưa cao. Điều này đặc biệt đáng chú ý nếu mỗi lượt thực thi giữ kết nối trong lúc chờ API đối tác

phản hồi. Vì vậy, khi đặt giới hạn concurrency, chúng ta cần kiểm tra **database còn đủ kết nối và khả năng xử lý cho các service khác hay không**.

Reserved concurrency vừa đặt trần số lượt thực thi đồng thời của một function, vừa dành riêng hạn mức đó cho function trong tổng hạn mức của tài khoản. Cơ chế này giúp giới hạn tải lên database và dịch vụ bên ngoài, tương tự việc đặt trần replica ở Chương 12. Phần được dành riêng là hạn mức thực thi; cấu hình này không khởi tạo sẵn môi trường chạy.

RDS Proxy có thể giúp tái sử dụng kết nối, nhưng khả năng xử lý transaction của database vẫn có giới hạn. Tương tự, giới hạn concurrency không tự làm giảm lượng yêu cầu gửi vào hệ thống. Nếu tốc độ tiếp nhận liên tục vượt tốc độ xử lý, công việc có thể tích tụ trong queue hoặc yêu cầu bị throttling, tùy cơ chế kích hoạt. Vì vậy, giới hạn thực thi cần đi cùng cách kiểm soát lượng công việc tiếp nhận và thời gian chờ. AWS — Lambda với RDS.

Kết nối mạng và phân quyền vẫn cần được cấu hình riêng. Để truy cập RDS trong private subnet, Lambda cần cấu hình VPC, định tuyến và các quy tắc mạng phù hợp. Nếu `car-service` còn gọi API công khai của hãng xe, chúng ta phải bảo đảm function có kết nối ra Internet; việc gắn Lambda vào VPC không tự cung cấp kết nối này. IAM role và quyền truy cập database cũng tiếp tục là hai cơ chế phân quyền riêng biệt. AWS — Kết nối mạng của Lambda.

13.3.3 Ảnh hưởng của cold start đến thời gian hoàn tất yêu cầu

Cold start xảy ra khi Lambda cần khởi tạo một môi trường thực thi mới. Điều này có thể xuất hiện sau một khoảng thời gian không có yêu cầu hoặc khi tải tăng và cần thêm môi trường để xử lý đồng thời. Để đánh giá ảnh hưởng, chúng ta cần đặt thời gian khởi tạo bên cạnh các khoảng thời gian khác trong quá trình hoàn tất công việc:

HÌNH 13.2 · SƠ ĐỒ LUỒNG



Với lời gọi HTTP đồng bộ, thời gian khởi tạo có thể làm tăng thời gian người dùng chờ phản hồi. Với cách xử lý bất đồng bộ của TravelNow, API có thể đã trả HTTP `202`, nhưng người dùng vẫn đang chờ kết quả booking và thời hạn hoàn tất vẫn tiếp tục được tính.

Vì vậy, cần đo **cold start đóng góp bao nhiêu vào tổng thời gian từ lúc tiếp nhận đến khi có kết quả nghiệp vụ**. Không nên mặc định thời gian này nhỏ hơn thời gian chờ trong queue hoặc chờ đối tác. Việc tối ưu khởi tạo chỉ cải thiện phần độ trễ tương ứng; nếu công việc chủ yếu bị chậm ở database hoặc API bên ngoài, chúng ta vẫn phải xử lý nguyên nhân tại các thành phần đó.

Khởi tạo sẵn môi trường thực thi có thể giảm độ trễ do cold start, đồng thời làm phát sinh chi phí duy trì tài nguyên sẵn sàng. Lựa chọn này cần được đánh giá bằng thời gian hoàn tất yêu cầu và tổng chi phí sau thay đổi.

13.3.4 Lựa chọn môi trường thực thi và database độc lập

TravelNow tiếp tục sử dụng RDS để duy trì transaction cục bộ và cơ chế outbox đã xây dựng. Việc chuyển `car-service` sang Lambda không buộc chúng ta thay đổi database. Nếu cách truy cập dữ liệu hoặc nhu cầu xử lý thay đổi, lựa chọn nơi lưu trữ dữ liệu cần được đánh giá riêng.

Chẳng hạn, Aurora Serverless v2 và DynamoDB có mô hình dữ liệu, cơ chế co giãn và giới hạn khác nhau. Việc lựa chọn cần dựa trên yêu cầu truy vấn, transaction và tính nhất quán đã phân tích ở Chương 9.

Dù thay đổi môi trường thực thi hay database, chúng ta vẫn phải kiểm chứng các yêu cầu nghiệp vụ:

- Thực hiện lại cùng một thao tác với cùng idempotency key không tạo thêm tác động nghiệp vụ.
- Các yêu cầu xử lý đồng thời không dẫn đến những cập nhật mâu thuẫn hoặc vi phạm quy tắc nghiệp vụ.
- Dữ liệu nghiệp vụ và bản ghi outbox tương ứng được lưu trong cùng một transaction.

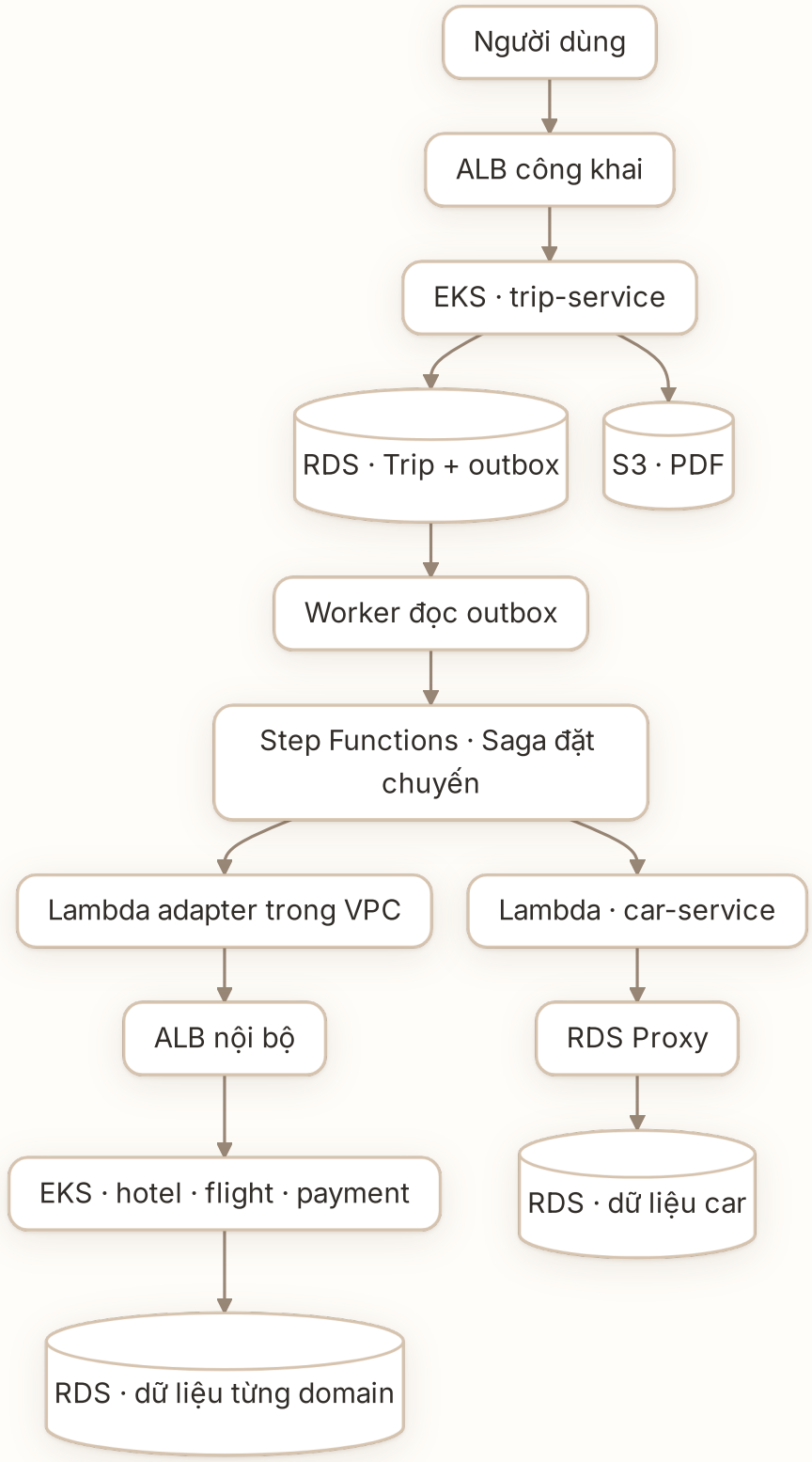
Thay đổi nền tảng chỉ được coi là phù hợp khi các yêu cầu này tiếp tục được đáp ứng và kiểm chứng bằng kiểm thử.

13.4 Kết hợp EKS và Lambda trong TravelNow

13.4.1 Triển khai riêng car-service trên Lambda

Giả sử kết quả đo cho thấy Lambda phù hợp với đặc điểm tải của `car-service`, trong khi các service còn lại đang hoạt động ổn định trên EKS. Chúng ta có thể chuyển riêng `car-service` sang Lambda, đồng thời giữ nguyên logic nghiệp vụ, quyền quản lý dữ liệu và vai trò của service trong quy trình booking:

HÌNH 13.3 · SƠ ĐỒ LUỒNG



Trong phương án này, Step Functions gọi trực tiếp các Lambda function của `car-service` để thực hiện thao tác giữ xe, hủy giữ xe hoặc tra cứu kết quả. Các function vẫn thuộc cùng một service và truy cập dữ liệu do domain car quản lý. Các service hotel, flight và payment tiếp tục được gọi qua Lambda adapter và ALB nội bộ như ở Chương 11.

Handler của mỗi function chuyển dữ liệu đầu vào tới logic nghiệp vụ, giữ nguyên định danh thao tác và trả kết quả theo contract đã thống nhất. Với cách gọi đồng bộ trong phương án này, function cần trả kết quả tương ứng với bước nghiệp vụ để workflow quyết định bước tiếp theo. Nếu bổ sung queue và để function chỉ gửi công việc vào đó, chúng ta phải thiết kế thêm cơ chế chuyển kết quả từ consumer về workflow; gửi message thành công chưa có nghĩa thao tác giữ xe đã hoàn tất. AWS — Lambda integration.

Đọc thêm — Tiếp nhận API và phân phối file

API Gateway có thể tập trung việc xác thực token, giới hạn tốc độ và định tuyến request. Service vẫn phải kiểm tra người dùng có quyền thực hiện thao tác trên Trip tương ứng hay không. CloudFront hỗ trợ phân phối nội dung từ các vị trí biên, chẳng hạn giao diện ứng dụng và tài liệu PDF. Việc bổ sung hai dịch vụ này cần xuất phát từ yêu cầu cụ thể của hệ thống.

Với PDF riêng tư của TravelNow, bucket S3 không được mở công khai. Nếu sử dụng CloudFront, Origin Access Control kiểm soát quyền truy cập từ CloudFront tới S3, còn signed URL hoặc signed cookie có thời hạn cho phép người dùng tải tài liệu. Trước khi cấp URL hoặc cookie, service phải kiểm tra quyền truy cập của người dùng đối với chuyến đi. Cơ chế cache và chữ ký chỉ hỗ trợ phân phối tài liệu theo quyền đã cấp; quyết định cấp quyền vẫn thuộc về ứng dụng. AWS — Hạn chế truy cập file CloudFront.

13.4.2 Chi phí vận hành khi kết hợp hai môi trường

Chuyển `car-service` sang Lambda có thể giảm tài nguyên nhàn rỗi hoặc công việc quản lý môi trường thực thi. Tuy nhiên, chúng ta cũng phải duy trì cách đóng gói, phát hành, cấp quyền và thu thập log, metric, trace phù hợp với cả Lambda lẫn EKS. Người xử lý sự cố cần hiểu cách theo dõi và chẩn đoán ứng dụng trong từng môi trường, đồng thời liên kết được thông tin của một booking xuyên suốt các service.

Vì vậy, **lợi ích của phương án kết hợp cần được đánh giá trên toàn hệ thống**. Phần chi phí tiết kiệm ở `car-service` cần được đặt cạnh công sức bổ sung cho công cụ, quy trình và việc vận hành hai môi trường. Nếu lợi ích không đủ bù phần việc phát sinh, duy trì một mô hình thực thi thống nhất có thể phù hợp hơn.

13.4.3 Phân biệt ranh giới nghiệp vụ và đơn vị triển khai

Chuyển `car-service` sang Lambda không có nghĩa mỗi function giữ xe, hủy giữ xe và tra cứu trở thành một microservice riêng. Các function có thể cùng thuộc một service, sử dụng dữ liệu của cùng domain và do cùng một nhóm phát triển, vận hành.

Một service cũng có thể gồm nhiều workload được triển khai riêng. Chẳng hạn, thành phần tiếp nhận HTTP request và worker xử lý nền có thể cần số lượng bản sao, tài nguyên hoặc lịch phát hành khác nhau. **Ranh giới nghiệp vụ xác định phạm vi logic và dữ liệu mà service quản lý; cách chia đơn vị triển khai còn phụ thuộc vào nhu cầu co giãn, độ trễ, kết nối và khả năng phục hồi.**

Sự khác biệt này thể hiện rõ ở hệ thống có phần mềm chạy trên cả thiết bị và Cloud. Ví dụ, hệ thống bán hàng tại quầy (POS) cần tiếp tục hoạt động khi mất mạng. Trong cùng một domain bán hàng, chúng ta vẫn phải xác định thao tác nào được phép thực hiện tại thiết bị, dữ liệu nào lấy Cloud làm nguồn chuẩn và cách đối soát khi kết nối được khôi phục. Ví dụ này cho thấy ranh giới domain chưa đủ để quyết định toàn bộ cách phân bố và triển khai ứng dụng.

Trước khi tách thêm một đơn vị triển khai, cần xác định lợi ích cụ thể và những phần việc đi kèm: duy trì contract, quản lý phiên bản, đồng bộ dữ liệu nếu cần, cấp quyền, theo dõi hoạt động và phân công xử lý sự cố. Khi chưa có nhu cầu phát hành hoặc co giãn độc lập, giữ các thao tác trong cùng một đơn vị triển khai có thể giảm công sức phối hợp mà vẫn duy trì ranh giới nghiệp vụ rõ ràng.

13.5 Tiêu chí lựa chọn và điều kiện đánh giá lại kiến trúc

Phương án triển khai cả năm service bằng Lambda ở Chương 9 vẫn là một lựa chọn phù hợp để bắt đầu. Với một nhóm nhỏ chưa có nền tảng Kubernetes, Lambda hoặc ECS trên Fargate có thể giảm công việc quản lý máy chủ và hạ tầng thực thi. Khi đã có nền tảng dùng chung đáp ứng yêu cầu, tiếp tục sử dụng nền tảng đó có thể giúp giảm công sức duy trì thêm một môi trường. **Việc hệ thống phát triển không tạo ra một lộ trình bắt buộc từ Lambda sang EKS hay ngược lại.** Lựa chọn cần dựa trên đặc điểm workload, hạ tầng hiện có và khả năng vận hành.

Với `car-service`, chúng ta có thể ghi lại cơ sở của quyết định qua bốn nội dung:

Nội dung	Câu hỏi cần trả lời
Lợi ích dự kiến	Mô hình tính phí theo mức sử dụng có giúp giảm tổng chi phí với đặc điểm tải hiện tại không?
Yêu cầu cần duy trì	Thời hạn hoàn tất, khả năng chống đặt trùng, đối soát kết quả và quyền truy cập dữ liệu có tiếp tục được đáp ứng không?
Cách kiểm chứng	Thời gian hoàn tất booking, chi phí cho cùng lượng công việc, tải lên RDS và công sức vận hành thay đổi như thế nào?
Điều kiện đánh giá lại	Tải trở nên ổn định hơn, chi phí mỗi booking tăng, giới hạn thực thi ảnh hưởng đến ứng dụng, hoặc xuất hiện nền tảng phù hợp hơn?

Cách tổ chức ứng dụng cũng ảnh hưởng đến công sức chuyển đổi sau này. Khi logic nghiệp vụ được tách khỏi phần tích hợp với nền tảng, chúng ta có thể tái sử dụng logic giữ xe và chống thao tác trùng. Handler Lambda, nguồn kích hoạt, cấu hình IAM, kết nối mạng và dashboard vẫn cần được điều chỉnh theo môi trường mới.

Nếu thay cả Step Functions bằng một cơ chế điều phối khác, chúng ta còn phải chuyển định nghĩa workflow, retry và bù trừ sang cơ chế tương ứng. Những phụ thuộc này cần được tính vào chi phí chuyển đổi; một lớp wrapper không loại bỏ được sự khác biệt về cách các nền tảng hoạt động. **Các bài kiểm thử lỗi ở Phần III cần được chạy lại để xác nhận phương án mới vẫn giữ đúng hành vi nghiệp vụ.**

13.6 Tóm tắt

Serverless giao việc cấp phát và duy trì môi trường thực thi cho nền tảng. Chúng ta vẫn chịu trách nhiệm về giới hạn xử lý, thời hạn hoàn tất và tính đúng đắn của nghiệp vụ. Với cùng thông lượng, thời lượng thực thi tăng sẽ làm tăng nhu cầu chạy đồng thời; tác động lên database và chi phí còn phụ thuộc vào cách sử dụng kết nối và cấu hình tính phí. Giới hạn concurrency giúp kiểm soát tải lên các thành phần phụ thuộc, nhưng công việc đang chờ hoặc bị gián đoạn vẫn cần cơ chế tiếp tục xử lý và đối soát.

Lựa chọn giữa tài nguyên tính phí theo mức sử dụng và tài nguyên duy trì sẵn cần dựa trên đặc điểm tải, mục tiêu phục vụ và tổng chi phí vận hành. Khi kết hợp Lambda với EKS, lợi ích của từng workload phải được đặt cạnh công sức duy trì cả hai môi trường.

Qua Phần IV, chúng ta đã tìm hiểu cách điều phối container, triển khai TravelNow trên EKS, điều chỉnh tài nguyên theo tải và đánh giá lại lựa chọn serverless. Mỗi phương án có cách phân chia trách nhiệm vận hành, cấp phát tài nguyên và phục hồi sau sự cố khác nhau. Các cơ chế nghiệp vụ ở Phần III vẫn cần được duy trì và kiểm chứng để TravelNow xử lý booking chính xác trong từng môi trường.

Phần V bắt đầu từ một tình huống cụ thể: người dùng báo “tiền đã bị trừ nhưng chưa có vé”. Dù service chạy trong Pod hay trên Lambda, chúng ta cần liên kết thông tin từ các thành phần để xác định booking đang ở đâu, điều gì đã xảy ra và cần xử lý thế nào.

PHẦN V

Quan sát và Kiểm soát

Observability & Control

Làm thế nào để biết hệ thống đang xử lý đúng yêu cầu của người dùng, kiểm soát được quyền truy cập và sử dụng tài nguyên với chi phí hợp lý? Phần cuối của eBook tìm hiểu cách liên kết thông tin từ các service để theo dõi hoạt động của hệ thống, đồng thời tổng hợp những quyết định về bảo mật và chi phí đã xuất hiện trong các phần trước.

Với TravelNow, một phản hồi HTTP thành công chưa đủ để kết luận booking đã hoàn tất. Saga có thể vẫn đang chạy, sự kiện còn nằm trong outbox hoặc kết quả thanh toán chưa được xác định. Chúng ta cần đối chiếu log, metric và trace với dữ liệu nghiệp vụ để biết booking đang ở bước nào, đã chờ bao lâu và cần ai xử lý tiếp. Từ đó, phần này mở rộng sang cách giới hạn quyền truy cập, khôi phục dữ liệu và đánh giá tổng chi phí, bao gồm cả hóa đơn Cloud lẫn công sức vận hành.

Những yêu cầu này cần được tính đến ngay từ khi hệ thống bắt đầu phục vụ người dùng. CloudWatch, X-Ray, IAM và các công cụ quản lý chi phí của AWS được giới thiệu qua các tình huống cụ thể: điều tra một booking chưa hoàn tất, cấp quyền vừa đủ cho `hotel-service` hoặc xác định chi phí xử lý một chuyến đi. Mỗi tình huống giúp làm rõ công cụ cung cấp thông tin gì, hỗ trợ kiểm soát đến đâu và phần nào ứng dụng vẫn phải tự xử lý. Khi chuyển sang công cụ khác hoặc tự vận hành giải pháp mã nguồn mở, chúng ta vẫn cần đáp ứng các yêu cầu về quan sát, bảo vệ hệ thống và phân bổ chi phí.

Chương	Nội dung	Câu hỏi dẫn đường
14	Distributed Tracing và Logging	Làm sao xác định nguyên nhân khi người dùng báo "tiền đã bị trừ nhưng chưa có vé"?
15	Bảo mật và Chi phí	Làm sao giới hạn thiệt hại khi một service bị chiếm quyền và xác định chi phí xử lý mỗi booking?

CHƯƠNG 14

Distributed Tracing và Logging

Phần V — Quan sát và Kiểm soát · *Observability & Control*

Một khách hàng phản ánh: "Tiền đã bị trừ nhưng tôi chưa thấy vé." Để xác định nguyên nhân, chúng ta cần đối chiếu dữ liệu chuyến đi, kết quả thanh toán, log của các service và tiến độ Saga. Booking có thể vẫn đang được xử lý, bước tạo PDF có thể gặp lỗi hoặc kết quả thanh toán chưa được cập nhật sau gián đoạn. Mỗi trường hợp cần một cách xử lý riêng, dù người dùng nhìn thấy cùng một biểu hiện.

Chương này xây dựng **khả năng quan sát — observability** cho TravelNow dựa trên những thông tin đã có: `tripId`, định danh thao tác, trạng thái workflow và công việc đang chờ xử lý. Chúng ta sẽ xác định dữ liệu ứng dụng cần ghi nhận, cách liên kết thông tin giữa các thành phần và cách thiết kế cảnh báo. CloudWatch, X-Ray và OpenTelemetry hỗ trợ thực hiện những việc này; mức độ hữu ích của chúng phụ thuộc vào dữ liệu mà ứng dụng cung cấp.

Các cơ chế ở Phần III giúp hệ thống tiếp tục công việc sau gián đoạn. Khả năng quan sát giúp chúng ta biết công việc đó có đang tiến triển hay cần can thiệp. Vì vậy, việc ghi log, truyền định danh và theo dõi tiến độ cần được thiết kế cùng quá trình xử lý nghiệp vụ ngay từ đầu.

14.1 Phân biệt nguyên nhân trước khi xử lý sự cố

14.1.1 Kiểm tra từng giả thuyết bằng dữ liệu

Phản ánh "tiền đã bị trừ nhưng chưa có vé" cho biết trải nghiệm của người dùng, nhưng chưa xác định được nguyên nhân. Nếu thực hiện lại toàn bộ booking ngay, chúng ta có thể lặp thao tác giữ chỗ hoặc thu tiền, như đã phân tích ở Chương 7. Trước khi can thiệp, cần tìm dữ liệu giúp phân biệt các khả năng:

Giả thuyết	Dữ liệu cần đối chiếu	Hành động sau khi xác minh
Thanh toán đã thành công nhưng phản hồi bị mất	Kết quả từ đối tác theo định danh thao tác, bản ghi payment và tiến độ Saga	Đối soát rồi tiếp tục từ bước phù hợp; không thu tiền lại bằng một key mới
Trip đã `BOOKED` nhưng PDF chưa được tạo hoặc người dùng chưa tải được	Trạng thái Trip, công việc tạo PDF, object trên S3 và quyền truy cập	Xử lý riêng bước tạo hoặc tải tài liệu; không thực hiện lại các bước giữ chỗ và thanh toán

Giả thuyết	Dữ liệu cần đối chiếu	Hành động sau khi xác minh
Booking không tiến triển sau khi một sự kiện được ghi nhận	Outbox, message, trạng thái workflow hoặc consumer và phiên bản phần mềm liên quan	Khắc phục nguyên nhân rồi tiếp tục xử lý có kiểm soát, giữ nguyên định danh chống trùng

Một booking có thể gặp nhiều vấn đề cùng lúc. Bảng trên giúp định hướng điều tra; kết luận vẫn cần dựa trên dữ liệu của trường hợp cụ thể.

14.1.2 Vai trò và giới hạn của metric, trace và log

Khi cảnh báo cho biết số booking quá hạn tăng, người trực có thể bắt đầu từ metric để xác định phạm vi, sau đó dùng trace và log để tìm nguyên nhân. Nếu khách hàng đã cung cấp `tripId`, chúng ta có thể tra cứu ngay trạng thái Trip và các thao tác liên quan. Điểm bắt đầu phụ thuộc vào thông tin đang có; các nguồn dữ liệu sẽ bổ sung cho nhau trong quá trình điều tra.

Telemetry là dữ liệu kỹ thuật mà hệ thống ghi nhận, gồm metric, trace và log. Những dữ liệu này giúp xác định thành phần nào đã xử lý, mất bao lâu và gặp lỗi gì. Tuy nhiên, quyết định thu tiền, hoàn tiền hoặc hủy chỗ cần được xác minh bằng **dữ liệu nghiệp vụ và kết quả từ đối tác**.

Telemetry có thể không đầy đủ: process dừng trước khi ghi log, hệ thống thu thập gặp lỗi, trace không được giữ lại do chính sách lấy mẫu hoặc dữ liệu đã hết thời gian lưu trữ. Vì vậy, không tìm thấy log thanh toán thành công chưa đủ để kết luận tiền chưa được thu.

Loại dữ liệu	Câu hỏi có thể trả lời	Giới hạn cần lưu ý
Metric — số booking quá hạn tăng	Bao nhiêu yêu cầu bị ảnh hưởng? Tình trạng bắt đầu từ khi nào?	Không cho biết đầy đủ từng booking đã thực hiện những thao tác nghiệp vụ nào
Trace — thời gian tập trung ở lời gọi payment	Thành phần nào chậm? Yêu cầu đi qua những bước nào đã được ghi nhận?	Có thể thiếu một số bước; timeout không chứng minh thanh toán chưa xảy ra
Log — kết quả, lần thử, phiên bản và định danh	Phiên bản nào xử lý thao tác nào, vào thời điểm nào và ghi nhận kết quả gì?	Thiếu log không đồng nghĩa thao tác chưa được thực hiện

Để chuyển từ một biểu hiện bất thường tới nguyên nhân cụ thể, chúng ta cần liên kết các dữ liệu này bằng `tripId`, định danh thao tác và phiên bản phát hành. Đó là cơ sở cho các phần tiếp theo của chương.

14.1.3 Theo dõi công việc từ lúc tiếp nhận đến khi hoàn tất

Giả sử `trip-service` đã commit Trip cùng bản ghi outbox rồi trả HTTP `202`, nhưng dispatcher chưa chuyển được công việc sang Step Functions. API không báo lỗi, các Pod vẫn hoạt động bình thường, còn workflow phía sau chưa được khởi chạy. Nếu chỉ theo dõi queue hoặc workflow, chúng ta có thể bỏ sót công việc vẫn đang nằm trong outbox.

Ý nghĩa của phản hồi HTTP cần được hiểu theo contract của API. Một hệ thống có thể trả `200` sau khi ghi nhận yêu cầu, trong khi phần xử lý phía sau vẫn chưa hoàn tất. Mã HTTP thành công không tự chứng minh kết quả nghiệp vụ cuối cùng.

TravelNow cần hỗ trợ tra cứu tiến độ theo `tripId`, dựa trên trạng thái đã lưu và các định danh liên quan. Việc này không yêu cầu gom toàn bộ dữ liệu về một database chung, nhưng phải giúp người xử lý sự cố trả lời được những câu hỏi sau:

Trạng thái công việc	Dữ liệu cần tra cứu	Nhóm hoặc người xử lý tiếp
Đã nhận Trip nhưng chưa khởi chạy Saga	Trip, outbox, thời điểm tạo, lần chuyển tiếp gần nhất và lỗi ghi nhận	Nhóm phụ trách trip kiểm tra dispatcher, lease và quyền khởi chạy workflow
Đang giữ phòng hoặc chờ thanh toán	Tiến độ Saga, định danh thao tác và trạng thái tại service liên quan	Nhóm phụ trách domain kiểm tra hoặc đối soát với đối tác
Đang bù trừ hoặc cần xử lý thủ công	Thao tác chưa hoàn tất, thời điểm bắt đầu chờ và kết quả đối soát	Người phụ trách phục hồi phối hợp các domain đến khi xác định được kết quả
Trip đã `BOOKED` nhưng chưa có PDF	Công việc tạo tài liệu, object trên S3 và quyền truy cập	Nhóm phụ trách tài liệu xử lý bước tạo hoặc tải PDF

Mỗi trạng thái chờ cần có mốc thời gian rõ ràng. Tuổi của booking được tính từ lúc tiếp nhận; thời gian chờ tại một bước được tính từ lúc bắt đầu bước đó. Thời điểm retry gần nhất phải được lưu riêng. Nếu mỗi lần retry lại đặt lại mốc bắt đầu chờ, một booking mắc kẹt nhiều giờ có thể luôn xuất hiện như công việc vừa được tiếp nhận.

Với orchestration, lịch sử execution giúp theo dõi tiến độ workflow. Với choreography, chúng ta cần liên kết trạng thái ở các service bằng `tripId` và event ID. Có thể xây dựng một mô hình đọc tổng hợp để tra cứu thuận tiện hơn, nhưng phải biết dữ liệu đã được cập nhật đến thời điểm nào. Trước khi thực hiện lại thao tác liên quan đến tiền hoặc chỗ giữ, vẫn cần xác minh tại service hoặc đối tác quản lý dữ liệu đó.

Nhóm nhận cảnh báo đầu tiên cần tiếp tục điều phối cho đến khi nhóm xử lý tiếp xác nhận tiếp nhận. Việc bàn giao phải rõ ràng để tránh tình trạng từng service đều được báo là "bình thường", trong khi booking của khách hàng vẫn chưa được giải quyết.

14.2 Log có cấu trúc và quy ước dữ liệu

14.2.1 Thống nhất cấu trúc log giữa các service

Một dòng log như `Payment declined for trip abc` dễ đọc, nhưng khó dùng để tổng hợp số liệu ổn định. Muốn đếm số lần thanh toán bị từ chối theo giờ, chúng ta phải phân tích nội dung chuỗi; truy vấn có thể không còn đúng khi câu chữ thay đổi.

Cách phù hợp hơn là ghi **log có cấu trúc**, chẳng hạn mỗi dòng là một đối tượng JSON:

```
{
  "timestamp": "2026-06-22T14:32:05.123Z",
  "level": "INFO",
  "service": "payment-service",
  "version": "a1b2c3d",
```

```
  "message": "payment declined",
  "tripId": "abc-123",
  "operationId": "charge:abc-123",
  "outcome": "BUSINESS_REJECTED",
  "reason": "card_declined",
  "attempt": 1,
  "durationMs": 840
}
```

JSON cho phép truy vấn theo từng trường. Để đối chiếu log giữa các service, TravelNow còn cần thống nhất tên trường, ý nghĩa và cách ghi giá trị:

Trường	Thông tin cung cấp	Mục đích
`service`, `version`	Service và phiên bản đã xử lý thao tác	Đối chiếu sự cố với các lần phát hành
`tripId`	Booking mà thao tác thuộc về	Liên kết thông tin của cùng một chuyến đi qua các service
`operationId`	Thao tác nghiệp vụ cụ thể	Nhận diện các lần thực thi lại của cùng một thao tác
`attempt`	Số thứ tự lần thử theo quy ước đã thống nhất	Phân biệt retry với công việc mới
`outcome`	Kết quả xử lý	Phân biệt thành công, từ chối nghiệp vụ và lỗi kỹ thuật

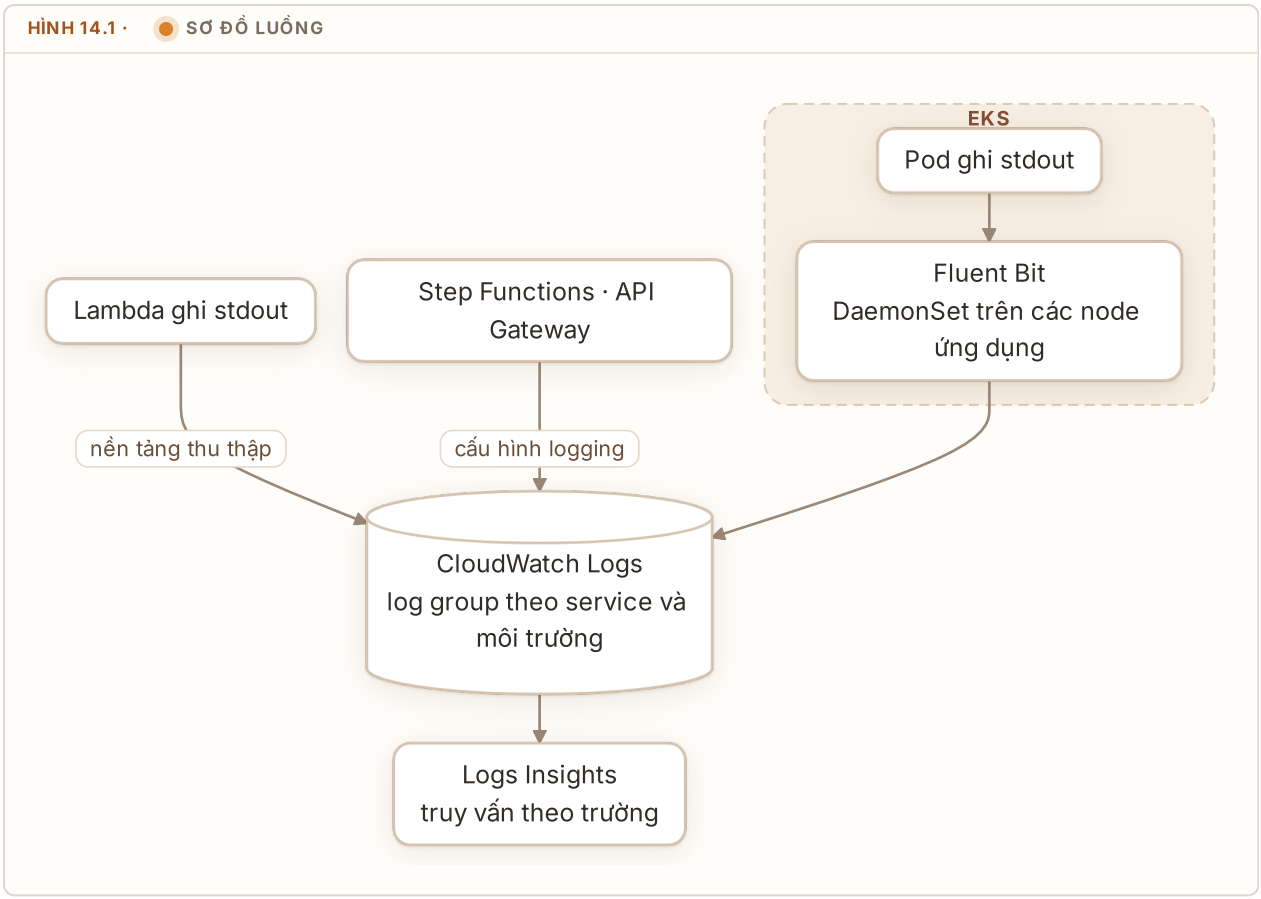
Trường `outcome` đặc biệt quan trọng khi thiết kế metric và cảnh báo. Thẻ bị từ chối là một kết quả nghiệp vụ đã xác định; mất kết nối với cổng thanh toán có thể khiến kết quả chưa rõ. Nếu gộp cả hai vào cùng một loại lỗi, chúng ta sẽ khó đánh giá tình trạng hệ thống và chọn cách xử lý.

Kết quả cần được phân loại tại nơi thực hiện thao tác, theo quy ước dùng chung giữa các service. Khi đó, truy vấn và cảnh báo mới có thể giữ cùng ý nghĩa.

Với Spring Boot, chúng ta có thể sử dụng khả năng JSON logging của phiên bản đang dùng hoặc encoder cho Logback. Cấu hình logger quyết định định dạng đầu ra; code ứng dụng hoặc interceptor vẫn phải cung cấp các trường nghiệp vụ như `tripId`, `operationId` và kết quả xử lý.

14.2.2 Thu thập và truy vấn log tập trung

Log chỉ nằm trong môi trường thực thi có thể mất khi Pod hoặc node bị thay thế. Nguyên tắc từ Chương 3 vẫn được giữ: **ứng dụng ghi log ra stdout, hạ tầng chịu trách nhiệm thu thập và chuyển tới nơi lưu trữ tập trung**.



Hệ thống thu thập log cũng cần được theo dõi. Lỗi chuyển dữ liệu hoặc độ trễ thu thập có thể khiến người trực nhìn thấy thông tin không đầy đủ. Log group, thời gian lưu giữ và quyền truy vấn cần được cấu hình theo từng môi trường và nhu cầu sử dụng.

Khi cấu trúc log đã thống nhất, chúng ta có thể tìm các thao tác của chuyến `abc-123` bằng truy vấn:

```
fields @timestamp, service, version, operationId, outcome, reason
| filter tripId = "abc-123"
| sort @timestamp asc
```

14.2.3 Gắn định danh nghiệp vụ vào log

Truy vấn theo booking chỉ có giá trị khi **các log nghiệp vụ quan trọng đều chứa** `tripId`. Trong Java, một cách phổ biến là đặt định danh vào **MDC — Mapped Diagnostic Context** khi tiếp nhận HTTP request hoặc message, rồi cấu hình logger đưa các trường này vào mỗi dòng log.

MDC thường gắn với thread. Khi công việc được chuyển sang executor, luồng reactive hoặc callback bất đồng bộ, chúng ta cần bảo đảm context được truyền tiếp. Nếu không, các dòng log phía sau có thể mất định danh dù vẫn thuộc cùng một thao tác.

Context cũng phải được xóa sau khi xử lý xong. Nếu thread được tái sử dụng mà vẫn giữ `tripId` cũ, log của yêu cầu tiếp theo có thể bị gắn nhầm vào booking trước. Đây là lỗi dễ gây hiểu sai khi điều tra vì từng dòng log vẫn có cấu trúc hợp lệ.

14.3 Distributed tracing: liên kết các bước xử lý giữa nhiều thành phần

14.3.1 Trace, span và cách truyền ngữ cảnh

Log ghi lại từng sự kiện. Distributed tracing liên kết những bước đã được ghi nhận để chúng ta theo dõi một lượt xử lý qua nhiều thành phần. Ba khái niệm chính gồm:

- **Trace:** tập hợp các span mô tả một lượt xử lý, được liên kết bằng `traceId`.
- **Span:** một bước xử lý có tên, thời điểm bắt đầu và thời lượng, chẳng hạn gọi HTTP, thực hiện câu SQL hoặc gửi message. Quan hệ cha–con thể hiện cấu trúc gọi; span link biểu diễn những mối liên hệ khác giữa các bước.
- **Context propagation:** truyền ngữ cảnh trace giữa các thành phần. Với HTTP, header W3C `traceparent` mang thông tin này. Với messaging, producer và consumer cần thống nhất cách ghi context vào message và đọc lại khi xử lý.

Việc ghi span và truyền context cần được thực hiện xuyên suốt những thành phần muốn quan sát. Một bước không được ghi nhận hoặc làm mất context có thể khiến trace bị thiếu hoặc tách thành nhiều phần.

14.3.2 Phân biệt các định danh trong một booking

TravelNow sử dụng nhiều loại định danh vì mỗi loại phục vụ một mục đích riêng:

Định danh	Phạm vi	Ví dụ	Mục đích
`tripId`	Toàn bộ vòng đời booking, có thể kéo dài nhiều phút hoặc nhiều ngày	`abc-123`	Liên kết mọi thông tin thuộc cùng một chuyến đi
`operationId`	Một thao tác nghiệp vụ cụ thể	`charge:abc-123`	Chống thực hiện trùng và đối soát với đối tác
`traceId`	Một lượt xử lý kỹ thuật	`4bf92f35…`	Theo dõi thời gian và các bước đã được ghi nhận
Execution ID	Một lần chạy workflow	ARN execution của Step Functions	Tra cứu tiến độ và lịch sử Saga

Một booking có thể liên quan đến nhiều trace. Request ban đầu trả `202` có thể thuộc một trace, còn workflow chạy sau đó thuộc trace khác. Các lần retry cũng có thể tạo trace riêng.

`tripId` liên kết thông tin nghiệp vụ xuyên suốt các trace; `traceId` liên kết các span của một lượt xử lý. Hai định danh bổ sung cho nhau. Chỉ truyền `tripId` chưa đủ để công cụ tracing dựng quan hệ giữa các span.

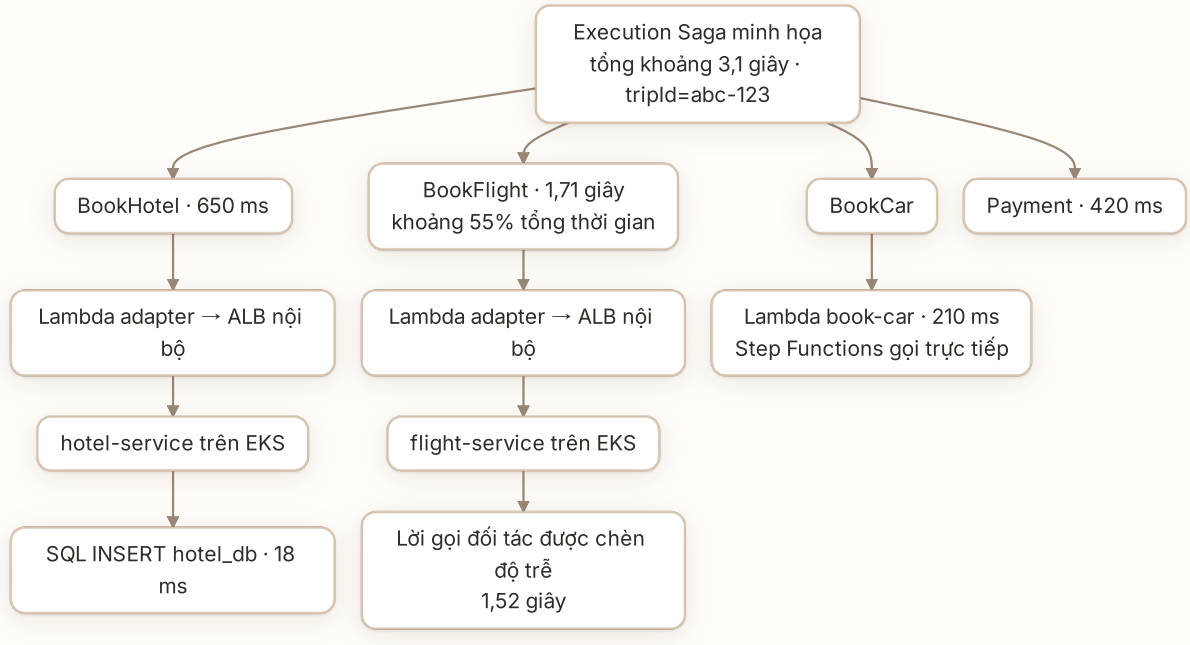
14.3.3 Quan sát các bước thực thi của Saga

OpenTelemetry (OTel) cung cấp các công cụ để ghi nhận và chuyển telemetry theo quy ước chung. Với Java, agent có thể tự động ghi nhiều lời gọi HTTP, SQL và AWS SDK. Những bước nghiệp vụ riêng vẫn cần được kiểm tra và bổ sung instrumentation khi cần.

Trên AWS, ADOT Collector có thể tiếp nhận, xử lý và chuyển telemetry tới X-Ray hoặc nơi lưu trữ khác. Lambda, API Gateway và Step Functions có cấu hình tracing riêng cần được phối hợp với ứng dụng. Việc tách phần ghi nhận dữ liệu khỏi nơi lưu trữ giúp tái sử dụng một phần instrumentation khi đổi công cụ; exporter, cách truy vấn và cảnh báo vẫn cần được kiểm tra lại. OpenTelemetry — Collector.

Với phương án kết hợp ở Chương 13, một sơ đồ quan sát execution Saga có thể được biểu diễn như sau:

HÌNH 14.2 · SƠ ĐỒ LUỒNG



Các thời lượng trên chỉ dùng để minh họa. Sơ đồ thể hiện những thành phần và mối liên hệ cần quan sát; cấu trúc span thực tế phụ thuộc vào instrumentation, cấu hình tracing và cách truyền context qua từng tích hợp.

Thông tin đáng chú ý là bước flight chiếm hơn một nửa tổng thời gian. Đây là nơi cần tiếp tục phân tích: thời gian nằm trong code, database hay lời gọi đối tác? Như đã trình bày ở Chương 12, xác định đúng thành phần gây chậm cần đi trước quyết định tăng tài nguyên.

14.3.4 Truyền ngữ cảnh qua giao tiếp bất đồng bộ

Với HTTP, lời gọi từ service này tới service khác thường có thể biểu diễn bằng quan hệ span cha–con. Với queue, consumer có thể xử lý message sau vài phút hoặc nhận một batch gồm message từ nhiều producer. Quan hệ giữa các bước vì thế cần được lựa chọn theo cách xử lý thực tế.

Consumer có thể tiếp tục trace nhận được, hoặc tạo trace mới và dùng **span link** để liên kết với context của producer. Sơ đồ sau minh họa hai cách giao tiếp HTTP và queue; đây là ví dụ về truyền context, không phải quy trình booking dùng Step Functions ở Mục 14.3.3:

Xem Hình 14.3 (sơ đồ khổ ngang) ở trang 259.

Khi xử lý một batch, các message có thể thuộc nhiều trace khác nhau. Instrumentation cần giữ được mối liên hệ với từng message; không nên mặc định cả batch đều bắt nguồn từ một trace. OpenTelemetry — Messaging spans.

Đọc thêm — Lấy mẫu trace, số chuỗi metric và dữ liệu nhạy cảm

Việc thu thập telemetry cần tính đến chi phí, khả năng xử lý và quyền truy cập dữ liệu:

- **Lấy mẫu trace — sampling:** lưu toàn bộ trace có thể phù hợp ở quy mô nhỏ, nhưng chi phí tăng theo tải. Head sampling quyết định giữ trace từ sớm. Tail sampling quyết định sau khi nhận các span, nhờ đó có thể ưu tiên trace lỗi hoặc chậm, nhưng cần tài nguyên và cơ chế tập hợp span theo trace tại collector. Chính sách cần cân bằng khả năng điều tra với chi phí thu thập và lưu trữ. OpenTelemetry — Sampling.
- **Số chuỗi metric — cardinality:** các nhãn như `service`, `operation` và `outcome` có thể được giới hạn bằng tập giá trị đã thống nhất. Ngược lại, `tripId`, `userId` hoặc URL chứa ID tạo ra rất nhiều tổ hợp nhãn. Những định danh này nên được ghi trong log và trace để tra cứu, tránh dùng làm nhãn metric thông thường.
- **Dữ liệu nhạy cảm:** không ghi JWT, mật khẩu, số thẻ hoặc toàn bộ request body vào log. Chỉ thu thập thông tin cá nhân khi có mục đích rõ ràng và biện pháp bảo vệ phù hợp. Cơ chế che dữ liệu, thời gian lưu giữ và quyền đọc telemetry cần được quản lý thống nhất.

Tail sampling chỉ lựa chọn trong số span đã đến collector; nó không khôi phục được dữ liệu đã bị loại từ phía ứng dụng. Vì vậy, các chỉ số như số booking quá hạn hoặc kết quả đối soát cần được tính từ nguồn dữ liệu đầy đủ trong phạm vi đánh giá, thay vì suy ra từ những trace được giữ lại.

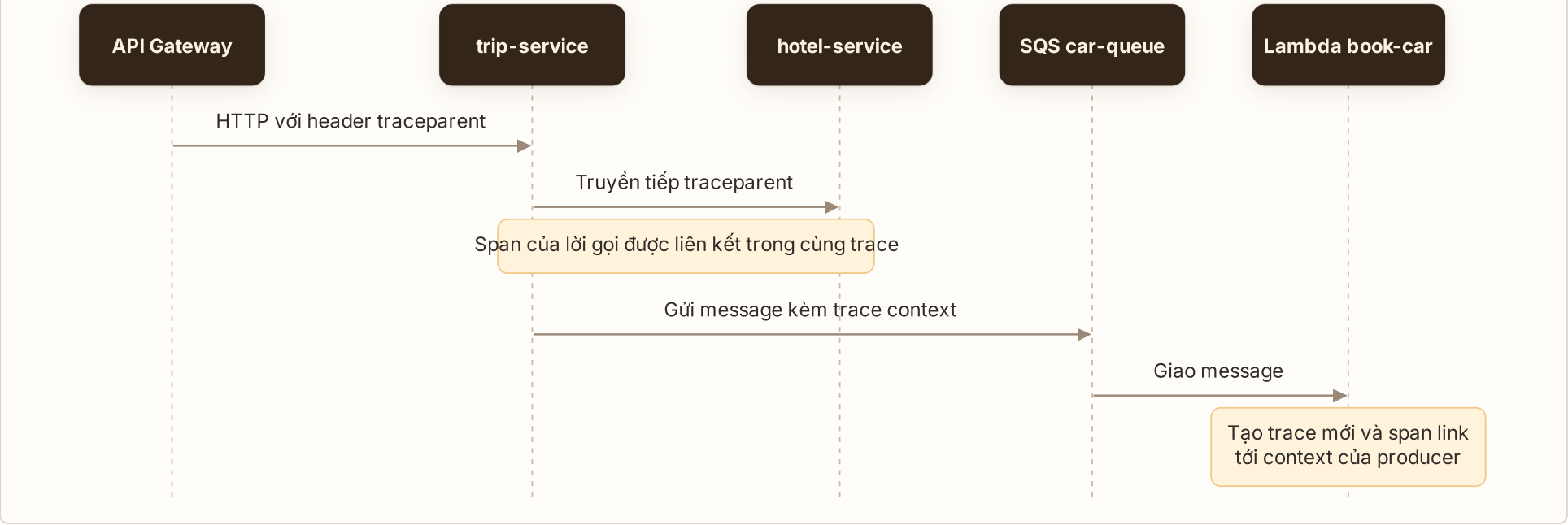
14.4 Thiết kế cảnh báo theo tác động và mức độ khẩn cấp

14.4.1 Xác định hành động cho từng loại cảnh báo

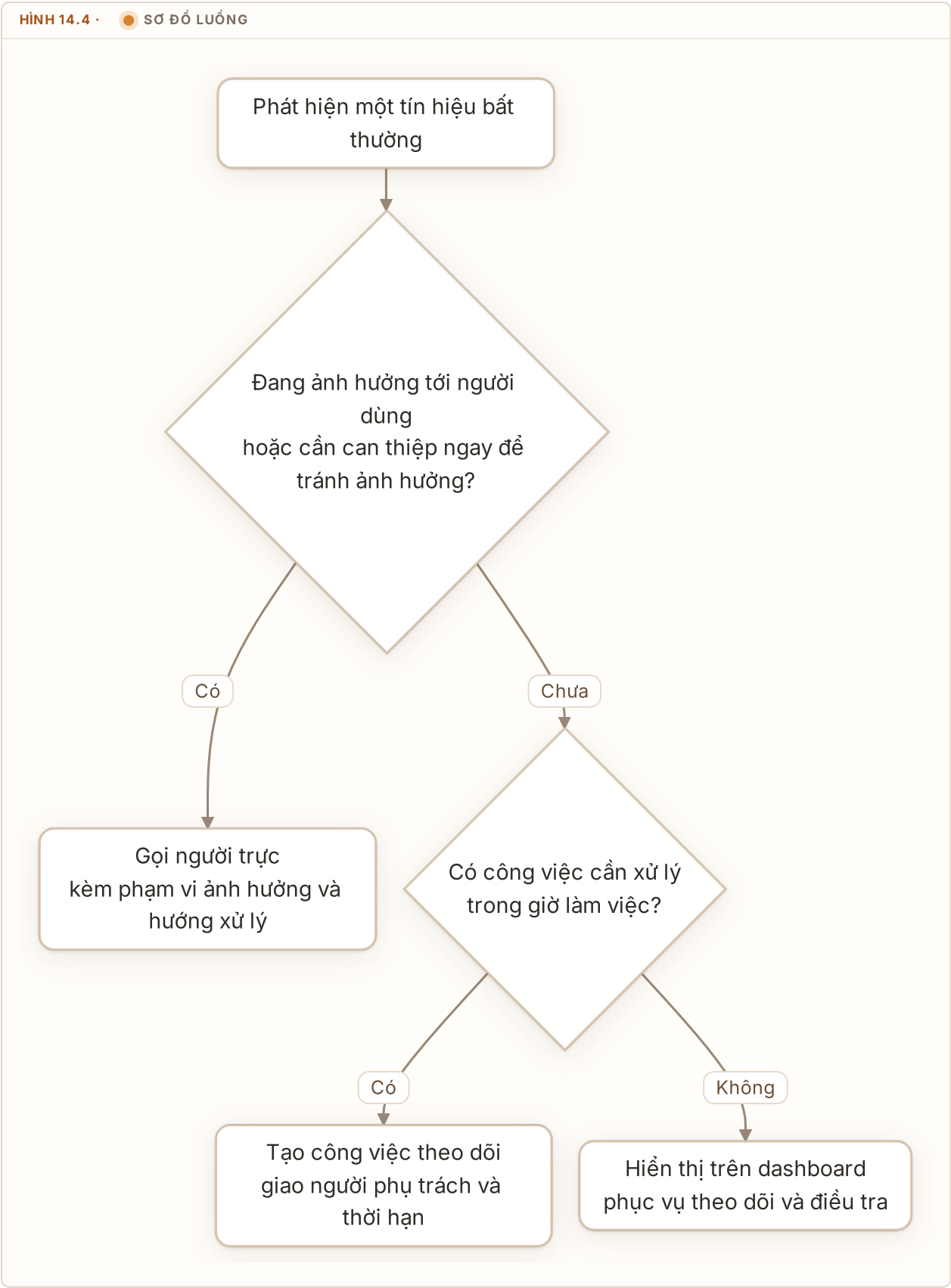
Giả sử CPU của `flight-service` tăng cao nhưng các booking vẫn hoàn tất trong thời gian mục tiêu. Cùng lúc, hotel-worker sử dụng ít CPU nhưng công việc chờ xử lý ngày càng lâu. Nếu chỉ cảnh báo theo CPU, chúng ta có thể tập trung vào thành phần chưa cần can thiệp và bỏ sót những booking đang quá hạn.

Mức độ khẩn cấp cần dựa trên ảnh hưởng tới người dùng và thời gian còn lại để xử lý:

HÌNH 14.3 · SƠ ĐỒ TUẦN TỰ



HÌNH 14.4 · SƠ ĐỒ LUỒNG



Cảnh báo cần giúp người nhận biết vì sao phải hành động và nên bắt đầu từ đâu. CPU, số Pod và số kết nối là dữ liệu hữu ích để điều tra. Việc gọi người trực cần gắn với nguy cơ suy giảm phục vụ, mức độ ảnh hưởng và khả năng trì hoãn xử lý.

Một cảnh báo lặp lại nhưng không có hành động rõ ràng dễ bị bỏ qua. Theo thời gian, điều này làm giảm hiệu quả của cả những cảnh báo nghiêm trọng.

TravelNow có thể thu thập metric từ Micrometer của Spring Boot, Container Insights và các metric dịch vụ như Lambda, SQS, Step Functions, API Gateway. Việc lựa chọn cần xuất phát từ câu hỏi vận hành cụ thể, đồng thời tính đến chi phí thu thập, lưu trữ và truy vấn.

14.4.2 Đo kết quả nghiệp vụ sau phản hồi `202`

SLI — Service Level Indicator là chỉ số đo chất lượng phục vụ. **SLO — Service Level Objective** là mục tiêu đặt cho chỉ số đó trong một khoảng thời gian. Chẳng hạn, với SLO 99,9% yêu cầu hợp lệ được xử lý đạt yêu cầu, tỷ lệ cho phép không đạt là 0,1%. Phần này được gọi là **error budget**.

Cảnh báo theo **burn rate**, tức tốc độ tiêu hao error budget, giúp phát hiện tình trạng không đạt mục tiêu đang tăng nhanh đến mức cần can thiệp.

Với TravelNow, tỷ lệ API trả `202` chỉ phản ánh khả năng tiếp nhận yêu cầu. Một SLO minh họa cho kết quả nghiệp vụ là:

> **Trong cửa sổ đánh giá đã chọn, 99% yêu cầu hợp lệ phải có kết quả nghiệp vụ xác định trong vòng 30 giây kể từ lúc tiếp nhận.**

Kết quả hợp lệ có thể là Trip chuyển sang `BOOKED`, hoặc yêu cầu bị từ chối và các bước bù trừ cần thiết đã hoàn tất. Tỷ lệ đặt chuyến thành công là một chỉ số sản phẩm riêng. Các Trip vẫn ở trạng thái `PENDING`, `COMPENSATING` hoặc `MANUAL_REVIEW` khi hết thời hạn phải được tính là không đạt SLO này.

Để đo nhất quán, chúng ta cần thống nhất tập yêu cầu được tính, mốc bắt đầu, trạng thái kết thúc hợp lệ và cửa sổ đánh giá.

Mục tiêu này cũng giúp kiểm tra lại thiết kế. Nếu relay chỉ quét outbox mỗi phút như mô hình cơ sở ở Chương 9, một yêu cầu có thể chờ quá 30 giây trước khi Saga được khởi chạy. Muốn đạt SLO trên, chúng ta cần đo thời gian tại từng bước và điều chỉnh cơ chế chuyển tiếp công việc.

Khi tính SLI, phải giữ cả những yêu cầu chưa hoàn tất trong tập đánh giá. Ví dụ, lấy toàn bộ yêu cầu hợp lệ được tiếp nhận trong một giờ và chờ thêm 30 giây để yêu cầu cuối cùng có đủ thời gian quan sát:

- **Tử số:** số yêu cầu có kết quả hợp lệ trong vòng 30 giây kể từ lúc tiếp nhận.
- **Mẫu số:** toàn bộ yêu cầu hợp lệ được tiếp nhận trong giờ đó.

Chỉ tính trên những booking đã kết thúc sẽ bỏ sót các booking đang mắc kẹt. Một booking hoàn tất muộn vẫn giải quyết được nhu cầu của khách hàng, nhưng không làm mất đi lần vi phạm thời hạn trước đó.

Dashboard, alarm và cấu hình kênh thông báo có thể được quản lý bằng Terraform để hỗ trợ review và triển khai nhất quán giữa các môi trường.

14.4.3 Theo dõi tính đúng đắn của nghiệp vụ

Hạ tầng có thể hoạt động bình thường trong khi một số booking chưa được xử lý đúng. **Semantic monitoring** bổ sung các kiểm tra dựa trên trạng thái và quy tắc nghiệp vụ, chẳng hạn:

- Saga chưa hoàn tất khi quá thời hạn, hoặc một bước bù trừ chờ lâu hơn mức cho phép.
- Thanh toán đã thành công nhưng trạng thái chuyến đi chưa được cập nhật.
- Khoản tiền cần hoàn cho Trip ở trạng thái `CANCELLING` vẫn chưa được xử lý.
- Bản ghi outbox chưa được chuyển tiếp hoặc PDF chưa được tạo trong thời hạn.
- Dữ liệu payment và trip có chênh lệch sau đối soát.

Mỗi kiểm tra cần quy định khoảng thời gian chờ hợp lệ, người phụ trách và cách xử lý. Điều này giúp phân biệt trạng thái trung gian bình thường của hệ thống bất đồng bộ với một vấn đề cần can thiệp.

Metric của queue chỉ phản ánh một phần quá trình. Ví dụ, `ApproximateAgeOfOldestMessage` của SQS không thể hiện toàn bộ tuổi của booking: công việc có thể đã chờ trong outbox trước khi được gửi vào queue, hoặc đã chuyển sang DLQ. Vì vậy, cần đối chiếu metric queue với tuổi công việc nghiệp vụ và trạng thái DLQ. Queue chính không còn message chưa đủ để kết luận mọi công việc đã hoàn tất. AWS — SQS metrics.

Một cách kiểm tra bổ sung là chạy **booking giả lập định kỳ — synthetic booking**. Kịch bản sử dụng tenant và nhà cung cấp giả được nhận diện rõ, không phát sinh giao dịch tiền thật và có cơ chế dọn dữ liệu. Bài kiểm tra này xác nhận một tình huống mẫu vẫn hoạt động, còn các kiểm tra trạng thái nghiệp vụ giúp tìm những booking thực tế đang chậm hoặc không nhất quán.

14.5 Điều tra và phục hồi sau sự cố

Để thấy cách các nguồn dữ liệu phối hợp với nhau, hãy xét một buổi diễn tập trên **phương án choreography** ở Chương 9. Trong phương án này, Lambda consumer của hotel nhận sự kiện `TripRequested` qua SQS event source mapping.

Giả sử `trip-service` được phát hành lúc **13:50**. Quá trình phát hiện và xử lý diễn ra như sau.

14:02 — Phát hiện công việc bị gián đoạn. Cảnh báo cho biết DLQ của `hotel-queue` có 12 message và tiếp tục tăng. Chưa có phản ánh từ khách hàng, nhưng tín hiệu này cho thấy một số công việc phía sau API chưa được xử lý thành công.

14:04 — Xác định thao tác gặp lỗi. Một message trong DLQ chứa sự kiện `TripRequested` cùng `tripId`. Truy vấn log theo định danh này cho thấy `hotel-service` lỗi khi đọc trường `checkIn`. Message thực tế chỉ có trường `checkInDate`.

14:07 — Xác minh nguyên nhân. Trường `version` trong log xác định phiên bản consumer đã xử lý message. Phiên bản producer được tra từ metadata của sự kiện hoặc lịch sử phát hành `trip-service`. Đối chiếu với bản phát hành lúc 13:50 và tái hiện bằng message lỗi xác nhận rằng producer đã đổi tên trường trước khi consumer hỗ trợ định dạng mới. Đây là vấn đề tương thích đã phân tích ở Chương 10.

14:10 — Ngăn lỗi tiếp diễn và phục hồi công việc. Nhóm xử lý xác định tập `tripId` bị ảnh hưởng và dừng phát sự kiện theo schema không tương thích. Trong diễn tập này, phiên bản producer cũ đã được kiểm chứng là vẫn hoạt động với dữ liệu và cấu hình hiện tại nên có thể rollback. Nếu điều kiện đó không còn đúng, cần tạm dừng việc phát sự kiện hoặc phát hành bản sửa phù hợp.

Consumer được cập nhật để đọc cả hai định dạng. Trước khi redrive DLQ, chúng ta kiểm tra thời hạn, trạng thái Trip và các thao tác đã thực hiện. Message được đưa trở lại xử lý với tốc độ có kiểm soát; những Trip đã hủy hoặc chưa rõ kết quả được chuyển sang quy trình xử lý tương ứng.

14:30 — Xác nhận kết quả và bổ sung biện pháp phòng ngừa. Từng booking bị ảnh hưởng được đối chiếu với dữ liệu tại các service: đã hoàn tất, đã bù trừ hoặc còn cần xử lý. Những trường hợp chưa giải quyết xong phải có người tiếp nhận và bước tiếp theo rõ ràng.

Sau đó, chúng ta bổ sung contract test cho thay đổi tên trường, đồng thời kiểm tra vì sao pipeline không chặn bản phát hành: thiếu test, test chưa chạy hay kết quả test không được dùng làm điều kiện phát hành. Việc này nối lại các yêu cầu về bàn giao và kiểm chứng ở Chương 11.

Cuộc điều tra sử dụng cảnh báo để phát hiện, log có định danh để khoanh vùng, thông tin phiên bản để đối chiếu thay đổi và dữ liệu nghiệp vụ để xác nhận kết quả phục hồi. CloudWatch, X-Ray và metric SQS hỗ trợ từng phần, nhưng kết luận về lỗi contract đến từ việc kiểm tra message cùng phiên bản producer, consumer. **Thay công cụ không làm mất nhu cầu liên kết những thông tin này.**

14.6 Tóm tắt

Metric giúp nhận biết bất thường và phạm vi ảnh hưởng. Trace giúp xác định những bước xử lý chậm hoặc lỗi. Log có cấu trúc cung cấp thông tin về từng thao tác, phiên bản và kết quả ghi nhận. Để sử dụng chúng cùng nhau, TravelNow cần quy ước log thống nhất, định danh có vai trò rõ ràng và cơ chế truyền context qua các thành phần. Những quyết định liên quan đến tiền hoặc chỗ giữ vẫn phải được xác minh bằng dữ liệu nghiệp vụ và kết quả từ đối tác.

Khả năng quan sát cần cho biết công việc đang ở bước nào, đã chờ bao lâu và ai xử lý tiếp. SLO phải bao phủ cả phần xử lý sau phản hồi `202`, bao gồm những yêu cầu chưa hoàn tất. Các kiểm tra dựa trên quy tắc nghiệp vụ giúp phát hiện sai lệch ngay cả khi hạ tầng chưa báo lỗi. Khi kết thúc xử lý sự cố, chúng ta cần xác nhận kết quả của những booking bị ảnh hưởng và bàn giao rõ các trường hợp còn lại.

Telemetry cũng cần được quản lý về dữ liệu nhạy cảm, quyền truy cập, thời gian lưu giữ và chi phí. Chương 15 mở rộng các yêu cầu này ra toàn hệ thống, từ bảo vệ dữ liệu đến đánh giá chi phí của từng quyết định kiến trúc.

CHƯƠNG 15

Bảo mật và Chi phí

Phần V — Quan sát và Kiểm soát · *Observability & Control*

Giả sử một lỗ hổng thư viện cho phép kẻ tấn công thực thi code trong `hotel-service`. Dù chạy trong private subnet, service vẫn có kết nối mạng và quyền truy cập dữ liệu để thực hiện nghiệp vụ. Khi những quyền này bị lợi dụng, hệ thống có thể giới hạn thiệt hại đến đâu? Nếu cần khôi phục database, làm sao tránh thực hiện lại một giao dịch thanh toán đã xảy ra? Trong quá trình xử lý sự cố, làm sao kiểm soát lượng tài nguyên tăng thêm do retry và autoscaling?

Chương này bắt đầu từ tình huống `hotel-service` bị chiếm quyền, sau đó xem xét các lớp bảo vệ, khả năng khôi phục dữ liệu và chi phí vận hành TravelNow. Các dịch vụ AWS cung cấp những cơ chế cụ thể để thực hiện. Chúng ta cần xác định **mỗi chủ thể được phép làm gì, dữ liệu sau phục hồi có còn đúng với thực tế nghiệp vụ hay không và tài nguyên được sử dụng cho những công việc nào**. Những yêu cầu này vẫn giữ nguyên khi chuyển sang nền tảng khác hoặc tự vận hành bằng phần mềm mã nguồn mở.

> **Cách đọc chương**
>
> Mục 15.1–15.5 tập trung vào quyền truy cập, bảo vệ dữ liệu và khả năng khôi phục. Mục 15.6–15.7 phân tích chi phí của các quyết định kiến trúc. Ở lượt đọc đầu, hãy tập trung vào ba câu hỏi: ai có quyền gì, dữ liệu được khôi phục thế nào và những khoản chi phí nào thay đổi theo workload. Các chi tiết cấu hình có thể tra cứu lại khi triển khai.

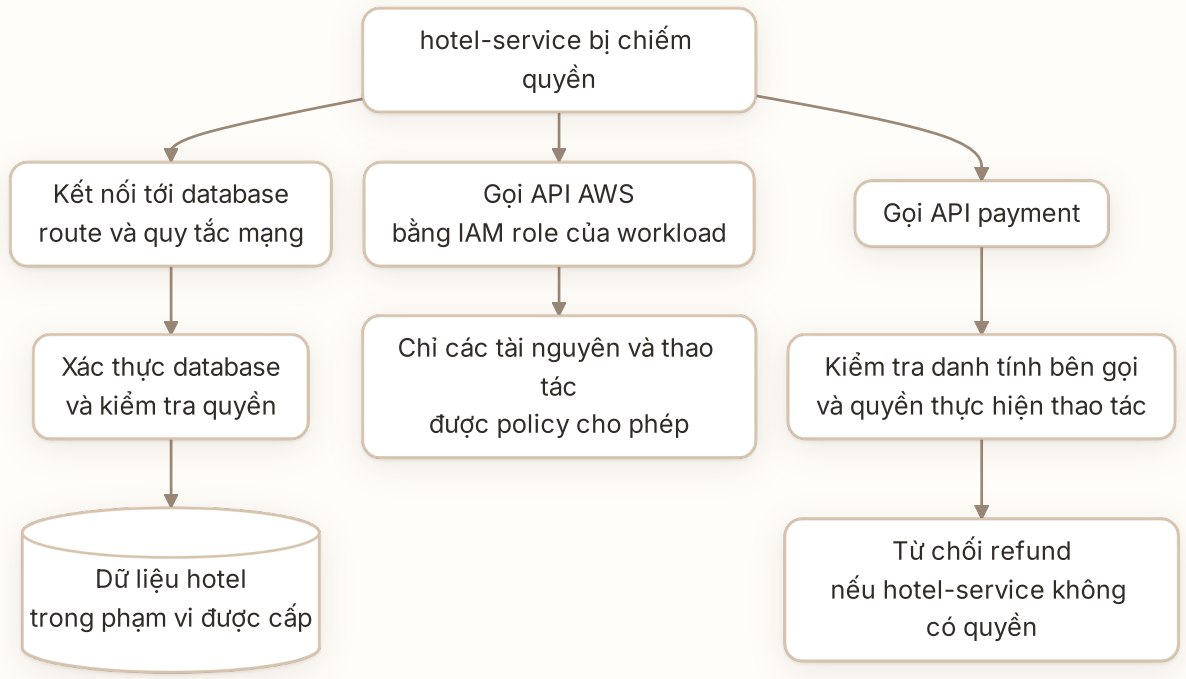
15.1 Giới hạn thiệt hại khi hotel-service bị chiếm quyền

15.1.1 Xác định phạm vi truy cập và quyền thao tác của workload

Khi code độc hại chạy trong Pod, kẻ tấn công có thể sử dụng những credential mà process đọc được và thực hiện các thao tác nằm trong phạm vi quyền đã cấp. Private subnet hạn chế khả năng truy cập trực tiếp từ Internet, nhưng không vô hiệu hóa quyền của `hotel-service`.

Vì vậy, bên cạnh việc ngăn chặn xâm nhập, chúng ta cần kiểm tra **nếu workload bị chiếm quyền, kẻ tấn công có thể truy cập dữ liệu nào và thực hiện những thao tác gì**:

HÌNH 15.1 · SƠ ĐỒ LUỒNG



Sơ đồ thể hiện các cơ chế cần cấu hình và kiểm chứng. Việc sử dụng EKS không tự tạo ra đầy đủ những giới hạn này.

Hành vi cần ngăn chặn	Cơ chế kiểm soát	Giới hạn cần hiểu
Đọc trực tiếp dữ liệu payment	Quy tắc mạng, tài khoản và quyền database riêng	Nếu các database logic dùng chung endpoint RDS, quy tắc mạng không phân biệt được từng database; quyền bên trong database mới quyết định dữ liệu được truy cập
Dùng API AWS để đọc file hoặc gửi message trái phép	IAM role riêng, giới hạn theo action và resource	Credential bị lộ vẫn có thể bị sử dụng trong phạm vi quyền đã cấp
Gọi API refund của payment	Xác thực workload và kiểm tra quyền tại service nhận	Cùng private subnet không đồng nghĩa có quyền hoàn tiền
Thay đổi code được triển khai ở lần phát hành sau	Kiểm soát pipeline, nguồn gốc artifact và chính sách triển khai	Thay Pod hoặc vá ứng dụng chưa đủ nếu quy trình phát hành cũng đã bị xâm nhập

15.1.2 Kiểm chứng tính độc lập của các lớp bảo vệ

Defense in depth, hay bảo vệ theo nhiều lớp, giúp duy trì một phần khả năng bảo vệ khi một cơ chế bị vượt qua. Hiệu quả phụ thuộc vào việc các lớp có cùng bị vô hiệu hóa bởi một điểm yếu hay không.

Chẳng hạn, tách dữ liệu thành năm database logic nhưng dùng chung một tài khoản có toàn quyền vẫn cho phép credential đó truy cập cả năm database. Tương tự, nhiều bước kiểm tra cùng dựa vào một IAM role quá rộng có thể cùng mất tác dụng khi credential của role bị lộ.

Vì vậy, chúng ta cần kiểm chứng từng giả định: khi credential của `hotel-service` bị sử dụng trái phép, dữ liệu payment có còn được bảo vệ không? Service đó có gửi được message tới queue ngoài phạm vi nghiệp vụ không? Nó có gọi được API hoàn tiền không?

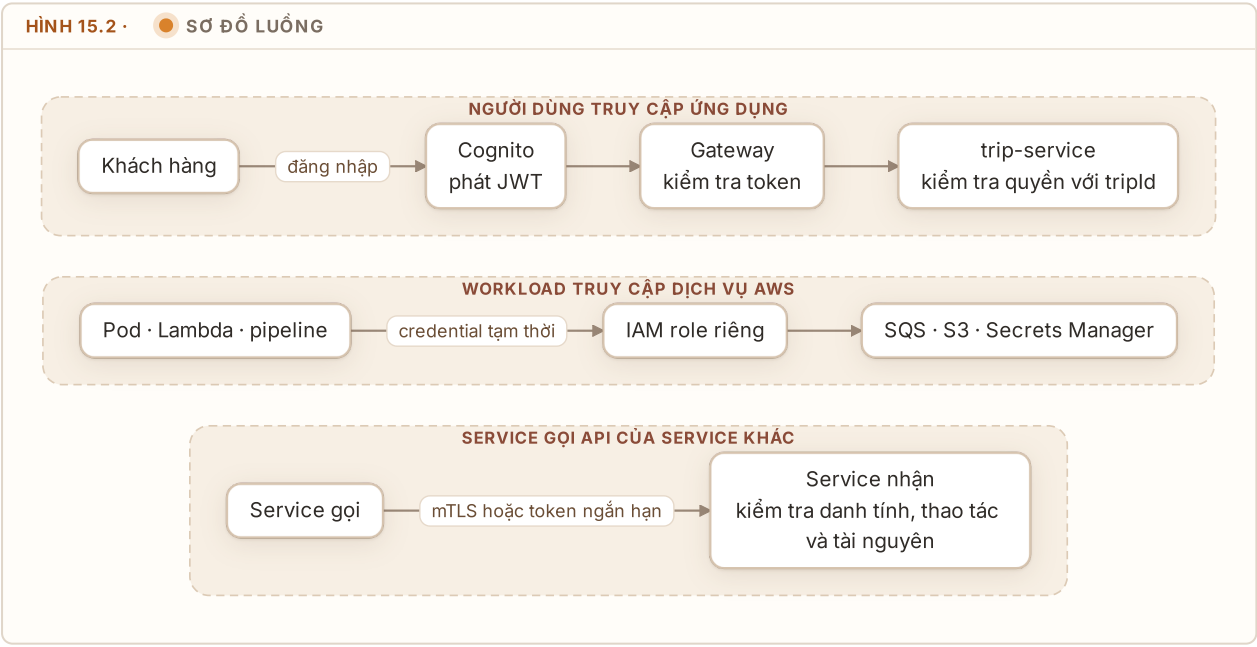
Nguyên tắc quyền tối thiểu — least privilege — giúp thu hẹp phạm vi thiệt hại. Tuy nhiên, `hotel-service` bị chiếm quyền vẫn có thể làm sai dữ liệu hotel mà nó được phép sửa. Giới hạn quyền vì thế cần đi cùng khả năng phát hiện, cô lập, thu hồi quyền, khôi phục dữ liệu và đối soát kết quả.

15.2 Phân biệt danh tính người dùng, workload và bên gọi API

15.2.1 Xác định danh tính tại từng điểm truy cập

Các chương trước đã đề cập đến quyền của người dùng, pipeline, node và workload. Khi xem xét toàn hệ thống, chúng ta cần phân biệt ba tình huống xác thực và phân quyền:

HÌNH 15.2 · SƠ ĐỒ LUỒNG



Sơ đồ minh họa các phạm vi kiểm tra quyền. Cognito và API Gateway là một phương án cho việc tiếp nhận request từ người dùng; nếu sử dụng ALB, chúng ta vẫn cần bố trí cơ chế xác thực phù hợp. Với chuỗi gọi Step Functions → Lambda adapter → domain service ở Chương 11, adapter là bên trực tiếp gửi request HTTP và cần được service nhận xác thực.

Workload truy cập AWS sử dụng IAM role. Policy xác định các thao tác được phép, còn trust policy quy định chủ thể nào có thể sử dụng role. Credential được cấp cho phiên làm việc có thời hạn. Pipeline có thể dùng OIDC như Chương 4; Pod dùng EKS Pod Identity hoặc IRSA như

Chương 11; Lambda dùng execution role như Chương 9; EC2 node sử dụng role thông qua instance profile.

Nhân viên truy cập AWS nên sử dụng cơ chế đăng nhập tập trung như IAM Identity Center để nhận credential ngắn hạn. Nếu có trường hợp phải dùng access key dài hạn của IAM user, cần quy định rõ cách lưu giữ, xoay vòng và thu hồi. AWS — Thực hành IAM.

Người dùng TravelNow sử dụng cơ chế xác thực của ứng dụng. Amazon Cognito User Pools có thể quản lý tài khoản, MFA và phát JWT. JWT authorizer của API Gateway HTTP API kiểm tra token theo cấu hình, bao gồm chữ ký, issuer, audience và thời hạn. Service vẫn phải kiểm tra quyền của người dùng đối với dữ liệu và thao tác được yêu cầu. AWS — JWT authorizer cho HTTP API.

15.2.2 Kiểm tra quyền sau khi xác thực danh tính

Một JWT hợp lệ giúp xác định danh tính và các claim được chấp nhận. Khi người dùng yêu cầu xem hoặc hủy một chuyến đi, `trip-service` còn phải kiểm tra người đó có quyền với `tripId` tương ứng hay không. Nếu thiếu bước này, người đã đăng nhập có thể thay ID trong URL để truy cập dữ liệu của người khác.

Vì vậy, **xác thực tại gateway và phân quyền nghiệp vụ tại service cần được thực hiện cùng nhau**.

Với API nội bộ, địa chỉ mạng không đủ để xác định quyền của bên gọi. Service nhận cần xác thực workload, chẳng hạn bằng mTLS với certificate riêng hoặc token ngắn hạn, rồi kiểm tra thao tác và tài nguyên được phép truy cập. Một API key dùng chung cho cả cluster không giúp phân biệt quyền của từng service.

Danh tính workload và danh tính người dùng cũng cần được xử lý riêng. Khi `trip-service` gọi `payment-service`, bên nhận cần biết caller trực tiếp là `trip-service`. Thông tin người dùng chỉ cần được truyền thêm khi quyết định nghiệp vụ tại payment phụ thuộc vào người dùng đó.

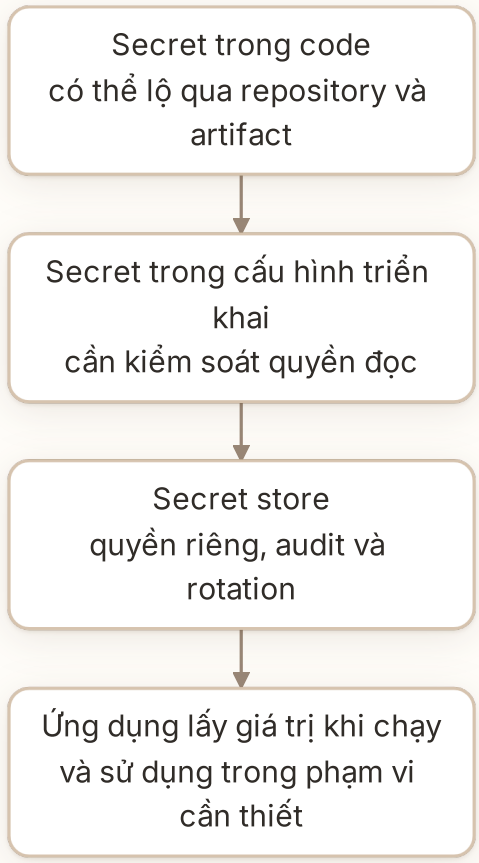
Việc chuyển tiếp nguyên JWT có quyền rộng qua mọi thành phần làm tăng phạm vi có thể lộ token và gây khó khăn khi xác định service nào đang hành động thay mặt người dùng. Kiểm tra rõ danh tính cùng quyền của từng caller giúp bảo đảm **quyền gọi refund không bị cấp mặc nhiên cho `hotel-service` chỉ vì các service cùng thuộc TravelNow**.

15.3 Quản lý secret, mã hóa và bảo vệ kết nối

15.3.1 Quản lý secret từ lúc cấp đến khi thu hồi

Tách cấu hình khỏi code, như ở Chương 3, là bước đầu. Với secret, chúng ta còn cần kiểm soát nơi lưu, quyền đọc, thời gian sử dụng và cách thay thế khi hết hạn hoặc bị lộ.

HÌNH 15.3 · SƠ ĐỒ LUỒNG



Với **AWS Secrets Manager**, cấu hình triển khai có thể chỉ chứa tham chiếu như `travelnow/prod/hotel/db`. Workload dùng IAM role của mình để đọc giá trị khi chạy. Cách này giúp tránh đưa secret trực tiếp vào repository hoặc manifest, đồng thời hỗ trợ kiểm tra lịch sử truy cập.

Secret vẫn tồn tại trong bộ nhớ ứng dụng khi được sử dụng. Vì vậy, secret store không loại bỏ yêu cầu bảo vệ process, tránh ghi secret vào log và giới hạn quyền đọc của workload.

Trên EKS, External Secrets Operator có thể đồng bộ giá trị từ secret store vào Kubernetes Secret. Khi đó, cần bảo vệ cả bản sao trong cluster bằng mã hóa dữ liệu lưu trữ và RBAC phù hợp.

Rotation cần được kiểm chứng đến tận ứng dụng sử dụng secret. Sau khi đổi credential RDS, ứng dụng phải nhận được giá trị mới và cập nhật kết nối đúng cách. Như đã phân tích ở Mục 10.6, nguồn cấu hình thay đổi chưa chứng minh process đang chạy đã sử dụng cấu hình mới.

15.3.2 Xác định phạm vi bảo vệ của mã hóa và TLS

Mã hóa khi lưu trữ — encryption at rest — bảo vệ dữ liệu của RDS, S3, EBS hoặc SQS ở tầng lưu trữ. Tuy nhiên, chủ thể có đủ quyền đọc dữ liệu vẫn có thể nhận được nội dung đã giải mã. Khi sử dụng KMS, chúng ta cần phân biệt quyền sử dụng khóa với quyền quản lý khóa.

Mã hóa khi truyền — encryption in transit — cần được cấu hình theo từng kết nối. TLS tại ALB, CloudFront hoặc API Gateway bảo vệ kết nối tới điểm kết thúc TLS đó. Các kết nối tiếp theo, như ALB tới ứng dụng hoặc ứng dụng tới RDS, cần được cấu hình riêng.

Vì vậy, với mỗi chặng, chúng ta cần xác định protocol, nơi kết thúc TLS và cách xác thực chứng chỉ. Những lựa chọn này nên được thể hiện trong module IaC và cấu hình ứng dụng để có thể kiểm tra nhất quán giữa các môi trường. Dữ liệu nhạy cảm được lưu qua Kubernetes API cũng cần cơ chế mã hóa và kiểm soát truy cập phù hợp. AWS — Mã hóa dữ liệu EKS.

15.4 Sao lưu và kiểm chứng khả năng khôi phục

15.4.1 Phân biệt mục đích của HA và backup

Chương 12 tập trung vào khả năng tiếp tục phục vụ khi một phần hạ tầng gặp sự cố. Backup bổ sung khả năng phục hồi dữ liệu khi dữ liệu hiện tại bị mất hoặc bị thay đổi sai.

Sự cố	Vai trò của RDS Multi-AZ	Vai trò của backup và point-in-time recovery
Máy chủ database hoặc một AZ gặp sự cố	Chuyển sang tài nguyên dự phòng theo cấu hình	Hỗ trợ khôi phục trong những tình huống cần thiết khác
Xóa nhầm hoặc migration làm sai dữ liệu	Thay đổi sai cũng có thể được sao chép sang bản dự phòng	Khôi phục dữ liệu về thời điểm trước khi xảy ra lỗi
Kẻ tấn công sửa hoặc phá dữ liệu bằng credential hợp lệ	Không ngăn thay đổi được sao chép	Cung cấp dữ liệu để khôi phục nếu backup được bảo vệ và còn bản phù hợp

HA và backup đáp ứng những nhu cầu phục hồi khác nhau; TravelNow cần kiểm chứng cả hai. Các phần cần chuẩn bị gồm:

- Bật automated backup và point-in-time recovery cho RDS, với thời gian lưu giữ phù hợp.
- Bật versioning cho tài liệu S3 cần khôi phục, đồng thời cấu hình lifecycle theo yêu cầu lưu giữ.
- Lưu phiên bản, mã hóa và giới hạn quyền truy cập Terraform state. Mất state không làm resource biến mất ngay, nhưng gây khó khăn cho việc quản lý và thay đổi an toàn.
- Thực hiện restore định kỳ trong môi trường cách ly, kiểm tra dữ liệu và khả năng hoạt động của ứng dụng.

Thông báo backup thành công cần được bổ sung bằng kết quả diễn tập: dữ liệu có khôi phục được không, ứng dụng có sử dụng được không và tổng thời gian có đáp ứng RTO hay không?

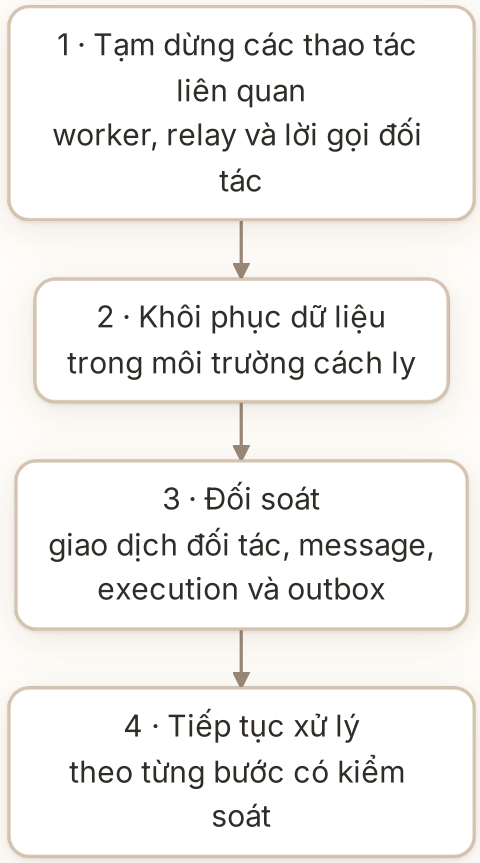
15.4.2 Đối soát nghiệp vụ sau khi khôi phục dữ liệu

Trong hệ phân tán, khôi phục database còn đòi hỏi đối chiếu với những giao dịch đã xảy ra ở các thành phần khác.

Giả sử RDS được khôi phục về thời điểm trước một lần thu tiền. Cổng thanh toán vẫn ghi nhận giao dịch thành công, còn queue vẫn chứa message liên quan. Bản ghi idempotency trong database có thể không còn sau restore. Nếu worker xử lý lại ngay, hệ thống có nguy cơ gửi một yêu cầu thu tiền không được nhận diện đúng là thao tác đã thực hiện.

Vì vậy, trước khi mở lại xử lý, chúng ta cần xác minh dữ liệu đã khôi phục với trạng thái thực tế bên ngoài:

HÌNH 15.4 · SƠ ĐỒ LUỒNG



Phạm vi tạm dừng và cách đối soát phụ thuộc vào sự cố, dữ liệu bị ảnh hưởng và nguồn được dùng để xác nhận kết quả. Những thao tác liên quan đến tiền hoặc chỗ giữ cần được kiểm tra trước khi thực hiện lại.

RTO cần bao gồm thời gian đưa nghiệp vụ trở lại trạng thái có thể phục vụ an toàn. Việc RDS đã sẵn sàng kết nối hoặc từng database đã restore thành công chưa chứng minh toàn bộ Saga đã nhất quán.

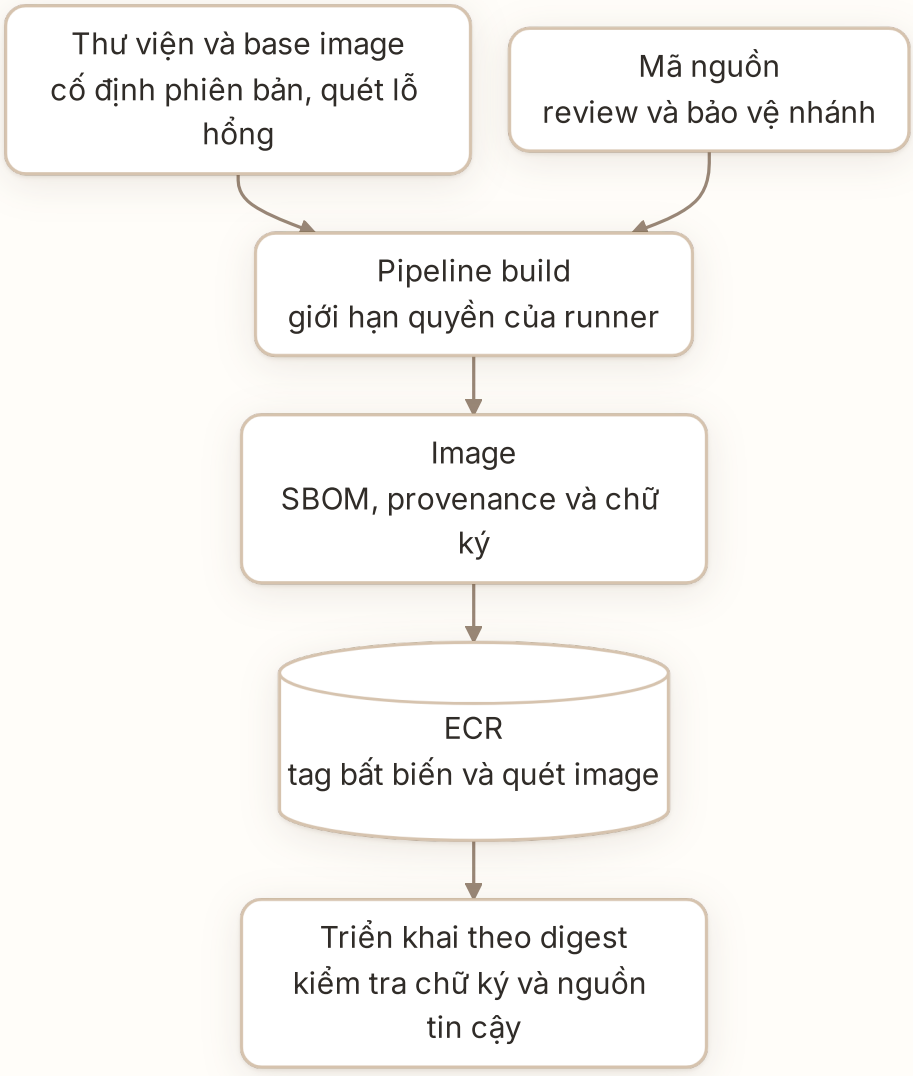
15.5 Bảo vệ quy trình phát hành và phân tích mối đe dọa

15.5.1 Kiểm soát nguồn gốc của bản phát hành

Chuỗi cung ứng phần mềm của TravelNow bao gồm mã nguồn, thư viện, base image, công cụ build và pipeline tạo artifact. Nếu một thành phần bị thay đổi trái phép, nội dung độc hại có thể đi vào bản phát hành dù ứng dụng vẫn khởi động bình thường.

Chúng ta cần kiểm soát nguồn gốc artifact và quyền thay đổi ở từng bước:

HÌNH 15.5 · SƠ ĐỒ LUỒNG



SBOM liệt kê các thành phần phần mềm trong artifact. **Provenance** ghi lại nguồn gốc và quá trình tạo artifact. Chữ ký, khi được kiểm tra theo chính sách tin cậy trước triển khai, giúp xác minh artifact đến từ nguồn được chấp nhận.

Kết quả quét lỗ hổng cần được đánh giá trong bối cảnh sử dụng. Mức độ nghiêm trọng là một yếu tố; chúng ta còn cần xét khả năng khai thác, mức phơi lộ của ứng dụng và phương án vá lỗi.

Ở tầng workload, có thể giảm quyền của process bằng cách chạy non-root, loại bỏ Linux capabilities không cần thiết, chặn nâng quyền và giới hạn nơi được ghi dữ liệu. Nếu dùng filesystem chỉ đọc, ứng dụng vẫn cần vùng ghi tạm cho các thao tác thực sự cần thiết; tài liệu PDF cần lưu bền vững tiếp tục được đưa lên S3.

Các cấu hình này phải được thử với ứng dụng thực tế để vừa hạn chế quyền vừa bảo đảm chức năng cần thiết hoạt động. Kubernetes — Security context.

15.5.2 Xây dựng mô hình mối đe dọa bằng bốn câu hỏi

Mô hình mối đe dọa giúp chúng ta xem xét cách tài sản có thể bị truy cập hoặc thay đổi trái phép, rồi kiểm tra biện pháp bảo vệ tương ứng.

1. **Cần bảo vệ những tài sản nào?** Với TravelNow, đó là dữ liệu giao dịch, định danh thanh toán, thông tin cá nhân, tài liệu PDF và những message chứa dữ liệu liên quan. Mỗi nơi lưu thêm dữ liệu nhạy cảm làm tăng phạm vi cần bảo vệ, nên event chỉ nên mang những trường cần thiết.
2. **Ai được phép truy cập và bằng cách nào?** `payment-service` sử dụng tài khoản database riêng và kết nối mạng được cho phép. Nhân viên truy cập production qua cơ chế đăng nhập tập trung và quyền được kiểm soát. Các giả định như "RDS không thể được truy cập trực tiếp từ Internet" cần được kiểm chứng bằng cấu hình và kiểm tra kết nối.
3. **Kẻ tấn công có thể lợi dụng những điểm nào?** Ví dụ, một dependency có lỗ hổng cho phép chạy code trong `hotel-service`, sau đó quyền quá rộng cho phép đọc dữ liệu khác hoặc phát sự kiện giả. Việc quét lỗ hổng, review policy và kiểm thử phân quyền cần gắn với từng bước của tình huống này.
4. **Làm sao phát hiện và xử lý khi có xâm nhập?** CloudTrail cung cấp lịch sử những sự kiện API được hỗ trợ; một số loại sự kiện cần bật riêng. GuardDuty hỗ trợ phát hiện các nhóm hành vi từ những nguồn dữ liệu được cấu hình. Các công cụ này cần phối hợp với audit ứng dụng, cảnh báo ở Chương 14 và quy trình ứng phó đã được diễn tập.

15.6 Phân tích chi phí từ các quyết định kiến trúc

Bảo mật và phục hồi đều cần nguồn lực: backup sử dụng dung lượng lưu trữ, audit tạo log, còn diễn tập cần tài nguyên và thời gian. Một lỗi retry hoặc cấu hình autoscaling không phù hợp cũng có thể làm tăng đồng thời tải hệ thống và hóa đơn.

Phân tích chi phí giúp chúng ta xác định các nguồn lực đang được sử dụng cho mục tiêu nào và phần nào cần điều chỉnh.

15.6.1 Xác định các khoản chi của từng thành phần

Mỗi quyết định kiến trúc đều sử dụng tài nguyên và công sức. Với AWS, một phần được thể hiện thành phí dịch vụ. Khi tự vận hành, chi phí chuyển sang máy chủ thuê hoặc sở hữu, tài nguyên dự phòng, nâng cấp và xử lý sự cố.

Vì vậy, các phương án cần được so sánh trên cùng lượng công việc, mục tiêu phục vụ và phạm vi trách nhiệm.

Có thể phân tích TravelNow theo các nhóm sau:

Nhóm	Thành phần	Yếu tố tạo chi phí
Compute	EC2 node của EKS	Loại instance, số lượng và thời gian duy trì; phương án cam kết giá được lựa chọn riêng
Compute	Control plane EKS	Thời gian sử dụng cluster và giai đoạn hỗ trợ phiên bản
Compute	Lambda	Số request, cấu hình bộ nhớ và thời lượng thực thi
Điều phối	Step Functions Standard	Số lần chuyển trạng thái
Messaging	SNS và SQS	Số request và kích thước dữ liệu theo cách tính của dịch vụ
Dữ liệu	RDS, Multi-AZ và RDS Proxy	Cấu hình compute, lưu trữ, I/O và các thành phần bổ sung
Dữ liệu	S3	Storage class, dung lượng, request và truyền dữ liệu
Mạng	NAT gateway, truyền dữ liệu giữa AZ và ra Internet	Tài nguyên mạng được duy trì và lượng dữ liệu đi qua
Quan sát	Log, metric và trace	Lượng dữ liệu thu thập, lưu trữ, truy vấn và các đơn vị tính của từng dịch vụ

Bảng này giúp nhận diện nguồn chi phí, không thay thế việc lập dự toán. Khi ước tính, cần kiểm tra Region, chế độ sử dụng và giá hiện hành. Ngay trong cùng một dịch vụ quan sát, thu thập log, truy vấn log và lưu metric có thể dùng các đơn vị tính khác nhau. AWS — CloudWatch pricing.

15.6.2 Tính cả công sức xây dựng và vận hành

Hóa đơn Cloud chỉ phản ánh một phần chi phí duy trì hệ thống. Giả sử TravelNow cân nhắc tự vận hành PostgreSQL thay cho RDS. Phí dịch vụ có thể giảm, nhưng chúng ta sẽ phải tự thực hiện thêm việc vá lỗi, replication, failover, backup, restore và xử lý sự cố.

Để so sánh, cần xác định cùng mục tiêu phục vụ và cùng khoảng thời gian, chẳng hạn một năm:

Nhóm chi phí	Nội dung cần tính
Tài nguyên và dịch vụ	Compute, lưu trữ, mạng, bản quyền nếu có và tài nguyên dự phòng
Xây dựng và chuyển đổi	Triển khai, di chuyển dữ liệu, kiểm thử, đào tạo và chạy song song
Duy trì	Nâng cấp, xử lý lỗ hổng, trực sự cố, diễn tập, tài liệu và hỗ trợ
Phối hợp và gián đoạn	Thời gian chờ giữa các nhóm, điều tra, phục hồi nghiệp vụ và ảnh hưởng tới khách hàng

Có thể ước lượng giờ công và quy đổi bằng một đơn giá nội bộ thống nhất, đồng thời ghi rõ giả định và mức độ không chắc chắn. Cách này giúp nhận ra một phương án đang thực sự tiết kiệm nguồn lực hay chỉ chuyển từ phí dịch vụ sang công việc vận hành.

Thời gian dành cho vận hành cũng ảnh hưởng đến khả năng phát triển tính năng. Khi quy đổi thành chi phí, cần tránh tính cùng một phần công sức nhiều lần dưới các tên khác nhau.

Việc tách thêm microservice cũng tạo ra những khoản tương tự. Mỗi service cần cấu hình, pipeline, quyền, dashboard, cảnh báo, quản lý phiên bản và người phụ trách. Công cụ dùng chung ở Chương 11 có thể giảm công việc lặp lại, nhưng chính công cụ đó cũng cần được duy trì.

Vì vậy, **dịch vụ được quản lý có phí cao hơn vẫn có thể mang lại tổng chi phí thấp hơn**. Quyết định tự xây hoặc tự vận hành cần dựa trên lợi ích cụ thể và khả năng duy trì trong thời gian dài.

Điều này cũng áp dụng khi chuyển workload từ AWS về hạ tầng sẵn có của tổ chức. Hóa đơn AWS có thể giảm đáng kể, nhưng cần tính phần tài nguyên nội bộ được sử dụng, backup, kết nối, vận hành và công sức chuyển đổi. Hạ tầng còn dư là một lợi thế nếu đáp ứng yêu cầu; việc một bộ phận khác thanh toán không làm chi phí đó biến mất.

15.6.3 Theo dõi chi phí nền, mạng và telemetry

Chi phí nền và chi phí theo mức sử dụng cùng tồn tại. Tài nguyên được duy trì sẵn tạo ra khoản chi ngay cả khi ít công việc. Các dịch vụ tính theo mức dùng tăng chi phí theo số yêu cầu, thời lượng xử lý và cả số lần retry. Như Chương 13 đã phân tích, đặc điểm tải ảnh hưởng trực tiếp đến phương án phù hợp.

Kết nối mạng cần được thể hiện trong dự toán. NAT gateway có thể phát sinh phí theo thời gian và lượng dữ liệu xử lý. Truyền dữ liệu giữa AZ hoặc ra Internet cũng cần được xem xét theo dịch vụ và tuyến kết nối. Thiết kế nhiều AZ để tăng độ sẵn sàng có thể làm thay đổi chi phí mạng.

Các lời gọi AWS API từ private subnet không nhất thiết đều đi qua NAT. VPC endpoint phù hợp có thể thay đổi cách kết nối và chi phí, nhưng cũng cần tính các khoản riêng của endpoint. Quyết định nên dựa trên lượng dữ liệu thực tế, báo cáo chi phí và giá hiện hành.

Telemetry cần có phạm vi thu thập rõ ràng. Ghi toàn bộ request body vừa tăng lượng dữ liệu vừa có nguy cơ lưu thông tin nhạy cảm. Mức log, thời gian lưu giữ và sampling cần được thiết kế cùng yêu cầu điều tra ở Chương 14.

15.6.4 Đọc đúng chỉ số chi phí mỗi booking

Để liên hệ nguồn lực với kết quả nghiệp vụ, chúng ta có thể theo dõi:

> **Chi phí mỗi booking thành công = tổng chi phí được phân bổ trong kỳ ÷ số booking thành công trong cùng kỳ**

Chỉ số này hữu ích khi phạm vi chi phí và quy tắc phân bổ được giữ nhất quán. Nếu chỉ tính hóa đơn Cloud, cần gọi rõ là "chi phí Cloud mỗi booking". Nếu gồm cả công sức và chi phí dùng chung, phải nêu cách phân bổ. Chi phí chuyển đổi một lần cũng cần được tách riêng hoặc phân bổ theo thời gian đã thống nhất.

Trước khi kết luận, cần kiểm tra cả tổng chi phí và số booking thành công. Giả sử một kỳ có 10.000 yêu cầu, tổng chi phí là 100 đơn vị tiền và 1.000 booking thành công. Chi phí mỗi booking là 0,1 đơn vị.

Kỳ sau vẫn có 10.000 yêu cầu và tổng chi phí 100 đơn vị, nhưng chỉ 500 booking thành công vì hết phòng. Chi phí mỗi booking tăng lên 0,2 đơn vị dù tổng chi phí không đổi. Đây là ví dụ số học để giải thích chỉ số, không phải dự toán AWS.

Vì vậy, nên theo dõi đồng thời:

- **Chi phí mỗi yêu cầu được tiếp nhận.**
- **Tỷ lệ booking thành công.**
- **Chi phí mỗi booking thành công.**

Retry kỹ thuật có thể làm tăng tổng chi phí, còn đối tác hết phòng có thể làm giảm số booking thành công. Hai nguyên nhân cần cách xử lý khác nhau.

Chi phí mỗi booking cũng không phải tiêu chí duy nhất. Một khoản đầu tư vào bảo mật hoặc phục hồi có thể làm chỉ số tăng nhưng vẫn cần thiết để đáp ứng yêu cầu của hệ thống.

Môi trường dev hoặc kiểm thử cần được đánh giá theo mục đích riêng, chẳng hạn chi phí mỗi lượt kiểm thử hoàn tất, thời gian chờ môi trường sẵn sàng và tổng chi phí duy trì mỗi tháng. Nếu phân bổ khoản này vào chi phí booking production, cần ghi rõ quy tắc. Credits hoặc ưu đãi cũng nên được tách khỏi chi phí sử dụng trước ưu đãi: số tiền thực trả thấp chưa cho biết cấu hình đó có phù hợp với ngân sách sau khi credits hết hạn hay không.

15.7 Kiểm soát và tối ưu chi phí

15.7.1 Xác định mục đích và thời gian sử dụng

Tối ưu chi phí bắt đầu từ việc hiểu tài nguyên đang phục vụ công việc gì. Với TravelNow, một môi trường dev có thể được dùng để phát triển tính năng ban ngày, chạy integration test ban đêm hoặc nhận callback từ sandbox của đối tác. Cùng mang nhãn dev nhưng những nhu cầu này dẫn đến lịch hoạt động khác nhau.

Trước khi điều chỉnh cấu hình, chúng ta cần thống nhất với người sử dụng: môi trường cần hoạt động khi nào, có thể chờ khởi động bao lâu, dữ liệu nào phải giữ và ai chịu trách nhiệm khi không còn nhu cầu sử dụng. Khi dự án chuyển từ phát triển sang bảo trì, những yêu cầu này

cũng cần được xem xét lại. Một môi trường chỉ dùng để tái hiện lỗi vài lần mỗi tháng có thể phù hợp với cách khởi tạo theo nhu cầu.

Dùng đúng mục đích còn có nghĩa là giữ đủ những đặc tính cần kiểm thử. Môi trường thử failover cần cấu hình dự phòng tương ứng, trong khi môi trường kiểm thử chức năng thông thường có thể có yêu cầu khác. Sau khi giảm tài nguyên, chúng ta cần biết những loại kiểm thử nào vẫn có giá trị và những loại nào phải thực hiện ở môi trường khác.

15.7.2 Phân loại workload trước khi chọn tài nguyên

Các workload trong cùng hệ thống có thể khác nhau về hiệu năng, nhu cầu co giãn, mức độ quan trọng và khả năng chịu gián đoạn. Việc phân loại giúp lựa chọn tài nguyên phù hợp cho từng công việc:

Workload hoặc dữ liệu	Yêu cầu cần xác định	Hướng tối ưu có thể xem xét
API phục vụ khách hàng	Độ trễ, SLO và khả năng phục vụ khi có sự cố	Điều chỉnh tài nguyên và co giãn trong giới hạn đã kiểm chứng
Worker kiểm thử với đối tác giả	Khả năng chạy lại và thời hạn hoàn tất	Sử dụng tài nguyên chịu gián đoạn nếu công việc có thể phục hồi an toàn
Môi trường tích hợp dùng chung	Giờ sử dụng, lịch chạy test và callback	Bật trước giờ cần dùng, thu nhỏ hoặc dừng phần không cần hoạt động ngoài giờ
Môi trường thử nghiệm tạm	Thời điểm hết nhu cầu và khả năng tái tạo dữ liệu	Gắn thời hạn sử dụng và tự động thu hồi
Database, volume và backup	Dữ liệu cần giữ và yêu cầu khôi phục	Điều chỉnh dung lượng, thời gian lưu giữ và chính sách thu hồi

Nhãn dev không tự chứng minh workload có thể bị dừng bất kỳ lúc nào. Một database dùng chung hoặc bài kiểm thử kéo dài vẫn có thể làm nhiều người phải chờ nếu bị gián đoạn. Tương tự, workload có idempotency chưa chắc đáp ứng thời hạn khi phải chạy lại nhiều lần.

Phân loại theo yêu cầu thực tế giúp xác định phần có thể dùng Spot, phần cần tài nguyên sẵn sàng và phần có thể dừng theo lịch. Các lựa chọn mua tài nguyên được xem xét tiếp ở Mục 15.7.5.

15.7.3 Tự động hóa và kiểm tra kết quả vận hành

Sau khi thống nhất nhu cầu, chúng ta có thể tự động hóa từng phần: bật môi trường trước giờ làm việc, thu nhỏ sau khi các bài test hoàn tất, thu hồi môi trường thử nghiệm hết hạn và dọn artifact theo chính sách lưu giữ. Việc tự động hóa nên được tính đến khi thiết kế môi trường, rồi triển khai dần theo mức độ lặp lại và khả năng kiểm soát từng thao tác.

Lịch hoạt động cần có múi giờ rõ ràng, cách gia hạn khi làm ngoài giờ và người nhận cảnh báo nếu thao tác thất bại. Khi khởi động lại, cần kiểm tra ứng dụng cùng các thành phần phụ thuộc đã sẵn sàng phục vụ.

Giả sử một phần compute phục vụ dev chỉ cần chạy 10 giờ mỗi ngày, 5 ngày mỗi tuần. So với 168 giờ chạy liên tục, lịch này còn 50 giờ, giảm khoảng 70% thời gian cấp tài nguyên cho phần compute đó. Đây là phép tính minh họa, chưa tính thời gian chuẩn bị, những lần sử dụng ngoài lịch và điều kiện tính phí. Tổng hóa đơn không giảm cùng tỷ lệ vì lưu trữ, backup, mạng và tài nguyên dùng chung có thể vẫn phát sinh phí. Chẳng hạn, dừng EC2 vẫn giữ các EBS volume liên quan. AWS — Dừng và khởi động EC2.

Trên EKS, giảm replica chưa chắc làm giảm số node. Lịch hoạt động cần phối hợp với HPA, KEDA và cơ chế cấp node để tránh việc một công cụ tạo lại tài nguyên vừa được thu nhỏ. Với môi trường có dữ liệu, cần xác định rõ phần được dừng, phần được xóa và cách tiếp tục công việc khi mở lại.

Tự động hóa cần đi cùng cảnh báo và kiểm kê định kỳ. Một job có thể chạy thành công nhưng bỏ sót tài nguyên mới tạo, sai tag hoặc nằm ngoài phạm vi quản lý. Chúng ta cần đối chiếu trạng thái mong đợi với tài nguyên thực tế và chi phí sau thay đổi.

Chẳng hạn, kiểm kê có thể phát hiện nhiều EBS volume không còn gắn vào instance. Trước khi thu hồi, cần xác định người phụ trách, mục đích dữ liệu và yêu cầu lưu giữ; trạng thái không gắn chưa đủ để kết luận volume có thể xóa. Có thể tự động lập danh sách cần kiểm tra, thông báo người phụ trách rồi thu hồi theo quy trình đã thống nhất. Những trường hợp phát hiện qua kiểm tra thủ công nên được bổ sung dần vào quy tắc tự động.

15.7.4 Phân bổ chi phí và phát hiện biến động

Trước khi tối ưu, chúng ta cần biết chi phí thuộc về đâu, thay đổi ra sao và ai có trách nhiệm xử lý:

1. **Gắn tag và thống nhất quy tắc phân bổ.** Các tag như `service`, `environment` và `owner` giúp nhận diện tài nguyên. Với node EKS, RDS, NAT hoặc ALB dùng chung, cần thêm quy tắc phân chia dựa trên dữ liệu phù hợp như tài nguyên được yêu cầu, mức sử dụng hoặc traffic. Kết quả là một mô hình phân bổ cần được giải thích và áp dụng nhất quán.
2. **Thiết lập ngân sách và cảnh báo bất thường.** Ngưỡng cảnh báo cần đủ sớm để điều tra và hành động. Retry tăng đột biến hoặc autoscaler cấu hình sai có thể xuất hiện đồng thời trong telemetry và dữ liệu chi phí.
3. **Đọc xu hướng bằng Cost Explorer.** Việc xem chi phí định kỳ giúp phát hiện thay đổi theo môi trường, dịch vụ và giai đoạn phát hành, thay vì chỉ kiểm tra khi hóa đơn vượt dự kiến.

Cảnh báo ngân sách không tự giới hạn số tiền hệ thống được phép sử dụng. Dữ liệu chi phí và thông báo có độ trễ. Các biện pháp trực tiếp hơn gồm trần replica, giới hạn concurrency và chính sách cấp tài nguyên ở Chương 12–13.

Budget Actions có thể tự động hóa một số phản ứng, nhưng cần cấu hình riêng và đánh giá ảnh hưởng tới khả năng phục vụ. AWS — Budgets.

15.7.5 Kiểm chứng hiệu quả của từng biện pháp tối ưu

Có thể bắt đầu từ ba nhóm biện pháp:

- **Thu hồi tài nguyên không còn cần thiết.** Môi trường tạm có thể được hủy theo lịch sau khi xác nhận đúng phạm vi và dữ liệu cần giữ. Môi trường có state cần phương án dừng, thu nhỏ hoặc sao lưu phù hợp. Lifecycle của ECR và S3 phải giữ đủ artifact và dữ liệu cho rollback, phục hồi và yêu cầu lưu trữ.
- **Điều chỉnh tài nguyên theo nhu cầu thực tế — right-sizing.** Requests của Pod ảnh hưởng đến cách scheduler và cơ chế cấp node bố trí workload. Khai báo dư có thể làm tăng số node phải duy trì, nhưng mức tăng không tỷ lệ trực tiếp với phần tài nguyên khai báo dư. Kích thước instance, ràng buộc phân bố, tài nguyên hệ thống và phần dự phòng đều ảnh hưởng. Giảm requests cần được thử tải để tránh tranh chấp tài nguyên hoặc làm sai tín hiệu autoscaling.
- **Chọn hình thức mua tài nguyên phù hợp.** Phần sử dụng ổn định có thể được xem xét cho Savings Plans hoặc Reserved Instances sau khi đã điều chỉnh lịch hoạt động và đo nhu cầu còn lại. Spot có thể phù hợp với workload chịu được gián đoạn và có cơ chế phục hồi đã kiểm chứng. Cần tính cả thời gian chờ tài nguyên thay thế và chi phí chạy lại công việc; idempotency không tự bảo đảm workload vẫn hoàn tất đúng hạn. AWS — Spot Instances.

Right-sizing cần được đo bằng chi phí cho công việc hoàn tất. Với Lambda, CPU được cấp theo cấu hình bộ nhớ. Giảm bộ nhớ có thể làm thời gian chạy dài hơn, khiến phần chi phí theo **bộ nhớ × thời lượng** không giảm như dự kiến.

Nếu TravelNow dùng Lambda để tạo PDF, chúng ta có thể chạy cùng một tập tài liệu ở nhiều mức bộ nhớ, rồi so sánh thời lượng, tỷ lệ lỗi và chi phí cho mỗi tài liệu hoàn tất. Khi CPU là nút thắt, thêm tài nguyên có thể giúp giảm thời gian đủ để cải thiện chi phí. Nếu phần lớn thời gian nằm ở chờ dịch vụ bên ngoài, cần xem xét nguyên nhân chờ đó. Góc nhìn này được trình bày trong *Serverless Architectures on AWS*, mục 10.3.2; cách Lambda cấp tài nguyên được mô tả trong tài liệu cấu hình bộ nhớ.

Quá trình tối ưu cần được lặp lại: **thiết kế, đo hoạt động, phân tích chi phí rồi điều chỉnh**. Mỗi thay đổi phải được đánh giá cùng mục tiêu phục vụ và khả năng phục hồi, với sự phối hợp giữa kỹ thuật, vận hành và người phụ trách ngân sách.

15.8 Tóm tắt

Bảo mật của TravelNow dựa trên nhiều cơ chế phối hợp: xác thực danh tính, giới hạn quyền, kiểm soát mạng, bảo vệ workload, quản lý dữ liệu và kiểm tra quy trình phát hành. Mỗi cơ chế cần có phạm vi rõ ràng và được kiểm chứng trong tình huống một thành phần đã bị chiếm quyền. Xác thực phải đi cùng phân quyền theo tài nguyên; HA cần được bổ sung bằng backup; khôi phục dữ liệu cần tính cả những giao dịch đã xảy ra bên ngoài database.

Chi phí bao gồm tài nguyên lẫn công sức: compute, lưu trữ, mạng, telemetry, phát triển, nâng cấp và xử lý sự cố. Tối ưu bắt đầu từ mục đích sử dụng, phân loại workload và tự động hóa phù hợp với nhu cầu thực tế. Cảnh báo, kiểm kê và đối chiếu chi phí giúp xác nhận các cơ chế ấy vẫn hoạt động đúng. Chỉ số chi phí mỗi booking có ý nghĩa khi phạm vi phân bổ rõ ràng và được đọc cùng tỷ lệ thành công. Mọi điều chỉnh cần giữ được mục tiêu phục vụ, khả năng phục hồi và năng lực duy trì hệ thống.

Qua mười lăm chương, chúng ta đã xây dựng các cơ chế để triển khai, phối hợp và vận hành TravelNow. Hiệu quả của từng lựa chọn vẫn cần được đo và kiểm chứng trong môi trường thực tế. Phần kết sẽ nhìn lại những quyết định này, cùng các dấu hiệu giúp nhận biết khi nào nên giữ nguyên và khi nào cần thay đổi.

PHẦN KẾT

Những đánh đổi có chủ đích

TravelNow bắt đầu từ một câu hỏi: thế nào là một chuyến đi được đặt thành công? Câu hỏi đó theo chúng ta xuyên suốt eBook, từ khi triển khai ứng dụng lên Cloud đến khi tách service, xử lý message được giao lại và thay thế instance gặp sự cố. Dù kiến trúc thay đổi, hệ thống vẫn cần tiếp nhận, tiếp tục và hoàn tất công việc cho khách hàng một cách chính xác.

Đó cũng là cách tiếp cận của eBook: bắt đầu từ yêu cầu cụ thể, lựa chọn cơ chế phù hợp, xác định trách nhiệm của từng thành phần rồi kiểm chứng kết quả. AWS cung cấp môi trường triển khai để chúng ta tìm hiểu những cơ chế này qua TravelNow. Mỗi công nghệ được xem xét cùng phạm vi bảo đảm, điều kiện sử dụng và công sức cần thiết để vận hành.

Nhìn lại các quyết định của TravelNow

Quyết định	Lợi ích hướng tới	Trách nhiệm và chi phí đi kèm
Tách dữ liệu cần lưu giữ khỏi vòng đời ứng dụng	Thay thế instance mà vẫn giữ dữ liệu; cho phép nhiều instance cùng xử lý	Kết nối mạng, phân quyền, phối hợp truy cập, sao lưu và phục hồi
Đóng gói ứng dụng và tự động hóa phát hành	Giảm sai lệch giữa các môi trường; kiểm tra và truy vết thay đổi	Duy trì image, pipeline, module hạ tầng và quyền thực thi
Tách service theo nhu cầu nghiệp vụ và vận hành	Phát hành, co giãn và phân công trách nhiệm độc lập	Quản lý contract, dữ liệu riêng, lỗi mạng và phối hợp giữa các nhóm
Sử dụng Saga và messaging	Tiếp tục hoặc bù trừ quy trình gồm nhiều transaction độc lập	Theo dõi trạng thái trung gian, xử lý trùng, giới hạn thời gian và đối soát
Sử dụng dịch vụ được quản lý	Giảm một phần công việc vận hành hạ tầng	Phí dịch vụ, giới hạn tùy biến và phạm vi hỗ trợ của nhà cung cấp
Co giãn tài nguyên và phân bố qua nhiều AZ	Đáp ứng tải thay đổi và duy trì khả năng phục vụ khi có sự cố	Tài nguyên dự phòng, thời gian cấp thêm tài nguyên và diễn tập phục hồi
Sử dụng serverless cho workload phù hợp	Giảm công việc quản lý máy chủ và tài nguyên chờ sẵn	Hạn mức, concurrency, giới hạn thực thi và chi phí theo mức sử dụng
Thu thập telemetry giữa các thành phần	Phát hiện bất thường, điều tra nguyên nhân và theo dõi tiến độ	Instrumentation, chất lượng dữ liệu, quyền truy cập và chi phí lưu giữ

Lựa chọn dữ liệu ở Chương 9 là một ví dụ. Cả năm service chạy trên Lambda, nhưng TravelNow vẫn sử dụng RDS với database riêng theo domain để duy trì mô hình transaction đã xây dựng. Chúng ta vì thế cần kiểm soát số kết nối, khả năng xử lý của RDS dùng chung và việc chuyển tiếp các bản ghi outbox.

Khi chuyển sang EKS, các instance có thể thay thế cho nhau nhờ trạng thái được lưu bền vững và các cơ chế phối hợp xử lý. Việc quản lý vòng đời, sao lưu và phục hồi dữ liệu vẫn cần được thực hiện riêng. Khi xem xét kết hợp Lambda với EKS, chúng ta tiếp tục đánh giá từng workload dựa trên đặc điểm tải và yêu cầu vận hành.

Những quyết định này không tạo thành một lộ trình bắt buộc. Một hệ thống có thể sử dụng dịch vụ được quản lý ngay từ đầu, duy trì monolith lâu dài hoặc kết hợp container với Lambda. Cơ sở lựa chọn là vấn đề cần giải quyết, lợi ích có thể kiểm chứng và khả năng duy trì phương án đó.

Hiểu hệ thống qua AWS, hiểu AWS qua TravelNow

AWS giúp các khái niệm trong eBook trở nên cụ thể. Chúng ta có thể xác định thành phần nào lưu tiến độ, thành phần nào chuyển message, thành phần nào cấp môi trường thực thi và dữ liệu nào hỗ trợ điều tra sự cố. TravelNow đặt những khả năng đó vào một quy trình có kết quả cần hoàn thành cho khách hàng.

Khi `payment-service` mất phản hồi, Step Functions vẫn lưu tiến độ Saga, nhưng ứng dụng phải đối soát để biết tiền đã được thu hay chưa. Khi hotel tạm ngừng xử lý, SQS có thể giữ message chờ trong thời gian lưu giữ đã cấu hình; ứng dụng vẫn cần xử lý lại an toàn và kiểm tra thời hạn nghiệp vụ. Khi Pod bị thay thế, Kubernetes duy trì workload, còn việc tiếp tục booking phụ thuộc vào trạng thái đã lưu. Khi khách hàng hỏi về chuyến đi, CloudWatch và X-Ray giúp tra cứu những thông tin mà ứng dụng đã ghi nhận.

Qua từng tình huống, chúng ta thấy rõ dịch vụ hỗ trợ phần nào và ứng dụng còn phải xử lý những gì.

Các yêu cầu tương tự có thể được thực hiện trên nền tảng Cloud khác hoặc bằng phần mềm mã nguồn mở, như những đối chiếu ở Chương 9, 10, 13 và 14. Tuy nhiên, chuyển nền tảng cần xem xét lại API, cách giao nhận message, mô hình dữ liệu, giới hạn và quy trình vận hành. Các tình huống lỗi cũng phải được kiểm thử lại để xác nhận hành vi nghiệp vụ vẫn được giữ đúng.

Hiểu những yêu cầu này giúp chúng ta có cơ sở để tiếp tục sử dụng AWS hoặc lựa chọn một nền tảng khác khi điều kiện thay đổi.

Áp dụng vào quyết định kiến trúc của bạn

Năm câu hỏi ở lời mở đầu có thể trở thành một bản ghi quyết định ngắn:

- **Vấn đề hoặc áp lực hiện tại là gì?**
- **Những yêu cầu nào phải tiếp tục được đáp ứng?**

- **Cơ chế nào phù hợp để giải quyết vấn đề?**
- **Trách nhiệm và chi phí sẽ thay đổi ra sao?**
- **Dùng kết quả nào để đánh giá quyết định?**

Bản ghi cần đủ rõ để người khác hiểu vì sao phương án được chọn, dựa trên những giả định nào và khi nào cần xem xét lại.

Chẳng hạn, một nhóm muốn tách payment khỏi monolith vì các lần phát hành thường xuyên phải chờ nhau. Trước hết, cần xác định nguyên nhân của thời gian chờ và xem việc cải thiện pipeline hoặc tổ chức lại module có giải quyết được vấn đề hay không.

Nếu vẫn chọn tách service, yêu cầu chống thu tiền trùng và khả năng đối soát phải được duy trì trong suốt quá trình chuyển đổi. Kế hoạch cần bao gồm dữ liệu, contract, phân quyền và trách nhiệm vận hành. Sau khi triển khai, chúng ta so sánh thời gian phát hành, mức độ sự cố và công sức duy trì với trước đó. Việc payment đã chạy độc lập mới xác nhận thay đổi kỹ thuật; kết quả đo mới cho biết thay đổi đó có giải quyết được vấn đề ban đầu hay không.

Cách đánh giá này cũng áp dụng khi thêm queue, bật autoscaling hoặc chuyển một bước xử lý sang Lambda. Chỉ số được chọn cần phản ánh mục tiêu của thay đổi.

Giữ nguyên kiến trúc cũng là một lựa chọn có cơ sở. Khi hệ thống đáp ứng yêu cầu với chi phí chấp nhận được, cải thiện kiểm thử, khả năng quan sát hoặc quy trình phục hồi có thể đem lại nhiều giá trị hơn việc thay đổi nền tảng. Khi dữ liệu thực tế cho thấy những giả định cũ không còn phù hợp, chúng ta có lý do rõ ràng để điều chỉnh.

Bắt đầu từ một quy trình nghiệp vụ

Hãy chọn một quy trình trong hệ thống của bạn và viết một trang ghi chú:

- Yêu cầu đi qua những thành phần nào?
- Dữ liệu nào đã được lưu sau mỗi bước?
- Nếu phản hồi bị mất, làm sao xác định thao tác đã thành công hay chưa?
- Khi một thành phần dừng hoạt động, công việc được tiếp tục bằng cách nào?
- Ai xử lý những trường hợp hệ thống không thể tự hoàn tất?

Sau đó, chọn một thành phần, chẳng hạn queue, và mô tả yêu cầu của nó bằng hành vi cần có thay vì tên sản phẩm. Đặt hai phương án triển khai cạnh nhau để so sánh cách giao nhận, thời gian lưu giữ, giới hạn, chi phí và trách nhiệm vận hành.

Cuối cùng, chọn một tình huống lỗi để kiểm thử. Ghi lại kết quả dự kiến, kết quả quan sát được và những điểm cần điều chỉnh. Bài tập này có thể thực hiện ngay trên hệ thống hiện tại, với phạm vi nhỏ và được kiểm soát.

Những chủ đề có thể tìm hiểu thêm

eBook chưa đi sâu vào triển khai nhiều Region, quản lý giao tiếp giữa các service bằng service mesh, lưu lịch sử thay đổi nghiệp vụ bằng event sourcing hay tổ chức mô hình đọc và ghi riêng với CQRS. Mỗi chủ đề giải quyết những nhóm vấn đề nhất định và có chi phí đi kèm.

Một yêu cầu cụ thể sẽ giúp định hướng việc tìm hiểu: cần tiếp tục phục vụ khi cả Region gặp sự cố, cần quản lý danh tính giữa nhiều workload, cần tái dựng lịch sử nghiệp vụ hoặc cần tối ưu cách đọc dữ liệu. Các tài liệu tham khảo là điểm bắt đầu để đánh giá những lựa chọn đó.

TravelNow vẫn là một mô hình học tập. Trước khi áp dụng cho người dùng thật, chúng ta cần kiểm chứng contract của đối tác, hành vi dưới tải, khả năng phục hồi và quy trình vận hành trên cấu hình, phiên bản cụ thể. **Phân biệt điều đã thiết kế với điều đã được đo và kiểm chứng là một phần quan trọng của tư duy hệ thống.**

Lời cuối

Ranh giới rõ ràng giúp chúng ta biết phần nào có thể thay đổi độc lập và phần nào cần phối hợp. Tự động hóa giúp triển khai thay đổi nhất quán hơn. Khả năng quan sát giúp kiểm tra hệ thống có đáp ứng yêu cầu và phát hiện nơi cần can thiệp. Giá trị của những cơ chế đó phụ thuộc vào việc chúng ta hiểu nghiệp vụ và theo dõi kết quả đến cùng.

Điều tôi mong bạn mang theo sau eBook là thói quen bắt đầu từ một vấn đề cụ thể, đưa ra quyết định có cơ sở và kiểm chứng quyết định ấy bằng thực tế. Công nghệ sẽ tiếp tục thay đổi; cách làm này vẫn giúp chúng ta xây dựng và vận hành những hệ thống đáng tin cậy.

Tài liệu tham khảo và đọc thêm

eBook ưu tiên xây bức tranh tổng thể qua TravelNow. Các nguồn dưới đây giúp bạn quay lại một câu hỏi cụ thể: vì sao chọn ranh giới service như vậy, một message có thể được giao lại khi nào, hoặc một dịch vụ AWS thực sự bảo đảm điều gì. Bạn không cần đọc hết trước khi thực hành; hãy chọn nguồn theo vấn đề đang gặp.

Sách và tài liệu về pattern giúp hiểu nguyên lý cùng các đánh đổi. Với cú pháp, phiên bản, quota và hành vi của dịch vụ cloud, hãy đối chiếu tài liệu chính thức tại thời điểm triển khai. Các liên kết ngay trong từng chương trỏ tới nguồn cho những điểm cần kiểm tra; danh sách này gom lại các hướng đọc tiếp.

Chọn nguồn theo câu hỏi

Khi đọc	Câu hỏi cần tra cứu	Nguồn nên bắt đầu
Chương 1–5	Làm sao quản lý dữ liệu, đóng gói và phát hành ứng dụng nhất quán?	Spring transaction, Docker, GitHub OIDC và Terraform ở phần tra cứu theo nhóm ví dụ.
Chương 6–9	Tách service, xử lý lại và bù trừ cần những cơ chế nào?	Building Microservices, Enterprise Integration Patterns, transactional outbox, Step Functions và Lambda với SQS.
Chương 10–12	Ai thay Pod, ai tăng replica và ai cấp thêm node?	Kubernetes Deployment, probes, HPA, KEDA, Karpenter và disruption.
Chương 13	Nền tảng quản lý phần nào và ứng dụng còn chịu trách nhiệm gì?	Lambda concurrency, ECS/EKS compute, Knative Serving và tài liệu database tương ứng.
Chương 14	Làm sao theo dõi booking và cảnh báo khi công việc quá hạn?	OpenTelemetry, W3C Trace Context và tài liệu SRE về SLO.
Chương 15	Làm sao giới hạn quyền, khôi phục dữ liệu và kiểm soát chi phí?	IAM, OWASP, Secrets Manager, RDS recovery, AWS Budgets và tài liệu chi phí.

Kiến trúc microservices và hệ thống phân tán

- Sam Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd edition, O'Reilly Media, 2021. Đặc biệt hữu ích cho ranh giới service, coupling/cohesion, tách monolith từng bước, communication style, workflow, contract testing, resiliency, observability và bảo mật giữa các service.

- Gregor Hohpe và Bobby Woolf, *Enterprise Integration Patterns*, Addison-Wesley, 2003. Nguồn nền tảng cho messaging, routing và integration pattern.
- Michael T. Nygard, *Release It!*, 2nd edition, Pragmatic Bookshelf, 2018. Đọc sâu về stability pattern, timeout, circuit breaker và failure mode trong production.
- Martin Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017. Đọc sâu về replication, consistency, transaction, stream và mô hình dữ liệu phân tán.
- RFC 9110 — HTTP Semantics — định nghĩa chính xác safe/idempotent method; hữu ích để phân biệt semantics của HTTP method với idempotency nghiệp vụ.
- RabbitMQ Documentation — exchange/queue/binding, acknowledgement và dead lettering; đối chiếu các cơ chế messaging được giải thích ở chương 8.
- Temporal Documentation — durable execution, yêu cầu determinism và kỷ luật versioning của workflow; đối chiếu với orchestrator tự xây ở chương 8.
- Apache Kafka — Design — đọc guarantees, transaction và phạm vi exactly-once; đặc biệt lưu ý ranh giới khi ghi sang hệ thống bên ngoài Kafka.

Kubernetes, nền tảng dùng chung và serverless

Ba nguồn sau bổ sung những góc nhìn cụ thể cho chương 10, 11 và 15. Các ví dụ TravelNow trong eBook là phần vận dụng của tác giả; số liệu và cấu hình trong tài liệu tham khảo cần được kiểm chứng lại khi triển khai.

Kubernetes: Up and Running

Brendan Burns, Joe Beda, Kelsey Hightower và Lachlan Evenson, *Kubernetes: Up and Running: Dive into the Future of Infrastructure*, 3rd edition, O'Reilly Media, 2022. Chương 13 của tài liệu tham khảo, phần "Managing ConfigMaps and Secrets", giúp hiểu vòng đời cấu hình và cách ứng dụng nhận thay đổi. Chương 10 của eBook này vận dụng vào việc truy vết bản phát hành và giới hạn của rollback.

Platform Engineering on Kubernetes

Mauricio Salatino, *Platform Engineering on Kubernetes*, Manning, 2023. Mục 1.3 của tài liệu tham khảo xem nền tảng như một sản phẩm nội bộ; chương 9 bàn về đo lường tác động. Chương 11 của eBook này vận dụng để đánh giá nền tảng dùng chung bằng khả năng hỗ trợ nhóm phát triển và giảm thời gian chờ.

Serverless Architectures on AWS

Peter Sbarski, Yan Cui và Ajay Nair, *Serverless Architectures on AWS*, 2nd edition, Manning, 2022. Mục 10.3.2 của tài liệu tham khảo, "Allocate sufficient resources to your execution environment", phân tích quan hệ giữa tài nguyên, thời lượng và chi phí. Chương 15 của eBook này vận dụng vào cách kiểm chứng tối ưu theo lượng công việc hoàn tất.

Bài viết của tác giả về Kubernetes

- Kubernetes Controllers, Deployment Strategies và một vài bài học — các tình huống rollout bị giới hạn bởi cổng trên node và storage. Chương 10 vận dụng vào việc chọn controller, chuẩn bị capacity và phân biệt thay Pod với phục hồi dữ liệu. Khi đọc về volume, đối chiếu access modes: `ReadWriteOnce` giới hạn theo node; `ReadWriteOncePod` giới hạn theo Pod.
- Về Ingress Controller trên EKS Cluster — đối chiếu định tuyến trên ALB với reverse proxy trong cluster. Chương 11 giữ cách phân tích trách nhiệm, bổ sung đường traffic và điều kiện chọn implementation. Ví dụ ingress-nginx trong bài viết năm 2025 cần được đọc cùng thông báo kết thúc bảo trì.

Delivery, reliability và vận hành

- Nicole Forsgren, Jez Humble và Gene Kim, *Accelerate*, IT Revolution, 2018. Nền tảng thực nghiệm cho delivery performance và các chỉ số DORA.
- Betsy Beyer, Chris Jones, Jennifer Petoff và Niall Richard Murphy (biên tập), *Site Reliability Engineering*, O'Reilly Media, 2016. Nguồn cho toil, SLO, error budget và vận hành hệ thống tin cậy; phần Eliminating Toil liên quan trực tiếp đến tự động hóa ở Chương 4.
- The Site Reliability Workbook — Alerting on SLOs — thiết kế cảnh báo theo error budget và burn rate, bổ sung cho Chương 14.
- AWS Well-Architected Framework — khung kiểm tra reliability, security, cost optimization, operational excellence, performance efficiency và sustainability.
- Kubernetes Documentation — nguồn chuẩn cho workload, probe, autoscaling, scheduling và disruption.
- OpenTelemetry Documentation — chuẩn instrument và truyền context cho logs, metrics, traces.
- Terraform S3 backend — remote state, S3 lockfile, versioning và quyền IAM; DynamoDB locking đã được đánh dấu deprecated.
- AWS Prescriptive Guidance — Transactional outbox — dual-write problem, outbox/CDC và yêu cầu idempotent consumer.

Tài liệu AWS cần kiểm tra khi thực hành

- Amazon EKS architecture, EKS Auto Mode, EKS Pod Identities và Security Groups for Pods.
- Amazon ECS, các lựa chọn compute của EKS và vòng đời phiên bản Kubernetes trên EKS — phân biệt lựa chọn điều phối, tài nguyên thực thi và trách nhiệm nâng cấp ở Chương 11.
- AWS Load Balancer Controller, Amazon VPC CNI và prefix delegation — đối chiếu cơ chế định tuyến, đăng ký target và nhu cầu địa chỉ IP ở Chương 11.

- Step Functions Standard và Express, Lambda integration và StartExecution — nối workflow, Lambda Task và contract khởi chạy của chương 9; phân biệt bảo đảm ở cấp execution với side effect nghiệp vụ.
- Step Functions versions and aliases — semantics phát hành state machine bất biến và canary theo alias.
- Step Functions HTTP Task — EventBridge connection, private connectivity, giới hạn timeout và contract của HTTP integration.
- Lambda concurrency and scaling — quota, reserved/provisioned concurrency và scaling rate; Lambda container image cho Java — base image, Runtime Interface Client và yêu cầu runtime.
- Lambda với RDS, RDS Proxy và kết nối mạng của Lambda trong VPC — đọc thêm cho năm service dùng Lambda và một RDS instance ở chương 9; đối chiếu connection, quyền và đường mạng.
- SNS message filtering và SNS dead-letter queues — phân biệt chặng phân phối event với chặng consumer xử lý message.
- Lambda xử lý lỗi từ SQS — retry, visibility timeout và partial batch response với `ReportBatchItemFailures`.
- Lambda với SQS, co giãn consumer SQS và SQS visibility timeout — bổ sung cơ chế polling, giới hạn concurrency và điều kiện message được nhận lại ở Chương 9.
- Lambda timeout và cấu hình bộ nhớ Lambda — kiểm tra giới hạn thực thi và quan hệ giữa tài nguyên, thời lượng, chi phí ở Chương 9, 13 và 15.
- API Gateway throttling — account/stage/route limits và phạm vi usage plan theo client.
- Amazon SQS quotas — throughput, retention, in-flight message và giới hạn theo loại queue.
- EventBridge Scheduler — kích hoạt Lambda relay định kỳ và đánh giá độ trễ polling outbox ở chương 9.
- RDS Multi-AZ — phân biệt dự phòng database với việc tách database deployment theo service.
- DynamoDB transactions, Streams và read consistency — đối chiếu idempotency, outbox và tra cứu kết quả khi đánh giá DynamoDB ở chương 9.
- Amazon EventBridge — event bus, rule/filter và danh mục target; lựa chọn thay thế SNS khi routing phức tạp hơn (chương 9).
- Aurora Serverless v2 và DynamoDB data modeling — đọc thêm khi đánh giá các lựa chọn lưu dữ liệu ở chương 9 và 13. Đây là những phương án cần phân tích riêng, không phải điều kiện để triển khai Saga của chương 9.
- API Gateway: REST API và HTTP API cùng JWT authorizer — kiểm tra tính năng của loại API đã chọn và phạm vi xác thực ở chương 13, 15.

Khi thực hành, hãy đối chiếu tài liệu với phiên bản công cụ và dịch vụ bạn đang sử dụng. Các trang có đường dẫn `latest`, `stable` hoặc `current` thường được cập nhật, còn tài liệu gắn với phiên bản cụ thể như KEDA 2.18 có thể khác với môi trường của bạn. Các sách được liệt kê theo ấn bản đã tham khảo để bạn tra cứu đúng nội dung được dẫn. Với AWS, cần kiểm tra thêm tính năng và giới hạn áp dụng tại Region triển khai.

Tra cứu theo nhóm ví dụ

Các nguồn dưới dùng để kiểm tra những điểm dễ nhầm trong ví dụ; cần đối chiếu lại phiên bản khi thực hành.

- **Nền tảng:** Spring transaction rollback, VPC subnet, S3 bucket và S3 storage class.
- **Container:** Docker image layer và vòng đời volume.
- **CI/CD:** GitHub OIDC trên AWS, ECR tag immutability và trạng thái lệnh SSM.
- **Cấu hình phát hành trên Kubernetes:** Helm template debugging, Kubernetes server-side dry-run và Argo CD automated sync — phạm vi kiểm tra manifest sau khi render và cách xử lý sai lệch giữa Git với cluster ở Chương 11.
- **IaC:** giới hạn `prevent_destroy`, dữ liệu nhạy cảm trong Terraform và module EKS cùng hướng dẫn nâng major version.
- **Luồng bất đồng bộ và quan sát:** Asynchronous Request-Reply và OpenTelemetry messaging spans — phân biệt request tiếp nhận, công việc chạy nền và cách liên kết các trace.
- **Saga và messaging:** RabbitMQ acknowledgements/confirms, dead letter exchanges, Step Functions Standard/Express và Retry/Catch.
- **Kubernetes:** kind quick start, Deployment và rolling update, disruption và phạm vi PDB, vòng đời Pod, startup/liveness/readiness probe, HPA và KEDA SQS scaler.
- **Vòng đời công cụ:** thông báo kết thúc bảo trì ingress-nginx — phân biệt dự án controller này với tài nguyên Ingress của Kubernetes.
- **Quan sát và kiểm soát:** OpenTelemetry sampling, EKS API data encryption, CloudFront signed URLs và AWS Budgets.
- **Đọc metric và chi phí quan sát:** SQS CloudWatch metrics và CloudWatch pricing — phân biệt tuổi message với tuổi công việc nghiệp vụ; kiểm tra đơn vị tính phí cho log, metric, trace và truy vấn ở Chương 14–15.

Database và các phương án triển khai ở Chương 9–10

- PostgreSQL — Managing Databases — đối chiếu việc tổ chức nhiều database trong cùng một hệ thống PostgreSQL; cần xem riêng quyền truy cập và tài nguyên dùng chung.
- Temporal Workflows và mã nguồn Temporal — tra cứu mô hình workflow và phương án tự vận hành được nhắc ở Chương 9.

- RabbitMQ — Publish/Subscribe với Java — ví dụ triển khai producer, exchange và consumer để đối chiếu với SNS/SQS.
- Docker Compose trong production, kind, minikube và k3d — phân biệt cấu hình chạy ứng dụng với môi trường Kubernetes phục vụ thực hành; không suy ra khả năng vận hành production chỉ từ việc chạy được ví dụ.

Vòng đời workload, mạng và co giãn ở Chương 10–13

- Kubernetes Services và networking, NetworkPolicy và Ingress controllers — phân biệt khai báo kết nối, chính sách mạng và thành phần thực thi.
- StatefulSet, xóa cưỡng bức Pod của StatefulSet và gắn EBS volume — đọc các điều kiện về danh tính Pod và storage trước khi xử lý lỗi hoặc thay thế Pod.
- Deployment bị đình trệ, cập nhật ứng dụng qua ConfigMap, chiến lược cập nhật StatefulSet và quá trình kết thúc Pod — kiểm tra giới hạn rollout, cập nhật cấu hình và graceful shutdown.
- Karpenter — cấp và quản lý node theo nhu cầu của Pod, bổ sung cho phần HPA/KEDA ở Chương 12.
- Knative Serving — triển khai và co giãn workload trên Kubernetes; đối chiếu phạm vi công việc của nhóm ứng dụng và nhóm nền tảng ở Chương 13.

Quan sát và truyền context ở Chương 14

- W3C Trace Context — đặc tả `traceparent` và `tracestate`; cơ sở để hiểu truyền ngữ cảnh trace qua HTTP.
- OpenTelemetry Collector — tiếp nhận, xử lý và xuất telemetry; đọc cùng tài liệu messaging spans và sampling ở trên để phân biệt instrumentation, truyền context và lưu trữ.

Bảo mật và khôi phục dữ liệu ở Chương 15

Các nguồn này bổ sung hướng triển khai cho những yêu cầu bảo vệ và phục hồi trong chương; cần đọc cùng các giả định về danh tính, phạm vi sự cố và dữ liệu nghiệp vụ của TravelNow.

- IAM security best practices — credential tạm thời, quyền tối thiểu và kiểm tra quyền truy cập.
- OWASP — Broken Object Level Authorization — vì sao token hợp lệ vẫn cần kiểm tra quyền đối với từng tài nguyên, chẳng hạn `tripId`.
- Secrets Manager — Rotation — vòng đời secret; cần kiểm chứng cả cách ứng dụng nhận credential mới.
- Kubernetes security context — quyền của process, capabilities và cấu hình tăng cường bảo vệ container.

- CloudFront — Private content — đọc cùng signed URLs ở trên để kiểm soát việc phân phối PDF; liên quan cả Chương 13.
- RDS — Point-in-time recovery — khôi phục database về một thời điểm; việc đối soát giao dịch bên ngoài database vẫn thuộc quy trình phục hồi nghiệp vụ.
- S3 Versioning — lưu nhiều phiên bản object; cần kết hợp quyền truy cập và chính sách lưu giữ phù hợp.
- SPDX Specifications và SLSA Provenance — phân biệt danh sách thành phần phần mềm với thông tin về quá trình tạo artifact ở Mục 15.5.
- CloudTrail management events, data events và nguồn dữ liệu nền tảng của GuardDuty — kiểm tra phạm vi audit và phát hiện; không mặc định mọi thao tác hoặc mọi hành vi xâm nhập đều được ghi nhận.

Phân tích và kiểm soát chi phí ở Chương 15

- AWS Cost Explorer — phân tích chi phí và mức sử dụng theo thời gian, đọc cùng quy tắc phân bổ tài nguyên dùng chung của TravelNow.
- AWS Cost Anomaly Detection — phát hiện biến động chi phí; cần phân biệt với AWS Budgets và các giới hạn tài nguyên trực tiếp trong ứng dụng.
- Khi so sánh phương án, dùng tài liệu giá của từng dịch vụ với cùng Region, lượng công việc và mục tiêu phục vụ. Chi phí vận hành, chuyển đổi và thời gian xử lý sự cố phải được ước lượng riêng; hóa đơn Cloud không bao phủ toàn bộ các khoản này.

Cảm ơn bạn đã đọc

eBook này miễn phí và sẽ luôn như vậy. Nếu eBook hữu ích với bạn, bạn có thể mời mình một ly cà phê để tiếp thêm động lực viết. Chia sẻ eBook với một người bạn hoặc gửi góp ý cũng là những cách ủng hộ mà mình rất trân trọng.

NGÂN HÀNG
TPBank (Tiên Phong)

CHỦ TÀI KHOẢN
HA HUU LINH

SỐ TÀI KHOẢN
0261 8975 555

NỘI DUNG GỢI Ý
cafe tu code den cloud

Thông tin chính thức luôn có tại ebook.fromcodetocloud.info/ung-ho/. Nếu bản PDF bạn nhận được có mã QR hay số tài khoản khác, đó không phải của tác giả.

— huulinhcvp